Bitcoin Book of BIPs

compiled by Adam Shannon

Abstract

Bitcoin Improvement Proposals are the documents that describe how Bitcoin actually works — consensus rules, wallet formats, peer messages, and the process for changing them. This book gathers the deployed and complete BIPs that shape Bitcoin today and groups them by theme so they can be read as a narrative instead of a numbered archive.

I did not write these proposals. The authors and contributors of bitcoin/bips did. I collected them, skipped withdrawn and closed work, and added short chapter notes so you can follow the connections between related specifications.

Contents

Introduction
How to Read This Book
Source snapshot
Process & History
- BIP 3
- BIP 123
- BIP 9
- BIP 8
- BIP 90
- BIP 50
- BIP 325
- BIP 94

Addresses and Encodings
- BIP 13
- BIP 173
- BIP 350
- BIP 321
- BIP 179

Consensus Changes That Shipped
- BIP 16
- BIP 30
- BIP 34
- BIP 42
- BIP 65
- BIP 66
- BIP 68
- BIP 112
- BIP 113
- BIP 141

- BIP 143
- BIP 147
- BIP 148
- BIP 91

Schnorr, Taproot, Tapscript

- BIP 340
- BIP 341
- BIP 342

Keys, Seeds, and Derivation

- BIP 32
- BIP 38
- BIP 39
- BIP 43
- BIP 44
- BIP 49
- BIP 84
- BIP 85
- BIP 86
- BIP 87
- BIP 48
- BIP 129
- BIP 328

Output Script Descriptors

- BIP 380
- BIP 381
- BIP 382
- BIP 383
- BIP 384
- BIP 385
- BIP 386
- BIP 387

Partially Signed Transactions

- BIP 174
- BIP 370
- BIP 371
- BIP 373

Peer-to-Peer Network

- BIP 14
- BIP 31
- BIP 35
- BIP 37
- BIP 61
- BIP 111
- BIP 130
- BIP 133
- BIP 144
- BIP 152

- BIP 155
- BIP 157
- BIP 158
- BIP 159
- BIP 324
- BIP 339
- BIP 434

Mining and Block Templates

- BIP 22
- BIP 23
- BIP 145

Paying People

- BIP 70
- BIP 71
- BIP 72
- BIP 73
- BIP 75
- BIP 78
- BIP 47
- BIP 352
- BIP 353
- BIP 327
- BIP 137
- BIP 322

Scripts, Miniscript, and Policy

- BIP 11
- BIP 67
- BIP 69
- BIP 125
- BIP 388

Conclusion

Introduction

Welcome to the Bitcoin Book of BIPs. This is a reading copy of the Bitcoin Improvement Proposals that actually shape Bitcoin today — the deployed and complete documents, grouped by theme instead of by number.

I did not write the BIPs. The authors and contributors of bitcoin/bips did. My job here is to collect their work, skip the closed and withdrawn proposals so the book stays readable, and put related documents next to each other so the connections are easier to see.

Each chapter starts with a short note from me, then the BIP texts themselves. Those texts are the originals. If you want to propose a change, take it upstream. If you want to understand how wallets, nodes, miners, and consensus rules fit together, this order should help.

Thanks for picking this up. I hope it makes the archive feel more like a book and less like a numbered attic.

How to Read This Book

This book is organized by theme, not by BIP number. Start with process and history, then addresses, then the consensus changes that shipped, then Taproot. After that the chapters follow how software is built: keys and wallets, descriptors, PSBTs, the peer-to-peer network, mining templates, payments, and script policy.

You do not have to read it front to back. A wallet engineer can jump to keys, descriptors, and PSBTs. A node implementer can live in the P2P chapter. Someone tracing a soft fork can read process, then consensus, then Taproot. The original BIP numbers stay in the headings so you can always find the same document in bitcoin/bips.

I left out closed and withdrawn BIPs on purpose. They matter as history, but they interrupt a reading copy. Drafts that are not yet complete are out too. What remains is the set of specifications that wallets and nodes actually implement.

A few of the included documents are informational rather than consensus rules. They still belong here because they describe how Bitcoin is used in practice — derivation paths, address encodings, payment URIs, and the like. When a chapter mixes those with consensus changes, the intro will say so.

Source snapshot

This book was built from the following commit of bitcoin/bips. Closed and withdrawn proposals are omitted so the book stays readable. If something here disagrees with upstream, upstream wins.

```
commit 61ff3f1361838fa7eb671e650b4e614207ca9df7
Merge: ea875b2 7d92f13
Author: Jon Atack <jon@atack.com>
Date:   Tue Nov 3 21:42:17 2026 -0600

    Merge pull request #2311 from jonatack/2026-09-bip375-reference-implementation

    bip375: replace stale TODO with reference implementation
```

Process & History

Bitcoin does not have a standards body that can decree a change. What it has is a written process, a classification of documents, and a few hard-won lessons about how rules actually activate on a live chain.

This chapter is the map of that process. BIP 3 is the current process document. BIP 123 explains how proposals are classified — consensus, peer services, API, applications — so you know what kind of thing you are reading. BIP 9 and BIP 8 describe version-bits activation, which is how most modern soft forks have been scheduled. BIP 90 is the cleanup that followed: once a deployment is buried under enough proof of work, nodes can stop carrying the old state machine.

BIP 50 sits next as a warning. It is the post-mortem of the March 2013 chain fork, when two versions of the software briefly disagreed about what a valid block was. The later activation BIPs make more sense after you have read that story.

The last two documents are the test networks wallets and nodes actually use. BIP 325 is Signet: a coordinated chain where blocks need a signature as well as proof of work. BIP 94 is Testnet 4, which replaced the old public testnet. Neither changes mainnet consensus.

In this chapter:

- BIP 3 — Updated BIP Process
- BIP 123 — BIP Classification
- BIP 9 — Version bits with timeout and delay
- BIP 8 — Version bits with lock-in by height
- BIP 90 — Buried Deployments
- BIP 50 — March 2013 Chain Fork Post-Mortem
- BIP 325 — Signet
- BIP 94 — Testnet 4

```
  BIP: 3
  Title: Updated BIP Process
  Authors: Murch <murch@murch.one>
  Status: Deployed
  Type: Process
  Assigned: 2025-01-09
  License: BSD-2-Clause
  Discussion: 2024-05-13: https://github.com/murchandamus/bips/pull/2
              2024-04-02: https://gnusha.org/pi/bitcoindev/
59fa94cea6f70e02b1ce0da07ae230670730171c.camel@timruffing.de/#t
  Version: 1.4.0
  Requires: 123
  Replaces: 2
```

BIP 3

Abstract

This *Bitcoin Improvement Proposal (BIP)* provides information about the preparation of BIPs and policies relating to the publication of BIPs. It replaces BIP 2 with a streamlined process, and may be amended to address the evolving needs of the BIP process.

Motivation

BIP 2 was written in 2016. This BIP revisits aspects of the BIP 2 process that did not achieve broad adoption, reduces the judgment calls assigned to the BIP Editor role, delineates the BIP types more clearly, and generalizes the BIP process to fit the community's use of the repository.

Fundamentals

What is a BIP?

BIPs are improvement proposals for Bitcoin. The main topic is information and technologies that support and expand the utility of the Bitcoin currency. Most BIPs provide a concise, self-contained, technical description of one new concept, feature, or standard. Some BIPs describe processes, implementation guidelines, best practices, incident reports (e.g., BIP 50), or other information relevant to the Bitcoin community. However, any topics related to the Bitcoin protocol, peer-to-peer network, and client software may be acceptable.

BIPs are intended to be a means for proposing new protocol features, coordinating client standards, and documenting design decisions that have gone into implementations. BIPs may be submitted by anyone, provided the content is of high quality, e.g., does not waste the community's time.

The scope of the BIPs repository is limited to BIPs that do not oppose the fundamental principle that Bitcoin constitutes a peer-to-peer electronic cash system for the Bitcoin currency.

BIP Ownership

Each BIP is primarily owned by its authors and represents the authors' opinion or recommendation. The authors are expected to foster discussion, address feedback and dissenting opinions, and, if applicable, advance the adoption of their proposal within the Bitcoin community. As a BIP progresses through the workflow, it becomes increasingly co-owned by the Bitcoin community.

Authors and Deputies

Authors may want additional help with the BIP process after writing an initial draft. In that case, they may assign one or more Deputies to their BIP. Deputies are stand-in owners of a BIP who were not involved in writing the document. They support the authors in advancing the proposal, or act as a point of contact for the BIP in the absence of the authors. Deputies may perform the role of Authors for any aspect of the BIP process unless overruled by an Author. Deputies share ownership of the BIP at the discretion of the Authors.

What is the Significance of BIPs?

BIPs do not define what Bitcoin is: individual BIPs do not represent Bitcoin community consensus or a general recommendation for implementation. A BIP represents a personal recommendation by the BIP authors to the Bitcoin community. Some BIPs may never be adopted. Some BIPs may be adopted by one or more Bitcoin clients or other related software. Some may even end up changing the consensus rules that the Bitcoin ecosystem jointly enforces.

What is the Purpose of the BIPs Repository?

The BIPs repository serves as a publication medium and archive for mature proposals. Through its high visibility, it facilitates the community-wide consideration of BIPs and provides a well-established source to retrieve the latest version of any BIP. The repository transparently records all changes to each BIP and allows any community member to retain a complete copy of the archive easily.

The BIPs repository neither tracks community sentiment[1] nor ecosystem adoption[2] of BIPs beyond the brief overview provided via the BIP status (see Workflow below). Proposals are published in this repository if they are on-topic and fulfill the editorial criteria. No formal or informal decision body governs Bitcoin development or decides adoption of BIPs.

BIP Format and Structure

Specification

Authors may choose to submit BIPs in MediaWiki or Markdown[3] format.

Each BIP must have a *Preamble*, an *Abstract*, a *Copyright*, and a *Motivation* section. Authors should consider all issues in the following list and address each as appropriate.

- Preamble — Headers containing metadata about the BIP (see the section BIP Header Preamble below).
- Abstract — A short description of the issue being addressed.
- Motivation — Why is this BIP being written? Clearly explain how the existing situation presents a problem and why the proposed idea resolves the issue or improves upon the current situation.
- Specification — The technical specification should describe the syntax and semantics of any new feature. The specification should be detailed enough to enable any Bitcoin project to create an interoperable implementation.
- Rationale — The rationale fleshes out the specification by describing what inspired the design and why particular design decisions were made. It should describe related work and alternate designs that were considered. The rationale should record relevant objections or important concerns that were raised and addressed as this proposal was developed.
- Backward Compatibility — Any BIP that introduces incompatibilities must include a section describing these incompatibilities and their severity as well as provide instructions on how implementers and users should deal with these incompatibilities.
- Reference Implementation — Where applicable, a reference implementation, test vectors, and documentation must be finished before the BIP can be given the status “Complete”. Test vectors must be provided in the BIP or as auxiliary files (see Auxiliary Files) under an acceptable license. The reference implementation can be provided in the BIP, as an auxiliary file, or per linking another code reference

that is expected to remain available permanently such as a pull request, a dedicated branch, a new repository, or similar.
- Changelog — A section to track modifications to a BIP after reaching Complete status.
- Copyright — The BIP must be placed under an acceptable license (see BIP Licensing below).

BIP Header Preamble

Each BIP must begin with an RFC 822-style header preamble. The headers must appear in the following order. Headers marked with "*" are optional. All other headers are required.

Overview

```
  BIP: <BIP number, or "?" before assignment>
* Layer: <Consensus (soft fork) | Consensus (hard fork) | Peer Services | API/RPC |
Applications>
  Title: <BIP title (≤ 50 characters)>
  Authors: <Authors' names and email addresses>
* Deputies: <Deputies' names and email addresses>
  Status: <Draft | Complete | Deployed | Closed>
  Type: <Specification | Informational | Process>
  Assigned: <Date of number assignment (yyyy-mm-dd), or "?" before assignment>
  License: <SPDX License Expression>
* Discussion: <Noteworthy discussion threads in "yyyy-mm-dd: URL" format>
* Version: <MAJOR.MINOR.PATCH>
* Requires: <BIP number(s)>
* Replaces: <BIP number(s)>
* Proposed-Replacement: <BIP number(s)>
```

Header Descriptions

- BIP — The assigned number of the BIP (without leading zeros). Please use "?" before a number has been assigned by the BIP Editors.
- Layer — The layer of Bitcoin the BIP applies to using the BIP classification defined in BIP 123.
- Authors — The names (or pseudonyms) and email addresses of all authors of the BIP. The format of each authors header value must be

  ```
  Random J. User <address@dom.ain>
  ```

  Multiple authors are recorded on separate lines:

  ```
  Authors: Random J. User <address@dom.ain>
           Anata Sample <anata@domain.example>
  ```
- Deputies — Additional owners of the BIP that are not authors. The Deputies header uses the same format as the Authors header. See the BIP Ownership section above.
- Status — The stage of the workflow of the proposal. See the Workflow section below.
- Type — See the BIP Types section below for a description of the three BIP types.

- Assigned – The date a BIP was assigned its number. Please use "?" before a number has been assigned by the BIP Editors.

- License — The License header specifies SPDX License Expressions describing the terms under which the BIP and its auxiliary files are available. See the BIP Licensing section below.

- Discussion — The Discussion header points the audience to relevant discussions of the BIP, e.g., the mailing list thread in which the idea for the BIP was discussed, a thread where a new version of the BIP was presented, or relevant discussion threads on other platforms. Entries take the format "yyyy-mm-dd: URL", e.g., `2009-01-09: https://www.mail-archive.com/cryptography@metzdowd.com/msg10142.html`, using the date and URL of the start of the conversation. Multiple discussions should be listed on separate lines.

- Version — The current version number of this BIP. See the Changelog section below.

- Requires — A list of existing BIPs the new proposal depends on. If multiple BIPs are required, they should be listed in one line separated by a comma and space (e.g., "1, 2").

- Replaces[4] — BIP authors may put the numbers of one or more prior BIPs in the Replaces header to recommend that their BIP succeeds, supersedes, or renders obsolete those prior BIPs.

- Proposed-Replacement[5] — When a later BIP indicates that it intends to supersede an existing BIP, the later BIP's number is added to the Proposed-Replacement header of the existing BIP to indicate the potential successor BIP.

Auxiliary Files

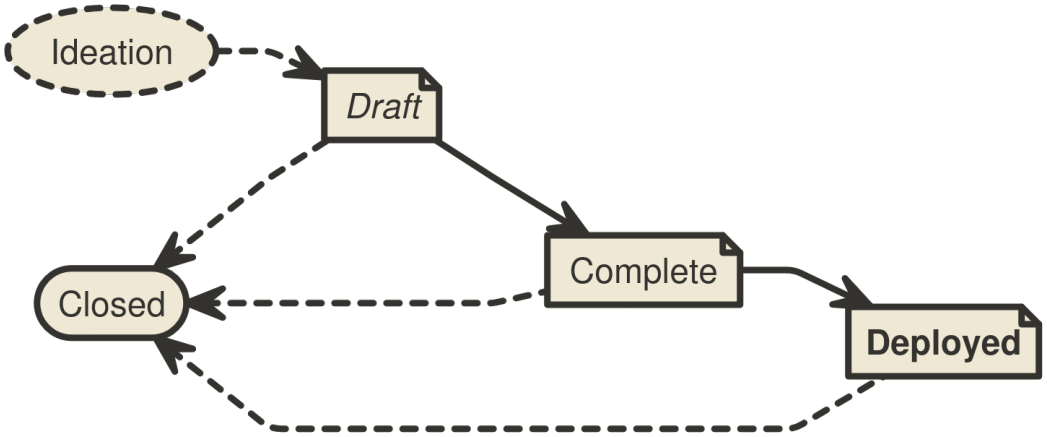
BIPs may include auxiliary files such as diagrams and source code. Auxiliary files must be included in a subdirectory for that BIP named `bip-XXXX`, where "XXXX" is the BIP number zero-padded to four digits. File names in the subdirectory do not need to adhere to a specific convention.

BIP Types

- A **Specification BIP** defines a set of technical rules describing a new feature or affecting the interoperability of implementations. The distinguishing characteristic of a Specification BIP is that it can be implemented, and implementations can be compliant with it. Specification BIPs must have a Specification section, must have a Backward Compatibility section (if incompatibilities are introduced), and can only be advanced to Complete after they contain or refer to a reference implementation and test vectors.
- An **Informational BIP** describes a Bitcoin design issue, or provides general guidelines or other information to the Bitcoin community.
- A **Process BIP** describes a process surrounding Bitcoin, or proposes a change to (or an event in) a process. Process BIPs are like Specification BIPs, but apply to topics other than the Bitcoin protocol and Bitcoin implementations. They often require community consensus and are typically binding for the corresponding process. Examples include procedures, guidelines, and changes to decision-making processes such as the BIP process.

Workflow

The BIP process starts with a new idea for Bitcoin. Each potential BIP must have authors—people who write the BIP, gather feedback, shepherd the discussion in the appropriate forums, and finally recommend a mature proposal to the community.



Status Diagram

Ideation

After having an idea, the authors should evaluate whether it meets the criteria to become a BIP, as described in this BIP. The idea must be of interest to the broader community or relevant to multiple software projects. Minor improvements and matters concerning only a single project usually do not require standardization and should instead be brought up directly to the relevant project.

The authors should first research whether their idea has been considered before. Ideas in Bitcoin are often rediscovered, and prior related discussions may inform the authors of the issues that may arise in its progression. After some investigation, the novelty and viability of the idea should be tested by posting a new, dedicated thread about the idea to the Bitcoin Development Mailing List. Prior correspondence can be found in the mailing list archive.

It is recommended that authors establish before or at the start of working on a draft whether their idea may be of interest to the Bitcoin community. Authors should avoid opening a pull request with a BIP draft out of the blue. Vetting an idea publicly before investing time and effort to formally describe the idea is meant to save time for both the authors and the community. Not only may someone point out relevant discussion topics that were missed in the authors' research, or that an idea is guaranteed to be rejected based on prior discussions, but describing an idea publicly also tests whether it is of interest to more people beside the authors.

As a first sketch of the proposal is taking shape, the authors should present it to the Bitcoin Development Mailing List. This gives the authors a chance to collect initial feedback and address fundamental concerns. If the authors wish to work in public on the proposal at this stage, it is recommended that they open a pull request against one of their forks of the BIPs repository instead of the main BIPs repository.

It is recommended that complicated proposals be split into separate BIPs that each focus on a specific component of the overall proposal.

Progression through BIP Statuses

The following sections refer to BIP Status field values. The BIP Status field is defined in the Header Preamble specification above.

Draft

After fleshing out the proposal further and ensuring that it is of high quality and properly formatted, the authors should open a pull request to the BIPs repository. The document must adhere to the formatting requirements specified above and should be provided as a file named with a working title of the form "bip-title.[md | mediawiki]". Only BIP Editors may assign BIP numbers. Until one has done so, authors should refer to their BIP by name only.

BIPs that (1) adhere to the formatting requirements, (2) are on-topic, and (3) have materially progressed beyond the ideation phase, e.g., by generating substantial public discussion and commentary from diverse contributors, by independent Bitcoin projects working on adopting the proposal, or by the authors working for an extended period toward improving the proposal based on community feedback, will be assigned a number by a BIP Editor. A number may be considered assigned only after it has been publicly announced in the pull request by a BIP Editor. The BIP Editors should not assign a number when they perceive a proposal being met with lack of interest: number assignment facilitates the distributed discussion of ideas, but before a proposal garners some interest in the Bitcoin community, there is no need to refer to it by a number.

Proposals are also not ready for number assignment if they duplicate efforts, disregard formatting rules, are too unfocused or too broad, fail to provide proper motivation, fail to address backward compatibility where necessary, or fail to specify the feature clearly and comprehensively. Reviewers and BIP Editors should provide guidance on how the proposal may be improved to progress toward readiness. Pull requests that are proposing off-topic ideas or have stopped making progress may be closed.

When the proposal is ready and has been assigned a number, a BIP Editor will merge it into the BIPs repository. After the BIP has been merged to the repository, its main focus should no longer shift significantly, even while the authors may continue to update the proposal as necessary. Updates to merged documents by the authors should also be submitted as pull requests.

Complete[6]

When the authors have concluded all planned work on their proposal, are confident that their BIP represents a net improvement, is clear, comprehensive, and is ready for adoption by the Bitcoin community, they may update the BIP's status to Complete to indicate that they recommend adoption, implementation, or deployment of the BIP. Where applicable, the authors must ensure that any proposed specification is solid, not unduly complicated, and definitive. Specification BIPs must come with or refer to a working reference implementation and comprehensive test vectors before they can be moved to Complete. Subsequently, the BIP's content should only be adjusted in minor details, e.g., to improve language, clarify ambiguities, backfill omissions in the specification, add test vectors for edge cases, or address other issues discovered as the BIP is being adopted.

A Complete BIP can only move to Deployed or Closed. Any necessary changes to the specification should be minimal and interfere as little as possible with ongoing adoption. If a Complete BIP is found to need substantial functional changes, it may be preferable to move it to Closed[7], and to start a new BIP with the changes instead. Otherwise, it could cause confusion as to what being compliant with the BIP means.

A BIP may remain in the Complete status indefinitely unless its authors or deputies decide to move it to Closed or it is advanced to Deployed. Complete is the final status for most successful Informational BIPs.

Deployed

A Complete BIP should only be moved to Deployed once it is settled: after its approach has solidified, its Specification has been put through its paces, feedback from early adopters has been processed, and amendments to the BIP have stopped. Then, a BIP may be advanced to Deployed upon request by any community member with evidence[8] that the BIP is in active use. Convincing evidence includes for example: an established project having deployed support for the BIP in mainnet software releases, a soft fork proposal's activation criteria having been met on the network, or rough consensus for the BIP having been demonstrated.

Once Deployed, the BIP is considered final. Any modifications to the BIP beyond bug fixes, other minor amendments, additions to the test vectors, or editorial changes should be avoided. Any breaking changes to the BIP's Specification should be proposed as a new separate BIP.[9]

Process BIPs

A Process BIP may change status from Complete to Deployed when it achieves rough consensus on the Bitcoin Development Mailing List. A proposal is said to have rough consensus if its advancement has been open to discussion on the mailing list for at least one month, the discussion achieved meaningful engagement, and no person maintains any unaddressed substantiated objections to it. Addressed or obstructive objections may be ignored/overruled by general agreement that they have been sufficiently addressed, but clear reasoning must be given in such circumstances. Deployed Process BIPs may be modified indefinitely as long as a proposed modification has rough consensus per the same criteria.[10]

Closed[11]

A BIP that is of historical interest only, and is not being actively worked on, promoted or in active use, should be marked as Closed. The reason for moving the proposal to (or from) Closed should be recorded in the Changelog section in the same commit that updates the status. BIPs do not get deleted, they are retained even after being updated to Closed. Transitions involving the Closed state are:

Draft ↦ Closed

BIP authors may decide on their own to change their BIP's status from Draft to Closed. If a Draft BIP stops making progress, sees accumulated feedback unaddressed, or otherwise appears stalled for a year, anyone may propose the BIP status be updated to Closed. The BIP is then updated to Closed unless the authors assert that they intend to continue work within four weeks of being contacted.

Complete ↦ Closed

BIPs that had attained the Complete status, i.e., that had been recommended for adoption, may be moved to Closed per the authors' announcement to the Bitcoin Development Mailing List[12]. However, if someone volunteers to adopt the proposal within four weeks, they become the BIP's author or deputy (see Transferring BIP Ownership below), and the BIP will remain Complete instead.

Deployed ↦ Closed

A BIP may evolve from Deployed to Closed when it is no longer in active use. Any community member may initiate this Status update by announcing it to the mailing list[13], and proceed if no objections have been raised for four weeks.

Closed ↦ Draft

The Closed status is generally intended to be a final status for BIPs, and if BIP authors decide to make another attempt at a previously Closed BIP, it is generally recommended to create a new proposal. (Obviously, the authors may borrow any amount of inspiration or actual text from any prior BIPs as licensing permits.) The authors should take special care to address the issues that caused the prior attempt's abandonment. Even if the prior attempt had been assigned a number, the new BIP will generally be assigned a distinct number. However, if it is obvious that the new attempt directly continues work on the same idea, it may be reasonable to return the Closed BIP to Draft status.

Changelog Section and Version Header

To help implementers understand updates to a BIP, any changes after it has reached Complete must be tracked with version, date, and description in a Changelog section sorted by most recent version first. The version number is inspired by semantic versioning (MAJOR.MINOR.PATCH). The MAJOR version is incremented if changes to the BIP's Specification are introduced that are incompatible with prior versions (which should be rare after a BIP is Complete, and only happen in well-grounded exceptional cases to a BIP that is Deployed). The MINOR version is incremented whenever the specification of the BIP is changed or extended in a backward-compatible way. The PATCH version is incremented for other changes to the BIP that are noteworthy (bug fixes, test vectors, important clarifications, etc.). Version 1.0.0 is used to label the promotion to Complete. A Changelog section may be introduced during the Draft phase to record significant changes (using versions 0.x.y).

Example:

> **Changelog**
>
> - **2.0.0** (2025-01-22):
>   - Introduce a breaking change in the specification to fix a bug.
> - **1.1.0** (2025-01-17):
>   - Add a backward compatible extension to the BIP.
> - **1.0.1** (2025-01-15):
>   - Clarify an edge case and add corresponding test vectors.
> - **1.0.0** (2025-01-14):
>   - Complete planned work on the BIP.

After a BIP receives a Changelog, the Preamble must indicate the latest version in the Version header. The Changelog highlights revisions to BIPs to human readers. A single BIP shall not recommend more than one variant of an idea at the same time. A different or competing variant of an existing BIP must be published as a separate BIP.

Adoption of Proposals

The BIPs repository does not track the sentiment on proposals and does not track the adoption of BIPs beyond whether they are in active use or not. It is not intended for BIPs to list additional implementations beyond the reference implementation: the BIPs repository is not a signpost where to find implementations.[14] After a BIP is advanced to Complete, it is up to the Bitcoin community to evaluate, adopt, ignore, or reject a BIP. Individual Bitcoin projects are encouraged to publish a list of BIPs they implement. A good example of this at the time of writing this BIP can be observed in Bitcoin Core's doc/bips.md file.

Transferring BIP Ownership

It occasionally becomes necessary to transfer ownership of BIPs to new owners. In general, it would be preferable to retain the original authors of the transferred BIP, but that is up to the original authors. A good reason to transfer ownership is because the original authors no longer have the time or interest in updating it or following through with the BIP process, or are unreachable or unresponsive. A bad reason to transfer ownership is because someone doesn't agree with the direction of the BIP. The community tries to build consensus around a BIP, but if that's not possible, rather than fighting over control, the dissenters should supply a competing BIP.

If someone is interested in assuming ownership of a BIP, they should send an email asking to take over, addressed to the original authors, the BIP Editors, and the Bitcoin Development Mailing List[15]. If the authors are unreachable or do not respond in a timely manner (e.g., four weeks), the BIP Editors will make a unilateral decision whether to appoint the applicants as Authors or Deputies (which may be amended should the original authors make a delayed reappearance).

BIP Licensing

The Bitcoin project develops a global peer-to-peer digital cash system. Open standards are indispensable for continued interoperability. Open standards reduce friction, and encourage anybody and everyone to contribute, compete, and innovate on a level playing field. Only freely licensed contributions are accepted to the BIPs repository.

Specification

Each new BIP must specify in two ways under which license terms it is made available. First, it must specify an SPDX License Expression in the License field in the preamble. Second, it must include a matching Copyright section, possibly providing further details on licensing.

For example, a preamble might include the following License header:

```
License: CC0-1.0 OR MIT
```

In this case, the BIP (including all auxiliary files) is made available under the terms of both CC0 1.0 Universal as well as the MIT License, and anyone may modify and redistribute it provided they comply with the terms of

either license, at their option. In other words, the license list is an "OR choice", not an "AND also" requirement. See the SPDX documentation and the SPDX License List for further details.

Wherever different from those specified in the License header, an auxiliary file or directory should specify the license terms under which it is made available as is common in software (e.g., with a `SPDX-License-Identifier: <SPDX License Expression>` comment, a license header, or a LICENSE/COPYING file). Such exceptions should also be mentioned in the Copyright section. It is recommended to make any test vectors available under CC0-1.0 or FSFAP in addition to any other licenses to allow anyone to copy test vectors into their implementations without introducing license hindrances.

A few BIP2-era BIPs (98, 116, 117, 330, 340) have a no longer used "License-Code" header indicating the license terms applicable to auxiliary source code files. For such cases, please refer to BIP2.

It is recommended that source code included in a BIP (whether within the text or in auxiliary files) be licensed under the same license terms as the project it is proposed to modify, if any. For example, changes intended for Bitcoin Core would ideally be licensed (also) under the MIT License.

In all cases, details of the licensing terms must be provided in the Copyright section of the BIP.

Acceptable Licenses[16]

Each new BIP must be made available under at least one acceptable license as listed below. BIPs are not required to be *exclusively* licensed under approved terms, and may also be licensed under other licenses *in addition to* at least one acceptable license.

In other words, a new BIP must specify an SPDX License Expression that is either "L" or equivalent to "L OR E" for some acceptable license L from the following list and another SPDX License Expression E.

- BSD-2-Clause: BSD 2-Clause License
- BSD-3-Clause: BSD 3-Clause License
- CC0-1.0: CC0 1.0 Universal
- FSFAP: FSF All Permissive License
- CC-BY-4.0: Creative Commons Attribution 4.0 International
- MIT: The MIT License
- MIT-0: MIT No Attribution License
- Apache-2.0: Apache License 2.0
- BSL-1.0: Boost Software License 1.0

Not Acceptable Licenses

All licenses not explicitly included in the above lists are not acceptable terms for a Bitcoin Improvement Proposal. However, BIPs predating this proposal were accepted under other terms, and should use one of the following identifiers.

- LicenseRef-PD: Placed into the public domain
- OPUBL-1.0: Open Publication License 1.0

BIP Editors

The current BIP Editors are:

- Bryan Bishop (kanzure@gmail.com)
- Jon Atack (jon@atack.com)
- Mark "Murch" Erhardt (murch@murch.one)
- Olaoluwa Osuntokun (laolu32@gmail.com)
- Ruben Somsen (rsomsen@gmail.com)

BIP Editor Responsibilities and Workflow

The BIP Editors subscribe to the Bitcoin Development Mailing List and watch the BIPs repository.

When either a new BIP idea or an early draft is submitted to the mailing list, BIP Editors or other community members should comment in regard to:

- Novelty of the idea
- Viability, utility, and relevance of the concept
- Readiness of the proposal
- On-topic for the Bitcoin community

Discussion in pull request comments can often be hard to follow as feedback gets marked as resolved when it is addressed by authors. Substantive discussion of ideas may be more accessible to a broader audience on the mailing list, where it is also more likely to be retained by the community memory.

If the BIP needs more work, an editor should ensure that constructive, actionable feedback is provided to the authors for revision. Once the BIP is ready it should be submitted as a "pull request" to the BIPs repository where it may get further feedback.

For each new BIP pull request that comes in, an editor checks the following:

- The idea has been previously proposed to the Bitcoin Development Mailing List and discussed there
- The described idea is on-topic for the repository
- A draft of the BIP by one of the authors has been previously discussed on the Bitcoin Development Mailing List
- Title accurately describes the content
- Proposal is of general interest and/or pertains to more than one Bitcoin project/implementation
- Document is properly formatted
- Licensing terms are acceptable
- Motivation, Rationale, and Backward Compatibility have been addressed
- Specification provides sufficient detail for implementation
- The defined Layer and Type headers must be correctly assigned for the given specification
- The BIP is ready: it is comprehensible, technically feasible and sound, and all aspects are addressed as necessary

Editors do NOT evaluate whether the proposal is likely to be adopted.

Then, a BIP Editor will:

- Assign a BIP number in the pull request
- Ensure that the BIP is listed in the README
- Merge the pull request when it is ready

The BIP Editors are intended to fulfill administrative and editorial responsibilities. The BIP Editors monitor BIP changes, and update BIP headers as appropriate.

BIP Editors may also, at their option, unilaterally make and merge strictly editorial changes to BIPs, such as correcting misspellings, mending grammar mistakes, fixing broken links, etc. as long as they do not change the meaning or conflict with the original intent of the authors. Such a change must be recorded in the Changelog if it's noteworthy per the criteria mentioned in the Changelog section.

Backward Compatibility

Changes from BIP 2

Workflow

- Status field values are reduced from nine to four:
  - Deferred, Obsolete, Rejected, Replaced, and Withdrawn are gathered up into Closed.[17]
  - Final and Active are collapsed into Deployed.
  - Proposed is renamed to Complete.
  - The remaining statuses are Draft, Complete, Deployed, and Closed.
- The comment system is abolished.[18]
- A BIP in Draft or Complete status may no longer be closed solely on grounds of not making progress for three years.[19]
  - A BIP in Draft status may be updated to Closed status if it appears to have stopped making progress for at least a year and its authors do not assert within four weeks of being contacted that they intend to continue working on it.
  - Complete BIPs can only be moved to Closed by its authors and may remain in Complete indefinitely.
- A Changelog section is introduced to track significant changes to BIPs after they have reached the Complete status.
- Process BIPs are living documents that do not ossify and may be modified indefinitely.
- Some judgment calls previously required from BIP Editors are reassigned either to the BIP authors or the repository's audience.

BIP Format

- The Standards Track type is superseded by the similar Specification type.[20]
- Many sections are declared optional; it is up to the authors and reviewers to judge whether all relevant topics have been comprehensively addressed and which topics require a designated section to do so.
- "Other Implementations" sections are discouraged.[21]
- Auxiliary files are only permitted in the corresponding BIP's subdirectory, as no one used the alternative of labeling them with the BIP number.

- Tracking of community consensus and adoption is out of scope for the BIPs repository, except to determine whether a BIP is in active use for the move into or out of the Deployed status.
- The distinction between recommended and acceptable licenses was dropped.
- Most licenses that have not been used in the BIP process have been dropped from the list of acceptable licenses.

Preamble

- "Comments-URI" and "Comments-Summary" headers are dropped from the preamble.[22]
- The "Superseded-By" header is replaced with the "Proposed-Replacement" header.
- The "Post-History" header is replaced with the "Discussion" header.
- The optional "Version" header is introduced.
- The "Discussions-To" header is dropped, as it has never been used in any BIP.
- The "License-Code" header has been sunset, as it was used by only five BIPs (98, 116, 117, 330, 340) and created more ambiguity than clarity.
- The "Created" header is renamed to "Assigned", as the header's value is the date of number assignment.[23]
- Introduce Deputies and optional "Deputies" header.
- The BIP "Title" header may now contain up to 50 characters (increased from 44 in BIP 2).
- The "Layer" header is optional for Specification BIPs or Informational BIPs, as it does not make sense for all BIPs.[24]
- Rename the "Author" field to "Authors".

Updates to Existing BIPs should this BIP be Activated

Previous BIP Process

This BIP replaces BIP 2 as the guideline for the BIP process.

BIP Types

Standards Track BIPs and eligible Informational BIPs are assigned the Specification type. The Standards Track type is considered obsolete. Specification BIPs use the Layer header rules specified in BIP 123.

Comments

The Comments-URI and Comments-Summary headers should be removed from all BIPs whose comment page in the wiki is empty. For existing BIPs whose comment page has content, BIP Authors may keep both headers or remove both headers at their discretion. It is recommended that existing wiki pages are not modified due to the activation of this BIP.

Status Field

After the activation of this BIP, the Status fields of existing BIPs that do not fit the specification in this BIP are updated to the corresponding values prescribed in this BIP. BIPs that have had Draft status for extended periods will be moved to Complete or Deployed as applicable in collaboration with their authors. The authors of incomplete Draft BIPs will be contacted to learn whether the BIPs are still in progress toward Complete, and will otherwise be updated to Closed as described in the Workflow section above.

Authors Header

The Author header is replaced with the Authors header in all BIPs.

Discussion Header

The Post-History header is replaced with the Discussion header in all BIPs.

Proposed-Replacement Header

The Superseded-By header is replaced with the Proposed-Replacement header in all BIPs.

Licenses

Existing BIPs retain their license terms unchanged. The License and License-Code headers of BIPs are updated to express those terms using SPDX License Expressions.

Changelog

- **1.4.0** (2025-12-09):
  - Revert AI guidance, add Changelog section, broaden reference implementation formats, move Type header responsibility to authors, other editorial changes
- **1.3.1** (2025-11-10):
  - Reiterate that numbers are assigned by BIP Editors in pull requests
- **1.3.0** (2025-10-22):
  - Restrict use of AI/LLM tools, require original work.
- **1.2.1** (2025-09-19):
  - Clarify that idea should be discussed on dedicated mailing list thread
- **1.2.0** (2025-09-19):
  - Rename Created header to Assigned to clarify that it holds the date of number assignment
- **1.1.0** (2025-07-18):
  - Switch to SPDX License Expressions, drop License-Code header, and make editorial changes to BIP Licensing section.
- **1.0.1** (2025-06-27):
  - Improve description of acceptance, purpose of the BIPs repository, when Draft BIPs can be closed due to not making progress, and make other minor improvements to phrasing.
- **1.0.0** (2025-03-18):
  - Complete planned work and move to Proposed.

Copyright

This BIP is licensed under the BSD-2-Clause License. Some content was adapted from BIP 2 which was also licensed under the BSD-2-Clause.

Related Work

- BIP 1: BIP Purpose and Guidelines
- BIP 2: BIP Process, revised

- BIP 123: BIP Classification
- RFC 822: Standard for ARPA Internet Text Messages
- RFC 2223: Instructions to RFC Authors
- RFC 7282: On Consensus and Humming in the IETF

Acknowledgements

We thank AJ Towns, Jon Atack, Jonas Nick, Larry Ruane, Pieter Wuille, Tim Ruffing, and others for their review, feedback, and helpful comments.

Rationale

```
BIP: 123
Title: BIP Classification
Authors: Eric Lombrozo <elombrozo@gmail.com>
Status: Deployed
Type: Process
Assigned: 2015-08-26
License: CC0-1.0 OR FSFAP
```

BIP 123

Abstract

This document describes a classification scheme for BIPs.

BIPs are classified by system layers with lower numbered layers involving more intricate interoperability requirements.

The specification defines the layers and sets forth specific criteria for deciding to which layer a particular standards BIP belongs.

Copyright

This BIP is dual-licensed under the Creative Commons CC0 1.0 Universal and FSF All Permissive licenses.

Motivation

Bitcoin is a system involving a number of different standards. Some standards are absolute requirements for interoperability while others can be considered optional, giving implementers a choice of whether to support them.

In order to have a BIP process which more closely reflects the interoperability requirements, it is necessary to categorize BIPs accordingly. Lower layers present considerably greater challenges in getting standards accepted and deployed.

Specification

Standards BIPs are placed in one of four layers:

1. Consensus
2. Peer Services
3. API/RPC
4. Applications

Non-standards BIPs may be placed in these layers, or none at all.

1. Consensus Layer

The consensus layer defines cryptographic commitment structures. Its purpose is ensuring that anyone can locally evaluate whether a particular state and history is valid, providing settlement guarantees, and assuring eventual convergence.

The consensus layer is not concerned with how messages are propagated on a network.

Disagreements over the consensus layer can result in network partitioning, or forks, where different nodes might end up accepting different incompatible histories. We further subdivide consensus layer changes into soft forks and hard forks.

Soft Forks

In a soft fork, some structures that were valid under the old rules are no longer valid under the new rules. Structures that were invalid under the old rules continue to be invalid under the new rules.

Hard Forks

In a hard fork, structures that were invalid under the old rules become valid under the new rules.

2. Peer Services Layer

The peer services layer specifies how nodes find each other and propagate messages.

Only a subset of all specified peer services are required for basic node interoperability. Nodes can support further optional extensions.

It is always possible to add new services without breaking compatibility with existing services, then gradually deprecate older services. In this manner, the entire network can be upgraded without serious risks of service disruption.

3. API/RPC Layer

The API/RPC layer specifies higher level calls accessible to applications. Support for these BIPs is not required for basic network interoperability but might be expected by some client applications.

There's room at this layer to allow for competing standards without breaking basic network interoperability.

4. Applications Layer

The applications layer specifies high level structures, abstractions, and conventions that allow different applications to support similar features and share data.

Classification of existing BIPs

Number	Layer	Title	Owner	Type	Status
1		BIP Purpose and Guidelines	Amir Taaki	Process	Active
2		BIP process, revised	Luke Dashjr	Process	Draft
9		Version bits with timeout and delay	Pieter Wuille, Peter Todd, Greg Maxwell, Rusty Russell	Informational	Final
10	Applications	Multi-Sig Transaction Distribution	Alan Reiner	Informational	Withdrawn
11	Applications	M-of-N Standard Transactions	Gavin Andresen	Standard	Final
12	Consensus (soft fork)	OP_EVAL	Gavin Andresen	Standard	Withdrawn
13	Applications	Address Format for pay-to-script-hash	Gavin Andresen	Standard	Final
14	Peer Services	Protocol Version and User Agent	Amir Taaki, Patrick Strateman	Standard	Final
15	Applications	Aliases	Amir Taaki	Standard	Deferred
16	Consensus (soft fork)	Pay to Script Hash	Gavin Andresen	Standard	Final
17	Consensus (soft fork)	OP_CHECKHASHVERIFY (CHV)	Luke Dashjr	Standard	Withdrawn
18	Consensus (soft fork)	hashScriptCheck	Luke Dashjr	Standard	Accepted
19	Applications	M-of-N Standard Transactions (Low SigOp)	Luke Dashjr	Standard	Draft
20	Applications	URI Scheme	Luke Dashjr	Standard	Replaced
21	Applications	URI Scheme	Nils Schneider, Matt Corallo	Standard	Final
22	API/RPC	getblocktemplate - Fundamentals	Luke Dashjr	Standard	Final

Number	Layer	Title	Owner	Type	Status
23	API/RPC	getblocktemplate - Pooled Mining	Luke Dashjr	Standard	Final
30	Consensus (soft fork)	Duplicate transactions	Pieter Wuille	Standard	Final
31	Peer Services	Pong message	Mike Hearn	Standard	Final
32	Applications	Hierarchical Deterministic Wallets	Pieter Wuille	Informational	Final
33	Peer Services	Stratized Nodes	Amir Taaki	Standard	Draft
34	Consensus (soft fork)	Block v2, Height in Coinbase	Gavin Andresen	Standard	Final
35	Peer Services	mempool message	Jeff Garzik	Standard	Final
36	Peer Services	Custom Services	Stefan Thomas	Standard	Draft
37	Peer Services	Connection Bloom filtering	Mike Hearn, Matt Corallo	Standard	Final
38	Applications	Passphrase-protected private key	Mike Caldwell, Aaron Voisine	Standard	Draft
39	Applications	Mnemonic code for generating deterministic keys	Marek Palatinus, Pavol Rusnak, Aaron Voisine, Sean Bowe	Standard	Accepted
42	Consensus (soft fork)	A finite monetary supply for Bitcoin	Pieter Wuille	Standard	Draft
43	Applications	Purpose Field for Deterministic Wallets	Marek Palatinus, Pavol Rusnak	Informational	Draft
44	Applications	Multi-Account Hierarchy for Deterministic Wallets	Marek Palatinus, Pavol Rusnak	Standard	Accepted
45	Applications	Structure for Deterministic P2SH Multisignature Wallets	Manuel Araoz, Ryan X. Charles, Matias Alejo Garcia	Standard	Accepted
47	Applications			Informational	Draft

Number	Layer	Title	Owner	Type	Status
		Reusable Payment Codes for Hierarchical Deterministic Wallets	Justus Ranvier		
49	Applications	Derivation scheme for P2WPKH-nested-in-P2SH based accounts	Daniel Weigl	Informational	Draft
50		March 2013 Chain Fork Post-Mortem	Gavin Andresen	Informational	Final
60	Peer Services	Fixed Length "version" Message (Relay-Transactions Field)	Amir Taaki	Standard	Draft
61	Peer Services	Reject P2P message	Gavin Andresen	Standard	Final
62	Consensus (soft fork)	Dealing with malleability	Pieter Wuille	Standard	Withdrawn
64	Peer Services	getutxo message	Mike Hearn	Standard	Draft
65	Consensus (soft fork)	OP_CHECKLOCKTIMEVERIFY	Peter Todd	Standard	Final
66	Consensus (soft fork)	Strict DER signatures	Pieter Wuille	Standard	Final
67	Applications	Deterministic Pay-to-script-hash multi-signature addresses through public key sorting	Thomas Kerin, Jean-Pierre Rupp, Ruben de Vries	Standard	Accepted
68	Consensus (soft fork)	Relative lock-time using consensus-enforced sequence numbers	Mark Friedenbach, BtcDrak, Nicolas Dorier, kinoshitajona	Standard	Final
69	Applications	Lexicographical Indexing of Transaction Inputs and Outputs	Kristov Atlas	Informational	Accepted
70	Applications	Payment Protocol	Gavin Andresen, Mike Hearn	Standard	Final
71	Applications	Payment Protocol MIME types	Gavin Andresen	Standard	Final
72	Applications	bitcoin: uri extensions for Payment Protocol	Gavin Andresen	Standard	Final

Number	Layer	Title	Owner	Type	Status
73	Applications	Use “Accept” header for response type negotiation with Payment Request URLs	Stephen Pair	Standard	Final
74	Applications	Allow zero value OP_RETURN in Payment Protocol	Toby Padilla	Standard	Draft
75	Applications	Out of Band Address Exchange using Payment Protocol Encryption	Justin Newton, Matt David, Aaron Voisine, James MacWhyte	Standard	Draft
80		Hierarchy for Non-Colored Voting Pool Deterministic Multisig Wallets	Justus Ranvier, Jimmy Song	Informational	Deferred
81		Hierarchy for Colored Voting Pool Deterministic Multisig Wallets	Justus Ranvier, Jimmy Song	Informational	Deferred
83	Applications	Dynamic Hierarchical Deterministic Key Trees	Eric Lombrozo	Standard	Draft
99		Motivation and deployment of consensus rule changes ([soft/hard]forks)	Jorge Timón	Informational	Draft
101	Consensus (hard fork)	Increase maximum block size	Gavin Andresen	Standard	Withdrawn
102	Consensus (hard fork)	Block size increase to 2MB	Jeff Garzik	Standard	Draft
103	Consensus (hard fork)	Block size following technological growth	Pieter Wuille	Standard	Draft
105	Consensus (hard fork)	Consensus based block size retargeting algorithm	BtcDrak	Standard	Draft
106	Consensus (hard fork)	Dynamically Controlled Bitcoin Block Size Max Cap	Upal Chakraborty	Standard	Draft
107	Consensus (hard fork)	Dynamic limit on the block size	Washington Y. Sanchez	Standard	Draft
109	Consensus (hard fork)	Two million byte size limit with sigop and sighash limits	Gavin Andresen	Standard	Draft
111	Peer Services	NODE_BLOOM service bit	Matt Corallo, Peter Todd	Standard	Accepted
112	Consensus (soft fork)	CHECKSEQUENCEVERIFY	BtcDrak, Mark	Standard	Final

Number	Layer	Title	Owner	Type	Status
			Friedenbach, Eric Lombrozo		
113	Consensus (soft fork)	Median time-past as endpoint for lock-time calculations	Thomas Kerin, Mark Friedenbach	Standard	Final
114	Consensus (soft fork)	Merkelized Abstract Syntax Tree	Johnson Lau	Standard	Draft
120	Applications	Proof of Payment	Kalle Rosenbaum	Standard	Draft
121	Applications	Proof of Payment URI scheme	Kalle Rosenbaum	Standard	Draft
122	Applications	URI scheme for Blockchain references / exploration	Marco Pontello	Standard	Draft
123		BIP Classification	Eric Lombrozo	Process	Draft
124	Applications	Hierarchical Deterministic Script Templates	Eric Lombrozo, William Swanson	Informational	Draft
125	Applications	Opt-in Full Replace-by-Fee Signaling	David A. Harding, Peter Todd	Standard	Accepted
126		Best Practices for Heterogeneous Input Script Transactions	Kristov Atlas	Informational	Draft
130	Peer Services	sendheaders message	Suhas Daftuar	Standard	Accepted
131	Consensus (hard fork)	“Coalescing Transaction” Specification (wildcard inputs)	Chris Priest	Standard	Draft
132		Committee-based BIP Acceptance Process	Andy Chase	Process	Withdrawn
133	Peer Services	feefilter message	Alex Morcos	Standard	Draft
134	Consensus (hard fork)	Flexible Transactions	Tom Zander	Standard	Draft
140	Consensus (soft fork)	Normalized TXID	Christian Decker	Standard	Draft
141	Consensus (soft fork)	Segregated Witness (Consensus layer)	Eric Lombrozo,	Standard	Draft

Number	Layer	Title	Owner	Type	Status
			Johnson Lau, Pieter Wuille		
142	Applications	Address Format for Segregated Witness	Johnson Lau	Standard	Deferred
143	Consensus (soft fork)	Transaction Signature Verification for Version 0 Witness Program	Johnson Lau, Pieter Wuille	Standard	Draft
144	Peer Services	Segregated Witness (Peer Services)	Eric Lombrozo, Pieter Wuille	Standard	Draft
145	API/RPC	getblocktemplate Updates for Segregated Witness	Luke Dashjr	Standard	Draft
146	Consensus (soft fork)	Dealing with signature encoding malleability	Johnson Lau, Pieter Wuille	Standard	Draft
147	Consensus (soft fork)	Dealing with dummy stack element malleability	Johnson Lau	Standard	Draft
150	Peer Services	Peer Authentication	Jonas Schnelli	Standard	Draft
151	Peer Services	Peer-to-Peer Communication Encryption	Jonas Schnelli	Standard	Draft
152	Peer Services	Compact Block Relay	Matt Corallo	Standard	Draft

```
BIP: 9
Title: Version bits with timeout and delay
Authors: Pieter Wuille <pieter.wuille@gmail.com>
         Peter Todd <pete@petertodd.org>
         Greg Maxwell <greg@xiph.org>
         Rusty Russell <rusty@rustcorp.com.au>
Status: Deployed
Type: Informational
Assigned: 2015-10-04
License: PD
```

BIP 9

Abstract

This document specifies a proposed change to the semantics of the 'version' field in Bitcoin blocks, allowing multiple backward-compatible changes (further called "soft forks") to be deployed in parallel. It relies on interpreting the version field as a bit vector, where each bit can be used to track an independent change. These are tallied each retarget period. Once the consensus change succeeds or times out, there is a "fallow" pause after which the bit can be reused for later changes.

Motivation

BIP 34 introduced a mechanism for doing soft-forking changes without a predefined flag timestamp (or flag block height), instead relying on measuring miner support indicated by a higher version number in block headers. As it relies on comparing version numbers as integers however, it only supports one single change being rolled out at once, requiring coordination between proposals, and does not allow for permanent rejection: as long as one soft fork is not fully rolled out, no future one can be scheduled.

In addition, BIP 34 made the integer comparison (nVersion >= 2) a consensus rule after its 95% threshold was reached, removing $2^{31}+2$ values from the set of valid version numbers (all negative numbers, as nVersion is interpreted as a signed integer, as well as 0 and 1). This indicates another downside this approach: every upgrade permanently restricts the set of allowed nVersion field values. This approach was later reused in BIP 66 and BIP 65, which further removed nVersions 2 and 3 as valid options. As will be shown further, this is unnecessary.

Specification

Each soft fork deployment is specified by the following per-chain parameters (further elaborated below):

1. The **name** specifies a very brief description of the soft fork, reasonable for use as an identifier. For deployments described in a single BIP, it is recommended to use the name "bipN" where N is the appropriate BIP number.
2. The **bit** determines which bit in the nVersion field of the block is to be used to signal the soft fork lock-in and activation. It is chosen from the set {0,1,2,…,28}.
3. The **starttime** specifies a minimum median time past of a block at which the bit gains its meaning.
4. The **timeout** specifies a time at which the deployment is considered failed. If the median time past of a block >= timeout and the soft fork has not yet locked in (including this block's bit state), the deployment is considered failed on all descendants of the block.

Selection guidelines

The following guidelines are suggested for selecting these parameters for a soft fork:

1. **name** should be selected such that no two softforks, concurrent or otherwise, ever use the same name.
2. **bit** should be selected such that no two concurrent softforks use the same bit.
3. **starttime** should be set to some date in the future, approximately one month after a software release date including the soft fork. This allows for some release delays, while preventing triggers as a result of parties running pre-release software.
4. **timeout** should be 1 year (31536000 seconds) after starttime.

A later deployment using the same bit is possible as long as the starttime is after the previous one's timeout or activation, but it is discouraged until necessary, and even then recommended to have a pause in between to detect buggy software.

States

With each block and soft fork, we associate a deployment state. The possible states are:

1. **DEFINED** is the first state that each soft fork starts out as. The genesis block is by definition in this state for each deployment.
2. **STARTED** for blocks past the starttime.
3. **LOCKED_IN** for one retarget period after the first retarget period with STARTED blocks of which at least threshold have the associated bit set in nVersion.
4. **ACTIVE** for all blocks after the LOCKED_IN retarget period.
5. **FAILED** for one retarget period past the timeout time, if LOCKED_IN was not reached.

Bit flags

The nVersion block header field is to be interpreted as a 32-bit little-endian integer (as present), and bits are selected within this integer as values (1 << N) where N is the bit number.

Blocks in the STARTED state get an nVersion whose bit position bit is set to 1. The top 3 bits of such blocks must be 001, so the range of actually possible nVersion values is [0x20000000…0x3FFFFFFF], inclusive.

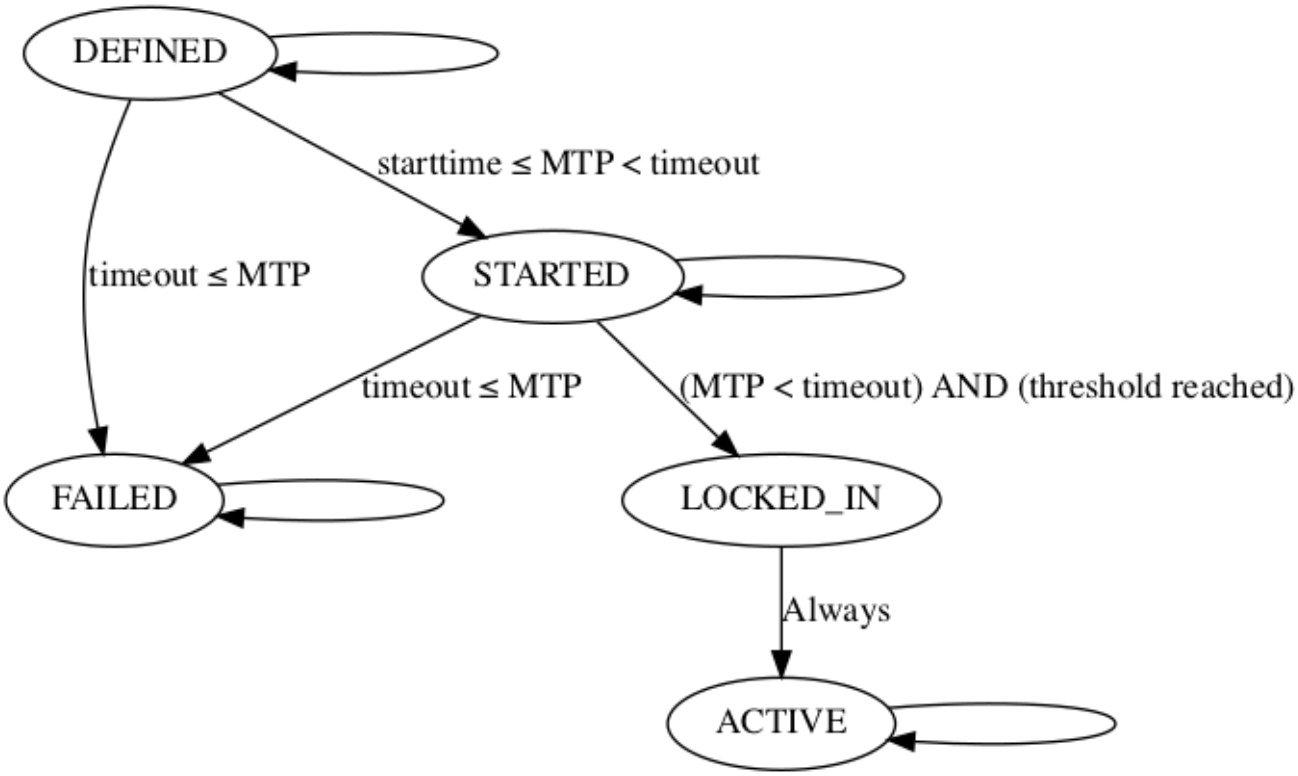
Due to the constraints set by BIP 34, BIP 66 and BIP 65, we only have 0x7FFFFFFB possible nVersion values available. This restricts us to at most 30 independent deployments. By restricting the top 3 bits to 001 we get 29 out of those for the purposes of this proposal, and support two future upgrades for different mechanisms (top bits 010 and 011). When a block nVersion does not have top bits 001, it is treated as if all bits are 0 for the purposes of deployments.

Miners should continue setting the bit in LOCKED_IN phase so uptake is visible, though this has no effect on consensus rules.

New consensus rules

The new consensus rules for each soft fork are enforced for each block that has ACTIVE state.

State transitions



</img>

The genesis block has state DEFINED for each deployment, by definition.

```
State GetStateForBlock(block) {
    if (block.height == 0) {
        return DEFINED;
    }
```

All blocks within a retarget period have the same state. This means that if floor(block1.height / 2016) = floor(block2.height / 2016), they are guaranteed to have the same state for every deployment.

```
    if ((block.height % 2016) != 0) {
        return GetStateForBlock(block.parent);
    }
```

Otherwise, the next state depends on the previous state:

```
    switch (GetStateForBlock(GetAncestorAtHeight(block, block.height - 2016))) {
```

We remain in the initial state until either we pass the start time or the timeout. GetMedianTimePast in the code below refers to the median nTime of a block and its 10 predecessors. The expression GetMedianTimePast(block.parent) is referred to as MTP in the diagram above, and is treated as a monotonic clock defined by the chain.

```
    case DEFINED:
        if (GetMedianTimePast(block.parent) >= timeout) {
```

```
            return FAILED;
        }
        if (GetMedianTimePast(block.parent) >= starttime) {
            return STARTED;
        }
        return DEFINED;
```

After a period in the STARTED state, if we're past the timeout, we switch to FAILED. If not, we tally the bits set, and transition to LOCKED_IN if a sufficient number of blocks in the past period set the deployment bit in their version numbers. The threshold is ≥1916 blocks (95% of 2016), or ≥1512 for testnet (75% of 2016). The transition to FAILED takes precedence, as otherwise an ambiguity can arise. There could be two non-overlapping deployments on the same bit, where the first one transitions to LOCKED_IN while the other one simultaneously transitions to STARTED, which would mean both would demand setting the bit.

Note that a block's state never depends on its own nVersion; only on that of its ancestors.

```
    case STARTED:
        if (GetMedianTimePast(block.parent) >= timeout) {
            return FAILED;
        }
        int count = 0;
        walk = block;
        for (i = 0; i < 2016; i++) {
            walk = walk.parent;
            if (walk.nVersion & 0xE0000000 == 0x20000000 && (walk.nVersion >> bit) & 1 == 1) {
                count++;
            }
        }
        if (count >= threshold) {
            return LOCKED_IN;
        }
        return STARTED;
```

After a retarget period of LOCKED_IN, we automatically transition to ACTIVE.

```
    case LOCKED_IN:
        return ACTIVE;
```

And ACTIVE and FAILED are terminal states, which a deployment stays in once they're reached.

```
    case ACTIVE:
        return ACTIVE;

    case FAILED:
        return FAILED;
    }
}
```

Implementation It should be noted that the states are maintained along block chain branches, but may need recomputation when a reorganization happens.

Given that the state for a specific block/deployment combination is completely determined by its ancestry before the current retarget period (i.e. up to and including its ancestor with height block.height - 1 - (block.height % 2016)), it is possible to implement the mechanism above efficiently and safely by caching the resulting state of every multiple-of-2016 block, indexed by its parent.

Warning mechanism

To support upgrade warnings, an extra "unknown upgrade" is tracked, using the "implicit bit" mask = (block.nVersion & ~expectedVersion) != 0. Mask will be non-zero whenever an unexpected bit is set in nVersion. Whenever LOCKED_IN for the unknown upgrade is detected, the software should warn loudly about the upcoming soft fork. It should warn even more loudly after the next retarget period (when the unknown upgrade is in the ACTIVE state).

getblocktemplate changes

The template request Object is extended to include a new item:

template request			
Key	Required	Type	Description
rules	No	Array of Strings	list of supported softfork deployments, by name

The template Object is also extended:

template			
Key	Required	Type	Description
rules	Yes	Array of Strings	list of softfork deployments, by name, that are active state
vbavailable	Yes	Object	set of pending, supported softfork deployments; each uses the softfork name as the key, and the softfork bit as its value
vbrequired	No	Number	bit mask of softfork deployment version bits the server requires enabled in submissions

The "version" key of the template is retained, and used to indicate the server's preference of deployments. If versionbits is being used, "version" MUST be within the versionbits range of [0x20000000…0x3FFFFFFF]. Miners MAY clear or set bits in the block version WITHOUT any special "mutable" key, provided they are listed among the template's "vbavailable" and (when clearing is desired) NOT included as a bit in "vbrequired".

Softfork deployment names listed in "rules" or as keys in "vbavailable" may be prefixed by a '!' character. Without this prefix, GBT clients may assume the rule will not impact usage of the template as-is; typical examples of this would be when previously valid transactions cease to be valid, such as BIPs 16, 65, 66, 68, 112, and 113. If a client does not understand a rule without the prefix, it may use it unmodified for mining. On the other hand, when this prefix is used, it indicates a more subtle change to the block structure or generation

transaction; examples of this would be BIP 34 (because it modifies coinbase construction) and 141 (since it modifies the txid hashing and adds a commitment to the generation transaction). A client that does not understand a rule prefixed by '!' must not attempt to process the template, and must not attempt to use it for mining even unmodified.

Support for future changes

The mechanism described above is very generic, and variations are possible for future soft forks. Here are some ideas that can be taken into account.

Modified thresholds The 1916 threshold (based on BIP 34's 95%) does not have to be maintained for eternity, but changes should take the effect on the warning system into account. In particular, having a lock-in threshold that is incompatible with the one used for the warning system may have long-term effects, as the warning system cannot rely on a permanently detectable condition anymore.

Conflicting soft forks At some point, two mutually exclusive soft forks may be proposed. The naive way to deal with this is to never create software that implements both, but that is making a bet that at least one side is guaranteed to lose. Better would be to encode "soft fork X cannot be locked-in" as consensus rule for the conflicting soft fork - allowing software that supports both, but can never trigger conflicting changes.

Multi-stage soft forks Soft forks right now are typically treated as booleans: they go from an inactive to an active state in blocks. Perhaps at some point there is demand for a change that has a larger number of stages, with additional validation rules that get enabled one by one. The above mechanism can be adapted to support this, by interpreting a combination of bits as an integer, rather than as isolated bits. The warning system is compatible with this, as (nVersion & ~nExpectedVersion) will always be non-zero for increasing integers.

Rationale

The failure timeout allows eventual reuse of bits even if a soft fork was never activated, so it's clear that the new use of the bit refers to a new BIP. It's deliberately very coarse-grained, to take into account reasonable development and deployment delays. There are unlikely to be enough failed proposals to cause a bit shortage.

The fallow period at the conclusion of a soft fork attempt allows some detection of buggy clients, and allows time for warnings and software upgrades for successful soft forks.

Deployments

A living list of deployment proposals can be found here.

Copyright

This document is placed in the public domain.

```
BIP: 8
Title: Version bits with lock-in by height
Authors: Shaolin Fry <shaolinfry@protonmail.ch>
         Luke Dashjr <luke+bip@dashjr.org>
Status: Complete
Type: Informational
Assigned: 2017-02-01
```

```
License: BSD-3-Clause OR CC0-1.0
Version: 1.0.0
```

BIP 8

Abstract

This document specifies an alternative to BIP9 that corrects for a number of perceived mistakes. Block heights are used for start and timeout rather than POSIX timestamps. It additionally introduces an activation parameter that can guarantee activation of backward-compatible changes (further called "soft forks").

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

Motivation

BIP9 introduced a mechanism for doing parallel soft forking deployments based on repurposing the block nVersion field. Activation is dependent on near unanimous hashrate signalling which may be impractical and result in veto by a small minority of non-signalling hashrate. Super majority hashrate based activation triggers allow for accelerated activation where the majority hash power enforces the new rules in lieu of full nodes upgrading. Since all consensus rules are ultimately enforced by full nodes, eventually any new soft fork will be enforced by the economy. This proposal combines these two aspects to provide optional flag day activation after a reasonable time, as well as for accelerated activation by majority of hash rate before the flag date.

Due to using timestamps rather than block heights, it was found to be a risk that a sudden loss of significant hashrate could interfere with a late activation.

Block time is somewhat unreliable and may be intentionally or unintentionally inaccurate, so thresholds based on block time are not ideal. Secondly, BIP9 specified triggers based on the first retarget after a given time, which is non-intuitive. Since each new block must increase the height by one, thresholds based on block height are much more reliable and intuitive and can be calculated exactly for difficulty retarget.

Specification

Parameters

Each soft fork deployment is specified by the following per-chain parameters (further elaborated below):

1. The **name** specifies a very brief description of the soft fork, reasonable for use as an identifier.
2. The **bit** determines which bit in the nVersion field of the block is to be used to signal the soft fork lock-in and activation. It is chosen from the set {0,1,2,…,28}.
3. The **startheight** specifies the height of the first block at which the bit gains its meaning.
4. The **timeoutheight** specifies a block height at which the miner signalling ends. Once this height has been reached, if the soft fork has not yet locked in (excluding this block's bit state), the deployment is considered failed on all descendants of the block.
5. The **threshold** specifies the minimum number of block per retarget period which indicate lock-in of the soft fork during the subsequent period.

6. The **minimum_activation_height** specifies the height of the first block at which the soft fork is allowed to become active.
7. The **lockinontimeout** boolean if set to true, blocks are required to signal in the final period, ensuring the soft fork has locked in by timeoutheight.

Selection guidelines

The following guidelines are suggested for selecting these parameters for a soft fork:

1. **name** should be selected such that no two softforks, concurrent or otherwise, ever use the same name. For deployments described in a single BIP, it is recommended to use the name "bipN" where N is the appropriate BIP number.
2. **bit** should be selected such that no two concurrent softforks use the same bit. The bit chosen should not overlap with active usage (legitimately or otherwise) for other purposes.
3. **startheight** should be set to some block height in the future. If **minimum_activation_height** is not going to be set, then **startheight** should be set to a height when a majority of economic activity is expected to have upgraded to software including the activation parameters. Some allowance should be made for potential release delays. If **minimum_activation_height** is going to be set, then **startheight** can be set to be soon after software with parameters is expected to be released. This shifts the time for upgrading from before signaling begins to during the LOCKED_IN state.
4. **timeoutheight** should be set to a block height when it is considered reasonable to expect the entire economy to have upgraded by, probably at least 1 year, or 52416 blocks (26 retarget intervals) after **startheight**.
5. **threshold** should be 1815 blocks (90% of 2016), or 1512 (75%) for testnet.
6. **minimum_activation_height** should be set to several retarget periods in the future if the **startheight** is to be very soon after software with parameters is expected to be released. **minimum_activation_height** should be set to a height when a majority of economic activity is expected to have upgraded to software including the activation parameters. This allows more time to be spent in the LOCKED_IN state so that nodes can upgrade. This may be set to 0 to have the LOCKED_IN state be a single retarget period.
7. **lockinontimeout** should be set to true for any softfork that is expected or found to have political opposition from a non-negligible percent of miners. (It can be set after the initial deployment, but cannot be cleared once set.)

A later deployment using the same bit is possible as long as the startheight is after the previous one's timeoutheight or activation, but it is discouraged until necessary, and even then recommended to have a pause in between to detect buggy software.

startheight, **timeoutheight**, and **minimum_activation_height** must be an exact multiple of 2016 (ie, at a retarget boundary), and **timeoutheight** must be at least 4032 blocks (2 retarget intervals) after **startheight**.

States

With each block and soft fork, we associate a deployment state. The possible states are:

1. **DEFINED** is the first state that each soft fork starts out as. The genesis block is by definition in this state for each deployment.
2. **STARTED** for blocks at or beyond the startheight.

3. **MUST_SIGNAL** for one retarget period prior to the timeout, if LOCKED_IN was not reached and **lockinontimeout** is true.
4. **LOCKED_IN** for at least one retarget period after the first retarget period with STARTED (or MUST_SIGNAL) blocks of which at least threshold have the associated bit set in nVersion. A soft fork remains in LOCKED_IN until at least **minimum_activation_height** is reached.
5. **ACTIVE** for all blocks after the LOCKED_IN state.
6. **FAILED** for all blocks after the timeoutheight if LOCKED_IN is not reached.

Bit flags

The nVersion block header field is to be interpreted as a 32-bit little-endian integer (as present), and bits are selected within this integer as values (1 << N) where N is the bit number.

Blocks in the STARTED state get an nVersion whose bit position bit is set to 1. The top 3 bits of such blocks must be 001, so the range of actually possible nVersion values is [0x20000000…0x3FFFFFFF], inclusive.

Due to the constraints set by BIP 34, BIP 66 and BIP 65, we only have 0x7FFFFFFB possible nVersion values available. This restricts us to at most 30 independent deployments. By restricting the top 3 bits to 001 we get 29 out of those for the purposes of this proposal, and support two future upgrades for different mechanisms (top bits 010 and 011). When a block nVersion does not have top bits 001, it is treated as if all bits are 0 for the purposes of deployments.

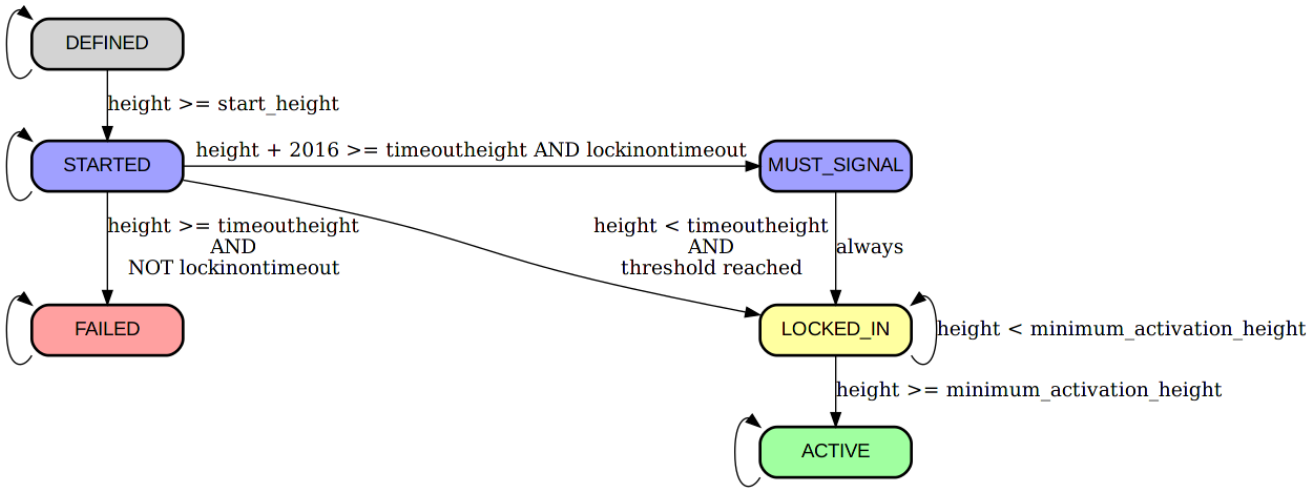
Miners should continue setting the bit in LOCKED_IN phase so uptake is visible, though this has no effect on consensus rules.

New consensus rules

The new consensus rules for each soft fork are enforced for each block that has ACTIVE state.

During the MUST_SIGNAL phase, if **(2016 - threshold)** blocks in the retarget period have already failed to signal, any further blocks that fail to signal are invalid.

State transitions



</img>

Note that when **lockinontimeout** is true, the LOCKED_IN state will be reached no later than at a height of **timeoutheight**. Regardless of the value of **lockinontimeout**, if LOCKED_IN is reached, ACTIVE will be reached either one retarget period later, or at **minimum_activation_height**, whichever comes later.

The genesis block has state DEFINED for each deployment, by definition.

```
State GetStateForBlock(block) {
    if (block.height == 0) {
        return DEFINED;
    }
```

All blocks within a retarget period have the same state. This means that if floor(block1.height / 2016) = floor(block2.height / 2016), they are guaranteed to have the same state for every deployment.

```
    if ((block.height % 2016) != 0) {
        return GetStateForBlock(block.parent);
    }
```

Otherwise, the next state depends on the previous state:

```
    switch (GetStateForBlock(GetAncestorAtHeight(block, block.height - 2016))) {
```

We remain in the initial state until we reach the start block height.

```
    case DEFINED:
        if (block.height >= startheight) {
            return STARTED;
        }
        return DEFINED;
```

After a period in the STARTED state, we tally the bits set, and transition to LOCKED_IN if a sufficient number of blocks in the past period set the deployment bit in their version numbers. If the threshold hasn't been met, lockinontimeout is true, and we are at the last period before the timeout, then we transition to MUST_SIGNAL. If the threshold hasn't been met and we reach the timeout, we transition directly to FAILED.

Note that a block's state never depends on its own nVersion; only on that of its ancestors.

```
    case STARTED:
        int count = 0;
        walk = block;
        for (i = 0; i < 2016; i++) {
            walk = walk.parent;
            if (walk.nVersion & 0xE0000000 == 0x20000000 && (walk.nVersion >> bit) & 1 == 1) {
                ++count;
            }
        }
        if (count >= threshold) {
            return LOCKED_IN;
        } else if (lockinontimeout && block.height + 2016 >= timeoutheight) {
            return MUST_SIGNAL;
```

```
            } else if (block.height >= timeoutheight) {
                return FAILED;
            }
            return STARTED;
```

If we have finished a period of MUST_SIGNAL, we transition directly to LOCKED_IN.

```
        case MUST_SIGNAL:
            return LOCKED_IN;
```

After at least one retarget period of LOCKED_IN, we automatically transition to ACTIVE if the minimum activation height is reached. Otherwise LOCKED_IN continues.

```
        case LOCKED_IN:
            if (block.height >= minimum_activation_height) {
                return ACTIVE;
            } else {
                return LOCKED_IN;
            }
```

And ACTIVE and FAILED are terminal states, which a deployment stays in once they're reached.

```
        case ACTIVE:
            return ACTIVE;

        case FAILED:
            return FAILED;
        }
    }
```

Implementation It should be noted that the states are maintained along block chain branches, but may need recomputation when a reorganization happens.

Given that the state for a specific block/deployment combination is completely determined by its ancestry before the current retarget period (i.e. up to and including its ancestor with height block.height - 1 - (block.height % 2016)), it is possible to implement the mechanism above efficiently and safely by caching the resulting state of every multiple-of-2016 block, indexed by its parent.

Mandatory signalling

Blocks received while in the MUST_SIGNAL phase must be checked to ensure that they signal as required. For example:

```
    if (GetStateForBlock(block) == MUST_SIGNAL) {
        int nonsignal = 0;
        walk = block;
        while (true) {
            if ((walk.nVersion & 0xE0000000) != 0x20000000 || ((walk.nVersion >> bit) & 1) !
= 1) {
                ++nonsignal;
```

```
                if (nonsignal > 2016 - threshold) {
                    return state.Invalid(BlockValidationResult::RECENT_CONSENSUS_CHANGE, "bad-
version-bip8-must-signal");
                }
            }
            if (walk.nHeight % 2016 == 0) {
                // checked every block in this retarget period
                break;
            }
            walk = walk.parent;
        }
    }
```

Implementations should be careful not to ban peers that send blocks that are invalid due to not signalling (or blocks that build on those blocks), as that would allow an incompatible chain that is only briefly longer than the compliant chain to cause a split of the p2p network. If that occurred, nodes that have not set *lockinontimeout* may not see new blocks in the compliant chain, and thus not reorg to it at the point when it has more work, and would thus not be following the valid chain with the most work.

Implementations with *lockinontimeout* set to true may potentially follow a lower work chain than nodes with *lockinontimeout* set to false for an extended period. In order for this not to result in a net split nodes with *lockinontimeout* set to true, those nodes may need to preferentially connect to each other. Deployments proposing that implementations set *lockinontimeout* to true should either use parameters that do not risk there being a higher work alternative chain, or specify a mechanism for implementations that support the deployment to preferentially peer with each other.

Warning mechanism

To support upgrade warnings, an extra "unknown upgrade" is tracked, using the "implicit bit" mask = (block.nVersion & ~expectedVersion) != 0. Mask will be non-zero whenever an unexpected bit is set in nVersion. Whenever LOCKED_IN for the unknown upgrade is detected, the software should warn loudly about the upcoming soft fork. It should warn even more loudly after the next retarget period (when the unknown upgrade is in the ACTIVE state).

getblocktemplate changes

The template request Object is extended to include a new item:

template request			
Key	Required	Type	Description
rules	No	Array of Strings	list of supported softfork deployments, by name

The template Object is also extended:

template			
Key	Required	Type	Description
rules	Yes		list of softfork deployments, by name, that are active state

template			
		Array of Strings	
vbavailable	Yes	Object	set of pending, supported softfork deployments; each uses the softfork name as the key, and the softfork bit as its value
vbrequired	No	Number	bit mask of softfork deployment version bits the server requires enabled in submissions

The "version" key of the template is retained, and used to indicate the server's preference of deployments. If versionbits is being used, "version" MUST be within the versionbits range of [0x20000000…0x3FFFFFFF]. Miners MAY clear or set bits in the block version WITHOUT any special "mutable" key, provided they are listed among the template's "vbavailable" and (when clearing is desired) NOT included as a bit in "vbrequired". Servers MUST set bits in "vbrequired" for deployments in MUST_SIGNAL state, to ensure blocks produced are valid.

Softfork deployment names listed in "rules" or as keys in "vbavailable" may be prefixed by a '!' character. Without this prefix, GBT clients may assume the rule will not impact usage of the template as-is; typical examples of this would be when previously valid transactions cease to be valid, such as BIPs 16, 65, 66, 68, 112, and 113. If a client does not understand a rule without the prefix, it may use it unmodified for mining. On the other hand, when this prefix is used, it indicates a more subtle change to the block structure or generation transaction; examples of this would be BIP 34 (because it modifies coinbase construction) and 141 (since it modifies the txid hashing and adds a commitment to the generation transaction). A client that does not understand a rule prefixed by '!' must not attempt to process the template, and must not attempt to use it for mining even unmodified.

Reference implementation

https://github.com/bitcoin/bitcoin/compare/master…luke-jr:bip8

Contrasted with BIP 9

- The **lockinontimeout** flag is added, providing a way to guarantee transition to LOCKED_IN.
- Block heights are used for the deployment monotonic clock, rather than median-time-past.

Backwards compatibility

BIP8 and BIP9 deployments should not share concurrent active deployment bits. Nodes that only implement BIP9 will not activate a BIP8 soft fork if hashpower threshold is not reached by **timeoutheight**, however, those nodes will still accept the blocks generated by activated nodes.

Deployments

A living list of deployment proposals can be found here.

References

BIP9

[Mailing list discussion]

Copyright

This document is dual licensed as BSD 3-clause, and Creative Commons CC0 1.0 Universal.

Changelog

- **1.0.0** (2026-08-03):
  - Advance to Complete.
- **0.3.2** (2021-02-17):
  - Make activation threshold configurable.
  - Reduce recommended threshold from 95% to 90%.
  - Update guidance for bit selection and naming conventions.
  - Clarify economic majority upgrade expectations.
- **0.3.1** (2021-02-04):
  - Simplify pseudocode for MUST_SIGNAL validation check.
- **0.3.0** (2021-02-02):
  - Make signaling during LOCKED_IN recommended instead of mandatory.
  - Allow some blocks to not signal during MUST_SIGNAL as long as threshold is met.
- **0.2.0** (2020-10-17):
  - Replace FAILING state with MUST_SIGNAL phase.
  - Require startheight and timeoutheight to be at retarget boundaries.
  - Require timeoutheight to be at least 4032 blocks after startheight.
  - Add note about lockinontimeout=true peer connectivity.
- **0.1.0** (2020-06-26):
  - Return document Draft status.
  - Fix timeout logic and state transition ordering.
  - Add minimum_activation_height parameter.
  - Update GBT section for MUST_SIGNAL state.
  - Add reference implementation link.
  - Fix pseudocode initialization bug.
- **0.0.2** (2020-02-26):
  - Move to Rejected due to 3-year time-out.
- **0.0.1** (2017-02-01):
  - Initial Draft published.

```
BIP: 90
Title: Buried Deployments
Authors: Suhas Daftuar <sdaftuar@chaincode.com>
Comments-Summary: Mostly Recommended for implementation, with some Discouragement
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0090
Status: Deployed
Type: Informational
Assigned: 2016-11-08
License: PD
```

BIP 90

Abstract

Prior soft forks (BIP 34, BIP 65, and BIP 66) were activated via miner signaling in block version numbers. Now that the chain has long since passed the blocks at which those consensus rules have triggered, we can (as a simplification) replace the trigger mechanism by caching the block heights at which those consensus rules became enforced.

Motivation

BIPs 34, 65 and 66 were deployed on mainnet using miner signaling using block version numbers. In short, new consensus rules were proposed for use in blocks with a higher version number (N+1) than the prevailing block version (N) in use on the network, and those rules became enforced under the following conditions:

1. 75% rule: If 750 of the prior 1000 blocks are version N+1 or higher, then blocks with version N+1 or higher must correctly enforce the new consensus rule.
2. 95% rule: If 950 of the prior 1000 blocks are version N+1 or higher, then blocks with version less than N+1 are invalid.

Please see those BIPs for more details.

Note that this trigger mechanism is dependent on the chain history. To validate a block, we must test whether the trigger was met by looking at the previous 1000 blocks in the chain before it, which can be inefficient.

In addition, this mechanism for code deployments have been deprecated in favor of BIP 9 deployments, which offer several advantages (please see BIP 9).

Thus we propose elimination of the logic implementing these kinds of deployments, by replacing the test which governs enforcement of BIP 34, BIP 65, and BIP 66 with simple height checks, which we choose to be the block height triggering the 95% activation rule on mainnet for each of those deployments. This simplification of the consensus rules would reduce the technical debt associated with deployment of those consensus changes.

Considerations

It is technically possible for this to be a non-backwards compatible change. For example, if an alternate chain were created in which BIP 34's 95% activation triggered at a lower height (H') than it did on the current mainnet chain (H), then older software would enforce that version 1 blocks were invalid at heights between H' and H, while newer software implementing this change would not. Similarly, this BIP proposes doing away with the 75% threshold check altogether, which means, for example, that a version 2 block forking off of mainnet at height H-1 which omitted the height in coinbase would be invalid to older software, while accepted by newer software.

However, while newer software and older software might validate old blocks differently, that could only cause a consensus split if there were an extremely large blockchain reorganization onto a chain built off such a block. As of November 2016, the most recent of these changes (BIP 65, enforced since December 2015) has nearly 50,000 blocks built on top of it. The occurrence of such a reorg that would cause the activating block to be disconnected would raise fundamental concerns about the security assumptions of Bitcoin, a far bigger issue than any non-backwards compatible change.

So while this proposal could *theoretically* result in a consensus split, it is extremely unlikely, and in particular any such circumstances would be sufficiently damaging to the Bitcoin network to dwarf any concerns about the effects of this proposed change.

Specification

The BIP 34, 66, and 65 activation heights are set to 227931, 363725, and 388381, respectively.

The 1000-block lookback test, first described in BIP 34, is no longer performed during validation of any blocks. Instead, a new check is added:

```
   if((block.nVersion < 2 && nHeight >= consensusParams.BIP34Height) ||
      (block.nVersion < 3 && nHeight >= consensusParams.BIP66Height) ||
      (block.nVersion < 4 && nHeight >= consensusParams.BIP65Height))
           return state.Invalid(false, REJECT_OBSOLETE, strprintf("bad-
version(0x%08x)", block.nVersion),
                                strprintf("rejected nVersion=0x%08x block", block.nVersion));
```

Furthermore, rather than consider the block versions of the prior 1000 blocks to determine whether to enforce BIP 34, BIP 65, or BIP 66 on a given block, we instead just compare the height of the block being validated with the stored activation heights:

```
   // Enforce rule that the coinbase starts with serialized block height
   if (nHeight >= consensusParams.BIP34Height)
   {
       CScript expect = CScript() << nHeight;
       if (block.vtx[0].vin[0].scriptSig.size() < expect.size() ||
           !std::equal(expect.begin(), expect.end(), block.vtx[0].vin[0].scriptSig.begin())) {
           return state.DoS(100, false, REJECT_INVALID, "bad-cb-
height", false, "block height mismatch in coinbase");
       }
   }
```

and

```
   // Start enforcing the DERSIG (BIP66) rule
   if (pindex->nHeight >= chainparams.GetConsensus().BIP66Height) {
       flags |= SCRIPT_VERIFY_DERSIG;
   }

   // Start enforcing CHECKLOCKTIMEVERIFY (BIP65) rule
   if (pindex->nHeight >= chainparams.GetConsensus().BIP65Height) {
       flags |= SCRIPT_VERIFY_CHECKLOCKTIMEVERIFY;
   }
```

Please see the implementation for additional details.

Implementation

https://github.com/bitcoin/bitcoin/pull/8391.

References

BIP34 Block v2, Height in Coinbase

BIP66 Strict DER signatures

BIP65 OP_CHECKLOCKTIMEVERIFY

BIP9 Version bits with timeout and delay

Acknowledgements

Thanks to Nicolas Dorier for drafting an initial version of this BIP, and to Alex Morcos, Matt Corallo, and Greg Maxwell for suggestions and feedback.

Copyright

This document is placed in the public domain.

```
BIP: 50
Title: March 2013 Chain Fork Post-Mortem
Authors: Gavin Andresen <gavinandresen@gmail.com>
Status: Deployed
Type: Informational
Assigned: 2013-03-20
License: PD
```

BIP 50

What went wrong

A block that had a larger number of total transaction inputs than previously seen was mined and broadcasted. Bitcoin 0.8 nodes were able to handle this, but some pre-0.8 Bitcoin nodes rejected it, causing an unexpected fork of the blockchain. The pre-0.8-incompatible chain (from here on, the 0.8 chain) at that point had around 60% of the mining hash power ensuring the split did not automatically resolve (as would have occurred if the pre-0.8 chain outpaced the 0.8 chain in total work, forcing 0.8 nodes to reorganise to the pre-0.8 chain).

In order to restore a canonical chain as soon as possible, BTCGuild and Slush downgraded their Bitcoin 0.8 nodes to 0.7 so their pools would also reject the larger block. This placed majority hashpower on the chain without the larger block, thus eventually causing the 0.8 nodes to reorganise to the pre-0.8 chain.

During this time there was at least one large double spend. However, it was done by someone experimenting to see if it was possible and was not intended to be malicious.

What went right

- The split was detected very quickly.
- The right people were online and available in IRC or could be contacted directly.

- Marek Palatinus (Slush) and Michael Marsee (Eleuthria of BTCGuild) quickly downgraded their nodes to restore a pre-0.8 chain as canonical, despite the fact that this caused them to sacrifice significant amounts of money.
- Deposits to the major exchanges and payments via BitPay were also suspended (and then un-suspended) very quickly.
- Fortunately, the only attack on a merchant was done by someone who was not intending to actually steal money.

Root cause

Bitcoin versions prior to 0.8 configure an insufficient number of Berkeley DB locks to process large but otherwise valid blocks. Berkeley DB locks have to be manually configured by API users depending on anticipated load. The manual says this:

The recommended algorithm for selecting the maximum number of locks, lockers, and lock objects is to run the application under stressful conditions and then review the lock system's statistics to determine the maximum number of locks, lockers, and lock objects that were used. Then, double these values for safety.

With the insufficiently high BDB lock configuration, it implicitly had become a network consensus rule determining block validity (albeit an inconsistent and unsafe rule, since the lock usage could vary from node to node).

Because max-sized blocks had been successfully processed on the testnet, it did not occur to anyone that there could be blocks that were smaller but require more locks than were available. Prior to 0.7 unmodified mining nodes self-imposed a maximum block size of 500,000 bytes, which further prevented this case from being triggered. 0.7 made the target size configurable and miners had been encouraged to increase this target in the week prior to the incident.

Bitcoin 0.8 did not use Berkeley DB. It switched to LevelDB instead, which did not implement the same locking limits as BDB. Therefore it was able to process the forking block successfully.

Note that BDB locks are also required during processing of re-organizations. Versions prior to 0.8 may be unable to process some valid re-orgs.

This would be an issue even if the entire network was running version 0.7.2. It is theoretically possible for one 0.7.2 node to create a block that others are unable to validate, or for 0.7.2 nodes to create block re-orgs that peers cannot validate, because the contents of each node's blkindex.dat database is not identical, and the number of locks required depends on the exact arrangement of the blkindex.dat on disk (locks are acquired per-page).

Action items

Immediately

Done: Release a version 0.8.1, forked directly from 0.8.0, that, for the next two months has the following new rules:

1. Reject blocks that would probably cause more than 10,000 locks to be taken.
2. Limit the maximum block-size created to 500,000 bytes

3. Release a patch for older versions that implements the same rules, but also increases the maximum number of locks to 537,000
4. Create a web page on bitcoin.org that will urge users to upgrade to 0.8.1, but will tell them how to set DB_CONFIG to 537,000 locks if they absolutely cannot.
5. Over the next 2 months, send a series of alerts to users of older versions, pointing to the web page.

Alert system

Done: Review who has access to the alert system keys, make sure they all have contact information for each other, and get good timezone overlap by people with access to the keys.

Done: Implement a new bitcoind feature so services can get timely notification of alerts:
-alertnotify=<command> Run command when an AppliesToMe() alert is received.

Done: Pre-generate 52 test alerts, and set a time every week when they are broadcast on -testnet (so -alertnotify scripts can be tested in as-close-to-real-world conditions as possible).

Idea from Michael Gronager: encourage merchants/exchanges (and maybe pools) to run new code behind a bitcoind running the network-majority version.

Safe mode

Done: Perhaps trigger an alert if there is a long enough side chain detected, even if it is not the main chain. Pools could use this to automatically suspend payouts if a long side-chain suddenly appeared out of nowhere (it's hard for an attacker to mine such a thing).

Testing

Start running bots on the testnet that grab some coins from a testnet faucet, generate large numbers of random transactions that split/recombine them and then send them back to the faucet. Randomized online testing on the testnet might have revealed the pathological block type earlier.

Double spending

A double spend attack was successful, despite that both sides of the chain heard about the transactions in the same order. The reason is most likely that the memory pools were cleared when the mining pool nodes were downgraded. A solution is for nodes to sync their mempools to each other at startup, however, this requires a memory pool expiry policy to be implemented as currently node restarts are the only way for unconfirmed transactions to be evicted from the system.

Resolution

On 16 August, 2013 block 252,451 (0x0000000000000024b58eeb1134432f00497a6a860412996e7a260f47126eed07) was accepted by the main network, forking unpatched nodes off the network.

Copyright

This document is placed in the public domain.

```
BIP: 325
Layer: Applications
Title: Signet
Authors: Karl-Johan Alm <karljohan-alm@garage.co.jp>
         Anthony Towns <aj@erisian.com.au>
Status: Complete
Type: Specification
Assigned: 2019-03-20
License: CC0-1.0
```

BIP 325

Abstract

A new type of test network where signatures are used in addition to proof of work for block progress, enabling much better coordination and robustness (be reliably unreliable), for persistent, longer-term testing scenarios involving multiple independent parties.

Motivation

Testnet is a great place to try out new things without risking real money, but it is notoriously unreliable. Huge block reorgs, long gaps in between blocks being mined or sudden bursts of blocks in rapid succession mean that realistic testing of software, especially involving multiple independent parties running software over an extended period of time, becomes infeasible in practice.

A new type of test network would be more suitable for integration testing by organizations such as exchanges, or testing of next generation Layer-2 protocols like Eltoo or sidechain pegs. The goal is not to be perfectly reliable but rather to have a predictable amount of unreliability. You want a test network to behave like mainnet (i.e. no thousands of block reorgs) while also making it easier to trigger expected but rare events like a 6-block reorg. Regtest is not suitable for longer-term scenarios involving multiple independent parties because creating blocks costs nothing, so any party can completely control the test network.

Specification

A new type of network ("signet"), which takes an additional consensus parameter called the challenge (scriptPubKey). The challenge can be a simple pubkey (P2PKH style), or a k-of-n multisig, or any other script you would want.

Signet requires all blocks to have a BIP 141 commitment in the coinbase transaction. In order to provide a non-empty solution to the block challenge the block's BIP 141 commitment's optional data must include an additional commitment of the signature/solution for the block:

```
1-5 bytes - Push the following (4 + x + y) bytes
4 bytes - Signet header (0xecc7daa2)
x bytes - scriptSig
y bytes - scriptWitness
```

In the special case where an empty solution is valid (ie scriptSig and scriptWitness are both empty) this additional commitment can optionally be left out. This special case is to allow non-signet-aware block generation code to be used to test a custom signet chain where the challenge is trivially true.

The scriptSig is serialized by first encoding its length as CompactSize. The scriptWitness stack is serialized as described in BIP 141.

Any push operations that do not start with the 4 byte Signet header are ignored. Multiple push operations with the 4 byte Signet header are ignored except for the first instance of the header.

To sign the block or verify a block signature, two virtual transactions, each with a single input and output are constructed from the block as follows.

The "to_spend" transaction is:

```
nVersion = 0
nLockTime = 0
vin[0].prevout.hash = 0000...000
vin[0].prevout.n = 0xFFFFFFFF
vin[0].nSequence = 0
vin[0].scriptSig = OP_0 PUSH72[ block_data ]
vin[0].scriptWitness = []
vout[0].nValue = 0
vout[0].scriptPubKey = signet_challenge
```

where block_data is the serialization of the block's nVersion, hashPrevBlock, signet_merkle_root, and nTime. The `signet_merkle_root` is obtained by generating the merkle root of the block transactions, after modifying the coinbase witness commitment by replacing the signet solution with an empty solution (that is, the witness commitment includes a four byte push of the Signet header with no additional solution data, and no prior pushes beginning with the Signet header). This means the merkle root of the block is different from the merkle root in the signet commitment. This is needed, because the signature can never be included in the very message (in this case, a block) that is being signed.

The "to_sign" transaction is:

```
nVersion = 0
nLockTime = 0
vin[0].prevout.hash = to_spend.txid
vin[0].prevout.n = 0
vin[0].nSequence = 0
vin[0].sigScript = [ signet_solution sigScript (x bytes), if any ]
vin[0].scriptWitness = [ signet_solution scriptWitness (y bytes), if any ]
vout[0].nValue = 0
vout[0].scriptPubKey = OP_RETURN
```

The scriptSig and/or scriptWitness for `vin[0]` are filled in from the Signet header push above.

To simplify block generation (mining), the signature also does not commit to the block nonce value, so that rolling the nonce to generate proof-of-work does not also require regenerating signatures. When grinding proof of work, the extended nonce cannot be used as it would invalidate the signature. Instead, simply resigning the same (or an updated) block will give a new search space.

A block is considered fully validated only if the to_sign transaction is a valid spend of the to_spend transaction. It is recommended that this verification is done directly before or after the witness commitment verification, as the data required to do both is approximately the same.

There is one other acceptable special case: if a block's challenge is e.g. `OP_TRUE` (`0x51`), where an empty solution would result in success, the block is also considered valid if the signet commitment is absent.

Minimum Difficulty

The minimum difficulty for blocks in signet is approximately 0.0011265, corresponding to a maximum target of `00000377ae0000000000000000000000000000000000000000000000000000` which is represented by an nBits value of `0x1e0377ae`.

This value is chosen so that multiplying the maximum target by maximum retarget timespan (8 weeks measured in seconds) does not overflow a uint256, making it easier to do difficulty adjustment calculations.

Genesis Block and Message Start

The genesis block is the same for all signet networks, whereas the message start is defined as the first four bytes of the sha256d of the challenge script as a single data push (see below).

Genesis Block

- Time stamp: 1598918400
- Nonce: 52613770
- Difficulty: 0x1e0377ae
- Version: 1

The resulting genesis block hash is 00000008819873e925422c1ff0f99f7cc9bbb232af63a077a480a3633bee1ef6, and the block hex is
010000000000000000000000000000000000000000000000000000000000000000000000000000003ba3edfd7a7b12b27ac72c3e67768f617fc81bc

Message Start

The message start is defined as the first four bytes of the sha256d of the challenge script, as a single push (i.e. prefixed with the challenge script length). Example:

- Challenge script = 512103ad5e0edad18cb1f0fc0d28a3d4f1f3e445640337489abb10404f2d1e086be43051ae
- Sha256d(len || challenge script) = sha256d(25512103ad…51ae) = 7ec653a59b1912f9db10da2c461ed827d48f9404d5ef0346a6c94aadd4203646
- First four bytes = the message start = 7ec653a5

Compatibility

This specification is backwards compatible in the sense that existing software can use Signet out of the box.

Simply by adding the network parameters for signet (magic number, etc), a client can connect to and use any signet network without further modifications. The block headers have valid proof of work, so clients can trivially check that blocks are "probably" valid.

However, anyone can mine blocks that are accepted by the client for any given signet network. These blocks do not contain the required signatures, however, so any fully validating node will promptly reject them. As such, clients need to either validate the block signature inside the coinbase transaction, or connect to trusted peers.

Other software need not add block signature validation code that they will not use in production. This is adequate for non-production test purposes where the goal is to have a network behave as much like mainnet as possible.

Reference implementation

Pull request at https://github.com/bitcoin/bitcoin/pull/18267

References

1. Original mailing list thread: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2019-March/016734.html
2. Bitcoin Wiki entry: https://en.bitcoin.it/wiki/Signet

Copyright

This document is licensed under the Creative Commons CC0 1.0 Universal license.

```
  BIP: 94
  Layer: Applications
  Title: Testnet 4
  Authors: Fabian Jahr <fjahr@protonmail.com>
  Status: Deployed
  Type: Specification
  Assigned: 2024-05-27
  License: CC0-1.0
  Discussion: https://gnusha.org/pi/bitcoindev/
CADL_X_eXjbRFROuJU0b336vPVy5Q2RJvhcx64NSNPH-3fDCUfw@mail.gmail.com/
              https://gnusha.org/pi/bitcoindev/a6e3VPsXJf9p3gt_FmNF_Up-wrFuNMKTN30-
xCSDHBKXzXnSpVflIZIj2NQ8Wos4PhQCzI2mWEMvIms_FAEs7rQdL15MpC_Phmu_fnR9iTg=@protonmail.com/
              https://github.com/bitcoin/bitcoin/pull/29775
```

BIP 94

Abstract

A new test network with the goal to replace Testnet 3. This network comes with small but important improvements of the consensus rules, that should make it impractical to attack the network using only CPU mining.

Motivation

Quoting the original mailing list post from Jameson Lopphttps://gnusha.org/pi/bitcoindev/CADL_X_eXjbRFROuJU0b336vPVy5Q2RJvhcx64NSNPH-3fDCUfw@mail.gmail.com/:

> <poem> Testnet3 has been running for 13 years. It's on block 2.5 million something and the block reward is down to ~0.014 TBTC, so mining is not doing a great job at distributing testnet coins anymore.
>
> The reason the block height is insanely high is due to a rather amusing edge case bug that causes the difficulty to regularly get reset to 1, which causes a bit of havoc. If you want a deep dive into the quirk: https://blog.lopp.net/the-block-storms-of-bitcoins-testnet/
>
> Testnet3 is being actively used for scammy airdrops; those of us who tend to be generous with our testnet coins are getting hounded by non-developers chasing cheap gains.
>
> As a result, TBTC is being actively bought and sold; one could argue that the fundamental principle of testnet coins having no value has been broken. </poem>

Since then the issue with block storms has been further demonstrated on Testnet 3 when three years' worth of blocks were mined in a few weeks while rendering the network practically unusable at the same time.

Specification

Testnet 4 follows the same consensus rules as mainnet with the following three exceptions. Additionally, all soft forks that are active on mainnet as of May 2024 are enforced from genesis.

1. 20-minute Exception Rule

This rule was implemented in Testnet 3https://github.com/bitcoin/bitcoin/pull/686 and is preserved in Testnet 4.

Rule Specification

1. For any block except the first block in a difficulty period:

```
a. If the block's timestamp is >20 minutes past the timestamp of the previous block
b. Then the block MUST use the minimum difficulty value (``nBits`` = ``0x1d00ffff``), regardless of the ne
```

2. The first block of each difficulty period MUST use the actual network difficulty.

This rule enables CPU mining on testnet but has led to block stormshttps://blog.lopp.net/the-block-storms-of-bitcoins-testnet/ which rule #2 below addresses.

2. Block Storm Fix

This is a new rule to address block storms caused by the 20-minute exception.

Problem Statement

In Mainnet and Testnet 3, the difficulty adjustment calculation uses the difficulty value of the last block in the previous period as its base. When the 20-minute exception is applied to this last block, it is mined at difficulty 1, causing the next period's difficulty to be constrained between 1-4, leading to block storms.

Rule Specification

1. For difficulty adjustment calculations between periods:

```
a. The base difficulty value MUST be taken from the first block of the previous difficulty period
b. NOT from the last block as in previous implementations
```

2. The adjustment factor calculation remains unchanged:

```
a. Multiplication factor based on the duration of the previous difficulty period
b. Limited to no less than 1/4 and no more than 4x
```

This change ensures that the actual network difficulty is used for adjustment calculations rather than potentially manipulated values from the last block in a period.

3. Time Warp Attack Prevention

This rule prevents time warp attacks that could otherwise be used to amplify block stormsA perpetual block storm attack with entire difficulty periods being authored in less than 3.5 days that resets the difficulty to the minimum in the last block of every difficulty period would adjust to a new actual difficulty of 4 every period. An attacker that additionally leverages a time warp attack would start their attack by holding back timestamps until the latest block's timestamp is at least two weeks in the past, and then limiting their block rate to six blocks per second, incrementing the timestamp on every sixth block. Only on the last block they would use the current time, which both resets the difficulty to one per the 20-minute exception and would result in a difficulty adjustment keeping the difficulty at the minimum due to the elapsed time exceeding the target. This would allow lower the difficulty for all blocks to difficulty 1 instead of difficulty 4.

Rule Specification

1. For any block whose height modulo 2016 equals 0 (i.e., the first block of each difficulty period):

```
a. The block's ``nTime`` field MUST be greater than or equal to the ``nTime`` field of the immediately pri
```

2. These blocks MUST still comply with existing Median-Time-Past `nTime` restrictions

This rule is based on The Great Consensus Cleanup proposalhttps://github.com/TheBlueMatt/bips/blob/cleanup-softfork/bip-XXXX.mediawiki and prevents miners from manipulating timestamps to artificially lower difficulty.

Rationale

The applied changes were the result of discussions on the mailing list and the PR. The selected changes try to strike a balance between minimal changes to the network (keeping it as close to mainnet as possible) while making it more robust against attackers that try to disrupt the network. Several alternative designs were considered:

- For the block storm fix an alternative fix could have been to prevent the last block in a difficulty period from applying the existing difficulty exception. Both solutions were deemed acceptable and there was no clear preference among reviewers.

- Removal of the 20-minute exception was discussed but dismissed since several reviewers insisted that it was a useful feature allowing non-standard transactions to be mined with just a CPU. The 20-minute exception also allows CPU users to move the chain forward (except on the first block that needs to be mined at actual difficulty) in case a large amount of hash power suddenly leaves the network. This would allow the chain to recover to a normal difficulty level faster if left stranded at high difficulty.
- Increase of minimum difficulty was discussed but dismissed as it would categorically prevent participation in the network using a CPU miner (utilizing the 20-minute exception).
- Increase of the delay in the 20-minute exception was suggested but did not receive significant support.
- Re-enabling `acceptnonstdtxn` in bitcoin core by default was dismissed as it had led to confusion among layer-2s that had used testnet for transaction propagation tests and expected it to behave similarly to mainnet.
- Motivating miners to re-org min difficulty blocks was suggested, but was considered out of scope for this BIP, since adoption of such a mining policy remains available after Testnet 4 is deployed. As 20-minute exception blocks only contribute work corresponding to difficulty one to the chaintip, and actual difficulty blocks should have a difficulty magnitudes higher, a block mined at actual difficulty could easily replace even multiple 20-minute exception blocks.
- Persisting the real difficulty in the version field was suggested to robustly prevent exploits of the 20-minute exception while allowing it to be used on any block, but did not receive a sufficient level of support to justify the more invasive change.

One known downside of the chosen approach is that if the difficulty is gradually raised by a miner with significant hash rate, and this miner disappears, then each difficulty adjustment period requires one block at the actual difficulty.

This would cause the network to stall once per difficulty adjustment period until the real difficulty is adjusted downwards enough for the remaining hash rate to find this block in reasonable time.

Network Parameters

Consensus Rules

All consensus rules active on mainnet at the time of this proposal are enforced from block 1, the newest of these rules being the Taproot softfork.

Genesis Block

- Message: `03/May/2024 000000000000000000001ebd58c244970b3aa9d783bb001011fbe8ea8e98e00e`
- Pubkey: `0000000000000000000000000000000000000000000000000000000000000000000000000000000000`
- Time stamp: 1714777860
- Nonce: 393743547
- Difficulty: `0x1d00ffff`
- Version: 1

The resulting genesis block hash is `00000000da84f2bafbbc53dee25a72ae507ff4914b867c565be350b0da8bf043`, and the block hex is
`0100000000000000000000000000000000000000000000000000000000000000000000004e7b2b9128fe0291db0693af2ae418b767e6`

Message Start

The message start is defined as `0x1c163f28`. These four bytes were randomly generated and have no special meaning.

Network Parameters

The default p2p port for Testnet 4 is `48333`.

Backwards Compatibility

The rules used by Testnet 4 are backwards compatible to the rules of Testnet 3. Existing software that implements support for Testnet 3 would only require addition of the network parameters (magic number, genesis block, etc.) to be able to follow Testnet 4.

However, implementations that only implement Testnet 3's rules would accept a chain that violates Testnet 4's rules and are therefore susceptible to being forked off. It is recommended that any implementations check blocks in regard to all the new rules of Testnet 4 and reject blocks that fail to comply.

Reference implementation

Pull request at https://github.com/bitcoin/bitcoin/pull/29775

References

Copyright

This document is licensed under the Creative Commons CC0 1.0 Universal license.

Addresses and Encodings

Before a wallet can pay anyone, it has to write down where the coins should go. That string in the UI is not the script itself. It is an encoding of a script or a witness program, plus a checksum, plus a network marker so you do not send mainnet coins to a testnet key.

This chapter follows that encoding as it evolved. BIP 13 is the Base58Check format for pay-to-script-hash, which is how most multisig looked for years. BIP 173 introduces Bech32 for native SegWit, and BIP 350 tightens the checksum (Bech32m) for Taproot and later witness versions. BIP 321 is the URI scheme wallets use when they share a payment request as a `bitcoin:` link.

BIP 179 is a small informational note with a large effect on language: what we casually call an "address" is really a payment recipient identifier. Reading it first or last both work. I put it at the end so the encodings have already earned the word.

In this chapter:

- BIP 13 — Address Format for pay-to-script-hash
- BIP 173 — Bech32

- BIP 350 — Bech32m
- BIP 321 — URI Scheme
- BIP 179 — Name for payment recipient identifiers

```
BIP: 13
Layer: Applications
Title: Address Format for pay-to-script-hash
Authors: Gavin Andresen <gavinandresen@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2011-10-18
```

BIP 13

Abstract

This BIP describes a new type of Bitcoin address to support arbitrarily complex transactions. Complexity in this context is defined as what information is needed by the recipient to respend the received coins, in contrast to needing a single ECDSA private key as in current implementations of Bitcoin.

In essence, an address encoded under this proposal represents the encoded hash of a script, rather than the encoded hash of an ECDSA public key.

Motivation

Enable "end-to-end" secure wallets and payments to fund escrow transactions or other complex transactions. Enable third-party wallet security services.

Specification

The new bitcoin address type is constructed in the same manner as existing bitcoin addresses (see Base58Check encoding):

```
base58-encode: [one-byte version][20-byte hash][4-byte checksum]
```

Version byte is 5 for a main-network address, 196 for a testnet address. The 20-byte hash is the hash of the script that will be used to redeem the coins. And the 4-byte checksum is the first four bytes of the double SHA256 hash of the version and hash.

Rationale

One criticism is that bitcoin addresses should be deprecated in favor of a more user-friendly mechanism for payments, and that this will just encourage continued use of a poorly designed mechanism.

Another criticism is that bitcoin addresses are inherently insecure because there is no identity information tied to them; if you only have a bitcoin address, how can you be certain that you're paying who or what you think you're paying?

Furthermore, truncating SHA256 is not an optimal checksum; there are much better error-detecting algorithms. If we are introducing a new form of Bitcoin address, then perhaps a better algorithm should be used.

This is one piece of the simplest path to a more secure bitcoin infrastructure. It is not intended to solve all of bitcoin's usability or security issues, but to be an incremental improvement over what exists today. A future BIP or BIPs should propose more user-friendly mechanisms for making payments, or for verifying that you're sending a payment to the Free Software Foundation and not Joe Random Hacker.

Assuming that typing in bitcoin addresses manually will become increasingly rare in the future, and given that the existing checksum method for bitcoin addresses seems to work "well enough" in practice and has already been implemented multiple times, the Author believes no change to the checksum algorithm is necessary.

The leading version bytes are chosen so that, after base58 encoding, the leading character is consistent: for the main network, byte 5 becomes the character '3'. For the testnet, byte 196 is encoded into '2'.

Backwards Compatibility

This proposal is not backwards compatible, but it fails gracefully– if an older implementation is given one of these new bitcoin addresses, it will report the address as invalid and will refuse to create a transaction.

Reference Implementation

See base58.cpp/base58.h at https://github.com/bitcoin/bitcoin/tree/master/src

See Also

- BIP 12: OP_EVAL, the original P2SH design
- BIP 16: Pay to Script Hash (aka "/P2SH/")
- BIP 17: OP_CHECKHASHVERIFY, another P2SH design

```
BIP: 173
Layer: Applications
Title: Base32 address format for native v0-16 witness outputs
Authors: Pieter Wuille <pieter.wuille@gmail.com>
         Greg Maxwell <greg@xiph.org>
Comments-Summary: No comments yet.
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0173
Status: Deployed
Type: Informational
Assigned: 2017-03-20
License: BSD-2-Clause
Replaces: 142
Proposed-Replacement: 350
```

BIP 173

Introduction

Abstract

This document proposes a checksummed base32 format, "Bech32", and a standard for native segregated witness output addresses using it.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

For most of its history, Bitcoin has relied on base58 addresses with a truncated double-SHA256 checksum. They were part of the original software and their scope was extended in BIP13 for Pay-to-script-hash (P2SH). However, both the character set and the checksum algorithm have limitations:

- Base58 needs a lot of space in QR codes, as it cannot use the *alphanumeric mode*.
- The mixed case in base58 makes it inconvenient to reliably write down, type on mobile keyboards, or read out loud.
- The double SHA256 checksum is slow and has no error-detection guarantees.
- Most of the research on error-detecting codes only applies to character-set sizes that are a prime power, which 58 is not.
- Base58 decoding is complicated and relatively slow.

Included in the Segregated Witness proposal are a new class of outputs (witness programs, see BIP141), and two instances of it ("P2WPKH" and "P2WSH", see BIP143). Their functionality is available indirectly to older clients by embedding in P2SH outputs, but for optimal efficiency and security it is best to use it directly. In this document we propose a new address format for native witness outputs (current and future versions).

This replaces BIP142, and was previously discussed here (summarized here).

Examples

All examples use public key `0279BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798`. The P2WSH examples use `key OP_CHECKSIG` as script.

- Mainnet P2WPKH: `bc1qw508d6qejxtdg4y5r3zarvary0c5xw7kv8f3t4`
- Testnet P2WPKH: `tb1qw508d6qejxtdg4y5r3zarvary0c5xw7kxpjzsx`
- Mainnet P2WSH: `bc1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3qccfmv3`
- Testnet P2WSH: `tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3q0sl5k7`

Specification

We first describe the general checksummed base32**Why use base32 at all?** The lack of mixed case makes it more efficient to read out loud or to put into QR codes. It does come with a 15% length increase, but that does not matter when copy-pasting addresses. format called *Bech32* and then define Segregated Witness addresses using it.

Bech32

A Bech32**Why call it Bech32?** “Bech” contains the characters BCH (the error detection algorithm used) and sounds a bit like “base”. string is at most 90 characters long and consists of:

- The **human-readable part**, which is intended to convey the type of data, or anything else that is relevant to the reader. This part MUST contain 1 to 83 US-ASCII characters, with each character having a value in the range [33-126]. HRP validity may be further restricted by specific applications.
- The **separator**, which is always “1”. In case “1” is allowed inside the human-readable part, the last one in the string is the separator**Why include a separator in addresses?** That way the human-readable

part is unambiguously separated from the data part, avoiding potential collisions with other human-readable parts that share a prefix. It also allows us to avoid having character-set restrictions on the human-readable part. The separator is *1* because using a non-alphanumeric character would complicate copy-pasting of addresses (with no double-click selection in several applications). Therefore an alphanumeric character outside the normal character set was chosen..

- The **data part**, which is at least 6 characters long and only consists of alphanumeric characters excluding “1”, “b”, “i”, and “o”**Why not use an existing character set like RFC3548 or z-base-32**?

The character set is chosen to minimize ambiguity according to this visual similarity data, and the ordering is chosen to minimize the number of pairs of similar characters (according to the same data) that differ in more than 1 bit. As the checksum is chosen to maximize detection capabilities for low numbers of bit errors, this choice improves its performance under some error models..

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
+0	q	p	z	r	y	9	x	8
+8	g	f	2	t	v	d	w	0
+16	s	3	j	n	5	4	k	h
+24	c	e	6	m	u	a	7	l

Checksum

The last six characters of the data part form a checksum and contain no information. Valid strings MUST pass the criteria for validity specified by the Python3 code snippet below. The function `bech32_verify_checksum` must return true when its arguments are:

- `hrp`: the human-readable part as a string
- `data`: the data part as a list of integers representing the characters after conversion using the table above

```
def bech32_polymod(values):
  GEN = [0x3b6a57b2, 0x26508e6d, 0x1ea119fa, 0x3d4233dd, 0x2a1462b3]
  chk = 1
  for v in values:
    b = (chk >> 25)
    chk = (chk & 0x1ffffff) << 5 ^ v
    for i in range(5):
      chk ^= GEN[i] if ((b >> i) & 1) else 0
  return chk
```

```
def bech32_hrp_expand(s):
  return [ord(x) >> 5 for x in s] + [0] + [ord(x) & 31 for x in s]

def bech32_verify_checksum(hrp, data):
  return bech32_polymod(bech32_hrp_expand(hrp) + data) == 1
```

This implements a BCH code that guarantees detection of **any error affecting at most 4 characters** and has less than a 1 in 10^9 chance of failing to detect more errors. More details about the properties can be found in the Checksum Design appendix. The human-readable part is processed by first feeding the higher bits of each character's US-ASCII value into the checksum calculation followed by a zero and then the lower bits of each**Why are the high bits of the human-readable part processed first?** This results in the actually checksummed data being *[high hrp] 0 [low hrp] [data]*. This means that under the assumption that errors to the human readable part only change the low 5 bits (like changing an alphabetical character into another), errors are restricted to the *[low hrp] [data]* part, which is at most 89 characters, and thus all error detection properties (see appendix) remain applicable..

To construct a valid checksum given the human-readable part and (non-checksum) values of the data-part characters, the code below can be used:

```
def bech32_create_checksum(hrp, data):
  values = bech32_hrp_expand(hrp) + data
  polymod = bech32_polymod(values + [0,0,0,0,0,0]) ^ 1
  return [(polymod >> 5 * (5 - i)) & 31 for i in range(6)]
```

Error correction

One of the properties of these BCH codes is that they can be used for error correction. An unfortunate side effect of error correction is that it erodes error detection: correction changes invalid inputs into valid inputs, but if more than a few errors were made then the valid input may not be the correct input. Use of an incorrect but valid input can cause funds to be lost irrecoverably. Because of this, implementations SHOULD NOT implement correction beyond potentially suggesting to the user where in the string an error might be found, without suggesting the correction to make.

Uppercase/lowercase

The lowercase form is used when determining a character's value for checksum purposes.

Encoders MUST always output an all lowercase Bech32 string. If an uppercase version of the encoding result is desired, (e.g.- for presentation purposes, or QR code use), then an uppercasing procedure can be performed external to the encoding process.

Decoders MUST NOT accept strings where some characters are uppercase and some are lowercase (such strings are referred to as mixed case strings).

For presentation, lowercase is usually preferable, but inside QR codes uppercase SHOULD be used, as those permit the use of *alphanumeric mode*, which is 45% more compact than the normal *byte mode*.

Segwit address format

A segwit address**Why not make an address format that is generic for all scriptPubKeys?** That would lead to confusion about addresses for existing scriptPubKey types. Furthermore, if addresses that do not have a one-to-one mapping with scriptPubKeys (such as ECDH-based addresses) are ever introduced, having a fully generic old address type available would permit reinterpreting the resulting scriptPubKeys using the old address format, with lost funds as a result if bitcoins are sent to them. is a Bech32 encoding of:

- The human-readable part "bc"**Why use 'bc' as human-readable part and not 'btc'?** 'bc' is shorter. for mainnet, and "tb"**Why use 'tb' as human-readable part for testnet?** It was chosen to

be of the same length as the mainnet counterpart (to simplify implementations' assumptions about lengths), but still be visually distinct. for testnet.

- The data-part values:
    - 1 character (representing 5 bits of data): the witness version
    - A conversion of the 2-to-40-byte witness program (as defined by BIP141) to base32:
        - Start with the bits of the witness program, most significant bit per byte first.
        - Re-arrange those bits into groups of 5, and pad with zeroes at the end if needed.
        - Translate those bits to characters using the table above.

Decoding

Software interpreting a segwit address:

- MUST verify that the human-readable part is "bc" for mainnet and "tb" for testnet.
- MUST verify that the first decoded data value (the witness version) is between 0 and 16, inclusive.
- Convert the rest of the data to bytes:
    - Translate the values to 5 bits, most significant bit first.
    - Re-arrange those bits into groups of 8 bits. Any incomplete group at the end MUST be 4 bits or less, MUST be all zeroes, and is discarded.
    - There MUST be between 2 and 40 groups, which are interpreted as the bytes of the witness program.

Decoders SHOULD enforce known-length restrictions on witness programs. For example, BIP141 specifies *If the version byte is 0, but the witness program is neither 20 nor 32 bytes, the script must fail.*

As a result of the previous rules, addresses are always between 14 and 74 characters long, and their length modulo 8 cannot be 0, 3, or 5. Version 0 witness addresses are always 42 or 62 characters, but implementations MUST allow the use of any version.

Implementations should take special care when converting the address to a scriptPubkey, where witness version *n* is stored as *OP_n*. OP_0 is encoded as 0x00, but OP_1 through OP_16 are encoded as 0x51 though 0x60 (81 to 96 in decimal). If a bech32 address is converted to an incorrect scriptPubKey the result will likely be either unspendable or insecure.

Compatibility

Only new software will be able to use these addresses, and only for receivers with segwit-enabled new software. In all other cases, P2SH or P2PKH addresses can be used.

Rationale

Reference implementations

- Reference encoder and decoder:
  - For C
  - For C++
  - For JavaScript
  - For Go
  - For Python
  - For Haskell
  - For Ruby
  - For Rust
- Fancy decoder that localizes errors:
  - For JavaScript (demo website)

Registered Human-readable Prefixes

SatoshiLabs maintains a full list of registered human-readable parts for other cryptocurrencies:

SLIP-0173 : Registered human-readable parts for BIP-0173

Appendices

Test vectors

The following strings are valid Bech32:

- `A12UEL5L`
- `a12uel5l`
- `an83characterlonghumanreadablepartthatcontainsthenumber1andtheexcludedcharactersbio1tt5tgs`
- `abcdef1qpzry9x8gf2tvdw0s3jn54khce6mua7lmqqqxw`
- `11qqqq

- `1pzry9x0s0muk`: Empty HRP
- `x1b4n0q5v`: Invalid data character
- `li1dgmt3`: Too short checksum
- `de1lg7wt` + 0xFF: Invalid character in checksum
- `A1G7SGD8`: checksum calculated with uppercase form of HRP
- `10a06t8`: empty HRP
- `1qzzfhee`: empty HRP

The following list gives valid segwit addresses and the scriptPubKey that they translate to in hex.

- `BC1QW508D6QEJXTDG4Y5R3ZARVARY0C5XW7KV8F3T4`: `0014751e76e8199196d454941c45d1b3a323f1433bd6`
- `tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3q0sl5k7`: `00201863143c14c5166804bd19203356da136c985678cd4d27a1b8c6329604903262`
- `bc1pw508d6qejxtdg4y5r3zarvary0c5xw7kw508d6qejxtdg4y5r3zarvary0c5xw7k7grplx`: `5128751e76e8199196d454941c45d1b3a323f1433bd6751e76e8199196d454941c45d1b3a323f1433bd6`
- `BC1SW50QA3JX3S`: `6002751e`
- `bc1zw508d6qejxtdg4y5r3zarvaryvg6kdaj`: `5210751e76e8199196d454941c45d1b3a323`
- `tb1qqqqqp399et2xygdj5xreqhjjvcmzhxw4aywxecjdzew6hylgvsesrxh6hy`: `0020000000c4a5cad46221b2a187905e5266362b99d5e91c6ce24d165dab93e86433`

The following list gives invalid segwit addresses and the reason for their invalidity.

- `tc1qw508d6qejxtdg4y5r3zarvary0c5xw7kg3g4ty`: Invalid human-readable part
- `bc1qw508d6qejxtdg4y5r3zarvary0c5xw7kv8f3t5`: Invalid checksum
- `BC13W508D6QEJXTDG4Y5R3ZARVARY0C5XW7KN40WF2`: Invalid witness version
- `bc1rw5uspcuh`: Invalid program length
- `bc10w508d6qejxtdg4y5r3zarvary0c5xw7kw508d6qejxtdg4y5r3zarvary0c5xw7kw5rljs90`: Invalid program length
- `BC1QR508D6QEJXTDG4Y5R3ZARVARYV98GJ9P`: Invalid program length for witness version 0 (per BIP141)
- `tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3q0sL5k7`: Mixed case
- `bc1zw508d6qejxtdg4y5r3zarvaryvqyzf3du`: zero padding of more than 4 bits
- `tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3pjxtptv`: Non-zero padding in 8-to-5 conversion
- `bc1gmk9yu`: Empty data section

Checksum design

Design choices

BCH codes can be constructed over any prime-power alphabet and can be chosen to have a good trade-off between size and error-detection capabilities. While most work around BCH codes uses a binary alphabet, that is not a requirement. This makes them more appropriate for our use case than CRC codes. Unlike Reed-Solomon codes, they are not restricted in length to one less than the alphabet size. While they also support efficient error correction, the implementation of just error detection is very simple.

We pick 6 checksum characters as a trade-off between length of the addresses and the error-detection capabilities, as 6 characters is the lowest number sufficient for a random failure chance below 1 per billion. For the length of data we're interested in protecting (up to 71 bytes for a potential future 40-byte witness program), BCH codes can be constructed that guarantee detecting up to 4 errors.

Selected properties

Many of these codes perform badly when dealing with more errors than they are designed to detect, but not all. For that reason, we consider codes that are designed to detect only 3 errors as well as 4 errors, and analyse how well they perform in practice.

The specific code chosen here is the result of:

- Starting with an exhaustive list of 159605 BCH codes designed to detect 3 or 4 errors up to length 93, 151, 165, 341, 1023, and 1057.
- From those, requiring the detection of 4 errors up to length 71, resulting in 28825 remaining codes.
- From those, choosing the codes with the best worst-case window for 5-character errors, resulting in 310 remaining codes.
- From those, picking the code with the lowest chance for not detecting small numbers of *bit* errors.

As a naive search would require over $6.5 * 10^{19}$ checksum evaluations, a collision-search approach was used for analysis. The code can be found here.

Properties

The following table summarizes the chances for detection failure (as multiples of 1 in 10^9).

Window length		**Number of wrong characters**					
Length	Description	≤4	5	6	7	8	≥9
8	Longest detecting 6 errors	0			1.127	0.909	n/a
18	Longest detecting 5 errors	0		0.965	0.929	0.932	0.931
19	Worst case for 6 errors	0	0.093	0.972	0.928	0.931	
39	Length for a P2WPKH address	0	0.756	0.935	0.932	0.931	
59	Length for a P2WSH address	0	0.805	0.933	0.931		
71	Length for a 40-byte program address	0	0.830	0.934	0.931		
89	Longest detecting 4 errors	0	0.867	0.933	0.931		

This means that when 5 changed characters occur randomly distributed in the 39 characters of a P2WPKH address, there is a chance of *0.756 per billion* that it will go undetected. When those 5 changes occur randomly within a 19-character window, that chance goes down to *0.093 per billion*. As the number of errors goes up, the chance converges towards $1 \text{ in } 2^{30} = 0.931 \text{ per billion}$.

Even though the chosen code performs reasonably well up to 1023 characters, other designs are preferable for lengths above 89 characters (excluding the separator).

Acknowledgements

This document is inspired by the address proposal by Rusty Russell, the base32 proposal by Mark Friedenbach, and had input from Luke Dashjr, Johnson Lau, Eric Lombrozo, Peter Todd, and various other reviewers.

Disclosures (added 2024)

Due to an oversight in the design of bech32, this checksum scheme is not always robust against the insertion and deletion of fewer than 5 consecutive characters. Due to this weakness, BIP-350 proposes using the scheme described in this BIP only for Native Segwit v0 outputs.

```
  BIP: 350
  Layer: Applications
  Title: Bech32m format for v1+ witness addresses
  Authors: Pieter Wuille <pieter@wuille.net>
  Status: Deployed
  Type: Specification
  Assigned: 2020-12-16
  License: BSD-2-Clause
  Discussion: 2021-01-05: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-January/
018338.html [bitcoin-dev] Bech32m BIP: new checksum, and usage for segwit address
  Replaces: 173
```

BIP 350

Introduction

Abstract

This document defines an improved variant of Bech32 called **Bech32m**, and amends BIP173 to use Bech32m for native segregated witness outputs of version 1 and later. Bech32 remains in use for segregated witness outputs of version 0.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

BIP173 defined a generic checksummed base 32 encoded format called Bech32. It is in use for segregated witness outputs of version 0 (P2WPKH and P2WSH, see BIP141), and other applications.

Bech32 has an unexpected weakness: whenever the final character is a 'p', inserting or deleting any number of 'q' characters immediately preceding it does not invalidate the checksum. This does not affect existing uses of witness version 0 BIP173 addresses due to their restriction to two specific lengths, but may affect future uses and/or other applications using the Bech32 encoding.

This document addresses that by specifying Bech32m, a variant of Bech32 that mitigates this insertion weakness and related issues.

Specification

We first specify the new checksum algorithm, and then document how it should be used for future Bitcoin addresses.

Bech32m

Bech32m modifies the checksum of the Bech32 specification, replacing the constant *1* that is xored into the checksum at the end with *0x2bc830a3*. The resulting checksum verification and creation algorithm (in Python, cf. the code in Bech32 section):

```
BECH32M_CONST = 0x2bc830a3

def bech32m_polymod(values):
  GEN = [0x3b6a57b2, 0x26508e6d, 0x1ea119fa, 0x3d4233dd, 0x2a1462b3]
  chk = 1
  for v in values:
    b = (chk >> 25)
    chk = (chk & 0x1ffffff) << 5 ^ v
    for i in range(5):
      chk ^= GEN[i] if ((b >> i) & 1) else 0
  return chk

def bech32m_hrp_expand(s):
  return [ord(x) >> 5 for x in s] + [0] + [ord(x) & 31 for x in s]

def bech32m_verify_checksum(hrp, data):
  return bech32m_polymod(bech32m_hrp_expand(hrp) + data) == BECH32M_CONST

def bech32m_create_checksum(hrp, data):
  values = bech32m_hrp_expand(hrp) + data
  polymod = bech32m_polymod(values + [0,0,0,0,0,0]) ^ BECH32M_CONST
  return [(polymod >> 5 * (5 - i)) & 31 for i in range(6)]
```

All other aspects of Bech32 remain unchanged, including its human-readable parts (HRPs).

A combined function to decode both Bech32 and Bech32m simultaneously could be written using:

```
class Encoding(Enum):
    BECH32 = 1
    BECH32M = 2

def bech32_bech32m_verify_checksum(hrp, data):
    check = bech32_polymod(bech32_hrp_expand(hrp) + data)
    if check == 1:
        return Encoding.BECH32
    elif check == BECH32M_CONST:
        return Encoding.BECH32M
    else:
        return None
```

which returns either None for failure, or one of the BECH32 / BECH32M enumeration values to indicate successful decoding according to the respective standard.

Addresses for segregated witness outputs

Version 0 outputs (specifically, P2WPKH and P2WSH addresses) continue to use Bech32**Why not permit both Bech32 and Bech32m for v0 addresses?** Permitting both encodings reduces the error detection capabilities (it makes it equivalent to only have 29 bits of checksum). as specified in BIP173. Addresses for segregated witness

outputs version 1 through 16 use Bech32m. Again, all other aspects of the encoding remain the same, including the 'bc' HRP.

To generate an address for a segregated witness output:

- If its witness version is 0, encode it using Bech32.
- If its witness version is 1 or higher, encode it using Bech32m.

To decode an address, client software should either decode with both a Bech32 and a Bech32m decoder**Can a single string simultaneously be valid as Bech32 and Bech32m?** No, a valid Bech32 and Bech32m string will always differ by at least 3 characters if they are the same length., or use a decoder that supports both simultaneously. In both cases, the address decoder has to verify that the encoding matches what is expected for the decoded witness version (Bech32 for version 0, Bech32m for others).

The following code demonstrates the checks that need to be performed. Refer to the Python code linked in the reference implementation section below for full details of the called functions.

```
def decode(hrp, addr):
    hrpgot, data, spec = bech32_decode(addr)
    if hrpgot != hrp:
        return (None, None)
    decoded = convertbits(data[1:], 5, 8, False)
    # Witness programs are between 2 and 40 bytes in length.
    if decoded is None or len(decoded) < 2 or len(decoded) > 40:
        return (None, None)
    # Witness versions are in range 0..16.
    if data[0] > 16:
        return (None, None)
    # Witness v0 programs must be exactly length 20 or 32.
    if data[0] == 0 and len(decoded) != 20 and len(decoded) != 32:
        return (None, None)
    # Witness v0 uses Bech32; v1 through v16 use Bech32m.
    if data[0] == 0 and spec != Encoding.BECH32 or data[0] != 0 and spec != Encoding.BECH32M:
        return (None, None)
    # Success.
    return (data[0], decoded)
```

Error locating

Bech32m, like Bech32, does support locating**What about error correction?** As explained in BIP173, introducing error correction reduces the ability to detect errors. While it is technically possible to correct a small number of errors due to Bech32(m)'s nature as a BCH code, implementations should refrain from using this for more than indicating where an error may be present. the positions of a few substitution errors. To combine this functionality with the segregated witness addresses proposed by this document, simply try locating errors for both Bech32 and Bech32m. If only one finds error locations, report that one. If both do (which should be very rare), there are a number of options:

- Report the one that needs fewer corrections (if they differ).
- Eliminate the response(s) that are inconsistent. Any symbol that isn't on an error location can be checked. For example, if the witness version symbol is not an error location, and it doesn't correspond to the specification used (0 for Bech32, 1+ for Bech32m), that response can be eliminated.

See the fancy Javascript decoder below for example of the above.

Compatibility

This document introduces a new encoding for v1 segregated witness outputs and higher versions. There should not be any compatibility issues on the receiver side; no wallets are creating v1 segregated witness addresses yet, as the output type is not usable on mainnet.

On the other hand, the Bech32m proposal breaks forward-compatibility for sending to v1 and higher version segregated witness addresses. This incompatibility is intentional. An alternative design was considered where Bech32 remained in use for certain subsets of future addresses, but ultimately discarded. By introducing a clean break, we protect not only new software but also existing senders from the mutation issue, as new addresses will be incompatible with the existing Bech32 address validation. Experiments by Taproot proponents had shown that hardly any wallets and services supported sending to higher segregated witness output versions, so little is lost by breaking forward-compatibility. Furthermore, those experiments identified cases in which segregated witness implementations would have caused wallets to burn funds when sending to version 1 addresses. In case it is still in use, the chosen approach will prevent such software from destroying funds when attempting to send to a Bech32m address.

Reference implementations

- Reference encoder and decoder:
  - Reference Python implementation
  - Reference C implementation
  - Reference C++ implementation
  - Bitcoin Core C++ implementation
  - Reference Javascript implementation

- Fancy decoder that localizes errors:
  - For JavaScript (demo website)

Test vectors

Implementation advice Experiments testing BIP173 implementations found that many wallets and services did not support sending to higher version segregated witness outputs. In anticipation of the proposed Taproot soft fork introducing v1 segregated witness outputs on the network, we emphatically recommend employing the complete set of test vectors provided below as well as ensuring that your implementation supports sending to v1 **and higher versions**. All higher versions of native segregated witness outputs should be recognized as valid recipients. As higher versions are not defined on the network, no wallet should ever create them and no recipient should ever provide them to a sender. Nor should a recipient ever want to falsely provide them as the recipient would simply see a payment intended to themselves burned instead. However, by defining higher versions as valid recipients now, future soft forks introducing higher versions of native segwit outputs will be forward-compatible to all wallets correctly implementing the Bech32m specification.

Test vectors for Bech32m

The following strings are valid Bech32m:

- `A1LQFN3A`
- `a1lqfn3a`
- `an83characterlonghumanreadablepartthatcontainsthetheexcludedcharactersbioandnumber11sg7hg6`
- `abcdef1l7aum6echk45nj3s0wdvt2fg8x9yrzpqzd3ryx`
- `11llllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllllludsr8`
- `split1checkupstagehandshakeupstreamerranterredcaperredlc445v`
- `?1v759aa`

No string can be simultaneously valid Bech32 and Bech32m, so the above examples also serve as invalid test vectors for Bech32.

The following string are not valid Bech32m (with reason for invalidity):

- 0x20 + `1xj0phk`: HRP character out of range
- 0x7F + `1g6xzxy`: HRP character out of range
- 0x80 + `1vctc34`: HRP character out of range
- `an84characterslonghumanreadablepartthatcontainsthetheexcludedcharactersbioandnumber11d6pts4`: overall max length exceeded
- `qyrz8wqd2c9m`: No separator character
- `1qyrz8wqd2c9m`: Empty HRP
- `y1b0jsk6g`: Invalid data character
- `lt1igcx5c0`: Invalid data character
- `in1muywd`: Too short checksum
- `mm1crxm3i`: Invalid character in checksum
- `au1s5cgom`: Invalid character in checksum
- `M1VUXWEZ`: checksum calculated with uppercase form of HRP
- `16plkw9`: empty HRP
- `1p2gdwpf`: empty HRP

Test vectors for v0-v16 native segregated witness addresses

The following list gives valid segwit addresses and the scriptPubKey that they translate to in hex.

- `BC1QW508D6QEJXTDG4Y5R3ZARVARY0C5XW7KV8F3T4`: `0014751e76e8199196d454941c45d1b3a323f1433bd6`
- `tb1qrp33g0q5c5txsp9arysrx4k6zdkfs4nce4xj0gdcccefvpysxf3q0sl5k7`: `00201863143c14c5166804bd19203356da136c985678cd4d27a1b8c6329604903262`
- `bc1pw508d6qejxtdg4y5r3zarvary0c5xw7kw508d6qejxtdg4y5r3zarvary0c5xw7kt5nd6y`: `5128751e76e8199196d454941c45d1b3a323f1433bd6751e76e8199196d454941c45d1b3a323f1433bd6`
- `BC1SW50QGDZ25J`: `6002751e`
- `bc1zw508d6qejxtdg4y5r3zarvaryvaxxpcs`: `5210751e76e8199196d454941c45d1b3a323`
- `tb1qqqqqp399et2xygdj5xreqhjjvcmzhxw4aywxecjdzew6hylgvsesrxh6hy`: `0020000000c4a5cad46221b2a187905e5266362b99d5e91c6ce24d165dab93e86433`
- `tb1pqqqqp399et2xygdj5xreqhjjvcmzhxw4aywxecjdzew6hylgvsesf3hn0c`: `5120000000c4a5cad46221b2a187905e5266362b99d5e91c6ce24d165dab93e86433`

- `bc1p0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vqzk5jj0`: `512079be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798`

The following list gives invalid segwit addresses and the reason for their invalidity.

- `tc1p0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vq5zuyut`: Invalid human-readable part
- `bc1p0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vqh2y7hd`: Invalid checksum (Bech32 instead of Bech32m)
- `tb1z0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vqglt7rf`: Invalid checksum (Bech32 instead of Bech32m)
- `BC1S0XLXVLHEMJA6C4DQV22UAPCTQUPFHLXM9H8Z3K2E72Q4K9HCZ7VQ54WELL`: Invalid checksum (Bech32 instead of Bech32m)
- `bc1qw508d6qejxtdg4y5r3zarvary0c5xw7kemeawh`: Invalid checksum (Bech32m instead of Bech32)
- `tb1q0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vq24jc47`: Invalid checksum (Bech32m instead of Bech32)
- `bc1p38j9r5y49hruaue7wxjce0updqjuyyx0kh56v8s25huc6995vvpql3jow4`: Invalid character in checksum
- `BC130XLXVLHEMJA6C4DQV22UAPCTQUPFHLXM9H8Z3K2E72Q4K9HCZ7VQ7ZWS8R`: Invalid witness version
- `bc1pw5dgrnzv`: Invalid program length (1 byte)
- `bc1p0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7v8n0nx0muaewav253zgeav`: Invalid program length (41 bytes)
- `BC1QR508D6QEJXTDG4Y5R3ZARVARYV98GJ9P`: Invalid program length for witness version 0 (per BIP141)
- `tb1p0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vq47Zagq`: Mixed case
- `bc1p0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7v07qwwzcrf`: zero padding of more than 4 bits
- `tb1p0xlxvlhemja6c4dqv22uapctqupfhlxm9h8z3k2e72q4k9hcz7vpggkg4j`: Non-zero padding in 8-to-5 conversion
- `bc1gmk9yu`: Empty data section

Appendix: checksum design & properties

Checksums are used to detect errors introduced into data during transfer. A hash function-based checksum such as Base58Check detects any type of error uniformly, but not all classes of errors are equally likely to occur in practice. Bech32 prioritizes detection of substitution errors, but improving detection of one error class inevitably worsens detection of other error classes. During the design of Bech32, it was assumed that other simple error patterns beside substitutions would have a similar detection rate as in a hash function-based design, and detection would only be worse for complex, impractical errors. The discovered insertion weakness shows that this is not the case.

For Bech32m, we aim to retain Bech32's guarantees for substitution errors, but make sure that other common errors don't perform worse than a hash function-based checksum would. To make sure the new standard is easy to implement, we restrict the design space to only amending the final constant that is xored in, as it was observed that is sufficient to mitigate the 'q' insertion issue while retaining the intended substitution error detection. In what follows, we explain how the new constant *0x2bc830a3* was chosen.

Error patterns & detection probability

We define an error pattern as a sequence of first one or more deletions, then swaps of adjacent characters, followed by substitutions, insertions, and duplications, in that order, all in specific positions, applied to a string

with valid checksum that is otherwise randomly chosen. For insertions and substitutions we assume a uniformly random new character. For example, “delete the 17th character, swap the 11th character with the 12th character, and insert a random character in the 24th position” is an error pattern. “Replace the 43rd through 48th character with ‘aardvark’” is not a valid error pattern, because the new characters are not random and there is no reason why this particular string is more likely than any other to be substituted.

A hash function-based checksum design with a 30-bit hash would have a probability of incorrectly accepting equal to 2^{-30}, for every error pattern. Bech32 has a probability of 0 to incorrectly accept error patterns consisting of up to 4 substitutions—they are always detected. The ‘q’-insertion issue shows that for Bech32 a simple error pattern (“insert a random character in the penultimate position”) with probability 2^{-10} exists: it requires the final character to be ‘p’ (leaving only 1 in 32 strings), and requires the inserted character to be ‘q’ (permitting only 1 of 32 possible inserted characters).

Note that the choice of *what* the error pattern is (which types of errors, and where) isn’t part of our probabilities: we try to make sure that *every* pattern behaves well, not just randomly chosen ones, because presumably humans make some kinds of errors more than others, and we cannot easily model which ones.

Detection properties of Bech32m

The table below shows the error detection properties of Bech32m, and a comparison with Bech32. The code used for this analysis can be found here. Every row specifies one error pattern via the constraints in the left four columns. The remaining columns report what percentage of those patterns have certain probabilities of not being detected. The columns are:

- **errors** The maximum number of individual errors considered
- **of type** What type of errors are considered (either “subst. only” for just substitutions, or “any” to also include deletions, swaps, insertions, and duplications)
- **window** The maximum size of the window in which the errors have to occur**What is an error pattern’s window size?** The window size of an error pattern is the length of the smallest consecutive range of characters that contains all modified characters (on input or output; whichever is larger). For example, an error pattern that turns “abcdef” into “accdbef” has a window size of 4, as it is replacing “bcd” with “ccdb”, a 4 character string. Window size is only meaningful when the pattern consists of two or more errors.
- **code/verifier** Whether this line is about Bech32 or Bech32m encoded strings, and whether those are evaluated regarding their probability of being accepted by either a Bech32 or a Bech32m verifier.**Why do we care about probability of accepting Bech32m strings in Bech32 verifiers?** For applications where Bech32m replaces an existing use of Bech32 (such as segregated witness addresses), we want to make sure that a Bech32m string created by new software won’t be erroneously accepted by old software that assumes Bech32 - even when a small number of errors were introduced as well.**Should we also take into account failures that occur due to taking a valid Bech32m string, and after errors it becoming acceptable to a Bech32 verifier?** This situation may in theory occur for segregated witness addresses when errors occur that change the version number in a v1+ address to v0. Due to the specificity of this type of error, plus the additional constraints that apply for v0 addresses, this is both unlikely and hard to analyze.
- **error patterns with failure probability** For each probability (0, 2^{-30}, 2^{-25}, 2^{-20}, 2^{-15}, and 2^{-10}) this reports what percentage of error patterns restricted by the constraints in the previous columns have those probabilities of being incorrectly accepted.

The properties are divided into two classes: those that hold over all strings when averaged over all possible HRPs (human readable parts), and those specific to the "bc1" HRP with the length restrictions imposed by segregated witness addresses**What restrictions were taken into account for the "bc1"-specific analysis?** The minimum length (due to witness programs being at least 2 bytes), the maximum length (due to witness programs being at most 40 bytes), and the fact that the witness programs are a multiple of 8 bits. The fact that the first data symbol cannot be over 16, or that the padding has to be 0, is not taken into account..

errors	**of type**	**window**	**code/verifier**	**error patterns with failure probability**					
0	2^{-30}	2^{-25}	2^{-20}	2^{-15}	2^{-10}				
Properties averaged over all HRPs									
≤ 4	only subst.	any	Bech32m/ Bech32m	100.00%	none[(a)]				
any	any	≤ 4		56.16%	43.84%	none[(b)]			
≤ 2	any	≤ 68		7.71%	92.28%	none[(b)]			
≤ 2	any	any		7.79%	92.20%	0.004%	none[(b)]		
≤ 3	any	≤ 69		7.73%	92.23%	0.033%[(d)]	none[(b)]		
≤ 3	any	any		7.77%	92.19%	0.034%	0.000065%	none[(b)]	
≤ 4	only subst.	any	Bech32/Bech32	100.00%	none				
any	any	≤ 4		54.00%	43.84%	1.08%	0.90%	0.17%	0.0091%
≤ 2	any	≤ 68		4.59%	92.29%	1.09%	1.01%	0.99%	0.039%
≤ 2	any	any		4.58%	92.21%	1.11%	1.04%	1.02%	0.038%
≤ 3	any	≤ 69		6.69%	92.23%	0.56%	0.48%	0.041%	0.00055%
≤ 3	any	any		6.66%	92.19%	0.59%	0.52%	0.041%	0.00053%
0	-	-	Bech32m/ Bech32	100.00%	none[(a)]				
1	any	-		46.53%	53.46%	none[(b)]			
≤ 2	any	any		22.18%	77.77%	0.048%	none[(b)]		
Properties for segregated witness addresses with HRP "bc"									
≤ 4	only subst.	any	Bech32m/ Bech32m	100.00%	none[(a)]				
1	any	-		24.34%	75.66%	none[(c)]			
≤ 2	any	≤ 28		16.85%	83.15%	none[(c)]			
any	any	≤ 4		74.74%	25.25%	0.0016%	none[(c)]		
≤ 2	any	any		15.72%	84.23%	0.039%	0.0053%	none[(c)]	
≤ 3	any	any		13.98%	85.94%	0.078%	0.00063%	none[(c)]	
≤ 4	only subst.	any	Bech32/Bech32	100.00%	none				

errors	of type	window	code/verifier	error patterns with failure probability					
1	any	-		14.63%	75.71%	2.43%	2.43%	2.43%	2.38%
≤ 2	any	≤ 28		14.22%	83.15%	0.94%	0.84%	0.79%	0.054%
any	any	≤ 4		73.23%	25.26%	0.76%	0.63%	0.12%	0.0064%
≤ 2	any	any		12.79%	84.24%	1.06%	0.95%	0.92%	0.041%
≤ 3	any	any		13.00%	85.94%	0.57%	0.45%	0.044%	0.00067%
≤ 3	only subst.	any	Bech32m / Bech32	100.00%	none[c]				
1	any	-		70.89%	29.11%	none[c]			
≤ 2	any	any		36.12%	63.79%	0.092%	0.00049%	none[c]	

The numbers in this table, as well as a comparison with the numbers for the ''1'' constant and earlier proposed improved constants, can be found here.

Selection process

The details of the selection process can be found here, but in short:

- Start with the set of all $2^{30}-1$ constants different from Bech32's *1*. All of these satisfy the properties marked [a] in the table above.
- Through exhaustive analysis, reject all constants that do not exhibit the properties**How were the properties to select for chosen?** All these properties are as strong as they can be without rejecting every constant: rejecting constants with lower probabilities, or more errors, or wider windows all result in nothing left. marked [b] in the table above (e.g. all constants that permit any error pattern of 2 errors or less in a window of 68 characters or less with a detection probability $\geq 2^{-20}$). This selection leaves us with 12054 candidates.
- Reject all constants that do not exhibit the [c] properties in the table above**Why optimize for segregated witness addresses (with HRP "bc1") specifically?** Our analysis for generic HRP has limitations (see the detailed description here, under "Technical details"). We optimize for generic usage first, but optimize for segregated witness addresses as a tiebreaker.. This leaves us with 79 candidates.
- Finally, select the candidate that minimizes the number of error classes matching [d] in the table above as a final tiebreaker. The result is the single constant *0x2bc830a3*.

Footnotes

Acknowledgements

Thanks to Greg Maxwell for doing most of the computation for code selection and analysis, and comments. Thanks to Mark Erhardt for help with writing and editing this document. Thanks to Rusty Russell and others on the bitcoin-dev list for the discussion around intentionally breaking compatibility with existing senders, which is used in this specification.

```
BIP: 321
Layer: Applications
```

```
Title: URI Scheme
Authors: Matt Corallo <bip21@bluematt.me>
Status: Complete
Type: Specification
Assigned: 2024-11-15
License: BSD-2-Clause
Replaces: 21
```

BIP 321

Copyright

This BIP is licensed under the BSD 2-clause license.

Abstract

This BIP proposes a URI scheme for describing Bitcoin payment instructions.

Motivation

The purpose of this URI scheme is to enable users to easily make payments by simply clicking links on webpages or scanning QR Codes.

This BIP is a modification and intended replacement of BIP 0021 to add information about the modern usage of bitcoin: URIs (including standard query parameters and modern address types) as well as provide forward-looking guidance on how to incorporate new payment instructions. It further adds an optional extension to provide the payment initiator with proof of payment. BIP 21 was based on BIP 20, which was, in turn based on an earlier document by Nils Schneider.

Specification

General rules for handling (important!)

Bitcoin clients MUST NOT act on URIs without getting the user's authorization. They SHOULD require the user to manually approve each payment individually, though in some cases they MAY allow the user to automatically make this decision.

Operating system integration

Graphical bitcoin clients SHOULD register themselves as the handler for the "bitcoin:" URI scheme by default, if no other handler is already registered. If there is already a registered handler, they MAY prompt the user to change it once when they first run the client.

General Format

Bitcoin URIs follow the general format for URIs as set forth in RFC 3986. The path component consists of a bitcoin address, and the query component provides additional payment options.

Elements of the query component may contain characters outside the valid range. These must first be encoded according to UTF-8, and then each octet of the corresponding UTF-8 sequence must be percent-encoded as described in RFC 3986.

ABNF grammar

```
bitcoinurn     = "bitcoin:" [ bitcoinaddress ] [ "?" bitcoinparams ]
bitcoinaddress = *base58 / *bech32 / *bech32m
bitcoinparams  = bitcoinparam [ "&" bitcoinparams ]
bitcoinparam   = [ amountparam / labelparam / messageparam / responseparam / otherparam / reqparam ]
amountparam    = "amount=" *digit [ "." *digit ]
labelparam     = "label=" *qchar
messageparam   = "message=" *qchar
responseparam  = [ "req-" ] "pop=" *qchar
otherparam     = qchar *qchar [ "=" *qchar ]
reqparam       = "req-" qchar *qchar [ "=" *qchar ]
```

Here, “qchar” corresponds to valid characters of an RFC 3986 URI query component, excluding the “=” and “&” characters, which this BIP takes as separators.

The scheme component (“bitcoin:”) is case-insensitive, and implementations must accept any combination of uppercase and lowercase letters. The query parameter keys are also case-insensitive. Query parameter values and bitcoin address fields may be case-sensitive depending on their content.

Multiple query parameters with the same key MAY be included for query parameters representing payment instructions. Multiple query parameters with the same key MUST NOT be included for keys “label”, “message”, or “pop”. Multiple query parameters with the same key for other keys MUST be allowed for unknown query parameters. Future query parameter keys may or may not allow for duplicate parameters.

Bitcoin Address

The bitcoinaddress body MUST be either a base58 P2SH or P2PKH address, bech32 Segwit version 0 address, bech32m Segwit address, or empty. Future address formats SHOULD instead be placed in query keys as optional payment instructions to provide backwards compatibility during upgrade cycles. The bitcoinaddress part of the URI MAY be left empty if there is at least one payment instruction provided in a query parameter, allowing for recipients wishing to avoid a standard on-chain fallback.

Query Keys

The following keys are defined generally and apply to any URI regardless of payment instructions:

- label: Label for the recipient (e.g. name of receiver)
- message: message that describes the transaction to the user (see examples below)
- pop: a URI which the Bitcoin Wallet may return to in order to provide the application which initiated the payment with proof that a payment was completed.

The following keys are currently defined for payment instructions of various forms:

- lightning: Lightning BOLT 11 invoices

- lno: Lightning BOLT12 offers
- pay: BIP 351 Private Payment addresses
- sp: BIP 352 Silent Payment addresses

New payment instructions using bech32 or bech32m encodings SHOULD reuse their address format's Human Readable Part as the parameter key.

Transfer amount

If an amount is provided, it MUST be specified in decimal BTC. All amounts MUST contain no commas and use a period (.) as the separating character to separate whole numbers and decimal fractions. I.e. amount=50.00 or amount=50 is treated as 50 BTC, and amount=50,000.00 is invalid.

Proof of Payment

The URI MAY include a "pop" (or "req-pop") parameter whose value can be used to build a URI which the wallet application can, after payment completes, "open" to provide proof the payment was completed or other information about the payment.

The value of a "pop" (or "req-pop") parameter shall be a percent-encoded (per RFC 3986 section 2.1) URI prefix. The wallet application, if it supports providing payment information SHOULD percent-decode the provided URI once, append the query parameter key from which the payment instructions used were read, append a single =, and finally append the Payment Information to the resulting URI and open it with the default system handler for the URI. For payment instructions read from the body of the URI, "onchain" SHALL be used in place of the key.

A wallet MUST validate that the provided URI's scheme is not (case-insensitive) "http", "https", "file", "javascript", "mailto" or any other scheme which will open in a web browser prior to opening it.

If a wallet will not open the pop scheme (either because it does not support returning payment information for the selected payment method or because it uses a URI scheme which should not be opened) and the parameter was passed as a "req-pop" parameter, the wallet MUST NOT initiate payment.

For payments made using an on-chain transaction, the Payment Information shall be the full (including witness data) Bitcoin transaction as it was broadcasted to the Bitcoin network, encoded in hex.

For payments made using a BOLT 11 invoice (communicated via the `lightning` parameter), the Payment Information shall be the hex-encoded payment preimage.

Other payment schemes will define their own Payment Information format. This BIP may be updated from time to time with Payment Information formats for other payment schemes.

Rationale

Payment identifiers, not person identifiers

Best practices are that a unique address should be used for every transaction on-chain. Therefore, a URI which contains an on-chain payment address MUST NOT represent an exchange of personal information, but a one-

time payment instruction. URIs which represent only reusable non-address-reusing payment instructions (like Lightning BOLT12 offers or Silent Payments) MAY be reused as a wallet sees fit.

Proof of Payment

On many mobile operating systems (especially, or any operating system more generally), applications may "open" a bitcoin: URI in order to initiate a payment with the user's default wallet application. These payment-initiating applications may wish to learn about the completed payment.

For payments completed on-chain, this is largely addressed by having the payment-initiating application monitor the blockchain for payment completion, however for other payment schemes (e.g. lightning), no such global ledger of transactions exists. In that case, proof of payment must be provided via some other mechanism.

In order to avoid inadvertently revealing the sender's IP address or other information to the recipient, proof URIs must only be opened when they will simply switch to another locally-installed application (i.e. the application which initiated the payment). When clicking a URI from a website, the website should already have plenty of logic on its backend to process payment completion and a proof-of-payment callback is unnecessary.

Reference Implementation

Documentation: https://docs.rs/bitcoin-payment-instructions

Code repository: https://github.com/rust-bitcoin/bitcoin-payment-instructions

Forward compatibility

Query parameter keys which are prefixed with a req- are considered required. If a client does not implement handling a query parameter which has a key prefixed with req-, it MUST consider the entire URI invalid. Any other query parameters which are not implemented, but which are not prefixed with a req-, can be safely ignored.

As future new address types should be added using query parameters rather than the `bitcoinaddress` field, URIs seamlessly support various payment instructions while senders only need to support legacy instructions. This allows old senders to pay newer recipients which offer more modern payment instruction formats.

Backward compatibility

Compared to BIP 21, this document describes standard query parameters containing payment instructions, makes query parameters case-insensitive, allows bech32 and bech32m addresses in the `bitcoinaddress` field, and allows for URIs with an empty `bitcoinaddress` field. Use of bech32 and bech32m `bitcoinaddress` fields were long-since common practice in 2024, and the `lightning` query parameter storing BOLT 11 payment instructions became common practice in the year or three leading up to 2024. Inclusion of standard query parameters was added to provide guidance on query parameter usage going forward.

Additionally, this BIP describes the "pop" query parameter, which was unused and will be ignored by BIP 21 implementations.

Any existing BIP 21 implementation should automatically be fully compliant with this BIP, as the changes only describe existing practice or impact future address format inclusion, with the one possible exception of query parameters being made case-insensitive. Note, however, that treating query parameters as case-insensitive is already common practice due to the use of mostly-uppercase URIs in QR codes.

Appendix

Examples

Note: The addresses used in these examples are intentionally invalid to prevent accidental transactions.

URIs

Just the address:

`bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W`

Address with recipient's name as label:

`bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?label=Luke-Jr`

Request 20.30 BTC to "Luke-Jr":

`bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?amount=20.3&label=Luke-Jr`

Request 50 BTC with message:

`bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?amount=50&label=Luke-Jr&message=Donation%20for%20project%20xyz`

Request funds to be paid over lightning to a BOLT 11 invoice with a fallback to on-chain payments:

`bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?lightning=lnbc420bogusinvoice`

Request funds to be paid over lightning to a BOLT 11 invoice with no fallback:

`bitcoin:?lightning=lnbc420bogusinvoice`

Request funds to be paid over lightning to a BOLT 12 offer with no fallback:

`bitcoin:?lno=lno1bogusoffer`

Request funds to be paid over lightning to a BOLT 12 offer or silent payments address with no fallback:

`bitcoin:?lno=lno1bogusoffer&sp=sp1qsilentpayment`

Request funds to be paid to a silent payments address with no fallback:

`bitcoin:?sp=sp1qsilentpayment`

Request funds to be paid to a silent payments address with a fallback:

`bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?sp=sp1qsilentpayment`

Some future version that has variables which are (currently) not understood and required and thus invalid:

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?req-somethingyoudontunderstand=50&req-somethingelseyoudontget=999

Some future version that has variables which are (currently) not understood but not required and thus valid:

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?somethingyoudontunderstand=50&somethingelseyoudontget=999

Multiple segwit addresses may be included for various versions of segwit, note that the human-readable part for all of them is `bc`

bitcoin:?bc=bc1qufgy354j3kmvuch987xe4s40836x3h0lg8f5nq&bc=bc1p5swkugezn97763tl0yty6556856uug0q6jflljvep9m4p7339x5qzyr

Many QR codes utilize all-uppercase URIs, which should be handled fine

BITCOIN:BC1QUFGY354J3KMVUCH987XE4S40836X3H0LG8F5NQ?BC=BC1P5SWKUGEZN97763TL0YTY6556856UUG0Q6JFLLJVEP9M4P7339X5QZYRH4Q
BITCOIN:?BC=BC1QUFGY354J3KMVUCH987XE4S40836X3H0LG8F5NQ&BC=BC1P5SWKUGEZN97763TL0YTY6556856UUG0Q6JFLLJVEP9M4P7339X5QZYR

A testnet segwit addresses must be included in the `tb` parameter

bitcoin:?tb=tb1qghfhmd4zh7ncpmxl3qzhmq566jk8ckq4gafnmq

Characters must be URI encoded properly.

Invalid URIs

Labels must not appear twice:

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?label=Luke-Jr&label=Matt

Amounts must not appear twice:

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?amount=42&amount=10

Amounts must not appear twice even if they are the same:

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?amount=42&amount=42

Multiple proof of payment URIs must not appear, even if they are sometimes prefixed with req-:

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?pop=callback%3a&req-pop=callback%3a

A testnet segwit addresses must be included in the `tb` parameter, not the `bc` parameter.

bitcoin:?bc=tb1qghfhmd4zh7ncpmxl3qzhmq566jk8ckq4gafnmq

Proof of Payment

If the original URI is

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?pop=initiatingapp%3a

the wallet should perform the payment information callback by opening

`initiatingapp:onchain=$HEX_ENCODED_TRANSACTION`

If the original URI is

bitcoin:?lightning=lnbc420bogusinvoice&pop=callbackuri%3abody%3fpop=

the wallet should perform the proof-of-payment callback by opening

`callbackuri:body?pop=lightning=$HEX_ENCODED_PAYMENT_PREIMAGE`

If the original URI is

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?lightning=lnbc420bogusinvoice&pop=app%3a%3f

and the wallet pays on-chain, it should perform the payment information callback by opening

`app:?onchain=$HEX_ENCODED_TRANSACTION`

but if the app pays using lightning, it should perform the proof-of-payment callback by opening

`app:?lightning=$HEX_ENCODED_PAYMENT_PREIMAGE`

If the original URI is

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?pop=https%3aiwantyouripaddress.com

the wallet should make a payment as it normally would but MUST NOT interact with iwantyouripaddress.com

If the original URI is

bitcoin:175tWpb8K1S7NmH4Zx6rewF9WQrcZv245W?req-pop=https%3aevilwebsite.com

the wallet MUST NOT make a payment

```
BIP: 179
Title: Name for payment recipient identifiers
Authors: Emil Engler <me@emilengler.com>
         Luke Dashjr <luke+bip@dashjr.org>
Status: Complete
Type: Informational
Assigned: 2019-10-17
License: CC0-1.0
```

BIP 179

Abstract

This BIP proposes a new term for ‘address’

Specification

The new term is: *Bitcoin* **Invoice** *Address*

The *Bitcoin* and *Address* parts are optional. The address suffix should only be used as a transitional step.

A *Bitcoin* Invoice *Address* is a string of characters that can be used to indicate the intended recipient and purpose of a transaction.

Motivation

Bitcoin addresses are intended to be only used **once** and you should generate a new one for every new incoming payment. The term ‘address’ however indicates consistency because nearly everything on the internet or the offline world with the term ‘address’ is something that rarely or even never changes (postal address, email address, IP addresses (depends heavily on the provider), etc.) The motivation for this BIP is to change the term address to something that indicates that the address is connected to a single transaction.

Rationale

The reason why we use *Bitcoin Invoice Address* or just *Invoice* is to emphasize that it is single-use. The terms *Bitcoin* and *Address* are optional for the following reasons: For *Bitcoin*:

- Useful for multicoin wallets to indicate that it belongs to Bitcoin
- Indicates a difference between a lightning and an on-chain invoice

For *Address*:

- To not confuse users with a completely new term
- To show that it is where you send something to
- To not break backwards compatibility

This gives us the four following possibilities:

- Bitcoin Invoice Address
- Bitcoin Invoice
- Invoice Address
- Invoice

Backwards Compatibility

To avoid issues, the ‘Address’ suffix is permitted, but not recommended. The suffix ‘Address’ remains so users should be immediately able to recognize it until the new term is widely known.

Acknowledgements

Thanks to Chris Belcher for the suggestion of the term ‘Bitcoin Invoice Address’

Copyright

This BIP is released under CC0-1.0 and therefore Public Domain.

Consensus Changes That Shipped

These are the rule changes that made it onto the chain. Not every proposal in bitcoin/bips did. The closed hard-fork block-size documents, the withdrawn script ideas, and the drafts still in flight are elsewhere. This chapter is the history of what nodes actually enforce.

It starts with pay-to-script-hash, then the early hygiene forks: no duplicate transactions, height in the coinbase, and a finite monetary supply. Then come the timelock opcodes and the signature-encoding rules that made contracts safer to write. SegWit is the long middle of the chapter — the consensus rules, the signature scheme, the dummy-stack fix, and the activation fight that BIP 148 and BIP 91 record.

Read these in order if you can. Each fork assumes the ones before it. By the time you reach SegWit you will have seen why malleability, script versioning, and honest lock-time were worth a soft fork.

In this chapter:

- BIP 16 — Pay to Script Hash
- BIP 30 — Duplicate transactions
- BIP 34 — Block v2, Height in Coinbase
- BIP 42 — A finite monetary supply for Bitcoin
- BIP 65 — OP_CHECKLOCKTIMEVERIFY
- BIP 66 — Strict DER signatures
- BIP 68 — Relative lock-time
- BIP 112 — CHECKSEQUENCEVERIFY
- BIP 113 — Median time-past as lock-time
- BIP 141 — Segregated Witness
- BIP 143 — SegWit signature verification
- BIP 147 — Dummy stack element malleability
- BIP 148 — Mandatory activation of segwit
- BIP 91 — Reduced threshold Segwit MASF

```
BIP: 16
Layer: Consensus (soft fork)
Title: Pay to Script Hash
Authors: Gavin Andresen <gavinandresen@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2012-01-03
```

BIP 16

Abstract

This BIP describes a new “standard” transaction type for the Bitcoin scripting system, and defines additional validation rules that apply only to the new transactions.

Motivation

The purpose of pay-to-script-hash is to move the responsibility for supplying the conditions to redeem a transaction from the sender of the funds to the redeemer.

The benefit is allowing a sender to fund any arbitrary transaction, no matter how complicated, using a fixed-length 20-byte hash that is short enough to scan from a QR code or easily copied and pasted.

Specification

A new standard transaction type that is relayed and included in mined blocks is defined:

```
OP_HASH160 [20-byte-hash-value] OP_EQUAL
```

[20-byte-hash-value] shall be the push-20-bytes-onto-the-stack opcode (0x14) followed by exactly 20 bytes.

This new transaction type is redeemed by a standard scriptSig:

```
...signatures... {serialized script}
```

Transactions that redeem these pay-to-script outpoints are only considered standard if the *serialized script* - also referred to as the *redeemScript* - is, itself, one of the other standard transaction types.

The rules for validating these outpoints when relaying transactions or considering them for inclusion in a new block are as follows:

1. Validation fails if there are any operations other than “push data” operations in the scriptSig.
2. Normal validation is done: an initial stack is created from the signatures and {serialized script}, and the hash of the script is computed and validation fails immediately if it does not match the hash in the outpoint.
3. {serialized script} is popped off the initial stack, and the transaction is validated again using the popped stack and the deserialized script as the scriptPubKey.

These new rules should only be applied when validating transactions in blocks with timestamps >= 1333238400 (Apr 1 2012) Remove -bip16 and -paytoscripthashtime command-line arguments. There are transactions earlier than 1333238400 in the block chain that fail these new validation rules. Transaction 6a26d2ecb67f27d1fa5524763b49029d7106e91e3cc05743073461a719776192. Older transactions must be validated under the old rules. (see the Backwards Compatibility section for details).

For example, the scriptPubKey and corresponding scriptSig for a one-signature-required transaction is:

```
scriptSig: [signature] {[pubkey] OP_CHECKSIG}
scriptPubKey: OP_HASH160 [20-byte-hash of {[pubkey] OP_CHECKSIG} ] OP_EQUAL
```

Signature operations in the {serialized script} shall contribute to the maximum number allowed per block (20,000) as follows:

1. OP_CHECKSIG and OP_CHECKSIGVERIFY count as 1 signature operation, whether or not they are evaluated.
2. OP_CHECKMULTISIG and OP_CHECKMULTISIGVERIFY immediately preceded by OP_1 through OP_16 are counted as 1 to 16 signature operation, whether or not they are evaluated.
3. All other OP_CHECKMULTISIG and OP_CHECKMULTISIGVERIFY are counted as 20 signature operations.

Examples:

+3 signature operations:

```
{2 [pubkey1] [pubkey2] [pubkey3] 3 OP_CHECKMULTISIG}
```

+22 signature operations:

```
{OP_CHECKSIG OP_IF OP_CHECKSIGVERIFY OP_ELSE OP_CHECKMULTISIGVERIFY OP_ENDIF}
```

Rationale

This BIP replaces BIP 12, which proposed a new Script opcode ("OP_EVAL") to accomplish everything in this BIP and more.

The Motivation for this BIP (and BIP 13, the pay-to-script-hash address type) is somewhat controversial; several people feel that it is unnecessary, and complex/multisignature transaction types should be supported by simply giving the sender the complete {serialized script}. The author believes that this BIP will minimize the changes needed to all of the supporting infrastructure that has already been created to send funds to a base58-encoded-20-byte bitcoin addresses, allowing merchants and exchanges and other software to start supporting multisignature transactions sooner.

Recognizing one 'special' form of scriptPubKey and performing extra validation when it is detected is ugly. However, the consensus is that the alternatives are either uglier, are more complex to implement, and/or expand the power of the expression language in dangerous ways.

The signature operation counting rules are intended to be easy and quick to implement by statically scanning the {serialized script}. Bitcoin imposes a maximum-number-of-signature-operations per block to prevent denial-of-service attacks on miners. If there was no limit, a rogue miner might broadcast a block that required hundreds of thousands of ECDSA signature operations to validate, and it might be able to get a head start computing the next block while the rest of the network worked to validate the current one.

There is a 1-confirmation attack on old implementations, but it is expensive and difficult in practice. The attack is:

1. Attacker creates a pay-to-script-hash transaction that is valid as seen by old software, but invalid for new implementation, and sends themselves some coins using it.
2. Attacker also creates a standard transaction that spends the pay-to-script-hash transaction, and pays the victim who is running old software.
3. Attacker mines a block that contains both transactions.

If the victim accepts the 1-confirmation payment, then the attacker wins because both transactions will be invalidated when the rest of the network overwrites the attacker's invalid block.

The attack is expensive because it requires the attacker create a block that they know will be invalidated by the rest of the network. It is difficult because creating blocks is difficult and users should not accept 1-confirmation transactions for higher-value transactions.

Backwards Compatibility

These transactions are non-standard to old implementations, which will (typically) not relay them or include them in blocks.

Old implementations will validate that the {serialize script}'s hash value matches when they validate blocks created by software that fully support this BIP, but will do no other validation.

Avoiding a block-chain split by malicious pay-to-script transactions requires careful handling of one case:

- A pay-to-script-hash transaction that is invalid for new clients/miners but valid for old clients/miners.

To gracefully upgrade and ensure no long-lasting block-chain split occurs, more than 50% of miners must support full validation of the new transaction type and must switch from the old validation rules to the new rules at the same time.

To judge whether or not more than 50% of hashing power supports this BIP, miners are asked to upgrade their software and put the string "/P2SH/" in the input of the coinbase transaction for blocks that they create.

On February 1, 2012, the block-chain will be examined to determine the number of blocks supporting pay-to-script-hash for the previous 7 days. If 550 or more contain "/P2SH/" in their coinbase, then all blocks with timestamps after 15 Feb 2012, 00:00:00 GMT shall have their pay-to-script-hash transactions fully validated. Approximately 1,000 blocks are created in a week; 550 should, therefore, be approximately 55% of the network supporting the new feature.

If a majority of hashing power does not support the new validation rules, then rollout will be postponed (or rejected if it becomes clear that a majority will never be achieved).

520-byte limitation on serialized script size

As a consequence of the requirement for backwards compatibility the serialized script is itself subject to the same rules as any other PUSHDATA operation, including the rule that no data greater than 520 bytes may be pushed to the stack. Thus it is not possible to spend a P2SH output if the redemption script it refers to is >520 bytes in length. For instance while the OP_CHECKMULTISIG opcode can itself accept up to 20 pubkeys, with 33-byte compressed pubkeys it is only possible to spend a P2SH output requiring a maximum of 15 pubkeys to redeem: 3 bytes + 15 pubkeys * 34 bytes/pubkey = 513 bytes.

Reference Implementation

https://gist.github.com/gavinandresen/3966071

See Also

- https://bitcointalk.org/index.php?topic=46538
- The Address format for Pay to Script Hash BIP
- M-of-N Multisignature Transactions BIP 11
- Quality Assurance test checklist

References

```
BIP: 30
Layer: Consensus (soft fork)
Title: Duplicate transactions
Authors: Pieter Wuille <pieter.wuille@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2012-02-22
License: BSD-2-Clause
```

BIP 30

Abstract

This document gives a specification for dealing with duplicate transactions in the block chain, in an attempt to solve certain problems the reference implementation has with them.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

So far, the Bitcoin reference implementation always assumed duplicate transactions (transactions with the same identifier) didn't exist. This is not true; in particular coinbases are easy to duplicate, and by building on duplicate coinbases, duplicate normal transactions are possible as well. Recently, an attack that exploits the reference implementation's dealing with duplicate transactions was described and demonstrated. It allows reverting fully-confirmed transactions to a single confirmation, making them vulnerable to become unspendable entirely. Another attack is possible that allows forking the block chain for a subset of the network.

Specification

To counter this problem, the following network rule is introduced:

- Blocks are not allowed to contain a transaction whose identifier matches that of an earlier, not-fully-spent transaction in the same chain.

This rule initially applied to all blocks whose timestamp is after March 15, 2012, 00:00 UTC (testnet: February 20, 2012 00:00 UTC). It was later extended by Commit Apply BIP30 checks to all blocks except the two historic violations. to apply to all blocks except the two historic blocks at heights 91842 and 91880 on the main chain that had to be grandfathered in.

Rationale

Whatever solution is used, the following law must be obeyed to guarantee sane behaviour: the set of usable transactions outputs must not be modified by adding blocks to the chain and removing them again. This happens during a reorganisation, and the current Bitcoin reference implementation does not obey this law in case the temporarily added blocks contain a duplicate transaction.

There are several potential solutions to this problem:

1. Guarantee that all coinbases are unique, making duplicate transactions very hard to create.
2. Remember previous remaining outputs of a given transaction identifier, in case a new transaction with the same identifier is added.
3. Only allow duplicate transactions in case the previous instance of the transaction had no spendable outputs left. Removing a block from the chain can then safely reset the removed transaction's outputs to nothing.

The first option is probably the most complete one, as it also guarantees transaction identifiers are unique. However, implementing it requires several changes that need to be accepted throughout the network. Furthermore, it does not prevent duplicate transactions based on earlier duplicate coinbases. The second option is impossible to implement in a forward-compatible way, as it potentially renders currently-invalid blocks valid. In this document we choose for the third option, because it only requires a trivial change.

Fully-spent transactions are allowed to be duplicated in order not to hinder pruning at some point in the future. Not allowing any transaction to be duplicated would require evidence to be kept for each transaction ever made.

Backward compatibility

The addition of this rule only makes some previously-valid blocks invalid. This implies that if the rule is implemented by a supermajority of miners, it is not possible to fork the block chain in a permanent way between nodes with and without the new rule.

Implementation

A patch for the reference client can be found on https://github.com/sipa/bitcoin/tree/nooverwritetx

This BIP was implemented in Commit Do not allow overwriting unspent transactions (BIP 30) There have been additional commits to refine the implementation of this BIP.

Acknowledgements

Thanks to Russell O'Connor for finding and demonstrating this problem, and helping test the patch.

```
BIP: 34
Layer: Consensus (soft fork)
Title: Block v2, Height in Coinbase
Authors: Gavin Andresen <gavinandresen@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2012-07-06
```

BIP 34

Abstract

Bitcoin blocks and transactions are versioned binary structures. Both currently use version 1. This BIP introduces an upgrade path for versioned transactions and blocks. A unique value is added to newly produced coinbase transactions, and blocks are updated to version 2.

Motivation

1. Clarify and exercise the mechanism whereby the bitcoin network collectively consents to upgrade transaction or block binary structures, rules and behaviors.
2. Enforce block and transaction uniqueness, and assist unconnected block validation.

Specification

1. Treat transactions with a version greater than 1 as non-standard (official Satoshi client will not mine or relay them).
2. Add height as the first item in the coinbase transaction's scriptSig, and increase block version to 2. The format of the height is "minimally encoded serialized CScript" – first byte is number of bytes in the number (will be 0x03 on main net for the next 150 or so years with 2^{23}-1 blocks), following bytes are little-endian representation of the number (including a sign bit). Height is the height of the mined block in the block chain, where the genesis block is height zero (0).
3. 75% rule: If 750 of the last 1,000 blocks are version 2 or greater, reject invalid version 2 blocks. (testnet3: 51 of last 100)
4. 95% rule ("Point of no return"): If 950 of the last 1,000 blocks are version 2 or greater, reject all version 1 blocks. (testnet3: 75 of last 100)

Backward compatibility

All older clients are compatible with this change. Users and merchants should not be impacted. Miners are strongly recommended to upgrade to version 2 blocks. Once 95% of the miners have upgraded to version 2, the remainder will be orphaned if they fail to upgrade.

Implementation

https://github.com/bitcoin/bitcoin/pull/1526

Result

Block number 227,835 (timestamp 2013-03-24 15:49:13 GMT) was the last version 1 block.

```
BIP: 42
Layer: Consensus (soft fork)
Title: A finite monetary supply for Bitcoin
Authors: Pieter Wuille <pieter.wuille@gmail.com>
Comments-Summary: Unanimously Recommended for implementation
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0042
Status: Deployed
```

```
Type: Specification
Assigned: 2014-04-01
License: PD
```

BIP 42

Abstract

Although it is widely believed that Satoshi was an inflation-hating goldbug he never said this, and in fact programmed Bitcoin's money supply to grow indefinitely, forever. He modeled the monetary supply as 4 gold mines being discovered per mibillennium (1024 years), with equal intervals between them, each one being depleted over the course of 140 years.

This poses obvious problems, however. Prominent among them is the discussion on what to call 1 billion bitcoin, which symbol color to use for it, and when wallet clients should switch to it by default.

To combat this, this document proposes a controversial change: making Bitcoin's monetary supply finite.

Details

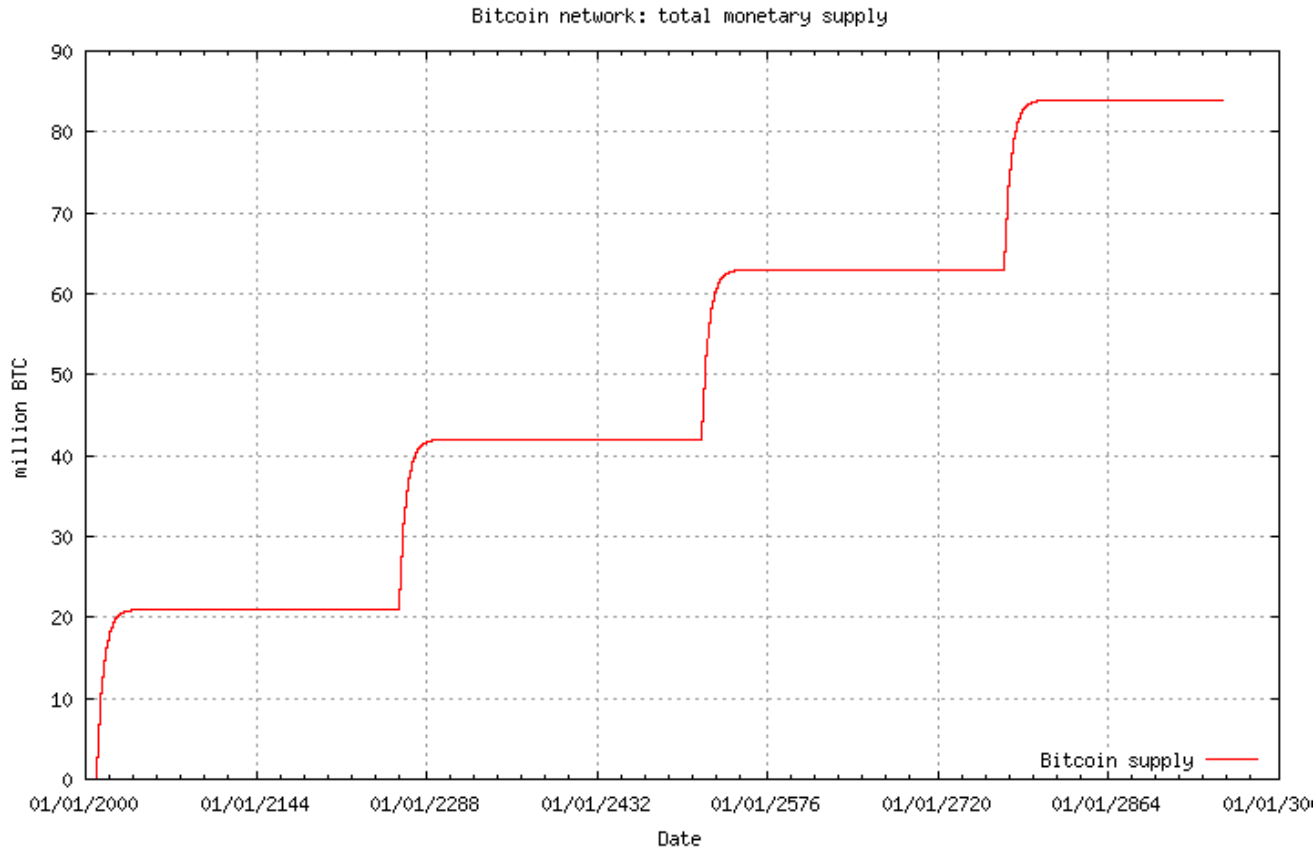
As is well known, Satoshi was a master programmer whose knowledge of C++ was surpassed only by his knowledge of Japanese culture. The code below:

```
int64_t nSubsidy = 50 * COIN;
// Subsidy is cut in half every 210,000 blocks
// which will occur approximately every 4 years.
nSubsidy >>= (nHeight / 210000);
```

is carefully written to rely on undefined behaviour in the C++ specification - perhaps so it can be hardware accelerated in future.

The block number is divided by 210000 (the "apparent" subsidy halving interval in blocks), and the result is used as input for a binary shift, applied to the original payout (50 BTC), expressed in base units. Thanks to the new-goldmine interval being exactly 64 times the halving interval, and 64 being the size in bits of the currency datatype, the cycle repeats itself every 64 halvings on all currently supported platforms.

Despite the nice showoff of underhanded programming skills - we want Bitcoin to be well-specified. Otherwise, we're clearly in for a bumpy ride:



</img>

Note that several other programming languages do not exhibit this behaviour, making new implementations likely to be slower and generally more bogus than Bitcoin Core. For example, Python unexpectedly returns 0 when shifting an integer beyond its size.

Other solutions

Floating-point approximation

An obvious solution would be to reimplement the shape of the subsidy curve using floating-point approximations, such as simulated annealing or quantitative easing, which have already proven their worth in consensus systems. Unfortunately, since the financial crisis everyone considers numbers with decimal points in them fishy, and integers are not well supported by Javascript.

Truncation

An alternative solution would be to represent the total number of bitcoins as a string:

```
"21000000000000000000000"
```

and then use string manipulation to remove the rightmost zero every 4 years, give or take a leap-year:

```
strSubsidy = strSubsidy.substr(0, strSubsidy.size() - 2);
```

This style relies less heavily on clever C++ and is more familiar to the Core Dev Team who are primarily PHP programmers.

Proposal

Instead, how about we stop thinking about long term issues when we'll all be dead (barring near lightspeed travel, cryogenic revival, or other technology— like cryptocurrency— which only exists in science fiction).

A softfork (see BIP16, BIP34, BIP62) will take place on april 1st 2214, permanently setting the subsidy to zero. The result of this will be that the total currency supply will be limited to 42 halfmillion (including the genesis coinbase output, which is not actually spendable).

Implementation

An implementation for the reference client can be found on https://github.com/bitcoin/bitcoin/pull/3842 .

Compatibility

Given the moderate time frame over which this change is to be implemented, we expect all miners to choose to screw themselves and deploy this change before 2214.

If they don't, and a minority remains on the old code base, a fork may occur. Essentially, they'll be mining fool's gold after that time.

Acknowledgements

Thanks to Gregory Maxwell for proposing this solution, and to Mike Hearn for insights into web development. Also thanks to "ditto-b" on github to implement a prototype ahead of time.

Copyright

This document is placed in the public domain.

```
BIP: 65
Layer: Consensus (soft fork)
Title: OP_CHECKLOCKTIMEVERIFY
Authors: Peter Todd <pete@petertodd.org>
Status: Deployed
Type: Specification
Assigned: 2014-10-01
License: PD
```

BIP 65

Abstract

This BIP describes a new opcode (OP_CHECKLOCKTIMEVERIFY) for the Bitcoin scripting system that allows a transaction output to be made unspendable until some point in the future.

Summary

CHECKLOCKTIMEVERIFY redefines the existing NOP2 opcode. When executed, if any of the following conditions are true, the script interpreter will terminate with an error:

- the stack is empty; or
- the top item on the stack is less than 0; or
- the lock-time type (height vs. timestamp) of the top stack item and the nLockTime field are not the same; or
- the top stack item is greater than the transaction's nLockTime field; or
- the nSequence field of the txin is 0xffffffff;

Otherwise, script execution will continue as if a NOP had been executed.

The nLockTime field in a transaction prevents the transaction from being mined until either a certain block height, or block time, has been reached. By comparing the argument to CHECKLOCKTIMEVERIFY against the nLockTime field, we indirectly verify that the desired block height or block time has been reached; until that block height or block time has been reached the transaction output remains unspendable.

Motivation

The nLockTime field in transactions can be used to prove that it is *possible* to spend a transaction output in the future, by constructing a valid transaction spending that output with the nLockTime field set.

However, the nLockTime field can't prove that it is *impossible* to spend a transaction output until some time in the future, as there is no way to know if a valid signature for a different transaction spending that output has been created.

Escrow

If Alice and Bob jointly operate a business they may want to ensure that all funds are kept in 2-of-2 multisig transaction outputs that require the co-operation of both parties to spend. However, they recognise that in exceptional circumstances such as either party getting "hit by a bus" they need a backup plan to retrieve the funds. So they appoint their lawyer, Lenny, to act as a third-party.

With a standard 2-of-3 CHECKMULTISIG at any time Lenny could conspire with either Alice or Bob to steal the funds illegitimately. Equally Lenny may prefer not to have immediate access to the funds to discourage bad actors from attempting to get the secret keys from him by force.

However, with CHECKLOCKTIMEVERIFY the funds can be stored in scriptPubKeys of the form:

```
IF
    <now + 3 months> CHECKLOCKTIMEVERIFY DROP
    <Lenny's pubkey> CHECKSIGVERIFY
    1
ELSE
    2
ENDIF
<Alice's pubkey> <Bob's pubkey> 2 CHECKMULTISIG
```

At any time the funds can be spent with the following scriptSig:

```
0 <Alice's signature> <Bob's signature> 0
```

After 3 months have passed Lenny and one of either Alice or Bob can spend the funds with the following scriptSig:

```
0 <Alice/Bob's signature> <Lenny's signature> 1
```

Non-interactive time-locked refunds

There exist a number of protocols where a transaction output is created that requires the co-operation of both parties to spend the output. To ensure the failure of one party does not result in the funds becoming lost, refund transactions are setup in advance using nLockTime. These refund transactions need to be created interactively, and additionally, are currently vulnerable to transaction malleability. CHECKLOCKTIMEVERIFY can be used in these protocols, replacing the interactive setup with a non-interactive setup, and additionally, making transaction malleability a non-issue.

Two-factor wallets

Services like GreenAddress store bitcoins with 2-of-2 multisig scriptPubKey's such that one keypair is controlled by the user, and the other keypair is controlled by the service. To spend funds the user uses locally installed wallet software that generates one of the required signatures, and then uses a 2nd-factor authentication method to authorize the service to create the second SIGHASH_NONE signature that is locked until some time in the future and sends the user that signature for storage. If the user needs to spend their funds and the service is not available, they wait until the nLockTime expires.

The problem is there exist numerous occasions the user will not have a valid signature for some or all of their transaction outputs. With CHECKLOCKTIMEVERIFY rather than creating refund signatures on demand scriptPubKeys of the following form are used instead:

```
IF
    <service pubkey> CHECKSIGVERIFY
ELSE
    <expiry time> CHECKLOCKTIMEVERIFY DROP
ENDIF
<user pubkey> CHECKSIG
```

Now the user is always able to spend their funds without the co-operation of the service by waiting for the expiry time to be reached.

Payment Channels

Jeremy Spilman style payment channels first setup a deposit controlled by 2-of-2 multisig, tx1, and then adjust a second transaction, tx2, that spends the output of tx1 to payor and payee. Prior to publishing tx1 a refund transaction is created, tx3, to ensure that should the payee vanish the payor can get their deposit back. The process by which the refund transaction is created is currently vulnerable to transaction malleability attacks, and additionally, requires the payor to store the refund. Using the same scriptPubKey form as in the Two-factor wallets example solves both these issues.

Trustless Payments for Publishing Data

The PayPub protocol makes it possible to pay for information in a trustless way by first proving that an encrypted file contains the desired data, and secondly crafting scriptPubKeys used for payment such that spending them reveals the encryption keys to the data. However the existing implementation has a significant flaw: the publisher can delay the release of the keys indefinitely.

This problem can be solved interactively with the refund transaction technique; with CHECKLOCKTIMEVERIFY the problem can be non-interactively solved using scriptPubKeys of the following form:

```
IF
    HASH160 <Hash160(encryption key)> EQUALVERIFY
    <publisher pubkey> CHECKSIG
ELSE
    <expiry time> CHECKLOCKTIMEVERIFY DROP
    <buyer pubkey> CHECKSIG
ENDIF
```

The buyer of the data is now making a secure offer with an expiry time. If the publisher fails to accept the offer before the expiry time is reached the buyer can cancel the offer by spending the output.

Proving sacrifice to miners' fees

Proving the sacrifice of some limited resource is a common technique in a variety of cryptographic protocols. Proving sacrifices of coins to mining fees has been proposed as a *universal public good* to which the sacrifice could be directed, rather than simply destroying the coins. However doing so is non-trivial, and even the best existing technique - announce-commit sacrifices - could encourage mining centralization. CHECKLOCKTIMEVERIFY can be used to create outputs that are provably spendable by anyone (thus to mining fees assuming miners behave optimally and rationally) but only at a time sufficiently far into the future that large miners can't profitably sell the sacrifices at a discount.

Freezing Funds

In addition to using cold storage, hardware wallets, and P2SH multisig outputs to control funds, now funds can be frozen in UTXOs directly on the blockchain. With the following scriptPubKey, nobody will be able to spend the encumbered output until the provided expiry time. This ability to freeze funds reliably may be useful in scenarios where reducing duress or confiscation risk is desired.

```
<expiry time> CHECKLOCKTIMEVERIFY DROP DUP HASH160 <pubKeyHash> EQUALVERIFY CHECKSIG
```

Replacing the nLockTime field entirely

As an aside, note how if the SignatureHash() algorithm could optionally cover part of the scriptSig the signature could require that the scriptSig contain CHECKLOCKTIMEVERIFY opcodes, and additionally, require that they be executed. (the CODESEPARATOR opcode came very close to making this possible in v0.1 of Bitcoin) This per-signature capability could replace the per-transaction nLockTime field entirely as a valid signature would now be the proof that a transaction output *can* be spent.

Detailed Specification

Refer to the reference implementation, reproduced below, for the precise semantics and detailed rationale for those semantics.

```
case OP_NOP2:
{
    // CHECKLOCKTIMEVERIFY
    //
    // (nLockTime -- nLockTime )

    if (!(flags & SCRIPT_VERIFY_CHECKLOCKTIMEVERIFY))
        break; // not enabled; treat as a NOP

    if (stack.size() < 1)
        return false;

    // Note that elsewhere numeric opcodes are limited to
    // operands in the range -2**31+1 to 2**31-1, however it is
    // legal for opcodes to produce results exceeding that
    // range. This limitation is implemented by CScriptNum's
    // default 4-byte limit.
    //
    // If we kept to that limit we'd have a year 2038 problem,
    // even though the nLockTime field in transactions
    // themselves is uint32 which only becomes meaningless
    // after the year 2106.
    //
    // Thus as a special case we tell CScriptNum to accept up
    // to 5-byte bignums, which are good until 2**32-1, the
    // same limit as the nLockTime field itself.
    const CScriptNum nLockTime(stacktop(-1), 5);

    // In the rare event that the argument may be < 0 due to
    // some arithmetic being done first, you can always use
    // 0 MAX CHECKLOCKTIMEVERIFY.
    if (nLockTime < 0)
        return false;

    // There are two types of nLockTime: lock-by-blockheight
    // and lock-by-blocktime, distinguished by whether
    // nLockTime < LOCKTIME_THRESHOLD.
    //
    // We want to compare apples to apples, so fail the script
    // unless the type of nLockTime being tested is the same as
    // the nLockTime in the transaction.
    if (!(
          (txTo.nLockTime <  LOCKTIME_THRESHOLD && nLockTime <  LOCKTIME_THRESHOLD) ||
          (txTo.nLockTime >= LOCKTIME_THRESHOLD && nLockTime >= LOCKTIME_THRESHOLD)
```

```
            ))
            return false;

        // Now that we know we're comparing apples-to-apples, the
        // comparison is a simple numeric one.
        if (nLockTime > (int64_t)txTo.nLockTime)
            return false;

        // Finally the nLockTime feature can be disabled and thus
        // CHECKLOCKTIMEVERIFY bypassed if every txin has been
        // finalized by setting nSequence to maxint. The
        // transaction would be allowed into the blockchain, making
        // the opcode ineffective.
        //
        // Testing if this vin is not final is sufficient to
        // prevent this condition. Alternatively we could test all
        // inputs, but testing just this input minimizes the data
        // required to prove correct CHECKLOCKTIMEVERIFY execution.
        if (txTo.vin[nIn].IsFinal())
            return false;

        break;

    }
```

https://github.com/petertodd/bitcoin/commit/ab0f54f38e08ee1e50ff72f801680ee84d0f1bf4

Deployment

We reuse the double-threshold IsSuperMajority() switchover mechanism used in BIP66 with the same thresholds, but for nVersion = 4. The new rules are in effect for every block (at height H) with nVersion = 4 and at least 750 out of 1000 blocks preceding it (with heights H-1000..H-1) also have nVersion >= 4. Furthermore, when 950 out of the 1000 blocks preceding a block do have nVersion >= 4, nVersion < 4 blocks become invalid, and all further blocks enforce the new rules.

It should be noted that BIP9 involves permanently setting a high-order bit to 1 which results in nVersion >= all prior IsSuperMajority() soft-forks and thus no bits in nVersion are permanently lost.

SPV Clients

While SPV clients are (currently) unable to validate blocks in general, trusting miners to do validation for them, they are able to validate block headers and thus can validate a subset of the deployment rules. SPV clients should reject nVersion < 4 blocks if 950 out of 1000 preceding blocks have nVersion >= 4 to prevent false confirmations from the remaining 5% of non-upgraded miners when the 95% threshold has been reached.

Credits

Thanks goes to Gregory Maxwell for suggesting that the argument be compared against the per-transaction nLockTime, rather than the current block height and time.

References

PayPub

- https://github.com/unsystem/paypub

Jeremy Spilman Payment Channels

- https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2013-April/002433.html

Implementations

Python / python-bitcoinlib

- https://github.com/petertodd/checklocktimeverify-demos

JavaScript / Node.js / bitcore

- https://github.com/mruddy/bip65-demos

Copyright

This document is placed in the public domain.

```
BIP: 66
Layer: Consensus (soft fork)
Title: Strict DER signatures
Authors: Pieter Wuille <pieter.wuille@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2015-01-10
License: BSD-2-Clause
```

BIP 66

Abstract

This document specifies proposed changes to the Bitcoin transaction validity rules to restrict signatures to strict DER encoding.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

Bitcoin's reference implementation currently relies on OpenSSL for signature validation, which means it is implicitly defining Bitcoin's block validity rules. Unfortunately, OpenSSL is not designed for consensus-critical behaviour (it does not guarantee bug-for-bug compatibility between versions), and thus changes to it can - and have - affected Bitcoin software.

One specifically critical area is the encoding of signatures. Until recently, OpenSSL's releases would accept various deviations from the DER standard and accept signatures as valid. When this changed in OpenSSL 1.0.0p and 1.0.1k, it made some nodes reject the chain.

This document proposes to restrict valid signatures to exactly what is mandated by DER, to make the consensus rules not depend on OpenSSL's signature parsing. A change like this is required if implementations would want to remove all of OpenSSL from the consensus code.

Specification

Every signature passed to OP_CHECKSIG, OP_CHECKSIGVERIFY, OP_CHECKMULTISIG, or OP_CHECKMULTISIGVERIFY, to which ECDSA verification is applied, must be encoded using strict DER encoding (see further).

These operators all perform ECDSA verifications on pubkey/signature pairs, iterating from the top of the stack backwards. For each such verification, if the signature does not pass the `IsValidSignatureEncoding` check below, the entire script evaluates to false immediately. If the signature is valid DER, but does not pass ECDSA verification, opcode execution continues as it used to, causing opcode execution to stop and push false on the stack (but not immediately fail the script) in some cases, which potentially skips further signatures (and thus does not subject them to `IsValidSignatureEncoding`).

DER encoding reference

The following code specifies the behaviour of strict DER checking. Note that this function tests a signature byte vector which includes the 1-byte sighash flag that Bitcoin adds, even though that flag falls outside of the DER specification, and is unaffected by this proposal. The function is also not called for cases where the length of sig is 0, in order to provide a simple, short and efficiently-verifiable encoding for deliberately invalid signatures.

DER is specified in https://www.itu.int/rec/T-REC-X.690/en .

```
bool static IsValidSignatureEncoding(const std::vector<unsigned char> &sig) {
    // Format: 0x30 [total-length] 0x02 [R-length] [R] 0x02 [S-length] [S] [sighash]
    // * total-length: 1-byte length descriptor of everything that follows,
    //   excluding the sighash byte.
    // * R-length: 1-byte length descriptor of the R value that follows.
    // * R: arbitrary-length big-endian encoded R value. It must use the shortest
    //   possible encoding for a positive integers (which means no null bytes at
    //   the start, except a single one when the next byte has its highest bit set).
    // * S-length: 1-byte length descriptor of the S value that follows.
    // * S: arbitrary-length big-endian encoded S value. The same rules apply.
    // * sighash: 1-byte value indicating what data is hashed (not part of the DER
    //   signature)

    // Minimum and maximum size constraints.
    if (sig.size() < 9) return false;
    if (sig.size() > 73) return false;

    // A signature is of type 0x30 (compound).
    if (sig[0] != 0x30) return false;

    // Make sure the length covers the entire signature.
    if (sig[1] != sig.size() - 3) return false;
```

```
    // Extract the length of the R element.
    unsigned int lenR = sig[3];

    // Make sure the length of the S element is still inside the signature.
    if (5 + lenR >= sig.size()) return false;

    // Extract the length of the S element.
    unsigned int lenS = sig[5 + lenR];

    // Verify that the length of the signature matches the sum of the length
    // of the elements.
    if ((size_t)(lenR + lenS + 7) != sig.size()) return false;

    // Check whether the R element is an integer.
    if (sig[2] != 0x02) return false;

    // Zero-length integers are not allowed for R.
    if (lenR == 0) return false;

    // Negative numbers are not allowed for R.
    if (sig[4] & 0x80) return false;

    // Null bytes at the start of R are not allowed, unless R would
    // otherwise be interpreted as a negative number.
    if (lenR > 1 && (sig[4] == 0x00) && !(sig[5] & 0x80)) return false;

    // Check whether the S element is an integer.
    if (sig[lenR + 4] != 0x02) return false;

    // Zero-length integers are not allowed for S.
    if (lenS == 0) return false;

    // Negative numbers are not allowed for S.
    if (sig[lenR + 6] & 0x80) return false;

    // Null bytes at the start of S are not allowed, unless S would otherwise be
    // interpreted as a negative number.
    if (lenS > 1 && (sig[lenR + 6] == 0x00) && !(sig[lenR + 7] & 0x80)) return false;

    return true;
}
```

Examples

Notation: P1 and P2 are valid, serialized, public keys. S1 and S2 are valid signatures using respective keys P1 and P2. S1′ and S2′ are non-DER but otherwise valid signatures using those same keys. F is any invalid but DER-compliant signature (including 0, the empty string). F′ is any invalid and non-DER-compliant signature.

1. `S1' P1 CHECKSIG` fails (**changed**)
2. `S1' P1 CHECKSIG NOT` fails (unchanged)
3. `F P1 CHECKSIG` fails (unchanged)
4. `F P1 CHECKSIG NOT` can succeed (unchanged)

5. `F' P1 CHECKSIG` fails (unchanged)
6. `F' P1 CHECKSIG NOT` fails (**changed**)

1. `0 S1' S2 2 P1 P2 2 CHECKMULTISIG` fails (**changed**)
2. `0 S1' S2 2 P1 P2 2 CHECKMULTISIG NOT` fails (unchanged)
3. `0 F S2' 2 P1 P2 2 CHECKMULTISIG` fails (unchanged)
4. `0 F S2' 2 P1 P2 2 CHECKMULTISIG NOT` fails (**changed**)
5. `0 S1' F 2 P1 P2 2 CHECKMULTISIG` fails (unchanged)
6. `0 S1' F 2 P1 P2 2 CHECKMULTISIG NOT` can succeed (unchanged)

Note that the examples above show that only additional failures are required by this change, as required for a soft forking change.

Deployment

We reuse the double-threshold switchover mechanism from BIP 34, with the same thresholds, but for nVersion = 3. The new rules are in effect for every block (at height H) with nVersion = 3 and at least 750 out of 1000 blocks preceding it (with heights H-1000..H-1) also have nVersion = 3. Furthermore, when 950 out of the 1000 blocks preceding a block do have nVersion = 3, nVersion = 2 blocks become invalid, and all further blocks enforce the new rules.

Compatibility

The requirement to have signatures that comply strictly with DER has been enforced as a relay policy by the reference client since v0.8.0, and very few transactions violating it are being added to the chain as of January 2015. In addition, every non-compliant signature can trivially be converted into a compliant one, so there is no loss of functionality by this requirement. This proposal has the added benefit of reducing transaction malleability (see BIP 62).

Implementation

An implementation for the reference client is available at https://github.com/bitcoin/bitcoin/pull/5713

Acknowledgements

This document is extracted from the previous BIP62 proposal, which had input from various people, in particular Greg Maxwell and Peter Todd, who gave feedback about this document as well.

Disclosures

- Subsequent to the network-wide adoption and enforcement of this BIP, the author disclosed that strict DER signatures provided an indirect solution to a consensus bug he had previously discovered.


```
BIP: 68
Layer: Consensus (soft fork)
Title: Relative lock-time using consensus-enforced sequence numbers
Authors: Mark Friedenbach <mark@friedenbach.org>
         BtcDrak <btcdrak@gmail.com>
         Nicolas Dorier <nicolas.dorier@gmail.com>
         kinoshitajona <kinoshitajona@gmail.com>
```

```
Status: Deployed
Type: Specification
Assigned: 2015-05-28
```

BIP 68

Abstract

This BIP introduces relative lock-time (RLT) consensus-enforced semantics of the sequence number field to enable a signed transaction input to remain invalid for a defined period of time after confirmation of its corresponding outpoint.

Motivation

Bitcoin transactions have a sequence number field for each input. The original idea appears to have been that a transaction in the mempool would be replaced by using the same input with a higher sequence value. Although this was not properly implemented, it assumes miners would prefer higher sequence numbers even if the lower ones were more profitable to mine. However, a miner acting on profit motives alone would break that assumption completely. The change described by this BIP repurposes the sequence number for new use cases without breaking existing functionality. It also leaves room for future expansion and other use cases.

The transaction nLockTime is used to prevent the mining of a transaction until a certain date. nSequence will be repurposed to prevent mining of a transaction until a certain age of the spent output in blocks or timespan. This, among other uses, allows bi-directional payment channels as used in Hashed Timelock Contracts (HTLCs) and BIP112.

Specification

This specification defines the meaning of sequence numbers for transactions with an nVersion greater than or equal to 2 for which the rest of this specification relies on.

All references to median-time-past (MTP) are as defined by BIP113.

If bit (1 << 31) of the sequence number is set, then no consensus meaning is applied to the sequence number and can be included in any block under all currently possible circumstances.

If bit (1 << 31) of the sequence number is not set, then the sequence number is interpreted as an encoded relative lock-time.

The sequence number encoding is interpreted as follows:

Bit (1 << 22) determines if the relative lock-time is time-based or block based: If the bit is set, the relative lock-time specifies a timespan in units of 512 seconds granularity. The timespan starts from the median-time-past of the output's previous block, and ends at the MTP of the previous block. If the bit is not set, the relative lock-time specifies a number of blocks.

The flag (1<<22) is the highest order bit in a 3-byte signed integer for use in bitcoin scripts as a 3-byte PUSHDATA with OP_CHECKSEQUENCEVERIFY (BIP 112).

This specification only interprets 16 bits of the sequence number as relative lock-time, so a mask of 0x0000ffff MUST be applied to the sequence field to extract the relative lock-time. The 16-bit specification allows for a year of relative lock-time and the remaining bits allow for future expansion.

For time based relative lock-time, 512 second granularity was chosen because bitcoin blocks are generated every 600 seconds. So when using block-based or time-based, the same amount of time can be encoded with the available number of bits. Converting from a sequence number to seconds is performed by multiplying by 512 = 2^9, or equivalently shifting up by 9 bits.

When the relative lock-time is time-based, it is interpreted as a minimum block-time constraint over the input's age. A relative time-based lock-time of zero indicates an input which can be included in any block. More generally, a relative time-based lock-time n can be included into any block produced 512 * n seconds after the mining date of the output it is spending, or any block thereafter. The mining date of the output is equal to the median-time-past of the previous block which mined it.

The block produced time is equal to the median-time-past of its previous block.

When the relative lock-time is block-based, it is interpreted as a minimum block-height constraint over the input's age. A relative block-based lock-time of zero indicates an input which can be included in any block. More generally, a relative block lock-time n can be included n blocks after the mining date of the output it is spending, or any block thereafter.

The new rules are not applied to the nSequence field of the input of the coinbase transaction.

Implementation

A reference implementation is provided by the following pull request

https://github.com/bitcoin/bitcoin/pull/7184

```
enum {
    /* Interpret sequence numbers as relative lock-time constraints. */
    LOCKTIME_VERIFY_SEQUENCE = (1 << 0),
};

/* Setting nSequence to this value for every input in a transaction
 * disables nLockTime. */
static const uint32_t SEQUENCE_FINAL = 0xffffffff;

/* Below flags apply in the context of BIP 68*/
/* If this flag set, CTxIn::nSequence is NOT interpreted as a
 * relative lock-time. */
static const uint32_t SEQUENCE_LOCKTIME_DISABLE_FLAG = (1 << 31);

/* If CTxIn::nSequence encodes a relative lock-time and this flag
 * is set, the relative lock-time has units of 512 seconds,
 * otherwise it specifies blocks with a granularity of 1. */
static const uint32_t SEQUENCE_LOCKTIME_TYPE_FLAG = (1 << 22);

/* If CTxIn::nSequence encodes a relative lock-time, this mask is
 * applied to extract that lock-time from the sequence field. */
```

```
static const uint32_t SEQUENCE_LOCKTIME_MASK = 0x0000ffff;

/* In order to use the same number of bits to encode roughly the
 * same wall-clock duration, and because blocks are naturally
 * limited to occur every 600s on average, the minimum granularity
 * for time-based relative lock-time is fixed at 512 seconds.
 * Converting from CTxIn::nSequence to seconds is performed by
 * multiplying by 512 = 2^9, or equivalently shifting up by
 * 9 bits. */
static const int SEQUENCE_LOCKTIME_GRANULARITY = 9;

/**
 * Calculates the block height and previous block's median time past at
 * which the transaction will be considered final in the context of BIP 68.
 * Also removes from the vector of input heights any entries which did not
 * correspond to sequence locked inputs as they do not affect the calculation.
 */
static std::pair<int, int64_t> CalculateSequenceLocks(const CTransaction &tx, int flags,
std::vector<int>* prevHeights, const CBlockIndex& block)
{
    assert(prevHeights->size() == tx.vin.size());

    // Will be set to the equivalent height- and time-based nLockTime
    // values that would be necessary to satisfy all relative lock-
    // time constraints given our view of block chain history.
    // The semantics of nLockTime are the last invalid height/time, so
    // use -1 to have the effect of any height or time being valid.
    int nMinHeight = -1;
    int64_t nMinTime = -1;

    // tx.nVersion is signed integer so requires cast to unsigned otherwise
    // we would be doing a signed comparison and half the range of nVersion
    // wouldn't support BIP 68.
    bool fEnforceBIP68 = static_cast<uint32_t>(tx.nVersion) >= 2
                      && flags & LOCKTIME_VERIFY_SEQUENCE;

    // Do not enforce sequence numbers as a relative lock time
    // unless we have been instructed to
    if (!fEnforceBIP68) {
        return std::make_pair(nMinHeight, nMinTime);
    }

    for (size_t txinIndex = 0; txinIndex < tx.vin.size(); txinIndex++) {
        const CTxIn& txin = tx.vin[txinIndex];

        // Sequence numbers with the most significant bit set are not
        // treated as relative lock-times, nor are they given any
        // consensus-enforced meaning at this point.
        if (txin.nSequence & CTxIn::SEQUENCE_LOCKTIME_DISABLE_FLAG) {
            // The height of this input is not relevant for sequence locks
            (*prevHeights)[txinIndex] = 0;
            continue;
        }

        int nCoinHeight = (*prevHeights)[txinIndex];
```

```
        if (txin.nSequence & CTxIn::SEQUENCE_LOCKTIME_TYPE_FLAG) {
            int64_t nCoinTime = block.GetAncestor(std::max(nCoinHeight-1, 0))-
>GetMedianTimePast();
            // NOTE: Subtract 1 to maintain nLockTime semantics
            // BIP 68 relative lock times have the semantics of calculating
            // the first block or time at which the transaction would be
            // valid. When calculating the effective block time or height
            // for the entire transaction, we switch to using the
            // semantics of nLockTime which is the last invalid block
            // time or height.  Thus we subtract 1 from the calculated
            // time or height.

            // Time-based relative lock-times are measured from the
            // smallest allowed timestamp of the block containing the
            // txout being spent, which is the median time past of the
            // block prior.
            nMinTime = std::max(nMinTime, nCoinTime + (int64_t)((txin.nSequence &
CTxIn::SEQUENCE_LOCKTIME_MASK) << CTxIn::SEQUENCE_LOCKTIME_GRANULARITY) - 1);
        } else {
            nMinHeight = std::max(nMinHeight, nCoinHeight + (int)(txin.nSequence &
CTxIn::SEQUENCE_LOCKTIME_MASK) - 1);
        }
    }

    return std::make_pair(nMinHeight, nMinTime);
}

static bool EvaluateSequenceLocks(const CBlockIndex& block, std::pair<int, int64_t> lockPair)
{
    assert(block.pprev);
    int64_t nBlockTime = block.pprev->GetMedianTimePast();
    if (lockPair.first >= block.nHeight || lockPair.second >= nBlockTime)
        return false;

    return true;
}

bool SequenceLocks(const CTransaction &tx, int flags, std::vector<int>* prevHeights, const
CBlockIndex& block)
{
    return EvaluateSequenceLocks(block, CalculateSequenceLocks(tx, flags, prevHeights, block));
}

bool CheckSequenceLocks(const CTransaction &tx, int flags)
{
    AssertLockHeld(cs_main);
    AssertLockHeld(mempool.cs);

    CBlockIndex* tip = chainActive.Tip();
    CBlockIndex index;
    index.pprev = tip;
    // CheckSequenceLocks() uses chainActive.Height()+1 to evaluate
    // height based locks because when SequenceLocks() is called within
    // ConnectBlock(), the height of the block *being*
```

```
    // evaluated is what is used.
    // Thus if we want to know if a transaction can be part of the
    // *next* block, we need to use one more than chainActive.Height()
    index.nHeight = tip->nHeight + 1;

    // pcoinsTip contains the UTXO set for chainActive.Tip()
    CCoinsViewMemPool viewMemPool(pcoinsTip, mempool);
    std::vector<int> prevheights;
    prevheights.resize(tx.vin.size());
    for (size_t txinIndex = 0; txinIndex < tx.vin.size(); txinIndex++) {
        const CTxIn& txin = tx.vin[txinIndex];
        CCoins coins;
        if (!viewMemPool.GetCoins(txin.prevout.hash, coins)) {
            return error("%s: Missing input", __func__);
        }
        if (coins.nHeight == MEMPOOL_HEIGHT) {
            // Assume all mempool transactions are confirmed in the next block
            prevheights[txinIndex] = tip->nHeight + 1;
        } else {
            prevheights[txinIndex] = coins.nHeight;
        }
    }

    std::pair<int, int64_t> lockPair = CalculateSequenceLocks(tx, flags, &prevheights, index);
    return EvaluateSequenceLocks(index, lockPair);
}
```

Acknowledgments

Credit goes to Gregory Maxwell for providing a succinct and clear description of the behavior of this change, which became the basis of this BIP text.

This BIP was edited by BtcDrak, Nicolas Dorier and kinoshitajona.

Deployment

This BIP is to be deployed by "versionbits" BIP9 using bit 0.

For Bitcoin **mainnet**, the BIP9 **starttime** will be midnight 1st May 2016 UTC (Epoch timestamp 1462060800) and BIP9 **timeout** will be midnight 1st May 2017 UTC (Epoch timestamp 1493596800).

For Bitcoin **testnet**, the BIP9 **starttime** will be midnight 1st March 2016 UTC (Epoch timestamp 1456790400) and BIP9 **timeout** will be midnight 1st May 2017 UTC (Epoch timestamp 1493596800).

This BIP must be deployed simultaneously with BIP112 and BIP113 using the same deployment mechanism.

Compatibility

The only use of sequence numbers by the Bitcoin Core reference client software is to disable checking the nLockTime constraints in a transaction. The semantics of that application are preserved by this BIP.

As can be seen from the specification section, a number of bits are undefined by this BIP to allow for other use cases by setting bit (1 << 31) as the remaining 31 bits have no meaning under this BIP. Additionally, bits (1 << 23) through (1 << 30) inclusive have no meaning at all when bit (1 << 31) is unset.

Additionally, this BIP specifies only 16 bits to actually encode relative lock-time meaning a further 6 are unused (1 << 16 through 1 << 21 inclusive). This allows the possibility to increase granularity by soft-fork, or for increasing the maximum possible relative lock-time in the future.

The most efficient way to calculate sequence number from relative lock-time is with bit masks and shifts:

```
// 0 <= nHeight <= 65,535 blocks (1.25 years)
nSequence = nHeight;
nHeight = nSequence & 0x0000ffff;

// 0 <= nTime < 33,554,431 seconds (1.06 years)
nSequence = (1 << 22) | (nTime >> 9);
nTime = (nSequence & 0x0000ffff) << 9;
```

References

Bitcoin mailing list discussion: https://www.mail-archive.com/bitcoin-development@lists.sourceforge.net/msg07864.html

BIP9: https://github.com/bitcoin/bips/blob/master/bip-0009.mediawiki

BIP112: https://github.com/bitcoin/bips/blob/master/bip-0112.mediawiki

BIP113: https://github.com/bitcoin/bips/blob/master/bip-0113.mediawiki

Hashed Timelock Contracts (HTLCs): https://github.com/ElementsProject/lightning/raw/master/doc/miscellaneous/deployable-lightning.pdf

```
BIP: 112
Layer: Consensus (soft fork)
Title: CHECKSEQUENCEVERIFY
Authors: BtcDrak <btcdrak@gmail.com>
         Mark Friedenbach <mark@friedenbach.org>
         Eric Lombrozo <elombrozo@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2015-08-10
License: PD
```

BIP 112

Abstract

This BIP describes a new opcode (CHECKSEQUENCEVERIFY) for the Bitcoin scripting system that in combination with BIP 68 allows execution pathways of a script to be restricted based on the age of the output being spent.

Summary

CHECKSEQUENCEVERIFY redefines the existing NOP3 opcode. When executed, if any of the following conditions are true, the script interpreter will terminate with an error:

- the stack is empty; or
- the top item on the stack is less than 0; or
- the top item on the stack has the disable flag (1 << 31) unset; and
    - the transaction version is less than 2; or
    - the transaction input sequence number disable flag (1 << 31) is set; or
    - the relative lock-time type is not the same; or
    - the top stack item is greater than the transaction input sequence (when masked according to the BIP68);

Otherwise, script execution will continue as if a NOP had been executed.

BIP 68 prevents a non-final transaction from being selected for inclusion in a block until the corresponding input has reached the specified age, as measured in block-height or block-time. By comparing the argument to CHECKSEQUENCEVERIFY against the nSequence field, we indirectly verify a desired minimum age of the output being spent; until that relative age has been reached any script execution pathway including the CHECKSEQUENCEVERIFY will fail to validate, causing the transaction not to be selected for inclusion in a block.

Motivation

BIP 68 repurposes the transaction nSequence field meaning by giving sequence numbers new consensus-enforced semantics as a relative lock-time. However, there is no way to build Bitcoin scripts to make decisions based on this field.

By making the nSequence field accessible to script, it becomes possible to construct code pathways that only become accessible some minimum time after proof-of-publication. This enables a wide variety of applications in phased protocols such as escrow, payment channels, or bidirectional pegs.

Contracts With Expiration Deadlines

Escrow with Timeout

An escrow that times out automatically 30 days after being funded can be established in the following way. Alice, Bob and Escrow create a 2-of-3 address with the following redeemscript.

```
IF
    2 <Alice's pubkey> <Bob's pubkey> <Escrow's pubkey> 3 CHECKMULTISIG
ELSE
    "30d" CHECKSEQUENCEVERIFY DROP
    <Alice's pubkey> CHECKSIG
ENDIF
```

At any time funds can be spent using signatures from any two of Alice, Bob or the Escrow.

After 30 days Alice can sign alone.

The clock does not start ticking until the payment to the escrow address confirms.

Retroactive Invalidation

In many instances, we would like to create contracts that can be revoked in case of some future event. However, given the immutable nature of the blockchain, it is practically impossible to retroactively invalidate a previous commitment that has already confirmed. The only mechanism we really have for retroactive invalidation is blockchain reorganization which, for fundamental security reasons, is designed to be very hard and very expensive to do.

Despite this limitation, we do have a way to provide something functionally similar to retroactive invalidation while preserving irreversibility of past commitments using CHECKSEQUENCEVERIFY. By constructing scripts with multiple branches of execution where one or more of the branches are delayed we provide a time window in which someone can supply an invalidation condition that allows the output to be spent, effectively invalidating the would-be delayed branch and potentially discouraging another party from broadcasting the transaction in the first place. If the invalidation condition does not occur before the timeout, the delayed branch becomes spendable, honoring the original contract.

Some more specific applications of this idea:

Hash Time-Locked Contracts

Hash Time-Locked Contracts (HTLCs) provide a general mechanism for off-chain contract negotiation. An execution pathway can be made to require knowledge of a secret (a hash preimage) that can be presented within an invalidation time window. By sharing the secret it is possible to guarantee to the counterparty that the transaction will never be broadcast since this would allow the counterparty to claim the output immediately while one would have to wait for the time window to pass. If the secret has not been shared, the counterparty will be unable to use the instant pathway and the delayed pathway must be used instead.

Bidirectional Payment Channels

Scriptable relative locktime provides a predictable amount of time to respond in the event a counterparty broadcasts a revoked transaction: Absolute locktime necessitates closing the channel and reopen it when getting close to the timeout, whereas with relative locktime, the clock starts ticking the moment the transactions confirms in a block. It also provides a means to know exactly how long to wait (in number of blocks) before funds can be pulled out of the channel in the event of a noncooperative counterparty.

Lightning Network

The lightning network extends the bidirectional payment channel idea to allow for payments to be routed over multiple bidirectional payment channel hops.

These channels are based on an anchor transaction that requires a 2-of-2 multisig from Alice and Bob, and a series of revocable commitment transactions that spend the anchor transaction. The commitment transaction splits the funds from the anchor between Alice and Bob and the latest commitment transaction may be published by either party at any time, finalising the channel.

Ideally then, a revoked commitment transaction would never be able to be successfully spent; and the latest commitment transaction would be able to be spent very quickly.

To allow a commitment transaction to be effectively revoked, Alice and Bob have slightly different versions of the latest commitment transaction. In Alice's version, any outputs in the commitment transaction that pay Alice also include a forced delay, and an alternative branch that allows Bob to spend the output if he knows that transaction's revocation code. In Bob's version, payments to Bob are similarly encumbered. When Alice and Bob negotiate new balances and new commitment transactions, they also reveal the old revocation code, thus committing to not relaying the old transaction.

A simple output, paying to Alice might then look like:

```
HASH160 <revokehash> EQUAL
IF
    <Bob's pubkey>
ELSE
    "24h" CHECKSEQUENCEVERIFY DROP
    <Alice's pubkey>
ENDIF
CHECKSIG
```

This allows Alice to publish the latest commitment transaction at any time and spend the funds after 24 hours, but also ensures that if Alice relays a revoked transaction, that Bob has 24 hours to claim the funds.

With CHECKLOCKTIMEVERIFY, this would look like:

```
HASH160 <revokehash> EQUAL
IF
    <Bob's pubkey>
ELSE
    "2015/12/15" CHECKLOCKTIMEVERIFY DROP
    <Alice's pubkey>
ENDIF
CHECKSIG
```

This form of transaction would mean that if the anchor is unspent on 2015/12/16, Alice can use this commitment even if it has been revoked, simply by spending it immediately, giving no time for Bob to claim it.

This means that the channel has a deadline that cannot be pushed back without hitting the blockchain; and also that funds may not be available until the deadline is hit. CHECKSEQUENCEVERIFY allows you to avoid making such a tradeoff.

Hashed Time-Lock Contracts (HTLCs) make this slightly more complicated, since in principle they may pay either Alice or Bob, depending on whether Alice discovers a secret R, or a timeout is reached, but the same principle applies – the branch paying Alice in Alice's commitment transaction gets a delay, and the entire output can be claimed by the other party if the revocation secret is known. With CHECKSEQUENCEVERIFY, a HTLC payable to Alice might look like the following in Alice's commitment transaction:

```
HASH160 DUP <R-HASH> EQUAL
IF
```

```
    "24h" CHECKSEQUENCEVERIFY
    2DROP
    <Alice's pubkey>
ELSE
    <Commit-Revocation-Hash> EQUAL
    NOTIF
        "2015/10/20 10:33" CHECKLOCKTIMEVERIFY DROP
    ENDIF
    <Bob's pubkey>
ENDIF
CHECKSIG
```

and correspondingly in Bob's commitment transaction:

```
HASH160 DUP <R-HASH> EQUAL
SWAP <Commit-Revocation-Hash> EQUAL ADD
IF
    <Alice's pubkey>
ELSE
    "2015/10/20 10:33" CHECKLOCKTIMEVERIFY
    "24h" CHECKSEQUENCEVERIFY
    2DROP
    <Bob's pubkey>
ENDIF
CHECKSIG
```

Note that both CHECKSEQUENCEVERIFY and CHECKLOCKTIMEVERIFY are used in the final branch of above to ensure Bob cannot spend the output until after both the timeout is complete and Alice has had time to reveal the revocation secret.

See the Deployable Lightning paper.

2-Way Pegged Sidechains

The 2-way pegged sidechain requires a new REORGPROOFVERIFY opcode, the semantics of which are outside the scope of this BIP. CHECKSEQUENCEVERIFY is used to make sure that sufficient time has passed since the return peg was posted to publish a reorg proof:

```
IF
    lockTxHeight <lockTxHash> nlocktxOut [<workAmount>] reorgBounty Hash160(<...>) <genesisHash> REORGPRO
ELSE
    withdrawLockTime CHECKSEQUENCEVERIFY DROP HASH160 p2shWithdrawDest EQUAL
ENDIF
```

Specification

Refer to the reference implementation, reproduced below, for the precise semantics and detailed rationale for those semantics.

```
/* Below flags apply in the context of BIP 68 */
/* If this flag set, CTxIn::nSequence is NOT interpreted as a
 * relative lock-time. */
static const uint32_t SEQUENCE_LOCKTIME_DISABLE_FLAG = (1 << 31);

/* If CTxIn::nSequence encodes a relative lock-time and this flag
 * is set, the relative lock-time has units of 512 seconds,
 * otherwise it specifies blocks with a granularity of 1. */
static const uint32_t SEQUENCE_LOCKTIME_TYPE_FLAG = (1 << 22);

/* If CTxIn::nSequence encodes a relative lock-time, this mask is
 * applied to extract that lock-time from the sequence field. */
static const uint32_t SEQUENCE_LOCKTIME_MASK = 0x0000ffff;

case OP_NOP3:
{
    if (!(flags & SCRIPT_VERIFY_CHECKSEQUENCEVERIFY)) {
        // not enabled; treat as a NOP3
        if (flags & SCRIPT_VERIFY_DISCOURAGE_UPGRADABLE_NOPS) {
            return set_error(serror, SCRIPT_ERR_DISCOURAGE_UPGRADABLE_NOPS);
        }
        break;
    }

    if (stack.size() < 1)
       return set_error(serror, SCRIPT_ERR_INVALID_STACK_OPERATION);

    // Note that elsewhere numeric opcodes are limited to
    // operands in the range -2**31+1 to 2**31-1, however it is
    // legal for opcodes to produce results exceeding that
    // range. This limitation is implemented by CScriptNum's
    // default 4-byte limit.
    //
    // Thus as a special case we tell CScriptNum to accept up
    // to 5-byte bignums, which are good until 2**39-1, well
    // beyond the 2**32-1 limit of the nSequence field itself.
    const CScriptNum nSequence(stacktop(-1), fRequireMinimal, 5);

    // In the rare event that the argument may be < 0 due to
    // some arithmetic being done first, you can always use
    // 0 MAX CHECKSEQUENCEVERIFY.
    if (nSequence < 0)
        return set_error(serror, SCRIPT_ERR_NEGATIVE_LOCKTIME);

    // To provide for future soft-fork extensibility, if the
    // operand has the disabled lock-time flag set,
    // CHECKSEQUENCEVERIFY behaves as a NOP.
    if ((nSequence & CTxIn::SEQUENCE_LOCKTIME_DISABLE_FLAG) != 0)
        break;

    // Compare the specified sequence number with the input.
    if (!checker.CheckSequence(nSequence))
        return set_error(serror, SCRIPT_ERR_UNSATISFIED_LOCKTIME);

    break;
```

```
}

bool TransactionSignatureChecker::CheckSequence(const CScriptNum& nSequence) const
{
    // Relative lock times are supported by comparing the passed
    // in operand to the sequence number of the input.
    const int64_t txToSequence = (int64_t)txTo->vin[nIn].nSequence;

    // Fail if the transaction's version number is not set high
    // enough to trigger BIP 68 rules.
    if (static_cast<uint32_t>(txTo->nVersion) < 2)
        return false;

    // Sequence numbers with their most significant bit set are not
    // consensus constrained. Testing that the transaction's sequence
    // number do not have this bit set prevents using this property
    // to get around a CHECKSEQUENCEVERIFY check.
    if (txToSequence & CTxIn::SEQUENCE_LOCKTIME_DISABLE_FLAG)
        return false;

    // Mask off any bits that do not have consensus-enforced meaning
    // before doing the integer comparisons
    const uint32_t nLockTimeMask = CTxIn::SEQUENCE_LOCKTIME_TYPE_FLAG |
CTxIn::SEQUENCE_LOCKTIME_MASK;
    const int64_t txToSequenceMasked = txToSequence & nLockTimeMask;
    const CScriptNum nSequenceMasked = nSequence & nLockTimeMask;

    // There are two kinds of nSequence: lock-by-blockheight
    // and lock-by-blocktime, distinguished by whether
    // nSequenceMasked < CTxIn::SEQUENCE_LOCKTIME_TYPE_FLAG.
    //
    // We want to compare apples to apples, so fail the script
    // unless the type of nSequenceMasked being tested is the same as
    // the nSequenceMasked in the transaction.
    if (!(
        (txToSequenceMasked <  CTxIn::SEQUENCE_LOCKTIME_TYPE_FLAG && nSequenceMasked <
CTxIn::SEQUENCE_LOCKTIME_TYPE_FLAG) ||
        (txToSequenceMasked >= CTxIn::SEQUENCE_LOCKTIME_TYPE_FLAG && nSequenceMasked >=
CTxIn::SEQUENCE_LOCKTIME_TYPE_FLAG)
    ))
        return false;

    // Now that we know we're comparing apples-to-apples, the
    // comparison is a simple numeric one.
    if (nSequenceMasked > txToSequenceMasked)
        return false;

    return true;
}
```

Reference Implementation

A reference implementation is provided by the following pull request:

https://github.com/bitcoin/bitcoin/pull/7524

Deployment

This BIP is to be deployed by "versionbits" BIP9 using bit 0.

For Bitcoin **mainnet**, the BIP9 **starttime** will be midnight 1st May 2016 UTC (Epoch timestamp 1462060800) and BIP9 **timeout** will be midnight 1st May 2017 UTC (Epoch timestamp 1493596800).

For Bitcoin **testnet**, the BIP9 **starttime** will be midnight 1st March 2016 UTC (Epoch timestamp 1456790400) and BIP9 **timeout** will be midnight 1st May 2017 UTC (Epoch timestamp 1493596800).

This BIP must be deployed simultaneously with BIP68 and BIP113 using the same deployment mechanism.

Credits

Mark Friedenbach invented the application of sequence numbers to achieve relative lock-time, and wrote the reference implementation of CHECKSEQUENCEVERIFY.

The reference implementation and this BIP was based heavily on work done by Peter Todd for the closely related BIP 65.

BtcDrak authored this BIP document.

Thanks to Eric Lombrozo and Anthony Towns for contributing example use cases.

References

BIP 9 Versionbits

BIP 68 Relative lock-time through consensus-enforced sequence numbers

BIP 65 OP_CHECKLOCKTIMEVERIFY

BIP 113 Median past block time for time-lock constraints

HTLCs using OP_CHECKSEQUENCEVERIFY/OP_LOCKTIMEVERIFY and revocation hashes

Lightning Network

Deployable Lightning

Scaling Bitcoin to Billions of Transactions Per Day

Softfork deployment considerations

Version bits

Jeremy Spilman Micropayment Channels

Copyright

This document is placed in the public domain.

```
BIP: 113
Layer: Consensus (soft fork)
Title: Median time-past as endpoint for lock-time calculations
Authors: Thomas Kerin <me@thomaskerin.io>
         Mark Friedenbach <mark@friedenbach.org>
Status: Deployed
Type: Specification
Assigned: 2015-08-10
License: PD
```

BIP 113

Abstract

This BIP is a proposal to redefine the semantics used in determining a time-locked transaction's eligibility for inclusion in a block. The median of the last 11 blocks is used instead of the block's timestamp, ensuring that it increases monotonically with each block.

Motivation

At present, transactions are excluded from inclusion in a block if the present time or block height is less than or equal to that specified in the locktime. Since the consensus rules do not mandate strict ordering of block timestamps, this has the unfortunate outcome of creating a perverse incentive for miners to lie about the time of their blocks in order to collect more fees by including transactions that by wall clock determination have not yet matured.

This BIP proposes comparing the locktime against the median of the past 11 block's timestamps, rather than the timestamp of the block including the transaction. Existing consensus rules guarantee this value to monotonically advance, thereby removing the capability for miners to claim more transaction fees by lying about the timestamps of their block.

This proposal seeks to ensure reliable behaviour in locktime calculations as required by BIP65 (CHECKLOCKTIMEVERIFY) and matching the behavior of BIP68 (sequence numbers) and BIP112 (CHECKSEQUENCEVERIFY).

Specification

The values for transaction locktime remain unchanged. The difference is only in the calculation determining whether a transaction can be included. Instead of an unreliable timestamp, the following function is used to determine the current block time for the purpose of checking lock-time constraints:

```
enum { nMedianTimeSpan=11 };

int64_t GetMedianTimePast(const CBlockIndex* pindex)
{
    int64_t pmedian[nMedianTimeSpan];
    int64_t* pbegin = &pmedian[nMedianTimeSpan];
    int64_t* pend = &pmedian[nMedianTimeSpan];
    for (int i = 0; i < nMedianTimeSpan && pindex; i++, pindex = pindex->pprev)
        *(--pbegin) = pindex->GetBlockTime();
```

```
    std::sort(pbegin, pend);
    return pbegin[(pend - pbegin)/2];
}
```

Lock-time constraints are checked by the consensus method IsFinalTx(). This method takes the block time as one parameter. This BIP proposes that after activation calls to IsFinalTx() within consensus code use the return value of `GetMedianTimePast(pindexPrev)` instead.

The new rule applies to all transactions, including the coinbase transaction.

A reference implementation of this proposal is provided by the following pull request:

https://github.com/bitcoin/bitcoin/pull/6566

Deployment

This BIP is to be deployed by "versionbits" BIP9 using bit 0.

For Bitcoin **mainnet**, the BIP9 **starttime** will be midnight 1st May 2016 UTC (Epoch timestamp 1462060800) and BIP9 **timeout** will be midnight 1st May 2017 UTC (Epoch timestamp 1493596800).

For Bitcoin **testnet**, the BIP9 **starttime** will be midnight 1st March 2016 UTC (Epoch timestamp 1456790400) and BIP9 **timeout** will be midnight 1st May 2017 UTC (Epoch timestamp 1493596800).

This BIP must be deployed simultaneously with BIP68 and BIP112 using the same deployment mechanism.

Acknowledgements

Mark Friedenbach for designing and authoring the reference implementation of this BIP.

Thanks go to Gregory Maxwell who came up with the original idea, in #bitcoin-wizards on 2013-07-16.

Thomas Kerin authored this BIP document.

Compatibility

Transactions generated using time-based lock-time will take approximately an hour longer to confirm than would be expected under the old rules. This is not known to introduce any compatibility concerns with existing protocols.

References

BIP9: Versionbits

BIP65: OP_CHECKLOCKTIMEVERIFY

BIP68: Consensus-enforced transaction replacement signaled via sequence numbers

BIP112: CHECKSEQUENCEVERIFY

Softfork deployment considerations

Version bits

Copyright

This document is placed in the public domain.

```
BIP: 141
Layer: Consensus (soft fork)
Title: Segregated Witness (Consensus layer)
Authors: Eric Lombrozo <elombrozo@gmail.com>
         Johnson Lau <jl2012@xbt.hk>
         Pieter Wuille <pieter.wuille@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2015-12-21
License: PD
```

BIP 141

Abstract

This BIP defines a new structure called a "witness" that is committed to blocks separately from the transaction merkle tree. This structure contains data required to check transaction validity but not required to determine transaction effects. In particular, scripts and signatures are moved into this new structure.

The witness is committed in a tree that is nested into the block's existing merkle root via the coinbase transaction for the purpose of making this BIP soft fork compatible. A future hard fork can place this tree in its own branch.

Motivation

The entirety of the transaction's effects are determined by output consumption (spends) and new output creation. Other transaction data, and signatures in particular, are only required to validate the blockchain state, not to determine it.

By removing this data from the transaction structure committed to the transaction merkle tree, several problems are fixed:

1. **Nonintentional malleability becomes impossible**. Since signature data is no longer part of the transaction hash, changes to how the transaction was signed are no longer relevant to transaction identification. As a solution of transaction malleability, this is superior to the canonical signature approach (BIP62):
    - It prevents involuntary transaction malleability for any type of scripts, as long as all inputs are signed (with at least one CHECKSIG or CHECKMULTISIG operation)
    - In the case of an m-of-n CHECKMULTISIG script, a transaction is malleable only with agreement of m private key holders (as opposed to only 1 private key holder with BIP62)
    - It prevents involuntary transaction malleability due to unknown ECDSA signature malleability
    - It allows creation of unconfirmed transaction dependency chains without counterparty risk, an important feature for offchain protocols such as the Lightning Network

2. **Transmission of signature data becomes optional**. It is needed only if a peer is trying to validate a transaction instead of just checking its existence. This reduces the size of SPV proofs and potentially improves the privacy of SPV clients as they can download more transactions using the same bandwidth.
3. **Some constraints could be bypassed with a soft fork** by moving part of the transaction data to a structure unknown to current protocol, for example:
    - Size of witness could be ignored / discounted when calculating the block size, effectively increasing the block size to some extent
    - Hard coded constants, such as maximum data push size (520 bytes) or sigops limit could be reevaluated or removed
    - New script system could be introduced without any limitation from the existing script semantic. For example, a new transaction digest algorithm for transaction signature verification is described in BIP143

Specification

Transaction ID

A new data structure, `witness`, is defined. Each transaction will have 2 IDs.

Definition of `txid` remains unchanged: the double SHA256 of the traditional serialization format:

```
[nVersion][txins][txouts][nLockTime]
```

A new `wtxid` is defined: the double SHA256 of the new serialization with witness data:

```
[nVersion][marker][flag][txins][txouts][witness][nLockTime]
```

Format of `nVersion`, `txins`, `txouts`, and `nLockTime` are same as traditional serialization.

The `marker` MUST be a 1-byte zero value: `0x00`.

The `flag` MUST be a 1-byte non-zero value. Currently, `0x01` MUST be used.

The `witness` is a serialization of all witness fields of the transaction. Each txin is associated with a witness field. A witness field starts with a `var_int` to indicate the number of stack items for the txin. It is followed by stack items, with each item starts with a `var_int` to indicate the length. Witness data is NOT script.

A non-witness program (defined hereinafter) txin MUST be associated with an empty witness field, represented by a `0x00`. If all txins are not witness program, a transaction's `wtxid` is equal to its `txid`.

Commitment structure

A new block rule is added which requires a commitment to the `wtxid`. The `wtxid` of coinbase transaction is assumed to be `0x0000....0000`.

A `witness root hash` is calculated with all those `wtxid` as leaves, in a way similar to the `hashMerkleRoot` in the block header.

The commitment is recorded in a `scriptPubKey` of the coinbase transaction. It must be at least 38 bytes, with the first 6-byte of `0x6a24aa21a9ed`, that is:

```
  1-byte - OP_RETURN (0x6a)
  1-byte - Push the following 36 bytes (0x24)
  4-byte - Commitment header (0xaa21a9ed)
 32-byte - Commitment hash: Double-SHA256(witness root hash|witness reserved value)

 39th byte onwards: Optional data with no consensus meaning
```

and the coinbase's input's witness must consist of a single 32-byte array for the `witness reserved value`.

If there are more than one `scriptPubKey` matching the pattern, the one with highest output index is assumed to be the commitment.

If all transactions in a block do not have witness data, the commitment is optional.

Witness program

A `scriptPubKey` (or `redeemScript` as defined in BIP16/P2SH) that consists of a 1-byte push opcode (one of `OP_0,OP_1,OP_2,...,OP_16`) followed by a direct data push between 2 and 40 bytes gets a new special meaning. The value of the first push is called the "version byte". The following byte vector pushed is called the "witness program". In more detail, this means a `scriptPubKey` or `redeemScript` which consists of (in order):

- First, byte 0x00 (`OP_0`) or any byte between 0x51 (`OP_1`) and 0x60 (`OP_16`) inclusive (the version byte).
- Then, a byte *L* between 0x02 (push of 2 bytes) and 0x28 (push of 40 bytes) inclusive.
- Finally, *L* arbitrary bytes (the witness program).

There are two cases in which witness validation logic are triggered. Each case determines the location of the witness version byte and program, as well as the form of the scriptSig:

1. Triggered by a `scriptPubKey` that is exactly a push of a version byte, plus a push of a witness program. The scriptSig must be exactly empty or validation fails. (*"native witness program"*)
2. Triggered when a `scriptPubKey` is a P2SH script, and the BIP16 `redeemScript` pushed in the `scriptSig` is exactly a push of a version byte plus a push of a witness program. The `scriptSig` must be exactly a push of the BIP16 `redeemScript` or validation fails. (*"P2SH witness program"*)

If the version byte is 0, and the witness program is 20 bytes ($L = 20$):

- It is interpreted as a pay-to-witness-public-key-hash (P2WPKH) program.
- The witness must consist of exactly 2 items (≤ 520 bytes each). The first one a signature, and the second one a public key.
- The HASH160 of the public key must match the 20-byte witness program.
- After normal script evaluation, the signature is verified against the public key with CHECKSIG operation. The verification must result in a single TRUE on the stack.

If the version byte is 0, and the witness program is 32 bytes ($L = 32$):

- It is interpreted as a pay-to-witness-script-hash (P2WSH) program.
- The witness must consist of an input stack to feed to the script, followed by a serialized script (`witnessScript`).
- The `witnessScript` (≤ 10,000 bytes) is popped off the initial witness stack. SHA256 of the `witnessScript` must match the 32-byte witness program.

- The `witnessScript` is deserialized, and executed after normal script evaluation with the remaining witness stack (≤ 520 bytes for each stack item).
- The script must not fail, and result in exactly a single TRUE on the stack.

If the version byte is 0, but the witness program is neither 20 nor 32 bytes, the script must fail.For example, a scriptPubKey with OP_0 followed by a 40-byte non-zero data push will fail due to incorrect program size. However, a scriptPubKey with OP_0 followed by a 41-byte non-zero data push will pass, since it is not considered to be a witness program

If the version byte is 1 to 16, no further interpretation of the witness program or witness stack happens, and there is no size restriction for the witness stack. These versions are reserved for future extensions.For backward compatibility, for any version byte from 0 to 16, the script must fail if the witness program has a `CastToBool` value of zero. However, having a hash like this is a successful preimage attack against the hash function, and the risk is negligible.

Other consensus critical limits

Block size

Blocks are currently limited to 1,000,000 bytes (1MB) total size. We change this restriction as follows:

Block weight is defined as *Base size* * 3 + *Total size*. (rationaleRationale of using a single composite constraint, instead of two separate limits such as 1MB base data and 3MB witness data: Using two separate limits would make mining and fee estimation nearly impossible. Miners would need to solve a complex non-linear optimization problem to find the set of transactions that maximize fees given both constraints, and wallets would not be able to know what to pay as it depends on which of the two conditions is most constrained by the time miners try to produce blocks with their transactions in. Another problem with such an approach is freeloading. Once a set of transactions hit the base data 1MB constraint, up to 3MB extra data could be added to the witness by just minimally increasing the fee. The marginal cost for extra witness space effectively becomes zero in that case.)

Base size is the block size in bytes with the original transaction serialization without any witness-related data, as seen by a non-upgraded node.

Total size is the block size in bytes with transactions serialized as described in BIP144, including base data and witness data.

The new rule is *block weight* ≤ 4,000,000.

Sigops

Sigops per block is currently limited to 20,000. We change this restriction as follows:

Sigops in the current pubkey script, signature script, and P2SH check script are counted at 4 times their previous value. The sigop limit is likewise quadrupled to ≤ 80,000.

Each P2WPKH input is counted as 1 sigop. In addition, opcodes within a P2WSH `witnessScript` are counted identically as previously within the P2SH `redeemScript`. That is, CHECKSIG is counted as only 1 sigop. When preceded by OP_1 to OP_16 CHECKMULTISIG is counted as 1 to 16 sigops respectively, otherwise it is counted as 20 sigops. This rule applies to both native witness program and P2SH witness program.

Additional definitions

The following definitions are not used for consensus limits, but are suggested to provide language consistent with the terminology introduced above.

Transaction size calculations

Transaction weight is defined as *Base transaction size* * 3 + *Total transaction size* (ie. the same method as calculating *Block weight* from *Base size* and *Total size*).

Virtual transaction size is defined as *Transaction weight* / 4 (rounded up to the next integer).

Base transaction size is the size of the transaction serialised with the witness data stripped.

Total transaction size is the transaction size in bytes serialized as described in BIP144, including base data and witness data.

New script semantics

Despite that the script language for P2WPKH and P2WSH looks very similar to pre-segregated witness script, there are several notable differences. Users MUST NOT assume that a script spendable in pre-segregated witness system would also be spendable as a P2WPKH or P2WSH script. Before large-scale deployment in the production network, developers should test the scripts on testnet with the default relay policy turned on, and with a small amount of money after BIP141 is activated on mainnet.

A major difference at consensus level is described in BIP143, as a new transaction digest algorithm for signature verification in version 0 witness program.

Three relay and mining policies are also included in the first release of segregated witness at reference implementation version 0.13.1. Softforks based on these policies are likely to be proposed in the near future. To avoid indefinite delay in transaction confirmation and permanent fund loss in a potential softfork, users MUST observe the new semantics carefully:

1. Only compressed public keys are accepted in P2WPKH and P2WSH (See BIP143)
2. The argument of OP_IF/NOTIF in P2WSH must be minimalhttps://lists.linuxfoundation.org/pipermail/bitcoin-dev/2016-August/013014.html
3. Signature(s) must be null vector(s) if an OP_CHECKSIG or OP_CHECKMULTISIG is failed (for both pre-segregated witness script and P2WSH. See BIP146)

Examples

P2WPKH

The following example is a version 0 pay-to-witness-public-key-hash (P2WPKH):

```
witness:      <signature> <pubkey>
scriptSig:    (empty)
scriptPubKey: 0 <20-byte-key-hash>
              (0x0014{20-byte-key-hash})
```

The '0' in scriptPubKey indicates the following push is a version 0 witness program. The length of the witness program indicates that it is a P2WPKH type. The witness must consist of exactly 2 items. The HASH160 of the pubkey in witness must match the witness program.

The signature is verified as

```
<signature> <pubkey> CHECKSIG
```

Comparing with a traditional P2PKH output, the P2WPKH equivalent occupies 3 less bytes in the scriptPubKey, and moves the signature and public key from scriptSig to witness.

P2WPKH nested in BIP16 P2SH

The following example is the same P2WPKH, but nested in a BIP16 P2SH output.

```
witness:      <signature> <pubkey>
scriptSig:    <0 <20-byte-key-hash>>
              (0x160014{20-byte-key-hash})
scriptPubKey: HASH160 <20-byte-script-hash> EQUAL
              (0xA914{20-byte-script-hash}87)
```

The only item in scriptSig is hashed with HASH160, compared against the 20-byte-script-hash in scriptPubKey, and interpreted as:

```
0 <20-byte-key-hash>
```

The public key and signature are then verified as described in the previous example.

Comparing with the previous example, the scriptPubKey is 1 byte bigger and the scriptSig is 23 bytes bigger. Although a nested witness program is less efficient, its payment address is fully transparent and backward compatible for all Bitcoin reference client since version 0.6.0.

P2WSH

The following example is an 1-of-2 multi-signature version 0 pay-to-witness-script-hash (P2WSH).

```
witness:      0 <signature1> <1 <pubkey1> <pubkey2> 2 CHECKMULTISIG>
scriptSig:    (empty)
scriptPubKey: 0 <32-byte-hash>
              (0x0020{32-byte-hash})
```

The '0' in scriptPubKey indicates the following push is a version 0 witness program. The length of the witness program indicates that it is a P2WSH type. The last item in the witness (the "witnessScript") is popped off, hashed with SHA256, compared against the 32-byte-hash in scriptPubKey, and deserialized:

```
1 <pubkey1> <pubkey2> 2 CHECKMULTISIG
```

The script is executed with the remaining data from witness:

```
0 <signature1> 1 <pubkey1> <pubkey2> 2 CHECKMULTISIG
```

P2WSH allows maximum script size of 10,000 bytes, as the 520-byte push limit is bypassed.

The scriptPubKey occupies 34 bytes, as opposed to 23 bytes of BIP16 P2SH. The increased size improves security against possible collision attacks, as 2^{80} work is not infeasible anymore (By the end of 2015, 2^{84} hashes have been calculated in Bitcoin mining since the creation of Bitcoin). The spending script is same as the one for an equivalent BIP16 P2SH output but is moved to witness.

P2WSH nested in BIP16 P2SH

The following example is the same 1-of-2 multi-signature P2WSH script, but nested in a BIP16 P2SH output.

```
witness:      0 <signature1> <1 <pubkey1> <pubkey2> 2 CHECKMULTISIG>
scriptSig:    <0 <32-byte-hash>>
              (0x220020{32-byte-hash})
scriptPubKey: HASH160 <20-byte-hash> EQUAL
              (0xA914{20-byte-hash}87)
```

The only item in scriptSig is hashed with HASH160, compared against the 20-byte-hash in scriptPubKey, and interpreted as:

```
0 <32-byte-hash>
```

The P2WSH witnessScript is then executed as described in the previous example.

Comparing with the previous example, the scriptPubKey is 11 bytes smaller (with reduced security) while witness is the same. However, it also requires 35 bytes in scriptSig.

Extensible commitment structure

The new commitment in coinbase transaction is a hash of the `witness root hash` and a `witness reserved value`. The `witness reserved value` currently has no consensus meaning, but in the future allows new commitment values for future softforks. For example, if a new consensus-critical commitment is required in the future, the commitment in coinbase becomes:

```
Double-SHA256(Witness root hash|Hash(new commitment|witness reserved value))
```

For backward compatibility, the `Hash(new commitment|witness reserved value)` will go to the coinbase witness, and the `witness reserved value` will be recorded in another location specified by the future softfork. Any number of new commitment could be added in this way.

Any commitments that are not consensus-critical to Bitcoin, such as merge-mining, MUST NOT use the `witness reserved value` to preserve the ability to do upgrades of the Bitcoin consensus protocol.

The optional data space following the commitment also leaves room for metadata of future softforks, and MUST NOT be used for other purpose.

Trust-free unconfirmed transaction dependency chain

Segregated witness fixes the problem of transaction malleability fundamentally, which enables the building of unconfirmed transaction dependency chains in a trust-free manner.

Two parties, Alice and Bob, may agree to send certain amount of Bitcoin to a 2-of-2 multisig output (the "funding transaction"). Without signing the funding transaction, they may create another transaction, time-locked in the future, spending the 2-of-2 multisig output to third account(s) (the "spending transaction"). Alice and Bob will sign the spending transaction and exchange the signatures. After examining the signatures, they will sign and commit the funding transaction to the blockchain. Without further action, the spending transaction will be confirmed after the lock-time and release the funding according to the original contract. It also retains the flexibility of revoking the original contract before the lock-time, by another spending transaction with shorter lock-time, but only with mutual-agreement of both parties.

Such setups are not possible with BIP62 as the malleability fix, since the spending transaction could not be created without both parties first signing the funding transaction. If Alice reveals the funding transaction signature before Bob does, Bob is able to lock up the funding indefinitely without ever signing the spending transaction.

Unconfirmed transaction dependency chain is a fundamental building block of more sophisticated payment networks, such as duplex micropayment channel and the Lightning Network, which have the potential to greatly improve the scalability and efficiency of the Bitcoin system.

Future extensions

Compact fraud proof for SPV nodes

Bitcoin right now only has two real security models. A user either runs a full-node which validates every block with all rules in the system, or a SPV (Simple Payment Verification) client which only validates the headers as a proof of publication of some transactions. The Bitcoin whitepaper suggested that SPV nodes may accept alerts from full nodes when they detect an invalid block, prompting the SPV node to download the questioned blocks and transactions for validation. This approach, however, could become a DoS attack vector as there is virtually no cost to generate a false alarm. An alarm must come with a compact, yet deterministic fraud proof.

In the current Bitcoin protocol, it is possible to generate compact fraud proof for almost all rules except a few:

1. It is not possible to prove a miner has introduced too many Bitcoins in the coinbase transaction outputs without showing the whole block itself and all input transactions.
2. It is not possible to prove the violation of any block specific constraints, such as size and sigop limits, without showing the whole block (and all input transactions in the case of sigop limit)
3. It is not possible to prove the spending of a non-existing input without showing all transaction IDs in the blockchain way back to the genesis block.

Extra witness data can be committed that allows short proofs of block invalidity that SPV nodes can quickly verify:

1. Sum trees for transaction fee can be committed making it possible to construct short proofs that the miner does not add excessive fees to the coinbase transaction. Similar for the block size and sigop count limit.
2. Backlinks for the outputs spent by the transaction's inputs can be provided. These backlinks consist of a block hash and an offset that thin clients can easily query and check to verify that the outputs exist.

These commitments could be included in the extensible commitment structure through a soft fork and will be transparent to nodes that do not understand such new rules.

New script system

Since a version byte is pushed before a witness program, and programs with unknown versions are always considered as anyone-can-spend script, it is possible to introduce any new script system with a soft fork. The witness as a structure is not restricted by any existing script semantics and constraints, the 520-byte push limit in particular, and therefore allows arbitrarily large scripts and signatures.

Examples of new script systems include Schnorr signatures, which reduce the size of multisig transactions dramatically; Lamport signatures, which are quantum computing resistant; and Merklized abstract syntax trees, which allow very compact witnesses for conditional scripts with extreme complexity.

Per-input lock-time and relative-lock-time

Currently there is only one nLockTime field in a transaction and all inputs must share the same value. BIP68 enables per-input relative-lock-time using the nSequence field, however, with a limited lock-time period and resolution.

With a soft fork, it is possible to introduce a separate witness structure to allow per-input lock-time and relative-lock-time, and a new script system that could sign and manipulate the new data (like BIP65 and BIP112).

Backward compatibility

As a soft fork, older software will continue to operate without modification. Non-upgraded nodes, however, will not see nor validate the witness data and will consider all witness programs as anyone-can-spend scripts (except a few edge cases where the witness programs are equal to 0, which the script must fail). Wallets should always be wary of anyone-can-spend scripts and treat them with suspicion. Non-upgraded nodes are strongly encouraged to upgrade in order to take advantage of the new features.

What a non-upgraded wallet can do

- Receiving bitcoin from non-upgraded and upgraded wallets
- Sending bitcoin to non-upgraded and upgraded wallets with traditional P2PKH address (without any benefit of segregated witness)
- Sending bitcoin to upgraded wallets using a P2SH address
- Sending bitcoin to upgraded wallets using a native witness program through BIP70 payment protocol

What a non-upgraded wallet cannot do

- Validating segregated witness transaction. It assumes such a transaction is always valid

Deployment

This BIP will be deployed by “version bits” BIP9 with the name “segwit” and using bit 1.

For Bitcoin mainnet, the BIP9 starttime will be midnight 15 November 2016 UTC (Epoch timestamp 1479168000) and BIP9 timeout will be midnight 15 November 2017 UTC (Epoch timestamp 1510704000).

For Bitcoin testnet, the BIP9 starttime will be midnight 1 May 2016 UTC (Epoch timestamp 1462060800) and BIP9 timeout will be midnight 1 May 2017 UTC (Epoch timestamp 1493596800).

Credits

Special thanks to Gregory Maxwell for originating many of the ideas in this BIP and Luke-Jr for figuring out how to deploy this as a soft fork.

Footnotes

Reference Implementation

https://github.com/bitcoin/bitcoin/pull/8149

References

- BIP16 Pay to Script Hash
- BIP143 Transaction Signature Verification for Version 0 Witness Program
- BIP144 Segregated Witness (Peer Services)
- BIP173 Base32 address format for native v0-16 witness outputs

Copyright

This document is placed in the public domain.

```
BIP: 143
Layer: Consensus (soft fork)
Title: Transaction Signature Verification for Version 0 Witness Program
Authors: Johnson Lau <jl2012@xbt.hk>
         Pieter Wuille <pieter.wuille@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2016-01-03
License: PD
```

BIP 143

Abstract

This proposal defines a new transaction digest algorithm for signature verification in version 0 witness program, in order to minimize redundant data hashing in verification, and to cover the input value by the signature.

Motivation

There are 4 ECDSA signature verification codes in the original Bitcoin script system: `CHECKSIG`, `CHECKSIGVERIFY`, `CHECKMULTISIG`, `CHECKMULTISIGVERIFY` ("sigops"). According to the sighash type (`ALL`, `NONE`, `SINGLE`, `ANYONECANPAY`), a transaction digest is generated with a double SHA256 of a serialized subset of the transaction, and the signature is verified against this digest with a given public key. The detailed procedure is described in a Bitcoin Wiki article. 1

Unfortunately, there are at least 2 weaknesses in the original SignatureHash transaction digest algorithm:

- For the verification of each signature, the amount of data hashing is proportional to the size of the transaction. Therefore, data hashing grows in $O(n^2)$ as the number of sigops in a transaction increases. While a 1 MB block would normally take 2 seconds to verify with an average computer in 2015, a 1MB transaction with 5569 sigops may take 25 seconds to verify. This could be fixed by optimizing the digest algorithm by introducing some reusable "midstate", so the time complexity becomes O(n). CVE-2013-2292New Bitcoin vulnerability: A transaction that takes at least 3 minutes to verifyThe Megatransaction: Why Does It Take 25 Seconds?
- The algorithm does not involve the amount of Bitcoin being spent by the input. This is usually not a problem for online network nodes as they could request for the specified transaction to acquire the output value. For an offline transaction signing device ("cold wallet"), however, the unknowing of input amount makes it impossible to calculate the exact amount being spent and the transaction fee. To cope with this problem a cold wallet must also acquire the full transaction being spent, which could be a big obstacle in the implementation of lightweight, air-gapped wallet. By including the input value of part of the transaction digest, a cold wallet may safely sign a transaction by learning the value from an untrusted source. In the case that a wrong value is provided and signed, the signature would be invalid and no funding might be lost. SIGHASH_WITHINPUTVALUE: Super-lightweight HW wallets and offline data

Deploying the aforementioned fixes in the original script system is not a simple task. That would be either a hardfork, or a softfork for new sigops without the ability to remove or insert stack items. However, the introduction of segregated witness softfork offers an opportunity to define a different set of script semantics without disrupting the original system, as the unupgraded nodes would always consider such a transaction output is spendable by arbitrary signature or no signature at all. BIP141: Segregated Witness (Consensus layer)

Specification

A new transaction digest algorithm is defined, but only applicable to sigops in version 0 witness program:

```
Double SHA256 of the serialization of:
   1. nVersion of the transaction (4-byte little endian)
   2. hashPrevouts (32-byte hash)
   3. hashSequence (32-byte hash)
   4. outpoint (32-byte hash + 4-byte little endian)
   5. scriptCode of the input (serialized as scripts inside CTxOuts)
   6. value of the output spent by this input (8-byte little endian)
   7. nSequence of the input (4-byte little endian)
   8. hashOutputs (32-byte hash)
   9. nLocktime of the transaction (4-byte little endian)
  10. sighash type of the signature (4-byte little endian)
```

Semantics of the original sighash types remain unchanged, except the following:

1. The way of serialization is changed;
2. All sighash types commit to the amount being spent by the signed input;
3. `FindAndDelete` of the signature is not applied to the `scriptCode`;

4. OP_CODESEPARATOR(s) after the last executed OP_CODESEPARATOR are not removed from the scriptCode (the last executed OP_CODESEPARATOR and any script before it are always removed);
5. SINGLE does not commit to the input index. When ANYONECANPAY is not set, the semantics are unchanged since hashPrevouts and outpoint together implicitly commit to the input index. When SINGLE is used with ANYONECANPAY, omission of the index commitment allows permutation of the input-output pairs, as long as each pair is located at an equivalent index.

The items 1, 4, 7, 9, 10 have the same meaning as the original algorithm.

The item 5:

- For P2WPKH witness program, the scriptCode is 0x1976a914{20-byte-pubkey-hash}88ac.
- For P2WSH witness program,
    - if the witnessScript does not contain any OP_CODESEPARATOR, the scriptCode is the witnessScript serialized as scripts inside CTxOut.
    - if the witnessScript contains any OP_CODESEPARATOR, the scriptCode is the witnessScript but removing everything up to and including the last executed OP_CODESEPARATOR before the signature checking opcode being executed, serialized as scripts inside CTxOut. (The exact semantics is demonstrated in the examples below)

The item 6 is a 8-byte value of the amount of bitcoin spent in this input.

hashPrevouts:

- If the ANYONECANPAY flag is not set, hashPrevouts is the double SHA256 of the serialization of all input outpoints;
- Otherwise, hashPrevouts is a uint256 of 0x0000......0000.

hashSequence:

- If none of the ANYONECANPAY, SINGLE, NONE sighash type is set, hashSequence is the double SHA256 of the serialization of nSequence of all inputs;
- Otherwise, hashSequence is a uint256 of 0x0000......0000.

hashOutputs:

- If the sighash type is neither SINGLE nor NONE, hashOutputs is the double SHA256 of the serialization of all output amount (8-byte little endian) with scriptPubKey (serialized as scripts inside CTxOuts);
- If sighash type is SINGLE and the input index is smaller than the number of outputs, hashOutputs is the double SHA256 of the output amount with scriptPubKey of the same index as the input;
- Otherwise, hashOutputs is a uint256 of 0x0000......0000.In the original algorithm, a uint256 of 0x0000......0001 is committed if the input index for a SINGLE signature is greater than or equal to the number of outputs. In this BIP a 0x0000......0000 is committed, without changing the semantics.

The hashPrevouts, hashSequence, and hashOutputs calculated in an earlier verification may be reused in other inputs of the same transaction, so that the time complexity of the whole hashing process reduces from $O(n^2)$ to $O(n)$.

Refer to the reference implementation, reproduced below, for the precise algorithm:

```
uint256 hashPrevouts;
uint256 hashSequence;
uint256 hashOutputs;

if (!(nHashType & SIGHASH_ANYONECANPAY)) {
    CHashWriter ss(SER_GETHASH, 0);
    for (unsigned int n = 0; n < txTo.vin.size(); n++) {
        ss << txTo.vin[n].prevout;
    }
    hashPrevouts = ss.GetHash();
}

if (!(nHashType & SIGHASH_ANYONECANPAY) && (nHashType & 0x1f) != SIGHASH_SINGLE && (nHashType
      & 0x1f) != SIGHASH_NONE) {
    CHashWriter ss(SER_GETHASH, 0);
    for (unsigned int n = 0; n < txTo.vin.size(); n++) {
        ss << txTo.vin[n].nSequence;
    }
    hashSequence = ss.GetHash();
}

if ((nHashType & 0x1f) != SIGHASH_SINGLE && (nHashType & 0x1f) != SIGHASH_NONE) {
    CHashWriter ss(SER_GETHASH, 0);
    for (unsigned int n = 0; n < txTo.vout.size(); n++) {
        ss << txTo.vout[n];
    }
    hashOutputs = ss.GetHash();
} else if ((nHashType & 0x1f) == SIGHASH_SINGLE && nIn < txTo.vout.size()) {
    CHashWriter ss(SER_GETHASH, 0);
    ss << txTo.vout[nIn];
    hashOutputs = ss.GetHash();
}

CHashWriter ss(SER_GETHASH, 0);
// Version
ss << txTo.nVersion;
// Input prevouts/nSequence (none/all, depending on flags)
ss << hashPrevouts;
ss << hashSequence;
// The input being signed (replacing the scriptSig with scriptCode + amount)
// The prevout may already be contained in hashPrevout, and the nSequence
// may already be contained in hashSequence.
ss << txTo.vin[nIn].prevout;
ss << static_cast<const CScriptBase&>(scriptCode);
ss << amount;
ss << txTo.vin[nIn].nSequence;
// Outputs (none/one/all, depending on flags)
ss << hashOutputs;
// Locktime
ss << txTo.nLockTime;
// Sighash type
ss << nHashType;

return ss.GetHash();
```

Restrictions on public key type

As a default policy, only compressed public keys are accepted in `P2WPKH` and `P2WSH`. Each public key passed to a sigop inside version 0 witness program must be a compressed key: the first byte MUST be either `0x02` or `0x03`, and the size MUST be 33 bytes. Transactions that break this rule will not be relayed or mined by default.

Since this policy is preparation for a future softfork proposal, to avoid potential future funds loss, users MUST NOT use uncompressed keys in version 0 witness programs.

Example

To ensure consistency in consensus-critical behaviour, developers should test their implementations against all the tests below. More tests related to this proposal could be found under https://github.com/bitcoin/bitcoin/tree/master/src/test/data .

Native P2WPKH

```
The following is an unsigned transaction:
  0100000002fff7f7881a8099afa6940d42d1e7f6362bec38171ea3edf433541db4e4ad969f0000000000eefffffffef51e1b804cc8

  nVersion:  01000000
  txin:      02 fff7f7881a8099afa6940d42d1e7f6362bec38171ea3edf433541db4e4ad969f 00000000 00 eefffffff
                ef51e1b804cc89d182d279655c3aa89e815b1b309fe287d9b2b55d57b90ec68a 01000000 00 ffffffff
  txout:     02 202cb20600000000 1976a9148280b37df378db99f66f85c95a783a76ac7a6d5988ac
                9093510d00000000 1976a9143bde42dbee7e4dbe6a21b2d50ce2f0167faa815988ac
  nLockTime: 11000000

The first input comes from an ordinary P2PK:
  scriptPubKey : 2103c9f4836b9a4f77fc0d81f7bcb01b7f1b35916864b9476c241ce9fc198bd25432ac value: 6.25
  private key  : bbc27228ddcb9209d7fd6f36b02f7dfa6252af40bb2f1cbc7a557da8027ff866

The second input comes from a P2WPKH witness program:
  scriptPubKey : 00141d0f172a0ecb48aee1be1f2687d2963ae33f71a1, value: 6
  private key  : 619c335025c7f4012e556c2a58b2506e30b8511b53ade95ea316fd8c3286feb9
  public key   : 025476c2e83188368da1ff3e292e7acafcdb3566bb0ad253f62fc70f07aeee6357

To sign it with a nHashType of 1 (SIGHASH_ALL):

hashPrevouts:
  dSHA256(fff7f7881a8099afa6940d42d1e7f6362bec38171ea3edf433541db4e4ad969f00000000ef51e1b804cc89d182d279655
= 96b827c8483d4e9b96712b6713a7b68d6e8003a781feba36c31143470b4efd37

hashSequence:
  dSHA256(eeffffffffffffff)
= 52b0a642eea2fb7ae638c36f6252b6750293dbe574a806984b8e4d8548339a3b

hashOutputs:
  dSHA256(202cb206000000001976a9148280b37df378db99f66f85c95a783a76ac7a6d5988ac9093510d000000001976a9143bde4
= 863ef3e1a92afbfdb97f31ad0fc7683ee943e9abcf2501590ff8f6551f47e5e5
```

```
hash preimage: 0100000096b827c8483d4e9b96712b6713a7b68d6e8003a781feba36c31143470b4efd3752b0a642eea2fb7ae638

  nVersion:     01000000
  hashPrevouts: 96b827c8483d4e9b96712b6713a7b68d6e8003a781feba36c31143470b4efd37
  hashSequence: 52b0a642eea2fb7ae638c36f6252b6750293dbe574a806984b8e4d8548339a3b
  outpoint:     ef51e1b804cc89d182d279655c3aa89e815b1b309fe287d9b2b55d57b90ec68a01000000
  scriptCode:   1976a9141d0f172a0ecb48aee1be1f2687d2963ae33f71a188ac
  amount:       0046c32300000000
  nSequence:    ffffffff
  hashOutputs:  863ef3e1a92afbfdb97f31ad0fc7683ee943e9abcf2501590ff8f6551f47e5e5
  nLockTime:    11000000
  nHashType:    01000000

sigHash:      c37af31116d1b27caf68aae9e3ac82f1477929014d5b917657d0eb49478cb670
signature:    304402203609e17b84f6a7d30c80bfa610b5b4542f32a8a0d5447a12fb1366d7f01cc44a0220573a954c451833156

The serialized signed transaction is: 01000000000102fff7f7881a8099afa6940d42d1e7f6362bec38171ea3edf433541db

  nVersion:  01000000
  marker:    00
  flag:      01
  txin:      02 fff7f7881a8099afa6940d42d1e7f6362bec38171ea3edf433541db4e4ad969f 00000000 494830450221008b9
                ef51e1b804cc89d182d279655c3aa89e815b1b309fe287d9b2b55d57b90ec68a 01000000 00 ffffffff
  txout:     02 202cb20600000000 1976a9148280b37df378db99f66f85c95a783a76ac7a6d5988ac
                9093510d00000000 1976a9143bde42dbee7e4dbe6a21b2d50ce2f0167faa815988ac
  witness    00
             02 47304402203609e17b84f6a7d30c80bfa610b5b4542f32a8a0d5447a12fb1366d7f01cc44a0220573a954c45183
  nLockTime: 11000000
```

P2SH-P2WPKH

```
The following is an unsigned transaction: 0100000001db6b1b20aa0fd7b23880be2ecbd4a98130974cf4748fb66092ac4d3

  nVersion:  01000000
  txin:      01 db6b1b20aa0fd7b23880be2ecbd4a98130974cf4748fb66092ac4d3ceb1a5477 01000000 00 fefffffff
  txout:     02 b8b4eb0b00000000 1976a914a457b684d7f0d539a46a45bbc043f35b59d0d96388ac
                0008af2f00000000 1976a914fd270b1ee6abcaea97fea7ad0402e8bd8ad6d77c88ac
  nLockTime: 92040000

The input comes from a P2SH-P2WPKH witness program:
  scriptPubKey : a9144733f37cf4db86fbc2efed2500b4f4e49f31202387, value: 10
  redeemScript : 001479091972186c449eb1ded22b78e40d009bdf0089
  private key  : eb696a065ef48a2192da5b28b694f87544b30fae8327c4510137a922f32c6dcf
  public key   : 03ad1d8e89212f0b92c74d23bb710c00662ad1470198ac48c43f7d6f93a2a26873

To sign it with a nHashType of 1 (SIGHASH_ALL):
```

```
hashPrevouts:
  dSHA256(db6b1b20aa0fd7b23880be2ecbd4a98130974cf4748fb66092ac4d3ceb1a547701000000)
= b0287b4a252ac05af83d2dcef00ba313af78a3e9c329afa216eb3aa2a7b4613a

hashSequence:
  dSHA256(fefffff)
= 18606b350cd8bf565266bc352f0caddcf01e8fa789dd8a15386327cf8cabe198

hashOutputs:
  dSHA256(b8b4eb0b000000001976a914a457b684d7f0d539a46a45bbc043f35b59d0d96388ac0008af2f000000001976a914fd270
= de984f44532e2173ca0d64314fcefe6d30da6f8cf27bafa706da61df8a226c83

hash preimage: 01000000b0287b4a252ac05af83d2dcef00ba313af78a3e9c329afa216eb3aa2a7b4613a18606b350cd8bf565266

  nVersion:     01000000
  hashPrevouts: b0287b4a252ac05af83d2dcef00ba313af78a3e9c329afa216eb3aa2a7b4613a
  hashSequence: 18606b350cd8bf565266bc352f0caddcf01e8fa789dd8a15386327cf8cabe198
  outpoint:     db6b1b20aa0fd7b23880be2ecbd4a98130974cf4748fb66092ac4d3ceb1a547701000000
  scriptCode:   1976a91479091972186c449eb1ded22b78e40d009bdf008988ac
  amount:       00ca9a3b00000000
  nSequence:    fefffff
  hashOutputs:  de984f44532e2173ca0d64314fcefe6d30da6f8cf27bafa706da61df8a226c83
  nLockTime:    92040000
  nHashType:    01000000

sigHash:      64f3b0f4dd2bb3aa1ce8566d220cc74dda9df97d8490cc81d89d735c92e59fb6
signature:    3044022047ac8e878352d3ebbde1c94ce3a10d057c24175747116f8288e5d794d12d482f0220217f36a485cae903c

The serialized signed transaction is: 01000000000101db6b1b20aa0fd7b23880be2ecbd4a98130974cf4748fb66092ac4d3
  nVersion:  01000000
  marker:    00
  flag:      01
  txin:      01 db6b1b20aa0fd7b23880be2ecbd4a98130974cf4748fb66092ac4d3ceb1a5477 01000000 17160014790919721
  txout:     02 b8b4eb0b00000000 1976a914a457b684d7f0d539a46a45bbc043f35b59d0d96388ac
                0008af2f00000000 1976a914fd270b1ee6abcaea97fea7ad0402e8bd8ad6d77c88ac
  witness    02 473044022047ac8e878352d3ebbde1c94ce3a10d057c24175747116f8288e5d794d12d482f0220217f36a485cae
  nLockTime: 92040000
```

Native P2WSH

This example shows how `OP_CODESEPARATOR` and out-of-range `SIGHASH_SINGLE` are processed:

```
The following is an unsigned transaction:
  0100000002fe3dc9208094f3ffd12645477b3dc56f60ec4fa8e6f5d67c565d1c6b9216b36e0000000000ffffffff0815cf020f013

  nVersion:  01000000
  txin:      02 fe3dc9208094f3ffd12645477b3dc56f60ec4fa8e6f5d67c565d1c6b9216b36e 00000000 00 ffffffff
                0815cf020f013ed6cf91d29f4202e8a58726b1ac6c79da47c23d1bee0a6925f8 00000000 00 ffffffff
```

```
  txout:     01 00f2052a01000000 1976a914a30741f8145e5acadf23f751864167f32e0963f788ac
  nLockTime: 00000000

The first input comes from an ordinary P2PK:
  scriptPubKey: 21036d5c20fa14fb2f635474c1dc4ef5909d4568e5569b79fc94d3448486e14685f8ac value: 1.5625
  private key:  b8f28a772fccbf9b4f58a4f027e07dc2e35e7cd80529975e292ea34f84c4580c
  signature:    304402200af4e47c9b9629dbecc21f73af989bdaa911f7e6f6c2e9394588a3aa68f81e9902204f3fcf6ade7e5ab

The second input comes from a native P2WSH witness program:
  scriptPubKey : 00205d1b56b63d714eebe542309525f484b7e9d6f686b3781b6f61ef925d66d6f6a0, value: 49
  witnessScript: 21026dccc749adc2a9d0d89497ac511f760f45c47dc5ed9cf352a58ac706453880aeadab210255a9626aebf5e2
                 <026dccc749adc2a9d0d89497ac511f760f45c47dc5ed9cf352a58ac706453880ae> CHECKSIGVERIFY CODESE

To sign it with a nHashType of 3 (SIGHASH_SINGLE):

hashPrevouts:
  dSHA256(fe3dc9208094f3ffd12645477b3dc56f60ec4fa8e6f5d67c565d1c6b9216b36e000000000815cf020f013ed6cf91d29f4
= ef546acf4a020de3898d1b8956176bb507e6211b5ed3619cd08b6ea7e2a09d41

  nVersion:     01000000
  hashPrevouts: ef546acf4a020de3898d1b8956176bb507e6211b5ed3619cd08b6ea7e2a09d41
  hashSequence: 0000000000000000000000000000000000000000000000000000000000000000
  outpoint:     0815cf020f013ed6cf91d29f4202e8a58726b1ac6c79da47c23d1bee0a6925f800000000
  scriptCode:   (see below)
  amount:       0011102401000000
  nSequence:    ffffffff
  hashOutputs:  0000000000000000000000000000000000000000000000000000000000000000 (this is the second input
  nLockTime:    00000000
  nHashType:    03000000

scriptCode:  4721026dccc749adc2a9d0d89497ac511f760f45c47dc5ed9cf352a58ac706453880aeadab210255a9626aebf5e29c
                                                                                ^^
             (please note that the not-yet-
executed OP_CODESEPARATOR is not removed from the scriptCode)
preimage:    01000000ef546acf4a020de3898d1b8956176bb507e6211b5ed3619cd08b6ea7e2a09d410000000000000000000000
sigHash:     82dde6e4f1e94d02c2b7ad03d2115d691f48d064e9d52f58194a6637e4194391
public key:  026dccc749adc2a9d0d89497ac511f760f45c47dc5ed9cf352a58ac706453880ae
private key: 8e02b539b1500aa7c81cf3fed177448a546f19d2be416c0c61ff28e577d8d0cd
signature:   3044022027dc95ad6b740fe5129e7e62a75dd00f291a2aeb1200b84b09d9e3789406b6c002201a9ecd315dd6a0e632

scriptCode:  23210255a9626aebf5e29c0e6538428ba0d1dcf6ca98ffdf086aa8ced5e0d0215ea465ac
             (everything up to the last executed OP_CODESEPARATOR, including that OP_CODESEPARATOR, are rem
preimage:    01000000ef546acf4a020de3898d1b8956176bb507e6211b5ed3619cd08b6ea7e2a09d410000000000000000000000
sigHash:     fef7bd749cce710c5c052bd796df1af0d935e59cea63736268bcbe2d2134fc47
public key:  0255a9626aebf5e29c0e6538428ba0d1dcf6ca98ffdf086aa8ced5e0d0215ea465
private key: 86bf2ed75935a0cbef03b89d72034bb4c189d381037a5ac121a70016db8896ec
signature:   304402200de66acf4527789bfda55fc5459e214fa6083f936b430a762c629656216805ac0220396f550692cd347171

The serialized signed transaction is: 01000000000102fe3dc9208094f3ffd12645477b3dc56f60ec4fa8e6f5d67c565d1c6
```

This example shows how unexecuted `OP_CODESEPARATOR` is processed, and `SINGLE|ANYONECANPAY` does not commit to the input index:

```
The following is an unsigned transaction:
  0100000002e9b542c5176808107ff1df906f46bb1f2583b16112b95ee5380665ba7fcfc0010000000000ffffffff80e6883151639

  nVersion:  01000000
  txin:      02 e9b542c5176808107ff1df906f46bb1f2583b16112b95ee5380665ba7fcfc001 00000000 00 ffffffff
             80e68831516392fcd100d186b3c2c7b95c80b53c77e77c35ba03a66b429a2a1b 00000000 00 ffffffff
  txout:     02 8096980000000000 1976a914de4b231626ef508c9a74a8517e6783c0546d6b2888ac
             8096980000000000 1976a9146648a8cd4531e1ec47f35916de8e259237294d1e88ac
  nLockTime: 00000000

The first input comes from a native P2WSH witness program:
  scriptPubKey: 0020ba468eea561b26301e4cf69fa34bde4ad60c81e70f059f045ca9a79931004a4d value: 0.16777215
  witnessScript:0063ab68210392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da98ac
             0 IF CODESEPARATOR ENDIF <0392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da9

The second input comes from a native P2WSH witness program:
  scriptPubKey: 0020d9bbfbe56af7c4b7f960a70d7ea107156913d9e5a26b0a71429df5e097ca6537 value: 0.16777215
  witnessScript:5163ab68210392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da98ac
             1 IF CODESEPARATOR ENDIF <0392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da9

To sign it with a nHashType of 0x83 (SINGLE|ANYONECANPAY):

  nVersion:     01000000
  hashPrevouts: 0000000000000000000000000000000000000000000000000000000000000000
  hashSequence: 0000000000000000000000000000000000000000000000000000000000000000
  outpoint:     (see below)
  scriptCode:   (see below)
  amount:       ffffff0000000000
  nSequence:    ffffffff
  hashOutputs:  (see below)
  nLockTime:    00000000
  nHashType:    83000000

outpoint:    e9b542c5176808107ff1df906f46bb1f2583b16112b95ee5380665ba7fcfc00100000000
scriptCode:  270063ab68210392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da98ac
             (since the OP_CODESEPARATOR is not executed, nothing is removed from the scriptCode)
hashOutputs: b258eaf08c39fbe9fbac97c15c7e7adeb8df142b0df6f83e017f349c2b6fe3d2
preimage:    0100000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
sigHash:     e9071e75e25b8a1e298a72f0d2e9f4f95a0f5cdf86a533cda597eb402ed13b3a
public key:  0392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da98
private key: f52b3484edd96598e02a9c89c4492e9c1e2031f471c49fd721fe68b3ce37780d
signature:   3045022100f6a10b8604e6dc910194b79ccfc93e1bc0ec7c03453caaa8987f7d6c3413566002206216229ede9b4d6e

outpoint:    80e68831516392fcd100d186b3c2c7b95c80b53c77e77c35ba03a66b429a2a1b00000000
scriptCode:  2468210392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da98ac
             (everything up to the last executed OP_CODESEPARATOR, including that OP_CODESEPARATOR, are rem
```

```
hashOutputs: 91ea93dd77f702b738ebdbf3048940a98310e869a7bb8fa2c6cb3312916947ca
preimage:    0100000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
sigHash:     cd72f1f1a433ee9df816857fad88d8ebd97e09a75cd481583eb841c330275e54
public key:  0392972e2eb617b2388771abe27235fd5ac44af8e61693261550447a4c3e39da98
private key: f52b3484edd96598e02a9c89c4492e9c1e2031f471c49fd721fe68b3ce37780d
signature:   304402200325218 02a76ad7bf74d0e2c218b72cf0cbc867066e2e53db905ba37f130397e02207709e2188ed7f08f4c

The serialized signed transaction is:
01000000000102e9b542c5176808107ff1df906f46bb1f2583b16112b95ee5380665ba7fcfc0010000000000ffffffff80e68831516
  nVersion:  01000000
  marker:    00
  flag:      01
  txin:      02 e9b542c5176808107ff1df906f46bb1f2583b16112b95ee5380665ba7fcfc001 00000000 00 ffffffff
             80e68831516392fcd100d186b3c2c7b95c80b53c77e77c35ba03a66b429a2a1b 00000000 00 ffffffff
  txout:     02 8096980000000000 1976a914de4b231626ef508c9a74a8517e6783c0546d6b2888ac
             8096980000000000 1976a9146648a8cd4531e1ec47f35916de8e259237294d1e88ac
  witness    02 483045022100f6a10b8604e6dc910194b79ccfc93e1bc0ec7c03453caaa8987f7d6c3413566002206216229ede9
             02 473044022003252 1802a76ad7bf74d0e2c218b72cf0cbc867066e2e53db905ba37f130397e02207709e2188ed7f
  nLockTime: 00000000

Since SINGLE|
ANYONECANPAY does not commit to the input index, the signatures are still valid when the input-
output pairs are swapped:
010000000010280e68831516392fcd100d186b3c2c7b95c80b53c77e77c35ba03a66b429a2a1b0000000000ffffffffe9b542c5176
  nVersion:  01000000
  marker:    00
  flag:      01
  txin:      02 80e68831516392fcd100d186b3c2c7b95c80b53c77e77c35ba03a66b429a2a1b 00000000 00 ffffffff
             e9b542c5176808107ff1df906f46bb1f2583b16112b95ee5380665ba7fcfc001 00000000 00 ffffffff
  txout:     02 8096980000000000 1976a9146648a8cd4531e1ec47f35916de8e259237294d1e88ac
             8096980000000000 1976a914de4b231626ef508c9a74a8517e6783c0546d6b2888ac
  witness    02 473044022003252 1802a76ad7bf74d0e2c218b72cf0cbc867066e2e53db905ba37f130397e02207709e2188ed7f
             02 483045022100f6a10b8604e6dc910194b79ccfc93e1bc0ec7c03453caaa8987f7d6c3413566002206216229ede9
  nLockTime: 00000000
```

P2SH-P2WSH

This example is a P2SH-P2WSH 6-of-6 multisig witness program signed with 6 different `SIGHASH` types.

```
The following is an unsigned transaction: 0100000001366418 69ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca

  nVersion:  01000000
  txin:      01 36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e 01000000 00 ffffffff
  txout:     02 00e9a43500000000 1976a914389ffce9cd9ae88dcc0631e88a821ffdbe9bfe2688ac
             c0832f0500000000 1976a9147480a33f950689af511e6e84c138dbbd3c3ee41588ac
  nLockTime: 00000000
```

```
The input comes from a P2SH-P2WSH 6-of-6 multisig witness program:
  scriptPubKey : a9149993a429037b5d912407a71c252019287b8d27a587, value: 9.87654321
  redeemScript : 0020a16b5755f7f6f96dbd65f5f0d6ab9418b89af4b1f14a1bb8a09062c35f0dcb54
  witnessScript: 56210307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba32103b28f0c28bfab5455

hashPrevouts:
  dSHA256(36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e01000000)
= 74afdc312af5183c4198a40ca3c1a275b485496dd3929bca388c4b5e31f7aaa0

hashSequence:
  dSHA256(ffffffff)
= 3bb13029ce7b1f559ef5e747fcac439f1455a2ec7c5f09b72290795e70665044

hashOutputs for ALL:
  dSHA256(00e9a435000000001976a914389ffce9cd9ae88dcc0631e88a821ffdbe9bfe2688acc0832f05000000001976a9147480a
= bc4d309071414bed932f98832b27b4d76dad7e6c1346f487a8fdbb8eb90307cc

hashOutputs for SINGLE:
  dSHA256(00e9a435000000001976a914389ffce9cd9ae88dcc0631e88a821ffdbe9bfe2688ac)
= 9efe0c13a6b16c14a41b04ebe6a63f419bdacb2f8705b494a43063ca3cd4f708

hash preimage for ALL: 0100000074afdc312af5183c4198a40ca3c1a275b485496dd3929bca388c4b5e31f7aaa03bb13029ce7b
  nVersion:     01000000
  hashPrevouts: 74afdc312af5183c4198a40ca3c1a275b485496dd3929bca388c4b5e31f7aaa0
  hashSequence: 3bb13029ce7b1f559ef5e747fcac439f1455a2ec7c5f09b72290795e70665044
  outpoint:     36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e01000000
  scriptCode:   cf56210307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba32103b28f0c28bfab545
  amount:       b168de3a00000000
  nSequence:    ffffffff
  hashOutputs:  bc4d309071414bed932f98832b27b4d76dad7e6c1346f487a8fdbb8eb90307cc
  nLockTime:    00000000
  nHashType:    01000000
sigHash:      185c0be5263dce5b4bb50a047973c1b6272bfbd0103a89444597dc40b248ee7c
public key:   0307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba3
private key:  730fff80e1413068a05b57d6a58261f07551163369787f349438ea38ca80fac6
signature:    304402206ac44d672dac41f9b00e28f4df20c52eeb087207e8d758d76d92c6fab3b73e2b0220367750dbbe1929006

hash preimage for NONE: 0100000074afdc312af5183c4198a40ca3c1a275b485496dd3929bca388c4b5e31f7aaa000000000000
  nVersion:     01000000
  hashPrevouts: 74afdc312af5183c4198a40ca3c1a275b485496dd3929bca388c4b5e31f7aaa0
  hashSequence: 0000000000000000000000000000000000000000000000000000000000000000
  outpoint:     36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e01000000
  scriptCode:   cf56210307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba32103b28f0c28bfab545
  amount:       b168de3a00000000
  nSequence:    ffffffff
  hashOutputs:  0000000000000000000000000000000000000000000000000000000000000000
  nLockTime:    00000000
  nHashType:    02000000
sigHash:        e9733bc60ea13c95c6527066bb975a2ff29a925e80aa14c213f686cbae5d2f36
```

```
 public key:     03b28f0c28bfab54554ae8c658ac5c3e0ce6e79ad336331f78c428dd43eea8449b
 private key:    11fa3d25a17cbc22b29c44a484ba552b5a53149d106d3d853e22fdd05a2d8bb3
 signature:      3044022068c7946a43232757cbdf9176f009a928e1cd9a1a8c212f15c1e11ac9f2925d9002205b75f937ff2f9f3

 hash preimage for SINGLE: 0100000074afdc312af5183c4198a40ca3c1a275b485496dd3929bca388c4b5e31f7aaa0000000000
   nVersion:     01000000
   hashPrevouts: 74afdc312af5183c4198a40ca3c1a275b485496dd3929bca388c4b5e31f7aaa0
   hashSequence: 0000000000000000000000000000000000000000000000000000000000000000
   outpoint:     36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e01000000
   scriptCode:   cf56210307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba32103b28f0c28bfab545
   amount:       b168de3a00000000
   nSequence:    ffffffff
   hashOutputs:  9efe0c13a6b16c14a41b04ebe6a63f419bdacb2f8705b494a43063ca3cd4f708
   nLockTime:    00000000
   nHashType:    03000000
 sigHash:        1e1f1c303dc025bd664acb72e583e933fae4cff9148bf78c157d1e8f78530aea
 public key:     034b8113d703413d57761b8b9781957b8c0ac1dfe69f492580ca4195f50376ba4a
 private key:    77bf4141a87d55bdd7f3cd0bdccf6e9e642935fec45f2f30047be7b799120661
 signature:      3044022059ebf56d98010a932cf8ecfec54c48e6139ed6adb0728c09cbe1e4fa0915302e022007cd986c8fa870f

 hash preimage for ALL|
ANYONECANPAY: 0100000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
   nVersion:     01000000
   hashPrevouts: 0000000000000000000000000000000000000000000000000000000000000000
   hashSequence: 0000000000000000000000000000000000000000000000000000000000000000
   outpoint:     36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e01000000
   scriptCode:   cf56210307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba32103b28f0c28bfab545
   amount:       b168de3a00000000
   nSequence:    ffffffff
   hashOutputs:  bc4d309071414bed932f98832b27b4d76dad7e6c1346f487a8fdbb8eb90307cc
   nLockTime:    00000000
   nHashType:    81000000
 sigHash:        2a67f03e63a6a422125878b40b82da593be8d4efaafe88ee528af6e5a9955c6e
 public key:     033400f6afecb833092a9a21cfdf1ed1376e58c5d1f47de74683123987e967a8f4
 private key:    14af36970f5025ea3e8b5542c0f8ebe7763e674838d08808896b63c3351ffe49
 signature:      3045022100fbefd94bd0a488d50b79102b5dad4ab6ced30c4069f1eaa69a4b5a763414067e02203156c6a5c9cf8

 hash preimage for NONE|
ANYONECANPAY: 0100000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
   nVersion:     01000000
   hashPrevouts: 0000000000000000000000000000000000000000000000000000000000000000
   hashSequence: 0000000000000000000000000000000000000000000000000000000000000000
   outpoint:     36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e01000000
   scriptCode:   cf56210307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba32103b28f0c28bfab545
   amount:       b168de3a00000000
   nSequence:    ffffffff
   hashOutputs:  0000000000000000000000000000000000000000000000000000000000000000
   nLockTime:    00000000
```

```
  nHashType:    82000000
 sigHash:       781ba15f3779d5542ce8ecb5c18716733a5ee42a6f51488ec96154934e2c890a
 public key:    03a6d48b1131e94ba04d9737d61acdaa1322008af9602b3b14862c07a1789aac16
 private key:   fe9a95c19eef81dde2b95c1284ef39be497d128e2aa46916fb02d552485e0323
 signature:     3045022100a5263ea0553ba89221984bd7f0b13613db16e7a70c549a86de0cc0444141a407022005c360ef0ae5a

 hash preimage for SINGLE|
ANYONECANPAY: 0100000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
  nVersion:     01000000
  hashPrevouts: 0000000000000000000000000000000000000000000000000000000000000000
  hashSequence: 0000000000000000000000000000000000000000000000000000000000000000
  outpoint:     36641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca29787b96e01000000
  scriptCode:   cf56210307b8ae49ac90a048e9b53357a2354b3334e9c8bee813ecb98e99a7e07e8c3ba32103b28f0c28bfab545
  amount:       b168de3a00000000
  nSequence:    ffffffff
  hashOutputs:  9efe0c13a6b16c14a41b04ebe6a63f419bdacb2f8705b494a43063ca3cd4f708
  nLockTime:    00000000
  nHashType:    83000000
 sigHash:       511e8e52ed574121fc1b654970395502128263f62662e076dc6baf05c2e6a99b
 public key:    02d8b661b0b3302ee2f162b09e07a55ad5dfbe673a9f01d9f0c19617681024306b
 private key:   428a7aee9f0c2af0cd19af3cf1c78149951ea528726989b2e83e4778d2c3f890
 signature:     30440220525406a1482936d5a21888260dc165497a90a15669636d8edca6b9fe490d309c022032af0c646a34a44

 The serialized signed transaction is: 01000000000010136641869ca081e70f394c6948e8af409e18b619df2ed74aa106c1ca
```

No FindAndDelete

These examples show that `FindAndDelete` for the signature is not applied. The transactions are generated in an unconventional way. Instead of signing using a private key, the signatures are pre-determined as part of `witnessScript`. The public keys are generated with key recovery, using the fixed signatures and the `sighash` defined in this proposal. Therefore, the private keys are unknown.

```
 The following is an unsigned transaction: 010000000169c12106097dc2e0526493ef67f21269fe888ef05c7a3a5dacab38e

  nVersion:  01000000
  txin:      01 69c12106097dc2e0526493ef67f21269fe888ef05c7a3a5dacab38e1ac8387f1 4c1d0000 00 ffffffff
  txout:     01 0100000000000000 00
  nLockTime: 00000000

 The input comes from a P2WSH witness program:
  scriptPubKey : 00209e1be07558ea5cc8e02ed1d80c0911048afad949affa36d5c3951e3159dbea19, value: 0.00200000
  redeemScript : OP_CHECKSIGVERIFY <0x30450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556d
                 ad4830450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556dcc9022100baf95feb

 To sign it with a nHashType of 1 (SIGHASH_ALL):

 hashPrevouts:
  dSHA256(69c12106097dc2e0526493ef67f21269fe888ef05c7a3a5dacab38e1ac8387f14c1d0000)
 = b67c76d200c6ce72962d919dc107884b9d5d0e26f2aea7474b46a1904c53359f
```

```
hashSequence:
  dSHA256(ffffffff)
= 3bb13029ce7b1f559ef5e747fcac439f1455a2ec7c5f09b72290795e70665044

hashOutputs:
  dSHA256(0100000000000000)
= e5d196bfb21caca9dbd654cafb3b4dc0c4882c8927d2eb300d9539dd0b934228

hash preimage: 01000000b67c76d200c6ce72962d919dc107884b9d5d0e26f2aea7474b46a1904c53359f3bb13029ce7b1f559ef5

  nVersion:     01000000
  hashPrevouts: b67c76d200c6ce72962d919dc107884b9d5d0e26f2aea7474b46a1904c53359f
  hashSequence: 3bb13029ce7b1f559ef5e747fcac439f1455a2ec7c5f09b72290795e70665044
  outpoint:     69c12106097dc2e0526493ef67f21269fe888ef05c7a3a5dacab38e1ac8387f14c1d0000
  scriptCode:   4aad4830450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556dcc9022100baf95fe
  amount:       400d030000000000
  nSequence:    ffffffff
  hashOutputs:  e5d196bfb21caca9dbd654cafb3b4dc0c4882c8927d2eb300d9539dd0b934228
  nLockTime:    00000000
  nHashType:    01000000

sigHash:      71c9cd9b2869b9c70b01b1f0360c148f42dee72297db312638df136f43311f23
signature:    30450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556dcc9022100baf95feb48f845d
pubkey:       02a9781d66b61fb5a7ef00ac5ad5bc6ffc78be7b44a566e3c87870e1079368df4c

The serialized signed transaction is: 0100000000010169c12106097dc2e0526493ef67f21269fe888ef05c7a3a5dacab38e

  nVersion:  01000000
  marker:    00
  flag:      01
  txin:      01 69c12106097dc2e0526493ef67f21269fe888ef05c7a3a5dacab38e1ac8387f1 4c1d0000 00 ffffffff
  txout:     01 0100000000000000 00
  witness:   03 4830450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556dcc9022100baf95feb48f
                2102a9781d66b61fb5a7ef00ac5ad5bc6ffc78be7b44a566e3c87870e1079368df4c
                4aad4830450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556dcc9022100baf95fe
  nLockTime: 00000000

The following transaction is a ``OP_CHECKMULTISIGVERIFY`` version of the ``FindAndDelete`` examples: 010000

redeemScript:  OP_2 OP_CHECKMULTISIGVERIFY <30450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d8
hash preimage: 0100000039283953eb1e26994dde57b7f9362a79a8c523e2f8deba943c27e826a005f1e63bb13029ce7b1f559ef5
sighash:       c1628a1e7c67f14ca0c27c06e4fdeec2e6d1a73c7a91d7c046ff83e835aebb72
witness:       07 00
                  4830450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556dcc9022100baf95feb4
                  48304502205286f726690b2e9b0207f0345711e63fa7012045b9eb0f19c2458ce1db90cf43022100e89f17f86
                  0102
                  2102966f109c54e85d3aee8321301136cedeb9fc710fdef58a9de8a73942f8e567c0
                  21034ffc99dd9a79dd3cb31e2ab3e0b09e0e67db41ac068c625cd1f491576016c84e
                  9552af4830450220487fb382c4974de3f7d834c1b617fe15860828c7f96454490edd6d891556dcc9022100baf
```

The new serialization format is described in BIP144 BIP144: Segregated Witness (Peer Services)

Deployment

This proposal is deployed with Segregated Witness softfork (BIP 141)

Backward compatibility

As a soft fork, older software will continue to operate without modification. Non-upgraded nodes, however, will not see nor validate the witness data and will consider all witness programs, including the redefined sigops, as anyone-can-spend scripts.

Reference Implementation

https://github.com/bitcoin/bitcoin/pull/8149

References

Copyright

This document is placed in the public domain.

```
BIP: 147
Layer: Consensus (soft fork)
Title: Dealing with dummy stack element malleability
Authors: Johnson Lau <jl2012@xbt.hk>
Status: Deployed
Type: Specification
Assigned: 2016-09-02
License: PD
```

BIP 147

Abstract

This document specifies proposed changes to the Bitcoin transaction validity rules to fix a malleability vector in the extra stack element consumed by `OP_CHECKMULTISIG` and `OP_CHECKMULTISIGVERIFY`.

Motivation

Signature malleability refers to the ability of any relay node on the network to transform the signature in transactions, with no access to the relevant private keys required. For non-segregated witness transactions, signature malleability will change the `txid` and invalidate any unconfirmed child transactions. Although the `txid` of segregated witness (BIP141) transactions is not third party malleable, this malleability vector will change the `wtxid` and may reduce the efficiency of compact block relay (BIP152).

A design flaw in `OP_CHECKMULTISIG` and `OP_CHECKMULTISIGVERIFY` causes them to consume an extra stack element (“dummy element”) after signature validation. The dummy element is not inspected in any manner,

and could be replaced by any value without invalidating the script. This document specifies a new rule to fix this signature malleability.

Specification

To fix the dummy element malleability, a new consensus rule ("`NULLDUMMY`") is deployed to require that the dummy element MUST be the empty byte array. Anything else makes the script evaluate to false immediately. The `NULLDUMMY` rule applies to `OP_CHECKMULTISIG` and `OP_CHECKMULTISIGVERIFY` in pre-segregated scripts, and also pay-to-witness-script-hash scripts described in BIP141.

Deployment

This BIP will be deployed by "version bits" BIP9 using the same parameters for BIP141 and BIP143, with the name "segwit" and using bit 1.

For Bitcoin mainnet, the BIP9 starttime is midnight 15 November 2016 UTC (Epoch timestamp 1479168000) and BIP9 timeout is midnight 15 November 2017 UTC (Epoch timestamp 1510704000).

For Bitcoin testnet, the BIP9 starttime is midnight 1 May 2016 UTC (Epoch timestamp 1462060800) and BIP9 timeout is midnight 1 May 2017 UTC (Epoch timestamp 1493596800).

Compatibility

The reference client has produced compatible signatures from the beginning, and the `NULLDUMMY` rule has been enforced as relay policy by the reference client since v0.10.0. There has been no transactions violating the requirement being added to the chain since at least August 2015.

For all scriptPubKey types in actual use, non-compliant signatures can trivially be converted into compliant ones, so there is no loss of functionality by this requirement. Users MUST pay extra attention to this new rule when designing exotic scripts.

Implementation

An implementation for the reference client is available at https://github.com/bitcoin/bitcoin/pull/8636

Acknowledgements

Peter Todd is the original author of NULLDUMMY. This document is extracted from the previous BIP62 proposal, which was composed by Pieter Wuille and had input from various people.

Copyright

This document is placed in the public domain.

```
BIP: 148
Layer: Consensus (soft fork)
Title: Mandatory activation of segwit deployment
Authors: Shaolin Fry <shaolinfry@protonmail.ch>
Status: Deployed
Type: Specification
```

```
  Assigned: 2017-03-12
  License: BSD-3-Clause OR CC0-1.0
```

BIP 148

Abstract

This document specifies a BIP16 like soft fork flag day activation of the segregated witness BIP9 deployment known as “segwit”.

Definitions

“existing segwit deployment” refer to the BIP9 “segwit” deployment using bit 1, between November 15th 2016 and November 15th 2017 to activate BIP141, BIP143 and BIP147.

Motivation

Segwit increases the blocksize, fixes transaction malleability, and makes scripting easier to upgrade as well as bringing many other benefits.

It is hoped that miners will respond to this BIP by activating segwit early, before this BIP takes effect. Otherwise this BIP will cause the mandatory activation of the existing segwit deployment before the end of midnight November 15th 2017.

Specification

All times are specified according to median past time.

This BIP will be active between midnight August 1st 2017 (epoch time 1501545600) and midnight November 15th 2017 (epoch time 1510704000) if the existing segwit deployment is not locked-in or activated before epoch time 1501545600. This BIP will cease to be active when segwit is locked-in.

While this BIP is active, all blocks must set the nVersion header top 3 bits to 001 together with bit field (1<<1) (according to the existing segwit deployment). Blocks that do not signal as required will be rejected.

Reference implementation

```
// Check if Segregated Witness is Locked In
bool IsWitnessLockedIn(const CBlockIndex* pindexPrev, const Consensus::Params& params)
{
    LOCK(cs_main);
    return (VersionBitsState(pindexPrev, params, Consensus::DEPLOYMENT_SEGWIT, versionbitscache)
== THRESHOLD_LOCKED_IN);
}

// BIP148 mandatory segwit signalling.
int64_t nMedianTimePast = pindex->GetMedianTimePast();
if ( (nMedianTimePast >= 1501545600) &&  // Tue 01 Aug 2017 00:00:00 UTC
     (nMedianTimePast <= 1510704000) &&  // Wed 15 Nov 2017 00:00:00 UTC
     (!IsWitnessLockedIn(pindex->pprev, chainparams.GetConsensus()) &&  // Segwit is not locked
in
```

```
      !IsWitnessEnabled(pindex->pprev, chainparams.GetConsensus())) )   // and is not active.
{
    bool fVersionBits = (pindex->nVersion & VERSIONBITS_TOP_MASK) == VERSIONBITS_TOP_BITS;
    bool fSegbit = (pindex->nVersion & VersionBitsMask(chainparams.GetConsensus(),
Consensus::DEPLOYMENT_SEGWIT)) != 0;
    if (!(fVersionBits && fSegbit)) {
        return state.DoS(0, error("ConnectBlock(): relayed block must signal for segwit, please
upgrade"), REJECT_INVALID, "bad-no-segwit");
    }
}
```

https://github.com/bitcoin/bitcoin/compare/master…shaolinfry:bip-segwit-flagday

Backwards Compatibility

This deployment is compatible with the existing "segwit" bit 1 deployment scheduled between midnight November 15th, 2016 and midnight November 15th, 2017.

Rationale

Historically, the P2SH soft fork (BIP16) was activated using a predetermined flag day where nodes began enforcing the new rules. P2SH was successfully activated with relatively few issues

By orphaning non-signalling blocks during the last month of the BIP9 bit 1 "segwit" deployment, this BIP can cause the existing "segwit" deployment to activate without needing to release a new deployment.

References

- Mailing list discussion
- P2SH flag day activation
- BIP9 Version bits with timeout and delay
- BIP16 Pay to Script Hash
- BIP141 Segregated Witness (Consensus layer)
- BIP143 Transaction Signature Verification for Version 0 Witness Program
- BIP147 Dealing with dummy stack element malleability
- Segwit benefits

Copyright

This document is dual licensed as BSD 3-clause, and Creative Commons CC0 1.0 Universal.

```
BIP: 91
Layer: Consensus (soft fork)
Title: Reduced threshold Segwit MASF
Authors: James Hilliard <james.hilliard1@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2017-05-22
License: BSD-3-Clause OR CC0-1.0
```

BIP 91

Abstract

This document specifies a method to activate the existing BIP9 segwit deployment with a majority hashpower less than 95%.

Definitions

"existing segwit deployment" refer to the BIP9 "segwit" deployment using bit 1, between November 15th 2016 and November 15th 2017 to activate BIP141, BIP143 and BIP147.

Motivation

Segwit increases the blocksize, fixes transaction malleability, and makes scripting easier to upgrade as well as bringing many other benefits.

This BIP provides a way for a simple majority of miners to coordinate activation of the existing segwit deployment with less than 95% hashpower. For a number of reasons a complete redeployment of segwit is difficult to do until the existing deployment expires. This is due to 0.13.1+ having many segwit related features active already, including all the P2P components, the new network service flag, the witness-tx and block messages, compact blocks v2 and preferential peering. A redeployment of segwit will need to redefine all these things and doing so before expiry would greatly complicate testing.

Specification

While this BIP is active, all blocks must set the nVersion header top 3 bits to 001 together with bit field (1<<1) (according to the existing segwit deployment). Blocks that do not signal as required will be rejected.

Deployment

This BIP will be deployed by a "version bits" with an 80%(this can be adjusted if desired) 269 block activation threshold and 336 block confirmation window BIP9 with the name "segsignal" and using bit 4.

This BIP will have a start time of midnight June 1st, 2017 (epoch time 1496275200) and timeout on midnight November 15th 2017 (epoch time 1510704000). This BIP will cease to be active when segwit (BIP141) is locked-in, active, or failed

Reference implementation

```
// Deployment of SEGSIGNAL
consensus.vDeployments[Consensus::DEPLOYMENT_SEGSIGNAL].bit = 4;
consensus.vDeployments[Consensus::DEPLOYMENT_SEGSIGNAL].nStartTime = 1496275200; // June 1st,
2017.
consensus.vDeployments[Consensus::DEPLOYMENT_SEGSIGNAL].nTimeout = 1510704000; // November 15th,
2017.
consensus.vDeployments[Consensus::DEPLOYMENT_SEGSIGNAL].nOverrideMinerConfirmationWindow =
336; // ~2.33 days
consensus.vDeployments[Consensus::DEPLOYMENT_SEGSIGNAL].nOverrideRuleChangeActivationThreshold =
```

```
269; // 80%

class VersionBitsConditionChecker : public AbstractThresholdConditionChecker {
private:
    const Consensus::DeploymentPos id;

protected:
    int64_t BeginTime(const Consensus::Params& params) const { return
params.vDeployments[id].nStartTime; }
    int64_t EndTime(const Consensus::Params& params) const { return
params.vDeployments[id].nTimeout; }
    int Period(const Consensus::Params& params) const {
        if (params.vDeployments[id].nOverrideMinerConfirmationWindow > 0)
            return params.vDeployments[id].nOverrideMinerConfirmationWindow;
        return params.nMinerConfirmationWindow;
    }
    int Threshold(const Consensus::Params& params) const {
        if (params.vDeployments[id].nOverrideRuleChangeActivationThreshold > 0)
            return params.vDeployments[id].nOverrideRuleChangeActivationThreshold;
        return params.nRuleChangeActivationThreshold;
    }

    bool Condition(const CBlockIndex* pindex, const Consensus::Params& params) const
    {
        return (((pindex->nVersion & VERSIONBITS_TOP_MASK) == VERSIONBITS_TOP_BITS) && (pindex-
>nVersion & Mask(params)) != 0);
    }

public:
    VersionBitsConditionChecker(Consensus::DeploymentPos id_) : id(id_) {}
    uint32_t Mask(const Consensus::Params& params) const { return ((uint32_t)1) <<
params.vDeployments[id].bit; }
};

// SEGSIGNAL mandatory segwit signalling.
if (VersionBitsState(pindex->pprev, chainparams.GetConsensus(), Consensus::DEPLOYMENT_SEGSIGNAL,
versionbitscache) == THRESHOLD_ACTIVE &&
    VersionBitsState(pindex->pprev, chainparams.GetConsensus(), Consensus::DEPLOYMENT_SEGWIT,
versionbitscache) == THRESHOLD_STARTED)
{
    bool fVersionBits = (pindex->nVersion & VERSIONBITS_TOP_MASK) == VERSIONBITS_TOP_BITS;
    bool fSegbit = (pindex->nVersion & VersionBitsMask(chainparams.GetConsensus(),
Consensus::DEPLOYMENT_SEGWIT)) != 0;
    if (!(fVersionBits && fSegbit)) {
        return state.DoS(0, error("ConnectBlock(): relayed block must signal for segwit, please
upgrade"), REJECT_INVALID, "bad-no-segwit");
    }
}
```

https://github.com/segsignal/bitcoin

Backwards Compatibility

This deployment is compatible with the existing "segwit" bit 1 deployment scheduled between midnight November 15th, 2016 and midnight November 15th, 2017. Miners will need to upgrade their nodes to support segsignal otherwise they may build on top of an invalid block. While this bip is active users should either upgrade to segsignal or wait for additional confirmations when accepting payments.

Rationale

Historically we have used IsSuperMajority() to activate soft forks such as BIP66 which has a mandatory signalling requirement for miners once activated, this ensures that miners are aware of new rules being enforced. This technique can be leveraged to lower the signalling threshold of a soft fork while it is in the process of being deployed in a backwards compatible way.

By orphaning non-signalling blocks during the BIP9 bit 1 "segwit" deployment, this BIP can cause the existing "segwit" deployment to activate without needing to release a new deployment.

References

- Mailing list discussion
- P2SH flag day activation
- BIP9 Version bits with timeout and delay
- BIP16 Pay to Script Hash
- BIP141 Segregated Witness (Consensus layer)
- BIP143 Transaction Signature Verification for Version 0 Witness Program
- BIP147 Dealing with dummy stack element malleability
- BIP148 Mandatory activation of segwit deployment
- BIP149 Segregated Witness (second deployment)
- Segwit benefits

Copyright

This document is dual licensed as BSD 3-clause, and Creative Commons CC0 1.0 Universal.

Schnorr, Taproot, Tapscript

Taproot is one upgrade in three documents. BIP 340 defines Schnorr signatures on secp256k1. BIP 341 defines the SegWit version 1 spending rules: a public key in the output, an optional script tree, and a way to spend with either a key path or a script path. BIP 342 is Tapscript, the validation rules inside those leaves.

Together they are the 2021 consensus change that wallets mean when they say "Taproot address." Key-path spends look like a single signature. Script-path spends reveal only the leaf they use. Multisig and more complicated policies can hide behind the same output type.

I kept this chapter short on purpose. The three BIPs are meant to be read as a set. If you skipped the SegWit documents in the previous chapter, go back — Taproot is version 1 witness, and BIP 341 assumes you already know version 0.

In this chapter:

- BIP 340 — Schnorr Signatures for secp256k1
- BIP 341 — Taproot: SegWit version 1 spending rules
- BIP 342 — Validation of Taproot Scripts

```
  BIP: 340
  Title: Schnorr Signatures for secp256k1
  Authors: Pieter Wuille <pieter.wuille@gmail.com>
           Jonas Nick <jonasd.nick@gmail.com>
           Tim Ruffing <crypto@timruffing.de>
  Comments-Summary: No comments yet.
  Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0340
  Status: Deployed
  Type: Specification
  Assigned: 2020-01-19
  License: BSD-2-Clause
  License-Code: BSD-2-Clause OR MIT OR CC0-1.0
  Discussion: 2018-07-06: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2018-July/
016203.html [bitcoin-dev] Schnorr signatures BIP
```

BIP 340

Introduction

Abstract

This document proposes a standard for 64-byte Schnorr signatures over the elliptic curve *secp256k1*.

Copyright

This document is licensed under the 2-clause BSD license.

Motivation

Bitcoin has traditionally used ECDSA signatures over the secp256k1 curve with SHA256 hashes for authenticating transactions. These are standardized, but have a number of downsides compared to Schnorr signatures over the same curve:

- **Provable security**: Schnorr signatures are provably secure. In more detail, they are *strongly unforgeable under chosen message attack (SUF-CMA)*Informally, this means that without knowledge of the secret key but given valid signatures of arbitrary messages, it is not possible to come up with further valid signatures. in the random oracle model assuming the hardness of the elliptic curve discrete logarithm problem (ECDLP) and in the generic group model assuming variants of preimage and second preimage resistance of the used hash functionA detailed security proof in the random oracle model, which essentially restates the original security proof by Pointcheval and Stern more explicitly, can be found in a paper by Kiltz, Masny and Pan. All these security proofs assume a variant of Schnorr signatures that use *(e,s)* instead of *(R,s)* (see Design above). Since we use a unique encoding of R, there is an efficiently computable bijection that maps *(R,s)* to *(e,s)*, which allows to convert a successful SUF-CMA attacker for the *(e,s)* variant to a successful SUF-CMA attacker for the *(R,s)* variant (and vice-versa). Furthermore, the

proofs consider a variant of Schnorr signatures without key prefixing (see Design above), but it can be verified that the proofs are also correct for the variant with key prefixing. As a result, all the aforementioned security proofs apply to the variant of Schnorr signatures proposed in this document.. In contrast, the best known results for the provable security of ECDSA rely on stronger assumptions.
- **Non-malleability**: The SUF-CMA security of Schnorr signatures implies that they are non-malleable. On the other hand, ECDSA signatures are inherently malleableIf *(r,s)* is a valid ECDSA signature for a given message and key, then *(r,n-s)* is also valid for the same message and key. If ECDSA is restricted to only permit one of the two variants (as Bitcoin does through a policy rule on the network), it can be proven non-malleable under stronger than usual assumptions.; a third party without access to the secret key can alter an existing valid signature for a given public key and message into another signature that is valid for the same key and message. This issue is discussed in BIP62 and BIP146.
- **Linearity**: Schnorr signatures provide a simple and efficient method that enables multiple collaborating parties to produce a signature that is valid for the sum of their public keys. This is the building block for various higher-level constructions that improve efficiency and privacy, such as multisignatures and others (see Applications below).

For all these advantages, there are virtually no disadvantages, apart from not being standardized. This document seeks to change that. As we propose a new standard, a number of improvements not specific to Schnorr signatures can be made:

- **Signature encoding**: Instead of using DER-encoding for signatures (which are variable size, and up to 72 bytes), we can use a simple fixed 64-byte format.
- **Public key encoding**: Instead of using *compressed* 33-byte encodings of elliptic curve points which are common in Bitcoin today, public keys in this proposal are encoded as 32 bytes.
- **Batch verification**: The specific formulation of ECDSA signatures that is standardized cannot be verified more efficiently in batch compared to individually, unless additional witness data is added. Changing the signature scheme offers an opportunity to address this.
- **Completely specified**: To be safe for usage in consensus systems, the verification algorithm must be completely specified at the byte level. This guarantees that nobody can construct a signature that is valid to some verifiers but not all. This is traditionally not a requirement for digital signature schemes, and the lack of exact specification for the DER parsing of ECDSA signatures has caused problems for Bitcoin in the past, needing BIP66 to address it. In this document we aim to meet this property by design. For batch verification, which is inherently non-deterministic as the verifier can choose their batches, this property implies that the outcome of verification may only differ from individual verifications with negligible probability, even to an attacker who intentionally tries to make batch- and non-batch verification differ.

By reusing the same curve and hash function as Bitcoin uses for ECDSA, we are able to retain existing mechanisms for choosing secret and public keys, and we avoid introducing new assumptions about the security of elliptic curves and hash functions.

Description

We first build up the algebraic formulation of the signature scheme by going through the design choices. Afterwards, we specify the exact encodings and operations.

Design

Schnorr signature variant Elliptic Curve Schnorr signatures for message m and public key P generally involve a point R, integers e and s picked by the signer, and the base point G which satisfy $e = hash(R \,||\, m)$ and $s \cdot G = R + e \cdot P$. Two formulations exist, depending on whether the signer reveals e or R:

1. Signatures are pairs (e, s) that satisfy $e = hash(s \cdot G - e \cdot P \,||\, m)$. This variant avoids minor complexity introduced by the encoding of the point R in the signature (see paragraphs "Encoding R and public key point P" and "Implicit Y coordinates" further below in this subsection). Moreover, revealing e instead of R allows for potentially shorter signatures: Whereas an encoding of R inherently needs about 32 bytes, the hash e can be tuned to be shorter than 32 bytes, and a short hash of only 16 bytes suffices to provide SUF-CMA security at the target security level of 128 bits. However, a major drawback of this optimization is that finding collisions in a short hash function is easy. This complicates the implementation of secure signing protocols in scenarios in which a group of mutually distrusting signers work together to produce a single joint signature (see Applications below). In these scenarios, which are not captured by the SUF-CMA model due its assumption of a single honest signer, a promising attack strategy for malicious co-signers is to find a collision in the hash function in order to obtain a valid signature on a message that an honest co-signer did not intend to sign.
2. Signatures are pairs (R, s) that satisfy $s \cdot G = R + hash(R \,||\, m) \cdot P$. This supports batch verification, as there are no elliptic curve operations inside the hashes. Batch verification enables significant speedups.The speedup that results from batch verification can be demonstrated with the cryptography library libsecp256k1.

Since we would like to avoid the fragility that comes with short hashes, the e variant does not provide significant advantages. We choose the R-option, which supports batch verification.

Key prefixing Using the verification rule above directly makes Schnorr signatures vulnerable to "related-key attacks" in which a third party can convert a signature (R, s) for public key P into a signature $(R, s + a \cdot hash(R \,||\, m))$ for public key $P + a \cdot G$ and the same message m, for any given additive tweak a to the signing key. This would render signatures insecure when keys are generated using BIP32's unhardened derivation and other methods that rely on additive tweaks to existing keys such as Taproot.

To protect against these attacks, we choose *key prefixed*A limitation of committing to the public key (rather than to a short hash of it, or not at all) is that it removes the ability for public key recovery or verifying signatures against a short public key hash. These constructions are generally incompatible with batch verification. Schnorr signatures which means that the public key is prefixed to the message in the challenge hash input. This changes the equation to $s \cdot G = R + hash(R \,||\, P \,||\, m) \cdot P$. It can be shown that key prefixing protects against related-key attacks with additive tweaks. In general, key prefixing increases robustness in multi-user settings, e.g., it seems to be a requirement for proving multiparty signing protocols (such as MuSig, MuSig2, and FROST) secure (see Applications below).

We note that key prefixing is not strictly necessary for transaction signatures as used in Bitcoin currently, because signed transactions indirectly commit to the public keys already, i.e., m contains a commitment to pk. However, this indirect commitment should not be relied upon because it may change with proposals such as SIGHASH_NOINPUT (BIP118), and would render the signature scheme unsuitable for other purposes than signing transactions, e.g., signing ordinary messages.

Encoding R and public key point P There exist several possibilities for encoding elliptic curve points:

1. Encoding the full X and Y coordinates of P and R, resulting in a 64-byte public key and a 96-byte signature.
2. Encoding the full X coordinate and one bit of the Y coordinate to determine one of the two possible Y coordinates. This would result in 33-byte public keys and 65-byte signatures.
3. Encoding only the X coordinate, resulting in 32-byte public keys and 64-byte signatures.

Using the first option would be slightly more efficient for verification (around 10%), but we prioritize compactness, and therefore choose option 3.

Implicit Y coordinates In order to support efficient verification and batch verification, the Y coordinate of P and of R cannot be ambiguous (every valid X coordinate has two possible Y coordinates). We have a choice between several options for symmetry breaking:

1. Implicitly choosing the Y coordinate that is in the lower half.
2. Implicitly choosing the Y coordinate that is evenSince p is odd, negation modulo p will map even numbers to odd numbers and the other way around. This means that for a valid X coordinate, one of the corresponding Y coordinates will be even, and the other will be odd..
3. Implicitly choosing the Y coordinate that is a quadratic residue (i.e. has a square root modulo p).

The second option offers the greatest compatibility with existing key generation systems, where the standard 33-byte compressed public key format consists of a byte indicating the oddness of the Y coordinate, plus the full X coordinate. To avoid gratuitous incompatibilities, we pick that option for P, and thus our X-only public keys become equivalent to a compressed public key that is the X-only key prefixed by the byte 0x02. For consistency, the same is done for RAn earlier version of this draft used the third option instead, based on a belief that this would in general trade signing efficiency for verification efficiency. When using Jacobian coordinates, a common optimization in ECC implementations, it is possible to determine if a Y coordinate is a quadratic residue by computing the Legendre symbol, without converting to affine coordinates first (which needs a modular inversion). As modular inverses and Legendre symbols have similar performance in practice, this trade-off is not worth it..

Despite halving the size of the set of valid public keys, implicit Y coordinates are not a reduction in security. Informally, if a fast algorithm existed to compute the discrete logarithm of an X-only public key, then it could also be used to compute the discrete logarithm of a full public key: apply it to the X coordinate, and then optionally negate the result. This shows that breaking an X-only public key can be at most a small constant term faster than breaking a full one.This can be formalized by a simple reduction that reduces an attack on Schnorr signatures with implicit Y coordinates to an attack to Schnorr signatures with explicit Y coordinates. The reduction works by reencoding public keys and negating the result of the hash function, which is modeled as random oracle, whenever the challenge public key has an explicit Y coordinate that is odd. A proof sketch can be found here..

Tagged Hashes Cryptographic hash functions are used for multiple purposes in the specification below and in Bitcoin in general. To make sure hashes used in one context can't be reinterpreted in another one, hash functions can be tweaked with a context-dependent tag name, in such a way that collisions across contexts can be assumed to be infeasible. Such collisions obviously can not be ruled out completely, but only for schemes using tagging with a unique name. As for other schemes collisions are at least less likely with tagging than without.

For example, without tagged hashing a BIP340 signature could also be valid for a signature scheme where the only difference is that the arguments to the hash function are reordered. Worse, if the BIP340 nonce derivation function was copied or independently created, then the nonce could be accidentally reused in the other scheme leaking the secret key.

This proposal suggests to include the tag by prefixing the hashed data with *SHA256(tag) || SHA256(tag)*. Because this is a 64-byte long context-specific constant and the *SHA256* block size is also 64 bytes, optimized implementations are possible (identical to SHA256 itself, but with a modified initial state). Using SHA256 of the tag name itself is reasonably simple and efficient for implementations that don't choose to use the optimization. In general, tags can be arbitrary byte arrays, but are suggested to be textual descriptions in UTF-8 encoding.

Final scheme As a result, our final scheme ends up using public key *pk* which is the X coordinate of a point *P* on the curve whose Y coordinate is even and signatures *(r,s)* where *r* is the X coordinate of a point *R* whose Y coordinate is even. The signature satisfies $s \cdot G = R + tagged_hash(r \,||\, pk \,||\, m) \cdot P$.

Specification

The following conventions are used, with constants as defined for secp256k1. We note that adapting this specification to other elliptic curves is not straightforward and can result in an insecure schemeAmong other pitfalls, using the specification with a curve whose order is not close to the size of the range of the nonce derivation function is insecure..

- Lowercase variables represent integers or byte arrays.
  - The constant *p* refers to the field size, *0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F*.
  - The constant *n* refers to the curve order, *0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141*.
- Uppercase variables refer to points on the curve with equation $y^2 = x^3 + 7$ over the integers modulo *p*.
  - *is_infinite(P)* returns whether or not *P* is the point at infinity.
  - *x(P)* and *y(P)* are integers in the range *0..p-1* and refer to the X and Y coordinates of a point *P* (assuming it is not infinity).
  - The constant *G* refers to the base point, for which *x(G) = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798* and *y(G) = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8*.
  - Addition of points refers to the usual elliptic curve group operation.
  - Multiplication (·) of an integer and a point refers to the repeated application of the group operation.
- Functions and operations:
  - || refers to byte array concatenation.
  - The function *x[i:j]*, where *x* is a byte array and $i, j \geq 0$, returns a *(j - i)*-byte array with a copy of the *i*-th byte (inclusive) to the *j*-th byte (exclusive) of *x*.
  - The function *bytes(x)*, where *x* is an integer, returns the 32-byte encoding of *x*, most significant byte first.
  - The function *bytes(P)*, where *P* is a point, returns *bytes(x(P))*.
  - The function *int(x)*, where *x* is a 32-byte array, returns the 256-bit unsigned integer whose most significant byte first encoding is *x*.
  - The function *has_even_y(P)*, where *P* is a point for which *not is_infinite(P)*, returns $y(P) \bmod 2 = 0$.

    - The function *lift_x(x)*, where x is a 256-bit unsigned integer, returns the point P for which $x(P) = x$

  Given a candidate X coordinate x in the range $0..p-1$, there exist either exactly two or exactly zero vali
= x. The valid Y coordinates for a given candidate x are the square roots of $c = x^3 + 7 \mod p$ and they can be computed as $y = \pm c^{(p+1)/4} \mod p$ (see Quadratic residue) if they exist, which can be checked by squaring and comparing with c. and *has_even_y(P)*, or fails i

- - Fail if $x \geq p$.
    - Let $c = x^3 + 7 \mod p$.
    - Let $y = c^{(p+1)/4} \mod p$.
    - Fail if $c \neq y^2 \mod p$.
    - Return the unique point P such that $x(P) = x$ and $y(P) = y$ if $y \mod 2 = 0$ or $y(P) = p-y$ otherwise.
- - The function $hash_{name}(x)$ where x is a byte array returns the 32-byte hash $SHA256(SHA256(tag) \,||\, SHA256(tag) \,||\, x)$, where *tag* is the UTF-8 encoding of *name*.

Public Key Generation

Input:

- The secret key *sk*: a 32-byte array, freshly generated uniformly at random

The algorithm *PubKey(sk)* is defined as:

- Let $d' = int(sk)$.
- Fail if $d' = 0$ or $d' \geq n$.
- Return $bytes(d' \cdot G)$.

Note that we use a very different public key format (32 bytes) than the ones used by existing systems (which typically use elliptic curve points as public keys, or 33-byte or 65-byte encodings of them). A side effect is that $PubKey(sk) = PubKey(bytes(n - int(sk)))$, so every public key has two corresponding secret keys.

Public Key Conversion

As an alternative to generating keys randomly, it is also possible and safe to repurpose existing key generation algorithms for ECDSA in a compatible way. The secret keys constructed by such an algorithm can be used as *sk* directly. The public keys constructed by such an algorithm (assuming they use the 33-byte compressed encoding) need to be converted by dropping the first byte. Specifically, BIP32 and schemes built on top of it remain usable.

Default Signing

Input:

- The secret key *sk*: a 32-byte array
- The message *m*: a byte array
- Auxiliary random data *a*: a 32-byte array

The algorithm *Sign(sk, m)* is defined as:

- Let $d' = int(sk)$
- Fail if $d' = 0$ or $d' \geq n$
- Let $P = d' \cdot G$
- Let $d = d'$ if *has_even_y(P)*, otherwise let $d = n - d'$.
- Let t be the byte-wise xor of *bytes(d)* and $hash_{BIP0340/aux}(a)$The auxiliary random data is hashed (with a unique tag) as a precaution against situations where the randomness may be correlated with the private key itself. It is xored with the private key (rather than combined with it in a hash) to reduce the number of operations exposed to the actual secret key..
- Let $rand = hash_{BIP0340/nonce}(t \mid\mid bytes(P) \mid\mid m)$Including the public key as input to the nonce hash helps ensure the robustness of the signing algorithm by preventing leakage of the secret key if the calculation of the public key P is performed incorrectly or maliciously, for example if it is left to the caller for performance reasons..
- Let $k' = int(rand) \bmod n$Note that in general, taking a uniformly random 256-bit integer modulo the curve order will produce an unacceptably biased result. However, for the secp256k1 curve, the order is sufficiently close to 2^{256} that this bias is not observable ($1 - n / 2^{256}$ is around $1.27 * 2^{-128}$)..
- Fail if $k' = 0$.
- Let $R = k' \cdot G$.
- Let $k = k'$ if *has_even_y(R)*, otherwise let $k = n - k'$.
- Let $e = int(hash_{BIP0340/challenge}(bytes(R) \mid\mid bytes(P) \mid\mid m)) \bmod n$.
- Let $sig = bytes(R) \mid\mid bytes((k + ed) \bmod n)$.
- If *Verify(bytes(P), m, sig)* (see below) returns failure, abortVerifying the signature before leaving the signer prevents random or attacker provoked computation errors. This prevents publishing invalid signatures which may leak information about the secret key. It is recommended, but can be omitted if the computation cost is prohibitive..
- Return the signature *sig*.

The auxiliary random data should be set to fresh randomness generated at signing time, resulting in what is called a *synthetic nonce*. Using 32 bytes of randomness is optimal. If obtaining randomness is expensive, 16 random bytes can be padded with 16 null bytes to obtain a 32-byte array. If randomness is not available at all at signing time, a simple counter wide enough to not repeat in practice (e.g., 64 bits or wider) and padded with null bytes to a 32 byte-array can be used, or even the constant array with 32 null bytes. Using any non-repeating value increases protection against fault injection attacks. Using unpredictable randomness additionally increases protection against other side-channel attacks, and is **recommended whenever available**. Note that while this means the resulting nonce is not deterministic, the randomness is only supplemental to security. The normal security properties (excluding side-channel attacks) do not depend on the quality of the signing-time RNG.

Alternative Signing

It should be noted that various alternative signing algorithms can be used to produce equally valid signatures. The 32-byte *rand* value may be generated in other ways, producing a different but still valid signature (in other words, this is not a *unique* signature scheme). **No matter which method is used to generate the *rand* value, the value must be a fresh uniformly random 32-byte string which is not even partially predictable for the attacker.** For nonces without randomness, this implies that the same inputs must not be presented in another context. This can be most reliably accomplished by not reusing the same private key across different signing

schemes. For example, if the *rand* value was computed as per RFC6979 and the same secret key is used in deterministic ECDSA with RFC6979, the signatures can leak the secret key through nonce reuse.

Nonce exfiltration protection It is possible to strengthen the nonce generation algorithm using a second device. In this case, the second device contributes randomness which the actual signer provably incorporates into its nonce. This prevents certain attacks where the signer's device is compromised and intentionally tries to leak the secret key through its nonce selection.

Multisignatures This signature scheme is compatible with various types of multisignature and threshold schemes such as MuSig2, where a single public key requires holders of multiple secret keys to participate in signing (see Applications below). **It is important to note that multisignature signing schemes in general are insecure with the *rand* generation from the default signing algorithm above (or any other deterministic method).**

Precomputed public key data For many uses, the compressed 33-byte encoding of the public key corresponding to the secret key may already be known, making it easy to evaluate *has_even_y(P)* and *bytes(P)*. As such, having signers supply this directly may be more efficient than recalculating the public key from the secret key. However, if this optimization is used and additionally the signature verification at the end of the signing algorithm is dropped for increased efficiency, signers must ensure the public key is correctly calculated and not taken from untrusted sources.

Verification

Input:

- The public key *pk*: a 32-byte array
- The message *m*: a byte array
- A signature *sig*: a 64-byte array

The algorithm *Verify(pk, m, sig)* is defined as:

- Let $P = lift_x(int(pk))$; fail if that fails.
- Let $r = int(sig[0:32])$; fail if $r \geq p$.
- Let $s = int(sig[32:64])$; fail if $s \geq n$.
- Let $e = int(hash_{BIP0340/challenge}(bytes(r) \mid\mid bytes(P) \mid\mid m)) \bmod n$.
- Let $R = s \cdot G - e \cdot P$.
- Fail if *is_infinite(R)*.
- Fail if *not has_even_y(R)*.
- Fail if $x(R) \neq r$.
- Return success iff no failure occurred before reaching this point.

For every valid secret key *sk* and message *m*, *Verify(PubKey(sk),m,Sign(sk,m))* will succeed.

Note that the correctness of verification relies on the fact that *lift_x* always returns a point with an even Y coordinate. A hypothetical verification algorithm that treats points as public keys, and takes the point *P* directly as input would fail any time a point with odd Y is used. While it is possible to correct for this by negating points with odd Y coordinate before further processing, this would result in a scheme where every (message, signature) pair is valid for two public keys (a type of malleability that exists for ECDSA as well, but we don't wish to retain). We avoid these problems by treating just the X coordinate as public key.

Batch Verification

Input:

- The number u of signatures
- The public keys $pk_{1..u}$: u 32-byte arrays
- The messages $m_{1..u}$: u byte arrays
- The signatures $sig_{1..u}$: u 64-byte arrays

The algorithm $BatchVerify(pk_{1..u}, m_{1..u}, sig_{1..u})$ is defined as:

- Generate $u-1$ random integers $a_{2...u}$ in the range $1...n-1$. They are generated deterministically using a CSPRNG seeded by a cryptographic hash of all inputs of the algorithm, i.e. $seed = seed_hash(pk_1..pk_u \,||\, m_1..m_u \,||\, sig_1..sig_u)$. A safe choice is to instantiate $seed_hash$ with SHA256 and use ChaCha20 with key $seed$ as a CSPRNG to generate 256-bit integers, skipping integers not in the range $1...n-1$.
- For $i = 1 .. u$:
  - Let $P_i = lift_x(int(pk_i))$; fail if it fails.
  - Let $r_i = int(sig_i[0:32])$; fail if $r_i \geq p$.
  - Let $s_i = int(sig_i[32:64])$; fail if $s_i \geq n$.
  - Let $e_i = int(hash_{BIP0340/challenge}(bytes(r_i) \,||\, bytes(P_i) \,||\, m_i)) \bmod n$.
  - Let $R_i = lift_x(r_i)$; fail if $lift_x(r_i)$ fails.
- Fail if $(s_1 + a_2s_2 + ... + a_us_u)\cdot G \neq R_1 + a_2\cdot R_2 + ... + a_u\cdot R_u + e_1\cdot P_1 + (a_2e_2)\cdot P_2 + ... + (a_ue_u)\cdot P_u$.
- Return success iff no failure occurred before reaching this point.

If all individual signatures are valid (i.e., *Verify* would return success for them), *BatchVerify* will always return success. If at least one signature is invalid, *BatchVerify* will return success with at most a negligible probability.

Usage Considerations

Messages of Arbitrary Size

The signature scheme specified in this BIP accepts byte strings of arbitrary size as input messages.In theory, the message size is restricted due to the fact that SHA256 accepts byte strings only up to size of 2^61-1 bytes. It is understood that implementations may reject messages which are too large in their environment or application context, e.g., messages which exceed predefined buffers or would otherwise cause resource exhaustion.

Earlier revisions of this BIP required messages to be exactly 32 bytes. This restriction puts a burden on callers who typically need to perform pre-hashing of the actual input message by feeding it through SHA256 (or another collision-resistant cryptographic hash function) to create a 32-byte digest which can be passed to signing or verification (as for example done in BIP341.)

Since pre-hashing may not always be desirable, e.g., when actual messages are shorter than 32 bytes,Another reason to omit pre-hashing is to protect against certain types of cryptanalytic advances against the hash function used for pre-hashing: If pre-hashing is used, an attacker that can find collisions in the pre-hashing function can necessarily forge signatures under chosen-message attacks. If pre-hashing is not used, an attacker that can find collisions in SHA256 (as used inside the signature scheme) may not be able to forge signatures. However, this seeming advantage is mostly irrelevant in the context of Bitcoin, which already relies on collision

resistance of SHA256 in other places, e.g., for transaction hashes. the restriction to 32-byte messages has been lifted. We note that pre-hashing is recommended for performance reasons in applications that deal with large messages. If large messages are not pre-hashed, the algorithms of the signature scheme will perform more hashing internally. In particular, the signing algorithm needs two sequential hashing passes over the message, which means that the full message must necessarily be kept in memory during signing, and large messages entail a runtime penalty.Typically, messages of 56 bytes or longer enjoy a performance benefit from pre-hashing, assuming the speed of SHA256 inside the signing algorithm matches that of the pre-hashing done by the calling application.

Domain Separation

It is good cryptographic practice to use a key pair only for a single purpose. Nevertheless, there may be situations in which it may be desirable to use the same key pair in multiple contexts, i.e., to sign different types of messages within the same application or even messages in entirely different applications (e.g., a secret key may be used to sign Bitcoin transactions as well plain text messages).

As a consequence, applications should ensure that a signed application message intended for one context is never deemed valid in a different context (e.g., a signed plain text message should never be misinterpreted as a signed Bitcoin transaction, because this could cause unintended loss of funds). This is called "domain separation" and it is typically realized by partitioning the message space. Even if key pairs are intended to be used only within a single context, domain separation is a good idea because it makes it easy to add more contexts later.

As a best practice, we recommend applications to use exactly one of the following methods to pre-process application messages before passing it to the signature scheme:

- Either, pre-hash the application message using $hash_{name}$, where *name* identifies the context uniquely (e.g., "foo-app/signed-bar"),
- or prefix the actual message with a 33-byte string that identifies the context uniquely (e.g., the UTF-8 encoding of "foo-app/signed-bar", padded with null bytes to 33 bytes).

As the two pre-processing methods yield different message sizes (32 bytes vs. at least 33 bytes), there is no risk of collision between them.

Applications

There are several interesting applications beyond simple signatures. While recent academic papers claim that they are also possible with ECDSA, consensus support for Schnorr signature verification would significantly simplify the constructions.

Multisignatures and Threshold Signatures

By means of an interactive scheme such as MuSig2 (BIP327), participants can aggregate their public keys into a single public key which they can jointly sign for. This allows *n*-of-*n* multisignatures which, from a verifier's perspective, are no different from ordinary signatures, giving improved privacy and efficiency versus *CHECKMULTISIG* or other means.

Moreover, Schnorr signatures are compatible with distributed key generation, which enables interactive threshold signatures schemes, e.g., the schemes by Stinson and Strobl (2001), by Gennaro, Jarecki, Krawczyk,

and Rabin (2007), or the FROST scheme including its variants such as FROST3. These protocols make it possible to realize k-of-n threshold signatures, which ensure that any subset of size k of the set of n signers can sign but no subset of size less than k can produce a valid Schnorr signature.

Adaptor Signatures

Adaptor signatures can be produced by a signer by offsetting his public nonce R with a known point $T = t \cdot G$, but not offsetting the signature's s value. A correct signature (or partial signature, as individual signers' contributions to a multisignature are called) on the same message with same nonce will then be equal to the adaptor signature offset by t, meaning that learning t is equivalent to learning a correct signature. This can be used to enable atomic swaps or even general payment channels in which the atomicity of disjoint transactions is ensured using the signatures themselves, rather than Bitcoin script support. The resulting transactions will appear to verifiers to be no different from ordinary single-signer transactions, except perhaps for the inclusion of locktime refund logic.

Adaptor signatures, beyond the efficiency and privacy benefits of encoding script semantics into constant-sized signatures, have additional benefits over traditional hash-based payment channels. Specifically, the secret values t may be reblinded between hops, allowing long chains of transactions to be made atomic while even the participants cannot identify which transactions are part of the chain. Also, because the secret values are chosen at signing time, rather than key generation time, existing outputs may be repurposed for different applications without recourse to the blockchain, even multiple times.

Blind Signatures

A blind signature protocol is an interactive protocol that enables a signer to sign a message at the behest of another party without learning any information about the signed message or the signature. Schnorr signatures admit a very simple blind signature scheme which is however insecure because it's vulnerable to Wagner's attack. Known mitigations are to let the signer abort a signing session with a certain probability, which can be proven secure under non-standard cryptographic assumptions, or to use zero-knowledge proofs.

Blind Schnorr signatures could for example be used in Partially Blind Atomic Swaps, a construction to enable transferring of coins, mediated by an untrusted escrow agent, without connecting the transactors in the public blockchain transaction graph.

Test Vectors and Reference Code

For development and testing purposes, we provide a collection of test vectors in CSV format, a naive, highly inefficient, and non-constant time pure Python 3.7 reference implementation of the signing and verification algorithm as well as the script used to generate the test vectors under the BSD-2-Clause License, or the MIT License, or CC0 1.0, at your choice. The reference implementation is for demonstration purposes only and not to be used in production environments.

Changelog

To help implementers understand updates to this BIP, we keep a list of substantial changes.

- 2022-08: Fix function signature of lift_x in reference code
- 2023-04: Allow messages of arbitrary size

- 2024-05: Update “Applications” section with more recent references
- 2025-04: Change license of test vectors and code

Footnotes

Acknowledgements

This document is the result of many discussions around Schnorr based signatures over the years, and had input from Johnson Lau, Greg Maxwell, Andrew Poelstra, Rusty Russell, and Anthony Towns. The authors further wish to thank all those who provided valuable feedback and reviews, including the participants of the structured reviews.

```
  BIP: 341
  Layer: Consensus (soft fork)
  Title: Taproot: SegWit version 1 spending rules
  Authors: Pieter Wuille <pieter.wuille@gmail.com>
           Jonas Nick <jonasd.nick@gmail.com>
           Anthony Towns <aj@erisian.com.au>
  Comments-Summary: No comments yet.
  Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0341
  Status: Deployed
  Type: Specification
  Assigned: 2020-01-19
  License: BSD-3-Clause
  Discussion: 2019-05-06: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2019-May/
016914.html [bitcoin-dev] Taproot proposal
              2019-10-09: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2019-October/
017378.html [bitcoin-dev] Taproot updates
  Requires: 340
```

BIP 341

Introduction

Abstract

This document proposes a new SegWit version 1 output type, with spending rules based on Taproot, Schnorr signatures, and Merkle branches.

Copyright

This document is licensed under the 3-clause BSD license.

Motivation

This proposal aims to improve privacy, efficiency, and flexibility of Bitcoin’s scripting capabilities without adding new security assumptions**What does not adding security assumptions mean?** Unforgeability of signatures is a necessary requirement to prevent theft. At least when treating script execution as a digital signature scheme itself, unforgeability can be proven in the Random Oracle Model assuming the Discrete Logarithm problem is hard. A proof for unforgeability of ECDSA in the current script system needs non-

standard assumptions on top of that. Note that it is hard in general to model exactly what security for script means, as it depends on the policies and protocols used by wallet software.. Specifically, it seeks to minimize how much information about the spendability conditions of a transaction output is revealed on chain at creation or spending time and to add a number of upgrade mechanisms, while fixing a few minor but long-standing issues.

Design

A number of related ideas for improving Bitcoin's scripting capabilities have been previously proposed: Schnorr signatures (BIP340), Merkle branches ("MAST", BIP114, BIP117), new sighash modes (BIP118), new opcodes like CHECKSIGFROMSTACK, Taproot, Graftroot, G'root, and cross-input aggregation.

Combining all these ideas in a single proposal would be an extensive change, be hard to review, and likely miss new discoveries that otherwise could have been made along the way. Not all are equally mature as well. For example, cross-input aggregation interacts in complex ways with upgrade mechanisms, and solutions to that are still in flux. On the other hand, separating them all into independent upgrades would reduce the efficiency and privacy gains to be had, and wallet and service providers may not be inclined to go through many incremental updates. Therefore, we're faced with a tradeoff between functionality and scope creep. In this design we strike a balance by focusing on the structural script improvements offered by Taproot and Merkle branches, as well as changes necessary to make them usable and efficient. For things like sighashes and opcodes we include fixes for known problems, but exclude new features that can be added independently with no downsides.

As a result we choose this combination of technologies:

- **Merkle branches** let us only reveal the actually executed part of the script to the blockchain, as opposed to all possible ways a script can be executed. Among the various known mechanisms for implementing this, one where the Merkle tree becomes part of the script's structure directly maximizes the space savings, so that approach is chosen.
- **Taproot** on top of that lets us merge the traditionally separate pay-to-pubkey and pay-to-scripthash policies, making all outputs spendable by either a key or (optionally) a script, and indistinguishable from each other. As long as the key-based spending path is used for spending, it is not revealed whether a script path was permitted as well, resulting in space savings and an increase in scripting privacy at spending time.
- Taproot's advantages become apparent under the assumption that most applications involve outputs that could be spent by all parties agreeing. That's where **Schnorr** signatures come in, as they permit key aggregation: a public key can be constructed from multiple participant public keys, and which requires cooperation between all participants to sign for. Such multi-party public keys and signatures are indistinguishable from their single-party equivalents. This means that with taproot most applications can use the key-based spending path, which is both efficient and private. This can be generalized to arbitrary M-of-N policies, as Schnorr signatures support threshold signing, at the cost of more complex setup protocols.
- As Schnorr signatures also permit **batch validation**, allowing multiple signatures to be validated together more efficiently than validating each one independently, we make sure all parts of the design are compatible with this.
- Where unused bits appear as a result of the above changes, they are reserved for mechanisms for **future extensions**. As a result, every script in the Merkle tree has an associated version such that new script

versions can be introduced with a soft fork while remaining compatible with BIP 341. Additionally, future soft forks can make use of the currently unused `annex` in the witness (see Rationale).

- While the core semantics of the **signature hashing algorithm** are not changed, a number of improvements are included in this proposal. The new signature hashing algorithm fixes the verification capabilities of offline signing devices by including amount and scriptPubKey in the signature message, avoids unnecessary hashing, uses **tagged hashes** and defines a default sighash byte.
- The **public key is directly included in the output** in contrast to typical earlier constructions which store a hash of the public key or script in the output. This has the same cost for senders and is more space efficient overall if the key-based spending path is taken. **Why is the public key directly included in the output?** While typical earlier constructions store a hash of a script or a public key in the output, this is rather wasteful when a public key is always involved. To guarantee batch verifiability, the public key must be known to every verifier, and thus only revealing its hash as an output would imply adding an additional 32 bytes to the witness. Furthermore, to maintain 128-bit collision security for outputs, a 256-bit hash would be required anyway, which is comparable in size (and thus in cost for senders) to revealing the public key directly. While the usage of public key hashes is often said to protect against ECDLP breaks or quantum computers, this protection is very weak at best: transactions are not protected while being confirmed, and a very large portion of the currency's supply is not under such protection regardless. Actual resistance to such systems can be introduced by relying on different cryptographic assumptions, but this proposal focuses on improvements that do not change the security model.

Informally, the resulting design is as follows: a new witness version is added (version 1), whose programs consist of 32-byte encodings of points Q. Q is computed as $P + hash(P||m)G$ for a public key P, and the root m of a Merkle tree whose leaves consist of a version number and a script. These outputs can be spent directly by providing a signature for Q, or indirectly by revealing P, the script and leaf version, inputs that satisfy the script, and a Merkle path that proves Q committed to that leaf. All hashes in this construction (the hash for computing Q from P, the hashes inside the Merkle tree's inner nodes, and the signature hashes used) are tagged to guarantee domain separation.

Specification

This section specifies the Taproot consensus rules. Validity is defined by exclusion: a block or transaction is valid if no condition exists that marks it failed.

The notation below follows that of BIP340. This includes the $hash_{tag}(x)$ notation to refer to $SHA256(SHA256(tag) || SHA256(tag) || x)$. To the best of the authors' knowledge, no existing use of SHA256 in Bitcoin feeds it a message that starts with two single SHA256 outputs, making collisions between $hash_{tag}$ with other hashes extremely unlikely.

Script validation rules

A Taproot output is a native SegWit output (see BIP141) with version number 1, and a 32-byte witness program. The following rules only apply when such an output is being spent. Any other outputs, including version 1 outputs with lengths other than 32 bytes, or P2SH-wrapped version 1 outputs**Why is P2SH-wrapping not supported?** Using P2SH-wrapped outputs only provides 80-bit collision security due to the use

of a 160-bit hash. This is considered low, and becomes a security risk whenever the output includes data from more than a single party (public keys, hashes, …)., remain unencumbered.

- Let q be the 32-byte array containing the witness program (the second push in the scriptPubKey) which represents a public key according to BIP340.
- Fail if the witness stack has 0 elements.
- If there are at least two witness elements, and the first byte of the last element is 0x50**Why is the first byte of the annex `0x50`?** The `0x50` is chosen as it could not be confused with a valid P2WPKH or P2WSH spending. As the control block's initial byte's lowest bit is used to indicate the parity of the public key's Y coordinate, each leaf version needs an even byte value and the immediately following odd byte value that are both not yet used in P2WPKH or P2WSH spending. To indicate the annex, only an "unpaired" available byte is necessary like `0x50`. This choice maximizes the available options for future script versions., this last element is called *annex a***What is the purpose of the annex?** The annex is a reserved space for future extensions, such as indicating the validation costs of computationally expensive new opcodes in a way that is recognizable without knowing the scriptPubKey of the output being spent. Until the meaning of this field is defined by another softfork, users SHOULD NOT include `annex` in transactions, or it may lead to PERMANENT FUND LOSS. and is removed from the witness stack. The annex (or the lack of thereof) is always covered by the signature and contributes to transaction weight, but is otherwise ignored during taproot validation.
- If there is exactly one element left in the witness stack, key path spending is used:
  - The single witness stack element is interpreted as the signature and must be valid (see the next section) for the public key q (see the next subsection).
- If there are at least two witness elements left, script path spending is used:
  - Call the second-to-last stack element s, the script.
  - The last stack element is called the control block c, and must have length $33 + 32m$, for a value of m that is an integer between 0 and 128**Why is the Merkle path length limited to 128?** The optimally space-efficient Merkle tree can be constructed based on the probabilities of the scripts in the leaves, using the Huffman algorithm. This algorithm will construct branches with lengths approximately equal to $log_2(1/probability)$, but to have branches longer than 128 you would need to have scripts with an execution chance below 1 in 2^{128}. As that is our security bound, scripts that truly have such a low chance can probably be removed entirely., inclusive. Fail if it does not have such a length.
  - Let $p = c[1:33]$ and let $P = lift_x(int(p))$ where $lift_x$ and $[:]$ are defined as in BIP340. Fail if this point is not on the curve.
  - Let $v = c[0] \,\&\, 0xfe$ and call it the *leaf version***What constraints are there on the leaf version?** First, the leaf version cannot be odd as $c[0] \,\&\, 0xfe$ will always be even, and cannot be $0x50$ as that would result in ambiguity with the annex. In addition, in order to support some forms of static analysis that rely on being able to identify script spends without access to the output being spent, it is recommended to avoid using any leaf versions that would conflict with a valid first byte of either a valid P2WPKH pubkey or a valid P2WSH script (that is, both v and $v \mid 1$ should be an undefined, invalid or disabled opcode or an opcode that is not valid as the first opcode). The values that comply to this rule are the 32 even values between $0xc0$ and $0xfe$ and also $0x66$, $0x7e$, $0x80$, $0x84$, $0x96$, $0x98$, $0xba$, $0xbc$, $0xbe$. Note also that this constraint implies that leaf versions should be shared amongst different witness versions, as knowing the witness version requires access to the output being spent..
  - Let $k_0 = hash_{TapLeaf}(v \mid\mid compact_size(size\ of\ s) \mid\mid s)$; also call it the *tapleaf hash*.

- For j in *[0,1,…,m-1]*:
  - Let $e_j = c[33+32j:65+32j]$.
  - Let k_{j+1} *depend on whether* $k_j < e_j$'' (lexicographically)**Why are child elements sorted before hashing in the Merkle tree?** By doing so, it is not necessary to reveal the left/right directions along with the hashes in revealed Merkle branches. This is possible because we do not actually care about the position of specific scripts in the tree; only that they are actually committed to.:
    - If $k_j < e_j$: $k_{j+1} = hash_{TapBranch}(k_j \mid\mid e_j)$**Why not use a more efficient hash construction for inner Merkle nodes?** The chosen construction does require two invocations of the SHA256 compression functions, one of which can be avoided in theory (see BIP98). However, it seems preferable to stick to constructions that can be implemented using standard cryptographic primitives, both for implementation simplicity and analyzability. If necessary, a significant part of the second compression function can be optimized out by specialization for 64-byte inputs..
    - If $k_j \geq e_j$: $k_{j+1} = hash_{TapBranch}(e_j \mid\mid k_j)$.
- Let $t = hash_{TapTweak}(p \mid\mid k_m)$.
- If $t \geq$ *0xFFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE BAAEDCE6 AF48A03B BFD25E8C D0364141* (order of secp256k1), fail.
- Let $Q = P + int(t)G$.
- If $q \neq x(Q)$ or $c[0] \,\&\, 1 \neq y(Q) \bmod 2$, fail**Why is it necessary to reveal a bit in a script path spend and check that it matches the parity of the Y coordinate of *Q*?** The parity of the Y coordinate is necessary to lift the X coordinate q to a unique point. While this is not strictly necessary for verifying the taproot commitment as described above, it is necessary to allow batch verification. Alternatively, Q could be forced to have an even Y coordinate, but that would require retrying with different internal public keys (or different messages) until Q has that property. There is no downside to adding the parity bit because otherwise the control block bit would be unused..
- Execute the script, according to the applicable script rules**What are the applicable script rules in script path spends?** BIP342 specifies validity rules that apply for leaf version 0xc0, but future proposals can introduce rules for other leaf versions., using the witness stack elements excluding the script s, the control block c, and the annex a if present, as initial stack. This implies that for the future leaf versions (non-*0xC0*) the execution must succeed.**Why we need to success on future leaf version validation** This is required to enable future leaf versions as soft forks.

q is referred to as *taproot output key* and p as *taproot internal key*.

Signature validation rules

We first define a reusable common signature message calculation function, followed by the actual signature validation as it's used in key path spending.

Common signature message

The function *SigMsg(hash_type, ext_flag)* computes the common portion of the message being signed as a byte array. It is implicitly also a function of the spending transaction and the outputs it spends, but these are not listed to keep notation simple.

The parameter *hash_type* is an 8-bit unsigned value. The `SIGHASH` encodings from the legacy script system are reused, including `SIGHASH_ALL`, `SIGHASH_NONE`, `SIGHASH_SINGLE`, and `SIGHASH_ANYONECANPAY`. We define a new *hashtype* `SIGHASH_DEFAULT` (value *0x00*) which results in signing over the whole transaction just as for `SIGHASH_ALL`. The following restrictions apply, which cause validation failure if violated:

- Using any undefined *hash_type* (not *0x00*, *0x01*, *0x02*, *0x03*, *0x81*, *0x82*, or *0x83***Why reject unknown *hash_type* values?** By doing so, it is easier to reason about the worst case amount of signature hashing an implementation with adequate caching must perform.).
- Using `SIGHASH_SINGLE` without a "corresponding output" (an output with the same index as the input being verified).

The parameter *ext_flag* is an integer in range 0-127, and is used for indicating (in the message) that extensions are appended to the output of *SigMsg()***What extensions use the *ext_flag* mechanism?** BIP342 reuses the same common signature message algorithm, but adds BIP342-specific data at the end, which is indicated using *ext_flag = 1*..

If the parameters take acceptable values, the message is the concatenation of the following data, in order (with byte size of each item listed in parentheses). Numerical values in 2, 4, or 8-byte are encoded in little-endian.

- Control:
  - *hash_type* (1).
- Transaction data:
  - *nVersion* (4): the *nVersion* of the transaction.
  - *nLockTime* (4): the *nLockTime* of the transaction.
  - If the *hash_type & 0x80* does not equal `SIGHASH_ANYONECANPAY`:
    - *sha_prevouts* (32): the SHA256 of the serialization of all input outpoints.
    - *sha_amounts* (32): the SHA256 of the serialization of all input amounts.
    - *sha_scriptpubkeys* (32): the SHA256 of all spent outputs' *scriptPubKeys*, serialized as script inside `CTxOut`.
    - *sha_sequences* (32): the SHA256 of the serialization of all input *nSequence*.
  - If *hash_type & 3* does not equal `SIGHASH_NONE` or `SIGHASH_SINGLE`:
    - *sha_outputs* (32): the SHA256 of the serialization of all outputs in `CTxOut` format.
- Data about this input:
  - *spend_type* (1): equal to *(ext_flag * 2) + annex_present*, where *annex_present* is 0 if no annex is present, or 1 otherwise (the original witness stack has two or more witness elements, and the first byte of the last element is *0x50*)
  - If *hash_type & 0x80* equals `SIGHASH_ANYONECANPAY`:
    - *outpoint* (36): the `COutPoint` of this input (32-byte hash + 4-byte little-endian).
    - *amount* (8): value of the previous output spent by this input.
    - *scriptPubKey* (35): *scriptPubKey* of the previous output spent by this input, serialized as script inside `CTxOut`. Its size is always 35 bytes.
    - *nSequence* (4): *nSequence* of this input.
  - If *hash_type & 0x80* does not equal `SIGHASH_ANYONECANPAY`:
    - *input_index* (4): index of this input in the transaction input vector. Index of the first input is 0.
  - If an annex is present (the lowest bit of *spend_type* is set):
    - *sha_annex* (32): the SHA256 of *(compact_size(size of annex) || annex)*, where *annex* includes the mandatory *0x50* prefix.

- Data about this output:
    - If *hash_type & 3* equals `SIGHASH_SINGLE`:
        - *sha_single_output* (32): the SHA256 of the corresponding output in `CTxOut` format.

The total length of *SigMsg()* is at most *206* bytes**What is the output length of *SigMsg()*?** The total length of *SigMsg()* can be computed using the following formula: *174 - is_anyonecanpay * 49 - is_none * 32 + has_annex * 32*.. Note that this does not include the size of sub-hashes such as *sha_prevouts*, which may be cached across signatures of the same transaction.

In summary, the semantics of the BIP143 sighash types remain unchanged, except the following:

1. The way and order of serialization is changed.**Why is the serialization in the signature message changed?** Hashes that go into the signature message and the message itself are now computed with a single SHA256 invocation instead of double SHA256. There is no expected security improvement by doubling SHA256 because this only protects against length-extension attacks against SHA256 which are not a concern for signature messages because there is no secret data. Therefore doubling SHA256 is a waste of resources. The message computation now follows a logical order with transaction level data first, then input data and output data. This allows to efficiently cache the transaction part of the message across different inputs using the SHA256 midstate. Additionally, sub-hashes can be skipped when calculating the message (for example `sha_prevouts` if `SIGHASH_ANYONECANPAY` is set) instead of setting them to zero and then hashing them as in BIP143. Despite that, collisions are made impossible by committing to the length of the data (implicit in *hash_type* and *spend_type*) before the variable length data.
2. The signature message commits to the *scriptPubKey* of the spent output and if the `SIGHASH_ANYONECANPAY` flag is not set, the message commits to the *scriptPubKey*s of *all* outputs spent by the transaction. **Why does the signature message commit to the *scriptPubKey*?** This prevents lying to offline signing devices about output being spent, even when the actually executed script (*scriptCode* in BIP143) is correct. This means it's possible to compactly prove to a hardware wallet what (unused) execution paths existed. Moreover, committing to all spent *scriptPubKey*s helps offline signing devices to determine the subset that belong to its own wallet. This is useful in automated coinjoins..
3. If the `SIGHASH_ANYONECANPAY` flag is not set, the message commits to the amounts of *all* transaction inputs.**Why does the signature message commit to the amounts of all transaction inputs?** This eliminates the possibility to lie to offline signing devices about the fee of a transaction.
4. The signature message commits to all input *nSequence* if `SIGHASH_NONE` or `SIGHASH_SINGLE` are set (unless `SIGHASH_ANYONECANPAY` is set as well).**Why does the signature message commit to all input *nSequence* if `SIGHASH_SINGLE` or `SIGHASH_NONE` are set?** Because setting them already makes the message commit to the `prevouts` part of all transaction inputs, it is not useful to treat the *nSequence* any different. Moreover, this change makes *nSequence* consistent with the view that `SIGHASH_SINGLE` and `SIGHASH_NONE` only modify the signature message with respect to transaction outputs and not inputs.
5. The signature message includes commitments to the taproot-specific data *spend_type* and *annex* (if present).

Taproot key path spending signature validation

A Taproot signature is a 64-byte Schnorr signature, as defined in BIP340, with the sighash byte appended in the usual Bitcoin fashion. This sighash byte is optional. If omitted, the resulting signatures are 64 bytes, and a SIGHASH_DEFAULT mode is implied.

To validate a signature *sig* with public key *q*:

- If the *sig* is 64 bytes long, return *Verify(q, $hash_{TapSighash}$(0x00 || SigMsg(0x00, 0)), sig)***Why is the input to $hash_{TapSighash}$ prefixed with 0x00?** This prefix is called the sighash epoch, and allows reusing the $hash_{TapSighash}$ tagged hash in future signature algorithms that make invasive changes to how hashing is performed (as opposed to the *ext_flag* mechanism that is used for incremental extensions). An alternative is having them use a different tag, but supporting a growing number of tags may become undesirable., where *Verify* is defined in BIP340.
- If the *sig* is 65 bytes long, return *sig[64] ≠ 0x00****Why can the `hash_type` not be `0x00` in 65-byte signatures?*** *Permitting that would enable malleating (by third parties, including miners) 64-byte signatures into 65-byte ones, resulting in a different `wtxid` and a different fee rate than the creator intended. and Verify(q, $hash_{TapSighash}$(0x00 || SigMsg(sig[64], 0)), sig[0:64])*.
- Otherwise, fail**Why permit two signature lengths?** By making the most common type of `hash_type` implicit, a byte can often be saved..

Constructing and spending Taproot outputs

This section discusses how to construct and spend Taproot outputs. It only affects wallet software that chooses to implement receiving and spending, and is not consensus critical in any way.

Conceptually, every Taproot output corresponds to a combination of a single public key condition (the internal key), and zero or more general conditions encoded in scripts organized in a tree. Satisfying any of these conditions is sufficient to spend the output.

Initial steps The first step is determining what the internal key and the organization of the rest of the scripts should be. The specifics are likely application dependent, but here are some general guidelines:

- When deciding between scripts with conditionals (`OP_IF` etc.) and splitting them up into multiple scripts (each corresponding to one execution path through the original script), it is generally preferable to pick the latter.
- When a single condition requires signatures with multiple keys, key aggregation techniques like MuSig can be used to combine them into a single key. The details are out of scope for this document, but note that this may complicate the signing procedure.
- If one or more of the spending conditions consist of just a single key (after aggregation), the most likely one should be made the internal key. If no such condition exists, it may be worthwhile adding one that consists of an aggregation of all keys participating in all scripts combined; effectively adding an "everyone agrees" branch. If that is inacceptable, pick as internal key a "Nothing Up My Sleeve" (NUMS) point, i.e., a point with unknown discrete logarithm. One example of such a point is *H = lift_x(0x50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0)* which is constructed by taking the hash of the standard uncompressed encoding of the secp256k1 base point *G* as X coordinate. In order to avoid leaking the information that key path spending is not possible it is recommended to pick a fresh integer *r* in the range *0…n-1* uniformly at random and use *H + rG* as internal key. It is possible to prove that this internal key does not have a known discrete logarithm with respect to *G* by revealing *r* to a verifier who can then reconstruct how the internal key was created.
- If the spending conditions do not require a script path, the output key should commit to an unspendable script path instead of having no script path. This can be achieved by computing the output key point as

$Q = P + int(hash_{TapTweak}(bytes(P)))G$. **Why should the output key always have a taproot commitment, even if there is no script path?**

If the taproot output key is an aggregate of keys, there is the possibility for a malicious party to add a script path without being noticed by the other parties. This allows to bypass the multiparty policy and to steal the coin. MuSig key aggregation does not have this issue because it already causes the internal key to be randomized.

The attack works as follows: Assume Alice and Mallory want to aggregate their keys into a taproot output key without a script path. In order to prevent key cancellation and related attacks they use MSDL-pop instead of MuSig. The MSDL-pop protocol requires all parties to provide a proof of possession of their corresponding secret key and the aggregated key is just the sum of the individual keys. After Mallory receives Alice's key A, Mallory creates $M = M_0 + int(t)G$ where M_0 is Mallory's original key and t allows a script path spend with internal key $P = A + M_0$ and a script that only contains Mallory's key. Mallory sends a proof of possession of M to Alice and both parties compute output key $Q = A + M = P + int(t)G$. Alice will not be able to notice the script path, but Mallory can unilaterally spend any coin with output key Q.

- The remaining scripts should be organized into the leaves of a binary tree. This can be a balanced tree if each of the conditions these scripts correspond to are equally likely. If probabilities for each condition are known, consider constructing the tree as a Huffman tree.

Computing the output script Once the spending conditions are split into an internal key `internal_pubkey` and a binary tree whose leaves are (leaf_version, script) tuples, the output script can be computed using the Python3 algorithms below. These algorithms take advantage of helper functions from the BIP340 reference code for integer conversion, point multiplication, and tagged hashes.

First, we define `taproot_tweak_pubkey` for 32-byte BIP340 public key arrays. The function returns a bit indicating the tweaked public key's Y coordinate as well as the public key byte array. The parity bit will be required for spending the output with a script path. In order to allow spending with the key path, we define `taproot_tweak_seckey` to compute the secret key for a tweaked public key. For any byte string `h` it holds that `taproot_tweak_pubkey(pubkey_gen(seckey), h)[1] == pubkey_gen(taproot_tweak_seckey(seckey, h))`.

Note that because tweaks are applied to 32-byte public keys, `taproot_tweak_seckey` may need to negate the secret key before applying the tweak.

```
def taproot_tweak_pubkey(pubkey, h):
    t = int_from_bytes(tagged_hash("TapTweak", pubkey + h))
    if t >= SECP256K1_ORDER:
        raise ValueError
    P = lift_x(int_from_bytes(pubkey))
    if P is None:
        raise ValueError
    Q = point_add(P, point_mul(G, t))
    return 0 if has_even_y(Q) else 1, bytes_from_int(x(Q))

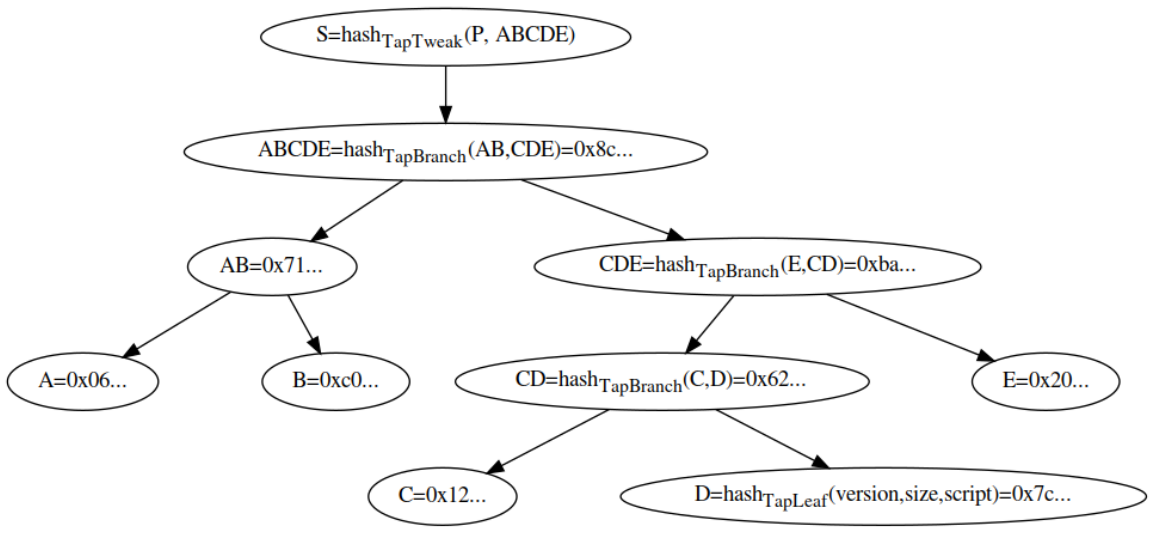
def taproot_tweak_seckey(seckey0, h):
    seckey0 = int_from_bytes(seckey0)
    P = point_mul(G, seckey0)
    seckey = seckey0 if has_even_y(P) else SECP256K1_ORDER - seckey0
    t = int_from_bytes(tagged_hash("TapTweak", bytes_from_int(x(P)) + h))
    if t >= SECP256K1_ORDER:
```

```
        raise ValueError
    return bytes_from_int((seckey + t) % SECP256K1_ORDER)
```

The following function, `taproot_output_script`, returns a byte array with the scriptPubKey (see BIP141). `ser_script` refers to a function that prefixes its input with a CompactSize-encoded length.

```
def taproot_tree_helper(script_tree):
    if isinstance(script_tree, tuple):
        leaf_version, script = script_tree
        h = tagged_hash("TapLeaf", bytes([leaf_version]) + ser_script(script))
        return ([((leaf_version, script), bytes())], h)
    left, left_h = taproot_tree_helper(script_tree[0])
    right, right_h = taproot_tree_helper(script_tree[1])
    ret = [(l, c + right_h) for l, c in left] + [(l, c + left_h) for l, c in right]
    if right_h < left_h:
        left_h, right_h = right_h, left_h
    return (ret, tagged_hash("TapBranch", left_h + right_h))

def taproot_output_script(internal_pubkey, script_tree):
    """Given a internal public key and a tree of scripts, compute the output script.
    script_tree is either:
     - a (leaf_version, script) tuple (leaf_version is 0xc0 for [[bip-0342.mediawiki|BIP342]]
        scripts)
     - a list of two elements, each with the same structure as script_tree itself
     - None
    """
    if script_tree is None:
        h = bytes()
    else:
        _, h = taproot_tree_helper(script_tree)
    _, output_pubkey = taproot_tweak_pubkey(internal_pubkey, h)
    return bytes([0x51, 0x20]) + output_pubkey
```



This diagram shows the hashing structure to obtain the tweak from an internal key *P* and a Merkle tree consisting of 5 script leaves. *A*, *B*, *C* and *E* are *TapLeaf* hashes similar to *D* and *AB* is a *TapBranch* hash. Note that when *CDE* is computed *E* is hashed first because *E* is less than *CD*.

To spend this output using script *D*, the control block would contain the following data in this order:

```
    <control byte with leaf version and parity bit> <internal key p> <C> <E> <AB>
```

The TapTweak would then be computed as described above like so:

```
D = tagged_hash("TapLeaf", bytes([leaf_version]) + ser_script(script))
CD = tagged_hash("TapBranch", C + D)
CDE = tagged_hash("TapBranch", E + CD)
ABCDE = tagged_hash("TapBranch", AB + CDE)
TapTweak = tagged_hash("TapTweak", p + ABCDE)
```

Spending using the key path A Taproot output can be spent with the secret key corresponding to the `internal_pubkey`. To do so, a witness stack consists of a single element: a BIP340 signature on the signature hash as defined above, with the secret key tweaked by the same `h` as in the above snippet. See the code below:

```
def taproot_sign_key(script_tree, internal_seckey, hash_type, bip340_aux_rand):
    if script_tree is None:
        h = bytes()
    else:
        _, h = taproot_tree_helper(script_tree)
    output_seckey = taproot_tweak_seckey(internal_seckey, h)
    sig = schnorr_sign(sighash(hash_type), output_seckey, bip340_aux_rand)
    if hash_type != 0:
        sig += bytes([hash_type])
    return [sig]
```

This function returns the witness stack necessary and a `sighash` function to compute the signature hash as defined above (for simplicity, the snippet above ignores passing information like the transaction, the input position, … to the sighashing code).

Spending using one of the scripts A Taproot output can be spent by satisfying any of the scripts used in its construction. To do so, a witness stack consisting of the script's inputs, plus the script itself and the control block are necessary. See the code below:

```
def taproot_sign_script(internal_pubkey, script_tree, script_num, inputs):
    info, h = taproot_tree_helper(script_tree)
    (leaf_version, script), path = info[script_num]
    output_pubkey_y_parity, _ = taproot_tweak_pubkey(internal_pubkey, h)
    pubkey_data = bytes([output_pubkey_y_parity + leaf_version]) + internal_pubkey
    return inputs + [script, pubkey_data + path]
```

Security

Taproot improves the privacy of Bitcoin because instead of revealing all possible conditions for spending an output, only the satisfied spending condition has to be published. Ideally, outputs are spent using the key path which prevents observers from learning the spending conditions of a coin. A key path spend could be a "normal" payment from a single- or multi-signature wallet or the cooperative settlement of hidden multiparty contract.

A script path spend leaks that there is a script path and that the key path was not applicable - for example because the involved parties failed to reach agreement. Moreover, the depth of a script in the Merkle root leaks information including the minimum depth of the tree, which suggests specific wallet software that created the output and helps clustering. Therefore, the privacy of script spends can be improved by deviating from the optimal tree determined by the probability distribution over the leaves.

Just like other existing output types, taproot outputs should never reuse keys, for privacy reasons. This does not only apply to the particular leaf that was used to spend an output but to all leaves committed to in the output. If leaves were reused, it could happen that spending a different output would reuse the same Merkle branches in the Merkle proof. Using fresh keys implies that taproot output construction does not need to take special measures to randomizing leaf positions because they are already randomized due to the branch-sorting Merkle tree construction used in taproot. This does not avoid leaking information through the leaf depth and therefore only applies to balanced (sub-) trees. In addition, every leaf should have a set of keys distinct from every other leaf. The reason for this is to increase leaf entropy and prevent an observer from learning an undisclosed script using brute-force search.

Test vectors

Test vectors for wallet operation (scriptPubKey computation, key path spending, control block construction) can be found here. It consists of two sets of vectors.

- The first "scriptPubKey" tests concern computing the scriptPubKey and (mainnet) BIP350 address given an internal public key, and a script tree. The script tree is encoded as `null` to represent no scripts, a JSON object to represent a leaf node, or a 2-element array to represent an inner node. The control blocks needed for script path spending are also provided for each of the script leaves.
- The second "keyPathSpending" tests consists of a list of test cases, each of which provides an unsigned transaction and the UTXOs it spends. For each of its BIP341 inputs, the internal private key and the Merkle root it was derived from is given, as well as the expected witness to spend it. All signatures are created with an all-zero (0x0000…0000) BIP340 auxiliary randomness array.
- In all cases, hexadecimal values represent byte arrays, not numbers. In particular, that means that provided hash values have the hex digits corresponding to the first bytes first. This differs from the convention used for txids and block hashes, where the hex strings represent numbers, resulting in a reversed order.

Validation test vectors used in the Bitcoin Core unit test framework can be found here.

Rationale

Deployment

This BIP is deployed concurrently with BIP342.

For Bitcoin signet, these BIPs are always active.

For Bitcoin mainnet and testnet3, these BIPs are deployed by "version bits" with the name "taproot" and bit 2, using BIP9 modified to use a lower threshold, with an additional *min_activation_height* parameter and replacing the state transition logic for the DEFINED, STARTED and LOCKED_IN states as follows:

```
case DEFINED:
    if (GetMedianTimePast(block.parent) >= starttime) {
        return STARTED;
    }
    return DEFINED;
```

```
case STARTED:
    int count = 0;
    walk = block;
    for (i = 0; i < 2016; i++) {
        walk = walk.parent;
        if ((walk.nVersion & 0xE0000000) == 0x20000000 && ((walk.nVersion >> bit) & 1) == 1) {
            count++;
        }
    }
    if (count >= threshold) {
        return LOCKED_IN;
    } else if (GetMedianTimePast(block.parent) >= timeout) {
        return FAILED;
    }
    return STARTED;

case LOCKED_IN:
    if (block.nHeight < min_activation_height) {
        return LOCKED_IN;
    }
    return ACTIVE;
```

For Bitcoin mainnet, the starttime is epoch timestamp 1619222400 (midnight 24 April 2021 UTC), timeout is epoch timestamp 1628640000 (midnight 11 August 2021 UTC), the threshold is 1815 blocks (90%) instead of 1916 blocks (95%), and the min_activation_height is block 709632. The deployment did activate at height 709632 on Bitcoin mainnet.

For Bitcoin testnet3, the starttime is epoch timestamp 1619222400 (midnight 24 April 2021 UTC), timeout is epoch timestamp 1628640000 (midnight 11 August 2021 UTC), the threshold is 1512 blocks (75%), and the min_activation_height is block 0. The deployment did activate at height 2011968 on Bitcoin testnet3.

Backwards compatibility

As a soft fork, older software will continue to operate without modification. Non-upgraded nodes, however, will consider all SegWit version 1 witness programs as anyone-can-spend scripts. They are strongly encouraged to upgrade in order to fully validate the new programs.

Non-upgraded wallets can receive and send bitcoin from non-upgraded and upgraded wallets using SegWit version 0 programs, traditional pay-to-pubkey-hash, etc. Depending on the implementation non-upgraded wallets may be able to send to Segwit version 1 programs if they support sending to BIP350 Bech32m addresses.

Acknowledgements

This document is the result of discussions around script and signature improvements with many people, and had direct contributions from Greg Maxwell and others. It further builds on top of earlier published proposals such as Taproot by Greg Maxwell, and Merkle branch constructions by Russell O'Connor, Johnson Lau, and Mark Friedenbach.

The authors wish the thank Arik Sosman for suggesting to sort Merkle node children before hashes, removing the need to transfer the position in the tree, as well as all those who provided valuable feedback and reviews, including the participants of the structured reviews.

```
  BIP: 342
  Layer: Consensus (soft fork)
  Title: Validation of Taproot Scripts
  Authors: Pieter Wuille <pieter.wuille@gmail.com>
           Jonas Nick <jonasd.nick@gmail.com>
           Anthony Towns <aj@erisian.com.au>
  Status: Deployed
  Type: Specification
  Assigned: 2020-01-19
  License: BSD-3-Clause
  Discussion: 2019-05-06: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2019-May/
016914.html [bitcoin-dev] Taproot proposal
  Requires: 340, 341
```

BIP 342

Introduction

Abstract

This document specifies the semantics of the initial scripting system under BIP341.

Copyright

This document is licensed under the 3-clause BSD license.

Motivation

BIP341 proposes improvements to just the script structure, but some of its goals are incompatible with the semantics of certain opcodes within the scripting language itself. While it is possible to deal with these in separate optional improvements, their impact is not guaranteed unless they are addressed simultaneously with BIP341 itself.

Specifically, the goal is making **Schnorr signatures**, **batch validation**, and **signature hash** improvements available to spends that use the script system as well.

Design

In order to achieve these goals, signature opcodes `OP_CHECKSIG` and `OP_CHECKSIGVERIFY` are modified to verify Schnorr signatures as specified in BIP340 and to use a signature message algorithm based on the common message calculation in BIP341. The tapscript signature message also simplifies `OP_CODESEPARATOR` handling and makes it more efficient.

The inefficient `OP_CHECKMULTISIG` and `OP_CHECKMULTISIGVERIFY` opcodes are disabled. Instead, a new opcode `OP_CHECKSIGADD` is introduced to allow creating the same multisignature policies in a batch-verifiable way.

Tapscript uses a new, simpler signature opcode limit fixing complicated interactions with transaction weight. Furthermore, a potential malleability vector is eliminated by requiring MINIMALIF.

Tapscript can be upgraded through soft forks by defining unknown key types, for example to add new `hash_types` or signature algorithms. Additionally, the new tapscript `OP_SUCCESS` opcodes allow introducing new opcodes more cleanly than through `OP_NOP`.

Specification

The rules below only apply when validating a transaction input for which all of the conditions below are true:

- The transaction input is a **segregated witness spend** (i.e., the scriptPubKey contains a witness program as defined in BIP141).
- It is a **taproot spend** as defined in BIP341 (i.e., the witness version is 1, the witness program is 32 bytes, and it is not P2SH wrapped).
- It is a **script path spend** as defined in BIP341 (i.e., after removing the optional annex from the witness stack, two or more stack elements remain).
- The leaf version is *0xc0* (i.e. the first byte of the last witness element after removing the optional annex is *0xc0* or *0xc1*), marking it as a **tapscript spend**.

Validation of such inputs must be equivalent to performing the following steps in the specified order.

1. If the input is invalid due to BIP141 or BIP341, fail.
2. The script as defined in BIP341 (i.e., the penultimate witness stack element after removing the optional annex) is called the **tapscript** and is decoded into opcodes, one by one:
    1. If any opcode numbered *80*, *98*, *126-129*, *131-134*, *137-138*, *141-142*, *149-153*, *187-254* is encountered, validation succeeds (none of the rules below apply). This is true even if later bytes in the tapscript would fail to decode otherwise. These opcodes are renamed to `OP_SUCCESS80`, …, `OP_SUCCESS254`, and collectively known as `OP_SUCCESSx`**`OP_SUCCESSx`** `OP_SUCCESSx` is a mechanism to upgrade the Script system. Using an `OP_SUCCESSx` before its meaning is defined by a softfork is insecure and leads to fund loss. The inclusion of `OP_SUCCESSx` in a script will pass it unconditionally. It precedes any script execution rules to avoid the difficulties in specifying various edge cases, for example: `OP_SUCCESSx` in a script with an input stack larger than 1000 elements, `OP_SUCCESSx` after too many signature opcodes, or even scripts with conditionals lacking `OP_ENDIF`. The mere existence of an `OP_SUCCESSx` anywhere in the script will guarantee a pass for all such cases. `OP_SUCCESSx` are similar to the `OP_RETURN` in very early bitcoin versions (v0.1 up to and including v0.3.5). The original `OP_RETURN` terminates script execution immediately, and return pass or fail based on the top stack element at the moment of termination. This was one of a major design flaws in the original bitcoin protocol as it permitted unconditional third party theft by placing an `OP_RETURN` in `scriptSig`. This is not a concern in the present proposal since it is not possible for a third party to inject an `OP_SUCCESSx` to the validation process, as the `OP_SUCCESSx` is part of the script (and thus committed to by the taproot output), implying the consent of the coin owner. `OP_SUCCESSx` can be used for a variety of upgrade possibilities:

- An `OP_SUCCESSx` could be turned into a functional opcode through a softfork. Unlike `OP_NOPx`-derived opcodes which only have read-only access to the stack, `OP_SUCCESSx` may also write to the stack. Any

rule changes to an `OP_SUCCESSx`-containing script may only turn a valid script into an invalid one, and this is always achievable with softforks.
- Since `OP_SUCCESSx` precedes size check of initial stack and push opcodes, an `OP_SUCCESSx`-derived opcode requiring stack elements bigger than 520 bytes may uplift the limit in a softfork.
- `OP_SUCCESSx` may also redefine the behavior of existing opcodes so they could work together with the new opcode. For example, if an `OP_SUCCESSx`-derived opcode works with 64-bit integers, it may also allow the existing arithmetic opcodes in the *same script* to do the same.
- Given that `OP_SUCCESSx` even causes potentially unparseable scripts to pass, it can be used to introduce multi-byte opcodes, or even a completely new scripting language when prefixed with a specific `OP_SUCCESSx` opcode..

1. 1. If any push opcode fails to decode because it would extend past the end of the tapscript, fail.
2. If the **initial stack** as defined in BIP341 (i.e., the witness stack after removing both the optional annex and the two last stack elements after that) violates any resource limits (stack size, and size of the elements in the stack; see "Resource Limits" below), fail. Note that this check can be bypassed using `OP_SUCCESSx`.
3. The tapscript is executed according to the rules in the following section, with the initial stack as input.
    1. If execution fails for any reason, fail.
    2. If the execution results in anything but exactly one element on the stack which evaluates to true with `CastToBool()`, fail.
4. If this step is reached without encountering a failure, validation succeeds.

Script execution

The execution rules for tapscript are based on those for P2WSH according to BIP141, including the `OP_CHECKLOCKTIMEVERIFY` and `OP_CHECKSEQUENCEVERIFY` opcodes defined in BIP65 and BIP112, but with the following modifications:

- **Disabled script opcodes** The following script opcodes are disabled in tapscript: `OP_CHECKMULTISIG` and `OP_CHECKMULTISIGVERIFY`**Why are `OP_CHECKMULTISIG` and `OP_CHECKMULTISIGVERIFY` disabled, and not turned into OP_SUCCESSx?** This is a precaution to make sure people who accidentally keep using `OP_CHECKMULTISIG` in Tapscript notice a problem immediately. It also avoids the complication of script disassemblers needing to become context-dependent.. The disabled opcodes behave in the same way as `OP_RETURN`, by failing and terminating the script immediately when executed, and being ignored when found in unexecuted branch of the script.
- **Consensus-enforced MINIMALIF** The MINIMALIF rules, which are only a standardness rule in P2WSH, are consensus enforced in tapscript. This means that the input argument to the `OP_IF` and `OP_NOTIF` opcodes must be either exactly 0 (the empty vector) or exactly 1 (the one-byte vector with value 1)**Why make MINIMALIF consensus?** This makes it considerably easier to write non-malleable scripts that take branch information from the stack..
- **OP_SUCCESSx opcodes** As listed above, some opcodes are renamed to `OP_SUCCESSx`, and make the script unconditionally valid.
- **Signature opcodes**. The `OP_CHECKSIG` and `OP_CHECKSIGVERIFY` are modified to operate on Schnorr public keys and signatures (see BIP340) instead of ECDSA, and a new opcode `OP_CHECKSIGADD` is added.
    - The opcode 186 (`0xba`) is named as `OP_CHECKSIGADD`. **`OP_CHECKSIGADD`** This opcode is added to compensate for the loss of `OP_CHECKMULTISIG`-like opcodes, which are incompatible with batch verification. `OP_CHECKSIGADD` is functionally equivalent to `OP_ROT OP_SWAP OP_CHECKSIG OP_ADD`,

but only takes 1 byte. All `CScriptNum`-related behaviours of `OP_ADD` are also applicable to `OP_CHECKSIGADD`.**Alternatives to `CHECKMULTISIG`** There are multiple ways of implementing a threshold *k*-of-*n* policy using Taproot and Tapscript:

- **Using a single `OP_CHECKSIGADD`-based script** A `CHECKMULTISIG` script `m <pubkey_1> ... <pubkey_n> n CHECKMULTISIG` with witness `0 <signature_1> ... <signature_m>` can be rewritten as script `<pubkey_1> CHECKSIG <pubkey_2> CHECKSIGADD ... <pubkey_n> CHECKSIGADD m NUMEQUAL` with witness `<w_n> ... <w_1>`. Every witness element `w_i` is either a signature corresponding to `pubkey_i` or an empty vector. A similar `CHECKMULTISIGVERIFY` script can be translated to BIP342 by replacing `NUMEQUAL` with `NUMEQUALVERIFY`. This approach has very similar characteristics to the existing `OP_CHECKMULTISIG`-based scripts.
- **Using a *k*-of-*k* script for every combination** A *k*-of-*n* policy can be implemented by splitting the script into several leaves of the Merkle tree, each implementing a *k*-of-*k* policy using `<pubkey_1> CHECKSIGVERIFY ... <pubkey_(n-1)> CHECKSIGVERIFY <pubkey_n> CHECKSIG`. This may be preferable for privacy reasons over the previous approach, as it only exposes the participating public keys, but it is only more cost effective for small values of *k* (1-of-*n* for any *n*, 2-of-*n* for $n \geq 6$, 3-of-*n* for $n \geq 9$, …). Furthermore, the signatures here commit to the branch used, which means signers need to be aware of which other signers will be participating, or produce signatures for each of the tree leaves.
- **Using an aggregated public key for every combination** Instead of building a tree where every leaf consists of *k* public keys, it is possible instead build a tree where every leaf contains a single *aggregate* of those *k* keys using MuSig. This approach is far more efficient, but does require a 3-round interactive signing protocol to jointly produce the (single) signature.
- **Native Schnorr threshold signatures** Multisig policies can also be realized with threshold signatures using verifiable secret sharing. This results in outputs and inputs that are indistinguishable from single-key payments, but at the cost of needing an interactive protocol (and associated backup procedures) before determining the address to send to.

Rules for signature opcodes

The following rules apply to `OP_CHECKSIG`, `OP_CHECKSIGVERIFY`, and `OP_CHECKSIGADD`.

- For `OP_CHECKSIGVERIFY` and `OP_CHECKSIG`, the public key (top element) and a signature (second to top element) are popped from the stack.
  - If fewer than 2 elements are on the stack, the script MUST fail and terminate immediately.
- For `OP_CHECKSIGADD`, the public key (top element), a `CScriptNum` `n` (second to top element), and a signature (third to top element) are popped from the stack.
  - If fewer than 3 elements are on the stack, the script MUST fail and terminate immediately.
  - If `n` is larger than 4 bytes, the script MUST fail and terminate immediately.
- If the public key size is zero, the script MUST fail and terminate immediately.
- If the public key size is 32 bytes, it is considered to be a public key as described in BIP340:
  - If the signature is not the empty vector, the signature is validated against the public key (see the next subsection). Validation failure in this case immediately terminates script execution with failure.
- If the public key size is not zero and not 32 bytes, the public key is of an *unknown public key type*.**Unknown public key types** allow adding new signature validation rules through softforks. A softfork could add actual signature validation which either passes or makes the script fail and terminate immediately. This way, new `SIGHASH` modes can be added, as well as NOINPUT-tagged public keys and a public key constant which is replaced by the taproot internal key for signature validation. and no

actual signature verification is applied. During script execution of signature opcodes they behave exactly as known public key types except that signature validation is considered to be successful.

- If the script did not fail and terminate before this step, regardless of the public key type:
  - If the signature is the empty vector:
    - For `OP_CHECKSIGVERIFY`, the script MUST fail and terminate immediately.
    - For `OP_CHECKSIG`, an empty vector is pushed onto the stack, and execution continues with the next opcode.
    - For `OP_CHECKSIGADD`, a `CScriptNum` with value `n` is pushed onto the stack, and execution continues with the next opcode.
  - If the signature is not the empty vector, the opcode is counted towards the sigops budget (see further).
    - For `OP_CHECKSIGVERIFY`, execution continues without any further changes to the stack.
    - For `OP_CHECKSIG`, a 1-byte value `0x01` is pushed onto the stack.
    - For `OP_CHECKSIGADD`, a `CScriptNum` with value of `n + 1` is pushed onto the stack.

Common Signature Message Extension

We define the tapscript message extension *ext* to BIP341 Common Signature Message, indicated by *ext_flag = 1*:

- *tapleaf_hash* (32): the tapleaf hash as defined in BIP341
- *key_version* (1): a constant value *0x00* representing the current version of public keys in the tapscript signature opcode execution.
- *codesep_pos* (4): the opcode position of the last executed `OP_CODESEPARATOR` before the currently executed signature opcode, with the value in little endian (or *0xffffffff* if none executed). The first opcode in a script has a position of 0. A multi-byte push opcode is counted as one opcode, regardless of the size of data being pushed. Opcodes in parsed but unexecuted branches count towards this value as well.

Signature validation

To validate a signature *sig* with public key *p*:

- Compute the tapscript message extension *ext* described above.
- If the *sig* is 64 bytes long, return $Verify(p, hash_{TapSighash}(0x00 \,||\, SigMsg(0x00, 1) \,||\, ext), sig)$, where *Verify* is defined in BIP340.
- If the *sig* is 65 bytes long, return $sig[64] \neq 0x00 \text{ and } Verify(p, hash_{TapSighash}(0x00 \,||\, SigMsg(sig[64], 1) \,||\, ext), sig[0:64])$.
- Otherwise, fail.

In summary, the semantics of signature validation is identical to BIP340, except the following:

1. The signature message includes the tapscript-specific data *key_version*.**Why does the signature message commit to the *key_version*?** This is for future extensions that define unknown public key types, making sure signatures can't be moved from one key type to another.
2. The signature message commits to the executed script through the *tapleaf_hash* which includes the leaf version and script instead of *scriptCode*. This implies that this commitment is unaffected by `OP_CODESEPARATOR`.
3. The signature message includes the opcode position of the last executed `OP_CODESEPARATOR`.**Why does the signature message include the position of the last executed `OP_CODESEPARATOR`?** This allows

continuing to use `OP_CODESEPARATOR` to sign the executed path of the script. Because the `codeseparator_position` is the last input to the hash, the SHA256 midstate can be efficiently cached for multiple `OP_CODESEPARATOR`s in a single script. In contrast, the BIP143 handling of `OP_CODESEPARATOR` is to commit to the executed script only from the last executed `OP_CODESEPARATOR` onwards which requires unnecessary rehashing of the script. It should be noted that the one known `OP_CODESEPARATOR` use case of saving a second public key push in a script by sharing the first one between two code branches can be most likely expressed even cheaper by moving each branch into a separate taproot leaf.

Resource limits

In addition to changing the semantics of a number of opcodes, there are also some changes to the resource limitations:

- **Script size limit** The maximum script size of 10000 bytes does not apply. Their size is only implicitly bounded by the block weight limit.**Why is a limit on script size no longer needed?** Since there is no `scriptCode` directly included in the signature hash (only indirectly through a precomputable tapleaf hash), the CPU time spent on a signature check is no longer proportional to the size of the script being executed.
- **Non-push opcodes limit** The maximum non-push opcodes limit of 201 per script does not apply.**Why is a limit on the number of opcodes no longer needed?** An opcode limit only helps to the extent that it can prevent data structures from growing unboundedly during execution (both because of memory usage, and because of time that may grow in proportion to the size of those structures). The size of stack and altstack is already independently limited. By using O(1) logic for `OP_IF`, `OP_NOTIF`, `OP_ELSE`, and `OP_ENDIF` as suggested here and implemented here, the only other instance can be avoided as well.
- **Sigops limit** The sigops in tapscripts do not count towards the block-wide limit of 80000 (weighted). Instead, there is a per-script sigops *budget*. The budget equals 50 + the total serialized size in bytes of the transaction input's witness (including the `CompactSize` prefix). Executing a signature opcode (`OP_CHECKSIG`, `OP_CHECKSIGVERIFY`, or `OP_CHECKSIGADD`) with a non-empty signature decrements the budget by 50. If that brings the budget below zero, the script fails immediately. Signature opcodes with unknown public key type and non-empty signature are also counted.**The tapscript sigop limit** The signature opcode limit protects against scripts which are slow to verify due to excessively many signature operations. In tapscript the number of signature opcodes does not count towards the BIP141 or legacy sigop limit. The old sigop limit makes transaction selection in block construction unnecessarily difficult because it is a second constraint in addition to weight. Instead, the number of tapscript signature opcodes is limited by witness weight. Additionally, the limit applies to the transaction input instead of the block and only actually executed signature opcodes are counted. Tapscript execution allows one signature opcode per 50 witness weight units plus one free signature opcode.**Parameter choice of the sigop limit** Regular witnesses are unaffected by the limit as their weight is composed of public key and (`SIGHASH_ALL`) signature pairs with *33 + 65* weight units each (which includes a 1 weight unit `CompactSize` tag). This is also the case if public keys are reused in the script because a signature's weight alone is 65 or 66 weight units. However, the limit increases the fees of abnormal scripts with duplicate signatures (and public keys) by requiring additional weight. The weight per sigop factor 50 corresponds to the ratio of BIP141 block limits: 4 mega weight units divided by 80,000 sigops. The "free" signature opcode permitted by the limit exists to account for the weight of the non-witness parts of the transaction input.**Why are only signature opcodes counted toward the budget, and not for example hashing opcodes or other expensive operations?** It turns out that the CPU cost per witness byte for verification of a script consisting of the maximum density of signature checking opcodes (taking the 50

WU/sigop limit into account) is already very close to that of scripts packed with other opcodes, including hashing opcodes (taking the 520 byte stack element limit into account) and `OP_ROLL` (taking the 1000 stack element limit into account). That said, the construction is very flexible, and allows adding new signature opcodes like `CHECKSIGFROMSTACK` to count towards the limit through a soft fork. Even if in the future new opcodes are introduced which change normal script cost there is no need to stuff the witness with meaningless data. Instead, the taproot annex can be used to add weight to the witness without increasing the actual witness size..

- **Stack + altstack element count limit** The existing limit of 1000 elements in the stack and altstack together after every executed opcode remains. It is extended to also apply to the size of initial stack.
- **Stack element size limit** The existing limit of maximum 520 bytes per stack element remains, both in the initial stack and in push opcodes.

Rationale

Deployment

This proposal is deployed identically to Taproot (BIP341).

Examples

The Taproot (BIP341) test vectors also contain examples for Tapscript execution.

Acknowledgements

This document is the result of many discussions and contains contributions by a number of people. The authors wish to thank all those who provided valuable feedback and reviews, including the participants of the structured reviews.

Keys, Seeds, and Derivation

A wallet is a way to turn one secret into many keys, and those keys into the scripts you actually receive on. This chapter is that pipeline.

BIP 32 is the root: hierarchical deterministic wallets, extended keys, and the child-key derivation everyone else builds on. BIP 38 encrypts a key with a passphrase. BIP 39 turns entropy into a mnemonic sentence. BIP 43 and BIP 44 add a purpose field and a multi-account tree so different applications can share one seed without colliding.

The later derivation BIPs pick a script type and pin a path for it: BIP 49 for P2WPKH nested in P2SH, BIP 84 for native P2WPKH, BIP 86 for single-key P2TR, BIP 48 and BIP 87 for multisig. BIP 85 is the odd one out in a useful way — it derives more entropy from a BIP 32 tree, so one seed can generate other seeds.

BIP 129 is how several devices agree on a multisig setup without a compromised coordinator. BIP 328 is the derivation scheme for a MuSig2 aggregate key — the wallet-side companion to BIP 327 in the payments chapter.

None of these documents change consensus. They are how wallets agree with each other about which key to use. If two wallets share a seed and disagree on the path, they will not see the same coins.

In this chapter:

- BIP 32 — Hierarchical Deterministic Wallets
- BIP 38 — Passphrase-protected private key
- BIP 39 — Mnemonic code for generating deterministic keys
- BIP 43 — Purpose Field for Deterministic Wallets
- BIP 44 — Multi-Account Hierarchy
- BIP 49 — P2WPKH-nested-in-P2SH derivation
- BIP 84 — P2WPKH derivation
- BIP 85 — Deterministic Entropy From BIP32 Keychains
- BIP 86 — Key Derivation for Single Key P2TR Outputs
- BIP 87 — Hierarchy for Deterministic Multisig Wallets
- BIP 48 — Multi-Script Hierarchy for Multi-Sig Wallets
- BIP 129 — Bitcoin Secure Multisig Setup
- BIP 328 — Derivation Scheme for MuSig2 Aggregate Keys

RECENT CHANGES:

- (16 Apr 2013) Added private derivation for i ≥ 0x80000000 (less risk of parent private key leakage)
- (30 Apr 2013) Switched from multiplication by I_L to addition of I_L (faster, easier implementation)
- (25 May 2013) Added test vectors
- (15 Jan 2014) Rename keys with index ≥ 0x80000000 to hardened keys, and add explicit conversion functions.
- (24 Feb 2017) Added test vectors for hardened derivation with leading zeros
- (4 Nov 2020) Added new test vectors for hardened derivation with leading zeros

```
BIP: 32
Layer: Applications
Title: Hierarchical Deterministic Wallets
Authors: Pieter Wuille <pieter.wuille@gmail.com>
Status: Deployed
Type: Informational
Assigned: 2012-02-11
License: BSD-2-Clause
```

BIP 32

Abstract

This document describes hierarchical deterministic wallets (or "HD Wallets"): wallets which can be shared partially or entirely with different systems, each with or without the ability to spend coins.

The specification is intended to set a standard for deterministic wallets that can be interchanged between different clients. Although the wallets described here have many features, not all are required by supporting clients.

The specification consists of two parts. In the first part, a system for deriving a tree of keypairs from a single seed is presented. The second part demonstrates how to build a wallet structure on top of such a tree.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

The Bitcoin reference client uses randomly generated keys. In order to avoid the necessity for a backup after every transaction, (by default) 100 keys are cached in a pool of reserve keys. Still, these wallets are not intended to be shared and used on several systems simultaneously. They support hiding their private keys by using the wallet encrypt feature and not sharing the password, but such "neutered" wallets lose the power to generate public keys as well.

Deterministic wallets do not require such frequent backups, and elliptic curve mathematics permit schemes where one can calculate the public keys without revealing the private keys. This permits for example a webshop business to let its webserver generate fresh addresses (public key hashes) for each order or for each customer, without giving the webserver access to the corresponding private keys (which are required for spending the received funds).

However, deterministic wallets typically consist of a single "chain" of keypairs. The fact that there is only one chain means that sharing a wallet happens on an all-or-nothing basis. However, in some cases one only wants some (public) keys to be shared and recoverable. In the example of a webshop, the webserver does not need access to all public keys of the merchant's wallet; only to those addresses which are used to receive customers' payments, and not for example the change addresses that are generated when the merchant spends money. Hierarchical deterministic wallets allow such selective sharing by supporting multiple keypair chains, derived from a single root.

Specification: Key derivation

Conventions

In the rest of this text we will assume the public key cryptography used in Bitcoin, namely elliptic curve cryptography using the field and curve parameters defined by secp256k1 (http://www.secg.org/sec2-v2.pdf). Variables below are either:

- Integers modulo the order of the curve (referred to as n).
- Coordinates of points on the curve.
- Byte sequences.

Addition (+) of two coordinate pair is defined as application of the EC group operation. Concatenation (||) is the operation of appending one byte sequence onto another.

As standard conversion functions, we assume:

- point(p): returns the coordinate pair resulting from EC point multiplication (repeated application of the EC group operation) of the secp256k1 base point with the integer p (i.e., the operation used to compute a public key from a private key).
- ser_{32}(i): serialize a 32-bit unsigned integer i as a 4-byte sequence, most significant byte first.
- ser_{256}(p): serializes the integer p as a 32-byte sequence, most significant byte first.

- $ser_P(P)$: serializes the coordinate pair P = (x,y) (i.e., the public key) as a byte sequence using SEC1's compressed form: (0x02 or 0x03) || $ser_{256}(x)$, where the header byte depends on the parity of the omitted y coordinate.
- $parse_{256}(p)$: interprets a 32-byte sequence as a 256-bit number, most significant byte first.

Extended keys

In what follows, we will define a function that derives a number of child keys from a parent key. In order to prevent these from depending solely on the key itself, we extend both private and public keys first with an extra 256 bits of entropy. This extension, called the chain code, is identical for corresponding private and public keys, and consists of 32 bytes.

We represent an extended private key as (k, c), with k the normal private key, and c the chain code. An extended public key is represented as (K, c), with K = point(k) and c the chain code.

Each extended key has 2^{31} normal child keys, and 2^{31} hardened child keys. Each of these child keys has an index. The normal child keys use indices 0 through 2^{31}-1. The hardened child keys use indices 2^{31} through 2^{32}-1. To ease notation for hardened key indices, a number i_H represents $i+2^{31}$.

Child key derivation (CKD) functions

Given a parent extended key and an index i, it is possible to compute the corresponding child extended key. The algorithm to do so depends on whether the child is a hardened key or not (or, equivalently, whether $i \geq 2^{31}$), and whether we're talking about private or public keys.

Private parent key → private child key

The function CKDpriv((k_{par}, c_{par}), i) → (k_i, c_i) computes a child extended private key from the parent extended private key:

- Check whether $i \geq 2^{31}$ (whether the child is a hardened key).
  - If so (hardened child): let I = HMAC-SHA512(Key = c_{par}, Data = 0x00 || $ser_{256}(k_{par})$ || $ser_{32}(i)$). (Note: The 0x00 pads the private key to make it 33 bytes long.)
  - If not (normal child): let I = HMAC-SHA512(Key = c_{par}, Data = $ser_P(point(k_{par}))$ || $ser_{32}(i)$).
- Split I into two 32-byte sequences, I_L and I_R.
- The returned child key k_i is $parse_{256}(I_L) + k_{par} \pmod n$.
- The returned chain code c_i is I_R.
- In case $parse_{256}(I_L) \geq n$ or $k_i = 0$, the resulting key is invalid, and one should proceed with the next value for i. (Note: this has probability lower than 1 in 2^{127}.)

The HMAC-SHA512 function is specified in RFC 4231.

Public parent key → public child key

The function CKDpub((K_{par}, c_{par}), i) → (K_i, c_i) computes a child extended public key from the parent extended public key. It is only defined for non-hardened child keys.

- Check whether $i \geq 2^{31}$ (whether the child is a hardened key).
  - If so (hardened child): return failure
  - If not (normal child): let I = HMAC-SHA512(Key = c_{par}, Data = $ser_P(K_{par}) \,||\, ser_{32}(i)$).
- Split I into two 32-byte sequences, I_L and I_R.
- The returned child key K_i is point($parse_{256}(I_L)$) + K_{par}.
- The returned chain code c_i is I_R.
- In case $parse_{256}(I_L) \geq n$ or K_i is the point at infinity, the resulting key is invalid, and one should proceed with the next value for i.

Private parent key → public child key

The function N((k, c)) → (K, c) computes the extended public key corresponding to an extended private key (the "neutered" version, as it removes the ability to sign transactions).

- The returned key K is point(k).
- The returned chain code c is just the passed chain code.

To compute the public child key of a parent private key:

- N(CKDpriv((k_{par}, c_{par}), i)) (works always).
- CKDpub(N(k_{par}, c_{par}), i) (works only for non-hardened child keys).

The fact that they are equivalent is what makes non-hardened keys useful (one can derive child public keys of a given parent key without knowing any private key), and also what distinguishes them from hardened keys. The reason for not always using non-hardened keys (which are more useful) is security; see further below for more information.

Public parent key → private child key

This is not possible.

The key tree

The next step is cascading several CKD constructions to build a tree. We start with one root, the master extended key m. By evaluating CKDpriv(m,i) for several values of i, we get a number of level-1 derived nodes. As each of these is again an extended key, CKDpriv can be applied to those as well.

To shorten notation, we will write CKDpriv(CKDpriv(CKDpriv(m,3_H),2),5) as m/3_H/2/5. Equivalently for public keys, we write CKDpub(CKDpub(CKDpub(M,3),2),5) as M/3/2/5. This results in the following identities:

- N(m/a/b/c) = N(m/a/b)/c = N(m/a)/b/c = N(m)/a/b/c = M/a/b/c.
- N(m/a_H/b/c) = N(m/a_H/b)/c = N(m/a_H)/b/c.

However, $N(m/a_H)$ cannot be rewritten as $N(m)/a_H$, as the latter is not possible.

Each leaf node in the tree corresponds to an actual key, while the internal nodes correspond to the collections of keys that descend from them. The chain codes of the leaf nodes are ignored, and only their embedded private or public key is relevant. Because of this construction, knowing an extended private key allows reconstruction of all descendant private keys and public keys, and knowing an extended public key allows reconstruction of all descendant non-hardened public keys.

Key identifiers

Extended keys can be identified by the Hash160 (RIPEMD160 after SHA256) of the serialized ECDSA public key K, ignoring the chain code. This corresponds exactly to the data used in traditional Bitcoin addresses. It is not advised to represent this data in base58 format though, as it may be interpreted as an address that way (and wallet software is not required to accept payment to the chain key itself).

The first 32 bits of the identifier are called the key fingerprint.

Serialization format

Extended public and private keys are serialized as follows:

- 4 bytes: version bytes (mainnet: 0x0488B21E public, 0x0488ADE4 private; testnet: 0x043587CF public, 0x04358394 private)
- 1 byte: depth: 0x00 for master nodes, 0x01 for level-1 derived keys, ….
- 4 bytes: the fingerprint of the parent's key (0x00000000 if master key)
- 4 bytes: child number. This is $ser_{32}(i)$ for i in $x_i = x_{par}/i$, with x_i the key being serialized. (0x00000000 if master key)
- 32 bytes: the chain code
- 33 bytes: the public key or private key data ($ser_P(K)$ for public keys, 0x00 || $ser_{256}(k)$ for private keys)

This 78 byte structure can be encoded like other Bitcoin data in Base58, by first adding 32 checksum bits (derived from the double SHA-256 checksum), and then converting to the Base58 representation. This results in a Base58-encoded string of exactly 111 characters. Because of the choice of the version bytes, the Base58 representation will start with "xprv" or "xpub" on mainnet, "tprv" or "tpub" on testnet.

Note that the fingerprint of the parent only serves as a fast way to detect parent and child nodes in software, and software must be willing to deal with collisions. Internally, the full 160-bit identifier could be used.

When importing a serialized extended public key, implementations must verify whether the X coordinate in the public key data corresponds to a point on the curve. If not, the extended public key is invalid.

Master key generation

The total number of possible extended keypairs is almost 2^{512}, but the produced keys are only 256 bits long, and offer about half of that in terms of security. Therefore, master keys are not generated directly, but instead from a potentially short seed value.

- Generate a seed byte sequence S of a chosen length (between 128 and 512 bits; 256 bits is advised) from a (P)RNG.

- Calculate I = HMAC-SHA512(Key = "Bitcoin seed", Data = S)
- Split I into two 32-byte sequences, I_L and I_R.
- Use $parse_{256}(I_L)$ as master secret key, and I_R as master chain code.

In case $parse_{256}(I_L)$ is 0 or $parse_{256}(I_L) \geq n$, the master key is invalid.

<img src=bip-0032/derivation.png></img>

Specification: Wallet structure

The previous sections specified key trees and their nodes. The next step is imposing a wallet structure on this tree. The layout defined in this section is a default only, though clients are encouraged to mimic it for compatibility, even if not all features are supported.

The default wallet layout

An HDW is organized as several 'accounts'. Accounts are numbered, the default account ("") being number 0. Clients are not required to support more than one account - if not, they only use the default account.

Each account is composed of two keypair chains: an internal and an external one. The external keychain is used to generate new public addresses, while the internal keychain is used for all other operations (change addresses, generation addresses, …, anything that doesn't need to be communicated). Clients that do not support separate keychains for these should use the external one for everything.

- $m/i_H/0/k$ corresponds to the k'th keypair of the external chain of account number i of the HDW derived from master m.
- $m/i_H/1/k$ corresponds to the k'th keypair of the internal chain of account number i of the HDW derived from master m.

Use cases

Full wallet sharing: m

In cases where two systems need to access a single shared wallet, and both need to be able to perform spendings, one needs to share the master private extended key. Nodes can keep a pool of N look-ahead keys cached for external chains, to watch for incoming payments. The look-ahead for internal chains can be very small, as no gaps are to be expected here. An extra look-ahead could be active for the first unused account's chains - triggering the creation of a new account when used. Note that the name of the account will still need to be entered manually and cannot be synchronized via the block chain.

Audits: N(m/*)

In case an auditor needs full access to the list of incoming and outgoing payments, one can share all account public extended keys. This will allow the auditor to see all transactions from and to the wallet, in all accounts, but not a single secret key.

Per-office balances: m/i_H

When a business has several independent offices, they can all use wallets derived from a single master. This will allow the headquarters to maintain a super-wallet that sees all incoming and outgoing transactions of all offices, and even permit moving money between the offices.

Recurrent business-to-business transactions: $N(m/i_H/0)$

In case two business partners often transfer money, one can use the extended public key for the external chain of a specific account (M/i h/0) as a sort of "super address", allowing frequent transactions that cannot (easily) be associated, but without needing to request a new address for each payment. Such a mechanism could also be used by mining pool operators as a variable payout address.

Unsecure money receiver: $N(m/i_H/0)$

When an unsecure webserver is used to run an e-commerce site, it needs to know public addresses that are used to receive payments. The webserver only needs to know the public extended key of the external chain of a single account. This means someone illegally obtaining access to the webserver can at most see all incoming payments but will not be able to steal the money, will not (trivially) be able to distinguish outgoing transactions, nor be able to see payments received by other webservers if there are several.

Compatibility

To comply with this standard, a client must at least be able to import an extended public or private key, to give access to its direct descendants as wallet keys. The wallet structure (master/account/chain/subchain) presented in the second part of the specification is advisory only, but is suggested as a minimal structure for easy compatibility - even when no separate accounts or distinction between internal and external chains is made. However, implementations may deviate from it for specific needs; more complex applications may call for a more complex tree structure.

Security

In addition to the expectations from the EC public-key cryptography itself:

- Given a public key K, an attacker cannot find the corresponding private key more efficiently than by solving the EC discrete logarithm problem (assumed to require 2^{128} group operations).

the intended security properties of this standard are:

- Given a child extended private key (k_i,c_i) and the integer i, an attacker cannot find the parent private key k_{par} more efficiently than a 2^{256} brute force of HMAC-SHA512.
- Given any number ($2 \le N \le 2^{32}$-1) of (index, extended private key) tuples $(i_j,(k_{i_j},c_{i_j}))$, with distinct i_j's, determining whether they are derived from a common parent extended private key (i.e., whether there exists a (k_{par},c_{par}) such that for each j in (0..N-1) $CKDpriv((k_{par},c_{par}),i_j)=(k_{i_j},c_{i_j})$), cannot be done more efficiently than a 2^{256} brute force of HMAC-SHA512.

Note however that the following properties do not exist:

- Given a parent extended public key (K_{par},c_{par}) and a child public key (K_i), it is hard to find i.
- Given a parent extended public key (K_{par},c_{par}) and a non-hardened child private key (k_i), it is hard to find k_{par}.

Implications

Private and public keys must be kept safe as usual. Leaking a private key means access to coins - leaking a public key can mean loss of privacy.

Somewhat more care must be taken regarding extended keys, as these correspond to an entire (sub)tree of keys.

One weakness that may not be immediately obvious, is that knowledge of a parent extended public key plus any non-hardened private key descending from it is equivalent to knowing the parent extended private key (and thus every private and public key descending from it). This means that extended public keys must be treated more carefully than regular public keys. It is also the reason for the existence of hardened keys, and why they are used for the account level in the tree. This way, a leak of account-specific (or below) private keys never risks compromising the master or other accounts.

Test Vectors

Test vector 1

Seed (hex): 000102030405060708090a0b0c0d0e0f

- Chain m
  - ext pub: xpub661MyMwAqRbcFtXgS5sYJABqqG9YLmC4Q1Rdap9gSE8NqtwybGhePY2gZ29ESFjqJoCu1Rupje8YtGqsef
  - ext prv: xprv9s21ZrQH143K3QTDL4LXw2F7HEK3wJUD2nW2nRk4stbPy6cq3jPPqjiChkVvvNKmPGJxWUtg6LnF5kejM
- Chain $m/0_H$
  - ext pub: xpub68Gmy5EdvgibQVfPdqkBBCHxA5htiqg55crXYuXoQRKfDBFA1WEjWgP6LHhwBZeNK1VTsfTFUHCdrfp
  - ext prv: xprv9uHRZZhk6KAJC1avXpDAp4MDc3sQKNxDiPvvkX8Br5ngLNv1TxvUxt4cV1rGL5hj6KCesnDYUhd7oWg
- Chain $m/0_H/1$
  - ext pub: xpub6ASuArnXKPbfEwhqN6e3mwBcDTgzisQN1wXN9BJcM47sSikHjJf3UFHKkNAWbWMiGj7Wf5uMash7Sy
  - ext prv: xprv9wTYmMFdV23N2TdNG573QoEsfRrWKQgWeibmLntzniatZvR9BmLnvSxqu53Kw1UmYPxLgboyZQaXw
- Chain $m/0_H/1/2_H$
  - ext pub: xpub6D4BDPcP2GT577Vvch3R8wDkScZWzQzMMUm3PWbmWvVJrZwQY4VUNgqFJPMM3No2dFDFGTsxp
  - ext prv: xprv9z4pot5VBttmtdRTWfWQmoH1taj2axGVzFqSb8C9xaxKymcFzXBDptWmT7FwuEzG3ryjH4ktypQSAewR

- Chain m/0_H/1/2_H/2
  - ext pub:
    xpub6FHa3pjLCk84BayeJxFW2SP4XRrFd1JYnxeLeU8EqN3vDfZmbqBqaGJAyiLjTAwm6ZLRQUMv1ZACTj37s
  - ext prv:
    xprvA2JDeKCSNNZky6uBCviVfJSKyQ1mDYahRjijr5idH2WwLsEd4Hsb2Tyh8RfQMuPh7f7RtyzTtdrbdqqsunu
- Chain m/0_H/1/2_H/2/1000000000
  - ext pub:
    xpub6H1LXWLaKsWFhvm6RVpEL9P4KfRZSW7abD2ttkWP3SSQvnyA8FSVqNTEcYFgJS2UaFcxupHiYkro49S
  - ext prv:
    xprvA41z7zogVVwxVSgdKUHDy1SKmdb533PjDz7J6N6mV6uS3ze1ai8FHa8kmHScGpWmj4WggLyQjgPie1rF

Test vector 2

Seed (hex):
fffcf9f6f3f0edeae7e4e1dedbd8d5d2cfccc9c6c3c0bdbab7b4b1aeaba8a5a29f9c999693908d8a8784817e7b7875726f6c696663605d5a5

- Chain m
  - ext pub:
    xpub661MyMwAqRbcFW31YEwpkMuc5THy2PSt5bDMsktWQcFF8syAmRUapSCGu8ED9W6oDMSgv6Zz8idc
  - ext prv:
    xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHN
- Chain m/0
  - ext pub:
    xpub69H7F5d8KSRgmmdJg2KhpAK8SR3DjMwAdkxj3ZuxV27CprR9LgpeyGmXUbC6wb7ERfvrnKZjXoUmmk
  - ext prv:
    xprv9vHkqa6EV4sPZHYqZznhT2NPtPCjKuDKGY38FBWLvgaDx45zo9WQRUT3dKYnjwih2yJD9mkrocEZXo1
- Chain m/0/2147483647_H
  - ext pub:
    xpub6ASAVgeehLbnwdqV6UKMHVzgqAG8Gr6riv3Fxxpj8ksbH9ebxaEyBLZ85ySDhKiLDBrQSARLq1uNRts8
  - ext prv:
    xprv9wSp6B7kry3Vj9m1zSnLvN3xH8RdsPP1Mh7fAaR7aRLcQMKTR2vidYEeEg2mUCTAwCd6vnxVrcjfy2kRg
- Chain m/0/2147483647_H/1
  - ext pub:
    xpub6DF8uhdarytz3FWdA8TvFSvvAh8dP3283MY7p2V4SeE2wyWmG5mg5EwVvmdMVCQcoNJxGoWaU9DC
  - ext prv:
    xprv9zFnWC6h2cLgpmSA46vutJzBcfJ8yaJGg8cX1e5StJh45BBciYTRXSd25UEPVuesF9yog62tGAQtHjXajPPdbR
- Chain m/0/2147483647_H/1/2147483646_H
  - ext pub:
    xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4A
  - ext prv:
    xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxF
- Chain m/0/2147483647_H/1/2147483646_H/2
  - ext pub:
    xpub6FnCn6nSzZAw5Tw7cgR9bi15UV96gLZhjDstkXXxvCLsUXBGXPdSnLFbdpq8p9HmGsApME5hQTZ3em
  - ext prv:
    xprvA2nrNbFZABcdryreWet9Ea4LvTJcGsqrMzxHx98MMrotbir7yrKCEXw7nadnHM8Dq38EGfSh6dqA9QWTy

Test vector 3

These vectors test for the retention of leading zeros. See bitpay/bitcore-lib#47 and iancoleman/bip39#58 for more information.

Seed (hex):
4b381541583be4423346c643850da4b320e46a87ae3d2a4e6da11eba819cd4acba45d239319ac14f863b8d5ab5a0d0c64d2e8a1e7d145

- Chain m
  - ext pub:
    xpub661MyMwAqRbcEZVB4dScxMAdx6d4nFc9nvyvH3v4gJL378CSRZiYmhRoP7mBy6gSPSCYk6SzXPTf3ND
  - ext prv:
    xprv9s21ZrQH143K25QhxbucbDDuQ4naNntJRi4KUfWT7xo4EKsHt2QJDu7KXp1A3u7Bi1j8ph3EGsZ9Xvz9dG
- Chain $m/0_H$
  - ext pub:
    xpub68NZiKmJWnxxS6aaHmn81bvJeTESw724CRDs6HbuccFQN9Ku14VQrADWgqbhhTHBaohPX4CjNLf9fq9
  - ext prv:
    xprv9uPDJpEQgRQfDcW7BkF7eTya6RPxXeJCqCJGHuCJ4GiRVLzkTXBAJMu2qaMWPrS7AANYqdq6vcBcBUd

Test vector 4

These vectors test for the retention of leading zeros. See btcsuite/btcutil#172 for more information.

Seed (hex): 3ddd5602285899a946114506157c7997e5444528f3003f6134712147db19b678

- Chain m
  - ext pub:
    xpub661MyMwAqRbcGczjuMoRm6dXaLDEhW1u34gKenbeYqAix21mdUKJyuyu5F1rzYGVxyL6tmgBUAEPrE
  - ext prv:
    xprv9s21ZrQH143K48vGoLGRPxgo2JNkJ3J3fqkirQC2zVdk5Dgd5w14S7fRDyHH4dWNHUgkvsvNDCkvAwcS
- Chain $m/0_H$
  - ext pub:
    xpub69AUMk3qDBi3uW1sXgjCmVjJ2G6WQoYSnNHyzkmdCHEhSZ4tBok37xfFEqHd2AddP56Tqp4o56AePAg
  - ext prv:
    xprv9vB7xEWwNp9kh1wQRfCCQMnZUEG21LpbR9NPCNN1dwhiZkjjeGRnaALmPXCX7SgjFTiCTT6bXes17b
- Chain $m/0_H/1_H$
  - ext pub:
    xpub6BJA1jSqiukeaesWfxe6sNK9CCGaujFFSJLomWHprUL9DePQ4JDkM5d88n49sMGJxrhpjazuXYWdMf17C9
  - ext prv:
    xprv9xJocDuwtYCMNAo3Zw76WENQeAS6WGXQ55RCy7tDJ8oALr4FWkuVoHJeHVAcAqiZLE7Je3vZJHxspZ

Test vector 5

These vectors test that invalid extended keys are recognized as invalid.

- xpub661MyMwAqRbcEYS8w7XLSVeEsBXy79zSzH1J8vCdxAZningWLdN3zgtU6LBpB85b3D2yc8sfvZU521AAwdZaf
  (pubkey version / prvkey mismatch)

- xprv9s21ZrQH143K24Mfq5zL5MhWK9hUhhGbd45hLXo2Pq2oqzMMo63oStZzFGTQQD3dC4H2D5GBj7vWvSQaaBv
  (prvkey version / pubkey mismatch)
- xpub661MyMwAqRbcEYS8w7XLSVeEsBXy79zSzH1J8vCdxAZningWLdN3zgtU6Txnt3siSujt9RCVYsx4qHZGc62TG4
  (invalid pubkey prefix 04)
- xprv9s21ZrQH143K24Mfq5zL5MhWK9hUhhGbd45hLXo2Pq2oqzMMo63oStZzFGpWnsj83BHtEy5Zt8CcDr1UiRXuW
  (invalid prvkey prefix 04)
- xpub661MyMwAqRbcEYS8w7XLSVeEsBXy79zSzH1J8vCdxAZningWLdN3zgtU6N8ZMMXctdiCjxTNq964yKkwrkBJJ
  (invalid pubkey prefix 01)
- xprv9s21ZrQH143K24Mfq5zL5MhWK9hUhhGbd45hLXo2Pq2oqzMMo63oStZzFAzHGBP2UuGCqWLTAPLcMtD9y5g
  (invalid prvkey prefix 01)
- xprv9s2SPatNQ9Vc6GTbVMFPFo7jsaZySyzk7L8n2uqKXJen3KUmvQNTuLh3fhZMBoG3G4ZW1N2kZuHEPY53qmbZ
  (zero depth with non-zero parent fingerprint)
- xpub661no6RGEX3uJkY4bNnPcw4URcQTrSibUZ4NqJEw5eBkv7ovTwgiT91XX27VbEXGENhYRCf7hyEbWrR3FewAT
  (zero depth with non-zero parent fingerprint)
- xprv9s21ZrQH4r4TsiLvyLXqM9P7k1K3EYhA1kkD6xuquB5i39AU8KF42acDyL3qsDbU9NmZn6MsGSUYZEsuoePmjz
  (zero depth with non-zero index)
- xpub661MyMwAuDcm6CRQ5N4qiHKrJ39Xe1R1NyfouMKTTWcguwVcfrZJaNvhpebzGerh7gucBvzEQWRugZDuDX
  (zero depth with non-zero index)
- DMwo58pR1QLEFihHiXPVykYB6fJmsTeHvyTp7hRThAtCX8CvYzgPcn8XnmdfHGMQzT7ayAmfo4z3gY5KfbrZWZ6S
  (unknown extended key version)
- DMwo58pR1QLEFihHiXPVykYB6fJmsTeHvyTp7hRThAtCX8CvYzgPcn8XnmdfHPmHJiEDXkTiJTVV9rHEBUem2mw
  (unknown extended key version)
- xprv9s21ZrQH143K24Mfq5zL5MhWK9hUhhGbd45hLXo2Pq2oqzMMo63oStZzF93Y5wvzdUayhgkkFoicQZcP3y52uP
  (private key 0 not in 1..n-1)
- xprv9s21ZrQH143K24Mfq5zL5MhWK9hUhhGbd45hLXo2Pq2oqzMMo63oStZzFAzHGBP2UuGCqWLTAPLcMtD5SDK
  (private key n not in 1..n-1)
- xpub661MyMwAqRbcEYS8w7XLSVeEsBXy79zSzH1J8vCdxAZningWLdN3zgtU6Q5JXayek4PRsn35jii4veMimro1xefsk
  (invalid pubkey 020000000000000000000000000000000000000000000000000000000000000007)
- xprv9s21ZrQH143K3QTDL4LXw2F7HEK3wJUD2nW2nRk4stbPy6cq3jPPqjiChkVvvNKmPGJxWUtg6LnF5kejMRNNU
  (invalid checksum)

Acknowledgements

- Gregory Maxwell for the original idea of type-2 deterministic wallets, and many discussions about it.
- Alan Reiner for the implementation of this scheme in Armory, and the suggestions that followed from that.
- Eric Lombrozo for reviewing and revising this BIP.
- Mike Caldwell for the version bytes to obtain human-recognizable Base58 strings.


```
BIP: 38
Layer: Applications
Title: Passphrase-protected private key
Authors: Mike Caldwell <mcaldwell@swipeclock.com>
         Aaron Voisine <voisine@gmail.com>
Comments-Summary: Unanimously Discourage for implementation
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0038
Status: Deployed
Type: Specification
```

```
Assigned: 2012-11-20
License: PD
```

BIP 38

Abstract

A method is proposed for encrypting and encoding a passphrase-protected Bitcoin private key record in the form of a 58-character Base58Check-encoded printable string. Encrypted private key records are intended for use on paper wallets and physical Bitcoins. Each record string contains all the information needed to reconstitute the private key except for a passphrase, and the methodology uses salting and *scrypt* to resist brute-force attacks.

The method provides two encoding methodologies - one permitting any known private key to be encrypted with any passphrase, and another permitting a shared private key generation scheme where the party generating the final key string and its associated Bitcoin address (such as a physical bitcoin manufacturer) knows only a string derived from the original passphrase, and where the original passphrase is needed in order to actually redeem funds sent to the associated Bitcoin address.

A 32-bit hash of the resulting Bitcoin address is encoded in plaintext within each encrypted key, so it can be correlated to a Bitcoin address with reasonable probability by someone not knowing the passphrase. The complete Bitcoin address can be derived through successful decryption of the key record.

Motivation

The motivation to make this proposal stems from observations of the way physical bitcoins and paper wallets are used.

An issuer of physical bitcoins must be trustworthy and trusted. Even if trustworthy, users are rightful to be skeptical about a third party with theoretical access to take their funds. A physical bitcoin that cannot be compromised by its issuer is always more intrinsically valuable than one that can.

A two-factor physical bitcoin solution is highly useful to individuals and organizations wishing to securely own bitcoins without any risk of electronic theft and without the responsibility of climbing the technological learning curve necessary to produce such an environment themselves. Two-factor physical bitcoins allow a secure storage solution to be put in a box and sold on the open market, greatly enlarging the number of people who are able to securely store bitcoins.

Existing methodologies for creating two-factor physical bitcoins are limited and cumbersome. At the time of this proposal, a user could create their own private key, submit the public key to the physical bitcoin issuer, and then receive a physical bitcoin that must be kept together with some sort of record of the user-generated private key, and finally, must be redeemed through a tool. The fact that the physical bitcoin must be kept together with a user-produced private key negates much of the benefit of the physical bitcoin - the user may as well just print and maintain a private key.

A standardized password-protected private key format makes acquiring and redeeming two-factor physical bitcoins simpler for the user. Instead of maintaining a private key that cannot be memorized, the user may choose a passphrase of their choice. The passphrase may be much shorter than the length of a typical private key, short enough that they could use a label or engraver to permanently commit their passphrase to their

physical Bitcoin piece once they have received it. By adopting a standard way to encrypt a private key, we maximize the possibility that they'll be able to redeem their funds in the venue of their choice, rather than relying on an executable redemption tool they may not wish to download.

Password and passphrase-protected private keys enable new practical use cases for sending bitcoins from person to person. Someone wanting to send bitcoins through postal mail could send a password-protected paper wallet and give the recipient the passphrase over the phone or e-mail, making the transfer safe from interception of either channel. A user of paper wallets or Bitcoin banknote-style vouchers ("cash") could carry funded encrypted private keys while leaving a copy at home as an element of protection against accidental loss or theft. A user of paper wallets who leaves bitcoins in a bank vault or safety deposit box could keep the password at home or share it with trusted associates as protection against someone at the bank gaining access to the paper wallets and spending from them. The foreseeable and unforeseeable use cases for password-protected private keys are numerous.

Copyright

This proposal is hereby placed in the public domain.

Rationale

User story: *As a Bitcoin user who uses paper wallets, I would like the ability to add encryption, so that my Bitcoin paper storage can be two factor: something I have plus something I know.*

User story: *As a Bitcoin user who would like to pay a person or a company with a private key, I do not want to worry that any part of the communication path may result in the interception of the key and theft of my funds. I would prefer to offer an encrypted private key, and then follow it up with the password using a different communication channel (e.g. a phone call or SMS).*

User story: *(EC-multiplied keys) As a user of physical bitcoins, I would like a third party to be able to create password-protected Bitcoin private keys for me, without them knowing the password, so I can benefit from the physical bitcoin without the issuer having access to the private key. I would like to be able to choose a password whose minimum length and required format does not preclude me from memorizing it or engraving it on my physical bitcoin, without exposing me to an undue risk of password cracking and/or theft by the manufacturer of the item.*

User story: *(EC-multiplied keys) As a user of paper wallets, I would like the ability to generate a large number of Bitcoin addresses protected by the same password, while enjoying a high degree of security (highly expensive scrypt parameters), but without having to incur the scrypt delay for each address I generate.*

Specification

This proposal makes use of the following functions and definitions:

- **AES256Encrypt, AES256Decrypt**: the simple form of the well-known AES block cipher without consideration for initialization vectors or block chaining. Each of these functions takes a 256-bit key and 16 bytes of input, and deterministically yields 16 bytes of output.
- **SHA256**, a well-known hashing algorithm that takes an arbitrary number of bytes as input and deterministically yields a 32-byte hash.

- **scrypt**: A well-known key derivation algorithm. It takes the following parameters: (string) password, (string) salt, (int) n, (int) r, (int) p, (int) length, and deterministically yields an array of bytes whose length is equal to the length parameter.
- **ECMultiply**: Multiplication of an elliptic curve point by a scalar integer with respect to the secp256k1 elliptic curve.
- **G, N**: Constants defined as part of the secp256k1 elliptic curve. G is an elliptic curve point, and N is a large positive integer.
- **Base58Check**: a method for encoding arrays of bytes using 58 alphanumeric characters commonly used in the Bitcoin ecosystem.

Prefix

It is proposed that the resulting Base58Check-encoded string start with a '6'. The number '6' is intended to represent, from the perspective of the user, "a private key that needs something else to be usable" - an umbrella definition that could be understood in the future to include keys participating in multisig transactions, and was chosen with deference to the existing prefix '5' most commonly observed in Wallet Import Format which denotes an unencrypted private key.

It is proposed that the second character ought to give a hint as to what is needed as a second factor, and for an encrypted key requiring a passphrase, the uppercase letter P is proposed.

To keep the size of the encrypted key down, no initialization vectors (IVs) are used in the AES encryption. Rather, suitable values for IV-like use are derived using scrypt from the passphrase and from using a 32-bit hash of the resulting Bitcoin address as salt.

Proposed specification

- Object identifier prefix: 0x0142 (non-EC-multiplied) or 0x0143 (EC-multiplied). These are constant bytes that appear at the beginning of the Base58Check-encoded record, and their presence causes the resulting string to have a predictable prefix.
- How the user sees it: 58 characters always starting with '6P'
    - Visual cues are present in the third character for visually identifying the EC-multiply and compress flag.
- Count of payload bytes (beyond prefix): 37
    - 1 byte (*flagbyte*):
        - the most significant two bits are set as follows to preserve the visibility of the compression flag in the prefix, as well as to keep the payload within the range of allowable values that keep the "6P" prefix intact. For non-EC-multiplied keys, the bits are 11. For EC-multiplied keys, the bits are 00.
        - the bit with value 0x20 when set indicates the key should be converted to a base58check encoded P2PKH bitcoin address using the DER compressed public key format. When not set, it should be a base58check encoded P2PKH bitcoin address using the DER uncompressed public key format.
        - the bits with values 0x10 and 0x08 are reserved for a future specification that contemplates using multisig as a way to combine the factors such that parties in possession of the separate factors can independently sign a proposed transaction without requiring that any party possess both factors. These bits must be 0 to comply with this version of the specification.

        - the bit with value 0x04 indicates whether a lot and sequence number are encoded into the first factor, and activates special behavior for including them in the decryption process. This applies to EC-multiplied keys only. Must be 0 for non-EC-multiplied keys.
        - remaining bits are reserved for future use and must all be 0 to comply with this version of the specification.
    - 4 bytes: SHA256(SHA256(expected_bitcoin_address))[0…3], used both for typo checking and as salt
    - 16 bytes: Contents depend on whether EC multiplication is used.
    - 16 bytes: lasthalf: An AES-encrypted key material record (contents depend on whether EC multiplication is used)
- Range in base58check encoding for non-EC-multiplied keys without compression (prefix 6PR):
    - Minimum value: 6PRHv1jg1ytiE4kT2QtrUz8gEjMQghZDWg1FuxjdYDzjUkcJeGdFj9q9Vi (based on 01 42 C0 plus thirty-six 00's)
    - Maximum value: 6PRWdmoT1ZursVcr5NiD14p5bHrKVGPG7yeEoEeRb8FVaqYSHnZTLEbYsU (based on 01 42 C0 plus thirty-six FF's)
- Range in base58check encoding for non-EC-multiplied keys with compression (prefix 6PY):
    - Minimum value: 6PYJxKpVnkXUsnZAfD2B5ZsZafJYNp4ezQQeCjs39494qUUXLnXijLx6LG (based on 01 42 E0 plus thirty-six 00's)
    - Maximum value: 6PYXg5tGnLYdXDRZiAqXbeYxwDoTBNthbi3d61mqBxPpwZQezJTvQHsCnk (based on 01 42 E0 plus thirty-six FF's)
- Range in base58check encoding for EC-multiplied keys without compression (prefix 6Pf):
    - Minimum value: 6PfKzduKZXAFXWMtJ19Vg9cSvbFg4va6U8p2VWzSjtHQCCLk3JSBpUvfpf (based on 01 43 00 plus thirty-six 00's)
    - Maximum value: 6PfYiPy6Z7BQAwEHLxxrCEHrH9kasVQ95ST1NnuEnnYAJHGsgpNPQ9dTHc (based on 01 43 00 plus thirty-six FF's)
- Range in base58check encoding for EC-multiplied keys with compression (prefix 6Pn):
    - Minimum value: 6PnM2wz9LHo2BEAbvoGpGjMLGXCom35XwsDQnJ7rLiRjYvCxjpLenmoBsR (based on 01 43 20 plus thirty-six 00's)
    - Maximum value: 6PnZki3vKspApf2zym6Anp2jd5hiZbuaZArPfa2ePcgVf196PLGrQNyVUh (based on 01 43 20 plus thirty-six FF's)

Encryption when EC multiply flag is not used

Encrypting a private key without the EC multiplication offers the advantage that any known private key can be encrypted. The party performing the encryption must know the passphrase.

Encryption steps:

1. Compute the Bitcoin address (ASCII), and take the first four bytes of SHA256(SHA256()) of it. Let's call this "addresshash".
2. Derive a key from the passphrase using scrypt
    - Parameters: *passphrase* is the passphrase itself encoded in UTF-8 and normalized using Unicode Normalization Form C (NFC). salt is *addresshash* from the earlier step, n=16384, r=8, p=8, length=64 (n, r, p are provisional and subject to consensus)
    - Let's split the resulting 64 bytes in half, and call them *derivedhalf1* and *derivedhalf*2.
3. Do AES256Encrypt(block = bitcoinprivkey[0…15] xor derivedhalf1[0…15], key = derivedhalf2), call the 16-byte result *encryptedhalf1*

4. Do AES256Encrypt(block = bitcoinprivkey[16…31] xor derivedhalf1[16…31], key = derivedhalf2), call the 16-byte result *encryptedhalf2*

The encrypted private key is the Base58Check-encoded concatenation of the following, which totals 39 bytes without Base58 checksum:

- 0x01 0x42 + *flagbyte* + *salt* + *encryptedhalf1* + *encryptedhalf2*

Decryption steps:

1. Collect encrypted private key and passphrase from user.
2. Derive *derivedhalf1* and *derivedhalf2* by passing the passphrase and *addresshash* into scrypt function.
3. Decrypt *encryptedhalf1* and *encryptedhalf2* using AES256Decrypt, merge them to form the encrypted private key.
4. Convert that private key into a Bitcoin address, honoring the compression preference specified in *flagbyte* of the encrypted key record.
5. Hash the Bitcoin address, and verify that *addresshash* from the encrypted private key record matches the hash. If not, report that the passphrase entry was incorrect.

Encryption when EC multiply mode is used

Encrypting a private key with EC multiplication offers the ability for someone to generate encrypted keys knowing only an EC point derived from the original passphrase and some salt generated by the passphrase's owner, and without knowing the passphrase itself. Only the person who knows the original passphrase can decrypt the private key. A code known as an *intermediate code* conveys the information needed to generate such a key without knowledge of the passphrase.

This methodology does not offer the ability to encrypt a known private key - this means that the process of creating encrypted keys is also the process of generating new addresses. On the other hand, this serves a security benefit for someone possessing an address generated this way: if the address can be recreated by decrypting its private key with a passphrase, and it's a strong passphrase one can be certain only he knows himself, then he can safely conclude that nobody could know the private key to that address.

The person who knows the passphrase and who is the intended beneficiary of the private keys is called the *owner*. He will generate one or more "intermediate codes", which are the first factor of a two-factor redemption system, and will give them to someone else we'll call *printer*, who generates a key pair with an intermediate code can know the address and encrypted private key, but cannot decrypt the private key without the original passphrase.

An intermediate code should, but is not required to, embed a printable "lot" and "sequence" number for the benefit of the user. The proposal forces these lot and sequence numbers to be included in any valid private keys generated from them. An owner who has requested multiple private keys to be generated for him will be advised by applications to ensure that each private key has a unique lot and sequence number consistent with the intermediate codes he generated. These mainly help protect *owner* from potential mistakes and/or attacks that could be made by *printer*.

The "lot" and "sequence" number are combined into a single 32 bit number. 20 bits are used for the lot number and 12 bits are used for the sequence number, such that the lot number can be any decimal number between 0 and 1048575, and the sequence number can be any decimal number between 0 and 4095. For programs that

generate batches of intermediate codes for an *owner*, it is recommended that lot numbers be chosen at random within the range 100000-999999 and that sequence numbers are assigned starting with 1.

Steps performed by *owner* to generate a single intermediate code, if lot and sequence numbers are being included:

1. Generate 4 random bytes, call them *ownersalt*.
2. Encode the lot and sequence numbers as a 4 byte quantity (big-endian): lotnumber * 4096 + sequencenumber. Call these four bytes *lotsequence*.
3. Concatenate *ownersalt* + *lotsequence* and call this *ownerentropy*.
4. Derive a key from the passphrase using scrypt
    - Parameters: *passphrase* is the passphrase itself encoded in UTF-8 and normalized using Unicode Normalization Form C (NFC). salt is *ownersalt*. n=16384, r=8, p=8, length=32.
    - Call the resulting 32 bytes *prefactor*.
    - Take SHA256(SHA256(*prefactor* + *ownerentropy*)) and call this *passfactor*. The "+" operator is concatenation.
5. Compute the elliptic curve point G * *passfactor*, and convert the result to compressed notation (33 bytes). Call this *passpoint*. Compressed notation is used for this purpose regardless of whether the intent is to create Bitcoin addresses with or without compressed public keys.
6. Convey *ownersalt* and *passpoint* to the party generating the keys, along with a checksum to ensure integrity.
    - The following Base58Check-encoded format is recommended for this purpose: magic bytes "2C E9 B3 E1 FF 39 E2 51" followed by *ownerentropy*, and then *passpoint*. The resulting string will start with the word "passphrase" due to the constant bytes, will be 72 characters in length, and encodes 49 bytes (8 bytes constant + 8 bytes *ownerentropy* + 33 bytes *passpoint*). The checksum is handled in the Base58Check encoding. The resulting string is called *intermediate_passphrase_string*.

If lot and sequence numbers are not being included, then follow the same procedure with the following changes:

- *ownersalt* is 8 random bytes instead of 4, and *lotsequence* is omitted. *ownerentropy* becomes an alias for *ownersalt*.
- The SHA256 conversion of *prefactor* to *passfactor* is omitted. Instead, the output of scrypt is used directly as *passfactor*.
- The magic bytes are "2C E9 B3 E1 FF 39 E2 53" instead (the last byte is 0x53 instead of 0x51).

Steps to create new encrypted private keys given *intermediate_passphrase_string* from *owner* (so we have *ownerentropy*, and *passpoint*, but we do not have *passfactor* or the passphrase):

1. Set *flagbyte*.
    - Turn on bit 0x20 if the Bitcoin address will be formed by hashing the compressed public key (optional, saves space, but many Bitcoin implementations aren't compatible with it)
    - Turn on bit 0x04 if *ownerentropy* contains a value for *lotsequence*. (While it has no effect on the keypair generation process, the decryption process needs this flag to know how to process *ownerentropy*)
2. Generate 24 random bytes, call this *seedb*. Take SHA256(SHA256(*seedb*)) to yield 32 bytes, call this *factorb*.
3. ECMultiply *passpoint* by *factorb*. Use the resulting EC point as a public key and hash it into a Bitcoin address using either compressed or uncompressed public key methodology (specify which methodology is used inside *flagbyte*). This is the generated Bitcoin address, call it *generatedaddress*.

4. Take the first four bytes of SHA256(SHA256(*generatedaddress*)) and call it *addresshash*.
5. Now we will encrypt *seedb*. Derive a second key from *passpoint* using scrypt
    - Parameters: *passphrase* is *passpoint* provided from the first party (expressed in binary as 33 bytes). *salt* is *addresshash* + *ownerentropy*, n=1024, r=1, p=1, length=64. The "+" operator is concatenation.
    - Split the result into two 32-byte halves and call them *derivedhalf1* and *derivedhalf2*.
6. Do AES256Encrypt(block = (seedb[0…15] xor derivedhalf1[0…15]), key = derivedhalf2), call the 16-byte result *encryptedpart1*
7. Do AES256Encrypt(block = ((encryptedpart1[8…15] + seedb[16…23]) xor derivedhalf1[16…31]), key = derivedhalf2), call the 16-byte result *encryptedpart2*. The "+" operator is concatenation.

The encrypted private key is the Base58Check-encoded concatenation of the following, which totals 39 bytes without Base58 checksum:

- 0x01 0x43 + *flagbyte* + *addresshash* + *ownerentropy* + *encryptedpart1*[0…7] + *encryptedpart2*

Confirmation code

The party generating the Bitcoin address has the option to return a *confirmation code* back to *owner* which allows *owner* to independently verify that he has been given a Bitcoin address that actually depends on his passphrase, and to confirm the lot and sequence numbers (if applicable). This protects *owner* from being given a Bitcoin address by the second party that is unrelated to the key derivation and possibly spendable by the second party. If a Bitcoin address given to *owner* can be successfully regenerated through the confirmation process, *owner* can be reasonably assured that any spending without the passphrase is infeasible. This confirmation code is 75 characters starting with "cfrm38".

To generate it, we need *flagbyte*, *ownerentropy*, *factorb*, *derivedhalf1* and *derivedhalf2* from the original encryption operation.

1. ECMultiply *factorb* by G, call the result *pointb*. The result is 33 bytes.
2. The first byte is 0x02 or 0x03. XOR it by (derivedhalf2[31] & 0x01), call the resulting byte *pointbprefix*.
3. Do AES256Encrypt(block = (pointb[1…16] xor derivedhalf1[0…15]), key = derivedhalf2) and call the result *pointbx1*.
4. Do AES256Encrypt(block = (pointb[17…32] xor derivedhalf1[16…31]), key = derivedhalf2) and call the result *pointbx2*.
5. Concatenate *pointbprefix* + *pointbx1* + *pointbx2* (total 33 bytes) and call the result *encryptedpointb*.

The result is a Base58Check-encoded concatenation of the following:

- 0x64 0x3B 0xF6 0xA8 0x9A + *flagbyte* + *addresshash* + *ownerentropy* + *encryptedpointb*

A confirmation tool, given a passphrase and a confirmation code, can recalculate the address, verify the address hash, and then assert the following: "It is confirmed that Bitcoin address *address* depends on this passphrase". If applicable: "The lot number is *lotnumber* and the sequence number is *sequencenumber*."

To recalculate the address:

1. Derive *passfactor* using scrypt with *ownerentropy* and the user's passphrase and use it to recompute *passpoint*
2. Derive decryption key for *pointb* using scrypt with *passpoint*, *addresshash*, and *ownerentropy*
3. Decrypt *encryptedpointb* to yield *pointb*

4. ECMultiply *pointb* by *passfactor*. Use the resulting EC point as a public key and hash it into *address* using either compressed or uncompressed public key methodology as specified in *flagbyte*.

Decryption

1. Collect encrypted private key and passphrase from user.
2. Derive *passfactor* using scrypt with *ownersalt* and the user's passphrase and use it to recompute *passpoint*
3. Derive decryption key for *seedb* using scrypt with *passpoint*, *addresshash*, and *ownerentropy*
4. Decrypt *encryptedpart2* using AES256Decrypt to yield the last 8 bytes of *seedb* and the last 8 bytes of *encryptedpart1*.
5. Decrypt *encryptedpart1* to yield the remainder of *seedb*.
6. Use *seedb* to compute *factorb*.
7. Multiply *passfactor* by *factorb* mod N to yield the private key associated with *generatedaddress*.
8. Convert that private key into a Bitcoin address, honoring the compression preference specified in the encrypted key.
9. Hash the Bitcoin address, and verify that *addresshash* from the encrypted private key record matches the hash. If not, report that the passphrase entry was incorrect.

Backwards compatibility

Backwards compatibility is minimally applicable since this is a new standard that at most extends Wallet Import Format. It is assumed that an entry point for private key data may also accept existing formats of private keys (such as hexadecimal and Wallet Import Format); this draft uses a key format that cannot be mistaken for any existing one and preserves auto-detection capabilities.

Suggestions for implementers of proposal with alt-chains

If this proposal is accepted into alt-chains, it is requested that the unused flag bytes not be used for denoting that the key belongs to an alt-chain.

Alt-chain implementers should exploit the address hash for this purpose. Since each operation in this proposal involves hashing a text representation of a coin address which (for Bitcoin) includes the leading '1', an alt-chain can easily be denoted simply by using the alt-chain's preferred format for representing an address. Alt-chain implementers may also change the prefix such that encrypted addresses do not start with "6P".

Discussion item: scrypt parameters

This proposal leaves the scrypt parameters up in the air. The following items are proposed for consideration:

The main goal of scrypt is to reduce the feasibility of brute force attacks. It must be assumed that an attacker will be able to use an efficient implementation of scrypt. The parameters should force a highly efficient implementation of scrypt to wait a decent amount of time to slow attacks.

On the other hand, an unavoidably likely place where scrypt will be implemented is using slow interpreted languages such as javascript. What might take milliseconds on an efficient scrypt implementation may take seconds in javascript.

It is believed, however, that someone using a javascript implementation is probably dealing with codes by hand, one at a time, rather than generating or processing large batches of codes. Thus, a wait time of several seconds is acceptable to a user.

A private key redemption process that forces a server to consume several seconds of CPU time would discourage implementation by the server owner, because they would be opening up a denial of service avenue by inviting users to make numerous attempts to invoke the redemption process. However, it's also feasible for the server owner to implement his redemption process in such a way that the decryption is done by the user's browser, offloading the task from his own server (and providing another reason why the chosen scrypt parameters should be tolerant of javascript-based decryptors).

The preliminary values of 16384, 8, and 8 are hoped to offer the following properties:

- Encryption/decryption in javascript requiring several seconds per operation
- Use of the parallelization parameter provides a modest opportunity for speedups in environments where concurrent threading is available - such environments would be selected for processes that must handle bulk quantities of encryption/decryption operations. Estimated time for an operation is in the tens or hundreds of milliseconds.

Reference implementation

Added to alpha version of Casascius Bitcoin Address Utility for Windows available at:

- https://github.com/casascius/Bitcoin-Address-Utility

Click "Tools" then "PPEC Keygen" (provisional name)

Test vectors

No compression, no EC multiply

Test 1:

- Passphrase: TestingOneTwoThree
- Encrypted: 6PRVWUbkzzsbcVac2qwfssoUJAN1Xhrg6bNk8J7Nzm5H7kxEbn2Nh2ZoGg
- Unencrypted (WIF): 5KN7MzqK5wt2TP1fQCYyHBtDrXdJuXbUzm4A9rKAteGu3Qi5CVR
- Unencrypted (hex): CBF4B9F70470856BB4F40F80B87EDB90865997FFEE6DF315AB166D713AF433A5

Test 2:

- Passphrase: Satoshi
- Encrypted: 6PRNFFkZc2NZ6dJqFfhRoFNMR9Lnyj7dYGrzdgXXVMXcxoKTePPX1dWByq
- Unencrypted (WIF): 5HtasZ6ofTHP6HCwTqTkLDuLQisYPah7aUnSKfC7h4hMUVw2gi5
- Unencrypted (hex): 09C2686880095B1A4C249EE3AC4EEA8A014F11E6F986D0B5025AC1F39AFBD9AE

Test 3:

- Passphrase ϓ␀🅾💩 (`\u03D2\u0301\u0000\U00010400\U0001F4A9`; GREEK UPSILON WITH HOOK, COMBINING ACUTE ACCENT, NULL, DESERET CAPITAL LETTER LONG I, PILE OF POO)
- Encrypted key: 6PRW5o9FLp4gJDDVqJQKJFTpMvdsSGJxMYHtHaQBF3ooa8mwD69bapcDQn

- Bitcoin Address: 16ktGzmfrurhbhi6JGqsMWf7TyqK9HNAeF
- Unencrypted private key (WIF): 5Jajm8eQ22H3pGWLEVCXyvND8dQZhiQhoLJNKjYXk9roUFTMSZ4
- *Note:* The non-standard UTF-8 characters in this passphrase should be NFC normalized to result in a passphrase of `0xcf9300f0909080f09f92a9` before further processing

Compression, no EC multiply

Test 1:

- Passphrase: TestingOneTwoThree
- Encrypted: 6PYNKZ1EAgYgmQfmNVamxyXVWHzK5s6DGhwP4J5o44cvXdoY7sRzhtpUeo
- Unencrypted (WIF): L44B5gGEpqEDRS9vVPz7QT35jcBG2r3CZwSwQ4fCewXAhAhqGVpP
- Unencrypted (hex): CBF4B9F70470856BB4F40F80B87EDB90865997FFEE6DF315AB166D713AF433A5

Test 2:

- Passphrase: Satoshi
- Encrypted: 6PYLtMnXvfG3oJde97zRyLYFZCYizPU5T3LwgdYJz1fRhh16bU7u6PPmY7
- Unencrypted (WIF): KwYgW8gcxj1JWJXhPSu4Fqwzfhp5Yfi42mdYmMa4XqK7NJxXUSK7
- Unencrypted (hex): 09C2686880095B1A4C249EE3AC4EEA8A014F11E6F986D0B5025AC1F39AFBD9AE

EC multiply, no compression, no lot/sequence numbers

Test 1:

- Passphrase: TestingOneTwoThree
- Passphrase code: passphrasepxFy57B9v8HtUsszJYKReoNDV6VHjUSGt8EVJmux9n1J3Ltf1gRxyDGXqnf9qm
- Encrypted key: 6PfQu77ygVyJLZjfvMLyhLMQbYnu5uguoJJ4kMCLqWwPEdfpwANVS76gTX
- Bitcoin address: 1PE6TQi6HTVNz5DLwB1LcpMBALubfuN2z2
- Unencrypted private key (WIF): 5K4caxezwjGCGfnoPTZ8tMcJBLB7Jvyjv4xxeacadhq8nLisLR2
- Unencrypted private key (hex): A43A940577F4E97F5C4D39EB14FF083A98187C64EA7C99EF7CE460833959A519

Test 2:

- Passphrase: Satoshi
- Passphrase code: passphraseoRDGAXTWzbp72eVbtUDdn1rwpgPUGjNZEc6CGBo8i5EC1FPW8wcnLdq4ThKzAS
- Encrypted key: 6PfLGnQs6VZnrNpmVKfjotbnQuaJK4KZoPFrAjx1JMJUa1Ft8gnf5WxfKd
- Bitcoin address: 1CqzrtZC6mXSAhoxtFwVjz8LtwLJjDYU3V
- Unencrypted private key (WIF): 5KJ51SgxWaAYR13zd9ReMhJpwrcX47xTJh2D3fGPG9CM8vkv5sH
- Unencrypted private key (hex): C2C8036DF268F498099350718C4A3EF3984D2BE84618C2650F5171DCC5EB660A

EC multiply, no compression, lot/sequence numbers

Test 1:

- Passphrase: MOLON LABE
- Passphrase code: passphraseaB8feaLQDENqCgr4gKZpmf4VoaT6qdjJNJiv7fsKvjqavcJxvuR1hy25aTu5sX
- Encrypted key: 6PgNBNNzDkKdhkT6uJntUXwwzQV8Rr2tZcbkDcuC9DZRsS6AtHts4Ypo1j
- Bitcoin address: 1Jscj8ALrYu2y9TD8NrpvDBugPedmbj4Yh
- Unencrypted private key (WIF): 5JLdxTtcTHcfYcmJsNVy1v2PMDx432JPoYcBTVVRHpPaxUrdtf8
- Unencrypted private key (hex): 44EA95AFBF138356A05EA32110DFD627232D0F2991AD221187BE356F19FA8190
- Confirmation code: cfrm38V8aXBn7JWA1ESmFMUn6erxeBGZGAxJPY4e36S9QWkzZKtaVqLNMgnifETYw7BPwWC9aPD
- Lot/Sequence: 263183/1

Test 2:

- Passphrase (all letters are Greek - test UTF-8 compatibility with this): ΜΟΛΩΝ ΛΑΒΕ
- Passphrase code: passphrased3z9rQJHSyBkNBwTRPkUGNVEVrUAcfAXDyRU1V28ie6hNFbqDwbFBvsTK7yWVK
- Encrypted private key: 6PgGWtx25kUg8QWvwuJAgorN6k9FbE25rv5dMRwu5SKMnfpfVe5mar2ngH
- Bitcoin address: 1Lurmih3KruL4xDB5FmHof38yawNtP9oGf
- Unencrypted private key (WIF): 5KMKKuUmAkiNbA3DazMQiLfDq47qs8MAEThm4yL8R2PhV1ov33D
- Unencrypted private key (hex): CA2759AA4ADB0F96C414F36ABEB8DB59342985BE9FA50FAAC228C8E7D90E3006
- Confirmation code: cfrm38V8G4qq2ywYEFfWLD5Cc6msj9UwsG2Mj4Z6QdGJAFQpdatZLavkgRd1i4iBMdRngDqDs51
- Lot/Sequence: 806938/1

```
BIP: 39
Layer: Applications
Title: Mnemonic code for generating deterministic keys
Authors: Marek Palatinus <slush@satoshilabs.com>
         Pavol Rusnak <stick@satoshilabs.com>
         Aaron Voisine <voisine@gmail.com>
         Sean Bowe <ewillbefull@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2013-09-10
License: MIT
```

BIP 39

Abstract

This BIP describes the implementation of a mnemonic code or mnemonic sentence – a group of easy to remember words – for the generation of deterministic wallets.

It consists of two parts: generating the mnemonic and converting it into a binary seed. This seed can be later used to generate deterministic wallets using BIP-0032 or similar methods.

Copyright

This BIP falls under the MIT License.

Motivation

A mnemonic code or sentence is superior for human interaction compared to the handling of raw binary or hexadecimal representations of a wallet seed. The sentence could be written on paper or spoken over the telephone.

This guide is meant to be a way to transport computer-generated randomness with a human-readable transcription. It's not a way to process user-created sentences (also known as brainwallets) into a wallet seed.

Generating the mnemonic

The mnemonic must encode entropy in a multiple of 32 bits. With more entropy security is improved but the sentence length increases. We refer to the initial entropy length as ENT. The allowed size of ENT is 128-256 bits.

First, an initial entropy of ENT bits is generated. A checksum is generated by taking the first `ENT / 32` bits of its SHA256 hash. This checksum is appended to the end of the initial entropy. Next, these concatenated bits are split into groups of 11 bits, each encoding a number from 0-2047, serving as an index into a wordlist. Finally, we convert these numbers into words and use the joined words as a mnemonic sentence.

The following table describes the relation between the initial entropy length (ENT), the checksum length (CS), and the length of the generated mnemonic sentence (MS) in words.

```
CS = ENT / 32
MS = (ENT + CS) / 11

|  ENT  | CS | ENT+CS |  MS  |
+-------+----+--------+------+
|  128  |  4 |   132  |  12  |
|  160  |  5 |   165  |  15  |
|  192  |  6 |   198  |  18  |
|  224  |  7 |   231  |  21  |
|  256  |  8 |   264  |  24  |
```

Wordlist

An ideal wordlist has the following characteristics:

a) smart selection of words

```
- the wordlist is created in such a way that it's enough to type the first four
  letters to unambiguously identify the word
```

b) similar words avoided

```
- word pairs like "build" and "built", "woman" and "women", or "quick" and "quickly"
  not only make remembering the sentence difficult but are also more error
  prone and more difficult to guess
```

c) sorted wordlists

```
- the wordlist is sorted which allows for more efficient lookup of the code words
  (i.e. implementations can use binary search instead of linear search)
- this also allows trie (a prefix tree) to be used, e.g. for better compression
```

The wordlist can contain native characters, but they must be encoded in UTF-8 using Normalization Form Compatibility Decomposition (NFKD).

From mnemonic to seed

A user may decide to protect their mnemonic with a passphrase. If a passphrase is not present, an empty string "" is used instead.

To create a binary seed from the mnemonic, we use the PBKDF2 function with a mnemonic sentence (in UTF-8 NFKD) used as the password and the string "mnemonic" + passphrase (again in UTF-8 NFKD) used as the salt. The iteration count is set to 2048 and HMAC-SHA512 is used as the pseudo-random function. The length of the derived key is 512 bits (= 64 bytes).

This seed can be later used to generate deterministic wallets using BIP-0032 or similar methods.

The conversion of the mnemonic sentence to a binary seed is completely independent from generating the sentence. This results in a rather simple code; there are no constraints on sentence structure and clients are free to implement their own wordlists or even whole sentence generators, allowing for flexibility in wordlists for typo detection or other purposes.

Although using a mnemonic not generated by the algorithm described in "Generating the mnemonic" section is possible, this is not advised and software must compute a checksum for the mnemonic sentence using a wordlist and issue a warning if it is invalid.

The described method also provides plausible deniability, because every passphrase generates a valid seed (and thus a deterministic wallet) but only the correct one will make the desired wallet available.

Wordlists

Since the vast majority of BIP39 wallets supports only the English wordlist, it is **strongly discouraged** to use non-English wordlists for generating the mnemonic sentences.

If you still feel your application really needs to use a localized wordlist, use one of the following instead of inventing your own.

- Wordlists

Test vectors

The test vectors include input entropy, mnemonic and seed. The passphrase "TREZOR" is used for all vectors.

https://github.com/trezor/python-mnemonic/blob/master/vectors.json

Also see https://github.com/bip32JP/bip32JP.github.io/blob/master/test_JP_BIP39.json

(Japanese wordlist test with heavily normalized symbols as passphrase)

Shortcomings

Some shortcomings have been identified with this proposal:

- Generated seed depends on the wordlist that was used for the mnemonic. Because the "mnemonic to seed" process uses the mnemonic sentence directly rather than the original entropy, translating the mnemonic to a different wordlist necessarily creates a completely different seed. This is not an issue if you only support the English wordlist, as recommended above.
- Because the seed is generated by hashing the mnemonic, it is not possible to represent an arbitrary BIP-0032 seed via a BIP-0039 sentence: the conversion is one-way only (from BIP-0039 sentence to BIP-0032 seed).
- The checksum is short. This means it only gives modest odds of catching random errors (1-in-256 errors will be missed). It is also not able to provide any assistance in correcting errors.
- No versioning scheme. When originally introduced, there was no way to distinguish the address format that should be used for a BIP-0039 key. This is now largely mitigated by use of descriptor wallets (BIP-0380) in addition to a seed however.

Related Work

The authors of BIP-0039 proposed the SLIP-0039 scheme as an intended successor to BIP-0039 improving on the above shortcomings.

Reference Implementation

Reference implementation including wordlists is available from

http://github.com/trezor/python-mnemonic

```
BIP: 43
Layer: Applications
Title: Purpose Field for Deterministic Wallets
Authors: Marek Palatinus <slush@satoshilabs.com>
         Pavol Rusnak <stick@satoshilabs.com>
Status: Deployed
Type: Specification
Assigned: 2014-04-24
```

BIP 43

Abstract

This BIP introduces a "Purpose Field" for use in deterministic wallets based on algorithm described in BIP-0032 (BIP32 from now on).

Motivation

Although Hierarchical Deterministic Wallet structure as described by BIP32 is an important step in user experience and security of the cryptocoin wallets, the BIP32 specification offers implementers too many degrees of freedom. Multiple implementations may claim they are BIP32 compatible, but in fact they can produce wallets with different logical structures making them non-interoperable. This situation unfortunately renders "BIP32 compatible" statement rather useless.

Purpose

We propose the first level of BIP32 tree structure to be used as "purpose". This purpose determines the further structure beneath this node.

```
m / purpose' / *
```

Apostrophe indicates that BIP32 hardened derivation is used.

We encourage different schemes to apply for assigning a separate BIP number and use the same number for purpose field, so addresses won't be generated from overlapping BIP32 spaces.

Purpose codes from 10001 to 19999 are reserved for SLIPs.

Example: Scheme described in BIP44 should use 44' (or 0x8000002C) as purpose.

Note that m / 0' / * is already taken by BIP32 (default account), which preceded this BIP.

Not all wallets may want to support the full range of features and possibilities described in these BIPs. Instead of choosing arbitrary subset of defined features and calling themselves BIPxx compatible, we suggest that software which needs only a limited structure should describe such structure in another BIP and use different "purpose" value.

Node serialization

Because this scheme can be used to generate nodes for more cryptocurrencies at once, or even something totally unrelated to cryptocurrencies, there's no point in using a special version magic described in section "Serialization format" of BIP32. We suggest to use always 0x0488B21E for public and 0x0488ADE4 for private nodes (leading to prefixes "xpub" and "xprv" respectively).

Reference

- BIP32 - Hierarchical Deterministic Wallets

```
BIP: 44
Layer: Applications
Title: Multi-Account Hierarchy for Deterministic Wallets
Authors: Marek Palatinus <slush@satoshilabs.com>
         Pavol Rusnak <stick@satoshilabs.com>
Comments-Summary: Mixed review (one person)
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0044
Status: Deployed
Type: Specification
```

```
Assigned: 2014-04-24
Requires: 32, 43
```

BIP 44

Abstract

This BIP defines a logical hierarchy for deterministic wallets based on an algorithm described in BIP-0032 (BIP32 from now on) and purpose scheme described in BIP-0043 (BIP43 from now on).

This BIP is a particular application of BIP43.

Motivation

The hierarchy proposed in this paper is quite comprehensive. It allows the handling of multiple coins, multiple accounts, external and internal chains per account and millions of addresses per chain.

Path levels

We define the following 5 levels in BIP32 path:

```
m / purpose' / coin_type' / account' / change / address_index
```

Apostrophe in the path indicates that BIP32 hardened derivation is used.

Each level has a special meaning, described in the chapters below.

Purpose

Purpose is a constant set to 44' (or 0x8000002C) following the BIP43 recommendation. It indicates that the subtree of this node is used according to this specification.

Hardened derivation is used at this level.

Coin type

One master node (seed) can be used for unlimited number of independent cryptocoins such as Bitcoin, Litecoin or Namecoin. However, sharing the same space for various cryptocoins has some disadvantages.

This level creates a separate subtree for every cryptocoin, avoiding reusing addresses across cryptocoins and improving privacy issues.

Coin type is a constant, set for each cryptocoin. Cryptocoin developers may ask for registering unused number for their project.

The list of already allocated coin types is in the chapter "Registered coin types" below.

Hardened derivation is used at this level.

Account

This level splits the key space into independent user identities, so the wallet never mixes the coins across different accounts.

Users can use these accounts to organize the funds in the same fashion as bank accounts; for donation purposes (where all addresses are considered public), for saving purposes, for common expenses etc.

Accounts are numbered from index 0 in sequentially increasing manner. This number is used as child index in BIP32 derivation.

Hardened derivation is used at this level.

Software should prevent a creation of an account if a previous account does not have a transaction history (meaning none of its addresses have been used before).

Software needs to discover all used accounts after importing the seed from an external source. Such an algorithm is described in "Account discovery" chapter.

Change

Constant 0 is used for external chain and constant 1 for internal chain (also known as change addresses). External chain is used for addresses that are meant to be visible outside of the wallet (e.g. for receiving payments). Internal chain is used for addresses which are not meant to be visible outside of the wallet and is used for return transaction change.

Public derivation is used at this level.

Index

Addresses are numbered from index 0 in sequentially increasing manner. This number is used as child index in BIP32 derivation.

Public derivation is used at this level.

Account discovery

When the master seed is imported from an external source the software should start to discover the accounts in the following manner:

1. derive the first account's node (index = 0)
2. derive the external chain node of this account
3. scan addresses of the external chain; respect the gap limit described below
4. if no transactions are found on the external chain, stop discovery
5. if there are some transactions, increase the account index and go to step 1

This algorithm is successful because software should disallow creation of new accounts if previous one has no transaction history, as described in chapter "Account" above.

Please note that the algorithm works with the transaction history, not account balances, so you can have an account with 0 total coins and the algorithm will still continue with discovery.

Address gap limit

Address gap limit is currently set to 20. If the software hits 20 unused addresses in a row, it expects there are no used addresses beyond this point and stops searching the address chain. We scan just the external chains, because internal chains receive only coins that come from the associated external chains.

Wallet software should warn when the user is trying to exceed the gap limit on an external chain by generating a new address.

Registered coin types

These are the default registered coin types for usage in level 2 of BIP44 described in chapter "Coin type" above.

All these constants are used as hardened derivation.

index	hexa	coin
0	0x80000000	Bitcoin
1	0x80000001	Bitcoin Testnet

This BIP is not a central directory for the registered coin types, please visit SatoshiLabs that maintains the full list:

SLIP-0044 : Registered coin types for BIP-0044

To register a new coin type, an existing wallet that implements the standard is required and a pull request to the above file should be created.

Examples

coin	account	chain	address	path
Bitcoin	first	external	first	m / 44′ / 0′ / 0′ / 0 / 0
Bitcoin	first	external	second	m / 44′ / 0′ / 0′ / 0 / 1
Bitcoin	first	change	first	m / 44′ / 0′ / 0′ / 1 / 0
Bitcoin	first	change	second	m / 44′ / 0′ / 0′ / 1 / 1
Bitcoin	second	external	first	m / 44′ / 0′ / 1′ / 0 / 0
Bitcoin	second	external	second	m / 44′ / 0′ / 1′ / 0 / 1
Bitcoin	second	change	first	m / 44′ / 0′ / 1′ / 1 / 0
Bitcoin	second	change	second	m / 44′ / 0′ / 1′ / 1 / 1
Bitcoin Testnet	first	external	first	m / 44′ / 1′ / 0′ / 0 / 0
Bitcoin Testnet	first	external	second	m / 44′ / 1′ / 0′ / 0 / 1
Bitcoin Testnet	first	change	first	m / 44′ / 1′ / 0′ / 1 / 0

coin	account	chain	address	path
Bitcoin Testnet	first	change	second	m / 44′ / 1′ / 0′ / 1 / 1
Bitcoin Testnet	second	external	first	m / 44′ / 1′ / 1′ / 0 / 0
Bitcoin Testnet	second	external	second	m / 44′ / 1′ / 1′ / 0 / 1
Bitcoin Testnet	second	change	first	m / 44′ / 1′ / 1′ / 1 / 0
Bitcoin Testnet	second	change	second	m / 44′ / 1′ / 1′ / 1 / 1

Reference

- BIP32 - Hierarchical Deterministic Wallets
- BIP43 - Purpose Field for Deterministic Wallets

```
BIP: 49
Layer: Applications
Title: Derivation scheme for P2WPKH-nested-in-P2SH based accounts
Authors: Daniel Weigl <DanielWeigl@gmx.at>
Comments-Summary: No comments yet.
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0049
Status: Deployed
Type: Specification
Assigned: 2016-05-19
License: PD
```

BIP 49

Abstract

This BIP defines the derivation scheme for HD wallets using the P2WPKH-nested-in-P2SH (BIP 141) serialization format for segregated witness transactions.

Motivation

With the usage of P2WPKH-nested-in-P2SH (BIP 141) transactions it is necessary to have a common derivation scheme. It allows the user to use different HD wallets with the same masterseed and/or a single account seamlessly.

Thus the user needs to create dedicated segregated witness accounts, which ensures that only wallets compatible with this BIP will detect the accounts and handle them appropriately.

Considerations

Two generally different approaches are possible for current BIP44 capable wallets:

1) Allow the user to use the same account(s) that they already use, but add segregated witness encoded addresses to it.

1.1) Use the same public keys as defined in BIP44, but in addition to the normal P2PKH address also derive the P2SH address from it.

1.2) Use the same account root, but branch off and derive different external and internal chain roots to derive dedicated public keys for the segregated witness addresses.

2) Create dedicated accounts used only for segregated witness addresses.

The solutions from point 1 have a common disadvantage: if a user imports/recovers a BIP49-compatible wallet masterseed into/in a non-BIP49-compatible wallet, the account might show up but also it might miss some UTXOs.

Therefore this BIP uses solution 2, which fails in a more visible way. Either the account shows up or not at all. The user does not have to check his balance after using the same seed in different wallets.

Specifications

This BIP defines the two needed steps to derive multiple deterministic addresses based on a BIP 32 root account.

Public key derivation

To derive a public key from the root account, this BIP uses the same account-structure as defined in BIP 44, but only uses a different purpose value to indicate the different transaction serialization method.

```
m / purpose' / coin_type' / account' / change / address_index
```

For the `purpose`-path level it uses `49'`. The rest of the levels are used as defined in BIP44.

Address derivation

To derive the P2SH address from the above calculated public key, we use the encapsulation defined in BIP 141:

```
witness:      <signature> <pubkey>
scriptSig:    <0 <20-byte-key-hash>>
              (0x160014{20-byte-key-hash})
scriptPubKey: HASH160 <20-byte-script-hash> EQUAL
              (0xA914{20-byte-script-hash}87)
```

Extended Key Version

When serializing extended keys, this scheme uses alternate version bytes. Extended public keys use `0x049d7cb2` to produce a “ypub” prefix, and private keys use `0x049d7878` to produce a “yprv” prefix. Testnet uses `0x044a5262` “upub” and `0x044a4e28` “uprv.”

Additional registered version bytes are listed in SLIP-0132.

Backwards Compatibility

This BIP is not backwards compatible by design as described under considerations. An incompatible wallet will not discover accounts at all and the user will notice that something is wrong.

Test vectors

```
  masterseedWords = abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon about
  masterseed =
uprv8tXDerPXZ1QsVNjUJWTurs9kA1KGfKUAts74GCkcXtU8GwnH33GDRbNJpEqTvipfCyycARtQJhmdfWf8oKt41X9LL1ze
D2pLsWmxEk3VAwd (testnet)

  // Account 0, root = m/49'/1'/0'
  account0Xpriv =
uprv91G7gZkzehuMVxDJTYE6tLivdF8e4rvzSu1LFfKw3b2Qx1Aj8vpoFnHdfUZ3hmi9jsvPifmZ24RTN2KhwB8BfMLTVqaB
ReibyaFFcTP1s9n (testnet)
  account0Xpub =
upub5EFU65HtV5TeiSHmZZm7FUffBGy8UKeqp7vw43jYbvZPpoVsgU93oac7Wk3u6moKegAEWtGNF8DehrnHtv21XXEMYRUo
cHqguyjknFHYfgY (testnet)

  // Account 0, first receiving private key = m/49'/1'/0'/0/0
  account0recvPrivateKey = cULrpoZGXiuC19Uhvykx7NugygA3k86b3hmdCeyvHYQZSxojGyXJ
  account0recvPrivateKeyHex = 0xc9bdb49cfbaedca21c4b1f3a7803c34636b1d7dc55a717132443fc3f4c5867e8
  account0recvPublicKeyHex =
0x03a1af804ac108a8a51782198c2d034b28bf90c8803f5a53f76276fa69a4eae77f

  // Address derivation
  keyhash = HASH160(account0recvPublicKeyHex) = 0x38971f73930f6c141d977ac4fd4a727c854935b3
  scriptSig = <0 <keyhash>> = 0x001438971f73930f6c141d977ac4fd4a727c854935b3
  addressBytes = HASH160(scriptSig) = 0x336caa13e08b96080a32b5d818d59b4ab3b36742

  // addressBytes base58check encoded for testnet
  address = base58check(prefix | addressBytes) = 2Mww8dCYPUpKHofjgcXcBCEGmniw9CoaiD2 (testnet)
```

Reference

- BIP16 - Pay to Script Hash
- BIP32 - Hierarchical Deterministic Wallets
- BIP43 - Purpose Field for Deterministic Wallets
- BIP44 - Multi-Account Hierarchy for Deterministic Wallets
- BIP141 - Segregated Witness (Consensus layer)

Copyright

This document is placed in the public domain.

```
  BIP: 84
  Layer: Applications
  Title: Derivation scheme for P2WPKH based accounts
  Authors: Pavol Rusnak <stick@satoshilabs.com>
  Comments-Summary: No comments yet.
  Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0084
  Status: Deployed
  Type: Specification
  Assigned: 2017-12-28
  License: CC0-1.0
```

BIP 84

Abstract

This BIP defines the derivation scheme for HD wallets using the P2WPKH (BIP 173) serialization format for segregated witness transactions.

Motivation

With the usage of P2WPKH transactions it is necessary to have a common derivation scheme. It allows the user to use different HD wallets with the same masterseed and/or a single account seamlessly.

Thus the user needs to create dedicated segregated witness accounts, which ensures that only wallets compatible with this BIP will detect the accounts and handle them appropriately.

Considerations

We use the same rationale as described in Considerations section of BIP 49.

Specifications

This BIP defines the two needed steps to derive multiple deterministic addresses based on a BIP 32 root account.

Public key derivation

To derive a public key from the root account, this BIP uses the same account-structure as defined in BIP 44 and BIP 49, but only uses a different purpose value to indicate the different transaction serialization method.

`m / purpose' / coin_type' / account' / change / address_index`

For the `purpose`-path level it uses `84'`. The rest of the levels are used as defined in BIP44 or BIP49.

Address derivation

To derive the P2WPKH address from the above calculated public key, we use the encapsulation defined in BIP 141:

```
witness:      <signature> <pubkey>
scriptSig:    (empty)
scriptPubKey: 0 <20-byte-key-hash>
              (0x0014{20-byte-key-hash})
```

Extended Key Version

When serializing extended keys, this scheme uses alternate version bytes. Extended public keys use `0x04b24746` to produce a “zpub” prefix, and private keys use `0x04b2430c` to produce a “zprv” prefix. Testnet uses `0x045f1cf6` “vpub” and `0x045f18bc` “vprv.”

Additional registered version bytes are listed in SLIP-0132.

Backwards Compatibility

This BIP is not backwards compatible by design as described under considerations. An incompatible wallet will not discover accounts at all and the user will notice that something is wrong.

Test vectors

```
  mnemonic = abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon about
  rootpriv =
zprvAWgYBBk7JR8Gjrh4UJQ2uJdG1r3WNRRfURiABBE3RvMXYSrRJL62XuezvGdPvG6GFBZduosCc1YP5wixPox7zhZLfiUm
8aunE96BBa4Kei5
  rootpub  =
zpub6jftahH18ngZxLmXaKw3GSZzZsszmt9WqedkyZdezFtWRFBZqsQH5hyUmb4pCEeZGmVfQuP5bedXTB8is6fTv19U1GQR
yQUKQGUTzyHACMF

  // Account 0, root = m/84'/0'/0'
  xpriv =
zprvAdG4iTXWBoARxkkzNpNh8r6Qag3irQB8PzEMkAFeTRXxHpbF9z4QgEvBRmfvqWvGp42t42nvgGpNgYSJA9iefm1yYNZK
Em7z6qUWCroSQnE
  xpub  =
zpub6rFR7y4Q2AijBEqTUquhVz398htDFrtymD9xYYfG1m4wAcvPhXNfE3EfH1r1ADqtfSdVCToUG868RvUUkgDKf31mGDtK
sAYz2oz2AGutZYs

  // Account 0, first receiving address = m/84'/0'/0'/0/0
  privkey = KyZpNDKnfs94vbrwhJneDi77V6jF64PWPF8x5cdJb8ifgg2DUc9d
  pubkey  = 0330d54fd0dd420a6e5f8d3624f5f3482cae350f79d5f0753bf5beef9c2d91af3c
  address = bc1qcr8te4kr609gcawutmrza0j4xv80jy8z306fyu

  // Account 0, second receiving address = m/84'/0'/0'/0/1
  privkey = Kxpf5b8p3qX56DKEe5NqWbNUP9MnqoRFzZwHRtsFqhzuvUJsYZCy
  pubkey  = 03e775fd51f0dfb8cd865d9ff1cca2a158cf651fe997fdc9fee9c1d3b5e995ea77
  address = bc1qnjg0jd8228aq7egyzacy8cys3knf9xvrerkf9g

  // Account 0, first change address = m/84'/0'/0'/1/0
  privkey = KxuoxufJL5csa1Wieb2kp29VNdn92Us8CoaUG3aGtPtcF3AzeXvF
  pubkey  = 03025324888e429ab8e3dbaf1f7802648b9cd01e9b418485c5fa4c1b9b5700e1a6
  address = bc1q8c6fshw2dlwun7ekn9qwf37cu2rn755upcp6el
```

Reference

- BIP32 - Hierarchical Deterministic Wallets
- BIP43 - Purpose Field for Deterministic Wallets
- BIP44 - Multi-Account Hierarchy for Deterministic Wallets
- BIP49 - Derivation scheme for P2WPKH-nested-in-P2SH based accounts
- BIP141 - Segregated Witness (Consensus layer)
- BIP173 - Base32 address format for native v0-16 witness outputs

```
BIP: 85
Layer: Applications
Title: Deterministic Entropy From BIP32 Keychains
```

Authors: Ethan Kosakovsky <ethankosakovsky@protonmail.com>
         Aneesh Karve <dowsing.seaport0d@icloud.com>
Status: Deployed
Type: Informational
Assigned: 2020-03-20
License: BSD-2-Clause OR OPUBL-1.0
Version: 2.1.0

BIP 85

Abstract

"One Seed to rule them all,
One Key to find them,
One Path to bring them all,
And in cryptography bind them."

It is not possible to maintain one single (mnemonic) seed backup for all keychains used across various wallets because there are a variety of incompatible standards. Sharing of seeds across multiple wallets is not desirable for security reasons. Physical storage of multiple seeds is difficult depending on the security and redundancy required.

As HD keychains are essentially derived from initial entropy, this proposal provides a way to derive entropy from the keychain which can be fed into whatever method a wallet uses to derive the initial mnemonic seed or root key.

Copyright

This BIP is dual-licensed under the Open Publication License and BSD 2-clause license.

Definitions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

The terminology related to keychains used in the wild varies widely, for example `seed` has various different meanings. In this document we define the terms

1. **BIP32 root key** is the root extended private key that is represented as the top root of the keychain in BIP32.
2. **BIP39 mnemonic** is the mnemonic phrase that is calculated from the entropy used before hashing of the mnemonic in BIP39.
3. **BIP39 seed** is the result of hashing the BIP39 mnemonic seed.

When in doubt, assume big endian byte serialization, such that the leftmost byte is the most significant.

Motivation

Most wallets implement BIP32 which defines how a BIP32 root key can be used to derive keychains. As a consequence, a backup of just the BIP32 root key is sufficient to include all keys derived from it. BIP32 does not have a human-friendly serialization of the BIP32 root key (or BIP32 extended keys in general), which makes paper backups or manually restoring the key more error-prone. BIP39 was designed to solve this problem, but rather than serialize the BIP32 root key, it takes some entropy, encoded to a "seed mnemonic", which is then hashed to derive the BIP39 seed, which can be turned into the BIP32 root key. Saving the BIP39 mnemonic is enough to reconstruct the entire BIP32 keychain, but a BIP32 root key cannot be reversed back to the BIP39 mnemonic.

Most wallets implement BIP39, so on initialization or restoration, the user must interact with a BIP39 mnemonic. Most wallets do not support BIP32 extended private keys, so each wallet must either share the same BIP39 mnemonic, or have a separate BIP39 mnemonic entirely. Neither scenario is particularly satisfactory for security reasons. For example, some wallets may be inherently less secure, like hot wallets on smartphones, JoinMarket servers, or Lightning Network nodes. Having multiple seeds is far from desirable, especially for those who rely on split key or redundancy backups in different geological locations. Adding keys is necessarily difficult and may result in users being more lazy with subsequent keys, resulting in compromised security or loss of keys.

There is an added complication with wallets that implement other standards, or no standards at all. The Bitcoin Core wallet uses a WIF as the *hdseed*, and yet other wallets, like Electrum, use different mnemonic schemes to derive the BIP32 root key. Other cryptocurrencies, like Monero, use an entirely different mnemonic scheme.

Ultimately, all of the mnemonic/seed schemes start with some "initial entropy" to derive a mnemonic/seed, and then process the mnemonic into a BIP32 key, or private key. We can use BIP32 itself to derive the "initial entropy" to then recreate the same mnemonic or seed according to the specific application standard of the target wallet. We can use a BIP44-like categorization to ensure uniform derivation according to the target application type.

Specification

We assume a single BIP32 master root key. This specification is not concerned with how this was derived (e.g. directly or via a mnemonic scheme such as BIP39).

For each application that requires its own wallet, a unique private key is derived from the BIP32 master root key using a fully hardened derivation path. The resulting private key (k) is then processed with HMAC-SHA512, where the key is "bip-entropy-from-k", and the message payload is the private key k: `HMAC-SHA512(key="bip-entropy-from-k", msg=k)`

The reason for running the derived key through HMAC-SHA512 and truncating the result as necessary is to prevent leakage of the parent tree should the derived key (*k*) be compromised. While the specification requires the use of hardened key derivation which would prevent this, we cannot enforce hardened derivation, so this method ensures the derived entropy is hardened. Also, from a semantic point of view, since the purpose is to derive entropy and not a private key, we are required to transform the child key. This is done out of an abundance of caution, in order to ward off unwanted side effects should *k* be used for a dual purpose, including as a nonce *hash(k)*, where undesirable and unforeseen interactions could occur. . The result produces

512 bits of entropy. Each application SHOULD use up to the required number of bits necessary for their operation, and truncate the rest.

The HMAC-SHA512 function is specified in RFC 4231.

Test vectors

Test case 1

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/0'/0'

OUTPUT:

- DERIVED KEY=cca20ccb0e9a90feb0912870c3323b24874b0ca3d8018c4b96d0b97c0e82ded0
- DERIVED ENTROPY=efecfbccffea313214232d29e71563d941229afb4338c21f9517c41aaa0d16f00b83d2a09ef747e7a64e8e2bd5a14869

Test case 2

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/0'/1'

OUTPUT

- DERIVED KEY=503776919131758bb7de7beb6c0ae24894f4ec042c26032890c29359216e21ba
- DERIVED ENTROPY=70c6e3e8ebee8dc4c0dbba66076819bb8c09672527c4277ca8729532ad711872218f826919f6b67218adde99018a6

BIP85-DRNG

BIP85-DRNG-SHAKE256 is a deterministic random number generator for cryptographic functions that require deterministic outputs, but where the input to that function requires more than the 64 bytes provided by BIP85's HMAC output. BIP85-DRNG-SHAKE256 uses BIP85 to seed a SHAKE256 stream (from the SHA-3 standard). The input must be exactly 64 bytes long (from the BIP85 HMAC output).

RSA key generation is an example of a function that requires orders of magnitude more than 64 bytes of random input. Further, it is not possible to precalculate the amount of random input required until the function has completed.

```
drng_reader = BIP85DRNG.new(bip85_entropy)
rsa_key = RSA.generate_key(4096, drng_reader.read)
```

Test Vectors

INPUT:
xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1LqdGhUt

- MASTER BIP32 ROOT KEY: m/83696968'/0'/0'

OUTPUT

- DERIVED KEY=cca20ccb0e9a90feb0912870c3323b24874b0ca3d8018c4b96d0b97c0e82ded0
- DERIVED ENTROPY=efecfbccffea313214232d29e71563d941229afb4338c21f9517c41aaa0d16f00b83d2a09ef747e7a64e8e2bd5a14869
- DRNG(80 bytes)=b78b1ee6b345eae6836c2d53d33c64cdaf9a696487be81b03e822dc84b3f1cd883d7559e53d175f243e4c349e822a957bb

Applications

The Application number defines how entropy will be used post processing. Some basic examples follow:

Derivation paths follow the format `m/83696968'/{app_no}'/{index}'`, where *{app_no}* is the path for the application, and *{index}* is the index.

Application numbers should be semantic in some way, such as a BIP number or ASCII character code sequence.

BIP39

Application number: 39'

Truncate trailing (least significant) bytes of the entropy to the number of bits required to map to the relevant word length: 128 bits for 12 words, 256 bits for 24 words.

The derivation path format is: `m/83696968'/39'/{language}'/{words}'/{index}'`

Example: a BIP39 mnemonic with 12 English words (first index) would have the path `m/83696968'/39'/0'/12'/0'`, the next key would be `m/83696968'/39'/0'/12'/1'` etc.

Language Table

Wordlist	**Code**
English	0'
Japanese	1'
Korean	2'
Spanish	3'
Chinese (Simplified)	4'
Chinese (Traditional)	5'
French	6'

Wordlist	Code
Italian	7′
Czech	8′
Portuguese	9′

Words Table

Words	Entropy	Code
12 words	128 bits	12′
15 words	160 bits	15′
18 words	192 bits	18′
21 words	224 bits	21′
24 words	256 bits	24′

12 English words

BIP39 English 12 word mnemonic seed

128 bits of entropy as input to BIP39 to derive 12 word mnemonic

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968′/39′/0′/12′/0′

OUTPUT:

- DERIVED ENTROPY=6250b68daf746d12a24d58b4787a714b
- DERIVED BIP39 MNEMONIC=girl mad pet galaxy egg matter matrix prison refuse sense ordinary nose

18 English words

BIP39 English 18 word mnemonic seed

192 bits of entropy as input to BIP39 to derive 18 word mnemonic

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968′/39′/0′/18′/0′

OUTPUT:

- DERIVED ENTROPY=938033ed8b12698449d4bbca3c853c66b293ea1b1ce9d9dc

- DERIVED BIP39 MNEMONIC=near account window bike charge season chef number sketch tomorrow excuse sniff circle vital hockey outdoor supply token

24 English words

Derives 24 word BIP39 mnemonic seed

256 bits of entropy as input to BIP39 to derive 24 word mnemonic

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/39'/0'/24'/0'

OUTPUT:

- DERIVED ENTROPY=ae131e2312cdc61331542efe0d1077bac5ea803adf24b313a4f0e48e9c51f37f
- DERIVED BIP39 MNEMONIC=puppy ocean match cereal symbol another shed magic wrap hammer bulb intact gadget divorce twin tonight reason outdoor destroy simple truth cigar social volcano

HD-Seed WIF

Application number: 2'

Uses the most significant 256 bits There is a very small chance that you'll make an invalid key that is zero or larger than the order of the curve. If this occurs, software should hard fail (forcing users to iterate to the next index). From BIP32:

> In case $parse_{256}(I_L) \geq n$ or $k_i = 0$, the resulting key is invalid, and one should proceed with the next value for i. (Note: this has probability lower than 1 in 2^{127}.)

of entropy as the secret exponent to derive a private key and encode as a compressed WIF that will be used as the hdseed for Bitcoin Core wallets.

Path format is `m/83696968'/2'/{index}'`

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/2'/0'

OUTPUT

- DERIVED ENTROPY=7040bb53104f27367f317558e78a994ada7296c6fde36a364e5baf206e502bb1
- DERIVED WIF=Kzyv4uF39d4Jrw2W7UryTHwZr1zQVNk4dAFyqE6BuMrMh1Za7uhp

XPRV

Application number: 32'

Taking 64 bytes of the HMAC digest, the first 32 bytes are the chain code, and the second 32 bytes are the private key for the BIP32 XPRV value. Child number, depth, and parent fingerprint are forced to zero.

Warning: The above order reverses the order of BIP32, which takes the first 32 bytes as the private key, and the second 32 bytes as the chain code.

Applications may support Testnet by emitting TPRV keys if and only if the input root key is a Testnet key.

Path format is `m/83696968'/32'/{index}'`

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/32'/0'

OUTPUT

- DERIVED ENTROPY=ead0b33988a616cf6a497f1c169d9e92562604e38305ccd3fc96f2252c177682
- DERIVED XPRV=xprv9s21ZrQH143K2srSbCSg4m4kLvPMzcWydgmKEnMmoZUurYuBuYG46c6P71UGXMzmriLzCCBvKQWB

HEX

Application number: 128169'

The derivation path format is: `m/83696968'/128169'/{num_bytes}'/{index}'`

`16 <= num_bytes <= 64`

Truncate trailing (least significant) bytes of the entropy after `num_bytes`.

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/128169'/64'/0'

OUTPUT

- DERIVED ENTROPY=492db4698cf3b73a5a24998aa3e9d7fa96275d85724a91e71aa2d645442f878555d078fd1f1f67e368976f04137b1f7

PWD BASE64

Application number: 707764'

The derivation path format is: `m/83696968'/707764'/{pwd_len}'/{index}'`

`20 <= pwd_len <= 86`

Base64 encode all 64 bytes of entropy. Remove any spaces or new lines inserted by Base64 encoding process. Slice Base64 result string on index 0 to `pwd_len`. This slice is the password. As `pwd_len` is limited to 86, passwords will not contain padding.

Entropy calculation:
R = 64 (Base64 - do not count padding)
L = pwd_len
Entropy = log2(R ** L)

pwd_length	(cca) entropy
20	120.0
24	144.0
32	192.0
64	384.0
86	516.0

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/707764'/21'/0'

OUTPUT

- DERIVED ENTROPY=74a2e87a9ba0cdd549bdd2f9ea880d554c6c355b08ed25088cfa88f3f1c4f74632b652fd4a8f5fda43074c6f6964a37
- DERIVED PWD=dKLoepugzdVJvdL56ogNV

PWD BASE85

Application number: 707785'

The derivation path format is: `m/83696968'/707785'/{pwd_len}'/{index}'`

`10 <= pwd_len <= 80`

Base85 encode all 64 bytes of entropy. Remove any spaces or new lines inserted by Base85 encoding process. Slice Base85 result string on index 0 to `pwd_len`. This slice is the password. `pwd_len` is limited to 80 characters.

Entropy calculation:
R = 85
L = pwd_len
Entropy = log2(R ** L)

pwd_length	(cca) entropy
10	64.0

pwd_length	(cca) entropy
15	96.0
20	128.0
30	192.0
80	512.0

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/707785'/12'/0'

OUTPUT

- DERIVED ENTROPY=f7cfe56f63dca2490f65fcbf9ee63dcd85d18f751b6b5e1c1b8733af6459c904a75e82b4a22efff9b9e69de2144b293a
- DERIVED PWD=_s`{TW89)i4`

RSA

Application number: 828365'

The derivation path format is: `m/83696968'/828365'/{key_bits}'/{key_index}'`

The RSA key generator should use BIP85-DRNG as the input RNG function.

RSA GPG

Keys allocated for RSA-GPG purposes use the following scheme:

```
- Main key ``m/83696968'/828365'/{key_bits}'/{key_index}'
- Sub keys:  ``m/83696968'/828365'/{key_bits}'/{key_index}'/{sub_key}'

   - key_index is the parent key for CERTIFY capability
   - sub_key ``0''' is used as the ENCRYPTION key
   - sub_key ``1''' is used as the AUTHENTICATION key
   - sub_key ``2''' is usually used as SIGNATURE key
```

Note on timestamps:

The resulting RSA key can be used to create a GPG key where the creation date MUST be fixed to UNIX Epoch timestamp 1231006505 (the Bitcoin genesis block time `'2009-01-03 18:15:05'` UTC)The human-readable datetime string was incorrectly noted as '2009-01-03 18:05:05' prior to v2.0.0 of this BIP, so implementations that relied on it rather than UNIX Epoch timestamp 1231006505 will produce different key fingerprints. because the key fingerprint is affected by the creation date (Epoch timestamp 0 was not chosen because of legacy behavior in GNUPG implementations for older keys). Additionally, when importing sub-keys under a key in GNUPG, the system time must be frozen to the same timestamp before importing (e.g. by use of `faketime`).

Note on GPG key capabilities on smartcard/hardware devices:

GPG capable smart-cards SHOULD be loaded as follows: The encryption slot SHOULD be loaded with the ENCRYPTION capable key; the authentication slot SHOULD be loaded with the AUTHENTICATION capable key. The signature capable slot SHOULD be loaded with the SIGNATURE capable key.

However, depending on available slots on the smart-card, and preferred policy, the CERTIFY capable key MAY be flagged with CERTIFY and SIGNATURE capabilities and loaded into the SIGNATURE capable slot (for example where the smart-card has only three slots and the CERTIFY capability is required on the same card). In this case, the SIGNATURE capable sub-key would be disregarded because the CERTIFY capable key serves a dual purpose.

DICE

Application number: 89101'

The derivation path format is: `m/83696968'/89101'/{sides}'/{rolls}'/{index}'`

```
2 <= sides <= 2^32 - 1
1 <= rolls <= 2^32 - 1
```

Use this application to generate PIN numbers, numeric secrets, and secrets over custom alphabets. For example, applications could generate alphanumeric passwords from a 62-sided die (26 + 26 + 10).

Roll values are zero-indexed, such that an N-sided die produces values in the range `[0, N-1]`, inclusive. Applications should separate printed rolls by a comma or similar.

Create a BIP85 DRNG whose seed is the derived entropy.

Calculate the following integers:

```
bits_per_roll = ceil(log_2(sides))
bytes_per_roll = ceil(bits_per_roll / 8)
```

Read `bytes_per_roll` bytes from the DRNG. Trim any bits in excess of `bits_per_roll` (retain the most significant bits). The resulting integer represents a single roll or trial. If the trial is greater than or equal to the number of sides, skip it and move on to the next one. Repeat as needed until all rolls are complete.

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/89101'/6'/10'/0'

OUTPUT

- DERIVED ENTROPY=5e41f8f5d5d9ac09a20b8a5797a3172b28c806aead00d27e36609e2dd116a59176a738804236586f668da8a51b90c
- DERIVED ROLLS=1,0,0,2,0,1,5,5,2,4

Nostr

Application number: 128002'

The derivation path format is: `m/83696968'/128002'/{identity}'/{account_index}'`

Each `identity` is an independent, unlinkable Nostr key namespace, and each `account_index` is a distinct key within that identity.

Identity index `0'` and account index `0'` are reserved for future key-management use, such as proof-of-linkage between an identity's keys, key rotation, and revocation. Their semantics are out of scope for this BIP and may be specified by a future NIP. Usable signing keys therefore start at `identity >= 1'` and `account_index >= 1'`.

The standard application of BIP-85 produces 64 bytes of entropy. Take the first 32 bytes (the 256 most significant bits) as the secp256k1 secret key, as in the HD-Seed WIF application, and discard the trailing 32 bytes. Encode the secret key in Bech32 with the `nsec` human-readable part as specified by NIP-19.

identity=1, account_index=1

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/128002'/1'/1'

OUTPUT

- DERIVED ENTROPY=ff6eb0fcdf1ef87a2a06b0d7884d495b486d0faa210e9f80f23fd649d6e114d27873d12f3b57c469f684e4dc9f4abf10
- DERIVED NSEC=nsec1lahtplxlrmu852sxkrtcsn2ftdyx6ra2yy8flq8j8ltyn4hpznfq23uvqz

identity=1, account_index=2

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/128002'/1'/2'

OUTPUT

- DERIVED ENTROPY=917628689652288f6983c8a01db516d0697d5198dcf9de9010597f5e322fba6e435a731d85fbcc5768c06ff95ff34bf5
- DERIVED NSEC=nsec1j9mzs6yk2g5g76vrezspmdgk6p5h65vcmnuaayqst9l4uv30hfhqje0jyh

identity=2, account_index=1

INPUT:

- MASTER BIP32 ROOT KEY: xprv9s21ZrQH143K2LBWUUQRFXhucrQqBpKdRRxNVq2zBqsx8HVqFk2uYo8kmbaLLHRdqtQpUm98uKfu3vca1Lq
- PATH: m/83696968'/128002'/2'/1'

OUTPUT

- DERIVED ENTROPY=fa2e784291b1fd347ba3624672e3090cb2cee34b8567180336e3aa290d02715b8f4b18f02f07cb2aa3023afd3473528
- DERIVED NSEC=nsec1lgh8ss53k87ng7arvfr89ccfpjevac6ts4n3sqekuw4zjrgzw9dsq3uelh

Backwards Compatibility

This specification is not backwards compatible with any other existing specification.

This specification relies on BIP32 but is agnostic to how the BIP32 root key is derived. As such, this standard is able to derive wallets with initialization schemes like BIP39 or Electrum wallet style mnemonics.

References

BIP32, BIP39, NIP-01, NIP-06, NIP-19

Reference Implementations

- 1.3.0 Python 3.x library implementation: 1

Changelog

2.1.0 (2026-08-02)

Added

- Nostr application 128002'

2.0.0 (2025-09-19)

Fixed

- Fixed the human-readable datetime string for BIP85 GPG Keys that was incorrectly stated as '2009-01-03 18:05:05' rather than '2009-01-03 18:15:05'. Implementations that relied on the previously incorrect datetime string instead of UNIX Epoch timestamp 1231006505 will produce different key fingerprints.

1.3.0 (2024-10-22)

Added

- Dice application 89101'
- Czech language code to application 39'
- TPRV guidance for application 32'
- Warning on application 32' key and chain code ordering

1.2.0 (2022-12-04)

Added

- Base64 application 707764'
- Base85 application 707785'

1.1.0 (2020-11-19)

Added

- BIP85-DRNG-SHAKE256
- RSA application 828365'

1.0.0 (2020-06-11)

- Initial version

Footnotes

Acknowledgements

Many thanks to Peter Gray and Christopher Allen for their input, and to Peter for suggesting extra application use cases.

```
BIP: 86
Layer: Applications
Title: Key Derivation for Single Key P2TR Outputs
Authors: Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
Assigned: 2021-06-22
License: BSD-2-Clause
```

BIP 86

Abstract

This document suggests a derivation scheme for HD wallets whose keys are involved in single key P2TR (BIP 341) outputs as the Taproot internal key.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

With the usage of single key P2TR transactions, it is useful to have a common derivation scheme so that HD wallets that only have a backup of the HD seed can be likely to recover single key Taproot outputs. Although

there are now solutions which obviate the need for fixed derivation paths for specific script types, many software wallets and hardware signers still use seed backups which lack derivation path and script information. Thus we largely use the same approach used in BIPs 49 and 84 for ease of implementation.

Specifications

This BIP defines the two needed steps to derive multiple deterministic addresses based on a BIP 32 master private key.

Public key derivation

To derive a public key from the root account, this BIP uses the same account-structure as defined in BIPs 44, 49, and 84, but with a different purpose value for the script type.

```
m / purpose' / coin_type' / account' / change / address_index
```

For the `purpose`-path level it uses `86'`. The rest of the levels are used as defined in BIPs 44, 49, and 84.

A key derived with this derivation path pattern will be referred to as `derived_key` further in this document.

Address derivation

BIP 341 states: "If the spending conditions do not require a script path, the output key should commit to an unspendable script path instead of having no script path. This can be achieved by computing the output key point as $Q = P + int(hash_{TapTweak}(bytes(P)))G$." Thus:

```
internal_key:       lift_x(derived_key)
32_byte_output_key: internal_key + int(HashTapTweak(bytes(internal_key)))G
```

In a transaction, the scripts and witnesses are as defined in BIP 341:

```
witness:      <signature>
scriptSig:    (empty)
scriptPubKey: 1 <32_byte_output_key>
              (0x5120{32_byte_output_key})
```

Backwards Compatibility

This BIP is not backwards compatible by design. An incompatible wallet will not discover these accounts at all and the user will notice that something is wrong.

However this BIP uses the same method used in BIPs 44, 49, and 84, so it should not be difficult to implement.

Test vectors

```
mnemonic = abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon about
rootpriv =
xprv9s21ZrQH143K3GJpoapnV8SFfukcVBSfeCficPSGfubmSFDxo1kuHnLisriDvSnRRuL2Qrg5ggqHKNVpxR86QEC8w35u
xmGoggxtQTPvfUu
rootpub  =
xpub661MyMwAqRbcFkPHucMnrGNzDwb6teAX1RbKQmqtEF8kK3Z7LZ59qafCjB9eCRLiTVG3uxBxgKvRgbubRhqSKXnGGb1a
```

```
oaqLrpMBDrVxga8

// Account 0, root = m/86'/0'/0'
xprv =
xprv9xgqHN7yz9MwCkxsBPN5qetuNdQSUttZNKw1dcYTV4mkaAFiBVGQziHs3NRSWMkCzvgjEe3n9xV8oYywvM8at9yRqyaZ
Vz6TYYhX98VjsUk
xpub =
xpub6BgBgsespWvERF3LHQu6CnqdvfEvtMcQjYrcRzx53QJjSxarj2afYWcLteoGVky7D3UKDP9QyrLprQ3VCECoY49yfdDE
HGCtMMj92pReUsQ

// Account 0, first receiving address = m/86'/0'/0'/0/0
xprv         =
xprvA449goEeU9okwCzzZaxiy475EQGQzBkc65su82nXEvcwzfSskb2hAt2WymrjyRL6kpbVTGL3cKtp9herYXSjjQ1j4sts
XXiRF7kXkCacK3T
xpub         =
xpub6H3W6JmYJXN49h5TfcVjLC3onS6uPeUTTJoVvRC8oG9vsTn2J8LwigLzq5tHbrwAzH9DGo6ThGUdWsqce8dGfwHVBxSb
ixjDADGGdzF7t2B
internal_key = cc8a4bc64d897bddc5fbc2f670f7a8ba0b386779106cf1223c6fc5d7cd6fc115
output_key   = a60869f0dbcf1dc659c9cecbaf8050135ea9e8cdc487053f1dc6880949dc684c
scriptPubKey = 5120a60869f0dbcf1dc659c9cecbaf8050135ea9e8cdc487053f1dc6880949dc684c
address      = bc1p5cyxnuxmeuwuvkwfem96lqzszd02n6xdcjrs20cac6yqjjwudpxqkedrcr

// Account 0, second receiving address = m/86'/0'/0'/0/1
xprv         =
xprvA449goEeU9okyiF1LmKiDaTgeXvmh87DVyRd35VPbsSop8n8uALpbtrUhUXByPFKK7C2yuqrB1FrhiDkEMC4RGmA5KTw
sE1aB5jRu9zHsuQ
xpub         =
xpub6H3W6JmYJXN4CCKUSnriaiQRCZmG6aq4sCMDqTu1ACyngw7HShf59hAxYjXgKDuuHThVEUzdHrc3aXCr9kfvQvZPit5d
nD3K9xVRBzjK3rX
internal_key = 83dfe85a3151d2517290da461fe2815591ef69f2b18a2ce63f01697a8b313145
output_key   = a82f29944d65b86ae6b5e5cc75e294ead6c59391a1edc5e016e3498c67fc7bbb
scriptPubKey = 5120a82f29944d65b86ae6b5e5cc75e294ead6c59391a1edc5e016e3498c67fc7bbb
address      = bc1p4qhjn9zdvkux4e44uhx8tc55attvtyu358kutcqkudyccelu0was9fqzwh

// Account 0, first change address = m/86'/0'/0'/1/0
xprv         =
xprvA3Ln3Gt3aphvUgzgEDT8vE2cYqb4PjFfpmbiFKphxLg1FjXQpkAk5M1ZKDY15bmCAHA35jTiawbFuwGtbDZogKF1Wfjw
xML4gK7WfYW5JRP
xpub         =
xpub6GL8SnQwRCGDhB59LEz9HMyM6sRYoByXBzXK3iEKWgCz8XrZNHUzd9L3AUBELW5NzA7dEFvMas1F84TuPH3xqdUA5tum
aGWFgihJzWytXe3
internal_key = 399f1b2f4393f29a18c937859c5dd8a77350103157eb880f02e8c08214277cef
output_key   = 882d74e5d0572d5a816cef0041a96b6c1de832f6f9676d9605c44d5e9a97d3dc
scriptPubKey = 5120882d74e5d0572d5a816cef0041a96b6c1de832f6f9676d9605c44d5e9a97d3dc
address      = bc1p3qkhfews2uk44qtvauqyr2ttdsw7svhkl9nkm9s9c3x4ax5h60wqwruhk7
```

Reference

- BIP32 - Hierarchical Deterministic Wallets
- BIP43 - Purpose Field for Deterministic Wallets
- BIP44 - Multi-Account Hierarchy for Deterministic Wallets
- BIP49 - Derivation scheme for P2WPKH-nested-in-P2SH based accounts
- BIP84 - Derivation scheme for P2WPKH based accounts
- BIP341 - Taproot: SegWit version 1 spending rules

```
BIP: 87
Layer: Applications
Title: Hierarchy for Deterministic Multisig Wallets
Authors: Robert Spigler <RobertSpigler@ProtonMail.ch>
Status: Complete
Type: Specification
Assigned: 2020-03-11
License: BSD-2-Clause
```

BIP 87

Abstract

This BIP defines a sane hierarchy for deterministic multisig wallets based on an algorithm described in BIP-0032 (BIP32 from now on), purpose scheme described in BIP-0043 (BIP43 from now on), and multi-account hierarchy described in BIP-0044 (BIP44 from now on).

This BIP is a particular application of BIP43.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

With the increase of more user friendly (offline) multisignature wallets, and adoption of new technologies such as the descriptor language and BIP-0174 (Partially Signed Bitcoin Transactions), it is necessary to create a common derivation scheme that makes use of all new technologies.

As background, BIP 44/49/84 specifies:

```
m / purpose' / coin_type' / account' / change / address_index
```

where the BIP43 `purpose'` path is separate for each script (P2PKH, P2WPKH-in-P2SH, and P2WPKH respectively). Having a script-per-derivation for single sig wallets allows for easy backup and restore, with just the private key information.

Multisignature wallets need more information to backup and restore (such as all cosigner public keys), and these per-script derivations are made redundant with descriptors, which provide that information (while also specifying a collection of output scripts). A modern standardization is needed for multisig derivation paths. There are some in existence, but all have issues. For example, BIP45 specifies:

```
m / purpose' / cosigner_index / change / address_index
```

BIP45 unnecessarily demands a single script type (here, P2SH). In addition, BIP45 sets `cosigner_index` in order to sort the `purpose'` public keys of each cosigner. This too is redundant, as descriptors can set the order of the public keys with `multi` or have them sorted lexicographically (as described in BIP67) with `sortedmulti`. Sorting public keys between cosigners in order to create the full derivation path, prior to sending the key record to the coordinator to create the descriptor, merely adds additional unnecessary communication rounds.

The second multisignature "standard" in use is m/48', which specifies:

`m / purpose' / coin_type' / account' / script_type' / change / address_index`

Rather than following in BIP 44/49/84's path and having a separate BIP per script after P2SH (BIP45), vendors decided to insert `script_type'` into the derivation path (where P2SH-P2WSH=1, P2WSH=2, Future_Script=3, etc). As described previously, this is unnecessary, as the descriptor sets the script. While it attempts to reduce maintenance work by getting rid of new BIPs-per-script, it still requires maintaining an updated, redundant, `script_type` list.

The structure proposed later in this paper solves these issues and is quite comprehensive. It allows for the handling of multiple accounts, external and internal chains per account, and millions of addresses per chain, in a multi-party, multisignature, hierarchical deterministic wallet regardless of the script type **Why propose this structure only for multisignature wallets?** Currently, single-sig wallets are able to restore funds using just the master private key data (in the format of BIP39 usually). Even if the user doesn't recall the derivation used, the wallet implementation can iterate through common schemes (BIP44/49/84). With this proposed hierarchy, the user would either have to now backup additional data (the descriptor), or the wallet would have to attempt all script types for every account level when restoring. Because of this, even though the descriptor language handles the signature type just like it does the script type, it is best to restrict this script-agnostic hierarchy to multisignature wallets only..

This paper was inspired from BIP44.

Specification

Key sorting

Any wallet that supports descriptors inherently supports deterministic key sorting as per BIP67 (through the `sortedmulti` function) so that all possible multisignature addresses/scripts are derived from deterministically sorted public keys.

Path levels

We should not be mixing keys and scripts in the same layer. The wallet should create extended private/public keys independent of the script type, whereas the descriptor language tells wallets to watch the multisig outputs with the specified public keys.

We define the following 5 levels in the BIP32 path:

`m / purpose' / coin_type' / account' / change / address_index`

`h` or `'` in the path indicates that BIP32 hardened derivation is used.

Each level has a special meaning, described in the chapters below.

Purpose

Purpose is a constant set to `87'` following the BIP43 recommendation. It indicates that the subtree of this node is used according to this specification.

Hardened derivation is used at this level.

Coin type

One master node (seed) can be used for multiple Bitcoin networks. Sharing the same space for various networks has some disadvantages.

This level creates a separate subtree for every network, avoiding reusing addresses across networks and improving privacy issues.

Coin type `0` for mainnet and `1` for testnets (testnet, regtest, and signet).

Hardened derivation is used at this level.

Account

This level splits the key space into independent user identities, following the BIP44 pattern, so the wallet never mixes the coins across different accounts.

Users can use these accounts to organize the funds in the same fashion as bank accounts; for donation purposes (where all addresses are considered public), for saving purposes, for common expenses, etc.

Accounts are numbered from index `0` in sequentially increasing manner. This number is used as child index in BIP32 derivation.

Hardened derivation is used at this level.

It is crucial that this level is increased for each new wallet joined or private/public keys created; for both privacy and cryptographic purposes. For example, before sending a new key record to a coordinator, the wallet must increment the `account'` level. This prevents key reuse - across ECDSA and Schnorr signatures, across different script types, and in between the same wallet types.

Change

Constant `0` is used for external chain and constant `1` for internal chain (also known as change addresses). External chain is used for addresses that are meant to be visible outside of the wallet (e.g. for receiving payments). Internal chain is used for addresses which are not meant to be visible outside of the wallet and is used for return transaction change.

Public derivation is used at this level.

Index

Addresses are numbered from index `0` in sequentially increasing manner. This number is used as child index in BIP32 derivation.

Public derivation is used at this level.

Address Discovery

The multisig descriptors or descriptor template that is generated from the cosigners' combined key records should be used to generate and discover addresses.

Please see BIP-0129 (Bitcoin Secure Multisig Setup) for an introduction on descriptor templates. The descriptor or descriptor template should contain the key origin information for maximum compatibility with BIP-0174.

For example:

The following descriptor template and derivation path restrictions:

```
wsh(sortedmulti(2,[xfpForA/87'/0'/0']XpubA/**,[xfpForB/87'/0'/0']XpubB/**))
```

```
/0/*,/1/*
```

Expands to the two concrete descriptors:

```
wsh(sortedmulti(2,[xfpForA/87'/0'/0']XpubA/0/*,[xfpForB/87'/0'/0']XpubB/0/*))
```

```
wsh(sortedmulti(2,[xfpForA/87'/0'/0']XpubA/1/*,[xfpForB/87'/0'/0']XpubB/1/*))
```

To discover addresses, import both the receiving and change descriptors; respect the gap limit described below.

Address Gap Limit

Address gap limit is currently set to 20. If the software hits 20 unused addresses in a row, it expects there are no used addresses beyond this point and stops searching the address chain.

Wallet software should warn when the user is trying to exceed the gap limit on an external descriptor by generating multiple unused addresses.

Backwards Compatibility

Any script that is supported by descriptors (and the specific wallet implementation) is compatible with this BIP.

As wallets complying with this BIP are descriptor wallets, this therefore necessitates that the cosigners backup their private key information and the descriptor, in order to properly restore at a later time. This shouldn't be a user burden, since (to much user surprise), all cosigner public keys need to be supplied in addition to `M` seeds in any `M` of `N` multisig restore operation. The descriptor provides this information in a standardized format, with key origin information and error detection.

Rationale

Examples

network	account	chain	address	path
mainnet	first	external	first	m / 87′ / 0′ / 0′ / 0 / 0
mainnet	first	external	second	m / 87′ / 0′ / 0′ / 0 / 1
mainnet	first	change	first	m / 87′ / 0′ / 0′ / 1 / 0
mainnet	first	change	second	m / 87′ / 0′ / 0′ / 1 / 1
mainnet	second	external	first	m / 87′ / 0′ / 1′ / 0 / 0

mainnet	second	external	second	m / 87′ / 0′ / 1′ / 0 / 1
testnet	first	external	first	m / 87′ / 1′ / 0′ / 0 / 0
testnet	first	external	second	m / 87′ / 1′ / 0′ / 0 / 1
testnet	first	change	first	m / 87′ / 1′ / 0′ / 1 / 0
testnet	first	change	second	m / 87′ / 1′ / 0′ / 1 / 1
testnet	second	external	first	m / 87′ / 1′ / 1′ / 0 / 0
testnet	second	external	second	m / 87′ / 1′ / 1′ / 0 / 1
testnet	second	change	first	m / 87′ / 1′ / 1′ / 1 / 0
testnet	second	change	second	m / 87′ / 1′ / 1′ / 1 / 1

Reference Implementation

None at the moment.

Acknowledgement

Special thanks to SomberNight, Craig Raw, David Harding, Jochen Hoenicke, Sjors Provoost, and others for their feedback on the specification.

References

Original mailing list thread: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-March/018630.html

- BIP-0032 (Hierarchical Deterministic Wallets)
- BIP-0043 (Purpose Field for Deterministic Wallets)
- BIP-0044 (Multi-Account Hierarchy for Deterministic Wallets)
- Output Descriptors
- BIP-0174 (Partially Signed Bitcoin Transaction Format)
- BIP-0067 (Deterministic Pay-to-script-hash multi-signature addresses through public key sorting)
- BIP-0129 (Bitcoin Secure Multisig Setup)


```
BIP: 48
Layer: Applications
Title: Multi-Script Hierarchy for Multi-Sig Wallets
Authors: Fontaine <dentondevelopment@protonmail.com>
Status: Deployed
Type: Specification
Assigned: 2020-12-16
License: MIT
```

BIP 48

Abstract

This BIP defines a logical hierarchy for deterministic multi-sig wallets based on an algorithm described in BIP-0067 (BIP67 from now on), BIP-0032 (BIP32 from now on), purpose scheme described in BIP-0043 (BIP43 from now on), and multi-account hierarchy described in BIP-0044 (BIP44 from now on).

This BIP is a particular application of BIP43.

Copyright

This BIP falls under the MIT License.

Motivation

The motivation of this BIP is to define the existing industry wide practice of utilizing m/48′ derivation paths in hierarchical deterministic multi-sig wallets so that other developers may benefit from a standard. This BIP allows for future script types to easily be appended to the specification so that a new BIP is not required for every future script type.

The hierarchy proposed in this paper is quite comprehensive. It allows the handling of multiple accounts, external and internal chains per account, multiple script types and millions of addresses per chain.

This paper was inspired from BIP44.

Backwards compatibility

Currently a number of wallets utilize the `m/48'` derivation scheme for HD multi-sig accounts. This BIP is intended to maintain the *existing* real world use of the `m/48'` derivation. No breaking changes are made so as to avoid “loss of funds” to existing users. Wallets which currently support the `m/48'` derivation will not need to make any changes to comply with this BIP.

Specification

Key sorting

Any wallet that supports BIP48 inherently supports deterministic key sorting as per BIP67 so that all possible multi-signature addresses/scripts are derived from deterministically sorted public keys.

Path levels

We define the following 6 levels in BIP32 path:

```
m / purpose' / coin_type' / account' / script_type' / change / address_index
```

`h` or `'` in the path indicates that BIP32 hardened derivation is used.

Each level has a special meaning, described in the chapters below.

Purpose

Purpose is a constant set to 48' following the BIP43 recommendation. It indicates that the subtree of this node is used according to this specification.

Hardened derivation is used at this level.

Coin type

One master node (seed) can be used for multiple Bitcoin networks. Sharing the same space for various networks has some disadvantages.

Avoiding reusing addresses across networks and improving privacy issues.

Coin type `0` for mainnet and `1` for testnet.

Hardened derivation is used at this level.

Account

This level splits the key space into independent user identities, following the BIP44 pattern, so the wallet never mixes the coins across different accounts.

Users can use these accounts to organize the funds in the same fashion as bank accounts; for donation purposes (where all addresses are considered public), for saving purposes, for common expenses etc.

Accounts are numbered from index 0 in sequentially increasing manner. This number is used as child index in BIP32 derivation.

Hardened derivation is used at this level.

Script

This level splits the key space into two separate `script_type`(s). To provide forward compatibility for future script types this specification can be easily extended.

Currently the only script types covered by this BIP are Native Segwit (p2wsh) and Nested Segwit (p2sh-p2wsh).

The following path represents Nested Segwit (p2sh-p2wsh) mainnet, account 0: `1'`: Nested Segwit (p2sh-p2wsh) `m/48'/0'/0'/1'`

The following path represents Native Segwit (p2wsh) mainnet, account 0: `2'`: Native Segwit (p2wsh) `m/48'/0'/0'/2'`

The recommended default for wallets is pay to witness script hash `m/48'/0'/0'/2'`.

Change

Constant 0 is used for external chain and constant 1 for internal chain (also known as change addresses). External chain is used for addresses that are meant to be visible outside of the wallet (e.g. for receiving

payments). Internal chain is used for addresses which are not meant to be visible outside of the wallet and is used for return transaction change.

Public derivation is used at this level.

Index

Addresses are numbered from index 0 in sequentially increasing manner. This number is used as child index in BIP32 derivation.

Public derivation is used at this level.

Examples

network	account	script	chain	address	path
mainnet	first	p2sh-p2wsh	external	first	m / 48′ / 0′ / 0′ / 1′ / 0 / 0
mainnet	first	p2wsh	external	first	m / 48′ / 0′ / 0′ / 2′ / 0 / 0
mainnet	first	p2wsh	external	second	m / 48′ / 0′ / 0′ / 2′ / 0 / 1
mainnet	first	p2wsh	change	first	m / 48′ / 0′ / 0′ / 2′ / 1 / 0
mainnet	first	p2wsh	change	second	m / 48′ / 0′ / 0′ / 2′ / 1 / 1
mainnet	second	p2wsh	external	first	m / 48′ / 0′ / 1′ / 2′ / 0 / 0
mainnet	second	p2wsh	external	second	m / 48′ / 0′ / 1′ / 2′ / 0 / 1
testnet	first	p2sh-p2wsh	external	first	m / 48′ / 1′ / 0′ / 1′ / 0 / 0
testnet	first	p2wsh	external	second	m / 48′ / 1′ / 0′ / 2′ / 0 / 1
testnet	first	p2wsh	change	first	m / 48′ / 1′ / 0′ / 2′ / 1 / 0
testnet	first	p2wsh	change	second	m / 48′ / 1′ / 0′ / 2′ / 1 / 1
testnet	second	p2wsh	external	first	m / 48′ / 1′ / 1′ / 2′ / 0 / 0
testnet	second	p2wsh	external	second	m / 48′ / 1′ / 1′ / 2′ / 0 / 1
testnet	second	p2wsh	change	first	m / 48′ / 1′ / 1′ / 2′ / 1 / 0
testnet	second	p2wsh	change	second	m / 48′ / 1′ / 1′ / 2′ / 1 / 1

Reference

- BIP67 - Deterministic Pay-to-script-hash multi-signature addresses through public key sorting
- BIP32 - Hierarchical Deterministic Wallets
- BIP43 - Purpose Field for Deterministic Wallets
- BIP44 - Multi-Account Hierarchy for Deterministic Wallets

```
BIP: 129
Layer: Applications
Title: Bitcoin Secure Multisig Setup (BSMS)
Authors: Hugo Nguyen <hugo@nunchuk.io>
         Peter Gray <peter@coinkite.com>
         Marko Bencun <marko@shiftcrypto.ch>
```

Aaron Chen <aarondongchen@gmail.com>
Rodolfo Novak <rodolfo@coinkite.com>
Status: Complete
Type: Specification
Assigned: 2020-11-10
License: BSD-2-Clause
Requires: 32, 174, 322, 380, 381, 382, 383

BIP 129

Introduction

Abstract

This document proposes a mechanism to set up multisig wallets securely.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

The Bitcoin multisig experience has been greatly streamlined under BIP-0174 (Partially Signed Bitcoin Transaction). However, what is still missing is a standardized process for setting up multisig wallets securely across different vendors.

There are a number of concerns when it comes to setting up a multisig wallet:

1. Whether the multisig configuration, such as Signer membership, script type, derivation paths and number of signatures required, is correct and not tampered with.
2. Whether the keys or the multisig configuration are leaked during the setup.
3. Whether the Signer persists the multisig configuration in their respective storage, and under what format.
4. Whether the Signer's storage is tamper-proof.
5. Whether the Signer subsequently uses the multisig configuration to generate and verify receive and change addresses.

An attacker who can modify the multisig configuration can steal or hold funds for ransom by duping the user into sending funds to the wrong address. An attacker who cannot modify the configuration but can learn about the keys and/or the configuration can monitor transactions in the wallet, resulting in loss of privacy.

This proposal seeks to address concerns #1, #2 and #3: to mitigate the risk of tampering during the initial setup phase, and to define an interoperable multisig configuration format.

Concerns #4 and #5 should be handled by Signers and are out of scope of this proposal.

Specification

Prerequisites

This proposal assumes the parties in the multisig support BIP-0032, BIP-0322, BIP-0380 Output Script Descriptors (BIP-0381,BIP-0382,BIP-0383) and AES encryption.

File Extensions

All descriptor and key records should have a `.bsms` file extension. Encrypted data should have a `.dat` extension.

Newline

This specification uses line feed (LF) control character `\n`.

Roles

Coordinator

The Coordinator initiates the multisig setup. The Coordinator determines what type of multisig is used and the exact policy script. If encryption is enabled, the Coordinator also distributes a shared secret or shared secrets to the parties involved for secure communication. The Coordinator gathers information from the Signers to generate a descriptor record. The Coordinator distributes the descriptor record back to the Signers.

Signer

The Signer is any software or hardware that controls the private keys and can sign using those keys. The Signer is a participating member in the multisig. Its responsibilities include providing its key record – which contains a public key or an Extended Public Key (XPUB) – to the Coordinator, verifying that its `KEY` is included in the descriptor record and persisting the descriptor record in its storage.

Setup Process

Round 1

Coordinator

- The Coordinator creates a new multisig wallet creation session. The Coordinator constructs the multisig script and its policy parameters, such as the required number of signatures and the total number of Signers (`M` and `N`).
- The session should expire after some time period determined by the Coordinator, e.g., 24 hours. The timeout allows the encryption key to have lower entropy.
- If encryption is enabled, the Coordinator distributes a secret `TOKEN` to each Signer over a secure channel. The Signer can use the `TOKEN` to derive an `ENCRYPTION_KEY`. Refer to the #encryption section below for details on the `TOKEN`, the key derivation function and the encryption scheme. Depending on the use case, the Coordinator can decide whether to share one common `TOKEN` for all Signers, or to have one per Signer.

- If encryption is disabled, the `TOKEN` is set to `0x00`, and all the encryption/decryption steps below can be skipped.

Signer

- The Signer initiates the multisig wallet creation session by setting the `TOKEN`. The Signer derives an `ENCRYPTION_KEY` from the `TOKEN`. The Signer can keep the session open until a different value for the `TOKEN` is set.
- The Signer generates a key record by prompting the user for a multisig derivation path and retrieves the `KEY` at that derivation path. Alternatively, the Signer can choose a path on behalf of the user. If the Signer chooses the path, it should try to avoid reusing `KEY`s for different wallets.
- The first line in the record must be the specification version (`BSMS 1.0` as of this writing). The second line must be the hex-encoded `TOKEN`. The third line must be the `KEY`. The `KEY` is a public key or an XPUB plus the key origin information, written in the descriptor-defined format, i.e.: `[{master key fingerprint}/{derivation path}]{KEY}`. The fourth line is a text description of the key, 80 characters maximum. The fifth line must be a `SIG`, whereas `SIG` is the signature generated by using the private key associated with the public key or XPUB to sign the first four lines. The signature should follow BIP-0322, legacy format accepted.
- The Signer calculates the Message Authentication Code (`MAC`) for the record. The first 16 bytes of the `MAC` serves as the Initialization Vector (`IV`) for the encryption.
- The Signer encrypts the key record with the `ENCRYPTION_KEY` and `IV`.
- The Signer encodes the `MAC` and the ciphertext into hexadecimal format, then concatenates the results: `(MAC || ciphertext)`.

Round 2

Coordinator

- The Coordinator gathers key records from all participating Signers. The Coordinator verifies that there are exactly `N` unique key records before the wallet setup session expires.
- For each key record, the Coordinator extracts the `MAC` from the data, sets `IV` to the first 16 bytes of the `MAC`, then decrypts the ciphertext using the `ENCRYPTION_KEY` and `IV`.
- The Coordinator verifies that the included `MAC` is valid given the plaintext.
- The Coordinator verifies that the key records have compatible specification versions.
- The Coordinator verifies that the included `SIG` is valid given the `KEY`.
- If all key records look good, the Coordinator fills in all necessary information to generate a descriptor record.
- The first line in the descriptor record must be the specification version (`BSMS 1.0` as of this writing). The second line must be a descriptor or a descriptor template. The third line must be a comma-separated list of derivation path restrictions. The paths must start with `/` and use non-hardened derivation. If there are no template or restrictions, it must say `No path restrictions`. The fourth line must be the wallet's first address. If there are path restrictions, use the first address from the first path restriction.
- The Coordinator calculates the `MAC` for the record. The first 16 bytes of the `MAC` serves as the `IV` for the encryption..
- The Coordinator encrypts the descriptor record with the `ENCRYPTION_KEY` and `IV`.
- The Coordinator encodes the `MAC` and the ciphertext into hexadecimal format, then concatenates the results: `(MAC || ciphertext)`.

- The Coordinator sends the encrypted descriptor record to all participating Signers.

Signer

- The Signer imports the descriptor record.
- The Signer extracts the `MAC` from the data, sets `IV` to the first 16 bytes of the `MAC`, then decrypts the ciphertext using the `ENCRYPTION_KEY` (derived from the open session) and `IV`.
- The Signer verifies that the included `MAC` is valid given the plaintext.
- The Signer verifies that it can support the included specification version.
- The Signer verifies that it can support the descriptor or descriptor template.
- The Signer checks that its `KEY` is included in the descriptor or descriptor template, using path and fingerprint information provided. The check must perform an exact match on the `KEY`s and not using shortcuts such as matching fingerprints, which is trivial to spoof.
- The Signer verifies that it is compatible with the derivation path restrictions.
- The Signer verifies that the wallet's first address is valid.
- For confirmation, the Signer must display to the user the wallet's first address and policy parameters, including, but not limited to: the derivation path restrictions, `M`, `N`, and the position(s) of the Signer's own `KEY` in the policy script. The total number of Signers, `N`, is important to prevent a `KEY` insertion attack. The position is important for scripts where `KEY` order matters. When applicable, all positions of the `KEY` must be displayed. The full descriptor or descriptor template must also be available for review upon user request.
- Parties must check with each other that all Signers have the same confirmation (except for the `KEY` positions).
- If all checks pass, the Signer must persist the descriptor record in its storage.

This completes the setup.

Encryption

The Token

We define three modes of encryption.

1. `NO_ENCRYPTION` : the `TOKEN` is set to `0x00`. Encryption is disabled.
2. `STANDARD` : the `TOKEN` is a 64-bit nonce.
3. `EXTENDED` : the `TOKEN` is a 128-bit nonce.

The `TOKEN` can be converted to one of these formats:

- A decimal number (recommended). The number must not exceed the maximum value of the nonce.
- A mnemonic phrase using BIP-0039 word list. This would be 6 words in `STANDARD` mode. This encoding is not recommended in `EXTENDED` mode as it can result in potential confusion between seed mnemonics and `TOKEN` mnemonics.
- A QR code.
- Other formats.

The flexibility in the data format allows each Signer to customize the User Experience based on its respective capabilities.

Key Derivation

The key derivation function is PBKDF2, with PRF = SHA512. Specifically:

```
DKey = PBKDF2(PRF, Password, Salt, c, dkLen)
```

Whereas:

- PRF = SHA512
- Password = "No SPOF"
- Salt = `TOKEN`
- c = 2048
- dkLen = 256 bits (32 bytes)
- DKey = Derived `ENCRYPTION_KEY`

Encryption Scheme

The encryption scheme is AES-256-CTR.

```
MAC = HMAC-SHA256(HMAC_Key, hex-encoded TOKEN || Data)

IV = First 16 bytes of MAC

Ciphertext = AES-256-CTR-Encrypt(Plaintext, DKey, IV)

Plaintext = AES-256-CTR-Decrypt(Ciphertext, DKey, IV)
```

Whereas:

- DKey = `ENCRYPTION_KEY`
- HMAC_Key = SHA256(`ENCRYPTION_KEY`)
- Data = the plaintext, e.g. the entire key record in round 1 and the entire descriptor record in round 2

The `MAC` is to be sent along with the key and descriptor record, as specified above. Because it is a `MAC` over the entire plaintext, this is essentially an Encrypt-and-MAC form of authenticated encryption.

Descriptor Template

The output descriptor language only supports one-dimensional lists. This proposal introduces a descriptor template to represent multi-dimensional lists:

```
XPUB/**
```

Whereas `/**` can be replaced by any number of derivation path restrictions.

A descriptor template must be accompanied by derivation path restrictions. Signers should expand the template into concrete descriptors by replacing `/**` with the restrictions.

For example, the following template and derivation path restrictions:

- `wsh(sortedmulti(2,XPUB1/**,XPUB2/**))`
- `/0/*,/1/*`

Should translate to two concrete descriptors:

- `wsh(sortedmulti(2,XPUB1/0/*,XPUB2/0/*))`
- `wsh(sortedmulti(2,XPUB1/1/*,XPUB2/1/*))`

QR Codes

For signers that use QR codes to transmit data, key and descriptor records can be converted to QR codes, following the BCR standard.

Also refer to UR Type Definition for BIP44 Accounts and UR Type Definition for Bitcoin Output Descriptors for more details.

Compatibility

This specification is not backwards compatible with existing multisig implementations.

BSMS is opt-in, meaning existing multisig implementations can continue working as-is, with the caveat that they are likely to have various pitfalls. Some of the problems with existing solutions have been described in the #motivation section.

To comply with this standard, a Signer must be able to persist the descriptor record in its storage.

To use BSMS for a multisig wallet, the user should wait until all participating Signers in the multisig have implemented BSMS.

Security

This proposal introduces two layers of protection. The first one is a temporary, secret `TOKEN`. The second one is the confirmation of the wallet's first address.

The `TOKEN` is used to encrypt the two rounds of communication between the Signer and the Coordinator. A `MAC` is also generated from the `TOKEN` and plaintext to authenticate the data being exchanged. The `TOKEN` is only needed during the setup phase, and can be safely discarded afterwards. It is not recommended to use the same `TOKEN` for multiple wallet creation sessions.

The wallet's first address, on the other hand, can be used to verify the integrity of the multisig configuration. An attacker who tampers with the multisig configuration must also change the wallet's first address. Parties must check with each other that all Signers confirm to the same address and policy parameters to reduce the chance of tampering.

Privacy

Encryption helps improve the privacy of the wallet by avoiding sharing keys and descriptors in plaintext.

If the parties wish to have stronger privacy, it is recommended to use a higher number of bits for the `TOKEN`, and to completely erase knowledge of the `TOKEN` after the multisig wallet has been set up.

Test Vectors

Mode: `NO_ENCRYPTION` with Public Keys

ROUND 1

- Coordinator
  - M-of-N: 1/2
  - ADDRESS_TYPE: NATIVE_SEGWIT
  - TOKEN: 0x00
- Signer 1
  - MASTER_KEY_FINGERPRINT: 59865f44
  - PRIVATE_KEY (m/48′/0′/0′/2′): L5TXU4SdD9e6QGgBjxeegJKxt4FgATLG1TCnFM8JLyEkFuyHEqNM
  - Public Key (m/48′/0′/0′/2′): 026d15412460ba0d881c21837bb999233896085a9ed4e5445bd637c10e579768ba
  - Legacy signature
  - `signer_1_key.bsms`:

```
BSMS 1.0
00
[59865f44/48'/0'/0'/2']026d15412460ba0d881c21837bb999233896085a9ed4e5445bd637c10e579768ba
Signer 1 key
H6DXgqkCb353BDPkzppMFpOcdJZlpur0WRetQhIBqSn6DFzoQWBtm+ibP5wERDRNi0bxxev9B+FIvyQWq0s6im4=
```

- Signer 2
  - MASTER_KEY_FINGERPRINT: b7044ca6
  - PRIVATE_KEY (m/48′/0′/0′/2′): KwT7BZDWjos4JAdfKi8NqF46Kj3rppTwN8KGhPbzmmugiZioFW3r
  - Public Key (m/48′/0′/0′/2′): 030baf0497ab406ff50cb48b4013abac8a0338758d2fd54cd934927afa57cc2062
  - Legacy signature
  - `signer_2_key.bsms`:

```
BSMS 1.0
00
[b7044ca6/48'/0'/0'/2']030baf0497ab406ff50cb48b4013abac8a0338758d2fd54cd934927afa57cc2062
Signer 2 key
H08mGNGN+NxX/snt+6eX2Q1HjjfDkOtotglshHi7xdsBdIrTVMCQbgQ5SdACNZ0B2AJcifK11nJj43SvaitSemI=
```

ROUND 2

- Coordinator
  - `my_multisig_wallet.bsms`:

```
BSMS 1.0
wsh(sortedmulti(1,[59865f44/48'/0'/0'/
2']026d15412460ba0d881c21837bb999233896085a9ed4e5445bd637c10e579768ba,[b7044ca6/48'/0'/0'/
2']030baf0497ab406ff50cb48b4013abac8a0338758d2fd54cd934927afa57cc2062))#rzx9dffd
```

```
No path restrictions
bc1quqy523xu3l8che3s8vja8n33qtg0uyugr9l5z092s3wa50p8t7rqy6zumf
```

Mode: `NO_ENCRYPTION`

ROUND 1

- Coordinator
  - M-of-N: 2/2
  - ADDRESS_TYPE: NATIVE_SEGWIT
  - TOKEN: 0x00

- Signer 1
  - MASTER_KEY_FINGERPRINT: 1cf0bf7e
  - PRIVATE_KEY (m/48′/0′/0′/2′): L3q1sg7iso1L3QfzB1riC9bQpqMynWyBeuLLSKwCDGkHkahB7MgU
  - XPUB (m/48′/0′/0′/2′): xpub6FL8FhxNNUVnG64YurPd16AfGyvFLhh7S2uSsDqR3Qfcm6o9jtcMYwh6DvmcBF9qozxNQmTCVvWtxLp
  - Legacy signature
  - `signer_1_key.bsms`:

```
BSMS 1.0
00
[1cf0bf7e/48'/0'/0'/
2']xpub6FL8FhxNNUVnG64YurPd16AfGyvFLhh7S2uSsDqR3Qfcm6o9jtcMYwh6DvmcBF9qozxNQmTCVvWtxLpKTnhVLN3Pg
nu2D3pAoXYFgVyd8Yz
Signer 1 key
IB7v+qi1b+Xrwm/3bF+Rjl8QbIJ/FMQ40kUsOOQo1SqUWn5QlFWbBD8BKPRetfo1L1N7DmYjVscZNsmMrqRJGWw=
```

- Signer 2
  - MASTER_KEY_FINGERPRINT: 4fc1dd4a
  - PRIVATE_KEY (m/48′/0′/0′/2′): L4JNkJfLBDyWfTLbKJ1H3w56GUMsvdfjCkzRo5RHXfJ6bdHqm6cN
  - XPUB (m/48′/0′/0′/2′): xpub6EebMbEps7ZcV3FYEnddRsvrFWDrt2tiPmCeM7pPXQEmphvq9ZfJ1LWFUDjf3vxCeBuPrfyGrMazWUsY
  - Legacy signature
  - `signer_2_key.bsms`:

```
BSMS 1.0
00
[4fc1dd4a/48'/0'/0'/
2']xpub6EebMbEps7ZcV3FYEnddRsvrFWDrt2tiPmCeM7pPXQEmphvq9ZfJ1LWFUDjf3vxCeBuPrfyGrMazWUsYsetrnHatQ
ZVLJH7LsgCjtMqdzgj
Signer 2 key
HzUa4Z76PFHMl54flIIF3XKiHZ+KbWjjxCEG5G3ZqZSqTd6OgTiFFLqq9PXJXdfYm6/cnL8IVWQgjFF9DQhIqQs=
```

ROUND 2

- Coordinator
  - `my_multisig_wallet.bsms`:

```
BSMS 1.0
wsh(sortedmulti(2,[1cf0bf7e/48'/0'/0'/
2']xpub6FL8FhxNNUVnG64YurPd16AfGyvFLhh7S2uSsDqR3Qfcm6o9jtcMYwh6DvmcBF9qozxNQmTCVvWtxLpKTnhVLN3Pg
nu2D3pAoXYFgVyd8Yz/**,[4fc1dd4a/48'/0'/0'/
2']xpub6EebMbEps7ZcV3FYEnddRsvrFWDrt2tiPmCeM7pPXQEmphvq9ZfJ1LWFUDjf3vxCeBuPrfyGrMazWUsYsetrnHatQ
ZVLJH7LsgCjtMqdzgj/**))
/0/*,/1/*
bc1qrgc6p3kylfztu06ysl752gwwuekhvtfh9vr7zg43jvu60mutamcsv948ej
```

Mode: `STANDARD` Encryption

ROUND 1

- Coordinator
  - M-of-N: 2/2
  - ADDRESS_TYPE: NATIVE_SEGWIT
  - TOKEN (hex): a54044308ceac9b7
    - TOKEN (decimal): 11907592390080907703
    - TOKEN (mnemonic): pipe acquire around border prosper swift
  - ENCRYPTION_KEY (hex): 7673ffd9efd70336a5442eda0b31457f7b6cdf7b42fe17f274434df55efa9839

- Signer 1
  - MASTER_KEY_FINGERPRINT: b7868815
  - PRIVATE_KEY (m/48′/0′/0′/2′):
    KyKvR9kf8r7ZVtdn3kB9ifipr6UKnTNTpWJkGZbHwARDCz5iZ39E
  - XPUB (m/48′/0′/0′/2′):
    xpub6FA5rfxJc94K1kNtxRby1hoHwi7YDyTWwx1KUR3FwskaF6HzCbZMz3zQwGnCqdiFeMTPV3YneTGS2Y
  - Legacy signature
  - `signer_1_key.bsms`:

```
BSMS 1.0
a54044308ceac9b7
[b7868815/48'/0'/0'/
2']xpub6FA5rfxJc94K1kNtxRby1hoHwi7YDyTWwx1KUR3FwskaF6HzCbZMz3zQwGnCqdiFeMTPV3YneTGS2YQPiuNYsSvtg
gWWMQpEJD4jXU7ZzEh
Signer 1 key
H8DYht5P6ko0bQqDV6MtUxpzBSK+aVHxbvMavA5byvLrOlCEGmO1WFR7k2wu42J6dxXD8vrmDQSnGq5MTMMbZ98=
```

- Signer 1 encryption
  - HMAC_KEY (hex): 3d4c422806ba8964c9ee45070cd675c024d96648a0ddb4001325818c84951de2
  - MAC (hex): fbdbdb64e6a8231c342131d9f13dcd5a954b4c5021658fa5afcb3fc74dc82706
  - IV (hex) : fbdbdb64e6a8231c342131d9f13dcd5a
  - CIPHERTEXT (hex):
    53f491cfd1431c292d922ea5a5dec3eb8ddaa6ed38ae109e7b040f0f23013e89a89b4d27476761a01197a3277850b2bc162
  - `signer_1_key.dat`:
    fbdbdb64e6a8231c342131d9f13dcd5a954b4c5021658fa5afcb3fc74dc8270653f491cfd1431c292d922ea5a5dec3eb8dd

- Signer 2
  - MASTER_KEY_FINGERPRINT: eedff89a
  - PRIVATE_KEY (m/48′/0′/0′/2′):
    Kz1ijnkDXmc65NWTYdg47DDaQgSGJAPfhJG9Unm36oqZPpPXuNR6

    - XPUB (m/48′/0′/0′/2′): xpub6EhJvMneoLWAf8cuyLBLQiKiwh89RAmqXEqYeFuaCEHdHwxSRfzLrUxKXEBap7nZSHAYP7Jfq6gZmuc
    - Legacy signature
    - `signer_2_key.bsms`:

```
BSMS 1.0
a54044308ceac9b7
[eedff89a/48'/0'/0'/
2']xpub6EhJvMneoLWAf8cuyLBLQiKiwh89RAmqXEqYeFuaCEHdHwxSRfzLrUxKXEBap7nZSHAYP7Jfq6gZmucotNzpMQ9Sb
1nTqerqW8hrtmx6Y6o
Signer 2 key
H/IHW5dMGYsrRdYEz3ux+kKnkWBtxHzfYkREpnYbco38VnMvIxCbDuf7iu6960qDhBLR/RLjlb9UPtLmCMbczDE=
```

- Signer 2 encryption
    - HMAC_KEY (hex): 3d4c422806ba8964c9ee45070cd675c024d96648a0ddb4001325818c84951de2
    - MAC (hex): 383d05b7351a2cef7cca2850450f5efbbc4a3f8ea35707dda87a3692f0f2ebae
    - IV (hex) : 383d05b7351a2cef7cca2850450f5efb
    - CIPHERTEXT (hex): 71860b7c69f3a7665c3c3e85c45735bff78535a37ec6610b724627c73696820d519a9251703b17626b63898580233bebbb3
    - `signer_2_key.dat`: 383d05b7351a2cef7cca2850450f5efbbc4a3f8ea35707dda87a3692f0f2ebae71860b7c69f3a7665c3c3e85c45735bff7853

ROUND 2

- Coordinator
    - `my_multisig_wallet.bsms`:

```
BSMS 1.0
wsh(sortedmulti(2,[b7868815/48'/0'/0'/
2']xpub6FA5rfxJc94K1kNtxRby1hoHwi7YDyTWwx1KUR3FwskaF6HzCbZMz3zQwGnCqdiFeMTPV3YneTGS2YQPiuNYsSvtg
gWWMQpEJD4jXU7ZzEh/**,[eedff89a/48'/0'/0'/
2']xpub6EhJvMneoLWAf8cuyLBLQiKiwh89RAmqXEqYeFuaCEHdHwxSRfzLrUxKXEBap7nZSHAYP7Jfq6gZmucotNzpMQ9Sb
1nTqerqW8hrtmx6Y6o/**))
/0/*,/1/*
bc1qhs4u273g4azq7kqqpe6vh5wfhasfmrq7nheyzsnq77humd7rwtkqagvakf
```

- Coordinator encryption
    - HMAC_KEY (hex): 3d4c422806ba8964c9ee45070cd675c024d96648a0ddb4001325818c84951de2
    - MAC (hex): 734ce791b466861945e1ef6f74c63faec590793de54831f0036b28d08714b71a
    - IV (hex) : 734ce791b466861945e1ef6f74c63fae
    - CIPHERTEXT (hex): 273cad18a5e1eff37dba6d850749594c9a3fd32b2069e8c69983ea269c5044b6bcaea26d9dbc8ad5d28bb8abfa02e3bfc76
    - `my_multisig_wallet.dat`: 734ce791b466861945e1ef6f74c63faec590793de54831f0036b28d08714b71a273cad18a5e1eff37dba6d850749594c9a3f

Mode: `EXTENDED` Encryption

ROUND 1

- Coordinator
    - M-of-N: 2/3

    - ADDRESS_TYPE: NESTED_SEGWIT
    - TOKEN for Signer 1 (hex): 108a2360adb302774eb521daebbeda5e
        - TOKEN (decimal): 21984902443033505423410071144203475550
        - ENCRYPTION_KEY (hex): 63dc1e57dfdc21fa11109d5088be01fb8078a383d2296925ad2b7612b7179777
    - TOKEN for Signer 2 (hex): d3fabc873b98165254fe18a71b5335b0
        - TOKEN (decimal): 28176900513250185974442197052809564715 2
        - ENCRYPTION_KEY (hex): 3dc860a53471ec03af14617fef60921cf215b45a9d684462fa65b9d804ad3ee7
    - TOKEN for Signer 3 (hex): 78a7d5e7549453d719150de5459c9ce5
        - TOKEN (decimal): 160378811550692397333855096016467696869
        - ENCRYPTION_KEY (hex): 62b90b4c08c03a0ee872e57aae73f9acfafb6cc09d20b5c9bc0bafaef33619db

- Signer 1
    - MASTER_KEY_FINGERPRINT: 793cc70b
    - PRIVATE_KEY (m/48′/0′/0′/1′): L1ZEgZ4zNYxyNc8UyeqwyKW1UHVMp9sxwPgSi3s9SW8mc7KsiSwJ
    - XPUB (m/48′/0′/0′/1′): xpub6ErVmcYYHmavsMgxEcTZyzN5sqth1ZyRpFNJC26ij1wYGC2SBKYrgt9yariSbn7HLRoZUvhUhmPfsRTPr
    - Legacy signature
    - `signer_1_key.bsms`:

```
BSMS 1.0
108a2360adb302774eb521daebbeda5e
[793cc70b/48'/0'/0'/
1']xpub6ErVmcYYHmavsMgxEcTZyzN5sqth1ZyRpFNJC26ij1wYGC2SBKYrgt9yariSbn7HLRoZUvhUhmPfsRTPrdhhGFscp
PZzmch6UTdmRP1aZUj
Signer 1 key
ILG47LpCtjoD9UxL87jo5QFqA90t8g9fDQp/KBojdKgPPGB1pMx2bf9hPdORNZIOdCc/2+Gs6AOs3BEK9ubIuBw=
```

- Signer 1 encryption
    - HMAC_KEY (hex): 1162cdace4ac9fcde1f96924b93714143d057a701de83ebaed248d1c9154f9fd
    - MAC (hex): ea12776c73de4bd5ea57c2d19eb8e0be856ac0d7f5651f7b74be4563d61ba5b1
    - IV (hex) : ea12776c73de4bd5ea57c2d19eb8e0be
    - CIPHERTEXT (hex): a36f34232bff47a853092654a718fea4f5f57d6a1f3d38fede04e2414da12c90cefc24ef662f736886d9a7fd6e7db636ca472
    - `signer_1_key.dat`: ea12776c73de4bd5ea57c2d19eb8e0be856ac0d7f5651f7b74be4563d61ba5b1a36f34232bff47a853092654a718fea4f5f5

- Signer 2
    - MASTER_KEY_FINGERPRINT: b3118e52
    - PRIVATE_KEY (m/48′/0′/0′/1′): L4SnPjcHszMg3Wi2YYxEYnzM2zFeFkFr5NcLZ18YQeyJwaSFbTud
    - XPUB (m/48′/0′/0′/1′): xpub6Du5Jn6eYZE96ccmAc1ZTFPzdnzrvqfG4mpamDun2qZYKywoiQJMCbS3kWWMr6U3XW6s125RLsaPAB
    - Legacy signature
    - `signer_2_key.bsms`:

```
BSMS 1.0
d3fabc873b98165254fe18a71b5335b0
[b3118e52/48'/0'/0'/
1']xpub6Du5Jn6eYZE96ccmAc1ZTFPzdnzrvqfG4mpamDun2qZYKywoiQJMCbS3kWWMr6U3XW6s125RLsaPABWgv2yA749ie
aMe67FxkTjMsbcxCch
Signer 2 key
IDK4d/oO0pgfrwRu4Zb8vqlPEmJb9aKT1K2CCnI3RKepVAKs3fZsBrypcCdQfUy1TG/3O5vAR3gjldxcCA1Wzg8=
```

- Signer 2 encryption
    - HMAC_KEY (hex): 43a4e704bd1bade703023004b00290f1a7b005474a581d869a217068eedf3f57
    - MAC (hex): 4a3ff970d027010e83b4fbf2845a23907a301b3df692a9265e2ca679697ac718
    - IV (hex) : 4a3ff970d027010e83b4fbf2845a2390
    - CIPHERTEXT (hex): c8f4a6a6714eff90aa48cbefe6750c2ee3cc72182eb455e964f0ba59ada3ecd758c29f0fab7e33aaa82a340a18d9c793ddab0
    - `signer_2_key.dat`: 4a3ff970d027010e83b4fbf2845a23907a301b3df692a9265e2ca679697ac718c8f4a6a6714eff90aa48cbefe6750c2ee3cc72

- Signer 3
    - MASTER_KEY_FINGERPRINT: 842bd2ed
    - PRIVATE_KEY (m/48′/0′/0′/1′): L1ehZHpo2UFHc1yaBWDU4bKVycUwcU2TESm92wbfq6xK6qpZZJP6
    - XPUB (m/48′/0′/0′/1′): xpub6Ex81KopPkEt9hJiWHabYy8LNsSR4A7sUQoFBk9dR8XxHrr4p9HrYWN3NCf5uwfopHnQkCG7FYnZMz
    - Legacy signature
    - `signer_3_key.bsms`:

```
BSMS 1.0
78a7d5e7549453d719150de5459c9ce5
[842bd2ed/48'/0'/0'/
1']xpub6Ex81KopPkEt9hJiWHabYy8LNsSR4A7sUQoFBk9dR8XxHrr4p9HrYWN3NCf5uwfopHnQkCG7FYnZMztKbtRtbh6tz
ZC4xtHPbmVVxRSN7ic
Signer 3 key
IL77mML0xo/O9dJn0T5EpQLuyRPPrdpgVJbtsdAugW5iX0MQ3Ci0f8jVnXu68Xm07CYjYGKX8af72jmkQKhNud0=
```

- Signer 3 encryption
    - HMAC_KEY (hex): ab93ce7bf0f91c62a66d00ea9bf5e5c00b854ee2cfc2fb06f6eeff738abcdc26
    - MAC (hex): e82cfcccbd4bd4d3b76e28133eecd13f7362f4a8b4c4baa3e5f6ba2dfb4d69b8
    - IV (hex) : e82cfcccbd4bd4d3b76e28133eecd13f
    - CIPHERTEXT (hex): b44433f0b564ec35a1e71371f25844088084b47402c90d52fee7237167b58a60a28c234af9123e104773136e8446d799541c
    - `signer_3_key.dat`: e82cfcccbd4bd4d3b76e28133eecd13f7362f4a8b4c4baa3e5f6ba2dfb4d69b8b44433f0b564ec35a1e71371f2584408808

ROUND 2

- Coordinator
    - `my_multisig_wallet.bsms`:

```
BSMS 1.0
sh(wsh(multi(2,[793cc70b/48'/0'/0'/
1']xpub6ErVmcYYHmavsMgxEcTZyzN5sqth1ZyRpFNJC26ij1wYGC2SBKYrgt9yariSbn7HLRoZUvhUhmPfsRTPrdhhGFscp
```

```
PZzmch6UTdmRP1aZUj/**,[b3118e52/48'/0'/0'/
1']xpub6Du5Jn6eYZE96ccmAc1ZTFPzdnzrvqfG4mpamDun2qZYKywoiQJMCbS3kWWMr6U3XW6s125RLsaPABWgv2yA749ie
aMe67FxkTjMsbcxCch/**,[842bd2ed/48'/0'/0'/
1']xpub6Ex81KopPkEt9hJiWHabYy8LNsSR4A7sUQoFBk9dR8XxHrr4p9HrYWN3NCf5uwfopHnQkCG7FYnZMztKbtRtbh6tz
ZC4xtHPbmVVxRSN7ic/**)))
/0/*,/1/*
3GzMtFXahiu4TpGNGFc4bHMvAcvz5vVQrT
```

- Send to Signer 1:
  - HMAC_KEY (hex): 1162cdace4ac9fcde1f96924b93714143d057a701de83ebaed248d1c9154f9fd
  - MAC (hex): 01bf557b6d44b3fbf07f8ec155cbdec42d85d856e174342563dd83b40ad7c025
  - IV (hex) : 01bf557b6d44b3fbf07f8ec155cbdec4
  - CIPHERTEXT (hex):
    617ed25b4b8fd88b806cbebcc1731b071465514a805f7ba2de60e291bc9493f31aa0f9b0665ba822cf9a2e21c02649b5c3f7
  - `my_multisig_wallet_for_signer_1.dat`:
    01bf557b6d44b3fbf07f8ec155cbdec42d85d856e174342563dd83b40ad7c025617ed25b4b8fd88b806cbebcc1731b0714
- Send to Signer 2:
  - HMAC_KEY (hex): 43a4e704bd1bade703023004b00290f1a7b005474a581d869a217068eedf3f57
  - MAC (hex): 974ba77900c43c463dadaa6eaf24aaeb1b25b443cf155229b719bcbf8b343092
  - IV (hex) : 974ba77900c43c463dadaa6eaf24aaeb
  - CIPHERTEXT (hex):
    86288c97a6341974a35015f97fbbc8db7655639c839fc438706f82fce36a82dd17e2d4d4a674516c4fc5c3a33d6097dd8fc5
  - `my_multisig_wallet_for_signer_2.dat`:
    974ba77900c43c463dadaa6eaf24aaeb1b25b443cf155229b719bcbf8b34309286288c97a6341974a35015f97fbbc8db765
- Send to Signer 3:
  - HMAC_KEY (hex): ab93ce7bf0f91c62a66d00ea9bf5e5c00b854ee2cfc2fb06f6eeff738abcdc26
  - MAC (hex): bb3c93b67d758f244de7ee73e5e61261cea6dff5b3852df8faf265cdf1c73dae
  - IV (hex) : bb3c93b67d758f244de7ee73e5e61261
  - CIPHERTEXT (hex):
    7ac33bd9719a3cef6c68e09b3c9677565418933f188bbe50dc70f46329706732fe28ab230468e2a8798d3fbf641867d5b332
  - `my_multisig_wallet_for_signer_3.dat`:
    bb3c93b67d758f244de7ee73e5e61261cea6dff5b3852df8faf265cdf1c73dae7ac33bd9719a3cef6c68e09b3c96775654189

Acknowledgement

Special thanks to Pavol Rusnak, Dmitry Petukhov, Christopher Allen, Craig Raw, Robert Spigler, Gregory Sanders, Ta Tat Tai, Michael Flaxman, Pieter Wuille, Salvatore Ingala, Ava Chow and others for their feedback on the specification.

References

Related mailing list threads:

- https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-February/018385.html
- https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-April/018732.html

```
BIP: 328
Layer: Applications
```

```
Title: Derivation Scheme for MuSig2 Aggregate Keys
Authors: Ava Chow <me@achow101.com>
Status: Complete
Type: Informational
Assigned: 2024-06-04
License: CC0-1.0
```

BIP 328

Abstract

This document specifies how BIP 32 extended public keys can be constructed from a BIP 327 MuSig2 aggregate public key and how such keys should be used for key derivation.

Copyright

This BIP is licensed under the Creative Commons CC0 1.0 Universal license.

Motivation

Multiple signers can create a single aggregate public key with MuSig2 that is indistinguishable from a random public key. The cosigners need a method for generating additional aggregate pubkeys to follow the best practice of using a new address for every payment.

The obvious method is for the cosigners to generate multiple public keys and produce a new aggregate pubkey every time one is needed. This is similar to how multisig using Bitcoin script works where all of the cosigners share their extended public keys and do derivation to produce the multisig script. The same could be done with MuSig2 and instead of producing a multisig script, the result would be a MuSig2 aggregate pubkey.

However, it is much simpler to be able to derive from a single extended public key instead of having to derive from many extended public keys and aggregate them. As MuSig2 produces a normal looking public key, the aggregate public can be used in this way. This reduces the storage and computation requirements for generating new aggregate pubkeys.

Specification

A synthetic xpub can be created from a BIP 327 MuSig2 plain aggregate public key by setting the depth to 0, the child number to 0, and attaching a chaincode with the byte string `868087ca02a6f974c4598924c36b57762d32cb45717167e300622c7167e38965`**Where does this constant chaincode come from?** It is the SHA256 of the text `MuSig2MuSig2MuSig2`. This fixed chaincode should be used by all such synthetic xpubs following this specification. Unhardened child public keys can be derived from the synthetic xpub as with any other xpub. Since the aggregate public key is all that is necessary to produce the synthetic xpub, any aggregate public key that will be used in this way shares the same privacy concerns as typical xpubs.

Furthermore, as there is no aggregate private key, only unhardened derivation from the aggregate public key is possible.

When signing, all signers must compute the tweaks used in the BIP 32 derivation for the child key being signed for. The I_L value computed in *CKDpub* is the tweak used at each derivation step. These are provided in the

session context, each with a tweak mode of plain (*is_xonly_t = false*). When the *Sign* algorithm is used, the tweaks will be applied to the partial signatures.

Test Vectors

- Aggregate pubkey `0354240c76b8f2999143301a99c7f721ee57eee0bce401df3afeaa9ae218c70f23`
  - Synthetic xpub
    `xpub661MyMwAqRbcFt6tk3uaczE1y6EvM1TqXvawXcYmFEWijEM4PDBnuCXwwXEKGEouzXE6QLLRxjatMcLLzJ5LV5Nib1BN`
  - Keys:
    - `03935F972DA013F80AE011890FA89B67A27B7BE6CCB24D3274D18B2D4067F261A9`
    - `02F9308A019258C31049344F85F89D5229B531C845836F99B08601F113BCE036F9`
- Aggregate pubkey `0290539eede565f5d054f32cc0c220126889ed1e5d193baf15aef344fe59d4610c`
  - Synthetic xpub
    `xpub661MyMwAqRbcFt6tk3uaczE1y6EvM1TqXvawXcYmFEWijEM4PDBnuCXwwVk5TFJk8Tw5WAdV3DhrGfbFA216sE9BsQQi`
  - Keys:
    - `02F9308A019258C31049344F85F89D5229B531C845836F99B08601F113BCE036F9`
    - `03DFF1D77F2A671C5F36183726DB2341BE58FEAE1DA2DECED843240F7B502BA659`
    - `023590A94E768F8E1815C2F24B4D80A8E3149316C3518CE7B7AD338368D038CA66`
- Aggregate pubkey `022479f134cdb266141dab1a023cbba30a870f8995b95a91fc8464e56a7d41f8ea`
  - Synthetic xpub
    `xpub661MyMwAqRbcFt6tk3uaczE1y6EvM1TqXvawXcYmFEWijEM4PDBnuCXwwUvaZYpysLX4wN59tjwU5pBuDjNrPEJbfxjL`
  - Keys:
    - `02DFF1D77F2A671C5F36183726DB2341BE58FEAE1DA2DECED843240F7B502BA659`
    - `023590A94E768F8E1815C2F24B4D80A8E3149316C3518CE7B7AD338368D038CA66`
    - `02F9308A019258C31049344F85F89D5229B531C845836F99B08601F113BCE036F9`
    - `03935F972DA013F80AE011890FA89B67A27B7BE6CCB24D3274D18B2D4067F261A9`

Backwards Compatibility

Once a synthetic xpub is created, it is fully backwards compatible with BIP 32 - only unhardened derivation can be done, and the signers will be able to produce a signature for any derived children.

Rationale

Reference Implementation

A Python reference implementation is available in this BIP's Auxiliary Files.

Acknowledgements

Thanks to Pieter Wuille, Andrew Poelstra, Sanket Kanjalkar, Salvatore Ingala, and all others who participated in discussions on this topic.

Output Script Descriptors

Derivation paths tell a wallet which keys to use. They do not tell it which script to wrap those keys in. Descriptors close that gap: a small language that names an output script, the keys inside it, and enough checksum to copy the string without corrupting it.

BIP 380 is the grammar. The rest of the family fills in the functions you actually type: non-SegWit scripts, SegWit scripts, multisig, `combo()`, `raw()` and `addr()`, Taproot `tr()`, and Tapscript multisig. Read 380 first. The others are short on purpose — they are the vocabulary, not a second language.

Wallets that speak descriptors can import a backup, a watch-only account, or a hardware-wallet policy without a side channel that explains "this is BIP 84, native SegWit." That is why this chapter sits between keys and PSBTs. You will need both.

In this chapter:

- BIP 380 — Output Script Descriptors General Operation
- BIP 381 — Non-Segwit Output Script Descriptors
- BIP 382 — Segwit Output Script Descriptors
- BIP 383 — Multisig Output Script Descriptors
- BIP 384 — combo() Output Script Descriptors
- BIP 385 — raw() and addr() Output Script Descriptors
- BIP 386 — tr() Output Script Descriptors
- BIP 387 — Tapscript Multisig Output Script Descriptors

```
BIP: 380
Layer: Applications
Title: Output Script Descriptors General Operation
Authors: Pieter Wuille <pieter@wuille.net>
         Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
Assigned: 2021-06-27
License: BSD-2-Clause
```

BIP 380

Abstract

Output Script Descriptors are a simple language which can be used to describe collections of output scripts. There can be many different descriptor fragments and functions. This document describes the general syntax for descriptors, descriptor checksums, and common expressions.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

Bitcoin wallets traditionally have stored a set of keys which are later serialized and mutated to produce the output scripts that the wallet watches and the addresses it provides to users. Typically backups have consisted of solely the private keys, nowadays primarily in the form of BIP 39 mnemonics. However this backup solution is insufficient, especially since the introduction of Segregated Witness which added new output types. Given just the private keys, it is not possible for restored wallets to know which kinds of output scripts and addresses to produce. This has lead to incompatibilities between wallets when restoring a backup or exporting data for a watch only wallet.

Further complicating matters are BIP 32 derivation paths. Although BIPs 44, 49, and 84 have specified standard BIP 32 derivation paths for different output scripts and addresses, not all wallets support those derivation paths nor use them. The lack of derivation path information in these backups and exports leads to further incompatibilities between wallets.

Current solutions to these issues have not been generic and can be viewed as being layer violations. Solutions such as introducing different version bytes for extended key serialization both are a layer violation (key derivation should be separate from script type meaning) and specific only to a particular derivation path and script type.

Output Script Descriptors introduce a generic solution to these issues. Script types are specified explicitly through the use of Script Expressions. Key derivation paths are specified explicitly in Key Expressions. These allow for creating wallet backups and exports which specify the exact scripts, subscripts (redeemScript, witnessScript, etc.), and keys to produce. With the general structure specified in this BIP, new Script Expressions can be introduced as new script types are added. Lastly, the use of common terminology and existing standards allow for Output Script Descriptors to be engineer readable so that the results can be understood at a glance.

Specification

Descriptors consist of several types of expressions. The top level expression is a `SCRIPT`. This expression may be followed by `#CHECKSUM`, where `CHECKSUM` is an 8 character alphanumeric descriptor checksum. Although the checksum is optional for parsing, applications may choose to reject descriptors that do not contain a checksum.

Script Expressions

Script Expressions (denoted `SCRIPT`) are expressions which correspond directly with a Bitcoin script. These expressions are written as functions and take arguments. Such expressions have a script template which is filled with the arguments correspondingly. Expressions are written with a human readable identifier string with the arguments enclosed with parentheses. The identifier string should be alphanumeric and may include underscores.

The arguments to a script expression are defined by that expression itself. They could be a script expression, a key expression, or some other expression entirely.

Key Expressions

A common expression used as an argument to script expressions are key expressions (denoted `KEY`). These represent a public or private key and, optionally, information about the origin of that key. Key expressions can only be used as arguments to script expressions.

Key expressions consist of:

- Optionally, key origin information, consisting of:
  - An open bracket `[`
  - Exactly 8 hex characters for the fingerprint of the key where the derivation starts (see BIP 32 for details)
  - Followed by zero or more `/NUM` or `/NUMh` path elements to indicate the unhardened or hardened derivation steps between the fingerprint and the key that follows.
  - A closing bracket `]`
- Followed by the actual key, which is either:
  - A hex encoded public key, which depending on the script expression, may be either:
    - 66 hex character string beginning with `02` or `03` representing a compressed public key
    - 130 hex character string beginning with `04` representing an uncompressed public key
  - A WIF encoded private key
  - `xpub` encoded extended public key or `xprv` encoded extended private key (as defined in BIP 32)
    - Followed by zero or more `/NUM` or `/NUMh` path elements indicating BIP 32 derivation steps to be taken after the given extended key.
    - Optionally followed by a single `/*` or `/*h` final step to denote all direct unhardened or hardened children.

If the `KEY` is a BIP 32 extended key, before output scripts can be created, child keys must be derived using the derivation information that follows the extended key. When the final step is `/*` or `/*'`, an output script will be produced for every child key index. The derived key must be not be serialized as an uncompressed public key. Script Expressions may have further requirements on how derived public keys are serialized for script creation.

In the above specification, the hardened indicator `h` may be replaced with alternative hardened indicator of `'`.

Normalization of Key Expressions with Hardened Derivation

When a descriptor is exported without private keys, it is necessary to do additional derivation to remove any intermediate hardened derivation steps for the exported descriptor to be useful. The exporter should derive the extended public key at the last hardened derivation step and use that extended public key as the key in the descriptor. The derivation steps that were taken to get to that key must be added to the previous key origin information. If there is no key origin information, then one must be added for the newly derived extended public key. If the final derivation is hardened, then it is not necessary to do additional derivation.

Character Set

The expressions used in descriptors must only contain characters within this character set so that the descriptor checksum will work.

The allowed characters are:

```
0123456789()[],'/*abcdefgh@:$%{}
IJKLMNOPQRSTUVWXYZ&+-.;<=>?!^_|~
ijklmnopqrstuvwxyzABCDEFGH`#"\<space>
```

Note that `<space>` on the last line is a space character.

This character set is written as 3 groups of 32 characters in this specific order so that the checksum below can identify more errors. The first group are the most common "unprotected" characters (i.e. things such as hex and keypaths that do not already have their own checksums). Case errors cause an offset that is a multiple of 32 while as many alphabetic characters are in the same group while following the previous restrictions.

Checksum

Following the top level script expression is a single octothorpe (`#`) followed by the 8 character checksum. The checksum is an error correcting checksum similar to bech32.

The checksum has the following properties:

- Mistakes in a descriptor string are measured in "symbol errors". The higher the number of symbol errors, the harder it is to detect:
    - An error substituting a character from `0123456789()[],'/*abcdefgh@:$%{}` for another in that set always counts as 1 symbol error.
        - Note that hex encoded keys are covered by these characters. Extended keys (`xpub` and `xprv`) use other characters too, but also have their own checksum mechanism.
        - `SCRIPT` expression function names use other characters, but mistakes in these would generally result in an unparsable descriptor.
    - A case error always counts as 1 symbol error.
    - Any other 1 character substitution error counts as 1 or 2 symbol errors.
- Any 1 symbol error is always detected.
- Any 2 or 3 symbol error in a descriptor of up to 49154 characters is always detected.
- Any 4 symbol error in a descriptor of up to 507 characters is always detected.
- Any 5 symbol error in a descriptor of up to 77 characters is always detected.
- Is optimized to minimize the chance of a 5 symbol error in a descriptor up to 387 characters is undetected
- Random errors have a chance of 1 in 2<super>40</super> of being undetected.

The checksum itself uses the same character set as bech32: `qpzry9x8gf2tvdw0s3jn54khce6mua7l`

Valid descriptor strings with a checksum must pass the criteria for validity specified by the Python3 code snippet below. The function `descsum_check` must return true when its argument `s` is a descriptor consisting in the form `SCRIPT#CHECKSUM`.

```
INPUT_CHARSET = "0123456789()[],'/*abcdefgh@:$%{}IJKLMNOPQRSTUVWXYZ&+-.;<=>?!^_|
~ijklmnopqrstuvwxyzABCDEFGH`#\"\\ "
CHECKSUM_CHARSET = "qpzry9x8gf2tvdw0s3jn54khce6mua7l"
GENERATOR = [0xf5dee51989, 0xa9fdca3312, 0x1bab10e32d, 0x3706b1677a, 0x644d626ffd]

def descsum_polymod(symbols):
    """Internal function that computes the descriptor checksum."""
    chk = 1
    for value in symbols:
```

```
        top = chk >> 35
        chk = (chk & 0x7ffffffff) << 5 ^ value
        for i in range(5):
            chk ^= GENERATOR[i] if ((top >> i) & 1) else 0
    return chk

def descsum_expand(s):
    """Internal function that does the character to symbol expansion"""
    groups = []
    symbols = []
    for c in s:
        if not c in INPUT_CHARSET:
            return None
        v = INPUT_CHARSET.find(c)
        symbols.append(v & 31)
        groups.append(v >> 5)
        if len(groups) == 3:
            symbols.append(groups[0] * 9 + groups[1] * 3 + groups[2])
            groups = []
    if len(groups) == 1:
        symbols.append(groups[0])
    elif len(groups) == 2:
        symbols.append(groups[0] * 3 + groups[1])
    return symbols

def descsum_check(s):
    """Verify that the checksum is correct in a descriptor"""
    if s[-9] != '#':
        return False
    if not all(x in CHECKSUM_CHARSET for x in s[-8:]):
        return False
    symbols = descsum_expand(s[:-9]) + [CHECKSUM_CHARSET.find(x) for x in s[-8:]]
    return descsum_polymod(symbols) == 1
```

This implements a BCH code that has the properties described above. The entire descriptor string is first processed into an array of symbols. The symbol for each character is its position within its group. After every 3rd symbol, a 4th symbol is inserted which represents the group numbers combined together. This means that a change that only affects the position within a group, or only a group number change, will only affect a single symbol.

To construct a valid checksum given a script expression, the code below can be used:

```
def descsum_create(s):
    """Add a checksum to a descriptor without"""
    symbols = descsum_expand(s) + [0, 0, 0, 0, 0, 0, 0, 0]
    checksum = descsum_polymod(symbols) ^ 1
    return s + '#' + ''.join(CHECKSUM_CHARSET[(checksum >> (5 * (7 - i))) & 31] for i in
range(8))
```

Test Vectors

The following tests cover the checksum and character set:

- Valid checksum: `raw(deadbeef)#89f8spxm`

- No checksum: `raw(deadbeef)`
- Missing checksum: `raw(deadbeef)#`
- Too long checksum (9 chars): `raw(deadbeef)#89f8spxmx`
- Too short checksum (7 chars): `raw(deadbeef)#89f8spx`
- Error in payload: `raw(deedbeef)#89f8spxm`
- Error in checksum: `raw(deedbeef)##9f8spxm`
- Invalid characters in payload: `raw(Ü)#00000000`

The following tests cover key expressions:

Valid expressions:

- Compressed public key: `0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Uncompressed public key: `04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c02559e3aa`
- Public key with key origin: `[deadbeef/0h/0h/0h]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Public key with key origin (`'` as hardened indicator): `[deadbeef/0'/0'/0']0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Public key with key origin (mixed hardened indicator): `[deadbeef/0'/0h/0']0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- WIF uncompressed private key `5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j299sBCnWjss`
- WIF compressed private key `L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1`
- Extended public key: `xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcgJvL`
- Extended public key with key origin: `[deadbeef/0h/1h/2h]xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcg`
- Extended public key with derivation: `[deadbeef/0h/1h/2h]xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcg3/4/5`
- Extended public key with derivation and children: `[deadbeef/0h/1h/2h]xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcg3/4/5/*`
- Extended public key with hardened derivation and unhardened children: `xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcgJvL3h/4h/5h/*`
- Extended public key with hardened derivation and children: `xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcgJvL3h/4h/5h/*h`
- Extended public key with key origin, hardened derivation and children: `[deadbeef/0h/1h/2]xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcgJ3h/4h/5h/*h`
- Extended private key: `xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLYnMp`
- Extended private key with key origin: `[deadbeef/0h/1h/2h]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLY`

- Extended private key with derivation: `[deadbeef/0h/1h/2h]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLY3/4/5`
- Extended private key with derivation and children: `[deadbeef/0h/1h/2h]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLY3/4/5/*`
- Extended private key with hardened derivation and unhardened children: `xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLYnMp3h/4h/5h/*`
- Extended private key with hardened derivation and children: `xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLYnMp3h/4h/5h/*h`
- Extended private key with key origin, hardened derivation and children: `[deadbeef/0h/1h/2]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLYn3h/4h/5h/*h`

Invalid expression:

- Children indicator in key origin: `[deadbeef/0h/0h/0h/*]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Trailing slash in key origin: `[deadbeef/0h/0h/0h/]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Too short fingerprint: `[deadbef/0h/0h/0h]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Too long fingerprint: `[deadbeeef/0h/0h/0h]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Invalid hardened indicators: `[deadbeef/0f/0f/0f]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Invalid hardened indicators: `[deadbeef/-0/-0/-0]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Invalid hardened indicators: `[deadbeef/0H/0H/0H]0260b2003c386519fc9eadf2b5cf124dd8eea4c4e68d5e154050a9346ea98ce600`
- Invalid hardened indicators: `[deadbeef/0h/1h/2]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLYn3H/4h/5h/*H`
- Private key with derivation: `L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1/0`
- Private key with derivation children: `L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1/*`
- Derivation index out of range: `xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHNqPqm55Qn3LqFtT2emdEXVYsCzC2U/2147483648`
- Invalid derivation index: `xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHNqPqm55Qn3LqFtT2emdEXVYsCzC2U/1aa`
- Multiple key origins: `[aaaaaaaa][aaaaaaaa]xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHNqPqm55Qn3/2147483647'/0`

- Missing key origin start: `aaaaaaaa]xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHNqPqm55Qn3L2147483647'/0`
- Non hex fingerprint: `[gaaaaaaa]xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHNqPqm55Qn32147483647'/0`
- Key origin with no public key: `[deadbeef]`

Backwards Compatibility

Output script descriptors are an entirely new language which is not compatible with any existing software. However many components of the expressions reuse encodings and serializations defined by previous BIPs.

Output script descriptors are designed for future extension with further fragment types and new script expressions. These will be specified in additional BIPs.

Reference Implementation

Descriptors have been implemented in Bitcoin Core since version 0.17.

Appendix A: Index of Expressions

Future BIPs may specify additional types of expressions. All available expression types are listed in this table.

Name	Denoted As	BIP
Script	`SCRIPT`	380
Key	`KEY`	380
Tree	`TREE`	386
Annotation	`ANNOT`	?

Appendix B: Index of Script Expressions

Script expressions will be specified in additional BIPs. This Table lists all available Script expressions and the BIPs specifying them.

Expression	BIP
`pk(KEY)`	381
`pkh(KEY)`	381
`sh(SCRIPT)`	381
`wpkh(KEY)`	382
`wsh(SCRIPT)`	382
`multi(NUM, KEY, ..., KEY)`	383
`sortedmulti(NUM, KEY, ..., KEY)`	383
`combo(KEY)`	384

Expression	BIP
`raw(HEX)`	385
`addr(ADDR)`	385
`tr(KEY)`, `tr(KEY, TREE)`	386
`musig(KEY, KEY, ..., KEY)`	390
`sp(KEY)`, `sp(KEY, KEY)`	392

```
BIP: 381
Layer: Applications
Title: Non-Segwit Output Script Descriptors
Authors: Pieter Wuille <pieter@wuille.net>
         Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
Assigned: 2021-06-27
License: BSD-2-Clause
Requires: 380
```

BIP 381

Abstract

This document specifies `pk()`, `pkh()`, and `sh()` output script descriptors. `pk()` descriptors take a key and produces a P2PK output script. `pkh()` descriptors take a key and produces a P2PKH output script. `sh()` descriptors take a script and produces a P2SH output script.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

Prior to the activation of Segregated Witness, there were 3 main standard output script formats: P2PK, P2PKH, and P2SH. These expressions allow specifying those formats as a descriptor.

Specification

Three new script expressions are defined: `pk()`, `pkh()`, and `sh()`.

`pk()`

The `pk(KEY)` expression can be used in any context or level of a descriptor. It takes a single key expression as an argument and produces a P2PK output script. Depending on the higher level descriptors, there may be restrictions on the type of public keys that can be included. Such restrictions will be specified by those descriptors.

The output script produced is:

`<KEY> OP_CHECKSIG`

pkh()

The `pkh(KEY)` expression can be used as a top level expression, or inside of either a `sh()` or `wsh()` descriptor. It takes a single key expression as an argument and produces a P2PKH output script. Depending on the higher level descriptors, there may be restrictions on the type of public keys that can be included. Such restrictions will be specified by those descriptors.

The output script produced is:

`OP_DUP OP_HASH160 <KEY_hash160> OP_EQUALVERIFY OP_CHECKSIG`

sh()

The `sh(SCRIPT)` expression can only be used as a top level expression. It takes a single script expression as an argument and produces a P2SH output script. `sh()` expressions also create a redeemScript which is required in order to spend outputs which use its output script. This redeemScript is the output script produced by the `SCRIPT` argument to `sh()`.

The output script produced is:

`OP_HASH160 <SCRIPT_hash160> OP_EQUAL`

Test Vectors

Valid descriptors followed by the scripts they produce. Descriptors involving derived child keys will have the 0th, 1st, and 2nd scripts listed.

- `pk(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1)`
  - `2103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bdac`
- `pk(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)`
  - `2103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bdac`
- `pkh([deadbeef/1/2'/3/4']L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1)`
  - `76a9149a1c78a507689f6f54b847ad1cef1e614ee23f1e88ac`
- `pkh([deadbeef/1/2'/3/4']03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)`
  - `76a9149a1c78a507689f6f54b847ad1cef1e614ee23f1e88ac`
- `pkh([deadbeef/1/2h/3/4h]03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)`
  - `76a9149a1c78a507689f6f54b847ad1cef1e614ee23f1e88ac`
- `pk(5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j299sBCnWjss)`
  - `4104a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c0`
- `pk(04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c02559e`
  - `4104a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c0`
- `pkh(5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j299sBCnWjss)`
  - `76a914b5bd079c4d57cc7fc28ecf8213a6b791625b818388ac`
- `pkh(04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c02559`
  - `76a914b5bd079c4d57cc7fc28ecf8213a6b791625b818388ac`

- sh(pk(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1))
    - a9141857af51a5e516552b3086430fd8ce55f7c1a52487
- sh(pk(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))
    - a9141857af51a5e516552b3086430fd8ce55f7c1a52487
- sh(pkh(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1))
    - a9141a31ad23bf49c247dd531a623c2ef57da3c400c587
- sh(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))
    - a9141a31ad23bf49c247dd531a623c2ef57da3c400c587
- pkh(xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHNqPqm55Qn3LqFtT2147483647'/0)
    - 76a914ebdc90806a9c4356c1c88e42216611e1cb4c1c1788ac
- pkh([bd16bee5/2147483647h]xpub69H7F5dQzmVd3vPuLKtcXJziMEQByuDidnX3YdwgtNsecY5HRGtAAQC5mXTt4dsv9RzyjgDj0)
    - 76a914ebdc90806a9c4356c1c88e42216611e1cb4c1c1788ac
- pk(xprv9uPDJpEQgRQfDcW7BkF7eTya6RPxXeJCqCJGHuCJ4GiRVLzkTXBAJMu2qaMWPrS7AANYqdq6vcBcBUdJCVVFceUvJFjaPdG0)
    - 210379e45b3cf75f9c5f9befd8e9506fb962f6a9d185ac87001ec44a8d3df8d4a9e3ac
- pk(xpub68NZiKmJWnxxS6aaHmn81bvJeTESw724CRDs6HbuccFQN9Ku14VQrADWgqbhhTHBaohPX4CjNLf9fq9MYo6oDaPPLPxSb7g0)
    - 210379e45b3cf75f9c5f9befd8e9506fb962f6a9d185ac87001ec44a8d3df8d4a9e3ac

Invalid descriptors

- pk() only accepts key expressions: pk(pk(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))
- pkh() only accepts key expressions: pkh(pk(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))
- sh() only accepts script expressions: sh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)
- sh() is top level only: sh(sh(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)))

Backwards Compatibility

pk(), pkh(), and sh() descriptors use the format and general operation specified in 380. As these are wholly new descriptors, they are not compatible with any implementation. However the scripts produced are standard scripts so existing software are likely to be familiar with them.

Reference Implementation

pk(), pkh(), and sh() descriptors have been implemented in Bitcoin Core since version 0.17.

```
BIP: 382
Layer: Applications
Title: Segwit Output Script Descriptors
Authors: Pieter Wuille <pieter@wuille.net>
         Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
```

```
Assigned: 2021-06-27
License: BSD-2-Clause
Requires: 380
```

BIP 382

Abstract

This document specifies `wpkh()`, and `wsh()` output script descriptors. `wpkh()` descriptors take a key and produces a P2WPKH output script. `wsh()` descriptors take a script and produces a P2WSH output script.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

Segregated Witness added 2 additional standard output script formats: P2WPKH and P2WSH. These expressions allow specifying those formats as a descriptor.

Specification

Two new script expressions are defined: `wpkh()`, and `wsh()`.

`wpkh()`

The `wpkh(KEY)` expression can be used as a top level expression, or inside of a `sh()` descriptor. It takes a single key expression as an argument and produces a P2WPKH output script. Only keys which are/have compressed public keys can be contained in a `wpkh()` expression.

The output script produced is:

```
OP_0 <KEY_hash160>
```

`wsh()`

The `wsh(SCRIPT)` expression can be used as a top level expression, or inside of a `sh()` descriptor. It takes a single script expression as an argument and produces a P2WSH output script. `wsh()` expressions also create a witnessScript which is required in order to spend outputs which use its output script. This redeemScript is the output script produced by the `SCRIPT` argument to `wsh()`. Any key expression found in any script expression contained by a `wsh()` expression must only produce compressed public keys.

The output script produced is:

```
OP_0 <SCRIPT_sha256>
```

Test Vectors

Valid descriptors followed by the scripts they produce. Descriptors involving derived child keys will have the 0th, 1st, and 2nd scripts listed.

- `wpkh(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1)`
  - `00149a1c78a507689f6f54b847ad1cef1e614ee23f1e`
- `wpkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)`
  - `00149a1c78a507689f6f54b847ad1cef1e614ee23f1e`
- `wpkh([ffffffff/13']xprv9vHkqa6EV4sPZHYqZznhT2NPtPCjKuDKGY38FBWLvgaDx45zo9WQRUT3dKYnjwih2yJD9mkrocEZXo1ex8G81dwSM1fwqWpWkeS3v86pgKt/1/2/0)`
  - `0014326b2249e3a25d5dc60935f044ee835d090ba859`
- `wpkh([ffffffff/13']xpub69H7F5d8KSRgmmdJg2KhpAK8SR3DjMwAdkxj3ZuxV27CprR9LgpeyGmXUbC6wb7ERfvrnKZjXoUmmDznezpbZb7ap6r1D3tgFxHmwMkQTPH/1/2/*)`
  - `0014326b2249e3a25d5dc60935f044ee835d090ba859`
  - `0014af0bd98abc2f2cae66e36896a39ffe2d32984fb7`
  - `00141fa798efd1cbf95cebf912c031b8a4a6e9fb9f27`
- `sh(wpkh(xprv9s21ZrQH143K3QTDL4LXw2F7HEK3wJUD2nW2nRk4stbPy6cq3jPPqjiChkVvvNKmPGJxWUtg6LnF5kejMRNNU3TGtRBeJgk33yuGBxrMPHi/10/20/30/40/*'))`
  - `a9149a4d9901d6af519b2a23d4a2f51650fcba87ce7b87`
  - `a914bed59fc0024fae941d6e20a3b44a109ae740129287`
  - `a9148483aa1116eb9c05c482a72bada4b1db24af654387`
- `sh(wpkh(xprv9s21ZrQH143K3QTDL4LXw2F7HEK3wJUD2nW2nRk4stbPy6cq3jPPqjiChkVvvNKmPGJxWUtg6LnF5kejMRNNU3TGtRBeJgk33yuGBxrMPHi/10/20/30/40/*h))`
  - `a9149a4d9901d6af519b2a23d4a2f51650fcba87ce7b87`
  - `a914bed59fc0024fae941d6e20a3b44a109ae740129287`
  - `a9148483aa1116eb9c05c482a72bada4b1db24af654387`
- `wsh(pkh(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1))`
  - `0020338e023079b91c58571b20e602d7805fb808c22473cbc391a41b1bd3a192e75b`
- `wsh(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`
  - `0020338e023079b91c58571b20e602d7805fb808c22473cbc391a41b1bd3a192e75b`
- `wsh(pk(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1))`
  - `00202e271faa2325c199d25d22e1ead982e45b64eeb4f31e73dbdf41bd4b5fec23fa`
- `wsh(pk(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`
  - `00202e271faa2325c199d25d22e1ead982e45b64eeb4f31e73dbdf41bd4b5fec23fa`
- `sh(wsh(pkh(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1)))`
  - `a914b61b92e2ca21bac1e72a3ab859a742982bea960a87`
- `sh(wsh(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)))`
  - `a914b61b92e2ca21bac1e72a3ab859a742982bea960a87`

Invalid descriptors with descriptions

- Uncompressed public key in `wpkh()`: `wpkh(5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j299sBCnWjss)`
- Uncompressed public key in `wpkh()`: `sh(wpkh(5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j299sBCnWjss))`

- Uncompressed public key in `wpkh()`:
  `wpkh(04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c0255`
- Uncompressed public key in `wpkh()`:
  `sh(wpkh(04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c0`
- Uncompressed public keys under `wsh()`:
  `wsh(pk(5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j299sBCnWjss))`
- Uncompressed public keys under `wsh()`:
  `wsh(pk(04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c02`
- `wpkh()` nested in `wsh()`:
  `wsh(wpkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`
- `wsh()` nested in `wsh()`:
  `wsh(wsh(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)))`
- `wsh()` nested in `wsh()`:
  `sh(wsh(wsh(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))))`
- Script in `wpkh()`:
  `wpkh(wsh(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)))`
- Key in `wsh()`: `wsh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)`

Backwards Compatibility

`wpkh()`, and `wsh()` descriptors use the format and general operation specified in 380. As these are a wholly new descriptors, they are not compatible with any implementation. However the scripts produced are standard scripts so existing software are likely to be familiar with them.

Reference Implementation

`wpkh()`, and `wsh()` descriptors have been implemented in Bitcoin Core since version 0.17.

```
BIP: 383
Layer: Applications
Title: Multisig Output Script Descriptors
Authors: Pieter Wuille <pieter@wuille.net>
         Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
Assigned: 2021-06-27
License: BSD-2-Clause
Requires: 380
```

BIP 383

Abstract

This document specifies `multi()`, and `sortedmulti()` output script descriptors. Both functions take a threshold and one or more public keys and produce a multisig output script. `multi()` specifies the public keys in the output script in the order given in the descriptor while `sortedmulti()` sorts the public keys lexicographically when the output script is produced.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

The most common complex script used in Bitcoin is a threshold multisig. These expressions allow specifying multisig scripts as a descriptor.

Specification

Two new script expressions are defined: `multi()`, and `sortedmulti()`. Both expressions produce the scripts of the same template and take the same arguments. They are written as `multi(k,KEY_1,KEY_2,...,KEY_n)`. `k` is the threshold - the number of keys that must sign the input for the script to be valid. `KEY_1,KEY_2,...,KEY_n` are the key expressions for the multisig. `k` must be less than or equal to `n`.

`multi()` and `sortedmulti()` expressions can be used as a top level expression, or inside of either a `sh()` or `wsh()` descriptor. Depending on the higher level descriptors, there may be restrictions on the type of public keys that can be included.

Depending on the higher level descriptors, there are also restrictions on the number of keys that can be present, i.e. the maximum value of `n`. When used at the top level, there can only be at most 3 keys. When used inside of a `sh()` expression, the output script produced is the redeem script, which is pushed as a single element in the spending scriptSig and must therefore not exceed the 520 byte limit on the size of a script element. This allows at most 15 compressed public keys, or at most 7 uncompressed ones, as an uncompressed key takes 66 bytes of the script rather than 34. Otherwise the maximum number of keys is 20.

The output script produced will be

```
k KEY_1 KEY_2 ... KEY_n n OP_CHECKMULTISIG
```

The values `k` and `n` must be minimally encoded integers. For values less than or equal to 16, they must be encoded using `OP_0` through `OP_16`. For values greater than 16, they must be a push of the signed little endian encoded value without padding.

`sortedmulti()`

The only change for `sortedmulti()` is that the keys are sorted lexicographically prior to the creation of the output script. This sorting is on the keys that are to be put into the output script, i.e. after all extended keys are derived.

Multiple Extended Keys

When one or more the key expressions in a `multi()` or `sortedmulti()` expression are extended keys, the derived keys use the same child index. This changes the keys in lockstep and allows for output scripts to be indexed in the same way that the derived keys are indexed.

Test Vectors

Valid descriptors followed by the scripts they produce. Descriptors involving derived child keys will have the 0th, 1st, and 2nd scripts listed.

- `multi(1,L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1,5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j2`
  - `512103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd4104a34b99f22c790c4e36b2b3`
- `multi(1,03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd,04a34b99f22c790c4e36b2b3c2c`
  - `512103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd4104a34b99f22c790c4e36b2b3`
- `sortedmulti(1,04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c`
  - `512103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd4104a34b99f22c790c4e36b2b3`
- `sh(multi(2,[00000000/111'/222]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbL0))`
  - `a91445a9a622a8b0a1269944be477640eedc447bbd8487`
- `sortedmulti(2,xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grB*,xpub68NZiKmJWnxxS6aaHmn81bvJeTESw724CRDs6HbuccFQN9Ku14VQrADWgqbhhTHBaohPX4CjNLf9fq9MYo6oDaPPLPxSb7gw0/0/*)`
  - `5221025d5fc65ebb8d44a5274b53bac21ff8307fec2334a32df05553459f8b1f7fe1b62102fbd47cc8034098f0e6a94c`
  - `52210264fd4d1f5dea8ded94c61e9641309349b62f27fbffe807291f664e286bfbe6472103f4ece6dfccfa37b211eb3d`
  - `5221022ccabda84c30bad578b13c89eb3b9544ce149787e5b538175b1d1ba259cbb83321024d902e1a2fc7a8755ab5b6`
- `wsh(multi(2,xprv9s21ZrQH143K31xYSDQpPDxsXRTUcvj2iNHm5NUtrGiGG5e2DtALGdso3pGz6ssrdK4PFmM8NSpSBHNqPqm55Q2147483647'/0,xprv9vHkqa6EV4sPZHYqZznhT2NPtPCjKuDKGY38FBWLvgaDx45zo9WQRUT3dKYnjwih2yJD9mkrocEZXo1ex8G81dwSM1fwqWpW1/2/*,xprv9s21ZrQH143K3QTDL4LXw2F7HEK3wJUD2nW2nRk4stbPy6cq3jPPqjiChkVvvNKmPGJxWUtg6LnF5kejMRNNU3TGtRBeJgk310/20/30/40/*'))`
  - `0020b92623201f3bb7c3771d45b2ad1d0351ea8fbf8cfe0a0e570264e1075fa1948f`
  - `002036a08bbe4923af41cf4316817c93b8d37e2f635dd25cfff06bd50df6ae7ea203`
  - `0020a96e7ab4607ca6b261bfe3245ffda9c746b28d3f59e83d34820ec0e2b36c139c`
- `sh(wsh(multi(16,03669b8afcec803a0d323e9a17f3ea8e68e8abe5a278020a929adbec52421adbd0,0260b2003c386519fc9`
  - `a9147fc63e13dc25e8a95a3cee3d9a714ac3afd96f1e87`
- `wsh(multi(20,KzoAz5CanayRKex3fSLQ2BwJpN7U52gZvxMyk78nDMHuqrUxuSJy,KwGNz6YCCQtYvFzMtrC6D3tKTKdBBboMrLTs`
  - `0020376bd8344b8b6ebe504ff85ef743eaa1aa9272178223bcb6887e9378efb341ac`
- `sh(wsh(multi(20,KzoAz5CanayRKex3fSLQ2BwJpN7U52gZvxMyk78nDMHuqrUxuSJy,KwGNz6YCCQtYvFzMtrC6D3tKTKdBBboMr`
  - `a914c2c9c510e9d7f92fd6131e94803a8d34a8ef675e87`

Invalid descriptors

- More than 15 keys in P2SH multisig: `sh(multi(16,03669b8afcec803a0d323e9a17f3ea8e68e8abe5a278020a929adbec52421adbd0,0260b2003c386519fc9eadf`
- Invalid threshold: `multi(a,03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd,04a34b99f22c790c4e36b2b3c2c`
- Threshold of 0: `multi(0,03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd,04a34b99f22c790c4e36b2b3c2c`
- Threshold larger than keys: `multi(3,L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1,5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j2`

Backwards Compatibility

`multi()`, and `sortedmulti()` descriptors use the format and general operation specified in 380. As these are a wholly new descriptors, they are not compatible with any implementation. However the scripts produced are standard scripts so existing software are likely to be familiar with them.

Reference Implementation

`multi()`, and `sortedmulti()` descriptors have been implemented in Bitcoin Core since version 0.17.

```
  BIP: 384
  Layer: Applications
  Title: combo() Output Script Descriptors
  Authors: Pieter Wuille <pieter@wuille.net>
           Ava Chow <me@achow101.com>
  Status: Deployed
  Type: Specification
  Assigned: 2021-06-27
  License: BSD-2-Clause
  Requires: 380
```

BIP 384

Abstract

This document specifies `combo()` output script descriptors. These take a key and produce P2PK, P2PKH, P2WPKH, and P2SH-P2WPKH output scripts if applicable to the key.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

In order to make the transition from traditional key based wallets to descriptor based wallets easier, it is useful to be able to take a key and produce the scripts which have traditionally been produced by wallet software.

Specification

A new top level script expression is defined: `combo(KEY)`. This expression can only be used as a top level expression. It takes a single key expression as an argument and produces either 2 or 4 output scripts, depending on the key. A `combo()` expression always produces a P2PK and P2PKH script, the same as putting the key in both a `pk()` and a `pkh()` expression. If the key is/has a compressed public key, then P2WPKH and P2SH-P2WPKH scripts are also produced, the same as putting the key in both a `wpkh()` and `sh(wpkh())` expression.

Test Vectors

Valid descriptors followed by the scripts they produce. Descriptors involving derived child keys will have the 0th, and 1st scripts in additional sub-bullets.

- `combo(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1)`
    - `2103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bdac`
    - `76a9149a1c78a507689f6f54b847ad1cef1e614ee23f1e88ac`
    - `00149a1c78a507689f6f54b847ad1cef1e614ee23f1e`
    - `a91484ab21b1b2fd065d4504ff693d832434b6108d7b87`
- `combo(04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c025`
    - `4104a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c0`
    - `76a914b5bd079c4d57cc7fc28ecf8213a6b791625b818388ac`
- `combo([01234567]xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2g`
    - `2102d2b36900396c9282fa14628566582f206a5dd0bcc8d5e892611806cafb0301f0ac`
    - `76a91431a507b815593dfc51ffc7245ae7e5aee304246e88ac`
    - `001431a507b815593dfc51ffc7245ae7e5aee304246e`
    - `a9142aafb926eb247cb18240a7f4c07983ad1f37922687`
- `combo(xprvA2JDeKCSNNZky6uBCviVfJSKyQ1mDYahRjijr5idH2WwLsEd4Hsb2Tyh8RfQMuPh7f7RtyzTtdrbdqqsunu5Mm3wDvUA*)`
    - Child 0
        - `2102df12b7035bdac8e3bab862a3a83d06ea6b17b6753d52edecba9be46f5d09e076ac`
        - `76a914f90e3178ca25f2c808dc76624032d352fdbdfaf288ac`
        - `0014f90e3178ca25f2c808dc76624032d352fdbdfaf2`
        - `a91408f3ea8c68d4a7585bf9e8bda226723f70e445f087`
    - Child 1
        - `21032869a233c9adff9a994e4966e5b821fd5bac066da6c3112488dc52383b4a98ecac`
        - `76a914a8409d1b6dfb1ed2a3e8aa5e0ef2ff26b15b75b788ac`
        - `0014a8409d1b6dfb1ed2a3e8aa5e0ef2ff26b15b75b7`
        - `a91473e39884cb71ae4e5ac9739e9225026c99763e6687`

Invalid descriptors

- `combo()` in `sh` : `sh(combo(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`
- `combo()` in `wsh` : `wsh(combo(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`
- Script in `combo()`: `combo(pkh(03a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`

Backwards Compatibility

`combo()` descriptors use the format and general operation specified in 380. As this is a wholly new descriptor, it is not compatible with any implementation. However the scripts produced are standard scripts so existing software are likely to be familiar with them.

Reference Implementation

`combo()` descriptors have been implemented in Bitcoin Core since version 0.17.

```
BIP: 385
Layer: Applications
Title: raw() and addr() Output Script Descriptors
Authors: Pieter Wuille <pieter@wuille.net>
         Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
Assigned: 2021-06-27
License: BSD-2-Clause
Requires: 380
```

BIP 385

Abstract

This document specifies `raw()` and `addr()` output script descriptors. `raw()` encapsulates a raw script as a descriptor. `addr()` encapsulates an address as a descriptor.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

In order to make descriptors maximally compatible with scripts in use today, it is useful to be able to wrap any arbitrary output script or an address into a descriptor.

Specification

Two new script expressions are defined: `raw()` and `addr()`.

`raw()`

The `raw(HEX)` expression can only be used as a top level descriptor. As the argument, it takes a hex string representing a Bitcoin script. The output script produced by this descriptor is the script represented by `HEX`.

`addr()`

The `addr(ADDR)` expression can only be used as a top level descriptor. It takes an address as its single argument. The output script produced by this descriptor is the output script produced by the address `ADDR`.

Test Vectors

Valid descriptors followed by the scripts they produce.

- `raw(deadbeef)`
  - `deadbeef`
- `raw(512103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd4104a34b99f22c790c4e36b2b3c2`
  - `512103a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd4104a34b99f22c790c4e36b2b3`

- `raw(a9149a4d9901d6af519b2a23d4a2f51650fcba87ce7b87)`
    - `a9149a4d9901d6af519b2a23d4a2f51650fcba87ce7b87`
- `addr(3PUNyaW7M55oKWJ3kDukwk9bsKvryra15j)`
    - `a914eeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeee87`

Invalid descriptors

- Non-hex script: `raw(asdf)`
- Invalid address: `addr(asdf)`
- `raw` nested in `sh`: `sh(raw(deadbeef))`
- `raw` nested in `wsh`: `wsh(raw(deadbeef))`
- `addr` nested in `sh`: `sh(addr(3PUNyaW7M55oKWJ3kDukwk9bsKvryra15j))`
- `addr` nested in `wsh`: `wsh(addr(3PUNyaW7M55oKWJ3kDukwk9bsKvryra15j))`

Backwards Compatibility

`raw()` and `addr()` descriptors use the format and general operation specified in 380. As this is a wholly new descriptor, it is not compatible with any implementation. The reuse of existing Bitcoin addresses allows for this to be more easily implemented.

Reference Implementation

`raw()` and `addr()` descriptors have been implemented in Bitcoin Core since version 0.17.

```
BIP: 386
Layer: Applications
Title: tr() Output Script Descriptors
Authors: Pieter Wuille <pieter@wuille.net>
         Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
Assigned: 2021-06-27
License: BSD-2-Clause
Requires: 380
```

BIP 386

Abstract

This document specifies `tr()` output script descriptors. `tr()` descriptors take a key and optionally a tree of scripts and produces a P2TR output script.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

Taproot added one additional standard output script format: P2TR. These expressions allow specifying those formats as a descriptor.

Specification

A new script expression is defined: `tr()`. A new expression is defined: Tree Expressions

Tree Expression

A Tree Expression (denoted `TREE`) is an expression which represents a tree of scripts. The way the tree is represented in an output script is dependent on the higher level expressions.

A Tree Expression is:

- Any Script Expression that is allowed at the level this Tree Expression is in.
- A pair of Tree Expressions consisting of:
    - An open brace `{`
    - A Tree Expression
    - A comma `,`
    - A Tree Expression
    - A closing brace `}`

`tr()`

The `tr(KEY)` or `tr(KEY, TREE)` expression can only be used as a top level expression. All key expressions under any `tr()` expression must create x-only public keys.

`tr(KEY)` takes a single key expression as an argument and produces a P2TR output script which does not have a script path. Each key produced by the key expression is used as the internal key of a P2TR output as specified by BIP 341. Specifically, "If the spending conditions do not require a script path, the output key should commit to an unspendable script path instead of having no script path. This can be achieved by computing the output key point as $Q = P + int(hash_{TapTweak}(bytes(P)))G$."

```
internal_key:       lift_x(KEY)
32_byte_output_key: internal_key + int(HashTapTweak(bytes(internal_key)))G
scriptPubKey:       OP_1 <32_byte_output_key>
```

`tr(KEY, TREE)` takes a key expression as the first argument, and a tree expression as the second argument and produces a P2TR output script which has a script path. The keys produced by the first key expression are used as the internal key as specified by BIP 341. The Tree expression becomes the Taproot script tree as described in BIP 341. A merkle root is computed from this tree and combined with the internal key to create the Taproot output key.

```
internal_key:       lift_x(KEY)
merkle_root:        HashTapBranch(TREE)
32_byte_output_key: internal_key + int(HashTapTweak(bytes(internal_key) || merkle_root))G
scriptPubKey:       OP_1 <32_byte_output_key>
```

Modified Key Expression

Key Expressions within a `tr()` expression must only create x-only public keys. Uncompressed public keys are not allowed, but compressed public keys would be implicitly converted to x-only public keys. The keys

derived from extended keys must be serialized as x-only public keys. An additional key expression is defined only for use within a `tr()` descriptor:

- A 64 hex character string representing an x-only public key

Test Vectors

Valid descriptors followed by the scripts they produce. Descriptors involving derived child keys will have the 0th, 1st, and 2nd scripts listed.

- `tr(a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd)`
    - `512077aab6e066f8a7419c5ab714c12c67d25007ed55a43cadcacb4d7a970a093f11`
- `tr(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1)`
    - `512077aab6e066f8a7419c5ab714c12c67d25007ed55a43cadcacb4d7a970a093f11`
- `tr(xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbLY0/*,pk(xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQb1/*))`
    - `512078bc707124daa551b65af74de2ec128b7525e10f374dc67b64e00ce0ab8b3e12`
    - `512001f0a02a17808c20134b78faab80ef93ffba82261ccef0a2314f5d62b6438f11`
    - `512021024954fcec88237a9386fce80ef2ced5f1e91b422b26c59ccfc174c8d1ad25`
- `tr(a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd,pk(669b8afcec803a0d323e9a17f3ea8e6`
    - `512017cf18db381d836d8923b1bdb246cfcd818da1a9f0e6e7907f187f0b2f937754`
- `tr(a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd,{pk(xprvA2JDeKCSNNZky6uBCviVfJSKyQ1mDYahRjijr5idH2WwLsEd4Hsb2Tyh8RfQMuPh7f7RtyzTtdrbdqqsunu5Mm3wDvUAKR`
    - `512071fff39599a7b78bc02623cbe814efebf1a404f5d8ad34ea80f213bd8943f574`
- `tr(a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd,pkh(L4rK1yDtCWekvXuE6oXD9jCYfFNV2c`

Invalid Descriptors

- Uncompressed private key: `tr(5KYZdUEo39z3FPrtuX2QbbwGnNP5zTd7yyr2SC1j299sBCnWjss)`
- Uncompressed public key: `tr(04a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd5b8dec5235a0fa8722476c7709c02559e`
- `tr()` nested in `wsh`: `wsh(tr(a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`
- `tr()` nested in `sh`: `sh(tr(a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd))`

Backwards Compatibility

`tr()` descriptors use the format and general operation specified in 380. As these are a set of wholly new descriptors, they are not compatible with any implementation. However the scripts produced are standard scripts so existing software are likely to be familiar with them.

Tree Expressions are largely incompatible with existing script expressions due to the restrictions in those expressions. Of the script expressions that existed when this BIP was written, only `pk()` can be used in a tree expression. Script expressions specified since then that can be used in a tree expression are the Miniscript expressions of 379, which include a `pkh()` fragment, and the `multi_a()` and `sortedmulti_a()` expressions of 387.

Reference Implementation

`tr()` descriptors have been implemented in Bitcoin Core since version 22.0.

```
  BIP: 387
  Layer: Applications
  Title: Tapscript Multisig Output Script Descriptors
  Authors: Pieter Wuille <pieter@wuille.net>
           Ava Chow <me@achow101.com>
  Status: Deployed
  Type: Specification
  Assigned: 2024-04-17
  License: BSD-2-Clause
  Requires: 380
```

BIP 387

Abstract

This document specifies `multi_a()` and `sortedmulti_a()` output script descriptors. Like BIP 383's `multi()` and `sortedmulti()`, both functions take a threshold and one or more public keys and produce a multisig script. The primary distinction is that `multi_a()` and `sortedmulti_a()` only produce tapscripts and are only allowed in a tapscript context.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

The most common complex script used in Bitcoin is a threshold multisig. These expressions allow specifying multisig scripts as a descriptor.

Specification

Two new script expressions are defined: `multi_a()` and `sortedmulti_a()`. Both expressions produce the scripts of the same template and take the same arguments. They are written as `multi_a(k,KEY_1,KEY_2,...,KEY_n)`. `k` is the threshold - the number of keys that must sign the input for the script to be valid. `KEY_1,KEY_2,...,KEY_n` are the key expressions for the multisig. `k` must be less than or equal to `n`.

`multi_a()` and `sortedmulti_a()` expressions can only be used inside of a `tr()` descriptor. The maximum number of keys is 999.

The output script produced also depends on the value of `k`. If `k` is less than or equal to 16:

```
KEY_1 OP_CHECKSIG KEY_2 OP_CHECKSIGADD ... KEY_n OP_CHECKSIGADD OP_k OP_NUMEQUAL
```

if `k` is greater than 16:

```
KEY_1 OP_CHECKSIG KEY_2 OP_CHECKSIGADD ... KEY_n OP_CHECKSIGADD k OP_NUMEQUAL
```

`sortedmulti_a()`

The only change for `sortedmulti_a()` is that the x-only public keys are sorted lexicographically prior to the creation of the output script. This sorting is on the keys that are to be put into the output script, i.e. after all extended keys are derived.

Multiple Extended Keys

When one or more of the key expressions in a `multi_a()` or `sortedmulti_a()` expression are extended keys, the derived keys use the same child index. This changes the keys in lockstep and allows for output scripts to be indexed in the same way that the derived keys are indexed.

Test Vectors

Valid descriptors followed by the scripts they produce. Descriptors involving derived child keys will have the 0th, 1st, and 2nd scripts listed.

- `tr(L4rK1yDtCWekvXuE6oXD9jCYfFNV2cWRpVuPLBcCU2z8TrisoyY1,multi_a(1,KzoAz5CanayRKex3fSLQ2BwJpN7U52gZvxMy`
    - `5120eb5bd3894327d75093891cc3a62506df7d58ec137fcd104cdd285d67816074f3`
- `tr(a34b99f22c790c4e36b2b3c2c35a36db06226e41c692fc82b8b56ac1c540c5bd,multi_a(1,669b8afcec803a0d323e9a17`
    - `5120eb5bd3894327d75093891cc3a62506df7d58ec137fcd104cdd285d67816074f3`
- `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,multi_a(2,`
  `[00000000/111'/`
  `222]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbL`
  `0))`
    - `51202eea93581594a43c0c8423b70dc112e5651df63984d108d4fc8ccd3b63b4eafa`
- `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,sortedmulti_a(2,`
  `[00000000/111'/`
  `222]xprvA1RpRA33e1JQ7ifknakTFpgNXPmW2YvmhqLQYMmrj4xJXXWYpDPS3xz7iAxn8L39njGVyuoseXzU6rcxFLJ8HFsTjSyQbL`
  `0))`
    - `512016fa6a6ba7e98c54b5bf43b3144912b78a61b60b02f6a74172b8dcb35b12bc30`
- `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,sortedmulti_a(2,xpub6ERApfZwUNrhLC`
  `*,xpub68NZiKmJWnxxS6aaHmn81bvJeTESw724CRDs6HbuccFQN9Ku14VQrADWgqbhhTHBaohPX4CjNLf9fq9MYo6oDaPPLPxSb7gw`
  `0/0/*))`
    - `5120abd47468515223f58a1a18edfde709a7a2aab2b696d59ecf8c34f0ba274ef772`
    - `5120fe62e7ed20705bd1d3678e072bc999acb014f07795fa02cb8f25a7aa787e8cbd`
    - `51201311093750f459039adaa2a5ed23b0f7a8ae2c2ffb07c5390ea37e2fb1050b41`
- `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,multi_a(2,xprv9s21ZrQH143K31xYSDQp`
  `2147483647'/`
  `0,xprv9vHkqa6EV4sPZHYqZznhT2NPtPCjKuDKGY38FBWLvgaDx45zo9WQRUT3dKYnjwih2yJD9mkrocEZXo1ex8G81dwSM1fwqWpW`
  `1/2/`
  `*,xprv9s21ZrQH143K3QTDL4LXw2F7HEK3wJUD2nW2nRk4stbPy6cq3jPPqjiChkVvvNKmPGJxWUtg6LnF5kejMRNNU3TGtRBeJgk3`
  `10/20/30/40/*'))`
    - `5120e4c8f2b0a7d3a688ac131cb03248c0d4b0a59bbd4f37211c848cfbd22a981192`
    - `5120827faedaa21e52fca2ac83b53afd1ab7d4d1e6ce67ff42b19f2723d48b5a19ab`
    - `5120647495ed09de61a3a324704f9203c130d655bf3141f9b748df8f7be7e9af55a4`

Invalid descriptors

- Unsupported top level:
  `multi_a(1,03669b8afcec803a0d323e9a17f3ea8e68e8abe5a278020a929adbec52421adbd0)`
- Unsupported `sh()` context:
  `sh(multi_a(1,03669b8afcec803a0d323e9a17f3ea8e68e8abe5a278020a929adbec52421adbd0))`
- Unsupported `wsh()` context:
  `wsh(multi_a(1,03669b8afcec803a0d323e9a17f3ea8e68e8abe5a278020a929adbec52421adbd0))`
- Invalid threshold:
  `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,multi_a(a,03a34b99f22c790c4e36b2b3`
- Threshold of 0:
  `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,multi_a(0,03a34b99f22c790c4e36b2b3`
- Uncompressed pubkey:
  `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,multi_a(1,04a34b99f22c790c4e36b2b3`
- Threshold larger than keys:
  `tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,multi_a(3,L4rK1yDtCWekvXuE6oXD9jCY`

Backwards Compatibility

`multi_a()` and `sortedmulti_a()` descriptors use the format and general operation specified in 380. As these are wholly new descriptors, they are not compatible with any implementation. However, the scripts produced are standard scripts, so existing software are likely to be familiar with them.

Reference Implementation

`multi_a()` and `sortedmulti_a()` descriptors were implemented in Bitcoin Core in https://github.com/bitcoin/bitcoin/pull/24043 and have been available since version 24.0.

Partially Signed Transactions

A PSBT is a transaction that is not finished yet. It carries the inputs, the outputs, and the extra data each signer needs — previous scripts, derivation paths, signatures already collected — in a format every wallet can parse.

BIP 174 is the original format. BIP 370 is version 2, which is easier to modify before anyone has signed. BIP 371 adds the Taproot fields. BIP 373 adds the MuSig2 fields. If you implement hardware signing, coinjoin, or multisig, you will live in these four documents.

The idea is simple even when the fields are not: never ask a signer to guess. Put the UTXO, the script, and the bip32 path in the PSBT so a device that cannot see the chain can still produce a valid signature, and so a coordinator can combine those signatures without holding the keys.

In this chapter:

- BIP 174 — Partially Signed Bitcoin Transaction Format
- BIP 370 — PSBT Version 2
- BIP 371 — Taproot Fields for PSBT
- BIP 373 — MuSig2 PSBT Fields

```
BIP: 174
Layer: Applications
Title: Partially Signed Bitcoin Transaction Format
Authors: Ava Chow <me@achow101.com>
Comments-Summary: No comments yet.
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0174
Status: Deployed
Type: Specification
Assigned: 2017-07-12
License: BSD-2-Clause
Version: 1.4.4
```

BIP 174

Introduction

Abstract

This document proposes a binary transaction format which contains the information necessary for a signer to produce signatures for the transaction and holds the signatures for an input while the input does not have a complete set of signatures. The signer can be offline as all necessary information will be provided in the transaction.

The generic format is described here in addition to the specification for version 0 of this format.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

Creating unsigned or partially signed transactions to be passed around to multiple signers is currently implementation dependent, making it hard for people who use different wallet software from being able to easily do so. One of the goals of this document is to create a standard and extensible format that can be used between clients to allow people to pass around the same transaction to sign and combine their signatures. The format is also designed to be easily extended for future use which is harder to do with existing transaction formats.

Signing transactions also requires users to have access to the UTXOs being spent. This transaction format will allow offline signers such as air-gapped wallets and hardware wallets to be able to sign transactions without needing direct access to the UTXO set and without risk of being defrauded.

Specification

The Partially Signed Bitcoin Transaction (PSBT) format consists of key-value maps. Each map consists of a sequence of key-value records, terminated by a `0x00` byte **Why is the separator here `0x00` instead of `0xff`?** The separator here is used to distinguish between each chunk of data. A separator of 0x00 would mean that the unserializer can read it as a key length of 0, which would never occur with actual keys. It can thus be used as a separator and allow for easier unserializer implementation..

```
<psbt> := <magic> <global-map> <input-map>* <output-map>*
<magic> := 0x70 0x73 0x62 0x74 0xFF
<global-map> := <keypair>* 0x00
<input-map> := <keypair>* 0x00
<output-map> := <keypair>* 0x00
<keypair> := <key> <value>
<key> := <keylen> <keytype> <keydata>
<value> := <valuelen> <valuedata>
```

Where:

`<keytype>`
A compact size unsigned integer representing the type. This compact size unsigned integer must be minimally encoded, i.e. if the value can be represented using one byte, it must be represented as one byte. There can be multiple entries with the same `<keytype>` within a specific `<map>`, but the `<key>` must be unique.

`<keylen>`
The compact size unsigned integer containing the combined length of `<keytype>` and `<keydata>`

`<valuelen>`
The compact size unsigned integer containing the length of `<valuedata>` (which must be exactly this many bytes).

`<magic>`
Magic bytes which are ASCII for psbt **Why use 4 bytes for psbt?** The

transaction format needed to start with a 5 byte header which uniquely identifies it. The first bytes were chosen to be the ASCII for psbt because that stands for Partially Signed Bitcoin Transaction. followed by a separator of `0xFF`**Why Use a separator after the magic bytes?** The separator is part of the 5 byte header for PSBT. This byte is a separator of `0xff` because this will cause any non-PSBT unserializer to fail to properly unserialize the PSBT as a normal transaction. Likewise, since the 5 byte header is fixed, no transaction in the non-PSBT format will be able to be unserialized by a PSBT unserializer.. This integer must be serialized in most significant byte order.

The global types are defined as follows:

Name	**<keytype>**	**<keydata>**	**<keydata> Description**	**<valuedata>**	**<valuedata> Description**	**Versions Requiring Inclusion**	**Versions Requirin Exclusio**
Unsigned Transaction	`PSBT_GLOBAL_UNSIGNED_TX = 0x00`	None	No key data	`<bytes transaction>`	The transaction in network serialization. The scriptSigs and witnesses for each input must be empty. The	0	2

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	Versions Requirin Exclusio
					transaction must be in the old serialization format (without witnesses).		
Extended Public Key	PSBT_GLOBAL_XPUB = 0x01	<bytes xpub>	The 78 byte serialized extended public key as defined by BIP 32. Extended public keys are those that can be used to derive public keys used in the inputs and outputs of this transaction. It should be the public key at the highest hardened derivation index so that the unhardened child keys used in the transaction can be derived.	<4 byte fingerprint> <32-bit little endian uint path element>*	The master key fingerprint as defined by BIP 32 concatenated with the derivation path of the public key. The derivation path is represented as 32-bit little endian unsigned integer indexes concatenated with each other. The number of 32 bit unsigned integer indexes must match the depth provided in the extended public key.		
PSBT Version Number	PSBT_GLOBAL_VERSION = 0xFB	None	No key data	<32-bit little	The 32-bit little endian unsigned integer	2	

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	Versions Requirin Exclusio
				endian uint version>	representing the version number of this PSBT. If omitted, the version number is 0.		
Proprietary Use Type	PSBT_GLOBAL_PROPRIETARY = 0xFC	<compact size uint identifier length> <bytes identifier> <compact size uint subtype> <bytes subkeydata>	Compact size unsigned integer of the length of the identifier, followed by identifier prefix, followed by a compact size unsigned integer subtype, followed by the key data itself.	<bytes data>	Any value data as defined by the proprietary type user.		

Per-input types are defined as follows:

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description
Non-Witness UTXO	PSBT_IN_NON_WITNESS_UTXO = 0x00	None	No key data	<bytes transaction>	The transaction in networ serialization format the cu input spends from. This sh be present for inputs that non-segwit outputs and c present for inputs that spe segwit outputs. An input have both PSBT_IN_NON_WITNESS_UT PSBT_IN_WITNESS_UTXO. **W can both UTXO types be**

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description
					provided? Many wallets b requiring the full previous transaction (i.e. PSBT_IN_NON_WITNESS for segwit inputs when PS was already in use. In ord be compatible with softwa which were expecting PSBT_IN_WITNESS_UTXO, b UTXO types must be allow
Witness UTXO	PSBT_IN_WITNESS_UTXO = 0x01	None	No key data	<64-bit little endian int amount> <compact size uint scriptPubKeylen> <bytes scriptPubKey>	The entire transaction out network serialization whi current input spends from should only be present for inputs which spend segw outputs, including P2SH embedded ones. An input have both PSBT_IN_NON_WITNESS_UT PSBT_IN_WITNESS_UTXO
Partial Signature	PSBT_IN_PARTIAL_SIG = 0x02	<bytes pubkey>	The public key which corresponds to this signature.	<bytes signature>	The signature as would be pushed to the stack from a scriptSig or witness. The signature should be a vali ECDSA signature corresp to the pubkey that would true when verified and no value that would return fa be invalid otherwise (such NULLDUMMY).
Sighash Type	PSBT_IN_SIGHASH_TYPE = 0x03	None	No key data	<32-bit little endian uint sighash type>	The 32-bit unsigned integ specifying the sighash typ used for this input. Signat for this input must use the sighash type, finalizers m to finalize inputs which ha signatures that do not ma specified sighash type. Sig who cannot produce signa with the sighash type mus provide a signature.
		None	No key data		

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description
Redeem Script	PSBT_IN_REDEEM_SCRIPT = 0x04			<bytes redeemScript>	The redeemScript for this if it has one.
Witness Script	PSBT_IN_WITNESS_SCRIPT = 0x05	None	No key data	<bytes witnessScript>	The witnessScript for this if it has one.
BIP 32 Derivation Path	PSBT_IN_BIP32_DERIVATION = 0x06	<bytes pubkey>	The public key	<4 byte fingerprint> <32-bit little endian uint path element>*	The master key fingerprin defined by BIP 32 concate with the derivation path c public key. The derivation is represented as 32 bit un integer indexes concatena with each other. Public ke those that will be needed this input.
Finalized scriptSig	PSBT_IN_FINAL_SCRIPTSIG = 0x07	None	No key data	<bytes scriptSig>	The Finalized scriptSig co a fully constructed scriptS with signatures and any c scripts necessary for the in pass validation.
Finalized scriptWitness	PSBT_IN_FINAL_SCRIPTWITNESS = 0x08	None	No key data	<bytes scriptWitness>	The Finalized scriptWitne contains a fully constructe scriptWitness with signatu and any other scripts nece for the input to pass valid
RIPEMD160 preimage	PSBT_IN_RIPEMD160 = 0x0a	<20-byte hash>	The resulting hash of the preimage	<bytes preimage>	The hash preimage, encod a byte vector, which must the key when run through RIPEMD160 algorithm
SHA256 preimage	PSBT_IN_SHA256 = 0x0b	<32-byte hash>	The resulting hash of the preimage	<bytes preimage>	The hash preimage, encod a byte vector, which must the key when run through SHA256 algorithm
HASH160 preimage	PSBT_IN_HASH160 = 0x0c	<20-byte hash>	The resulting hash of the preimage	<bytes preimage>	The hash preimage, encod a byte vector, which must the key when run through SHA256 algorithm followe the RIPEMD160 algorithm
HASH256 preimage	PSBT_IN_HASH256 = 0x0d	<32-byte hash>	The resulting hash of the preimage	<bytes preimage>	The hash preimage, encod a byte vector, which must the key when run through SHA256 algorithm twice

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description
Proprietary Use Type	PSBT_IN_PROPRIETARY = 0xFC	<compact size uint identifier length> <bytes identifier> <compact size uint subtype> <bytes subkeydata>	Compact size unsigned integer of the length of the identifier, followed by identifier prefix, followed by a compact size unsigned integer subtype, followed by the key data itself.	<bytes data>	Any value data as defined the proprietary type user.

The per-output **Why do we need per-output data?** Per-output data allows signers to verify that the outputs are going to the intended recipient. The output data can also be use by signers to determine which outputs are change outputs and verify that the change is returning to the correct place. types are defined as follows:

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	Vers Requ Excl
Redeem Script	PSBT_OUT_REDEEM_SCRIPT = 0x00	None	No key data	<bytes redeemScript>	The redeemScript for this output if it has one.		
Witness Script	PSBT_OUT_WITNESS_SCRIPT = 0x01	None	No key data	<bytes witnessScript>	The witnessScript for this output if it has one.		
BIP 32 Derivation Path	PSBT_OUT_BIP32_DERIVATION = 0x02	<bytes public key>	The public key	<4 byte fingerprint> <32–bit little endian uint path element>*	The master key fingerprint concatenated with the		

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	Vers Requ Excl
					derivation path of the public key. The derivation path is represented as 32-bit little endian unsigned integer indexes concatenated with each other. Public keys are those needed to spend this output.		
Proprietary Use Type	PSBT_OUT_PROPRIETARY = 0xFC	<compact size uint identifier length> <bytes identifier> <compact size uint subtype> <bytes subkeydata>	Compact size unsigned integer of the length of the identifier, followed by identifier prefix, followed by a compact size unsigned integer subtype, followed by the key data itself.	<bytes data>	Any value data as defined by the proprietary type user.		

Types can be skipped when they are unnecessary. For example, if an input is a witness input, then it should not have a Non-Witness UTXO key-value pair.

If the signer encounters key-value pairs that it does not understand, it must pass those key-value pairs through when re-serializing the transaction.

All keys must have the data that they specify. If any key or value does not match the specified format for that type, the PSBT must be considered invalid. For example, any key that has no data except for the type specifier must only have the type specifier in the key.

Additional types may be defined by other BIPs. They should be added to the accompanying type registry tables to ease coordination.

Handling Duplicated Keys

Keys within each scope should never be duplicated; all keys in the format are unique. PSBTs containing duplicate keys are invalid. However implementers will still need to handle events where keys are duplicated when combining transactions with duplicated fields. In this event, the software may choose whichever value it wishes.**Why can the values be arbitrarily chosen?** When there are duplicated keys, the values that can be chosen will either be valid or invalid. If the values are invalid, a signer would simply produce an invalid signature and the final transaction itself would be invalid. If the values are valid, then it does not matter which is chosen as either way the transaction is still valid.

Proprietary Use Type

For all global, per-input, and per-output maps, the type `0xFC` is reserved for proprietary use. The proprietary use type requires keys that follow the type with a compact size unsigned integer representing the length of the string identifier, followed by the string identifier, then a subtype, and finally any key data.

The identifier can be any variable length string that software can use to identify whether the particular data in the proprietary type can be used by it. It can also be the empty string although this is not recommended.

The subtype is defined by the proprietary type user and can mean whatever they want it to mean. The subtype must also be a compact size unsigned integer in the same form as the normal types. The key data and value data are defined by the proprietary type user.

The proprietary use type is for private use by individuals and organizations who wish to use PSBT in their processes. It is useful when there are additional data that they need attached to a PSBT but such data are not useful or available for the general public. The proprietary use type is not to be used by any public specification and there is no expectation that any publicly available software be able to understand any specific meanings of it and the subtypes. This type must be used for internal processes only.

Version 0

Partially Signed Bitcoin Transactions version 0 is the first version of the PSBT format. Version 0 PSBTs must either omit PSBT_GLOBAL_VERSION or include it and set it to 0. Version 0 PSBTs must include PSBT_GLOBAL_UNSIGNED_TX, if omitted, the PSBT is invalid.

Roles

Using the transaction format involves many different roles. Multiple roles can be handled by a single entity, but each role is specialized in what it should be capable of doing.

Creator

The Creator creates a new PSBT. It must create an unsigned transaction and place it in the PSBT. The Creator must create empty input and output fields.

Updater

The Updater must only accept a PSBT. The Updater adds information to the PSBT that it has access to. If it has the UTXO for an input, it should add it to the PSBT. The Updater should also add redeemScripts, witnessScripts, and BIP 32 derivation paths to the input and output data if it knows them.

A single entity is likely to be both a Creator and Updater.

Signer

The Signer must only accept a PSBT. The Signer must only use the UTXOs provided in the PSBT to produce signatures for inputs. Before signing a non-witness input, the Signer must verify that the TXID of the non-witness UTXO matches the TXID specified in the unsigned transaction. Before signing a witness input, the Signer must verify that the witnessScript (if provided) matches the hash specified in the UTXO or the redeemScript, and the redeemScript (if provided) matches the hash in the UTXO. The Signer may choose to fail to sign a segwit input if a non-witness UTXO is not provided. **Why would non-witness UTXOs be provided for segwit inputs?** The sighash algorithm for Segwit specified in BIP 143 is known to have an issue where an attacker could trick a user to sending Bitcoin to fees if they are able to convince the user to sign a malicious transaction multiple times. This is possible because the amounts in `PSBT_IN_WITNESS_UTXO` of other segwit inputs can be modified without effecting the signature for a particular input. In order to prevent this kind of attack, many wallets are requiring that the full previous transaction (i.e. `PSBT_IN_NON_WITNESS_UTXO`) be provided to ensure that the amounts of other inputs are not being tampered with. The Signer should not need any additional data sources, as all necessary information is provided in the PSBT format. The Signer must only add data to a PSBT. Any signatures created by the Signer must be added as a "Partial Signature" key-value pair for the respective input it relates to. If a Signer cannot sign a transaction, it must not add a Partial Signature.

The Signer can additionally compute the addresses and values being sent, and the transaction fee, optionally showing this data to the user as a confirmation of intent and the consequences of signing the PSBT.

Signers do not need to sign for all possible input types. For example, a signer may choose to only sign Segwit inputs.

A single entity is likely to be both a Signer and an Updater as it can update a PSBT with necessary information prior to signing it.

Data Signers Check For

For a Signer to only produce valid signatures for what it expects to sign, it must check that the following conditions are true:

- If a non-witness UTXO is provided, its hash must match the hash specified in the prevout
- If a witness UTXO is provided, no non-witness signature may be created
- If a redeemScript is provided, the scriptPubKey must be for that redeemScript
- If a witnessScript is provided, the scriptPubKey or the redeemScript must be for that witnessScript

- If a sighash type is provided, the signer must check that the sighash is acceptable. If unacceptable, they must fail.
- If a sighash type is not provided, the signer should sign using SIGHASH_ALL, but may use any sighash type they wish.

Simple Signer Algorithm

A simple signer can use the following algorithm to determine what and how to sign

```
sign_witness(script_code, i):
    for key, sighash_type in psbt.inputs[i].items:
        if sighash_type == None:
            sighash_type = SIGHASH_ALL
        if IsMine(key) and IsAcceptable(sighash_type):
            sign(witness_sighash(script_code, i, input))

sign_non_witness(script_code, i):
    for key, sighash_type in psbt.inputs[i].items:
        if sighash_type == None:
            sighash_type = SIGHASH_ALL
        if IsMine(key) and IsAcceptable(sighash_type):
            sign(non_witness_sighash(script_code, i, input))

for input, i in enumerate(psbt.inputs):
    if witness_utxo.exists:
        if redeemScript.exists:
            assert(witness_utxo.scriptPubKey == P2SH(redeemScript))
            script = redeemScript
        else:
            script = witness_utxo.scriptPubKey
        if IsP2WPKH(script):
            sign_witness(P2PKH(script[2:22]), i)
        else if IsP2WSH(script):
            assert(script == P2WSH(witnessScript))
            sign_witness(witnessScript, i)
    else if non_witness_utxo.exists:
        assert(sha256d(non_witness_utxo) == psbt.tx.input[i].prevout.hash)
        if redeemScript.exists:
            assert(non_witness_utxo.vout[psbt.tx.input[i].prevout.n].scriptPubKey ==
P2SH(redeemScript))
            sign_non_witness(redeemScript, i)
        else:
            sign_non_witness(non_witness_utxo.vout[psbt.tx.input[i].prevout.n].scriptPubKey, i)
    else:
        assert False
```

Change Detection

Signers may wish to display the inputs and outputs to users for extra verification. In such displays, signers may wish to identify which outputs are change outputs in order to omit them to avoid additional user confusion. In order to detect change, a signer can use the BIP 32 derivation paths provided in inputs and outputs as well as the extended public keys provided globally.

For a single key output, a signer can observe whether the master fingerprint for the public key for that output belongs to itself. If it does, it can then derive the public key at the specified derivation path and check whether that key is the one present in that output.

For outputs involving multiple keys, a signer can first examine the inputs that it is signing. It should determine the general pattern of the script and internally produce a representation of the policy that the script represents. Such a policy can include things like how many keys are present, what order they are in, how many signers are necessary, which signers are required, etc. The signer can then use the BIP 32 derivation paths for each of the pubkeys to find which global extended public key is the one that can derive that particular public key. To do so, the signer would extract the derivation path to the highest hardened index and use that to lookup the public key with that index and master fingerprint. The signer would construct this script policy with extended public keys for all of the inputs and outputs. Change outputs would then be identified as being the outputs which have the same script policy as the inputs that are being signed.

Combiner

The Combiner can accept 1 or many PSBTs. The Combiner must merge them into one PSBT (if possible), or fail. The resulting PSBT must contain all of the key-value pairs from each of the PSBTs. The Combiner must remove any duplicate key-value pairs, in accordance with the specification. It can pick arbitrarily when conflicts occur. A Combiner must not combine two different PSBTs. PSBTs can be uniquely identified by `0x00` global transaction typed key-value pair. For every type that a Combiner understands, it may refuse to combine PSBTs if it detects that there will be inconsistencies or conflicts for that type in the combined PSBT.

The Combiner does not need to know how to interpret scripts in order to combine PSBTs. It can do so without understanding scripts or the network serialization format.

In general, the result of a Combiner combining two PSBTs from independent participants A and B will be functionally equivalent to a result obtained from processing the original PSBT by A and then B in a sequence. Or, for participants performing fA(psbt) and fB(psbt) where fA() and fB() only add unique fields to the PSBT: Combine(fA(psbt), fB(psbt)) == fA(fB(psbt)) == fB(fA(psbt)) Combining two PSBTs is commutative unless the PSBTs have conflicting fields, i.e., fields with duplicated keys but differing values.

As discussed in Handling Duplicated Keys, when a combiner is asked to combine PSBTs with conflicting fields, the combiner may arbitrarily choose the value for the combined PSBT from any of the original PSBTs. Alternatively, the combiner may refuse to combine PSBTs with conflicting content. In this case, the previously discussed commutative property no longer holds.

Input Finalizer

The Input Finalizer must only accept a PSBT. For each input, the Input Finalizer determines if the input has enough data to pass validation. If it does, it must construct the `0x07` Finalized scriptSig and `0x08` Finalized scriptWitness and place them into the input key-value map. If the input has a `PSBT_IN_SIGHASH_TYPE` field, the Input Finalizer must fail to finalize that input if any signature does not match the specified sighash type. If scriptSig is empty for an input, `0x07` should remain unset rather than assigned an empty array. Likewise, if no scriptWitness exists for an input, `0x08` should remain unset rather than assigned an empty array. All other data except the UTXO and unknown fields (including `PSBT_IN_PROPRIETARY` fields the Input Finalizer does not understand) in the input key-value map should be cleared from the PSBT. The UTXO should be kept to allow Transaction Extractors to verify the final network serialized transaction.

Transaction Extractor

The Transaction Extractor must only accept a PSBT. It checks whether all inputs have complete scriptSigs and scriptWitnesses by checking for the presence of `0x07` Finalized scriptSig and `0x08` Finalized scriptWitness typed records. If they do, the Transaction Extractor should construct complete scriptSigs and scriptWitnesses and encode them into network serialized transactions. Otherwise the Extractor must not modify the PSBT. The Extractor should produce a fully valid, network serialized transaction if all inputs are complete.

The Transaction Extractor does not need to know how to interpret scripts in order to extract the network serialized transaction. However it may be able to in order to validate the network serialized transaction at the same time.

A single entity is likely to be both a Transaction Extractor and an Input Finalizer.

Encoding

A PSBT can be represented in two ways: in binary (as a file) or as a Base64 string using the encoding described in RFC4648.

Binary PSBT files should use the `.psbt` file extension. A MIME type name will be added to this document once one has been registered.

Extensibility

The Partially Signed Transaction format can be extended in the future by adding new types for key-value pairs. Backwards compatibility will still be maintained as those new types will be ignored and passed-through by signers which do not know about them.

Version Numbers

The Version number field exists only as a safeguard in the event that a backwards incompatible change is introduced to PSBT. If a parser encounters a version number it does not recognize, it should exit immediately as this indicates that the PSBT will contain types that it does not know about and cannot be ignored. Current PSBTs are Version 0. Any PSBT that does not have the version field is version 0. It is not expected that any backwards incompatible change will be introduced to PSBT, so it is not expected that the version field will ever actually be seen.

Updaters and combiners that need to add a version number to a PSBT should use the highest version number required. For example, if a combiner sees two PSBTs for the same transaction, one with version 0, and the other with version 1, then it should combine them and produce a PSBT with version 1. If an updater is updating a PSBT and needs to add a field that is only available in version 1, then it should set the PSBT version number to 1 unless a version higher than that is already specified.

Procedure For New Fields

New fields should first be proposed on the bitcoin-dev mailing list, and their usage should then be defined in a separate BIP. New fields must be added to the type registry tables. Although some PSBT version 0 implementations encode types as uint8_t rather than compact size, it is still safe to add >0xFD fields to PSBT 0, because these old parsers ignore unknown fields, and <keytype> is prefixed by its length.

Procedure For New Versions

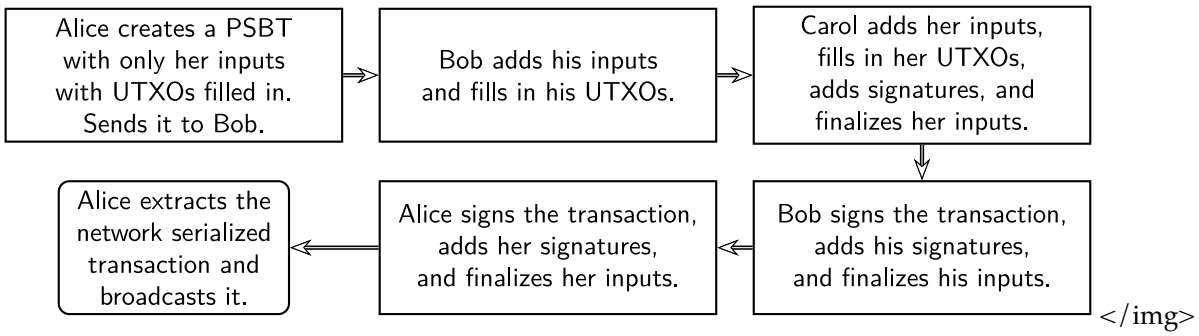
New PSBT versions must be described in a separate BIP. Such new BIPs may reference this BIP and any components of PSBT version 0 that are retained in the new version. Any new fields described in a new version must be added to the type registry tables.

Compatibility

This transaction format is designed so that it is unable to be properly unserialized by normal transaction unserializers. Likewise, a normal transaction will not be able to be unserialized by an unserializer for the PSBT format.

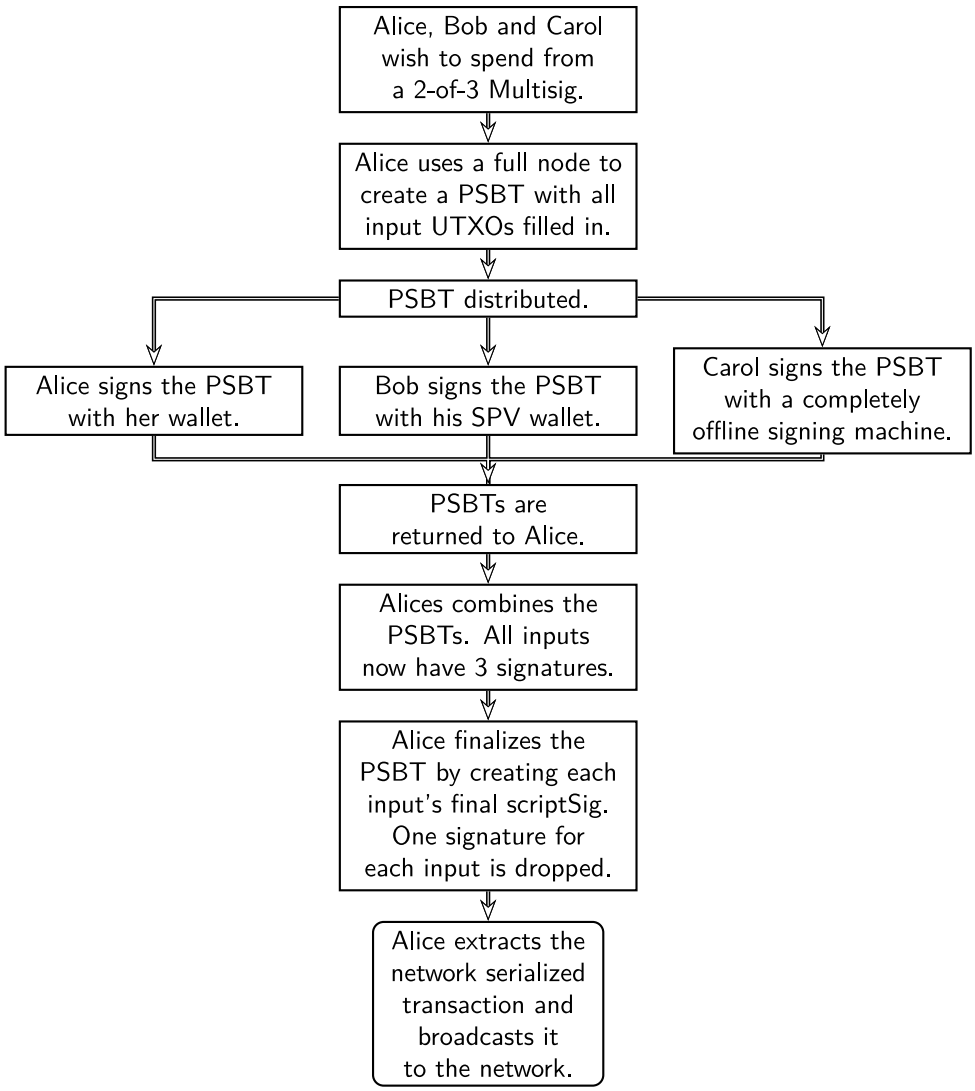
Examples

Manual CoinJoin Workflow



</img>

2-of-3 Multisig Workflow



</img>

Changelog

- **1.4.4** (2026-09-23):
  - Clarify that combiner test vectors must pass in any input order unless there are conflicting fields
  - Add combiner test vectors for preserving PSBT_IN_SIGHASH_TYPE
- **1.4.3** (2026-07-16):
  - Clarify that combining PSBTs is commutative unless they have conflicting fields
- **1.4.2** (2026-04-08):
  - Introduce type registry auxiliary file
  - Add changelog
- **1.4.1** (2021-01-14):
  - Mark Final (Later updated to "Deployed" with adoption of BIP3)
- **1.4.0** (2019-10-02):
  - Add preimage fields

- **1.3.0** (2019-10-02):
    - Add proprietary fields
- **1.2.0** (2019-08-01):
    - Types are compact size unsigned ints
    - Add global PSBT version field
- **1.1.0** (2019-05-13):
    - Add global xpub field
- **1.0.0** (2018-07-11):
    - Mark Proposed

Test Vectors

The following are invalid PSBTs:

- Case: Network transaction, not PSBT format
    - Bytes in Hex:
      020000000126817137 1edff285e937adeea4b37b78000c0566cbb3ad64641713ca42171bf6000000006a473044022070b22
    - Base64 String: AgAAAAEmgXE3Ht/
      yhek3re6ks3t4AAwFZsuzrWRkFxPKQhcb9gAAAABqRzBEAiBwsiRRI+a/
      R01gxbUMBD1MaRpdJDXwmjSnZiqdwlF5CgIgATKcqdrPKAvfMHQOwDkEIkIsgctFg5RXrrdvwS7dlbMBIQJ

- Case: PSBT missing outputs
    - Bytes in Hex:
      70736274ff0100750200000001268171371edff285e937adeea4b37b78000c0566cbb3ad64641713ca42171bf6000000000
    - Base64 String: cHNidP8BAHUCAAAAASaBcTce3/
      KF6Tet7qSze3gADAVmy7OtZGQXE8pCFxv2AAAAAAD+////
      AtPf9QUAAAAAGXapFNDFmQPFusKGh2DpD9UhpGZap2UgiKwA4fUFAAAAABepFDVF5uM7gyxHBQ8k0
      hTPMti+VZ+UBAAAAFxYAFL4Y0VKpsBIDna89p95PUzSe7LmF/////
      4b4qkOnHf8USIk6UwpyN+9rRgi7st0tAXHmOuxqSJC0AQAAABcWABT+Pp7xp0XpdNkCxDVZQ6vLNL1TU/
      8CAMLrCwAAAAAZdqkUhc/xCX/Z4Ai7NK9wnGIZeziXikiIrHL+
      +E4sAAAAF6kUM5cluiHv1irHU6m80GfWx6ajnQWHAkcwRAIgJxK+IuAnDzlPVoMR3HyppolwuAJf3TskAin
      jnF6efkE/IQUCSDBFAiEA0SuFLYXc2WHS9fSrZgZU327tzHlMDDPOXMMJ/
      7X85Y0CIGczio4OFyXBl/
      saiK9Z9R5E5CVbIBZ8hoQDHAXR8lkqASECI7cr7vCWXRC+B3jv7NYfysb3mk6haTkzgHNEZPhPKrMAAAAA

- Case: PSBT where one input has a filled scriptSig in the unsigned tx
    - Bytes in Hex:
      70736274ff0100fd0a010200000002ab0949a08c5af7c49b8212f417e2f15ab3f5c33dcf153821a8139f877a5b7be40000000
    - Base64 String:
      cHNidP8BAP0KAQIAAAACqwlJoIxa98SbghL0F+LxWrP1wz3PFTghqBOfh3pbe+QAAAAAakcwRAIgR1lmF5
      REf3xM286KdqGbX+EhtdVRs7tr5MZASEDXNxh/
      HupccC1AaZGoqg7ECy0OIEhfKaC3Ibi1z+ogpL+////
      qwlJoIxa98SbghL0F+LxWrP1wz3PFTghqBOfh3pbe+QBAAAAAP7///8CYDvqCwAAAAZdqkUdopAu9dAy

p+gOcykWXiKwAAAAAAAABASAA4fUFAAAAABepFDVF5uM7gyxHBQ8k0+65PJwDlIvHhwEEFgAUhdE1
LiZUBaNNuvqePdoB+4IwgAAAA=

- Case: PSBT where inputs and outputs are provided but without an unsigned tx
  - Bytes in Hex:
    70736274ff000100fda501010000000000010289a3c71eab4d20e0371bbba4cc698fa295c9463afa2e397f8533ccb62f9567e50
  - Base64 String: cHNidP8AAQD9pQEBAAAAAAECiaPHHqtNIOA3G7ukzGmPopXJRjr6Ljl/
    hTPMti+VZ+UBAAAAFxYAFL4Y0VKpsBIDna89p95PUzSe7LmF/////
    4b4qkOnHf8USIk6UwpyN+9rRgi7st0tAXHmOuxqSJC0AQAAABcWABT+Pp7xp0XpdNkCxDVZQ6vLNL1TU
    8CAMLrCwAAAAAZdqkUhc/xCX/Z4Ai7NK9wnGIZeziXikiIrHL+
    +E4sAAAAF6kUM5cluiHv1irHU6m80GfWx6ajnQWHAkcwRAIgJxK+IuAnDzlPVoMR3HyppolwuAJf3TskAin
    jnF6efkE/IQUCSDBFAiEA0SuFLYXc2WHS9fSrZgZU327tzHlMDDPOXMMJ/
    7X85Y0CIGczio4OFyXBl/
    saiK9Z9R5E5CVbIBZ8hoQDHAXR8lkqASECI7cr7vCWXRC+B3jv7NYfysb3mk6haTkzgHNEZPhPKrMAAAAA

- Case: PSBT with duplicate keys in an input
  - Bytes in Hex:
    70736274ff01007502000000012681713 71edff285e937adeea4b37b78000c0566cbb3ad64641713ca42171bf6000000000
  - Base64 String: cHNidP8BAHUCAAAAASaBcTce3/
    KF6Tet7qSze3gADAVmy7OtZGQXE8pCFxv2AAAAAAD+////
    AtPf9QUAAAAAGXapFNDFmQPFusKGh2DpD9UhpGZap2UgiKwA4fUFAAAAABepFDVF5uM7gyxHBQ8k0
    hTPMti+VZ+UBAAAAFxYAFL4Y0VKpsBIDna89p95PUzSe7LmF/////
    4b4qkOnHf8USIk6UwpyN+9rRgi7st0tAXHmOuxqSJC0AQAAABcWABT+Pp7xp0XpdNkCxDVZQ6vLNL1TU
    8CAMLrCwAAAAAZdqkUhc/xCX/Z4Ai7NK9wnGIZeziXikiIrHL+
    +E4sAAAAF6kUM5cluiHv1irHU6m80GfWx6ajnQWHAkcwRAIgJxK+IuAnDzlPVoMR3HyppolwuAJf3TskAin
    jnF6efkE/IQUCSDBFAiEA0SuFLYXc2WHS9fSrZgZU327tzHlMDDPOXMMJ/
    7X85Y0CIGczio4OFyXBl/
    saiK9Z9R5E5CVbIBZ8hoQDHAXR8lkqASECI7cr7vCWXRC+B3jv7NYfysb3mk6haTkzgHNEZPhPKrMAAAAA
    AgAAAAH/////////////////////////////////////wAAAAAA/////
    wEAAAAAAAAAANqAQAAAAAAAAA

- Case: PSBT with invalid global transaction typed key
  - Bytes in Hex:
    70736274ff02000155020000000127 9a2323a5dfb51fc45f220fa58b0fc13e1e3342792a85d7e36cd6333b5cbc390000000000
  - Base64 String: cHNidP8CAAFVAgAAAAEnmiMjpd+1H8RfIg+liw/
    BPh4zQnkqhdfjbNYzO1y8OQAAAAAA/////wGgWuoLAAAAABl2qRT/
    6cAGEJfMO2NvLLBGD6T8Qn0rRYisAAAAAAABASCVXuoLAAAAABepFGNFIA9o0YnhrcDfHE0W6o8UwN
    6qmlOFVnqXJO7/UqJBkIkBVzfBwtncUaUQtBwIfXI6w/qZRbWC4rLM61k7eYOh4W/
    s6qUuZvfhhUduamgEBBCIAIHcf0YrUWWZt1J89Vk49vEL0yEd042CtoWgWqO1IjVaBAQVHUiEDsTQcy6doO
    KSZXyb4IIO9Uq4iBgOxNBzLp2g7avTxI4zW6X5xZ9Vp+sR/
    HkjUdUGEQ1W9RhC0prpnAAAAgAAAAIAEAACAIgYD3lXR4drIBeP4pYwfv5uUwC89uq/
    hJ/78pJlfJvggg70QtKa6ZwAAAIAAAACABQAAgAAA

- Case: PSBT with invalid input witness utxo typed key
  - Bytes in Hex:
    70736274ff0100550200000001279a2323a5dfb51fc45f220fa58b0fc13e1e3342792a85d7e36cd6333b5cbc39000000000000
  - Base64 String:
    cHNidP8BAFUCAAAAASeaIyOl37UfxF8iD6WLD8E+HjNCeSqF1+Ns1jM7XLw5AAAAAAD/////

AaBa6gsAAAAAGXapFP/
pwAYQl8w7Y28ssEYPpPxCfStFiKwAAAAAAAIBACCVXuoLAAAAABepFGNFIA9o0YnhrcDfHE0W6o8UwN
6qmlOFVnqXJO7/UqJBkIkBVzfBwtncUaUQtBwIfXI6w/qZRbWC4rLM61k7eYOh4W/
s6qUuZvfhhUduamgEBBCIAIHcf0YrUWWZt1J89Vk49vEL0yEd042CtoWgWqO1IjVaBAQVHUiEDsTQcy6doO
KSZXyb4IIO9Uq4iBgOxNBzLp2g7avTxI4zW6X5xZ9Vp+sR/
HkjUdUGEQ1W9RhC0prpnAAAAgAAAAIAEAACAIgYD3lXR4drIBeP4pYwfv5uUwC89uq/
hJ/78pJlfJvggg70QtKa6ZwAAAIAAAACABQAAgAAA

- Case: PSBT with invalid pubkey length for input partial signature typed key
  - Bytes in Hex:
    70736274ff0100550200000001279a2323a5dfb51fc45f220fa58b0fc13e1e3342792a85d7e36cd6333b5cbc390000000000f
  - Base64 String:
    cHNidP8BAFUCAAAAASeaIyOl37UfxF8iD6WLD8E+HjNCeSqF1+Ns1jM7XLw5AAAAAAD/////
    AaBa6gsAAAAAGXapFP/
    pwAYQl8w7Y28ssEYPpPxCfStFiKwAAAAAAAEBIJVe6gsAAAAF6kUY0UgD2jRieGtwN8cTRbqjxTA2+uHIQ
    1KiQZCJAVc3wcLZ3FGlELQcCH1yOsP6mUW1guKyzOtZO3mDoeFv7OqlLmb34YVHbmpoBAQQiACB3H9G
    iljB+/m5TALz26r+En/
    vykmV8m+CCDvVKuIgYDsTQcy6doO2r08SOM1ul+cWfVafrEfx5I1HVBhENVvUYQtKa6ZwAAAIAAAACAE
    KSZXyb4IIO9ELSmumcAAACAAAAAgAUAAIAAAA==

- Case: PSBT with invalid redeemscript typed key
  - Bytes in Hex:
    70736274ff0100550200000001279a2323a5dfb51fc45f220fa58b0fc13e1e3342792a85d7e36cd6333b5cbc390000000000f
  - Base64 String:
    cHNidP8BAFUCAAAAASeaIyOl37UfxF8iD6WLD8E+HjNCeSqF1+Ns1jM7XLw5AAAAAAD/////
    AaBa6gsAAAAAGXapFP/
    pwAYQl8w7Y28ssEYPpPxCfStFiKwAAAAAAAEBIJVe6gsAAAAF6kUY0UgD2jRieGtwN8cTRbqjxTA2+uHIg
    qqaU4VWepck7v9SokGQiQFXN8HC2dxRpRC0HAh9cjrD+plFtYLisszrWTt5g6Hhb+zqpS5m9+GFR25qaAQIEA
    KSZXyb4IIO9Uq4iBgOxNBzLp2g7avTxI4zW6X5xZ9Vp+sR/
    HkjUdUGEQ1W9RhC0prpnAAAAgAAAAIAEAACAIgYD3lXR4drIBeP4pYwfv5uUwC89uq/
    hJ/78pJlfJvggg70QtKa6ZwAAAIAAAACABQAAgAAA

- Case: PSBT with invalid witnessscript typed key
  - Bytes in Hex:
    70736274ff0100550200000001279a2323a5dfb51fc45f220fa58b0fc13e1e3342792a85d7e36cd6333b5cbc390000000000f
  - Base64 String:
    cHNidP8BAFUCAAAAASeaIyOl37UfxF8iD6WLD8E+HjNCeSqF1+Ns1jM7XLw5AAAAAAD/////
    AaBa6gsAAAAAGXapFP/
    pwAYQl8w7Y28ssEYPpPxCfStFiKwAAAAAAAEBIJVe6gsAAAAF6kUY0UgD2jRieGtwN8cTRbqjxTA2+uHIg
    qqaU4VWepck7v9SokGQiQFXN8HC2dxRpRC0HAh9cjrD+plFtYLisszrWTt5g6Hhb+zqpS5m9+GFR25qaAQEE
    RitRZZm3Unz1WTj28QvTIR3TjYK2haBao7UiNVoECBQBHUiEDsTQcy6doO2r08SOM1ul+cWfVafrEfx5I1HVBh
    KSZXyb4IIO9Uq4iBgOxNBzLp2g7avTxI4zW6X5xZ9Vp+sR/
    HkjUdUGEQ1W9RhC0prpnAAAAgAAAAIAEAACAIgYD3lXR4drIBeP4pYwfv5uUwC89uq/
    hJ/78pJlfJvggg70QtKa6ZwAAAIAAAACABQAAgAAA

- Case: PSBT with invalid pubkey in input BIP 32 derivation paths typed key
  - Bytes in Hex:
    70736274ff0100550200000001279a2323a5dfb51fc45f220fa58b0fc13e1e3342792a85d7e36cd6333b5cbc390000000000f

    - Base64 String:
      cHNidP8BAFUCAAAAASeaIyOl37UfxF8iD6WLD8E+HjNCeSqF1+Ns1jM7XLw5AAAAAAD/////
      AaBa6gsAAAAAGXapFP/
      pwAYQl8w7Y28ssEYPpPxCfStFiKwAAAAAAAEBIJVe6gsAAAAF6kUY0UgD2jRieGtwN8cTRbqjxTA2+uHIg
      qqaU4VWepck7v9SokGQiQFXN8HC2dxRpRC0HAh9cjrD+plFtYLisszrWTt5g6Hhb+zqpS5m9+GFR25qaAQEE
      RitRZZm3Unz1WTj28QvTIR3TjYK2haBao7UiNVoEBBUdSIQOxNBzLp2g7avTxI4zW6X5xZ9Vp+sR/
      HkjUdUGEQ1W9RiED3lXR4drIBeP4pYwfv5uUwC89uq/hJ/
      78pJlfJvggg71SriEGA7E0HMunaDtq9PEjjNbpfnFn1Wn6xH8eSNR1QYRDVb0QtKa6ZwAAAIAAAACABAAAg
      KSZXyb4IIO9ELSmumcAAACAAAAAgAUAAIAAAA==

- Case: PSBT with invalid non-witness utxo typed key
    - Bytes in Hex:
      70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000
    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      HxqM+GzFUjUgcgmwBW9MBNarULNZ3kNq2bSrSQ7AiBKHO0mBMZzW2OT5bQWkd14sA8MWUL7n3UY
      AoDw+gIAAAAAF6kUD7lGNCFpa4LIM68kHHjBfdveSTSH0PIKJwEAAAAXqRQpynT4oI+BmZQoGFyXtdhS
      YYdlAAAAAQfaAEcwRAIgdAGK1BgAl7hzMjwAFXILNoTMgSOJEEjn282bVa1nnJkCIHPTabdA4+tT3O+jOCI
      Oa4KYJdHrRma3dY0+mEIVZ1sXNObTCGD8auW4H8hAtq2H/
      SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXUq4AAQEgAMLrCwAAAAAXqRS39fr0Dj1ApaRZsds1NfK
      n2ddJfjsgKJAwEI2gQARzBEAiBi63pVYQenxz9FrEq1od3fb3B1+xJ1lpp/
      OD7/94S8sgIgDAXbt0cNvy8IVX3TVscyXB7TCRPpls04QJRdsSIo2l8BRzBEAiBl9FulmYtZon/
      +GnvtAWrx8fkNVLOqj3RQql9WolEDvQIgf3JHA60e25ZoCyhLVtT/
      y4j3+3Weq74IqjDym4UTg9IBR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PW
      +57VDl6GFRkmhgIh8Oc1KuACICA6mkw39ZltOqJdusa1cK8GUDlEkpQkYLNUdT7Z7spYdxENkMak8AAAC
      1WhNq0CxoSxg4tlVuXxtrNCgqlLa1AFEJYQ2QxqTwAAAIAAAACABQAAgAA=

- Case: PSBT with invalid final scriptsig typed key
    - Bytes in Hex:
      70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000
    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      Oa4KYJdHrRma3dY0+mEIVZ1sXNObTCGD8auW4H8hAtq2H/
      SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXUq4AAQEgAMLrCwAAAAAXqRS39fr0Dj1ApaRZsds1NfK
      n2ddJfjsgKJAwEI2gQARzBEAiBi63pVYQenxz9FrEq1od3fb3B1+xJ1lpp/
      OD7/94S8sgIgDAXbt0cNvy8IVX3TVscyXB7TCRPpls04QJRdsSIo2l8BRzBEAiBl9FulmYtZon/
      +GnvtAWrx8fkNVLOqj3RQql9WolEDvQIgf3JHA60e25ZoCyhLVtT/
      y4j3+3Weq74IqjDym4UTg9IBR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PW
      +57VDl6GFRkmhgIh8Oc1KuACICA6mkw39ZltOqJdusa1cK8GUDlEkpQkYLNUdT7Z7spYdxENkMak8AAAC
      1WhNq0CxoSxg4tlVuXxtrNCgqlLa1AFEJYQ2QxqTwAAAIAAAACABQAAgAA=

- Case: PSBT with invalid final script witness typed key
    - Bytes in Hex:
      70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000

    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP/////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      EsYMStbTlTa5Nui+/
      71NtdSrgABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohwEHIyIAIIwjUxc3Q7WV37Sge3K6j
      OD7/94S8sgIgDAXbt0cNvy8IVX3TVscyXB7TCRPpls04QJRdsSIo2l8BRzBEAiBl9FulmYtZon/
      +GnvtAWrx8fkNVLOqj3RQql9WolEDvQIgf3JHA60e25ZoCyhLVtT/
      y4j3+3Weq74IqjDym4UTg9IBR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PW
      +57VDl6GFRkmhgIh8Oc1KuACICA6mkw39ZltOqJdusa1cK8GUDlEkpQkYLNUdT7Z7spYdxENkMak8AAAC
      1WhNq0CxoSxg4tlVuXxtrNCgqlLa1AFEJYQ2QxqTwAAAIAAAACABQAAgAA=

- Case: PSBT with invalid pubkey in output BIP 32 derivation paths typed key
    - Bytes in Hex:
      70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000
    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP/////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      EsYMStbTlTa5Nui+/
      71NtdSrgABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohwEHIyIAIIwjUxc3Q7WV37Sge3K6j
      3hLyyAiAMBdu3Rw2/
      LwhVfdNWxzJcHtMJE+mWzThAlF2xIijaXwFHMEQCIGX0W6WZi1mif/
      4ae+0BavHx+Q1Us6qPdFCqX1aiUQO9AiB/ckcDrR7blmgLKEtW1P/
      LiPf7dZ6rvgiqMPKbhROD0gFHUiEDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN50bmQMLAtwhAjrdkE89bc

- Case: PSBT with invalid input sighash type typed key
    - Bytes in Hex:
      70736274ff0100730200000001301ae986e516a1ec8ac5b4bc6573d32f83b465e23ad76167d68b38e730b4dbdb000000000
    - Base64 String:
      cHNidP8BAHMCAAAAATAa6YblFqHsisW0vGVz0y+DtGXiOtdhZ9aLOOcwtNvbAAAAAAD/////
      AnR7AQAAAAAAF6kUA6oXrogrXQ1Usl1jEE5P/s57nqKHYEOZOwAAAAAXqRS5IbG6b3IuS/
      qDtlV6MTmYakLsg4cAAAAAAAEBHwDKmjsAAAAFgAU0tlLZK4IWH7vyO6xh8YB6Tn5A3wCAwABAAA
      zTdgjlhAxawkM0qO3R8sAAQAiACCHa62DLx0WgBXtQSMqnqZaGBXZ7xPA74dZ9ktbKyeKZQEBJVEhA7fO

- Case: PSBT with invalid output redeemScript typed key
    - Bytes in Hex:
      70736274ff0100730200000001301ae986e516a1ec8ac5b4bc6573d32f83b465e23ad76167d68b38e730b4dbdb000000000
    - Base64 String:
      cHNidP8BAHMCAAAAATAa6YblFqHsisW0vGVz0y+DtGXiOtdhZ9aLOOcwtNvbAAAAAAD/////
      AnR7AQAAAAAAF6kUA6oXrogrXQ1Usl1jEE5P/s57nqKHYEOZOwAAAAAXqRS5IbG6b3IuS/
      qDtlV6MTmYakLsg4cAAAAAAAEBHwDKmjsAAAAFgAU0tlLZK4IWH7vyO6xh8YB6Tn5A3wAAgAAFgA
      zTdgjlhAxawkM0qO3R8sAAQAiACCHa62DLx0WgBXtQSMqnqZaGBXZ7xPA74dZ9ktbKyeKZQEBJVEhA7fO

- Case: PSBT with invalid output witnessScript typed key
    - Bytes in Hex:
      70736274ff0100730200000001301ae986e516a1ec8ac5b4bc6573d32f83b465e23ad76167d68b38e730b4dbdb000000000

    - Base64 String:
      cHNidP8BAHMCAAAAATAa6YblFqHsisW0vGVz0y+DtGXiOtdhZ9aLOOcwtNvbAAAAAAD/////
      AnR7AQAAAAAAF6kUA6oXrogrXQ1Usl1jEE5P/s57nqKHYEOZOwAAAAAXqRS5IbG6b3IuS/
      qDtlV6MTmYakLsg4cAAAAAAAEBHwDKmjsAAAAAFgAU0tlLZK4IWH7vyO6xh8YB6Tn5A3wAAQAWABI
      NN2COWEDFrCQzSo7dHywABACIAIIdrrYMvHRaAFe1BIyqeploYFdnvE8Dvh1n2S1srJ4plIQEAJVEhA7fOI6

- Case: PSBT with unsigned tx serialized with witness serialization format
    - Bytes in Hex:
      70736274ff01007802000000000101268171371edff285e937adeea4b37b78000c0566cbb3ad64641713ca42171bf6000000
    - Base64 String:
      cHNidP8BAHgCAAAAAAEBJoFxNx7f8oXpN63upLN7eAAMBWbLs61kZBcTykIXG/
      YAAAAAAP7///8C09/1BQAAAAAZdqkU0MWZA8W6woaHYOkP1SGkZlqnZSCIrADh9QUAAAAAF6kUN
      hTPMti+VZ+UBAAAAFxYAFL4Y0VKpsBIDna89p95PUzSe7LmF/////
      4b4qkOnHf8USIk6UwpyN+9rRgi7st0tAXHmOuxqSJC0AQAAABcWABT+Pp7xp0XpdNkCxDVZQ6vLNL1TU/
      8CAMLrCwAAAAAZdqkUhc/xCX/Z4Ai7NK9wnGIZeziXikiIrHL+
      +E4sAAAAF6kUM5cluiHv1irHU6m80GfWx6ajnQWHAkcwRAIgJxK+IuAnDzlPVoMR3HyppolwuAJf3TskAin
      jnF6efkE/IQUCSDBFAiEA0SuFLYXc2WHS9fSrZgZU327tzHlMDDPOXMMJ/
      7X85Y0CIGczio4OFyXBl/
      saiK9Z9R5E5CVbIBZ8hoQDHAXR8lkqASECI7cr7vCWXRC+B3jv7NYfysb3mk6haTkzgHNEZPhPKrMAAAAA

- Case: PSBT with an invalid value data due to its size being not the stated size
    - Bytes in Hex:
      70736274ff0100337401ff0700010000000100ff01000a73317428ff0000000001ff0103010000010000000000000000760100
    - Base64 String: cHNidP8BADN0Af8HAAEAAAABAP8BAApzMXQo/wAAAAAB/
      wEDAQAAAQAAAAAAAAAAdgEAAABBAAkAAAAAAA==

The following are valid PSBTs:

- Case: PSBT with one P2PKH input. Outputs are empty
    - Bytes in Hex:
      70736274ff0100750200000001268171371edff285e937adeea4b37b78000c0566cbb3ad64641713ca42171bf60000000000
    - Base64 String: cHNidP8BAHUCAAAAASaBcTce3/
      KF6Tet7qSze3gADAVmy7OtZGQXE8pCFxv2AAAAAAD+////
      AtPf9QUAAAAAGXapFNDFmQPFusKGh2DpD9UhpGZap2UgiKwA4fUFAAAAABepFDVF5uM7gyxHBQ8k0
      hTPMti+VZ+UBAAAAFxYAFL4Y0VKpsBIDna89p95PUzSe7LmF/////
      4b4qkOnHf8USIk6UwpyN+9rRgi7st0tAXHmOuxqSJC0AQAAABcWABT+Pp7xp0XpdNkCxDVZQ6vLNL1TU/
      8CAMLrCwAAAAAZdqkUhc/xCX/Z4Ai7NK9wnGIZeziXikiIrHL+
      +E4sAAAAF6kUM5cluiHv1irHU6m80GfWx6ajnQWHAkcwRAIgJxK+IuAnDzlPVoMR3HyppolwuAJf3TskAin
      jnF6efkE/IQUCSDBFAiEA0SuFLYXc2WHS9fSrZgZU327tzHlMDDPOXMMJ/
      7X85Y0CIGczio4OFyXBl/
      saiK9Z9R5E5CVbIBZ8hoQDHAXR8lkqASECI7cr7vCWXRC+B3jv7NYfysb3mk6haTkzgHNEZPhPKrMAAAAA

- Case: PSBT with one P2PKH input and one P2SH-P2WPKH input. First input is signed and finalized. Outputs are empty
    - Bytes in Hex:
      70736274ff0100a00200000002ab0949a08c5af7c49b8212f417e2f15ab3f5c33dcf153821a8139f877a5b7be40000000000f
    - Base64 String:
      cHNidP8BAKACAAAAAqsJSaCMWvfEm4IS9Bfi8Vqz9cM9zxU4IagTn4d6W3vkAAAAAAD+////

qwlJoIxa98SbghL0F+LxWrP1wz3PFTghqBOfh3pbe+QBAAAAAP7///8CYDvqCwAAAAAZdqkUdopAu9dAy
p+gOcykWXiKwAAAAAAAEHakcwRAIgR1lmF5fAGwNrJZKJSGhiGDR9iYZLcZ4ff89X0eURZYcCIFMJ6r9W
REf3xM286KdqGbX+EhtdVRs7tr5MZASEDXNxh/
HupccC1AaZGoqg7ECy0OIEhfKaC3Ibi1z+ogpIAAQEgAOH1BQAAAAAXqRQ1RebjO4MsRwUPJNPuuTycA5

- Case: PSBT with one P2PKH input which has a non-final scriptSig and has a sighash type specified. Outputs are empty
  - Bytes in Hex:
    70736274ff01007502000000012681713 71edff285e937adeea4b37b78000c0566cbb3ad64641713ca42171bf6000000000
  - Base64 String: cHNidP8BAHUCAAAAASaBcTce3/
    KF6Tet7qSze3gADAVmy7OtZGQXE8pCFxv2AAAAAAD+////
    AtPf9QUAAAAAGXapFNDFmQPFusKGh2DpD9UhpGZap2UgiKwA4fUFAAAAABepFDVF5uM7gyxHBQ8k0
    hTPMti+VZ+UBAAAAFxYAFL4Y0VKpsBIDna89p95PUzSe7LmF/////
    4b4qkOnHf8USIk6UwpyN+9rRgi7st0tAXHmOuxqSJC0AQAAABcWABT+Pp7xp0XpdNkCxDVZQ6vLNL1TU/
    8CAMLrCwAAAAAZdqkUhc/xCX/Z4Ai7NK9wnGIZeziXikiIrHL+
    +E4sAAAAF6kUM5cluiHv1irHU6m80GfWx6ajnQWHAkcwRAIgJxK+IuAnDzlPVoMR3HyppolwuAJf3TskAin
    jnF6efkE/IQUCSDBFAiEA0SuFLYXc2WHS9fSrZgZU327tzHlMDDPOXMMJ/
    7X85Y0CIGczio4OFyXBl/
    saiK9Z9R5E5CVbIBZ8hoQDHAXR8lkqASECI7cr7vCWXRC+B3jv7NYfysb3mk6haTkzgHNEZPhPKrMAAAAA

- Case: PSBT with one P2PKH input and one P2SH-P2WPKH input both with non-final scriptSigs. P2SH-P2WPKH input's redeemScript is available. Outputs filled.
  - Bytes in Hex:
    70736274ff0100a00200000002ab0949a08c5af7c49b8212f417e2f15ab3f5c33dcf153821a8139f877a5b7be40000000000f
  - Base64 String:
    cHNidP8BAKACAAAAAqsJSaCMWvfEm4IS9Bfi8Vqz9cM9zxU4IagTn4d6W3vkAAAAAAD+////
    qwlJoIxa98SbghL0F+LxWrP1wz3PFTghqBOfh3pbe+QBAAAAAP7///8CYDvqCwAAAAAZdqkUdopAu9dAy
    p+gOcykWXiKwAAAAAAAEA3wIAAAABJoFxNx7f8oXpN63upLN7eAAMBWbLs61kZBcTykIXG/
    YAAAAAakcwRAIgcLIkUSPmv0dNYMW1DAQ9TGkaXSQ18Jo0p2YqncJReQoCIAEynKnazygL3zB0DsA5BCJ
    kzjH+////
    AtPf9QUAAAAAGXapFNDFmQPFusKGh2DpD9UhpGZap2UgiKwA4fUFAAAAABepFDVF5uM7gyxHBQ8k0

- Case: PSBT with one P2SH-P2WSH input of a 2-of-2 multisig, redeemScript, witnessScript, and keypaths are available. Contains one signature.
  - Bytes in Hex:
    70736274ff0100550200000001279a2323a5dfb51fc45f220fa58b0fc13e1e3342792a85d7e36cd6333b5cbc390000000000f
  - Base64 String:
    cHNidP8BAFUCAAAAASeaIyOl37UfxF8iD6WLD8E+HjNCeSqF1+Ns1jM7XLw5AAAAAAD/////
    AaBa6gsAAAAAGXapFP/
    pwAYQl8w7Y28ssEYPpPxCfStFiKwAAAAAAAEBIJVe6gsAAAAF6kUY0UgD2jRieGtwN8cTRbqjxTA2+uHIg
    qqaU4VWepck7v9SokGQiQFXN8HC2dxRpRC0HAh9cjrD+plFtYLisszrWTt5g6Hhb+zqpS5m9+GFR25qaAQEE
    RitRZZm3Unz1WTj28QvTIR3TjYK2haBao7UiNVoEBBUdSIQOxNBzLp2g7avTxI4zW6X5xZ9Vp+sR/
    HkjUdUGEQ1W9RiED3lXR4drIBeP4pYwfv5uUwC89uq/hJ/

78pJlfJvggg71SriIGA7E0HMunaDtq9PEjjNbpfnFn1Wn6xH8eSNR1QYRDVb1GELSmumcAAACAAAAAgAQA
iljB+/m5TALz26r+En/vykmV8m+CCDvRC0prpnAAAAgAAAAIAFAACAAAA=

- Case: PSBT with one P2WSH input of a 2-of-2 multisig. witnessScript, keypaths, and global xpubs are available. Contains no signatures. Outputs filled.
  - Bytes in Hex:
    70736274ff01005202000000019dfc6628c26c5899fe1bd3dc338665bfd55d7ada10f6220973df2d386dec1276010000000
  - Base64 String: cHNidP8BAFICAAAAAZ38ZijCbFiZ/hvT3DOGZb/
    VXXraEPYiCXPfLTht7BJ2AQAAAAD/////
    AfA9zR0AAAAAFgAUezoAv9wU0neVwrdJAdCdpu8TNXkAAAAATwEENYfPAto/
    0AiAAAAAlwSLGtBEWx7IJ1UXcnyHtOTrwYogP/
    oPlMAVZr046QADUbdDiH7h1A3DKmBDck8tZFmztaTXPa7I+64EcvO8Q+IM2QxqT64AAIAAAACATwEENY
    0AiAAAABuQRSQnE5zXjCz/
    JES+NTzVhgXj5RMoXlKLQH+uP2FzUD0wpel8itvFV9rCrZp+OcFyLrrGnmaLbyZnzB1nHIPKsM2QxqT64AAI
    v5ckcOrlHPY5DtTODarRWKZyIcSEDNys0I07Xz5wf6l0F1EFVeSe+lUKxYusC4ass6AIkwAtSriIGAp2hLNtbI1aS
    lyRw6uUc9jkO1M4NqtFYpnIhxENkMak+uAACAAAAAgAAAAAAiBgM3KzQjTtfPnB/
    qXQXUQVV5J76VQrFi6wLhqyzoAiTACxDZDGpPrgAAgAEAAIAAAAAAAACICA57/
    H1R6HV+S36K6evaslxpL0DukpzSwMVaiVritOh75EO3kXMUAAACAAAAAgAEAAIAA

- Case: PSBT with unknown types in the inputs.
  - Bytes in Hex:
    70736274ff01003f0200000001ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff00000000000fffffff0100000000000000000
  - Base64 String:
    cHNidP8BAD8CAAAAAf//
    AAAAAAD/////
    AQAAAAAAAAAAA2oBAAAAAAAACvABAgMEBQYHCAkPAQIDBAUGBwgJCgsMDQ4PAAA=

- Case: PSBT with `PSBT_GLOBAL_XPUB`.
  - Bytes in Hex:
    70736274ff01009d0100000002710ea76ab45c5cb6438e607e59cc037626981805ae9e0dfd9089012abb0be535010000000
  - Base64 String: cHNidP8BAJ0BAAAAAnEOp2q0XFy2Q45gflnMA3YmmBgFrp4N/
    ZCJASq7C+U1AQAAAAD/////
    GQmU1qizyMgsy8+y+6QQaqBmObhyqNRHRlwNQliNbWcAAAAAAP////
    8CAOH1BQAAAAAZdqkUtrwsDuVlWoQ9ea/
    t0MzD991kNAmIrGBa9AUAAAAAFgAUEYjvjkzgRJ6qyPsUHL9aEXbmoIgAAAAATwEEiLIeA55TDKyAAAA
    7iAjtCQOSQ+OtXBgnVpxQMQAAgAAAAIAAAACAAAAAAAEAAAAAAQEfAOH1BQAAAAAWABQ4j7lE
    S4QsRAYJ1acUDEAAIAAAACAAAAAgAAAAAAAAAAAAAAiAgLSDKUC7iiWhtIYFb1DqAY3sGmOH7zb5

- Case: PSBT with global unsigned tx that has 0 inputs and 0 outputs
  - Bytes in Hex: 70736274ff01000a00000000000000000000
  - Base64 String: cHNidP8BAAoAAAAAAAAAAAAA==

- Case: PSBT with 0 inputs
  - Bytes in Hex:
    70736274ff01004c020000000002d3dff505000000001976a914d0c59903c5bac2868760e90fd521a4665aa7652088ac00e1
  - Base64 String: cHNidP8BAEwCAAAAALT3/UFAAAAABl2qRTQxZkDxbrChodg6Q/
    VIaRmWqdlIIisAOH1BQAAAAXqRQ1RebjO4MsRwUPJNPuuTycA5SLx4ezLhMAAAAA

Fails Signer checks

- Case: A Witness UTXO is provided for a non-witness input
  - Bytes in Hex:
    70736274ff0100a00200000002ab0949a08c5af7c49b8212f417e2f15ab3f5c33dcf153821a8139f877a5b7be40000000000f
  - Base64 String:
    cHNidP8BAKACAAAAAqsJSaCMWvfEm4IS9Bfi8Vqz9cM9zxU4IagTn4d6W3vkAAAAAAD+////
    qwlJoIxa98SbghL0F+LxWrP1wz3PFTghqBOfh3pbe+QBAAAAAP7///8CYDvqCwAAAAZdqkUdopAu9dAy
    p+gOcykWXiKwAAAAAAAEBItPf9QUAAAAAGXapFNSO0xELlAFMsRS9Mtb00GbcdCVriKwAAQEgAOH1

- Case: redeemScript with non-witness UTXO does not match the scriptPubKey
  - Bytes in Hex:
    70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000
  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP/////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
    vU210gwRQIhAPYQOLMI3B2oZaNIUnRvAVdyk0IIxtJEVDk82ZvfIhd3AiAFbmdaZ1ptCgK4WxTl4pB02KJam1
    Oa4KYJdHrRma3dY0+mEIVZ1sXNObTCGD8auW4H8hAtq2H/
    SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXUq8iBgKVg785rgpgl0etGZrd1jT6YQhVnWxc05tMIYPxq5bg
    EsYMStbTlTa5Nui+/
    71NtcQ2QxqTwAAAIAAAACAAQAAgAABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohyIC
    +GnvtAWrx8fkNVLOqj3RQql9WolEDvQIgf3JHA60e25ZoCyhLVtT/
    y4j3+3Weq74IqjDym4UTg9IBAQMEAQAAAAEEIgAgjCNTFzdDtZXftKB7crqOQuN5fadOh/
    59nXSX47ICiQMBBUdSIQMIncEMesbbVPkTKa9hczPbOIzq0MIx9yM3nRuZAwsC3CECOt2QTz1tz1nduQaw3u
    ue1Q5ehhUZJoYCIfDnNSriIGAjrdkE89bc9Z3bkGsN7iNSm3/7ntUOXoYVGSaGAiHw5zENkMak8AAACAAA
    1WhNq0CxoSxg4tlVuXxtrNCgqlLa1AFEJYQ2QxqTwAAAIAAAACABQAAgAA=

- Case: redeemScript with witness UTXO does not match the scriptPubKey
  - Bytes in Hex:
    70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000
  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP/////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
    vU210gwRQIhAPYQOLMI3B2oZaNIUnRvAVdyk0IIxtJEVDk82ZvfIhd3AiAFbmdaZ1ptCgK4WxTl4pB02KJam1
    Oa4KYJdHrRma3dY0+mEIVZ1sXNObTCGD8auW4H8hAtq2H/
    SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXUq4iBgKVg785rgpgl0etGZrd1jT6YQhVnWxc05tMIYPxq5bg
    EsYMStbTlTa5Nui+/
    71NtcQ2QxqTwAAAIAAAACAAQAAgAABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohyIC
    +GnvtAWrx8fkNVLOqj3RQql9WolEDvQIgf3JHA60e25ZoCyhLVtT/
    y4j3+3Weq74IqjDym4UTg9IBAQMEAQAAAAEEIgAgjCNTFzdDtZXftKB7crqOQuN5fadOh/
    59nXSX47ICiQABBUdSIQMIncEMesbbVPkTKa9hczPbOIzq0MIx9yM3nRuZAwsC3CECOt2QTz1tz1nduQaw3u

ue1Q5ehhUZJoYCIfDnNSriIGAjrdkE89bc9Z3bkGsN7iNSm3/7ntUOXoYVGSaGAiHw5zENkMak8AAACAAA
1WhNq0CxoSxg4tlVuXxtrNCgqlLa1AFEJYQ2QxqTwAAAIAAAACABQAAgAA=

- Case: witnessScript with witness UTXO does not match the redeemScript
  - Bytes in Hex:
    70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000
  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
    vU210gwRQIhAPYQOLMI3B2oZaNIUnRvAVdyk0IIxtJEVDk82ZvfIhd3AiAFbmdaZ1ptCgK4WxTl4pB02KJam1
    Oa4KYJdHrRma3dY0+mEIVZ1sXNObTCGD8auW4H8hAtq2H/
    SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXUq4iBgKVg785rgpgl0etGZrd1jT6YQhVnWxc05tMIYPxq5bg
    EsYMStbTlTa5Nui+/
    71NtcQ2QxqTwAAAIAAAACAAQAAgAABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohyIC
    +GnvtAWrx8fkNVLOqj3RQql9WolEDvQIgf3JHA60e25ZoCyhLVtT/
    y4j3+3Weq74IqjDym4UTg9IBAQMEAQAAAAEEIgAgjCNTFzdDtZXftKB7crqOQuN5fadOh/
    59nXSX47ICiQMBBUdSIQMIncEMesbbVPkTKa9hczPbOIzq0MIx9yM3nRuZAwsC3CECOt2QTz1tz1nduQaw3u
    ue1Q5ehhUZJoYCIfDnNSrSIGAjrdkE89bc9Z3bkGsN7iNSm3/7ntUOXoYVGSaGAiHw5zENkMak8AAACAAA
    1WhNq0CxoSxg4tlVuXxtrNCgqlLa1AFEJYQ2QxqTwAAAIAAAACABQAAgAA=

The private keys in the tests below are derived from the following master private key:

- Extended Private Key:
  tprv8ZgxMBicQKsPd9TeAdPADNnSyH9SSUUbTVeFszDE23Ki6TBB5nCefAdHkK8Fm3qMQR6sHwA56zqRmKmxnH

  - Seed: cUkG8i1RFfWGWy5ziR11zJ5V4U4W3viSFCfyJmZnvQaUsd1xuF3T

A creator creating a PSBT for a transaction which creates the following outputs:

- scriptPubKey: `0014d85c2b71d0060b09c9886aeb815e50991dda124d`, Amount: `1.49990000`
- scriptPubKey: `001400aea9a2e5f0f876a588df5546e8742d1d87008f`, Amount: `1.00000000`

and spends the following inputs:

- TXID: `75ddabb27b8845f5247975c8a5ba7c6f336c4570708ebe230caf6db5217ae858`, Index: `0`
- TXID: `1dea7cd05979072a3578cab271c02244ea8a090bbb46aa680a65ecd027048d83`, Index: `1`

must create this PSBT:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff

  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu

Given the above PSBT, an updater with only the following:

- Redeem Scripts:
  - `5221029583bf39ae0a609747ad199addd634fa6108559d6c5cd39b4c2183f1ab96e07f2102dab61ff49a14db6a7d02b0`
  - `00208c2353173743b595dfb4a07b72ba8e42e3797da74e87fe7d9d7497e3b2028903`
- Witness Scripts:
  - `522103089dc10c7ac6db54f91329af617333db388cead0c231f723379d1b99030b02dc21023add904f3d6dcf59ddb906`
- Previous Transactions:
  - `0200000000010158e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd75010000`
    `00171600145f275f436b09a8cc9a2eb2a2f528485c68a56323feffffff02d8231f1b0100000017a914ae`
    `d962d6654f9a2b36608eb9d64d2b260db4f1118700c2eb0b0000000017a914b7f5faf40e3d40a5a459b1`
    `db3535f2b72fa921e88702483045022100a22edcc6e5bc511af4cc4ae0de0fcd75c7e04d8c1c3a8aa9d8`
    `20ed4b967384ec02200642963597b9b1bc22c75e9f3e117284a962188bf5e8a74c895089046a20ad7701`
    `21035509a48eb623e10aace8bfd0212fdb8a8e5af3c94b0b133b95e114cab89e4f7965000000`
  - `0200000001aad73931018bd25f84ae400b68848be09db706eac2ac18298babee71ab656f8b0000000048`
    `473044022058f6fc7c6a33e1b31548d481c826c015bd30135aad42cd67790dab66d2ad243b02204a1ced`
    `2604c6735b6393e5b41691dd78b00f0c5942fb9f751856faa938157dba01feffffff0280f0fa02000000`
    `0017a9140fb9463421696b82c833af241c78c17ddbde493487d0f20a270100000017a91429ca74f8a08f`
    `81999428185c97b5d852e4063f618765000000`
- Public Keys
  - Key: `029583bf39ae0a609747ad199addd634fa6108559d6c5cd39b4c2183f1ab96e07f`, Derivation Path: `m/0'/0'/0'`
  - Key: `02dab61ff49a14db6a7d02b0cd1fbb78fc4b18312b5b4e54dae4dba2fbfef536d7`, Derivation Path: `m/0'/0'/1'`
  - Key: `03089dc10c7ac6db54f91329af617333db388cead0c231f723379d1b99030b02dc`, Derivation Path: `m/0'/0'/2'`
  - Key: `023add904f3d6dcf59ddb906b0dee23529b7ffb9ed50e5e86151926860221f0e73`, Derivation Path: `m/0'/0'/3'`
  - Key: `03a9a4c37f5996d3aa25dbac6b570af0650394492942460b354753ed9eeca58771`, Derivation Path: `m/0'/0'/4'`
  - Key: `027f6399757d2eff55a136ad02c684b1838b6556e5f1b6b34282a94b6b50051096`, Derivation Path: `m/0'/0'/5'`

Must create this PSBT:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff
  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    EsYMStbTlTa5Nui+/71NtdSriIGApWDvzmuCmCXR60Zmt3WNPphCFWdbFzTm0whg/GrluB/
    ENkMak8AAACAAAAAgAAAAIAiBgLath/0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
    vU21xDZDGpPAAAAgAAAAIABAACAAAEBIADC6wsAAAAAF6kUt/
    X69A49QKWkWbHbNTXyty+pIeiHAQQiACCMI1MXN0O1ld+0oHtyuo5C43l9p06H/
    n2ddJfjsgKJAwEFR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PWd25BrDe4jU
    +57VDl6GFRkmhgIh8Oc1KuIgYCOt2QTz1tz1nduQaw3uI1Kbf/

ue1Q5ehhUZJoYCIfDnMQ2QxqTwAAAIAAAACAAwAAgCIGAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG
Y5l1fS7/VaE2rQLGhLGDi2VW5fG2s0KCqUtrUAUQlhDZDGpPAAAAgAAAAIAFAACAAA==

An updater which adds SIGHASH_ALL to the above PSBT must create this PSBT:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff
  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    Oa4KYJdHrRma3dY0+mEIVZ1sXNObTCGD8auW4H8hAtq2H/
    SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXUq4iBgKVg785rgpgl0etGZrd1jT6YQhVnWxc05tMIYPxq5bg
    EsYMStbTlTa5Nui+/
    71NtcQ2QxqTwAAAIAAAACAAQAAgAABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohwE
    +57VDl6GFRkmhgIh8OcxDZDGpPAAAAgAAAAIADAACAIgYDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN
    WZbTqiXbrGtXCvBlA5RJKUJGCzVHU+2e7KWHcRDZDGpPAAAAgAAAAIAEAACAACICAn9jmXV9Lv9Vo

Given the above updated PSBT, a signer that supports SIGHASH_ALL for P2PKH and P2WPKH spends and uses RFC6979 for nonce generation and has the following keys:

- `cP53pDbR5WtAD8dYAW9hhTjuvvTVaEiQBdrz9XPrgLBeRFiyCbQr` (`m/0'/0'/0'`)
- `cR6SXDoyfQrcp4piaiHE97Rsgta9mNhGTen9XeonVgwsh4iSgw6d` (`m/0'/0'/2'`)

must create this PSBT:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff
  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    EsYMStbTlTa5Nui+/71NtdSriIGApWDvzmuCmCXR60Zmt3WNPphCFWdbFzTm0whg/GrluB/
    ENkMak8AAACAAAAAgAAAAIAiBgLath/0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
    vU21xDZDGpPAAAAgAAAAIABAACAAAEBIADC6wsAAAAAF6kUt/
    X69A49QKWkWbHbNTXyty+pIeiHIgIDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN50bmQMLAtxHMEQCIG
    3hLyyAiAMBdu3Rw2/
    LwhVfdNWxzJcHtMJE+mWzThAlF2xIijaXwEBAwQBAAAAAQQiACCMI1MXN0O1ld+0oHtyuo5C43l9p06H/
    n2ddJfjsgKJAwEFR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PWd25BrDe4jU
    +57VDl6GFRkmhgIh8Oc1KuIgYCOt2QTz1tz1nduQaw3uI1Kbf/
    ue1Q5ehhUZJoYCIfDnMQ2QxqTwAAAIAAAACAAwAAgCIGAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG
    Y5l1fS7/VaE2rQLGhLGDi2VW5fG2s0KCqUtrUAUQlhDZDGpPAAAAgAAAAIAFAACAAA==

Given the above updated PSBT, a signer with the following keys:

- `cT7J9YpCwY3AVRFSjN6ukeEeWY6mhpbJPxRaDaP5QTdygQRxP9Au` (`m/0'/0'/1'`)
- `cNBc3SWUip9PPm1GjRoLEJT6T41iNzCYtD7qro84FMnM5zEqeJsE` (`m/0'/0'/3'`)

must create this PSBT:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff

  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP/////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
    vU210gwRQIhAPYQOLMI3B2oZaNIUnRvAVdyk0IIxtJEVDk82ZvfIhd3AiAFbmdaZ1ptCgK4WxTl4pB02KJam1
    Oa4KYJdHrRma3dY0+mEIVZ1sXNObTCGD8auW4H8hAtq2H/
    SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXUq4iBgKVg785rgpgl0etGZrd1jT6YQhVnWxc05tMIYPxq5bg
    EsYMStbTlTa5Nui+/
    71NtcQ2QxqTwAAAIAAAACAAQAAgAABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohyIC
    +GnvtAWrx8fkNVLOqj3RQql9WolEDvQIgf3JHA60e25ZoCyhLVtT/
    y4j3+3Weq74IqjDym4UTg9IBAQMEAQAAAAEEIgAgjCNTFzdDtZXftKB7crqOQuN5fadOh/
    59nXSX47ICiQMBBUdSIQMIncEMesbbVPkTKa9hczPbOIzq0MIx9yM3nRuZAwsC3CECOt2QTz1tz1nduQaw3u
    ue1Q5ehhUZJoYCIfDnNSriIGAjrdkE89bc9Z3bkGsN7iNSm3/7ntUOXoYVGSaGAiHw5zENkMak8AAACAAA
    1WhNq0CxoSxg4tlVuXxtrNCgqlLa1AFEJYQ2QxqTwAAAIAAAACABQAAgAA=

Given both of the above PSBTs, a combiner must create this PSBT. A combiner must produce the given result for any order of its inputs, unless there are conflicting fields. The current combiner test vectors do not have conflicting fields:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff

  - Base64 String:
    cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
    g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP/////
    8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
    SaFNtqfQKwzR+7ePxLGDErW05U2uTbovv+9TbXSDBFAiEA9hA4swjcHahlo0hSdG8BV3KTQgjG0kRUOTzZm
    EsYMStbTlTa5Nui+/71NtdSriIGApWDvzmuCmCXR60Zmt3WNPphCFWdbFzTm0whg/GrluB/
    ENkMak8AAACAAAAAgAAAAIAiBgLath/0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
    vU21xDZDGpPAAAAgAAAAIABAACAAAEBIADC6wsAAAAAF6kUt/
    X69A49QKWkWbHbNTXyty+pIeiHIgIDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN50bmQMLAtxHMEQCIG
    3hLyyAiAMBdu3Rw2/
    LwhVfdNWxzJcHtMJE+mWzThAlF2xIijaXwEiAgI63ZBPPW3PWd25BrDe4jUpt/
    +57VDl6GFRkmhgIh8Oc0cwRAIgZfRbpZmLWaJ//
    hp77QFq8fH5DVSzqo90UKpfVqJRA70CIH9yRwOtHtuWaAsoS1bU/8uI9/
    t1nqu+CKow8puFE4PSAQEDBAEAAAABBCIAIIwjUxc3Q7WV37Sge3K6jkLjeX2nTof+fZ10l+OyAokDAQVHU
    +57VDl6GFRkmhgIh8OcxDZDGpPAAAAgAAAAIADAACAIgYDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN
    WZbTqiXbrGtXCvBlA5RJKUJGCzVHU+2e7KWHcRDZDGpPAAAAgAAAAIAEAACAACICAn9jmXV9Lv9Vo

Given the above PSBT, an input finalizer must create this PSBT:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff8

    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      EsYMStbTlTa5Nui+/
      71NtdSrgABASAAwusLAAAAABepFLf1+vQOPUClpFmx2zU18rcvqSHohwEHIyIAIIwjUxc3Q7WV37Sge3K6j
      3hLyyAiAMBdu3Rw2/
      LwhVfdNWxzJcHtMJE+mWzThAlF2xIijaXwFHMEQCIGX0W6WZi1mif/
      4ae+0BavHx+Q1Us6qPdFCqX1aiUQO9AiB/ckcDrR7blmgLKEtW1P/
      LiPf7dZ6rvgiqMPKbhROD0gFHUiEDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN50bmQMLAtwhAjrdkE89bc
      Y5l1fS7/VaE2rQLGhLGDi2VW5fG2s0KCqUtrUAUQlhDZDGpPAAAAgAAAAIAFAACAAA==

Given the above PSBT, a transaction extractor must create this Bitcoin transaction:

- Bytes in Hex:
  0200000000010258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd7500000000da00473044022074018

Given these two PSBTs with unknown key-value pairs:

- Bytes in Hex:
  70736274ff01003f0200000001ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff00000000000ffffffff0100000000000000000036a01

    - Base64 String:
      cHNidP8BAD8CAAAAAf//
      AAAAAAD/////
      AQAAAAAAAAAAA2oBAAAAAAAK8AECAwQFBgcICQ8BAgMEBQYHCAkKCwwNDg8ACvABAgMEBQ

- Bytes in Hex:
  70736274ff01003f0200000001ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff00000000000ffffffff0100000000000000000036a01

    - Base64 String:
      cHNidP8BAD8CAAAAAf//
      AAAAAAD/////
      AQAAAAAAAAAAA2oBAAAAAAAK8AECAwQFBgcIEA8BAgMEBQYHCAkKCwwNDg8ACvABAgMEBQ

A combiner which orders keys lexicographically must produce the following PSBT:

- Bytes in Hex:
  70736274ff01003f0200000001ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff00000000000ffffffff0100000000000000000036a01

    - Base64 String:
      cHNidP8BAD8CAAAAAf//
      AAAAAAD/////
      AQAAAAAAAAAAA2oBAAAAAAAK8AECAwQFBgcICQ8BAgMEBQYHCAkKCwwNDg8K8AECAwQFBg

Given these two PSBTs:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff

    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      EsYMStbTlTa5Nui+/71NtdSriIGApWDvzmuCmCXR60Zmt3WNPphCFWdbFzTm0whg/GrluB/
      ENkMak8AAACAAAAAgAAAAIAiBgLath/0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
      vU21xDZDGpPAAAAgAAAAIABAACAAAEBIADC6wsAAAAF6kUt/
      X69A49QKWkWbHbNTXyty+pIeiHAQQiACCMI1MXN0O1ld+0oHtyuo5C43l9p06H/
      n2ddJfjsgKJAwEFR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PWd25BrDe4jU
      +57VDl6GFRkmhgIh8Oc1KuIgYCOt2QTz1tz1nduQaw3uI1Kbf/
      ue1Q5ehhUZJoYCIfDnMQ2QxqTwAAAIAAAACAAwAAgCIGAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PWd25BrDe4jUY5l1fS7/VaE2rQLGhLGDi2VW5fG2s0KCqUtrUAUQlhDZDGpPAAAAgAAAAIAFAACAAA==

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff

    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      EsYMStbTlTa5Nui+/71NtdSriIGApWDvzmuCmCXR60Zmt3WNPphCFWdbFzTm0whg/GrluB/
      ENkMak8AAACAAAAAgAAAAIAiBgLath/0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
      vU21xDZDGpPAAAAgAAAAIABAACAAAEBIADC6wsAAAAF6kUt/
      X69A49QKWkWbHbNTXyty+pIeiHIgIDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN50bmQMLAtxHMEQCIA
      oQ/
      AiBbz5DTn8kiRHdCLpwvlqyo9WaRXO+k7NTlxSqZtYiSkYEBAwSBAAAAAQQiACCMI1MXN0O1ld+0oHtyuo5C43l9p06H/
      n2ddJfjsgKJAwEFR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PWd25BrDe4jU
      +57VDl6GFRkmhgIh8Oc1KuIgYCOt2QTz1tz1nduQaw3uI1Kbf/
      ue1Q5ehhUZJoYCIfDnMQ2QxqTwAAAIAAAACAAwAAgCIGAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PWd25BrDe4jUY5l1fS7/VaE2rQLGhLGDi2VW5fG2s0KCqUtrUAUQlhDZDGpPAAAAgAAAAIAFAACAAA==

A combiner must create this PSBT:

- Bytes in Hex:
  70736274ff01009a020000000258e87a21b56daf0c23be8e7070456c336f7cbaa5c8757924f545887bb2abdd750000000000ffffffff

    - Base64 String:
      cHNidP8BAJoCAAAAAljoeiG1ba8MI76OcHBFbDNvfLqlyHV5JPVFiHuyq911AAAAAAD/////
      g40EJ9DsZQpoqka7CwmK6kQiwHGyyng1Kgd5WdB86h0BAAAAAP////
      8CcKrwCAAAAAAWABTYXCtx0AYLCcmIauuBXlCZHdoSTQDh9QUAAAAAFgAUAK6pouXw+HaliN9VRu
      EsYMStbTlTa5Nui+/71NtdSriIGApWDvzmuCmCXR60Zmt3WNPphCFWdbFzTm0whg/GrluB/
      ENkMak8AAACAAAAAgAAAAIAiBgLath/0mhTban0CsM0fu3j8SxgxK1tOVNrk26L7/
      vU21xDZDGpPAAAAgAAAAIABAACAAAEBIADC6wsAAAAF6kUt/
      X69A49QKWkWbHbNTXyty+pIeiHIgIDCJ3BDHrG21T5EymvYXMz2ziM6tDCMfcjN50bmQMLAtxHMEQCIA

oQ/
AiBbz5DTn8kiRHdCLpwvlqyo9WaRXO+k7NTlxSqZtYiSkYEBAwSBAAAAAQQiACCMI1MXN0O1ld+0oHtyu
n2ddJfjsgKJAwEFR1IhAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG5kDCwLcIQI63ZBPPW3PWd25BrDe4jU
+57VDl6GFRkmhgIh8Oc1KuIgYCOt2QTz1tz1nduQaw3uI1Kbf/
ue1Q5ehhUZJoYCIfDnMQ2QxqTwAAAIAAAACAAwAAgCIGAwidwQx6xttU+RMpr2FzM9s4jOrQwjH3IzedG
Y5l1fS7/VaE2rQLGhLGDi2VW5fG2s0KCqUtrUAUQlhDZDGpPAAAAgAAAAIAFAACAAA==

Rationale

Reference implementation

The reference implementation of the PSBT format is available at https://github.com/achow101/bitcoin/tree/psbt.

Acknowledgements

Special thanks to Pieter Wuille for suggesting that such a transaction format should be made and for coming up with the name and abbreviation of PSBT.

Thanks to Pieter Wuille, Gregory Maxwell, Jonathan Underwood, Daniel Cousens and those who commented on the bitcoin-dev mailing list for additional comments and suggestions for improving this proposal.

```
BIP: 370
Layer: Applications
Title: PSBT Version 2
Authors: Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
Assigned: 2021-01-14
License: BSD-2-Clause
```

BIP 370

Introduction

Abstract

This document proposes a second version of the Partially Signed Bitcoin Transaction format described in BIP 174 which allows for inputs and outputs to be added to the PSBT after creation.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

Partially Signed Bitcoin Transaction Version 0 as described in BIP 174 is unable to have new inputs and outputs be added to the transaction. The fixed global unsigned transaction cannot be changed which prevents any additional inputs or outputs to be added. PSBT Version 2 is intended to rectify this problem.

An additional beneficial side effect is that all information for a given input or output will be provided by its <input-map> or <output-map>. With Version 0, to retrieve all of the information for an input or output, data would need to be found in two locations: the <input-map>/<output-map> and the global unsigned transaction. PSBT Version 2 now moves all related information to one place.

Specification

PSBT Version 2 (PSBTv2) only specifies new fields and field inclusion/exclusion requirements.

PSBT_GLOBAL_UNSIGNED_TX must be excluded in PSBTv2. PSBT_GLOBAL_VERSION must be included in PSBTv2 and set to version number 2**What happened to version number 1?** Version number 1 is skipped because PSBT Version 0 has been colloquially referred to as version 1. Originally this BIP was to be version 1, but because it has been colloquially referred to as version 2 during its design phase, it was decided to change the version number to 2 so that there would not be any confusion.

The new global types for PSBT Version 2 are as follows:

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Vers Req Inclu
Transaction Version	PSBT_GLOBAL_TX_VERSION = 0x02	None	No key data	<32-bit little endian int version>	The 32-bit little endian signed integer representing the version number of the transaction being created. Note that this is not the same as the PSBT version number specified by the PSBT_GLOBAL_VERSION field.	2
Fallback Locktime	PSBT_GLOBAL_FALLBACK_LOCKTIME = 0x03	None	No key data	<32-bit little endian uint locktime>	The 32-bit little endian unsigned integer representing the transaction locktime to use if no inputs specify a required locktime.	
Input Count	PSBT_GLOBAL_INPUT_COUNT = 0x04	None	No key data	<compact size uint input count>	Compact size unsigned integer representing the number of inputs in this PSBT.	2
Output Count	PSBT_GLOBAL_OUTPUT_COUNT = 0x05	None	No key data	<compact size uint output count>	Compact size unsigned integer representing the number of outputs in this PSBT.	2
	PSBT_GLOBAL_TX_MODIFIABLE = 0x06	None	No key data	<8-bit uint flags>	An 8 bit unsigned integer as a bitfield for various	

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Vers Req Inclu
Transaction Modifiable Flags					transaction modification flags. Bit 0 is the Inputs Modifiable Flag, set to 1 to indicate whether inputs can be added or removed. Bit 1 is the Outputs Modifiable Flag, set to 1 to indicate whether outputs can be added or removed. Bit 2 is the Has SIGHASH_SINGLE flag, set to 1 to indicate whether the transaction has a SIGHASH_SINGLE signature who's input and output pairing must be preserved. Bit 2 essentially indicates that the Constructor must iterate the inputs to determine whether and how to add or remove an input.	

The new per-input types for PSBT Version 2 are defined as follows:

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	V R In
Previous TXID	`PSBT_IN_PREVIOUS_TXID = 0x0e`	None	No key data	`<32 byte txid>`	32 byte txid in standard byte order, not display byte order, of the previous transaction whose output at PSBT_IN_OUTPUT_INDEX is being spent.	2
Spent Output Index	`PSBT_IN_OUTPUT_INDEX = 0x0f`	None	No key data	`<32-bit little endian uint index>`	32 bit little endian integer representing the index of the output being spent in the transaction with the txid of PSBT_IN_PREVIOUS_TXID.	2
Sequence Number	`PSBT_IN_SEQUENCE = 0x10`	None	No key data	`<32-bit little`	The 32 bit unsigned little endian integer for the sequence number of this input. If omitted, the	

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	V R In
				endian uint sequence>	sequence number is assumed to be the final sequence number (0xffffffff).	
Required Time-based Locktime	PSBT_IN_REQUIRED_TIME_LOCKTIME = 0x11	None	No key data	<32-bit little endian uint locktime>	32 bit unsigned little endian integer greater than or equal to 500000000 representing the minimum Unix timestamp that this input requires to be set as the transaction's lock time.	
Required Height-based Locktime	PSBT_IN_REQUIRED_HEIGHT_LOCKTIME = 0x12	None	No key data	<32-bit uint locktime>	32 bit unsigned little endian integer greater than 0 and less than 500000000 representing the minimum block height that this input requires to be set as the transaction's lock time.	

The new per-output types for PSBT Version 2 are defined as follows:

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	Versions Requiring Exclusion	Versions Allowing Inclusion
Output Amount	PSBT_OUT_AMOUNT = 0x03	None	No key data	<64-bit little endian int amount>	64 bit signed little endian integer representing the output's amount in satoshis.	2	0	2
Output Script	PSBT_OUT_SCRIPT = 0x04	None	No key data	<bytes script>	The script for this output, also known as the scriptPubKey.	2	0	2

Determining Lock Time

The nLockTime field of a transaction is determined by inspecting the PSBT_GLOBAL_FALLBACK_LOCKTIME and each input's PSBT_IN_REQUIRED_TIME_LOCKTIME and PSBT_IN_REQUIRED_HEIGHT_LOCKTIME fields. If none of the inputs have a PSBT_IN_REQUIRED_TIME_LOCKTIME and PSBT_IN_REQUIRED_HEIGHT_LOCKTIME, then

PSBT_GLOBAL_FALLBACK_LOCKTIME must be used. If PSBT_GLOBAL_FALLBACK_LOCKTIME is not provided, then it is assumed to be 0.

If one or more inputs have a PSBT_IN_REQUIRED_TIME_LOCKTIME or PSBT_IN_REQUIRED_HEIGHT_LOCKTIME, then the field chosen is the one which is supported by all of the inputs. This can be determined by looking at all of the inputs which specify a locktime in either of those fields, and choosing the field which is present in all of those inputs. Inputs not specifying a lock time field can take both types of lock times, as can those that specify both. The lock time chosen is then the maximum value of the chosen type of lock time.

If a PSBT has both types of locktimes possible because one or more inputs specify both PSBT_IN_REQUIRED_TIME_LOCKTIME and PSBT_IN_REQUIRED_HEIGHT_LOCKTIME, then locktime determined by looking at the PSBT_IN_REQUIRED_HEIGHT_LOCKTIME fields of the inputs must be chosen.**Why choose the height based locktime?** In the event of a tie for the locktime type, signers need to be able to know which locktime to use as their signatures will commit to the locktime in the transaction, so choosing the wrong one will result in an invalid transaction. Height based locktime is preferred over time based as Bitcoin's unit of time is the block height, so a height makes more sense in the context of Bitcoin.

Unique Identification

PSBTv2s can be uniquely identified by constructing an unsigned transaction given the information provided in the PSBT, and computing the transaction ID of that transaction. Since PSBT_IN_SEQUENCE can be changed by Updaters and Combiners, the sequence number in this unsigned transaction must be set to 0 (not final, nor the sequence in PSBT_IN_SEQUENCE). The lock time in this unsigned transaction must be computed as described previously.

Roles

PSBTv2 introduces new roles and modifies some existing roles.

Creator

In PSBTv2, the Creator initializes the PSBT with 0 inputs and 0 outputs. The PSBT version number is set to 2. The Creator should also set PSBT_GLOBAL_FALLBACK_LOCKTIME. If the Creator is not also a Constructor and will be giving the PSBT to others to add inputs and outputs, the PSBT_GLOBAL_TX_MODIFIABLE field must be present and the Inputs Modifiable and Outputs Modifiable flags set appropriately; moreover, the transaction version number must be set to at least 2. **Why does the transaction version number need to be at least 2?** The transaction version number is part of the validation rules for some features such as OP_CHECKSEQUENCEVERIFY. Since it is backwards compatible, and there are other ways to disable those features (e.g. through sequence numbers), it is easier to require transactions be able to support these features than to try to negotiate the transaction version number. If the Creator is a Constructor and no inputs and outputs will be added by other entities, PSBT_GLOBAL_TX_MODIFIABLE may be omitted.

Constructor

This Constructor is only present for PSBTv2. Once a Creator initializes the PSBT, a constructor will add inputs and outputs. Before any input or output may be added, the constructor must check the

PSBT_GLOBAL_TX_MODIFIABLE field. Inputs may only be added if the Inputs Modifiable flag is True. Outputs may only be added if the Outputs Modifiable flag is True.

When an input or output is added, the corresponding PSBT_GLOBAL_INPUT_COUNT or PSBT_GLOBAL_OUTPUT_COUNT must be incremented to reflect the number of inputs and outputs in the PSBT. When an input is added, it must have PSBT_IN_PREVIOUS_TXID and PSBT_IN_OUTPUT_INDEX set. When an output is added, it must have PSBT_OUT_VALUE and PSBT_OUT_OUTPUT_SCRIPT set. If the input has a required timelock, Constructors must set the requisite timelock field. If the input has a required time based timelock, then PSBT_IN_REQUIRED_TIME_TIMELOCK must be set. If the input has a required height based timelock, then PSBT_IN_REQUIRED_HEIGHT_TIMELOCK must be set. If an input has both types of timelocks, then both may be set. In some cases, an input that can allow both types, but a particular branch supporting only one type of timelock will be taken, then the type of timelock that will be used can be the only one set.

If an input being added specifies a required time lock, then the Constructor must iterate through all of the existing inputs and ensure that the time lock types are compatible. Additionally, if during this iteration, it finds that any inputs have signatures, it must ensure that the newly added input does not change the transaction's locktime. If the newly added input has an incompatible time lock, then it must not be added. If it changes the transaction's locktime when there are existing signatures, it must not be added.

If the Has SIGHASH_SINGLE flag is True, then the Constructor must iterate through the inputs and find the inputs which have signatures that use SIGHASH_SINGLE. The same number of inputs and outputs must be added before those inputs and their corresponding outputs.

A Constructor may choose to declare that no further inputs and outputs can be added to the transaction by setting the appropriate bits in PSBT_GLOBAL_TX_MODIFIABLE to 0 or by removing the field entirely.

A single entity is likely to be both a Creator and Constructor.

Updater

For PSBTv2, an Updater can set the sequence number.

Signer

For PSBTv2s, a signer must update the PSBT_GLOBAL_TX_MODIFIABLE field after signing inputs so that it accurately reflects the state of the PSBT. If the Signer added a signature that does not use SIGHASH_ANYONECANPAY, the Input Modifiable flag must be set to False. If the Signer added a signature that does not use SIGHASH_NONE, the Outputs Modifiable flag must be set to False. If the Signer added a signature that uses SIGHASH_SINGLE, the Has SIGHASH_SINGLE flag must be set to True.

Input Finalizer

For PSBTv2s, when the Input Finalizer clears a finalized input's fields as BIP 174 asks, it must keep PSBT_IN_PREVIOUS_TXID, PSBT_IN_OUTPUT_INDEX, PSBT_IN_SEQUENCE, PSBT_IN_REQUIRED_TIME_LOCKTIME and PSBT_IN_REQUIRED_HEIGHT_LOCKTIME. The Transaction Extractor constructs the transaction from these fields, and an input missing either PSBT_IN_PREVIOUS_TXID or PSBT_IN_OUTPUT_INDEX is invalid.

Transaction Extractor

For PSBTv2s, the transaction is constructed using the PSBTv2 fields. The lock time for this transaction is determined as described in the Determining Lock Time section. The Extractor should produce a fully valid, network serialized transaction if all inputs are complete.

Backwards Compatibility

PSBTv2 shares the same generic format as PSBTv0 as defined in BIP 174. Parsers for PSBTv0 should be able to deserialize PSBTv2 with only changes to support the new fields.

However PSBTv2 is incompatible with PSBTv0, and vice versa due to the use of the PSBT_GLOBAL_VERSION. This incompatibility is intentional so that PSBT_GLOBAL_UNSIGNED_TX could be removed in PSBTv2. However it is possible to convert a PSBTv2 to a PSBTv0 by creating an unsigned transaction from the PSBTv2 fields.

Test Vectors

The following are invalid PSBTs:

- Case: PSBTv0 but with PSBT_GLOBAL_VERSION set to 2.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJzO
    O18/+NBRkwbjUV11FaXoBbEgAAAAAA/////
    wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFLUtsK7wbgAAAAAAEBHxjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFLUtsK7wbgAAAAAA
    DQKJwAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAAA

- Case: PSBTv0 but with PSBT_GLOBAL_TX_VERSION.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJzO
    O18/+NBRkwbjUV11FaXoBbEgAAAAAA/////
    wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFLUtsK7wbgAAAAAA
    DQKJwAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAAA

- Case: PSBTv0 but with PSBT_GLOBAL_FALLBACK_LOCKTIME.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////

AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
O18/+NBRkwbjUV11FaXoBbEgAAAAAA/////
wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAFgAUsKOvFEIIQSaT
DQKJwAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAA

- Case: PSBTv0 but with PSBT_GLOBAL_INPUT_COUNT.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
    O18/+NBRkwbjUV11FaXoBbEgAAAAAA/////
    wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAFgAUsKOvFEIIQSaT
    DQKJwAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAA

- Case: PSBTv0 but with PSBT_GLOBAL_OUTPUT_COUNT.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
    O18/+NBRkwbjUV11FaXoBbEgAAAAAA/////
    wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAFgAUsKOvFEIIQSaT
    DQKJwAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAA

- Case: PSBTv0 but with PSBT_GLOBAL_TX_MODIFIABLE.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
    O18/+NBRkwbjUV11FaXoBbEgAAAAAA/////
    wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAFgAUsKOvFEIIQSaT
    DQKJwAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAA

- Case: PSBTv0 but with PSBT_IN_PREVIOUS_TXID.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////

ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
KfFRTIxScEjhKAKwQdT8NAonAQ4gCwrZIUGcHIcZc11y3HOfnqngY40f5MHu8PmUQISBX8gAIgIC1gH4SEa

- Case: PSBTv0 but with PSBT_IN_OUTPUT_INDEX.
  - Bytes in Hex:
  70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
  cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
  IAAAAAAD+////
  AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
  87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
  ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
  KfFRTIxScEjhKAKwQdT8NAonAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj

- Case: PSBTv0 but with PSBT_IN_SEQUENCE.
  - Bytes in Hex:
  70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
  cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
  IAAAAAAD+////
  AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
  87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
  ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
  KfFRTIxScEjhKAKwQdT8NAonARAE/////
  wAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAACoAA

- Case: PSBTv0 but with PSBT_IN_REQUIRED_TIME_LOCKTIME.
  - Bytes in Hex:
  70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
  cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
  IAAAAAAD+////
  AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
  87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
  ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
  KfFRTIxScEjhKAKwQdT8NAonAREEjI3EYgAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nY

- Case: PSBTv0 but with PSBT_IN_REQUIRED_HEIGHT_LOCKTIME.
  - Bytes in Hex:
  70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
  cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
  IAAAAAAD+////
  AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJz
  87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////

ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
KfFRTIxScEjhKAKwQdT8NAonARIEECcAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2r

- Case: PSBTv0 but with PSBT_OUT_AMOUNT.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJzG
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    KfFRTIxScEjhKAKwQdT8NAonACICAtYB+EhGpnVfd2vgDj2d6PsQrMk1+4PEX7AWLUytWreSGPadhz5UAA

- Case: PSBTv0 but with PSBT_OUT_SCRIPT.
  - Bytes in Hex:
    70736274ff01007102000000010b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9944084815fc80000000000f
  - Base64 String:
    cHNidP8BAHECAAAAAQsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5lECEgV/
    IAAAAAAD+////
    AgAIry8AAAAAFgAUxDD2TEdW2jENvRoIVXLvKZkmJyyLvesLAAAAABYAFKB9rIq2ypQtN57Xlfg1unHJzG
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    KfFRTIxScEjhKAKwQdT8NAonACICAtYB+EhGpnVfd2vgDj2d6PsQrMk1+4PEX7AWLUytWreSGPadhz5UAA

- Case: PSBTv2 but with PSBT_GLOBAL_UNSIGNED_TX.
  - Bytes in Hex:
    70736274ff0100520200000001c1aa256e214b96a1822f93de42bff3b5f3ff8d0519306e3515d7515a5e805b120000000000f
  - Base64 String: cHNidP8BAFICAAAAAcGqJW4hS5ahgi+T3kK/87Xz/
    40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQIEAgAAAAEDBAAAAAABBAEBAQU
    ztfP/jQUZMG41FddRWl6AWxIAAAAAAP////
    8BGMaaOwAAAAAWABSwo68UQghBJpPKfRZoUrUtsK7wbgAAAAABAR8Yxpo7AAAAABYAFLCjrxRCCEEm
    v///
    wERBIyNxGIBEgQQJwAAACICAtYB+EhGpnVfd2vgDj2d6PsQrMk1+4PEX7AWLUytWreSGPadhz5UAACAA
    TlmFGgPNXqbhdtzQL8c+nRdKtezQBj2nYc+VAAAgAEAAIAAAACAAQAAAGQAAAABAwiLvesLAAAAA

- Case: PSBTv2 missing PSBT_GLOBAL_INPUT_COUNT.
  - Bytes in Hex:
    70736274ff0102040200000001030400000000010501020 1fb0402000000000100520200000001c1aa256e214b96a1822f93
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEFAQIB+wQCAAAAAAEAUgIAAAABwaolbiFLlqGCL5PeQr/
    ztfP/jQUZMG41FddRWl6AWxIAAAAAAP////
    8BGMaaOwAAAAAWABSwo68UQghBJpPKfRZoUrUtsK7wbgAAAAABAR8Yxpo7AAAAABYAFLCjrxRCCEEm
    v///
    wAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAAACoAA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/

HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
i9HT4UTAA==

- Case: PSBTv2 missing PSBT_GLOBAL_OUTPUT_COUNT.
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010101fb040200000000010052020000001c1aa256e214b96a1822f93
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQEB+wQCAAAAAAEAUgIAAAABwaolbiFLlqGCL5PeQr/
    ztfP/jQUZMG41FddRWl6AWxIAAAAAAP////
    8BGMaaOwAAAAAWABSwo68UQghBJpPKfRZoUrUtsK7wbgAAAAABAR8Yxpo7AAAAABYAFLCjrxRCCEI
    v///
    wAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAACoAA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
    i9HT4UTAA==

- Case: PSBTv2 missing PSBT_GLOBAL_TX_VERSION.
  - Bytes in Hex:
    70736274ff010401010105010201fb040200000000010e200b0ad921419c1c8719735d72dc739f9ea9e0638d1fe4c1eef0f9
  - Base64 String:
    cHNidP8BBAEBAQUBAgH7BAIAAAAAAQ4gCwrZIUGcHIcZc11y3HOfnqngY40f5MHu8PmUQISBX8gBDwC

- Case: PSBTv2 missing PSBT_IN_PREVIOUS_TXID.
  - Bytes in Hex:
    70736274ff0102040200000001030400000000010401010105010201fb040200000000010052020000001c1aa256e214b96
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQEBBQECAfsEAgAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    ACICAtYB+EhGpnVfd2vgDj2d6PsQrMk1+4PEX7AWLUytWreSGPadhz5UAACAAQAAgAAAAIAAAAAAKg
    TlmFGgPNXqbhdtzQL8c+nRdKtezQBj2nYc+VAAAgAEAAIAAAACAAQAAAGQAAAABAwiLvesLAAAAA

- Case: PSBTv2 missing PSBT_IN_OUTPUT_INDEX.
  - Bytes in Hex:
    70736274ff0102040200000001030400000000010401010105010201fb040200000000010052020000001c1aa256e214b96
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQEBBQECAfsEAgAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IARAE/v///
    wAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAACoAA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
    i9HT4UTAA==

- Case: PSBTv2 missing PSBT_OUT_AMOUNT.
  - Bytes in Hex:
    70736274ff0102040200000001030400000000010401010105010201fb040200000000010052020000001c1aa256e214b96

    - Base64 String:
      cHNidP8BAgQCAAAAAQMEAAAAAAEEAQEBBQECAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK
      87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
      ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
      IAQ8EAAAAAAEQBP7///8AIgIC1gH4SEamdV93a+AOPZ3o+xCsyTX7g8RfsBYtTK1at5IY9p2HPlQAAIABAA
      TlmFGgPNXqbhdtzQL8c+nRdKtezQBj2nYc+VAAAgAEAAIAAAACAAQAAAGQAAAABAwiLvesLAAAAA

wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAAFgAUsKOvFEIIQSaT
kwe7w+ZRAhIFfyAEPBAAAAAABEAT+////
AREEjI3EYgESBAAAAAAAIgIC1gH4SEamdV93a+AOPZ3o+xCsyTX7g8RfsBYtTK1at5IY9p2HPlQAAIABAAC
U91TQHDPj9OWYUaA81epuF23NAvxz6dF0q17NAGPadhz5UAACAAQAAgAAAAIABAAAAZAAAAAEDC

The following are valid PSBTs

- Case: 1 input, 2 output PSBTv2, required fields only.
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201fb040200000000010e200b0ad921419c1c8719735d72dc739f9ea9e063
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIB+wQCAAAAAAEOIAsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5
    IAQ8EAAAAAAABAwgACK8vAAAAAAEEFgAUxDD2TEdW2jENvRoIVXLvKZkmJywAAQMIi73rCwAAAA
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2.
  - Bytes in HEx:
    70736274ff01020402000000010401010105010201fb040200000000010052020000000 1c1aa256e214b96a1822f93de42bf
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIB+wQCAAAAAAEAUgIAAAABwaolbiFLlqGCL5PeQr/
    ztfP/jQUZMG41FddRWl6AWxIAAAAAAP////
    8BGMaaOwAAAAAWABSwo68UQghBJpPKfRZoUrUtsK7wbgAAAAABAR8Yxpo7AAAAABYAFLCjrxRCCEI
    TlmFGgPNXqbhdtzQL8c+nRdKtezQBj2nYc+VAAAgAEAAIAAAACAAQAAAGQAAAABAwiLvesLAAAAA

- Case: 1 input, 2 output updated PSBTv2, with PSBT_IN_SEQUENCE.
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201fb040200000000010052020000000 1c1aa256e214b96a1822f93de42bf
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIB+wQCAAAAAAEAUgIAAAABwaolbiFLlqGCL5PeQr/
    ztfP/jQUZMG41FddRWl6AWxIAAAAAAP////
    8BGMaaOwAAAAAWABSwo68UQghBJpPKfRZoUrUtsK7wbgAAAAABAR8Yxpo7AAAAABYAFLCjrxRCCEI
    v////
    wAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACAAAAAACoAA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAABBBYAFE3Rk6yWSlasG
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with PSBT_IN_SEQUENCE, and all locktime fields
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010101050102 01fb040200000000010052020000000 1c1aa256e214b96
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQEBBQECAfsEAgAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAEQBP7///8BEQSMjcRiARIEECcAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1Mr
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/

HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with Inputs Modifiable Flag (bit 0) of PSBT_GLOBAL_TX_MODIFIABLE set
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201060101 01fb040200000000010052020000000 1c1aa256e214b96a1822
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgEBAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with Outputs Modifiable Flag (bit 1) of PSBT_GLOBAL_TX_MODIFIABLE set
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201060102 01fb040200000000010052020000000 1c1aa256e214b96a1822
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgECAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with Has SIGHASH_SINGLE Flag (bit 2) of PSBT_GLOBAL_TX_MODIFIABLE set
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201060104 01fb040200000000010052020000000 1c1aa256e214b96a1822
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgEEAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with an undefined flag (bit 3) of PSBT_GLOBAL_TX_MODIFIABLE set
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201060108 01fb040200000000010052020000000 1c1aa256e214b96a1822
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgEIAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/

87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas0
i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with both Inputs Modifiable Flag (bit 0) and Outputs Modifiable Flag (bit 1) of PSBT_GLOBAL_TX_MODIFIABLE set
    - Bytes in Hex:
    70736274ff01020402000000010401010105010201060103 01fb040200000000010052020000000 1c1aa256e214b96a1822f
    - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgEDAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas0
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with both Inputs Modifiable Flag (bit 0) and Has SIGHASH_SINGLE Flag (bit 2) of PSBT_GLOBAL_TX_MODIFIABLE set
    - Bytes in Hex:
    70736274ff01020402000000010401010105010201060105 01fb040200000000010052020000000 1c1aa256e214b96a1822f
    - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgEFAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas0
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with both Outputs Modifiable Flag (bit 1) and Has SIGHASH_SINGLE FLag (bit 2) of PSBT_GLOBAL_TX_MODIFIABLE set
    - Bytes in Hex:
    70736274ff01020402000000010401010105010201060106 01fb040200000000010052020000000 1c1aa256e214b96a1822f
    - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgEGAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/

HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with all defined PSBT_GLOBAL_TX_MODIFIABLE flags set
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201060107 01fb040200000000010052020000000 1c1aa256e214b96a1822f
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIBBgEHAfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/
    87Xz/40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with all possible PSBT_GLOBAL_TX_MODIFIABLE flags set
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201060 1ff01fb040200000000010052020000000 1c1aa256e214b96a1822f
  - Base64 String: cHNidP8BAgQCAAAAAQQBAQEFAQIBBgH/
    AfsEAgAAAAABAFICAAAAAcGqJW4hS5ahgi+T3kK/87Xz/
    40FGTBuNRXXUVpegFsSAAAAAAD/////
    ARjGmjsAAAAAFgAUsKOvFEIIQSaTyn0WaFK1LbCu8G4AAAAAAQEfGMaaOwAAAAAWABSwo68UQghB
    IAQ8EAAAAAAAiAgLWAfhIRqZ1X3dr4A49nej7EKzJNfuDxF+wFi1MrVq3khj2nYc+VAAAgAEAAIAAAACA
    9T3VNAcM+P05ZhRoDzV6m4Xbc0C/
    HPp0XSrXs0AY9p2HPlQAAIABAACAAAAAgAEAAABkAAAAAQMIi73rCwAAAAABBBYAFE3Rk6yWSlas
    i9HT4UTAA==

- Case: 1 input, 2 output updated PSBTv2, with all PSBTv2 fields
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010101050102010601 0701fb040200000000010052020000000 1c1aa25
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQEBBQECAQYBBwH7BAIAAAAAAQBSAgAAAAHBqiVuIUuW
    O18/+NBRkwbjUV11FaXoBbEgAAAAAA/////
    wEYxpo7AAAAABYAFLCjrxRCCEEmk8p9FmhStS2wrvBuAAAAAAEBHxjGmjsAAAAAFgAUsKOvFEIIQSaT
    kwe7w+ZRAhIFfyAEPBAAAAAAABEAT+/////
    AREEjI3EYgESBBAnAAAAIgIC1gH4SEamdV93a+AOPZ3o+xCsyTX7g8RfsBYtTK1at5IY9p2HPlQAAIABAAC
    U91TQHDPj9OWYUaA81epuF23NAvxz6dF0q17NAGPadhz5UAACAAQAAgAAAAIABAAAAZAAAAEDC

The following tests are the timelock determination algorithm.

The timelock for the following PSBTs should be computed to be 0:

- Case: No locktimes specified
  - Bytes in Hex:
    70736274ff01020402000000010401010105010201fb040200000000010e200b0ad921419c1c8719735d72dc739f9ea9e063
  - Base64 String:
    cHNidP8BAgQCAAAAAQQBAQEFAQIB+wQCAAAAAAEOIAsK2SFBnByHGXNdctxzn56p4GONH+TB7vD5

IAQ8EAAAAAAABAwgACK8vAAAAAAEEFgAUxDD2TEdW2jENvRoIVXLvKZkmJywAAQMIi73rCwAAA
i9HT4UTAA==

- Case: Fallback locktime of 0
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
    vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAAAAQ4gOhs7PIN9ZInqejHY5sfdUDwAG+8

The timelock for the following PSBTs should be computed to be 10000:

- Case: Input 1 has PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 10000, Input 2 has no locktime fields
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
    vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEgQQJwAAAAEOIDobOzyDfWSJ6nox2
    PI1jiGcbNKXhEA

- Case: Input 1 has PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 10000, Input 2 has
  PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 9000
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
    vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEgQQJwAAAAEOIDobOzyDfWSJ6nox2

- Case: Input 1 has PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 10000, Input 2 has
  PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 9000 and PSBT_IN_REQUIRED_TIME_LOCKTIME of
  1657048460
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
    vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEgQQJwAAAAEOIDobOzyDfWSJ6nox2

- Case: Input 1 has PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 10000 and
  PSBT_IN_REQUIRED_TIME_LOCKTIME of 1657048459, Input 2 has
  PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 9000 and PSBT_IN_REQUIRED_TIME_LOCKTIME of
  1657048460
  - Bytes in Hex:
    70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
  - Base64 String:
    cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
    vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEQSLjcRiARIEECcAAAABDiA6Gzs8g31kiep6MdjmyN0PAAvvAAEAAAABEgQQJwAAAAEOIDobOzyDfWSJ6nox2
    PI1jiGcbNKXhEA

The timelock for the following PSBTs should be computed to be 1657048460:

- Case: Input 1 has PSBT_IN_REQUIRED_TIME_LOCKTIME of 1657048459, Input 2 has PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 9000 and PSBT_IN_REQUIRED_TIME_LOCKTIME of 1657048460
    - Bytes in Hex:
      70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
    - Base64 String:
      cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
      vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEQSLjcRiAAEOIDobOzyDfWSJ6nox2Ob
- Case: Input 1 has PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 10000 and PSBT_IN_REQUIRED_TIME_LOCKTIME of 1657048459, Input 2 has PSBT_IN_REQUIRED_TIME_LOCKTIME of 1657048460
    - Bytes in Hex:
      70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
    - Base64 String:
      cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
      vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEQSLjcRiARIEECcAAAABDiA6Gzs8g31
-
    - Bytes in Hex:
      70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
    - Base64 String:
      cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
      vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEQSLjcRiAAAAAQ4gOhs7PIN9ZInqejHY5sfdUDwAG+8
      PI1jiGcbNKXhEA

The timelock for the following PSBTs cannot be computed:

- Case: Input 1 has PSBT_IN_REQUIRED_HEIGHT_LOCKTIME of 10000, Input 2 has PSBT_IN_REQUIRED_TIME_LOCKTIME of 1657048460
    - Bytes in Hex:
      70736274ff01020402000000010304000000000104010201050101fb040200000000010e200f758dbfbd4da7c16c8a3309
    - Base64 String:
      cHNidP8BAgQCAAAAAQMEAAAAAAEEAQIBBQEBAfsEAgAAAAABDiAPdY2/
      vU2nwWyKMwnDyB4RAPVh6mRttbAXUsSF4b3enwEPBAEAAAABEgQQJwAAAAEOIDobOzyDfWSJ6nox2

Rationale

Reference implementation

The reference implementation of the PSBT format is available in Bitcoin Core PR 21283.

```
BIP: 371
Layer: Applications
Title: Taproot Fields for PSBT
Authors: Ava Chow <me@achow101.com>
Status: Deployed
Type: Specification
```

```
Assigned: 2021-06-21
License: BSD-2-Clause
```

BIP 371

Introduction

Abstract

This document proposes additional fields for BIP 174 PSBTv0 and BIP 370 PSBTv2 that allow for BIP 340/341/342 Taproot data to be included in a PSBT of any version. These will be fields for signatures and scripts that are relevant to the creation of Taproot inputs.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

BIPs 340, 341, and 342 specify Taproot which provides a wholly new way to create and spend Bitcoin outputs. The existing PSBT fields are unable to support Taproot due to the new signature algorithm and the method by which scripts are embedded inside of a Taproot output. Therefore new fields must be defined to allow PSBTs to carry the information necessary for signing Taproot inputs.

Specification

The new per-input types are defined as follows:

Name	**<keytype>**	**<keydata>**	**<keydata> Description**	**<valuedata>**	**<valuedata> De**
Taproot Key Spend Signature	PSBT_IN_TAP_KEY_SIG = 0x13	None	No key data **Why is there no key data for PSBT_IN_TAP_KEY_SIG**The signature in a key path spend corresponds directly with the pubkey provided in the output script. Thus it is not necessary to provide any metadata that attaches the key path spend signature to a particular pubkey.	<64 or 65 byte signature>	The 64 or 65 byt signature for ke spending a Tapr Finalizers shoul field after PSBT_IN_FINAL_ is constructed.
Taproot Script	PSBT_IN_TAP_SCRIPT_SIG = 0x14	<xonlypubkey> <leafhash>	A 32 byte X-only public key involved in a leaf script concatenated with	<64 or 65 byte signature>	The 64 or 65 byt signature for thi leaf combinatio

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> De
Spend Signature			the 32 byte hash of the leaf it is part of.		should remove t PSBT_IN_FINAL_ is constructed.
Taproot Leaf Script	PSBT_IN_TAP_LEAF_SCRIPT = 0x15	<bytes control block>	The control block for this leaf as specified in BIP 341. The control block contains the merkle tree path to this leaf.	<bytes script> <8-bit uint leaf version>	The script for th would be provid witness stack fo single byte leaf v that the leaves i field should be t signers of this in expected to be a Finalizers shoul field after PSBT_IN_FINAL_ is constructed.
Taproot Key BIP 32 Derivation Path	PSBT_IN_TAP_BIP32_DERIVATION = 0x16	<32 byte xonlypubkey>	A 32 byte X-only public key involved in this input. It may be the output key, the internal key, or a key present in a leaf script.	<compact size uint number of hashes> <32 byte leaf hash>* <4 byte fingerprint> <32-bit little endian uint path element>*	A compact size u integer represen number of leaf h followed by a lis hashes, followed master key finge concatenated wi derivation path key. The derivat represented as 3 endian unsigned indexes concate each other. Publ those needed to output. The leaf the leaves which public key. The i does not have le can be indicated len of 0. Finalize remove this field PSBT_IN_FINAL_ is constructed.
Taproot Internal Key	PSBT_IN_TAP_INTERNAL_KEY = 0x17	None	No key data	<32 byte xonlypubkey>	The X-only publ internal key in th output.**Why is t key provided?**T

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> De
					is not necessaril as in the Taproo BIP 341 recomm the key with the It may be necess to know what th actually is so tha to determine wh can be signed by should remove t PSBT_IN_FINAL_ is constructed.
Taproot Merkle Root	PSBT_IN_TAP_MERKLE_ROOT = 0x18	None	No key data	<32-byte hash>	The 32 byte Mer Finalizers shoul field after PSBT_IN_FINAL_ is constructed.

The new per-output types are defined as follows:

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	V R E
Taproot Internal Key	PSBT_OUT_TAP_INTERNAL_KEY = 0x05	None	No key data	<32 byte xonlypubkey>	The X-only pubkey used as the internal key in this output.		
Taproot Tree	PSBT_OUT_TAP_TREE = 0x06	None	No key data	{<8-bit uint depth> <8-bit uint leaf version> <compact size uint scriptlen> <bytes script>}*	One or more tuples representing the depth, leaf version, and script for a leaf in the Taproot tree, allowing the entire tree to be reconstructed. The tuples must be in depth first		

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	V F E
					search order so that the tree is correctly reconstructed. Each tuple is an 8-bit unsigned integer representing the depth in the Taproot tree for this script, an 8-bit unsigned integer representing the leaf version, the length of the script as a compact size unsigned integer, and the script itself.		
Taproot Key BIP 32 Derivation Path	PSBT_OUT_TAP_BIP32_DERIVATION = 0x07	<32 byte xonlypubkey>	A 32 byte X-only public key involved in this output. It may be the output key, the internal key, or a key present in a leaf script.	<compact size uint number of hashes> <32 byte leaf hash>* <4 byte fingerprint> <32-bit little endian uint path element>*	A compact size unsigned integer representing the number of leaf hashes, followed by a list of leaf hashes, followed by the 4 byte master key fingerprint concatenated with the derivation path of the		

Name	<keytype>	<keydata>	<keydata> Description	<valuedata>	<valuedata> Description	Versions Requiring Inclusion	V F E
					public key. The derivation path is represented as 32-bit little endian unsigned integer indexes concatenated with each other. Public keys are those needed to spend this output. The leaf hashes are of the leaves which involve this public key. The internal key does not have leaf hashes, so can be indicated with a hashes len of 0.		

UTXO Types

BIP 174 recommends using `PSBT_IN_NON_WITNESS_UTXO` for all inputs because of potential attacks involving an updater lying about the amounts in an output. Because a Taproot signature will commit to all of the amounts and output scripts spent by the inputs of the transaction, such attacks are prevented as any such lying would result in an invalid signature. Thus Taproot inputs can use just `PSBT_IN_WITNESS_UTXO`.

Compatibility

These are simply new fields added to the existing PSBT format. Because PSBT is designed to be extensible, old software will ignore the new fields.

Test Vectors

The following are invalid PSBTs:

- Case: PSBT With `PSBT_IN_TAP_INTERNAL_KEY` key that is too long (incorrectly serialized as compressed DER)
    - Bytes in Hex:
-
    - Base64 String: cHNidP8BAHECAAAAASd0Srq/
      MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
      Anh8AQAAAAAAFgAUg6fjS9mf8DpJYu+KGhAbspVGHs5gawQqAQAAABYAFHrDad8bIOAz1hFmI5V7CsS
      8DGD7Nj2CcdRCuikjgORCgdXARchAv40kGTJjW4qhT+jybEr2LMEoZwZXGDvp+4jkwRtP6IyAAAA

- Case: PSBT With `PSBT_IN_TAP_KEY_SIG` signature that is too short
    - Bytes in Hex:
      <70736274ff01007102000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000
    - Base64 String: cHNidP8BAHECAAAAASd0Srq/
      MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
      Anh8AQAAAAAAFgAUg6fjS9mf8DpJYu+KGhAbspVGHs5gawQqAQAAABYAFHrDad8bIOAz1hFmI5V7CsS
      8DGD7Nj2CcdRCuikjgORCgdXARchAv40kGTJjW4qhT+jybEr2LMEoZwZXGDvp+4jkwRtP6IyAAAA
      Fzuz02wHSvtxb+xjB6BpouRQuZXzyCeFlFq43w4kJg3NcDsMvzTeOZGEqUgawrNYbbZgHwJqd/
      fkk4SBvDR1AAAA

- Case: PSBT With `PSBT_IN_TAP_KEY_SIG` signature that is too long
    - Bytes in Hex:
      <70736274ff01007102000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000
    - Base64 String: cHNidP8BAHECAAAAASd0Srq/
      MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
      Anh8AQAAAAAAFgAUg6fjS9mf8DpJYu+KGhAbspVGHs5gawQqAQAAABYAFHrDad8bIOAz1hFmI5V7CsS
      8DGD7Nj2CcdRCuikjgORCgdXARchAv40kGTJjW4qhT+jybEr2LMEoZwZXGDvp+4jkwRtP6IyAAAA
      fkk4SBvDR1FwGqAAAA

- Case: PSBT With `PSBT_IN_TAP_BIP32_DERIVATION` key that is too long (incorrectly serialized as compressed DER)
    - Bytes in Hex:
      <70736274ff01007102000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000
    - Base64 String: cHNidP8BAHECAAAAASd0Srq/
      MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
      Anh8AQAAAAAAFgAUg6fjS9mf8DpJYu+KGhAbspVGHs5gawQqAQAAABYAFHrDad8bIOAz1hFmI5V7CsS
      8DGD7Nj2CcdRCuikjgORCgdXIhYC/jSQZMmNbiqFP6PJsSvYswShnBlcYO+n7iOTBG0/
      ojIZAHcrLadWAACAAQAAgAAAAIABAAAAAAAAAAAAAA==

- Case: PSBT With `PSBT_OUT_TAP_INTERNAL_KEY` key that is too long (incorrectly serialized as compressed DER)
    - Bytes in Hex:
      70736274ff01007d020000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000
    - Base64 String: cHNidP8BAH0CAAAAASd0Srq/
      MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
      Aoh7AQAAAAAAFgAUI4KHHH6EIaAAk/

dU2RKB5nWHS59gawQqAQAAACJRIFosLPW1LPMfg60ujaY/
8DGD7Nj2CcdRCuikjgORCgdXAAAAAAAABASsA8gUqAQAAACJRIFosLPW1LPMfg60ujaY/
8DGD7Nj2CcdRCuikjgORCgdXAAABBSEC/jSQZMmNbiqFP6PJsSvYswShnBlcYO+n7iOTBG0/
ojIA

- Case: PSBT With `PSBT_OUT_TAP_BIP32_DERIVATION` key that is too long (incorrectly serialized as compressed DER)
  - Bytes in Hex:
    70736274ff01007d020000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000
  - Base64 String: cHNidP8BAH0CAAAAASd0Srq/
    MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
    Aoh7AQAAAAAAFgAUI4KHHH6EIaAAk/
    dU2RKB5nWHS59gawQqAQAAACJRIFosLPW1LPMfg60ujaY/
    8DGD7Nj2CcdRCuikjgORCgdXAAAAAAAABASsA8gUqAQAAACJRIFosLPW1LPMfg60ujaY/
    8DGD7Nj2CcdRCuikjgORCgdXAAAiBwL+NJBkyY1uKoU/
    o8mxK9izBKGcGVxg76fuI5MEbT+iMhkAdystp1YAAIABAACAAAAAgAEAAAAAAAAAAA==

- Case: PSBT With `PSBT_IN_TAP_SCRIPT_SIG` key that is too long (incorrectly serialized as compressed DER)
  - Bytes in Hex:
    70736274ff01005e02000000019bd48765230bf9a72e662001f972556e54f0c6f97feb56bcb5600d817f6995260100000000f
  - Base64 String: cHNidP8BAF4CAAAAAZvUh2UjC/mnLmYgAflyVW5U8Mb5f+tWvLVgDYF/
    aZUmAQAAAAD/////AUjmBSoBAAAAIlEgAw2k/
    OT32yjCyylRYx4ANxOFZZf+ljiCy1AOaBEsymMAAAAAAAEBKwDyBSoBAAAAIlEgwiR++/
    2SrEf29AuNQtFpF1oZ+p+hDkol1/
    NetN2FtpJCFAIssTrGgkjegGqmo2Wc88A+toIdCcgRSk6Gj+vehlu20s2XDhX1P8DIL5UP1WD/
    qRm3YXK+AXNoqJkTrwdPQAsJQIl1aqNznMxonsD886NgvjLMC1mxbpOh6LtGBXJrLKej/
    3BsQXZkljKyzGjh+RK4pXjjcZzncQiFx6lm9JvNQ8sAAA==

- Case: PSBT With `PSBT_IN_TAP_SCRIPT_SIG` signature that is too long
  - Bytes in Hex:
    70736274ff01005e02000000019bd48765230bf9a72e662001f972556e54f0c6f97feb56bcb5600d817f6995260100000000f
  - Base64 String: cHNidP8BAF4CAAAAAZvUh2UjC/mnLmYgAflyVW5U8Mb5f+tWvLVgDYF/
    aZUmAQAAAAD/////AUjmBSoBAAAAIlEgAw2k/
    OT32yjCyylRYx4ANxOFZZf+ljiCy1AOaBEsymMAAAAAAAEBKwDyBSoBAAAAIlEgwiR++/
    2SrEf29AuNQtFpF1oZ+p+hDkol1/
    NetN2FtpJBFCyxOsaCSN6AaqajZZzzwD62gh0JyBFKToaP696GW7bSzZcOFfU/wMgvlQ/
    VYP+pGbdhcr4Bc2iomROvB09ACwlCiXVqo3OczGiewPzzo2C+MswLWbFuk6Hou0YFcmssp6P/
    cGxBdmSWMrLMaOH5ErileONxnOdxCIXHqWb0m81DywEBAAA=

- Case: PSBT With `PSBT_IN_TAP_SCRIPT_SIG` signature that is too short
  - Bytes in Hex:
    70736274ff01005e02000000019bd48765230bf9a72e662001f972556e54f0c6f97feb56bcb5600d817f6995260100000000f
  - Base64 String: cHNidP8BAF4CAAAAAZvUh2UjC/mnLmYgAflyVW5U8Mb5f+tWvLVgDYF/
    aZUmAQAAAAD/////AUjmBSoBAAAAIlEgAw2k/
    OT32yjCyylRYx4ANxOFZZf+ljiCy1AOaBEsymMAAAAAAAEBKwDyBSoBAAAAIlEgwiR++/
    2SrEf29AuNQtFpF1oZ+p+hDkol1/
    NetN2FtpJBFCyxOsaCSN6AaqajZZzzwD62gh0JyBFKToaP696GW7bSzZcOFfU/wMgvlQ/

VYP+pGbdhcr4Bc2iomROvB09ACwk/
iXVqo3OczGiewPzzo2C+MswLWbFuk6Hou0YFcmssp6P/
cGxBdmSWMrLMaOH5ErileONxnOdxCIXHqWb0m81DAAA=

- Case: PSBT With `PSBT_IN_TAP_LEAF_SCRIPT` Control block that is too long
  - Bytes in Hex:
    70736274ff01005e02000000019bd48765230bf9a72e662001f972556e54f0c6f97feb56bcb5600d817f6995260100000000f
  - Base64 String: cHNidP8BAF4CAAAAAZvUh2UjC/mnLmYgAflyVW5U8Mb5f+tWvLVgDYF/
    aZUmAQAAAAD/////AUjmBSoBAAAAIlEgAw2k/
    OT32yjCyylRYx4ANxOFZZf+ljiCy1AOaBEsymMAAAAAAAEBKwDyBSoBAAAAIlEgwiR++/
    2SrEf29AuNQtFpF1oZ+p+hDkol1/
    NetN2FtpJjFcFQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA6wG99YgWelJehpKJnVp2YdtpgEBr/
    OONSm5uTnOf5GulwEV8uSQr3zEXE94UR82BXzlxaXFYyWin7RN/CA/
    NW4fgAIyAssTrGgkjegGqmo2Wc88A+toIdCcgRSk6Gj+vehlu20qzAAAA=

- Case: PSBT With `PSBT_IN_TAP_LEAF_SCRIPT` Control block that is too short
  - Bytes in Hex:
    70736274ff01005e02000000019bd48765230bf9a72e662001f972556e54f0c6f97feb56bcb5600d817f6995260100000000f
  - Base64 String: cHNidP8BAF4CAAAAAZvUh2UjC/mnLmYgAflyVW5U8Mb5f+tWvLVgDYF/
    aZUmAQAAAAD/////AUjmBSoBAAAAIlEgAw2k/
    OT32yjCyylRYx4ANxOFZZf+ljiCy1AOaBEsymMAAAAAAAEBKwDyBSoBAAAAIlEgwiR++/
    2SrEf29AuNQtFpF1oZ+p+hDkol1/
    NetN2FtpJhFcFQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA6wG99YgWelJehpKJnVp2YdtpgEBr/
    OONSm5uTnOf5GulwEV8uSQr3zEXE94UR82BXzlxaXFYyWin7RN/CA/
    NW4SMgLLE6xoJI3oBqpqNlnPPAPraCHQnIEUpOho/r3oZbttKswAAA

The following are valid PSBTs:

- Case: PSBT with one P2TR key only input with internal key and its derivation path
  - Bytes in Hex:
    70736274ff0100520200000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000f
  - Base64 String: cHNidP8BAFICAAAAASd0Srq/
    MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
    AUjmBSoBAAAAFgAUdo4e60z0IIZgM/
    gKzv8PlyB0SWkAAAAAAAEBKwDyBSoBAAAAIlEgWiws9bUs8x+DrS6Npj/
    wMYPs2PYJx1EK6KSOA5EKB1chFv40kGTJjW4qhT+jybEr2LMEoZwZXGDvp+4jkwRtP6IyGQB3Ky2nVgAAg/
    o8mxK9izBKGcGVxg76fuI5MEbT+AwRIQkLs5z2Bh3Ky2nV

- Case: PSBT with one P2TR key only input with internal key, its derivation path, and signature
  - Bytes in Hex:
    70736274ff0100520200000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000f
  - Base64 String: cHNidP8BAFICAAAAASd0Srq/
    MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
    AUjmBSoBAAAAFgAUdo4e60z0IIZgM/
    gKzv8PlyB0SWkAAAAAAAEBKwDyBSoBAAAAIlEgWiws9bUs8x+DrS6Npj/
    wMYPs2PYJx1EK6KSOA5EKB1cBE0C7U+yRe62dkGrxuocYHEi4as5aritTYFpyXKdGJWMUdvxvW67a9PLuD0
    NvWPOXDVuCc7fkl7l68uPxJcl680IRb+NJBkyY1uKoU/
    o8mxK9izBKGcGVxg76fuI5MEbT+MhkAdystp1YAAIABAACAAAAAgAEAAAAAAAAAARcg/

jSQZMmNbiqFP6PJsSvYswShnBlcYO+n7iOTBG0/
ojIAIgIDa3cqbbdNh1PJioJ5WN5seKszEhCfN9PgMESEJC7Oc9gYdystp1QAAIABAACAAAAAgAAAAAAAA

- Case: PSBT with one P2TR key only output with internal key and its derivation path
  - Bytes in Hex:
    70736274ff01005e020000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000f
  - Base64 String: cHNidP8BAF4CAAAAASd0Srq/
    MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
    AUjmBSoBAAAAIlEgg2mORYxmZOFZXXXaJZfeHiLul9eY5wbEwKS1qYI810MAAAAAAAEBKwDyBSoBAA
    wMYPs2PYJx1EK6KSOA5EKB1chFv40kGTJjW4qhT+jybEr2LMEoZwZXGDvp+4jkwRtP6IyGQB3Ky2nVgAAgA
    o8mxK9izBKGcGVxg76fuI5MEbT+iMgABBSARJNp67JLM0GyVRWJkf0N7E4uVchqEvivyJ2u92rPmcSEHESTae

- Case: PSBT with one P2TR script path only input with dummy internal key, scripts, derivation paths for keys in the scripts, and merkle root
  - Bytes in Hex:
    70736274ff01005e02000000019bd48765230bf9a72e662001f972556e54f0c6f97feb56bcb5600d817f6995260100000000f
  - Base64 String: cHNidP8BAF4CAAAAAZvUh2UjC/mnLmYgAflyVW5U8Mb5f+tWvLVgDYF/
    aZUmAQAAAAD/////
    AUjmBSoBAAAAIlEgg2mORYxmZOFZXXXaJZfeHiLul9eY5wbEwKS1qYI810MAAAAAAAEBKwDyBSoBAA
    +/2SrEf29AuNQtFpF1oZ+p+hDkol1/
    NetN2FtpJiFcFQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA6wG99YgWelJehpKJnVp2YdtpgEBr/
    OONSm5uTnOf5GulwEV8uSQr3zEXE94UR82BXzlxaXFYyWin7RN/CA/
    NW4fgjICyxOsaCSN6AaqajZZzzwD62gh0JyBFKToaP696GW7bSrMBCFcFQkpt0waBJVLeLS2A16XpeB4paDyjs
    3FP9XJEmZkIEOQG6YqhD1v35fZ4S8HQqabOIyBDILC/FvARtT6nvmFZJKp/
    J+XSmtIOoRVdhIZ2w7rRsqzAYhXBUJKbdMGgSVS3i0tgNel6XgeKWg8o7JbVR7/
    ums6AOsDNlw4V9T/AyC+VD9Vg/
    6kZt2FyvgFzaKiZE68HT0ALCRFfLkkK98xFxPeFEfNgV85cWlxWMlop+0TfwgPzVuH4IyD6D3o87zsdDAps59J
    wMgvlQ/
    VYP+pGbdhcr4Bc2iomROvB09ACwl3Ky2nVgAAgAEAAIACAACAAAAAAAAAAAAhFkMgsL8W8BG1Pqe+
    ums6AOsAFAHxGHl0hFvoPejzvOx0MCmzn0m4XraCy5cktGe+tSLQYWcuKRRypOQFvfWIFnpSXoaSiZ1admI
    zjjUpubk5zn+RrpcHcrLadWAACAAQAAgAMAAIAAAAAAAAAAAAAEXIFCSm3TBoElUt4tLYDXpel4HiloPK
    7prOgDrAARgg8DYuL3Wm9CClvePrIh2WrmcgzyX4GJDJWx13WstRXmUAAQUgESTaeuySzNBslUViZH9Dex
    Q3sTi5VyGoS+K/Ina73as+ZxGQB3Ky2nVgAAgAEAAIAAAACAAAAAAAUAAAAA

- Case: PSBT with one P2TR script path only output with dummy internal key, taproot tree, and script key derivation paths
  - Bytes in Hex:
    70736274ff01005e020000000127744ababf3027fe0d6cf23a96eee2efb188ef52301954585883e69b6624b2420000000000f
  - Base64 String: cHNidP8BAF4CAAAAASd0Srq/
    MCf+DWzyOpbu4u+xiO9SMBlUWFiD5ptmJLJCAAAAAAD/////
    AUjmBSoBAAAAIlEgCoy9yG3hzhwPnK6yLW33ztNoP+Qj4F0eQCqHk0HW9vUAAAAAAAEBKwDyBSoBAA
    wMYPs2PYJx1EK6KSOA5EKB1chFv40kGTJjW4qhT+jybEr2LMEoZwZXGDvp+4jkwRtP6IyGQB3Ky2nVgAAgA
    o8mxK9izBKGcGVxg76fuI5MEbT+iMgABBSBQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA6wAEGbwLA
    s37bzKfOAvVZu8gyN9tgT6rHEJzrCEHRPqkmgM43kiMjf/
    +zftvMp84C9Vm7yDI322BPqscQnM5AfBreYuSoQ7ZqdC7/

Trxc6U7FhfaOkFZygCCFs2Fay4Odystp1YAAIABAACAAQAAgAAAAAADAAAAIQdQkpt0waBJVLeLS2A16
ozJgVtC0CIy5M8rngmy42Cx3Ky2nVgAAgAEAAIADAACAAAAAAAMAAAAA

- Case: PSBT with one P2TR script path only input with dummy internal key, scripts, script key derivation paths, merkle root, and script path signatures
  - Bytes in Hex:
    70736274ff01005e02000000019bd48765230bf9a72e662001f972556e54f0c6f97feb56bcb5600d817f6995260100000000f
  - Base64 String: cHNidP8BAF4CAAAAAZvUh2UjC/mnLmYgAflyVW5U8Mb5f+tWvLVgDYF/
    aZUmAQAAAAD/////
    AUjmBSoBAAAAIlEgg2mORYxmZOFZXXXaJZfeHiLul9eY5wbEwKS1qYI810MAAAAAAAEBKwDyBSoBAA
    +/2SrEf29AuNQtFpF1oZ+p+hDkol1/
    NetN2FtpJBFCyxOsaCSN6AaqajZZzzwD62gh0JyBFKToaP696GW7bSzZcOFfU/wMgvlQ/
    VYP+pGbdhcr4Bc2iomROvB09ACwlAv4GNl1fW/+tTi6BX+0wfxOD17xhudlvrVkeR4Cr1/
    T1eJVHU404z2G8na4LJnHmu0/A5Wgge/
    NLMLGXdfmk9eUEUQyCwvxbwEbU+p75hWSSqfyfl0prSDqEVXYSGdsO60bIRXy5JCvfMRcT3hRHzYFfOXFp
    omsjbyGDNxncHUKKt2jYD5H5mI2KvvR7+4Y7sfKlKfdowV8AzjTsKDzcB+iPhCi+KPbvZAQ8MpEYEaQRT6D
    OONSm5uTnOf5GulwQOwfA3kgZGHIM0IoVCMyZwirAx8NpKJT7kWq+luMkgNNi2BUkPjNE+APmJmJuX4
    ZiFcFQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA6wG99YgWelJehpKJnVp2YdtpgEBr/
    OONSm5uTnOf5GulwEV8uSQr3zEXE94UR82BXzlxaXFYyWin7RN/CA/
    NW4fgjICyxOsaCSN6AaqajZZzzwD62gh0JyBFKToaP696GW7bSrMBCFcFQkpt0waBJVLeLS2A16XpeB4paDyjs
    3FP9XJEmZkIEOQG6YqhD1v35fZ4S8HQqabOIyBDILC/FvARtT6nvmFZJKp/
    J+XSmtIOoRVdhIZ2w7rRsqzAYhXBUJKbdMGgSVS3i0tgNel6XgeKWg8o7JbVR7/
    ums6AOsDNlw4V9T/AyC+VD9Vg/
    6kZt2FyvgFzaKiZE68HT0ALCRFfLkkK98xFxPeFEfNgV85cWlxWMlop+0TfwgPzVuH4IyD6D3o87zsdDAps59J
    wMgvlQ/
    VYP+pGbdhcr4Bc2iomROvB09ACwl3Ky2nVgAAgAEAAIACAACAAAAAAAAAAAAhFkMgsL8W8BG1Pqe+
    ums6AOsAFAHxGHl0hFvoPejzvOx0MCmzn0m4XraCy5cktGe+tSLQYWcuKRRypOQFvfWIFnpSXoaSiZ1admI
    zjjUpubk5zn+RrpcHcrLadWAACAAQAAgAMAAIAAAAAAAAAAAAAAEXIFCSm3TBoElUt4tLYDXpel4HiloPK
    7prOgDrAARgg8DYuL3Wm9CClvePrIh2WrmcgzyX4GJDJWx13WstRXmUAAQUgESTaeuySzNBslUViZH9Dex
    Q3sTi5VyGoS+K/Ina73as+ZxGQB3Ky2nVgAAgAEAAIAAAACAAAAAAAUAAAAA

Rationale

Reference implementation

The reference implementation of the PSBT format is available at https://github.com/achow101/bitcoin/tree/taproot-psbt.

Acknowledgements

TBD

```
BIP: 373
Layer: Applications
Title: MuSig2 PSBT Fields
Authors: Ava Chow <me@achow101.com>
Status: Complete
Type: Specification
Assigned: 2024-06-04
```

```
License: CC0-1.0
Requires: 32, 174, 327, 328, 370
```

BIP 373

Introduction

Abstract

This document proposes additional fields for BIP 174 PSBTv0 and BIP 370 PSBTv2 that allow for BIP 327 MuSig2 Multi-Signature data to be included in a PSBT of any version. These will be fields for the participants' keys, the public nonces, and the partial signatures produced with MuSig2.

Copyright

This BIP is licensed under the Creative Commons CC0 1.0 Universal license.

Motivation

BIP 327 specifies a way to create BIP 340 compatible public keys and signatures using the MuSig2 Multi-Signature scheme. The existing PSBT fields are unable to support MuSig2 as it introduces new concepts and additional rounds of communication. Therefore new fields must be defined to allow PSBTs to carry the information necessary to produce a valid signature with MuSig2.

Specification

The new per-input types are defined as follows:

Name	**<keytype>**	**<keydata>**	**<valuedata>**	**Versions Requiring Inclusion**
MuSig2 Participant Public Keys	PSBT_IN_MUSIG2_PARTICIPANT_PUBKEYS = 0x1a	<33 byte aggregate pubkey (compressed)>	<33 byte participant pubkey (compressed)>*	
		The MuSig2 aggregate public key (compressed) **Why the compressed aggregate public key instead of x-only?** BIP 32 public keys can be derived from a BIP 327 MuSig2 aggregate public key (see: BIP 328). But since BIP 32 requires public keys to include their evenness byte, BIP 327 MuSig2 aggregate public keys must include their evenness byte as well. Furthermore, PSBT_IN_TAP_BIP32_DERIVATION	A list of the compressed public keys of the participants in the MuSig2 aggregate key in the order required for aggregation. If sorting was done, then the	

Name	<keytype>	<keydata>	<valuedata>	Versions Requiring Inclusion
		fields include fingerprints to identify master keys, and these fingerprints require the y-coordinate of the public key, so x-only serialization can't be used. By including the aggregate key as a full public key, signers that are unaware of the MuSig2 outside of the PSBT will still be able to identify which keys are derived from the aggregate key by computing and then comparing the fingerprints. This is necessary for the signer to apply the correct tweaks to their partial signature. from the `KeyAgg` algorithm. This key may or may not appear (as x-only) in the Taproot output key, the internal key, or in a script. It may instead be a parent public key from which the Taproot output key, internal key, or keys in a script were derived.	keys must be in the sorted order.	
MuSig2 Public Nonce	`PSBT_IN_MUSIG2_PUB_NONCE = 0x1b`	`<33 byte participant pubkey (compressed)> <33 byte aggregate pubkey (compressed)> <32 byte hash or omitted>`	`<66 byte public nonce>`	
		The compressed public key of the participant providing this nonce, followed by the compressed aggregate public key the participant is providing the nonce for, followed by the BIP 341 tapleaf hash of the Taproot leaf script that will be signed. If the aggregate key is the Taproot internal key or the Taproot output key, then the tapleaf hash must be omitted. The compressed participant public key must be the Taproot output key or found in a script. It is not the internal key nor the aggregate public key that it was	The public nonce produced by the `NonceGen` algorithm.	

Name	<keytype>	<keydata>	<valuedata>	Versions Requiring Inclusion
		derived from, if it was derived from an aggregate key.		
MuSig2 Participant Partial Signature	PSBT_IN_MUSIG2_PARTIAL_SIG = 0x1c	<33 byte participant pubkey (compressed)> <33 byte aggregate pubkey (compressed)> <32 byte hash or omitted>	<32 byte partial signature>	
		The compressed public key of the participant providing this partial signature, followed by the compressed aggregate public key the participant is providing the signature for, followed by the BIP 341 tapleaf hash of the Taproot leaf script that will be signed. If the aggregate key is the Taproot internal key or the Taproot output key, then the tapleaf hash must be omitted. Note that the compressed participant public key must be the Taproot output key or found in a script. It is not the internal key nor the aggregate public key that it was derived from, if it was derived from an aggregate key.	The partial signature produced by the Sign algorithm.	

The new per-output types are defined as follows:

Name	<keytype>	<keydata>	<valuedata>	Versions Requiring Inclusion	Versions Requiring Exclusion	Versio Allowi Inclusi
MuSig2 Participant Public Keys	PSBT_OUT_MUSIG2_PARTICIPANT_PUBKEYS = 0x08	<33 byte aggregate pubkey (compressed)>	<33 byte participant pubkey (compressed)>*			0, 2
		The MuSig2 compressed aggregate public key from the KeyAgg algorithm. This key may or	A list of the compressed public keys of the participants in the MuSig2 aggregate key in the order required for			

Name	<keytype>	<keydata>	<valuedata>	Versions Requiring Inclusion	Versions Requiring Exclusion	Versio Allowi Inclusi
		may not appear (as x-only) in the Taproot output key, the internal key, or in a script. It may instead be a parent public key from which the Taproot output key, internal key, or keys in a script were derived.	aggregation. If sorting was done, then the keys must be in the sorted order.			

Roles

Updater

When an updater observes a Taproot output which involves a MuSig2 aggregate public key that it is aware of, it can add a `PSBT_IN_MUSIG2_PARTICIPANT_PUBKEYS` field containing the public keys of the participants. This aggregate public key may be directly in the script, the Taproot internal key, the Taproot output key, or a public key from which the key in the script was derived from.

An aggregate public key that appears directly in the script or internal key may be from the result of deriving child pubkeys from participant xpubs. If the updater has this derivation information, it should also add `PSBT_IN_TAP_BIP32_DERIVATION` for each participant public key.

If the public key found was derived from an aggregate public key, then all MuSig2 PSBT fields for that public key should contain the aggregate public key rather than the found pubkey itself. The updater should also add `PSBT_IN_TAP_BIP32_DERIVATION` that contains the derivation path used to derive the found pubkey from the aggregate pubkey. Derivation from the aggregate pubkey can be assumed to follow BIP 328 if there is no `PSBT_GLOBAL_XPUB` that specifies the synthetic xpub for the aggregate public key.

Updaters should add `PSBT_OUT_MUSIG2_PARTICIPANT_PUBKEYS` and `PSBT_OUT_TAP_BIP32_DERIVATION` similarly to inputs to aid in change detection.

Signer

To determine whether a signer is a participant in the MuSig2 aggregate key, the signer should first look at all `PSBT_IN_MUSIG2_PARTICIPANT_PUBKEYS` and see if any key which it knows the private key for appears as a participant in any aggregate pubkey. Signers should also check whether any of the keys in

`PSBT_IN_TAP_BIP32_DERIVATION` belong to it, and if any of those keys appear in as a participant in `PSBT_IN_MUSIG2_PARTICIPANT_PUBKEYS`.

For each aggregate public key that the signer is a participant of that it wants to produce a signature for, if the signer does not find an existing `PSBT_IN_MUSIG2_PUB_NONCE` field for its key, then it should add one using the `NonceGen` algorithm (or one of its variations) to produce a public nonce that is added in a `PSBT_IN_MUSIG2_PUB_NONCE` field. However signers must keep in mind that **improper nonce usage can compromise private keys.** Please see BIP 327 for best practices on nonce generation and usage.

Once all signers have added their `PSBT_IN_MUSIG2_PUB_NONCE` fields, each signer will perform the `NonceAgg` algorithm followed by the `Sign` algorithm in order to produce the partial signature for their key. The result will be added to the PSBT in a `PSBT_IN_MUSIG2_PARTIAL_SIG` field.

Signers must remember to apply any relevant tweaks such as a tweak that is the result of performing BIP 32 unhardened derivation with the aggregate public key as the parent key.

If all other signers have provided a `PSBT_IN_MUSIG2_PARTIAL_SIG`, then the final signer may perform the `PartialSigAgg` algorithm and produce a BIP 340 compatible signature that can be placed into a `PSBT_IN_TAP_KEY_SIG` or a `PSBT_IN_TAP_SCRIPT_SIG`.

Finalizer

A finalizer may perform the same `PartialSigAgg` step as the final signer if it has not already been done.

Otherwise, the resulting signature is a BIP 340 compatible signature and finalizers should treat it as such.

Backwards Compatibility

These are simply new fields added to the existing PSBT format. Because PSBT is designed to be extensible, old software will ignore the new fields.

Reusing `PSBT_IN_TAP_BIP32_DERIVATION` to provide derivation paths for participant public keys may cause software unaware of MuSig2 to produce a signature for that public key. This is still safe. If that public key does not directly appear in the leaf script that was signed, then the signature produced will not be useful and so cannot be replayed. If the public key does directly appear in the leaf script, then the signer will have validated the script as if it did not involve a MuSig2 and will have found it acceptable in order for it to have produced a signature. In either case, producing a signature does not give rise to the possibility of losing funds.

Test Vectors

The following are valid PSBTs

All of the following test cases use the aggregate pubkey `030b58e337aa4d3852a8c29387c42408d8cfbe3a613a5e397e0a9f01a5fb7107d4` which has the following participant keys:

1. `02346B99593357107C9D3459E9DEBA8D3EAF44E6636C85C7F853EB90BA52E8CD00`,
   `L2XJhGmS9rkNwzn1eFJVD5ydKpA5K5p54uk9qqWpURj85VkEPuNE`,
   `cStJABmHavSe7SFH2f7caQUgx3TUyXum8wtcxFyKyYP8LEqnMiEh`

2. `024fafd65f8169186fc2bfdb2233c77e630d10be280a24c7165c09a27611775c2c`,
   `L19kEzCkrce5E3v7CWKn9pTceVzFwwdorjNXfqtUc5nKLXz4qXSX`,
   `cRWjhuCcHgLLPVPNav8uX8xgGjHfcPjVvmWznGLz7CSKbGzmfvRQ`
3. `02f9308a019258c31049344f85f89d5229b531c845836f99b08601f113bce036f9`,
   `KwDiBf89QgGbjEhKnhXJuH7LrciVrZi3qYjgd9M7rFU74sHUHy8S`,
   `cMahea7zqjxrtgAbB7LSGbcQUr1uX1ojuat9jZodMN87KcLPVfXz`

- Case: Spend of a Taproot output where the output key is a MuSig2 Aggregate Pubkey
  - With participant pubkeys only
    - Bytes in Hex:
      70736274ff0100520200000000015686dff400165f4e040a5855f658093472c9bcf8108b272a5d31f181f7b4ffb1010000
    - Base64 String: cHNidP8BAFICAAAAAVaG3/
      QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
      ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN
      +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZ
      hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
      Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+Zs
      +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
      bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5AAA
  - With all pubnonces
    - Bytes in Hex:
      70736274ff0100520200000000015686dff400165f4e040a5855f658093472c9bcf8108b272a5d31f181f7b4ffb1010000
    - Base64 String: cHNidP8BAFICAAAAAVaG3/
      QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
      ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN
      +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZ
      hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
      Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+Zs
      +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
      bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5QxsC
      +OmE6Xjl+Cp8BpftxB9RCAlKbGdeHnMwEyRVIf18TQbxoWLC8dOUDbGQ9N7U6Q3G1A7f4r+MmP8s
      WX4FpGG/
      Cv9siM8d+Yw0QvigKJMcWXAmiDhF3XCwDC1jjN6pNOFKowpOHxCQI2M+
      +OmE6Xjl+Cp8BpftxB9RCA6gdlzSZ50+C0rWrE/
      Hmnp4Rpf20D1Rm72cubkzroL55A0mVk3ZxUDlO7OOE0FHEW2FcUtvhyOs2ENezRLQ7fa1tQxsC+TCI
      +OmE6Xjl+Cp8BpftxB9RCAzlJ2/8xudUCUdiCHmbMXJe346/X2Vv9x3UA/
      aiUfzrgAj8g5PC/C3bw1AzTq/5843PWaFu9d7X6KOf0n7Qp0YmwAAA=
  - With all partial signatures
    - Bytes in Hex:
      70736274ff0100520200000000015686dff400165f4e040a5855f658093472c9bcf8108b272a5d31f181f7b4ffb1010000
    - Base64 String: cHNidP8BAFICAAAAAVaG3/
      QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
      ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN
      +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZ
      hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
      Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+Zs
      +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/

bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5QxsC
+OmE6Xjl+Cp8BpftxB9RCAlKbGdeHnMwEyRVIf18TQbxoWLC8dOUDbGQ9N7U6Q3G1A7f4r+MmP8s
WX4FpGG/
Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwDC1jjN6pNOFKowpOHxCQI2M+
+OmE6Xjl+Cp8BpftxB9RCA6gdlzSZ50+C0rWrE/
Hmnp4Rpf20D1Rm72cubkzroL55A0mVk3ZxUDlO7OOE0FHEW2FcUtvhyOs2ENezRLQ7fa1tQxsC+TCI
+OmE6Xjl+Cp8BpftxB9RCAzlJ2/8xudUCUdiCHmbMXJe346/X2Vv9x3UA/
aiUfzrgAj8g5PC/C3bw1AzTq/
5843PWaFu9d7X6KOf0n7Qp0YmwQxwCNGuZWTNXEHydNFnp3rqNPq9E5mNshcf4U+uQulLozQAD
+OmE6Xjl+Cp8BpftxB9QgDlfKTKDeGjEW0/1rrxnThXLkfo/
wJOfvw5USdR5U7TFDHAJPr9ZfgWkYb8K/
2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1CByVAe
- Case: Spend of a Taproot output where the internal key is a MuSig2 Aggregate Pubkey
  - With participant pubkeys only
    - Bytes in Hex:
    70736274ff0100520200000000015818a9cd644b369c306c7fb191ec014ff625e63c283f00f9d17a959fefa3e8f6000000
    - Base64 String: cHNidP8BAFICAAAAAVgYqc1kSzacMGx/sZHsAU/2JeY8KD8A+dF6lZ/
    vo+j2AAAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgKWf
    KJbsOV+kcWz56gav6cjKjSUIhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+Zs
    +OmE6Xjl+Cp8BpftxB9QiGgMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1GMCNGuZWTN
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgN
  - With all pubnonces
    - Bytes in Hex:
    70736274ff0100520200000000015818a9cd644b369c306c7fb191ec014ff625e63c283f00f9d17a959fefa3e8f6000000
    - Base64 String: cHNidP8BAFICAAAAAVgYqc1kSzacMGx/sZHsAU/2JeY8KD8A+dF6lZ/
    vo+j2AAAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgKWf
    KJbsOV+kcWz56gav6cjKjSUIhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+Zs
    +OmE6Xjl+Cp8BpftxB9QiGgMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1GMCNGuZWTN
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgN
    hT65C6UujNAAMpZ9LQIKl5XacrUb5PP8oluw5X6RxbPnqBq/
    pyMqNJQkIDzBdIXKAcLrsOuhyAs+ra9e6cwUYp+k8QcdGYIOLQf8wCrfytRaaM+2d1nJhJzdV5Y55NG
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMpZ9LQIKl5XacrUb5PP8oluw5X6RxbPnqBq/
    pyMqNJQkICNDu4NZxDT19qnWtG91cyWPv7tWG7UhLL9YUKgqmgL34CRugyIWZEJ6ym0TwQgJostl
    pyMqNJQkIDne5CWLjf40RgCG/xcDIJ5HhDfGqw9Zii5ugJ/TCp/
    zoD4GtOBOxN5PdXyE1RrNr2yx70vMv9gQNwO8AahF3PM2UAAA==
  - With all partial signatures
    - Bytes in Hex:
    70736274ff0100520200000000015818a9cd644b369c306c7fb191ec014ff625e63c283f00f9d17a959fefa3e8f6000000

        - Base64 String: cHNidP8BAFICAAAAAVgYqc1kSzacMGx/sZHsAU/2JeY8KD8A+dF6lZ/
          vo+j2AAAAAAD9////
          ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgKWf
          KJbsOV+kcWz56gav6cjKjSUIBE0Auiae9+QhcZDjRXd8ahncqZSIiRCdukwL/
          3Z+pOxwgrlimsRpr6YsVHYWC2qhMEAF8mU2SNbE+xRipR4LGfEDiIRYLWOM3qk04UqjCk4fEJAjYz7
          bIjPHfmMNEL4oCiTHFlwJonYRd1wsBQDDJJqCIRb5MIoBkljDEEk0T4X4nVIptTHIRYNvmbCGAfETvC
          +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
          bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5QxsC
          KJbsOV+kcWz56gav6cjKjSUJCA8wXSFygHC67DrocgLPq2vXunMFGKfpPEHHRmCDi0H/
          MAq38rUWmjPtndZyYSc3VeWOeTRhnJyuZGgNj0cp2KD78QxsCT6/WX4FpGG/
          Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwDKWfS0CCpeV2nK1G+Tz/
          KJbsOV+kcWz56gav6cjKjSUJCAjQ7uDWcQ09fap1rRvdXMlj7+7Vhu1ISy/
          WFCoKpoC9+AkboMiFmRCesptE8EICaLLS1Ta6aey9at2QhPdR5pqlCQxsC+TCKAZJYwxBJNE+F+J1SK
          KJbsOV+kcWz56gav6cjKjSUJCA53uQli43+NEYAhv8XAyCeR4Q3xqsPWYouboCf0wqf86A+BrTgTsTeT3V
          YEDcDvAGoRdzzNlQxwCNGuZWTNXEHydNFnp3rqNPq9E5mNshcf4U+uQulLozQADKWfS0CCpeV
          KJbsOV+kcWz56gav6cjKjSUIgEJrTIVVyIBTdyiGuO408OpOm0KtcfhnQxpPVZcR8FiZDHAJPr9ZfgWkYl
          2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMpZ9LQIKl5XacrUb5PP8oluw5X6RxbPnqBq/
          pyMqNJQiA11e7MQE+ipjZE8wz4r0P9vYKeXNnHRwfKmzOpE0x1bkMcAvkwigGSWMMQSTRPhfidUi
          yiW7DlfpHFs+eoGr+nIyo0lCILnIVd79RGdv8/xTQq28kMLewVYk6rVbH/
          KXZTFWA9tuAAA=
- Case: Spend of a Taproot output where a key in a script is a MuSig2 Aggregate Pubkey
    - With participant pubkeys only
        - Bytes in Hex:
          70736274ff01005202000000019a8b4a50796b9600990f7fe11dfa00bc70efd296048afc86719af0fb1fa919370100000
        - Base64 String:
          cHNidP8BAFICAAAAAZqLSlB5a5YAmQ9/4R36ALxw79KWBIr8hnGa8PsfqRk3AQAAAAD9////
          ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgVr20
          +OmE6Xjl+Cp8BpftxB9SswCEWC1jjN6pNOFKowpOHxCQI2M+
          +OmE6Xjl+Cp8BpftxB9QlAbEf7apjoJVlAacwjJO1Y3Nx52E9m4reF4PUnibAbPosJoDdbiEWNGuZWTNX
          WX4FpGG/
          Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwlAbEf7apjoJVlAacwjJO1Y3Nx52E9m4reF4PUnibAbPoswyS
          ums6AOsAFAHxGHl0hFvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5JQGxH+2qY6CVZQ
          +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
          bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5QAAA
    - With all pubnonces
        - Bytes in Hex:
          70736274ff01005202000000019a8b4a50796b9600990f7fe11dfa00bc70efd296048afc86719af0fb1fa919370100000
        - Base64 String:
          cHNidP8BAFICAAAAAZqLSlB5a5YAmQ9/4R36ALxw79KWBIr8hnGa8PsfqRk3AQAAAAD9////
          ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgVr20
          +OmE6Xjl+Cp8BpftxB9SswCEWC1jjN6pNOFKowpOHxCQI2M+
          +OmE6Xjl+Cp8BpftxB9QlAbEf7apjoJVlAacwjJO1Y3Nx52E9m4reF4PUnibAbPosJoDdbiEWNGuZWTNX
          WX4FpGG/
          Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwlAbEf7apjoJVlAacwjJO1Y3Nx52E9m4reF4PUnibAbPoswyS
          ums6AOsAFAHxGHl0hFvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5JQGxH+2qY6CVZQ
          +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
          bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5YxsC

+OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LEIC2Z58hxmzrQhWa
2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjo
clIoO3JuFbBO0EuVAM4agKY8wj7MJkVV3LkWyqo53D0Nb7f4gQdHMTT03U4xmMbAvkwigGSWMMQ
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixCArHZEtRddTzu0JVEF7qYJlbSrsU/
hji9byl9rjt0PXGwAy9CRTfVmdKPFNWf4KEbgv6iqiJqKYD/2srV/asg+AaDAAA=

◦ With all partial signatures

▪ Bytes in Hex:
70736274ff01005202000000019a8b4a50796b9600990f7fe11dfa00bc70efd296048afc86719af0fb1fa919370100

▪ Base64 String:
cHNidP8BAFICAAAAAZqLSlB5a5YAmQ9/4R36ALxw79KWBIr8hnGa8PsfqRk3AQAAAAD9////
ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgVr20
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixAJmfVL2zAf+BtsxsaX37+gZA/
nL7vQPpk2vygHSyx1WQDvHUEiY5VhyVX0W0sp5vFX+8QlzhBoz7AMtiEdYyf5iIVwFCSm3TBoElUt4t
7prOgDrAIyALWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1KzAIRYLWOM3qk04UqjCk4fEJAj
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwmgN1uIRY0a5lZM1cQfJ00Weneuo0+r0TmY2yFx/
hT65C6UujNACUBsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixYCwiHIRZPr9ZfgWkYb8K/
2yIzx35jDRC+KAokxxZcCaJ2EXdcLCUBsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+izDJJqCIRZQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+
7prOgDrAARggsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwiGgMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwG
WX4FpGG/
Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgN
hT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAacwjJO1Y3Nx52
bIjPHfmMNEL4oCiTHFlwJonYRd1wsAwtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixCAucCpwdSJqDZMTp35tsQuOdC1M/
9yUig7cm4VsE7QS5UAzhqApjzCPswmRVXcuRbKqjncPQ1vt/
iBB0cxNPTdTjGYxsC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkDC1jjN6pNOFKowpC
+OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LEICsdkS1F11PO7QlU
goRuC/qKqImopgP/aytX9qyD4BoNjHAI0a5lZM1cQfJ00Weneuo0+r0TmY2yFx/
hT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAacwjJO1Y3Nx52
WX4FpGG/
Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwDC1jjN6pNOFKowpOHxCQI2M+
+OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LCAfqLfmdBwh642bI
6yobYWMcAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5AwtY4zeqTThSqMKTh8QkCNjI
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwgNmwxrWXlM/bStF5ZEPP1h1Q/
gGGRgWe/WoGLE75XkqQAAA==

• Case: Spend of a Taproot output where the internal key is derived from a MuSig2 Aggregate Pubkey

◦ With participant pubkeys only

▪ Bytes in Hex:
70736274ff0100520200000000012589e7767958ba154f9018cccf0dedea6147bb60cd1a194b6e3590a9965690d6010

▪ Base64 String:
cHNidP8BAFICAAAAASWJ53Z5WLoVT5AYzM8N7ephR7tgzRoZS241kKmWVpDWAQAAAAD9////
ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEg0LIm
mhKtsMCgIG+6KLL7TGhNvWGX2z6QhFjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0ABQBY
2yIzx35jDRC+KAokxxZcCaJ2EXdcLAUAwySagiEWjdlquFiyWcUYIYwBSkbrTmrImeUcZ173dPu2ioeZz

hT65C6UujNAAJPr9ZfgWkYb8K/
2yIzx35jDRC+KAokxxZcCaJ2EXdcLAL5MIoBkljDEEk0T4X4nVIptTHIRYNvmbCGAfETvOA2+QAA

- With all pubnonces
  - Bytes in Hex:
    70736274ff0100520200000000012589e7767958ba154f9018cccf0dedea6147bb60cd1a194b6e3590a9965690d6010
  - Base64 String:
    cHNidP8BAFICAAAAASWJ53Z5WLoVT5AYzM8N7ephR7tgzRoZS241kKmWVpDWAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEg0LIm
    mhKtsMCgIG+6KLL7TGhNvWGX2z6QhFjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0ABQBY
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLAUAwySagiEWjdlquFiyWcUYIYwBSkbrTmrImeUcZ173dPu2ioeZz
    hT65C6UujNAAJPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLAL5MIoBkljDEEk0T4X4nVIptTHIRYNvmbCGAfETvOA2+UMbAjR
    OgiQTlVI68EJivwOMJ26DKq1L+56QSFFi9XTIsmGd9b0Z24/6LrAFlJO/
    G0MbAk+v1l+BaRhvwr/
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsAtCyJsZZnyc4dN+P5oSrbDAoCBvuiiy+0xoTb1hl9s+kQgOjJKP0I
    R+12ZI2iXmJ9ukS3CFHbSdxh0EUkwAA

- With all partial signatures
  - Bytes in Hex:
    70736274ff0100520200000000012589e7767958ba154f9018cccf0dedea6147bb60cd1a194b6e3590a9965690d6010
  - Base64 String:
    cHNidP8BAFICAAAAASWJ53Z5WLoVT5AYzM8N7ephR7tgzRoZS241kKmWVpDWAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEg0LIm
    mhKtsMCgIG+6KLL7TGhNvWGX2z6QBE0CeOYl6wv/
    idSXcRg+FhP3dEf6al84uUMFIm4waTpL8wH5I22OhpMy50pdTfQwDiDg3i78njeeqGhKJldFiXMXNIRY0
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFo3ZarhYslnFGCGMAUpG605qyJnlHGde93T
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgN
    hT65C6UujNAALQsibGWZ8nOHTfj+aEq2wwKAgb7oosvtMaE29YZfbPpEICTu/J/
    dEr50dGSFxni4JolJzJI26fzoIkE5VSOvBCYr8DjCdugyqtS/
    uekEhRYvV0yLJhnfW9GduP+i6wBZSTvxtDGwJPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLALQsibGWZ8nOHTfj+aEq2wwKAgb7oosvtMaE29YZfbPpEIDoySj9
    Yd8GEoWlgJEDoI5BsmKULG5ZMv0ftdmSNol5ifbpEtwhR20ncYdBFJNDHAI0a5lZM1cQfJ00Weneuo0+
    hT65C6UujNAALQsibGWZ8nOHTfj+aEq2wwKAgb7oosvtMaE29YZfbPpCBlfbKGvhToDs4N2EtNF8TcQ
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsAtCyJsZZnyc4dN+P5oSrbDAoCBvuiiy+0xoTb1hl9s+kIOeFUvtM
    mhKtsMCgIG+6KLL7TGhNvWGX2z6Qgy8yVeGocZ01I2KxS4yLdfG6rrDv8S+ebdQ3ijfguL3MAAA==

- Case: Receiving a Taproot output where the internal key is a MuSig2 Aggregate Pubkey
  - Bytes in Hex:
    70736274ff01007d020000000012589e7767958ba154f9018cccf0dedea6147bb60cd1a194b6e3590a9965690d600000000
  - Base64 String:
    cHNidP8BAH0CAAAAASWJ53Z5WLoVT5AYzM8N7ephR7tgzRoZS241kKmWVpDWAAAAAAD9////
    AoCWmAAAAAAAIlEgKWfS0CCpeV2nK1G+Tz/
    KJbsOV+kcWz56gav6cjKjSUIPwJJ8AAAAABYAFDScXTMCeMMAKmT1l9KwGqPcG9kDAAAAAAABAH0CA
    AolcK30AAAAAFgAUz9mLoQJ+pO1L0q4bNIthVqAVA3UA4fUFAAAAACJRINCyJsZZnyc4dN+P5oSrbDAoC
    +k8InrIu3NvnYWZF/2zRgKNkhNS8gQVFlbexi//
    0SjVAAAgAEAAIAAAACAAQAAAIoCAAAAAQUgC1jjN6pNOFKowpOHxCQI2M+
    +OmE6Xjl+Cp8BpftxB9QhBwtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIQc0a5lZM1cQ

hT65C6UujNAAUAWAsIhyEHT6/WX4FpGG/
Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhB/
kwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5BQB91lWSIggDC1jjN6pNOFKowpOHxCQI2M+
+OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5ACICA75K5
RKNUAACAAQAAgAAAAIABAAAAjQIAAAA=

- Case: Receiving a Taproot output where the internal key is derived from a MuSig2 Aggregate Pubkey
  - Bytes in Hex:
    70736274ff01007d0200000001f835a5ec8e4008f96f17407e13f0c34912c39d27620800fae02baf86b8a78e760000000000f
  - Base64 String:
    cHNidP8BAH0CAAAAAfg1peyOQAj5bxdAfhPww0kSw50nYggA+uArr4a4p452AAAAAAD9////
    AoCWmAAAAAAAIlEg0LImxlmfJzh034/
    mhKtsMCgIG+6KLL7TGhNvWGX2z6SeQF0FAAAAABYAFJ+UrC20ZCC5XcDbHMj0vsC7kjTYAAAAAAABA
    QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAQEfGN31BQAAAAAWABTJEj4G6Nfwl
    RKNUAACAAQAAgAAAAIAAAAAAlwEAAAABBSCN2Wq4WLJZxRghjAFKRutOasiZ5RxnXvd0+7aKh5nC
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsBQDDJJqCIQeN2Wq4WLJZxRghjAFKRutOasiZ5RxnXvd0+7aKh5nOLw
    hT65C6UujNAAJPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLAL5MIoBkljDEEk0T4X4nVIptTHIRYNvmbCGAfETvOA2+QAiAgLOad6W
    bBHsTwXHnkXT8zhi//0SjVAAAgAEAAIAAAACAAQAAAI4CAAAA

The following are invalid PSBTs

- Case: PSBT with x-only aggregate pubkey in input participant pubkeys keydata
  - Bytes in Hex:
  - Base64 String: cHNidP8BAFICAAAAAVaG3/
    QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN6pNO
    +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZM1cQf
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5ACICA75K5
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkAAA
- Case: PSBT with an x-only input participant pubkey
  - Bytes in Hex:
    70736274ff0100520200000001568 6dff400165f4e040a5855f658093472c9bcf8108b272a5d31f181f7b4ffb10100000000fd
  - Base64 String: cHNidP8BAFICAAAAAVaG3/
    QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN6pNO
    +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZM1cQf
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5ACICA75K5
    +OmE6Xjl+Cp8BpftxB9RiNGuZWTNXEHydNFnp3rqNPq9E5mNshcf4U+uQulLozQACT6/
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkAAA

- Case: PSBT with x-only aggregate pubkey in output participant pubkeys keydata
  - Bytes in Hex:
  - Base64 String:
    cHNidP8BAH0CAAAAASWJ53Z5WLoVT5AYzM8N7ephR7tgzRoZS241kKmWVpDWAAAAAAD9////
    AoCWmAAAAAAAIlEgKWfS0CCpeV2nK1G+Tz/
    KJbsOV+kcWz56gav6cjKjSUIPwJJ8AAAAABYAFDScXTMCeMMAKmT1l9KwGqPcG9kDAAAAAAABAH0CA
    AolcK30AAAAAFgAUz9mLoQJ+pO1L0q4bNIthVqAVA3UA4fUFAAAAACJRINCyJsZZnyc4dN+P5oSrbDAoC
    +k8InrIu3NvnYWZF/2zRgKNkhNS8gQVFlbexi//
    0SjVAAAgAEAAIAAAACAAQAAAIoCAAAAAQUgC1jjN6pNOFKowpOHxCQI2M+
    +OmE6Xjl+Cp8BpftxB9QhBwtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIQc0a5lZM1cQ
    hT65C6UujNAAUAWAsIhyEHT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhB/
    kwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5BQB91lWSIQgLWOM3qk04UqjCk4fEJAjYz746YTpeC
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkAIgI
    9Eo1QAAIABAACAAAAAgAEAAACNAgAAAA==
- Case: PSBT with an x-only output participant pubkey
  - Bytes in Hex:
    70736274ff01007d02000000012589e7767958ba154f9018cccf0dedea6147bb60cd1a194b6e3590a9965690d6000000000
  - Base64 String:
    cHNidP8BAH0CAAAAASWJ53Z5WLoVT5AYzM8N7ephR7tgzRoZS241kKmWVpDWAAAAAAD9////
    AoCWmAAAAAAAIlEgKWfS0CCpeV2nK1G+Tz/
    KJbsOV+kcWz56gav6cjKjSUIPwJJ8AAAAABYAFDScXTMCeMMAKmT1l9KwGqPcG9kDAAAAAAABAH0CA
    AolcK30AAAAAFgAUz9mLoQJ+pO1L0q4bNIthVqAVA3UA4fUFAAAAACJRINCyJsZZnyc4dN+P5oSrbDAoC
    +k8InrIu3NvnYWZF/2zRgKNkhNS8gQVFlbexi//
    0SjVAAAgAEAAIAAAACAAQAAAIoCAAAAAQUgC1jjN6pNOFKowpOHxCQI2M+
    +OmE6Xjl+Cp8BpftxB9QhBwtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIQc0a5lZM1cQ
    hT65C6UujNAAUAWAsIhyEHT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhB/
    kwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5BQB91lWSIQgLWOM3qk04UqjCk4fEJAjYz746YTpeC
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkAIgI
    9Eo1QAAIABAACAAAAAgAEAAACNAgAAAA==
- Case: PSBT with x-only aggregate pubkey in public nonce keydata
  - Bytes in Hex:
    70736274ff0100520200000000015686dff400165f4e040a5855f658093472c9bcf8108b272a5d31f181f7b4ffb10100000000fd
  - Base64 String: cHNidP8BAFICAAAAAVaG3/
    QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN6pNC
    +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZM1cQf
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5QhsCNGuZ
    +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5QhsCNGuZ
    XxNBvGhYsLx05QNsZD03tTpDcbUDt/iv4yY/yz7yRU/
    hbzpnWcVgDHjmN7jfoNg+hVKIIFZDGwJPr9ZfgWkYb8K/

2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1EIDqB2XNJnn
bQPVGbvZy5uTOugvnkDSZWTdnFQOU7s44TQUcRbYVxS2+HI6zYQ17NEtDt9rW1DGwL5MIoBkljDEEk0T4
zG51QJR2IIeZsxcl7fjr9fZW/3HdQD9qJR/OuACPyDk8L8LdvDUDNOr/nzjc9ZoW713tfoo5/
SftCnRibAAAA==

- Case: PSBT with an x-only participant pubkey in public nonce keydata
  - Bytes in Hex:
    70736274ff01005202000000015686dff400165f4e040a5855f658093472c9bcf8108b272a5d31f181f7b4ffb10100000000fd
  - Base64 String: cHNidP8BAFICAAAAAVaG3/
    QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN6pNC
    +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZM1cQf
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO
    +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5Qhs0a5lZM
    hT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1EICUpsZ14eczATJFUh/
    XxNBvGhYsLx05QNsZD03tTpDcbUDt/iv4yY/yz7yRU/
    hbzpnWcVgDHjmN7jfoNg+hVKIIFZDGwJPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1EIDqB2XNJnn
    bQPVGbvZy5uTOugvnkDSZWTdnFQOU7s44TQUcRbYVxS2+HI6zYQ17NEtDt9rW1DGwL5MIoBkljDEEk0T4
    zG51QJR2IIeZsxcl7fjr9fZW/3HdQD9qJR/OuACPyDk8L8LdvDUDNOr/nzjc9ZoW713tfoo5/
    SftCnRibAAAA==

- Case: PSBT with invalid public nonce valuedata length
  - Bytes in Hex:
    70736274ff01005202000000015686dff400165f4e040a5855f658093472c9bcf8108b272a5d31f181f7b4ffb10100000000fd
  - Base64 String: cHNidP8BAFICAAAAAVaG3/
    QAFl9OBApYVfZYCTRyybz4EIsnKl0x8YH3tP+xAQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgC1jjN6pNC
    +OmE6Xjl+Cp8BpftxB9QhFgtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUBQAmgN1uIRY0a5lZM1cQf
    hT65C6UujNAAUAWAsIhyEWT6/WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwFAMMkmoIhFvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO
    +OmE6Xjl+Cp8BpftxB9RjAjRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAk+v1l+BaRhvwr/
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsAvkwigGSWMMQSTRPhfidUim1MchFg2+ZsIYB8RO84Db5Qhs0a5lZMhT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1EICUpsZ14eczATJFUh/
    XxNBvGhYsLx05QNsZD03tTpDcbUDt/iv4yY/yz7yRU/
    hbzpnWcVgDHjmN7jfoNg+hVKIIFZDGwJPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1EIDqB2XNJnn
    bQPVGbvZy5uTOugvnkDSZWTdnFQOU7s44TQUcRbYVxS2+HI6zYQ17NEtDt9rW1DGwL5MIoBkljDEEk0T4
    zG51QJR2IIeZsxcl7fjr9fZW/3HdQD9qJR/OuACPyDk8L8LdvDUDNOr/nzjc9ZoW713tfoo5/
    SftCnRibAAAA==

- Case: PSBT with x-only aggregate pubkey in partial sig keydata
  - Bytes in Hex:
    70736274ff01005202000000019a8b4a50796b9600990f7fe11dfa00bc70efd296048afc86719af0fb1fa919370100000000fd
  - Base64 String:
    cHNidP8BAFICAAAAAZqLSlB5a5YAmQ9/4R36ALxw79KWBIr8hnGa8PsfqRk3AQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgVr20gbTWd
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixAJmfVL2zAf+BtsxsaX37+gZA/
    nL7vQPpk2vygHSyx1WQDvHUEiY5VhyVX0W0sp5vFX+8QlzhBoz7AMtiEdYyf5iIVwFCSm3TBoElUt4tLYDXp
    7prOgDrAIyALWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1KzAIRYLWOM3qk04UqjCk4fEJAjYz746Y

tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwmgN1uIRY0a5lZM1cQfJ00Weneuo0+r0TmY2yFx/
hT65C6UujNACUBsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixYCwiHIRZPr9ZfgWkYb8K/
2yIzx35jDRC+KAokxxZcCaJ2EXdcLCUBsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+izDJJqCIRZQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA
7prOgDrAARggsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwiGgMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH
WX4FpGG/
Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvljGwl
hT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAacwjJO1Y3Nx52E9m4r
bIjPHfmMNEL4oCiTHFlwJonYRd1wsAwtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixCAucCpwdSJqDZMTp35tsQuOdC1M/
9yUig7cm4VsE7QS5UAzhqApjzCPswmRVXcuRbKqjncPQ1vt/
iBB0cxNPTdTjGYxsC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkDC1jjN6pNOFKowpOHxCQ
+OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LEICsdkS1F11PO7QlUQXup
goRuC/qKqImopgP/aytX9qyD4BoNiHAI0a5lZM1cQfJ00Weneuo0+r0TmY2yFx/
hT65C6UujNAAtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUsR/
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwgraeLcK+M/
6TFCGOu9RWsWKMnzVjFW60poWLmfZxBMyJjHAJPr9ZfgWkYb8K/
2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAa
DkK0ukYQgxX/
rKhthYxwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkDC1jjN6pNOFKowpOHxCQI2M+
+OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LCA2bDGtZeUz9tK0XlkQ8/
WHVD+AYZGBZ79agYsTvleSpAAA

- Case: PSBT with an x-only participant pubkey in partial sig keydata
  - Bytes in Hex:
    70736274ff01005202000000019a8b4a50796b9600990f7fe11dfa00bc70efd296048afc86719af0fb1fa919370100000000fc
  - Base64 String:
    cHNidP8BAFICAAAAAZqLSlB5a5YAmQ9/4R36ALxw79KWBIr8hnGa8PsfqRk3AQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAEBKwDh9QUAAAAAIlEgVr20gbTWc
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixAJmfVL2zAf+BtsxsaX37+gZA/
    nL7vQPpk2vygHSyx1WQDvHUEiY5VhyVX0W0sp5vFX+8QlzhBoz7AMtiEdYyf5iIVwFCSm3TBoElUt4tLYDXp
    7prOgDrAIyALWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1KzAIRYLWOM3qk04UqjCk4fEJAjYz746Y
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwmgN1uIRY0a5lZM1cQfJ00Weneuo0+r0TmY2yFx/
    hT65C6UujNACUBsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixYCwiHIRZPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLCUBsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+izDJJqCIRZQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA
    7prOgDrAARggsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwiGgMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvljGwl
    hT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAacwjJO1Y3Nx52E9m4r
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsAwtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixCAucCpwdSJqDZMTp35tsQuOdC1M/
    9yUig7cm4VsE7QS5UAzhqApjzCPswmRVXcuRbKqjncPQ1vt/
    iBB0cxNPTdTjGYxsC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkDC1jjN6pNOFKowpOHxCQ

+OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LEICsdkS1F11PO7QlUQXup
goRuC/qKqImopgP/
aytX9qyD4BoNiHDRrmVkzVxB8nTRZ6d66jT6vROZjbIXH+FPrkLpS6M0AAwtY4zeqTThSqMKTh8QkCNjPvjp
tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwgraeLcK+M/
6TFCGOu9RWsWKMnzVjFW60poWLmfZxBMyJjHAJPr9ZfgWkYb8K/
2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAa
DkK0ukYQgxX/
rKhthYxwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkDC1jjN6pNOFKowpOHxCQI2M+
+OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LCA2bDGtZeUz9tK0XlkQ8/
WHVD+AYZGBZ79agYsTvleSpAAA

- Case: PSBT with invalid partial sig valuedata length
  - Bytes in Hex:
    70736274ff01005202000000019a8b4a50796b9600990f7fe11dfa00bc70efd296048afc86719af0fb1fa919370100000000fc
  - Base64 String:
    cHNidP8BAFICAAAAAZqLSlB5a5YAmQ9/4R36ALxw79KWBIr8hnGa8PsfqRk3AQAAAAD9////
    ARjd9QUAAAAAFgAUyRI+BujX8JZsXRzQ+TMALU63V80AAAAAAAAEBKwDh9QUAAAAAIlEgVr20gbTWc
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixAJmfVL2zAf+BtsxsaX37+gZA/
    nL7vQPpk2vygHSyx1WQDvHUEiY5VhyVX0W0sp5vFX+8QlzhBoz7AMtiEdYyf5iIVwFCSm3TBoElUt4tLYDXp
    7prOgDrAIyALWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1KzAIRYLWOM3qk04UqjCk4fEJAjYz746Y
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwmgN1uIRY0a5lZM1cQfJ00Weneuo0+r0TmY2yFx/
    hT65C6UujNACUBsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixYCwiHIRZPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLCUBsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+izDJJqCIRZQkpt0waBJVLeLS2A16XpeB4paDyjsltVHv+6azoA6
    7prOgDrAARggsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+iwiGgMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH
    WX4FpGG/
    Cv9siM8d+Yw0QvigKJMcWXAmidhF3XCwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvljGwI
    hT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAacwjJO1Y3Nx52E9m4r
    bIjPHfmMNEL4oCiTHFlwJonYRd1wsAwtY4zeqTThSqMKTh8QkCNjPvjphOl45fgqfAaX7cQfUsR/
    tqmOglWUBpzCMk7Vjc3HnYT2bit4Xg9SeJsBs+ixCAucCpwdSJqDZMTp35tsQuOdC1M/
    9yUig7cm4VsE7QS5UAzhqApjzCPswmRVXcuRbKqjncPQ1vt/
    iBB0cxNPTdTjGYxsC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkDC1jjN6pNOFKowpOHxCQ
    +OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LEICsdkS1F11PO7QlUQXup
    goRuC/qKqImopgP/aytX9qyD4BoNjHAI0a5lZM1cQfJ00Weneuo0+r0TmY2yFx/
    hT65C6UujNAAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAacwjJO1Y3Nx52E9m4r
    6TFCGOu9RWsWKMnzVjFW60poWLmfZxBMyJjHAJPr9ZfgWkYb8K/
    2yIzx35jDRC+KAokxxZcCaJ2EXdcLAMLWOM3qk04UqjCk4fEJAjYz746YTpeOX4KnwGl+3EH1LEf7apjoJVlAa
    DkK0ukYQgxX/
    rKhthYxwC+TCKAZJYwxBJNE+F+J1SKbUxyEWDb5mwhgHxE7zgNvkDC1jjN6pNOFKowpOHxCQI2M+
    +OmE6Xjl+Cp8BpftxB9SxH+2qY6CVZQGnMIyTtWNzcedhPZuK3heD1J4mwGz6LCA2bDGtZeUz9tK0XlkQ8/
    WHVD+AYZGBZ79agYsTvleSpAAA

Rationale

Reference implementation

The reference implementation of the PSBT format is available at Bitcoin Core PR #31247.

Acknowledgements

Thanks to Sanket Kanjalkar whose notes on this topic formed the initial basis of this BIP. Also thanks to Pieter Wuille, Jonas Nick, Tim Ruffing, Marko Bencun, Salvatore Ingala, and all others who have participated in discussions about these fields.

Peer-to-Peer Network

Bitcoin nodes do not share a server. They share a gossip protocol. This chapter is that protocol as it exists now: how peers introduce themselves, how they keep the connection alive, how they announce blocks and transactions, and how light clients ask for less than the full chain.

The early documents are small and still in use — version and user agent, ping/pong, the mempool message, bloom filters and the service bit that opts into them, the reject message. Then the chapter moves into the performance work that followed: headers-first announcements, fee filters, SegWit inventory, compact blocks, compact client-side filters, and pruned-node service bits.

The last three are the modern network. BIP 155 lets peers advertise addresses that are not IPv4. BIP 324 encrypts the transport so a listener on the wire cannot trivially see which transactions you relay. BIP 339 relays by witness transaction id. BIP 434 is how peers negotiate optional features without inventing a new handshake every time.

Bloom filters (BIP 37) are deployed and still specified here, even though most new light clients use compact filters instead. I left them in because the service bits and the historical clients still refer to them.

In this chapter:

- BIP 14 — Protocol Version and User Agent
- BIP 31 — Pong message
- BIP 35 — mempool message
- BIP 37 — Connection Bloom filtering
- BIP 61 — Reject P2P message
- BIP 111 — NODE_BLOOM service bit
- BIP 130 — sendheaders message
- BIP 133 — feefilter message
- BIP 144 — Segregated Witness (Peer Services)
- BIP 152 — Compact Block Relay
- BIP 155 — addrv2 message
- BIP 157 — Client Side Block Filtering
- BIP 158 — Compact Block Filters for Light Clients
- BIP 159 — NODE_NETWORK_LIMITED service bit
- BIP 324 — Version 2 P2P Encrypted Transport Protocol

- BIP 339 — WTXID-based transaction relay
- BIP 434 — Peer Feature Negotiation

```
  BIP: 14
  Layer: Peer Services
  Title: Protocol Version and User Agent
  Authors: Amir Taaki <genjix@riseup.net>
           Patrick Strateman <bitcoin-bips@covertinferno.org>
  Status: Deployed
  Type: Specification
  Assigned: 2011-11-10
  Discussion: 2011-11-02: https://gnusha.org/pi/bitcoindev/
1320268981.72296.YahooMailNeo@web121003.mail.ne1.yahoo.com/
              2011-11-10: https://gnusha.org/pi/bitcoindev/
1320959761.36702.YahooMailNeo@web121014.mail.ne1.yahoo.com/
```

In this document, bitcoin will be used to refer to the protocol while Satoshi will refer to the current client in order to prevent confusion.

BIP 14

Past Situation

Bitcoin as a protocol began life with the Satoshi client. Now that the community is diversifying, a number of alternative clients with their own codebases written in a variety of languages (Java, Python, Javascript, C++) are rapidly developing their own feature-sets.

Embedded in the protocol is a version number. Primarily this version number is in the “version” and “getblocks” messages, but is also in the “block” message to indicate the software version that created that block. Currently this version number is the same version number as that of the client. This document is a proposal to separate the protocol version from the client version, together with a proposed method to do so.

Rationale

With non-separated version numbers, every release of the Satoshi client will increase its internal version number. Primarily this holds every other client hostage to a game of catch-up with Satoshi version number schemes. This plays against the decentralised nature of bitcoin, by forcing every software release to remain in step with the release schedule of one group of bitcoin developers.

Version bumping can also introduce incompatibilities and fracture the network. In order that the health of the network is maintained, the development of the protocol as a shared common collaborative process requires being split off from the implementation of that protocol. Neutral third entities to guide the protocol with representatives from all groups, present the chance for bitcoin to grow in a positive manner with minimal risks.

By using a protocol version, we set all implementations on the network to a common standard. Everybody is able to agree within their confines what is protocol and what is implementation-dependent. A user agent string is offered as a ‘vanity-plate’ for clients to distinguish themselves in the network.

Separation of the network protocol from the implementation, and forming development of said protocol by means of a mutual consensus among participants, has the democratic disadvantage when agreement is hard to

reach on contentious issues. To mitigate this issue, strong communication channels and fast release schedules are needed, and are outside the scope of this document (concerning a process-BIP type).

User agents provide extra tracking information that is useful for keeping tabs on network data such as client implementations used or common architectures/operating-systems. In the rare case they may even provide an emergency method of shunning faulty clients that threaten network health- although this is strongly unrecommended and extremely bad form. The user agent does not provide a method for clients to work around and behave differently to different implementations, as this will lead to protocol fracturing.

In short:

- Protocol version: way to distinguish between nodes and behave different accordingly.
- User agent: simple informational tool. Protocol should not be modified depending on user agent.

Browser User-Agents

RFC 1945 vaguely specifies a user agent to be a string of the product with optional comments.

```
Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.9.1.6) Gecko/20100127 Gentoo Shiretoko/3.5.6
```

User agents are most often parsed by computers more than humans. The space delimited format, does not provide an easy, fast or efficient way for parsing. The data contains no structure indicating hierarchy in this placement.

The most immediate pieces of information there are the browser product, rendering engine and the build (Gentoo Shiretoko) together with version number. Various other pieces of information as included as comments such as desktop environment, platform, language and revision number of the build.

Proposal

The version field in “version” and “getblocks” packets will become the protocol version number. The version number in the “blocks” reflects the protocol version from when that block was created.

The currently unused sub_version_num field in “version” packets will become the new user-agent string.

Bitcoin user agents are a modified browser user agent with more structure to aid parsers and provide some coherence. In bitcoin, the software usually works like a stack starting from the core code-base up to the end graphical interface. Therefore the user agent strings codify this relationship.

Basic format:

```
/Name:Version/Name:Version/.../
```

Example:

```
/Satoshi:5.64/bitcoin-qt:0.4/
/Satoshi:5.12/Spesmilo:0.8/
```

Here bitcoin-qt and Spesmilo may use protocol version 5.0, however the internal codebase they use are different versions of the same software. The version numbers are not defined to any strict format, although this guide recommends:

- Version numbers in the form of Major.Minor.Revision (2.6.41)
- Repository builds using a date in the format of YYYYMMDD (20110128)

For git repository builds, implementations are free to use the git commitish. However the issue lies in that it is not immediately obvious without the repository which version precedes another. For this reason, we lightly recommend dates in the format specified above, although this is by no means a requirement.

Optional -r1, -r2, … can be appended to user agent version numbers. This is another light recommendation, but not a requirement. Implementations are free to specify version numbers in whatever format needed insofar as it does not include (,), : or / to interfere with the user agent syntax.

An optional comments field after the version number is also allowed. Comments should be delimited by brackets (…). The contents of comments is entirely implementation defined although this BIP recommends the use of semi-colons ; as a delimiter between pieces of information.

Example:

```
/BitcoinJ:0.2(iPad; U; CPU OS 3_2_1)/AndroidBuild:0.8/
```

Reserved symbols are therefore: / : ( )

They should not be misused beyond what is specified in this section.

- / separates the code-stack
- specifies the implementation version of the particular stack
- (and) delimits a comment which optionally separates data using ;

Timeline

When this document was published, the bitcoin protocol and Satoshi client versions were currently at 0.5 and undergoing changes. In order to minimise disruption and allow the undergoing changes to be completed, the next protocol version at 0.6 became peeled from the client version (also at 0.6). As of that time (January 2012), protocol and implementation version numbers are distinct from each other.

```
BIP: 31
Layer: Peer Services
Title: Pong message
Authors: Mike Hearn <hearn@google.com>
Status: Deployed
Type: Specification
Assigned: 2012-04-11
```

BIP 31

Abstract

This document describes a trivial protocol extension that makes it easier for clients to detect dead peer connections.

Motivation

Today there are a few network related problems that can degrade the Bitcoin user experience:

1) Some Bitcoin clients run on platforms that can go to sleep and essentially stop running at any time without warning. Notably, this is very common on both mobiles and laptops (shut the lid). When the system comes back, TCP connections that existed before the sleep still exist but may no longer function correctly, eg, because the IP address has changed, or because the remote peer went away or the connection was timed out by some other system. Currently it can often take a while to notice this has happened.

2) The reference Satoshi client is largely single threaded and when placed under heavy load (e.g., because it is downloading the block chain) becomes very slow to respond to network messages. There's no easy way to detect this has occurred, especially if you are just passively waiting for broadcasts from that peer. A way to detect overloaded remote peers and avoid them would both help balance load and provide a better, more responsive system.

3) When downloading large data structures like the block chain it is efficient to choose a peer that is near to you network-wise, in order to reduce load on often congested trans-national links and ensure lower latency. Currently it is difficult to measure the latency to a remote peer so clients don't bother, and instead just select a random peer to download from.

All of these can be solved by a backwards compatible protocol modification.

Specification

When the protocol version as negotiated in the "ver" message is greater than 60000, the "ping" message must contain a uint64 field called "nonce". A peer sending "ping" should set the nonce to a random value, and it is then echoed back by the recipient in a new "pong" message that also contains a single uint64 field.

In this way, the client can send a ping and measure the time taken to receive the corresponding pong. If a client sends two pings before hearing back the first pong, the responses can be distinguished using the nonce. If the client chooses to never overlap pings in this way it should simply set the nonce value to zero.

Backward compatibility

Clients must opt-in to the new feature by advertising a protocol version > 60000. Clients with older protocol versions are not expected to provide a nonce in the ping message and will not be sent a pong.

Implementation

https://github.com/bitcoin/bitcoin/pull/932/files

```
BIP: 35
Layer: Peer Services
Title: mempool message
Authors: Jeff Garzik <jgarzik@exmulti.com>
Status: Deployed
Type: Specification
Assigned: 2012-08-16
```

BIP 35

Abstract

Make a network node's transaction memory pool accessible via a new "mempool" message. Extend the existing "getdata" message behavior to permit accessing the transaction memory pool.

Motivation

Several use cases make it desirable to expose a network node's transaction memory pool:

1. SPV clients, wishing to obtain zero-confirmation transactions sent or received.
2. Miners, to avoid missing lucrative fees, downloading existing network transactions after a restart.
3. Remote network diagnostics.

Specification

1. The mempool message is defined as an empty message where pchCommand == "mempool"
2. Upon receipt of a "mempool" message, the node will respond with an "inv" message containing MSG_TX hashes of all the transactions in the node's transaction memory pool, if any.
3. The typical node behavior in response to an "inv" is "getdata". However, the reference Satoshi implementation ignores requests for transaction hashes outside that which is recently relayed. To support "mempool", an implementation must extend its "getdata" message support to querying the memory pool.
4. Feature discovery is enabled by checking two "version" message attributes:
    1. Protocol version >= 60002
    2. NODE_NETWORK bit set in nServices

Note that existing implementations drop "inv" messages with a vector size > 50000.

Backward compatibility

Older clients remain 100% compatible and interoperable after this change.

Implementation

https://github.com/bitcoin/bitcoin/pull/1641

```
BIP: 37
Layer: Peer Services
Title: Connection Bloom filtering
Authors: Mike Hearn <hearn@google.com>
```

Matt Corallo <bip37@bluematt.me>
Comments-Summary: No comments yet.
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0037
Status: Deployed
Type: Specification
Assigned: 2012-10-24
License: PD

BIP 37

Abstract

This BIP adds new support to the peer-to-peer protocol that allows peers to reduce the amount of transaction data they are sent. Peers have the option of setting *filters* on each connection they make after the version handshake has completed. A filter is defined as a Bloom filter on data derived from transactions. A Bloom filter is a probabilistic data structure which allows for testing set membership - they can have false positives but not false negatives.

This document will not go into the details of how Bloom filters work and the reader is referred to Wikipedia for an introduction to the topic.

Motivation

As Bitcoin grows in usage the amount of bandwidth needed to download blocks and transaction broadcasts increases. Clients implementing *simplified payment verification* do not attempt to fully verify the block chain, instead just checking that block headers connect together correctly and trusting that the transactions in a chain of high difficulty are in fact valid. See the Bitcoin paper for more detail on this mode.

Today, SPV clients have to download the entire contents of blocks and all broadcast transactions, only to throw away the vast majority of the transactions that are not relevant to their wallets. This slows down their synchronization process, wastes users bandwidth (which on phones is often metered) and increases memory usage. All three problems are triggering real user complaints for the Android "Bitcoin Wallet" app which implements SPV mode. In order to make chain synchronization fast, cheap and able to run on older phones with limited memory we want to have remote peers throw away irrelevant transactions before sending them across the network.

Design rationale

The most obvious way to implement the stated goal would be for clients to upload lists of their keys to the remote node. We take a more complex approach for the following reasons:

- Privacy: Because Bloom filters are probabilistic, with the false positive rate chosen by the client, nodes can trade off precision vs bandwidth usage. A node with access to lots of bandwidth may choose to have a high FP rate, meaning the remote peer cannot accurately know which transactions belong to the client and which don't. A node with very little bandwidth may choose to use a very accurate filter meaning that they only get sent transactions actually relevant to their wallet, but remote peers may be able to correlate transactions with IP addresses (and each other).
- Bloom filters are compact and testing membership in them is fast. This results in satisfying performance characteristics with minimal risk of opening up potential for DoS attacks.

Specification

New messages

We start by adding three new messages to the protocol:

- `filterload`, which sets the current Bloom filter on the connection
- `filteradd`, which adds the given data element to the connections current filter without requiring a completely new one to be set
- `filterclear`, which deletes the current filter and goes back to regular pre-BIP37 usage.

Note that there is no filterremove command because by their nature, Bloom filters are append-only data structures. Once an element is added it cannot be removed again without rebuilding the entire structure from scratch.

The `filterload` command is defined as follows:

Field Size	Description	Data type	Comments
?	filter	uint8_t[]	The filter itself is simply a bit field of arbitrary byte-aligned size. The maximum size is 36,000 bytes.
4	nHashFuncs	uint32_t	The number of hash functions to use in this filter. The maximum value allowed in this field is 50.
4	nTweak	uint32_t	A random value to add to the seed value in the hash function used by the bloom filter.
1	nFlags	uint8_t	A set of flags that control how matched items are added to the filter.

See below for a description of the Bloom filter algorithm and how to select nHashFuncs and filter size for a desired false positive rate.

Upon receiving a `filterload` command, the remote peer will immediately restrict the broadcast transactions it announces (in inv packets) to transactions matching the filter, where the matching algorithm is specified below. The flags control the update behaviour of the matching algorithm.

The `filteradd` command is defined as follows:

Field Size	Description	Data type	Comments
?	data	uint8_t[]	

Field Size	Description	Data type	Comments
			The data element to add to the current filter.

The data field must be smaller than or equal to 520 bytes in size (the maximum size of any potentially matched object).

The given data element will be added to the Bloom filter. A filter must have been previously provided using `filterload`. This command is useful if a new key or script is added to a clients wallet whilst it has connections to the network open, it avoids the need to re-calculate and send an entirely new filter to every peer (though doing so is usually advisable to maintain anonymity).

The `filterclear` command has no arguments at all.

After a filter has been set, nodes don't merely stop announcing non-matching transactions, they can also serve filtered blocks. A filtered block is defined by the `merkleblock` message and is defined like this:

Field Size	Description	Data type	Comments
4	version	uint32_t	Block version information, based upon the software version creating this block
32	prev_block	char[32]	The hash value of the previous block this particular block references
32	merkle_root	char[32]	The reference to a Merkle tree collection which is a hash of all transactions related to this block
4	timestamp	uint32_t	A timestamp recording when this block was created (Limited to 2106!)
4	bits	uint32_t	The calculated difficulty target being used for this block
4	nonce	uint32_t	The nonce used to generate this block… to allow variations of the header and compute different hashes
4	total_transactions	uint32_t	Number of transactions in the block (including unmatched ones)
?	hashes	uint256[]	

Field Size	Description	Data type	Comments
			hashes in depth-first order (including standard varint size prefix)
?	flags	byte[]	flag bits, packed per 8 in a byte, least significant bit first (including standard varint size prefix)

See below for the format of the partial merkle tree hashes and flags.

Thus, a `merkleblock` message is a block header, plus a part of a merkle tree which can be used to extract identifying information for transactions that matched the filter and prove that the matching transaction data really did appear in the solved block. Clients can use this data to be sure that the remote node is not feeding them fake transactions that never appeared in a real block, although lying through omission is still possible.

Extensions to existing messages

The `version` command is extended with a new field:

Field Size	Description	Data type	Comments
1 byte	fRelay	bool	If false then broadcast transactions will not be announced until a filter{load,add,clear} command is received. If missing or true, no change in protocol behaviour occurs.

SPV clients that wish to use Bloom filtering would normally set fRelay to false in the version message, then set a filter based on their wallet (or a subset of it, if they are overlapping different peers). Being able to opt-out of inv messages until the filter is set prevents a client being flooded with traffic in the brief window of time between finishing version handshaking and setting the filter.

The `getdata` command is extended to allow a new type in the `inv` submessage. The type field can now be `MSG_FILTERED_BLOCK (== 3)` rather than `MSG_BLOCK`. If no filter has been set on the connection, a request for filtered blocks is ignored. If a filter has been set, a `merkleblock` message is returned for the requested block hash. In addition, because a `merkleblock` message contains only a list of transaction hashes, transactions matching the filter should also be sent in separate tx messages after the merkleblock is sent. This avoids a slow roundtrip that would otherwise be required (receive hashes, didn't see some of these transactions yet, ask for them). Note that because there is currently no way to request transactions which are already in a block from a node (aside from requesting the full block), the set of matching transactions that the requesting node hasn't either received or announced with an inv must be sent and any additional transactions which match the filter may also be sent. This allows for clients (such as the reference client) to limit the number of invs it must

remember a given node to have announced while still providing nodes with, at a minimum, all the transactions it needs.

Filter matching algorithm

The filter can be tested against arbitrary pieces of data, to see if that data was inserted by the client. Therefore the question arises of what pieces of data should be inserted/tested.

To determine if a transaction matches the filter, the following algorithm is used. Once a match is found the algorithm aborts.

1. Test the hash of the transaction itself.
2. For each output, test each data element of the output script. This means each hash and key in the output script is tested independently. **Important:** if an output matches whilst testing a transaction, the node might need to update the filter by inserting the serialized COutPoint structure. See below for more details.
3. For each input, test the serialized COutPoint structure.
4. For each input, test each data element of the input script (note: input scripts only ever contain data elements).
5. Otherwise there is no match.

In this way addresses, keys and script hashes (for P2SH outputs) can all be added to the filter. You can also match against classes of transactions that are marked with well known data elements in either inputs or outputs, for example, to implement various forms of Smart property.

The test for outpoints is there to ensure you can find transactions spending outputs in your wallet, even though you don't know anything about their form. As you can see, once set on a connection the filter is **not static** and can change throughout the connections lifetime. This is done to avoid the following race condition:

1. A client sets a filter matching a key in their wallet. They then start downloading the block chain. The part of the chain that the client is missing is requested using getblocks.
2. The first block is read from disk by the serving peer. It contains TX 1 which sends money to the clients key. It matches the filter and is thus sent to the client.
3. The second block is read from disk by the serving peer. It contains TX 2 which spends TX 1. However TX 2 does not contain any of the clients keys and is thus not sent. The client does not know the money they received was already spent.

By updating the bloom filter atomically in step 2 with the discovered outpoint, the filter will match against TX 2 in step 3 and the client will learn about all relevant transactions, despite that there is no pause between the node processing the first and second blocks.

The nFlags field of the filter controls the nodes precise update behaviour and is a bit field.

- `BLOOM_UPDATE_NONE (0)` means the filter is not adjusted when a match is found.
- `BLOOM_UPDATE_ALL (1)` means if the filter matches any data element in a scriptPubKey the outpoint is serialized and inserted into the filter.
- `BLOOM_UPDATE_P2PUBKEY_ONLY (2)` means the outpoint is inserted into the filter only if a data element in the scriptPubKey is matched, and that script is of the standard "pay to pubkey" or "pay to multisig" forms.

These distinctions are useful to avoid too-rapid degradation of the filter due to an increasing false positive rate. We can observe that a wallet which expects to receive only payments of the standard pay-to-address form doesn't need automatic filter updates because any transaction that spends one of its own outputs has a predictable data element in the input (the pubkey that hashes to the address). If a wallet might receive pay-to-address outputs and also pay-to-pubkey or pay-to-multisig outputs then BLOOM_UPDATE_P2PUBKEY_ONLY is appropriate, as it avoids unnecessary expansions of the filter for the most common types of output but still ensures correct behaviour with payments that explicitly specify keys.

Obviously, nFlags == 1 or nFlags == 2 mean that the filter will get dirtier as more of the chain is scanned. Clients should monitor the observed false positive rate and periodically refresh the filter with a clean one.

Partial Merkle branch format

A *Merkle tree* is a way of arranging a set of items as leaf nodes of tree in which the interior nodes are hashes of the concatenations of their child hashes. The root node is called the *Merkle root*. Every Bitcoin block contains a Merkle root of the tree formed from the blocks transactions. By providing some elements of the trees interior nodes (called a *Merkle branch*) a proof is formed that the given transaction was indeed in the block when it was being mined, but the size of the proof is much smaller than the size of the original block.

Constructing a partial merkle tree object

- Traverse the merkle tree from the root down, and for each encountered node:
  - Check whether this node corresponds to a leaf node (transaction) that is to be included OR any parent thereof:
    - If so, append a '1' bit to the flag bits
    - Otherwise, append a '0' bit.
  - Check whether this node is an internal node (non-leaf) AND is the parent of an included leaf node:
    - If so:
      - Descend into its left child node, and process the subtree beneath it entirely (depth-first).
      - If this node has a right child node too, descend into it as well.
    - Otherwise: append this node's hash to the hash list.

Parsing a partial merkle tree object

As the partial block message contains the number of transactions in the entire block, the shape of the merkle tree is known before hand. Again, traverse this tree, computing traversed node's hashes along the way:

- Read a bit from the flag bit list:
  - If it is '0':
    - Read a hash from the hashes list, and return it as this node's hash.
  - If it is '1' and this is a leaf node:
    - Read a hash from the hashes list, store it as a matched txid, and return it as this node's hash.
  - If it is '1' and this is an internal node:
    - Descend into its left child tree, and store its computed hash as L.

        - If this node has a right child as well:
            - Descend into its right child, and store its computed hash as R.
            - If L == R, the partial merkle tree object is invalid.
            - Return Hash(L || R).
        - If this node has no right child, return Hash(L || L).

The partial merkle tree object is only valid if:

- All hashes in the hash list were consumed and no more.
- All bits in the flag bits list were consumed (except padding to make it into a full byte), and no more.
- The hash computed for the root node matches the block header's merkle root.
- The block header is valid, and matches its claimed proof of work.
- In two-child nodes, the hash of the left and right branches was never equal.

Bloom filter format

A Bloom filter is a bit-field in which bits are set based on feeding the data element to a set of different hash functions. The number of hash functions used is a parameter of the filter. In Bitcoin we use version 3 of the 32-bit Murmur hash function. To get N "different" hash functions we simply initialize the Murmur algorithm with the following formula:

```
nHashNum * 0xFBA4C795 + nTweak
```

i.e. if the filter is initialized with 4 hash functions and a tweak of 0x00000005, when the second function (index 1) is needed h1 would be equal to 4221880218.

When loading a filter with the `filterload` command, there are two parameters that can be chosen. One is the size of the filter in bytes. The other is the number of hash functions to use. To select the parameters you can use the following formulas:

Let N be the number of elements you wish to insert into the set and P be the probability of a false positive, where 1.0 is "match everything" and zero is unachievable.

The size S of the filter in bytes is given by `(-1 / pow(log(2), 2) * N * log(P)) / 8`. Of course you must ensure it does not go over the maximum size (36,000: selected as it represents a filter of 20,000 items with false positive rate of < 0.1% or 10,000 items and a false positive rate of < 0.0001%).

The number of hash functions required is given by `S * 8 / N * log(2)`.

Copyright

This document is placed in the public domain.

```
BIP: 61
Layer: Peer Services
Title: Reject P2P message
Authors: Gavin Andresen <gavinandresen@gmail.com>
Comments-Summary: Controversial; some recommendation, and some discouragement
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0061
Status: Deployed
```

```
Type: Specification
Assigned: 2014-06-18
```

BIP 61

Abstract

This BIP describes a new message type for the Bitcoin peer-to-peer network.

Motivation

Giving peers feedback about why their blocks or transactions are rejected, or why they are being banned for not following the protocol helps interoperability between different implementations.

It also gives SPV (simplified payment verification) clients a hint that something may be wrong when their transactions are rejected due to insufficient priority or fees.

Specification

Data types in this specification are as described at https://en.bitcoin.it/wiki/Protocol_specification

reject

One new message type "reject" is introduced. It is sent directly to a peer in response to a "version", "tx" or "block" message.

For example, the message flow between two peers for a relayed transaction that is rejected for some reason would be:

```
--> inv
<-- getdata
--> tx
<-- reject
```

All implementations of the P2P protocol version 70,002 and later should support the reject message.

common payload

Every reject message begins with the following fields. Some messages append extra, message-specific data.

Field Size	Name	Data type	Comments
variable	response-to-msg	var_str	Message that triggered the reject
1	reject-code	uint8_t	0x01 through 0x4f (see below)
variable	reason	var_string	Human-readable message for debugging

The human-readable string is intended only for debugging purposes; in particular, different implementations may use different strings. The string should not be shown to users or used for anything besides diagnosing interoperability problems.

The following reject code categories are used; in the descriptions below, “server” is the peer generating the reject message, “client” is the peer that will receive the message.

Range	Category
0x01-0x0f	Protocol syntax errors
0x10-0x1f	Protocol semantic errors
0x40-0x4f	Server policy rule

rejection codes common to all message types

Code	Description
0x01	Message could not be decoded

reject version codes

Codes generated during the initial connection process in response to a “version” message:

Code	Description
0x11	Client is an obsolete, unsupported version
0x12	Duplicate version message received

reject tx payload, codes

Transaction rejection messages extend the basic message with the transaction id hash:

Field Size	Name	Data type	Comments
32	hash	char[32]	transaction that is rejected

The following codes are used:

Code	Description
0x10	Transaction is invalid for some reason (invalid signature, output value greater than input, etc.)
0x12	An input is already spent
0x40	Not mined/relayed because it is “non-standard” (type or version unknown by the server)
0x41	One or more output amounts are below the ‘dust’ threshold
0x42	Transaction does not have enough fee/priority to be relayed or mined

payload, reject block

Block rejection messages extend the basic message with the block header hash:

Field Size	Name	Data type	Comments
32	hash	char[32]	block (hash of block header) that is rejected

Rejection codes:

code	description
0x10	Block is invalid for some reason (invalid proof-of-work, invalid signature, etc)
0x11	Block's version is no longer supported
0x43	Inconsistent with a compiled-in checkpoint

Note: blocks that are not part of the server's idea of the current best chain, but are otherwise valid, should not trigger reject messages.

Compatibility

The reject message is backwards-compatible; older peers that do not recognize the reject message will ignore it.

Implementation notes

Implementers must consider what happens if an attacker either sends them reject messages for valid transactions/blocks or sends them random reject messages, and should beware of possible denial-of-service attacks. For example, notifying the user of every reject message received would make it trivial for an attacker to mount an annoy-the-user attack. Even merely writing every reject message to a debugging log could make an implementation vulnerable to a fill-up-the-users-disk attack.

```
BIP: 111
Layer: Peer Services
Title: NODE_BLOOM service bit
Authors: Matt Corallo <bip111@bluematt.me>
         Peter Todd <pete@petertodd.org>
Status: Deployed
Type: Specification
Assigned: 2015-08-20
License: PD
```

BIP 111

Abstract

This BIP extends BIP 37, Connection Bloom filtering, by defining a service bit to allow peers to advertise that they support bloom filters explicitly. It also bumps the protocol version to allow peers to identify old nodes which allow bloom filtering of the connection despite lacking the new service bit.

Motivation

BIP 37 did not specify a service bit for the bloom filter service, thus implicitly assuming that all nodes that serve peers data support it. However, the connection filtering algorithm proposed in BIP 37, and implemented in several clients today, has been shown to provide little to no privacyhttp://eprint.iacr.org/2014/763, as well as being a large DoS risk on some nodes1 is one example where the issues were found, though others independently discovered issues as well. Sample DoS exploit code available at https://github.com/petertodd/bloom-io-attack.. Thus, allowing node operators to disable connection bloom filtering is a much-needed feature.

Specification

The following protocol bit is added:

```
NODE_BLOOM = (1 << 2)
```

Nodes which support bloom filters should set that protocol bit. Otherwise it should remain unset. In addition the protocol version is increased from 70002 to 70011 in the reference implementation. It is often the case that nodes which have a protocol version smaller than 70011, but larger than 70000 support bloom filtered connections without the NODE_BLOOM bit set, however clients which require bloom filtered connections should avoid making this assumption.

NODE_BLOOM is distinct from NODE_NETWORK, and it is legal to advertise NODE_BLOOM but not NODE_NETWORK (though there is little reason to do so now, some proposals may make this more useful in the future)

If a node does not support bloom filters but receives a “filterload”, “filteradd”, or “filterclear” message from a peer the node should disconnect that peer immediately. For backwards compatibility, in initial implementations, nodes may choose to only disconnect nodes which have the new protocol version set and attempt to send a filter command.

While outside the scope of this BIP it is suggested that DNS seeds and other peer discovery mechanisms support the ability to specify the services required; current implementations simply check only that NODE_NETWORK is set.

Design rational

A service bit was chosen as applying a bloom filter is a service.

The increase in protocol version is for backwards compatibility. In initial implementations, old nodes which are not yet aware of NODE_BLOOM and use a protocol version < 70011 may still send filter messages to a node without NODE_BLOOM. This feature may be removed after there are sufficient NODE_BLOOM nodes available and SPV clients have upgraded, allowing node operators to fully close the bloom-related DoS vectors.

Reference Implementation

https://github.com/bitcoin/bitcoin/pull/6579

Copyright

This document is placed in the public domain.

References

```
BIP: 130
Layer: Peer Services
Title: sendheaders message
Authors: Suhas Daftuar <sdaftuar@chaincode.com>
Status: Deployed
Type: Specification
Assigned: 2015-05-08
License: PD
```

BIP 130

Abstract

Add a new message, “sendheaders”, which indicates that a node prefers to receive new block announcements via a “headers” message rather than an “inv”.

Motivation

Since the introduction of “headers-first” downloading of blocks in 0.10, blocks will not be processed unless they are able to connect to a (valid) headers chain. Consequently, block relay generally works as follows:

1. A node (N) announces the new tip with an “inv” message, containing the block hash
2. A peer (P) responds to the “inv” with a “getheaders” message (to request headers up to the new tip) and a “getdata” message for the new tip itself
3. N responds with a “headers” message (with the header for the new block along with any preceding headers unknown to P) and a “block” message containing the new block

However, in the case where a new block is being announced that builds on the tip, it would be generally more efficient if the node N just announced the block header for the new block, rather than just the block hash, and saved the peer from generating and transmitting the getheaders message (and the required block locator).

In the case of a reorg, where 1 or more blocks are disconnected, nodes currently just send an “inv” for the new tip. Peers currently are able to request the new tip immediately, but wait until the headers for the intermediate blocks are delivered before requesting those blocks. By announcing headers from the last fork point leading up to the new tip in the block announcement, peers are able to request all the intermediate blocks immediately.

Specification

1. The sendheaders message is defined as an empty message where pchCommand == “sendheaders”
2. Upon receipt of a “sendheaders” message, the node will be permitted, but not required, to announce new blocks by sending the header of the new block (along with any other blocks that a node believes a peer might need in order for the block to connect).
3. Feature discovery is enabled by checking protocol version >= 70012

Additional constraints

As support for sendheaders is optional, software that implements this may also optionally impose additional constraints, such as only honoring sendheaders messages shortly after a connection is established.

Backward compatibility

Older clients remain fully compatible and interoperable after this change.

Implementation

https://github.com/bitcoin/bitcoin/pull/6494

Copyright

This document is placed in the public domain.

```
BIP: 133
Layer: Peer Services
Title: feefilter message
Authors: Alex Morcos <morcos@chaincode.com>
Status: Deployed
Type: Specification
Assigned: 2016-02-13
License: PD
```

BIP 133

Abstract

Add a new message, “feefilter”, which serves to instruct peers not to send “inv”’s to the node for transactions with fees below the specified fee rate.

Motivation

The concept of a limited mempool was introduced in Bitcoin Core 0.12 to provide protection against attacks or spam transactions of low fees that are not being mined. A reject filter was also introduced to help prevent repeated requests for the same transaction that might have been recently rejected for insufficient fee. These methods help keep resource utilization on a node from getting out of control.

However, there are limitations to the effectiveness of these approaches. The reject filter is reset after every block which means transactions that are inv’ed over a longer time period will be rerequested and there is no method to prevent requesting the transaction the first time. Furthermore, inv data is sent at least once either to or from each peer for every transaction accepted to the mempool and there is no mechanism by which to know that an inv sent to a given peer would not result in a getdata request because it represents a transaction with too little fee.

After receiving a feefilter message, a node can know before sending an inv that a given transaction's fee rate is below the minimum currently required by a given peer, and therefore the node can skip relaying an inv for that transaction to that peer.

Specification

1. The feefilter message is defined as a message containing an int64_t where pchCommand == "feefilter"
2. Upon receipt of a "feefilter" message, the node will be permitted, but not required, to filter transaction invs for transactions that fall below the feerate provided in the feefilter message interpreted as satoshis per kilobyte.
3. The fee filter is additive with a bloom filter for transactions so if an SPV client were to load a bloom filter and send a feefilter message, transactions would only be relayed if they passed both filters.
4. Inv's generated from a mempool message are also subject to a fee filter if it exists.
5. Feature discovery is enabled by checking protocol version >= 70013

Considerations

The propagation efficiency of transactions across the network should not be adversely affected by this change. In general, transactions which are not accepted to a node's mempool are not relayed by this node and the functionality implemented with this message is meant only to filter those transactions. There could be a small number of edge cases where a node's mempool min fee is actually less than the filter value a peer is aware of and transactions with fee rates between these values will now be newly inhibited.

Feefilter messages are not sent to whitelisted peers if the "-whitelistforcerelay" option is set. In that case, transactions are intended to be relayed even if they are not accepted to the mempool.

There are privacy concerns with deanonymizing a node by the fact that it is broadcasting identifying information about its mempool min fee. To help ameliorate this concern, the implementation quantizes the filter value broadcast with a small amount of randomness, in addition, the messages are broadcast to different peers at individually randomly distributed times.

If a node is using prioritisetransaction to accept transactions whose actual fee rates might fall below the node's mempool min fee, it may want to consider disabling the fee filter to make sure it is exposed to all possible txid's.

Backward compatibility

Older clients remain fully compatible and interoperable after this change. Also, clients implementing this BIP can choose to not send any feefilter messages.

Implementation

https://github.com/bitcoin/bitcoin/pull/7542

Copyright

This document is placed in the public domain.

```
BIP: 144
Layer: Peer Services
Title: Segregated Witness (Peer Services)
Authors: Eric Lombrozo <elombrozo@gmail.com>
         Pieter Wuille <pieter.wuille@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2016-01-08
License: PD
```

BIP 144

Abstract

This BIP defines new messages and serialization formats for propagation of transactions and blocks committing to segregated witness structures.

Motivation

In addition to defining witness structures and requiring commitments in future blocks (BIP141 - Consensus segwit BIP), new mechanisms must be defined to allow peers to advertise support for segregated witness and to relay the witness structures and request them from other peers without breaking compatibility with older nodes.

Specification

Serialization

A new serialization format for tx messages is added to the peer-to-peer protocol.

The serialization has the following structure:

Field Size	Name	Type	Description
4	version	int32_t	Transaction data format version
1	marker	char	Must be zero
1	flag	char	Must be nonzero
1+	txin_count	var_int	Number of transaction inputs
41+	txins	txin[]	A list of one or more transaction inputs
1+	txout_count	var_int	Number of transaction outputs
9+	txouts	txouts[]	A list of one or more transaction outputs
1+	script_witnesses	script_witnesses[]	

Field Size	Name	Type	Description
			The witness structure as a serialized byte array
4	lock_time	uint32_t	The block number or timestamp until which the transaction is locked

Parsers supporting this BIP will be able to distinguish between the old serialization format (without the witness) and this one. The marker byte is set to zero so that this structure will never parse as a valid transaction in a parser that does not support this BIP. If parsing were to succeed, such a transaction would contain no inputs and a single output.

If the witness is empty, the old serialization format must be used.

Currently, the only witness objects type supported are script witnesses which consist of a stack of byte arrays. It is encoded as a var_int item count followed by each item encoded as a var_int length followed by a string of bytes. Each txin has its own script witness. The number of script witnesses is not explicitly encoded as it is implied by txin_count. Empty script witnesses are encoded as a zero byte. The order of the script witnesses follows the same order as the associated txins.

- **Rationale for not having an independent message type with its own serialization**: this would require separate "tx" and "block" messages, and all RPC calls operating on raw transactions would need to be duplicated, or need inefficient or nondeterministic guesswork to know which type is to be used.
- **Rationale for not using just a single 0x00 byte as marker**: that would lead to empty transactions (no inputs, no outputs, which are used in some tests) to be interpreted as new serialized data.
- **Rationale for the 0x01 flag byte in between**: this will allow us to easily add more extra non-committed data to transactions (like txouts being spent, …). It can be interpreted as a bitvector.

Handshake

A node will signal that it can provide witnesses using the following service bit

```
NODE_WITNESS = (1 << 3)
```

Hashes

Transaction hashes used in the transaction merkle tree and txin outpoints are always computed using the old non-witness serialization.

Support for a new hash including the witness data is added that is computed from the new witness serialization. (Note that transactions with an empty witness always use the old serialization, and therefore, they have witness hash equal to normal hash.)

<img src=bip-0144/witnesstx.png></img>

Relay

New inv types MSG_WITNESS_TX (0x40000001, or (1<<30)+MSG_TX) and MSG_WITNESS_BLOCK (0x40000002, or (1<<30)+MSG_BLOCK) are added, only for use in getdata. Inventory messages themselves still use just MSG_TX and MSG_BLOCK, similar to MSG_FILTERED_BLOCK. A further inv type MSG_FILTERED_WITNESS_BLOCK (0x40000003, or (1<<30)+MSG_FILTERED_BLOCK) is reserved for future use.

- **Rationale for not advertizing witnessness in invs**: we don't always use invs anymore (with 'sendheaders' BIP 130), plus it's not useful: implicitly, every transaction and block have a witness, old ones just have empty ones.

MSG_WITNESS_TX getdata requests should use the non-witness serialized hash. The peer shall respond with a tx message, and if the witness structure is nonempty, the witness serialization shall be used.

MSG_WITNESS_BLOCK requests will return a block message with transactions that have a witness using witness serialization.

Credits

Special thanks to Gregory Maxwell for originating many of the ideas in this BIP and Luke-Jr for figuring out how to deploy this as a soft fork.

Reference Implementation

https://github.com/bitcoin/bitcoin/pull/8149

Copyright

This document is placed in the public domain.

```
BIP: 152
Layer: Peer Services
Title: Compact Block Relay
Authors: Matt Corallo <bip152@bluematt.me>
Comments-Summary: Unanimously Recommended for implementation
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0152
Status: Deployed
Type: Specification
Assigned: 2016-04-27
License: PD
```

BIP 152

Abstract

Compact blocks on the wire as a way to save bandwidth for nodes on the P2P network.

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

Motivation

Historically, the Bitcoin P2P protocol has not been very bandwidth efficient for block relay. Every transaction in a block is included when relayed, even though a large number of the transactions in a given block are already available to nodes before the block is relayed. This causes moderate inbound bandwidth spikes for nodes when receiving blocks, but can cause very significant outbound bandwidth spikes for some nodes which receive a block before their peers. When such spikes occur, buffer bloat can make consumer-grade internet connections temporarily unusable, and can delay the relay of blocks to remote peers who may choose to wait instead of redundantly requesting the same block from other, less congested, peers.

Thus, decreasing the bandwidth used during block relay is very useful for many individuals running nodes.

While the goal of this work is explicitly not to reduce block transfer latency, it does, as a side effect reduce block transfer latencies in some rather significant ways. Additionally, this work forms a foundation for future work explicitly targeting low-latency block transfer.

Specification for version 1

Intended Protocol Flow

<img src=bip-0152/protocol-flow.png></img>

The protocol is intended to be used in two ways, depending on the peers and bandwidth available, as discussed later. The “high-bandwidth” mode, which nodes may only enable for a few of their peers, is enabled by setting the first boolean to 1 in a `sendcmpct` message. In this mode, peers send new block announcements with the short transaction IDs already (via a `cmpctblock` message), possibly even before fully validating the block (as indicated by the grey box in the image above). In some cases no further round-trip is needed, and the receiver can reconstruct the block and process it as usual immediately. When some transactions were not available from local sources (ie mempool), a `getblocktxn`/`blocktxn` roundtrip is necessary, bringing the best-case latency to the same 1.5*RTT minimum time that nodes take today, though with significantly less bandwidth usage.

The “low-bandwidth” mode is enabled by setting the first boolean to 0 in a `sendcmpct` message. In this mode, peers send new block announcements with the usual inv/headers announcements (as per BIP130, and after fully validating the block). The receiving peer may then request the block using a MSG_CMPCT_BLOCK `getdata` request, which will receive a response of the header and short transaction IDs. In some cases no further round-trip is needed, and the receiver can reconstruct the block and process it as usual, taking the same 1.5*RTT minimum time that nodes take today, though with significantly less bandwidth usage. When some transactions were not available from local sources (ie mempool), a `getblocktxn`/`blocktxn` roundtrip is necessary, bringing the latency to at least 2.5*RTT in this case, again with significantly less bandwidth usage than today. Because TCP often exhibits worse transfer latency for larger data sizes (as a multiple of RTT), total latency is expected to be reduced even when the full 2.5*RTT transfer mechanism is used.

New data structures

Several new data structures are added to the P2P network to relay compact blocks: PrefilledTransaction, HeaderAndShortIDs, BlockTransactionsRequest, and BlockTransactions.

For the purposes of this section, CompactSize refers to the variable-length integer encoding used across the existing P2P protocol to encode array lengths, among other things, in 1, 3, 5 or 9 bytes. Only CompactSize encodings which are minimally-encoded (ie the shortest length possible) are used by this spec. Any other CompactSize encodings are left with undefined behavior.

Several uses of CompactSize below are "differentially encoded". For these, instead of using raw indexes, the number encoded is the difference between the current index and the previous index, minus one. For example, a first index of 0 implies a real index of 0, a second index of 0 thereafter refers to a real index of 1, etc.

PrefilledTransaction

A PrefilledTransaction structure is used in HeaderAndShortIDs to provide a list of a few transactions explicitly.

Field Name	Type	Size	Encoding	Purpose
index	CompactSize	1, 3 bytes	Compact Size, differentially encoded since the last PrefilledTransaction in a list	The index into the block at which this transaction is
tx	Transaction	variable	As encoded in "tx" messages sent in response to getdata MSG_TX	The transaction which is in the block at index index.

HeaderAndShortIDs

A HeaderAndShortIDs structure is used to relay a block header, the short transactions IDs used for matching already-available transactions, and a select few transactions which we expect a peer may be missing.

Field Name	Type	Size	Encoding	Purpose
header	Block header	80 bytes	First 80 bytes of the block as defined by the encoding used by "block" messages	The header of the block being provided
nonce	uint64_t	8 bytes	Little Endian	A nonce for use in short transaction ID calculations
shortids_length	CompactSize	1 or 3 bytes		

			As used to encode array lengths elsewhere	
shortids	List of 6-byte integers	6*shortids_length bytes	Little Endian	The short transaction IDs calculated from the transactions which were not provided explicitly in prefilledtxn
prefilledtxn_length	CompactSize	1 or 3 bytes	As used to encode array lengths elsewhere	
prefilledtxn	List of PrefilledTransactions	variable size*prefilledtxn_length	As defined by PrefilledTransaction definition, above	Used to provide the coinbase transaction and a select few which we expect a peer may be missing

BlockTransactionsRequest

A BlockTransactionsRequest structure is used to list transaction indexes in a block being requested.

Field Name	Type	Size	Encoding	Purpose
blockhash	Binary blob	32 bytes	The output from a double-SHA256 of the block header, as used elsewhere	The blockhash of the block which the transactions being requested are in
indexes_length	CompactSize	1 or 3 bytes	As used to encode array lengths elsewhere	
indexes	List of CompactSizes	1 or 3 bytes*indexes_length	Differentially encoded	The indexes of the transactions being requested in the block

BlockTransactions

A BlockTransactions structure is used to provide some of the transactions in a block, as requested.

Field Name	Type	Size	Encoding	Purpose
blockhash	Binary blob	32 bytes		

			The output from a double-SHA256 of the block header, as used elsewhere	The blockhash of the block which the transactions being provided are in
transactions_length	CompactSize	1 or 3 bytes	As used to encode array lengths elsewhere	
transactions	List of Transactions	variable	As encoded in “tx” messages in response to getdata MSG_TX	The transactions provided

Short transaction IDs

Short transaction IDs are used to represent a transaction without sending a full 256-bit hash. They are calculated by:

1. single-SHA256 hashing the block header with the nonce appended (in little-endian)
2. Running SipHash-2-4 with the input being the transaction ID and the keys (k0/k1) set to the first two little-endian 64-bit integers from the above hash, respectively.
3. Dropping the 2 most significant bytes from the SipHash output to make it 6 bytes.

New messages

A new inv type (MSG_CMPCT_BLOCK == 4) and several new protocol messages are added: sendcmpct, cmpctblock, getblocktxn, and blocktxn.

sendcmpct

1. The sendcmpct message is defined as a message containing a 1-byte integer followed by a 8-byte integer where pchCommand == “sendcmpct”.
2. The first integer SHALL be interpreted as a boolean (and MUST have a value of either 1 or 0)
3. The second integer SHALL be interpreted as a little-endian version number. Nodes sending a sendcmpct message MUST currently set this value to 1.
4. Upon receipt of a “sendcmpct” message with the first and second integers set to 1, the node SHOULD announce new blocks by sending a cmpctblock message.
5. Upon receipt of a “sendcmpct” message with the first integer set to 0, the node SHOULD NOT announce new blocks by sending a cmpctblock message, but SHOULD announce new blocks by sending invs or headers, as defined by BIP130.
6. Upon receipt of a “sendcmpct” message with the second integer set to something other than 1, nodes MUST treat the peer as if they had not received the message (as it indicates the peer will provide an unexpected encoding in cmpctblock, and/or other, messages). This allows future versions to send duplicate sendcmpct messages with different versions as a part of a version handshake for future versions. See Protocol Versioning section, below, for more info on the specifics of the version-negotiation mechanics.
7. Nodes SHOULD check for a protocol version of >= 70014 before sending sendcmpct messages.

8. Nodes MUST NOT send a request for a MSG_CMPCT_BLOCK object to a peer before having received a sendcmpct message from that peer.
9. Nodes MUST NOT request a MSG_CMPCT_BLOCK object before having sent all sendcmpct messages to that peer which they intend to send, as the peer cannot know what version protocol to use in the response.

MSG_CMPCT_BLOCK

1. getdata messages may now contain requests for MSG_CMPCT_BLOCK objects.
2. Upon receipt of a getdata containing a request for a MSG_CMPCT_BLOCK object with the hash of a block which was recently announced and is close to the tip of the best chain of the receiver and after having sent the requesting peer a sendcmpct message, nodes MUST respond with a cmpctblock message containing appropriate data representing the block being requested.
3. Upon receipt of a getdata containing a request for a MSG_CMPCT_BLOCK object for which a cmpctblock message is not sent in response, a block message containing the requested block in non-compact form MUST be sent.
4. MSG_CMPCT_BLOCK inv objects MUST NOT appear anywhere except for in getdata messages.

cmpctblock

1. The cmpctblock message is defined as a message containing a serialized HeaderAndShortIDs message and pchCommand == “cmpctblock”.
2. Upon receipt of a cmpctblock message after sending a sendcmpct message, nodes SHOULD calculate the short transaction ID for each unconfirmed transaction they have available (ie in their mempool) and compare each to each short transaction ID in the cmpctblock message.
3. After finding already-available transactions, nodes which do not have all transactions available to reconstruct the full block SHOULD request the missing transactions using a getblocktxn message.
4. A node MUST NOT send a cmpctblock message unless they are able to respond to a getblocktxn message which requests every transaction in the block.
5. A node MUST NOT send a cmpctblock message without having validated that the header properly commits to each transaction in the block, and properly builds on top of the existing, fully-validated chain with a valid proof-of-work either as a part of the current most-work valid chain, or building directly on top of it. A node MAY send a cmpctblock before validating that each transaction in the block validly spends existing UTXO set entries.

getblocktxn

1. The getblocktxn message is defined as a message containing a serialized BlockTransactionsRequest message and pchCommand == “getblocktxn”.
2. Upon receipt of a properly-formatted getblocktxn message, nodes which recently provided the sender of such a message a cmpctblock for the block hash identified in this message MUST respond with either an appropriate blocktxn message, or a full block message. A blocktxn response MUST contain exactly and only each transaction which is present in the appropriate block at the index specified in the getblocktxn indexes list, in the order requested.

blocktxn

1. The blocktxn message is defined as a message containing a serialized BlockTransactions message and pchCommand == "blocktxn".
2. Upon receipt of a properly-formatted requested blocktxn message, nodes SHOULD attempt to reconstruct the full block by:
    1. Taking the prefilledtxn transactions from the original cmpctblock and placing them in the marked positions.
    2. For each short transaction ID from the original cmpctblock, in order, find the corresponding transaction either from the blocktxn message or from other sources and place it in the first available position in the block.
3. Once the block has been reconstructed, it shall be processed as normal, keeping in mind that short transaction IDs are expected to occasionally collide, and that nodes MUST NOT be penalized for such collisions, wherever they appear.

Protocol Versioning

1. The protocol version negotiation allows two nodes to agree on the versions of compact blocks which they will exchange. As it is only in a single field, it does not allow a node to support a specific version in only one direction (sending or receiving).
2. Upon connection establishment, a node SHOULD send a burst of sendcmpct messages containing every version of compact block encodings for which they are willing to support sending cmpctblock and blocktxn messages, and receiving getblocktxn messages. These messages SHOULD be ordered in the order of the priority which the node wishes to receive cmpctblock/blocktxn messages, with the highest-priority version sendcmpct message sent first.
3. The encoding version used to send a cmpctblock or blocktxn message or to receive a getblocktxn message MUST be the second integer (version number) in the first sendcmpct message received for which a sendcmpct message with the same version number was sent.
4. Nodes MUST NOT send a sendcmpct message which contains a version number other than the version number which has been negotiated for receiving cmpctblock/blocktxn messages after sending a request for a MSG_CMPCT_BLOCK object, sending a cmpctblock, getblocktxn, blocktxn, or pong message.
5. As a node must send all sendcmpct messages which contain a novel version announcement before any other compact block-related messages, it is possible to determine which version of compact blocks will be used for each object received. It is, however, not possible to know which version will be used to encode the response to a request for a compact block object before any MSG_CMPCT_BLOCK-containing getdata, cmpctblock, getblocktxn, blocktxn, or ping/pong messages have been exchanged.
6. Thus, if a node wishes to determine exactly which version of compact blocks will be used before requesting a compact block object, it must send all of its sendcmpct version announcements, followed by a ping, and wait for the pong response to ensure it has received all sendcmpctblock version announcement messages from the remote peer. Nodes can, obviously, however, determine that the version used will be at least a certain version (in their priority order) after having received a sendcmpct message from the remote peer containing that version as the second integer.

Sample Version Implementation

1. By way of example, an implementation of the above protocol might look like the following.

2. Upon exchanging version/verack messages, a node immediately sends its list of sendcmpct announcements to the other side, with the version which it wants to receive sent first.
3. Upon receiving the first sendcmpct announcement with a protocol version which is understood from the remote peer, a node will "lock in" the compact block encoding version which will be used to encode compact blocks to that peer.
4. The node then sets the current receive-protocol-version in use on the connection to that version, and uses it to decode new compact block messages.
5. Upon receiving subsequent sendcmpct announcements with a protocol version which is understood from the remote peer (ie a version which has been announced using a sendcmpct in the other direction), a node will check if that protocol version is higher-receive-priority than the current receive-protocol-version in use on the connection, and switch to that version for decoding new compact block messages received.
6. A node might wish to keep a flag for each peer which indicates compact block version negotiation is complete, which can be set upon receiving any compact block-related, or pong message.
7. The above implementation requires only a compile-time list of supported versions in some static priority order, two version fields per peer, and an optional negotiation-complete boolean per-peer.

Specification for version 2

Compact blocks version 2 is almost identical to version 1, but supports segregated witness transactions (BIP 141 and BIP 144). The changes are:

1. The second integer (version number) inside sendcmpct is 2 instead of 1 (see Protocol Versioning section, above).
2. Transactions inside cmpctblock messages (both those used as direct announcement and those in response to getdata) and in blocktxn should include witness data, using the same format as responses to getdata MSG_WITNESS_TX, specified in BIP144.
3. Short transaction IDs sent to us in cmpctblock messages, and sent by us in getblocktxn messages, are computed using the same process as in version 1, but using the wtxid as defined in BIP 141 instead of the txid. Note that, though a node normally SHOULD, if a node does not include (ie must then include the short ID for) the coinbase transaction, it must be computed by encoding the transaction in witness format as defined by BIP 141.
4. Upon receipt of a getdata containing a request for a MSG_CMPCT_BLOCK object for which a cmpctblock message is not sent in response, the block message containing the requested block in non-compact form MUST be encoded with witnesses (as is sent in reply to a MSG_WITNESS_BLOCK getdata) if the protocol version used to encode the cmpctblock message would have been 2, and encoded without witnesses (as is sent in response to a MSG_BLOCK getdata) if the protocol version used to encode the cmpctblock message would have been 1.

Implementation Notes

1. For nodes which have sufficient inbound bandwidth, sending a sendcmpct message with the first integer set to 1 to up to 3 peers is RECOMMENDED. If possible, it is RECOMMENDED that those peers be selected based on their past performance in providing blocks quickly (eg the three peers which

provided the highest number of the recent N blocks the quickest), allowing nodes to receive blocks which come from those peers in only 0.5*RTT.

1. Nodes MUST NOT send such sendcmpct messages to more than three peers, as it encourages wasting outbound bandwidth across the network.

1. All nodes SHOULD send a sendcmpct message to all appropriate peers. This will reduce their outbound bandwidth usage by allowing their peers to request compact blocks instead of full blocks.

1. Nodes with limited inbound bandwidth SHOULD request blocks using MSG_CMPCT_BLOCK/ getblocktxn requests, when possible. While this increases worst-case message round-trips, it is expected to reduce overall transfer latency as TCP is more likely to exhibit poor throughput on low-bandwidth nodes.

1. Nodes sending cmpctblock messages SHOULD limit prefilledtxn to 10KB of transactions. When in doubt, nodes SHOULD only include the coinbase transaction in prefilledtxn.

1. Nodes MAY pick one nonce per block they wish to send, and only build a cmpctblock message once for all peers which they wish to send a given block to. Nodes SHOULD NOT use the same nonce across multiple different blocks.

1. Nodes MAY impose additional requirements on when they announce new blocks by sending cmpctblock messages. For example, nodes with limited outbound bandwidth MAY choose to announce new blocks using inv/header messages (as per BIP130) to conserve outbound bandwidth.

1. Note that the MSG_CMPCT_BLOCK section does not require that nodes respond to MSG_CMPCT_BLOCK getdata requests for blocks which they did not recently announce. This allows nodes to calculate cmpctblock messages at announce-time instead of at request-time. Blocks which are requested with a MSG_CMPCT_BLOCK getdata, but which are not responded to with a cmpctblock message MUST be responded to with a block message, allowing nodes to request all blocks using MSG_CMPCT_BLOCK getdatas and rely on their peer to pick an appropriate response.

1. While the current version sends transactions with the same encodings as are used in tx messages and elsewhere in the protocol, the version field in sendcmpct is intended to allow this to change in the future. For this reason, it is recommended that the code used to decode PrefilledTransaction and BlockTransactions messages be prepared to take a different transaction encoding, if and when the version field in sendcmpct changes in a future BIP.

1. Any undefined behavior in this spec may cause failure to transfer block to, peer disconnection by, or self-destruction by the receiving node. A node receiving non-minimally-encoded CompactSize encodings should make a best-effort to eat the sender's cat.

Pre-Validation Relay and Consistency Considerations

1. As high-bandwidth mode permits relaying of CMPCTBLOCK messages prior to full validation (requiring only that the block header is valid before relay), nodes SHOULD NOT ban a peer for announcing a new block with a CMPCTBLOCK message that is invalid, but has a valid header. For avoidance of doubt, nodes SHOULD bump their peer-to-peer protocol version to 70015 or higher to signal that they will not ban or punish a peer for announcing compact blocks prior to full validation,

and nodes SHOULD NOT announce a CMPCTBLOCK to a peer with a version number below 70015 before fully validating the block.

1. SPV nodes which implement this spec must consider the implications of accepting blocks which were not validated by the node which provided them. Especially SPV nodes which allow users to select a "trusted full node" to sync from may wish to avoid implementing this spec in high-bandwidth mode.

1. Note that this spec does not change the requirement that nodes only relay information about blocks which they have fully validated in response to GETDATA/GETHEADERS/GETBLOCKS/etc requests. Nodes which announce using CMPCTBLOCK message and then receive a request for associated block data SHOULD ensure that messages do not go unresponded to, and that the appropriate data is provided after the block has been validated, subject to standard message-response ordering requirements. Note that no requirement is added that the node respond to the request with the new block included in eg GETHEADERS or GETBLOCKS messages, but the node SHOULD re-announce the block using the associated announcement methods after validation has completed if it is not included in the original response. On the other hand, nodes SHOULD delay responding to GETDATA requests for the block until validation has completed, stalling all message processing for the associated peer. REJECT messages are not considered "responses" for the purpose of this section.

1. As a result of the above requirements, implementers may wish to consider the potential for the introduction of delays in responses while remote peers validate blocks, avoiding delay-causing requests where possible.

Justification

Protocol design

There have been many proposals to save wire bytes when relaying blocks. Many of them have a two-fold goal of reducing block relay time and thus rely on the use of significant processing power in order to avoid introducing additional worst-case RTTs. Because this work is not focused primarily on reducing block relay time, its design is much simpler (ie does not rely on set reconciliation protocols). Still, in testing at the time of writing, nodes are able to relay blocks without the extra getblocktxn/blocktxn RTT around 90% of the time. With a smart compact-block-announcement policy, it is thus expected that this work might allow blocks to be relayed between nodes in 0.5*RTT instead of 1.5*RTT at least 75% of the time.

Short transaction ID calculation

There are several design goals for the Short ID calculation:

- **Performance** The sender needs to compute short IDs for all block transactions, and the receiver for all mempool transactions they are being compared to. As we're easily talking about several thousand transactions, sub-microsecond processing per-transactions is needed.
- **Space** cmpctblock messages are never optional in this protocol, and contain a short ID for each non-prefilled transaction in the block. Thus, the size of short IDs is directly proportional to the maximum bandwidth savings possible.
- **Collision resistance** It should be hard for network participants to create transactions that cause collisions. If an attacker were able to cause such collisions, filling mempools (and, thus, blocks) with them would cause poor network propagation of new (or non-attacker, in the case of a miner) blocks.

SipHash is a secure, fast, and simple 64-bit MAC designed for network traffic authentication and collision-resistant hash tables. We truncate the output from SipHash-2-4 to 48 bits (see next section) in order to minimize space. The resulting 48-bit hash is certainly not large enough to avoid intentionally created individual collisions, but by using the block hash as a key to SipHash, an attacker cannot predict what keys will be used once their transactions are actually included in a relayed block. We mix in a per-connection 64-bit nonce to obtain independent short IDs on every connection, so that even block creators cannot control where collisions occur, and random collisions only ever affect a small number of connections at any given time. The mixing is done using SHA256(block_header || nonce), which is slow compared to SipHash, but only done once per block. It also adds the ability for nodes to choose the nonce in a better than random way to minimize collisions, though that is not necessary for correct behaviour. Conversely, nodes can also abuse this ability to increase their ability to introduce collisions in the blocks they relay themselves. However, they can already cause more problems by simply refusing to relay blocks. That is inevitable, and this design only seeks to prevent network-wide misbehavior.

Random collision probability

Thanks to the block-header-based SipHash keys, we can assume that the only collisions on links between honest nodes are random ones.

For each of the t block transactions, the receiver will compare its received short ID with that of a set of m mempool transactions. We assume that each of those t has a chance r to be included in that set of m. If we use B bits short IDs, for each comparison between a received short ID and a mempool transaction, there is a chance of $P = 1 - 1 / 2^B$ that a mismatch is detected as such.

When comparing a given block transaction to the whole set of mempool transactions, there are 5 cases to distinguish:

1. The receiver has exactly one match, which is the correct one. This has chance $r * P^(m - 1)$.
2. The receiver has no matches. This has chance $(1 - r) * P^m$.
3. The receiver has at least two matches, one of which is correct. This has chance $r * (1 - P^(m - 1))$.
4. The receiver has at least two matches, both of which are incorrect. This has chance $(1 - r) * (1 - P^m - m * (1 - P) * P^(m - 1))$.
5. The receiver has exactly one match, but an incorrect one. This has chance $(1 - r) * m * (1 - P) * P^(m - 1)$.

(note that these 5 numbers always add up to 100%)

In case 1, we're good. In cases 2, 3, or 4, we request the full transaction because we know we're uncertain. Only in case 5, we fail to reconstruct. The chance that case 5 does not occur in any of the t transactions in a block is $(1 - (1 - r) * m * (1 - P) * P^(m - 1))^t$. This expression is well approximated by $1 - (1 - r) * m * (1 - P) * t = 1 - (1 - r) * m * t / 2^B$. Thus, if we want only one in F block transmissions between honest nodes to fail under the conservative $r = 0$ assumption, we need $log2(F * m * t)$ bits hash functions.

This means that $B = 48$ bits short IDs suffice for blocks with up to $t = 10000$ transactions, mempools up to $m = 100000$ transactions, with failure to reconstruct at most one in $F = 281474$ blocks. Since failure to reconstruct just means we fall back to normal inv/header based relay, it isn't necessary to avoid such failure completely. It just needs to be sufficiently rare they have a lower impact than random transmission failures (for example, network disconnection, node overloaded, …).

Separate version for segregated witness

The changes to transaction and block relay in BIP 144 introduce separate MSG_FILTERED_ versions of messages in getdata, allowing a receiver to choose individually where witness data is wanted.

This method is not useful for compact blocks because `cmpctblock` blocks can be sent unsolicitedly in high-bandwidth mode, so we need to negotiate at least whether those should include witness data up front. There is little use for a validating node that only sometimes processes witness data, so we may as well use that negotiation for everything and turn it into a separate protocol version. We also need a means to distinguish different versions of the same transaction with different witnesses for correct reconstruction, so this also forces us to use wtxids instead of txids for short IDs everywhere in that case.

Backward compatibility

Older clients remain fully compatible and interoperable after this change.

Implementation

https://github.com/bitcoin/bitcoin/pull/8068 for version 1. https://github.com/bitcoin/bitcoin/pull/8393 for version 2.

Acknowledgements

Thanks to Gregory Maxwell for the initial suggestion as well as a lot of back-and-forth design and significant testing. Thanks to Nicolas Dorier for the protocol flow diagram.

Copyright

This document is placed in the public domain.

```
BIP: 155
Layer: Peer Services
Title: addrv2 message
Authors: Wladimir J. van der Laan <laanwj@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2019-02-27
License: BSD-2-Clause
Version: 2.1.1
```

BIP 155

Introduction

Abstract

This document proposes a new P2P message to gossip longer node addresses over the P2P network. This is required to support new-generation Onion addresses, I2P, and potentially other networks that have longer endpoint addresses than fit in the 128 bits of the current `addr` message.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

Tor v3 hidden services are part of the stable release of Tor since version 0.3.2.9. They have various advantages compared to the old hidden services, among which are better encryption and privacy Tor Rendezvous Specification - Version 3. These services have 256 bit addresses and thus do not fit in the existing `addr` message, which encapsulates onion addresses in OnionCat IPv6 addresses.

Other transport-layer protocols such as I2P have always used longer addresses. This change would make it possible to gossip such addresses over the P2P network, so that other peers can connect to them.

Specification

> The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119RFC 2119.

The `addrv2` message is defined as a message where `pchCommand == "addrv2"`. It is serialized in the standard encoding for P2P messages. Its format is similar to the current `addr` message format, with the difference that the fixed 16-byte IP address is replaced by a network ID and a variable-length address, and the services format has been changed to CompactSize.

This means that the message contains a serialized `std::vector` of the following structure:

Type	Name	Description
`uint32_t`	`time`	Time that this node was last seen as connected to the network. A time in Unix epoch time format.
`CompactSize`	`services`	Service bits. A bit field that is 64 bits wide, encoded in CompactSize.
`uint8_t`	`networkID`	Network identifier. An 8-bit value that specifies which network is addressed.
`std::vector<uint8_t>`	`addr`	Network address. The interpretation depends on networkID.
`uint16_t`	`port`	Network port. If not relevant for the network this MUST be 0.

One message can contain up to 1,000 addresses. Clients SHOULD reject messages with more addresses.

Field `addr` has a variable length, with a maximum of 512 bytes (4096 bits). Clients SHOULD reject messages with longer addresses, irrespective of the network ID.

The list of reserved network IDs is as follows:

Network ID	Enumeration	Address length (bytes)	Description
`0x01`	`IPV4`	4	IPv4 address (globally routed internet)
`0x02`	`IPV6`	16	IPv6 address (globally routed internet)
`0x03`	`TORV2`	10	Tor v2 hidden service address (no longer usedTor v2 is no longer operational; clients MUST NOT gossip or relay Tor v2 addresses and MUST ignore them on receive)
`0x04`	`TORV3`	32	Tor v3 hidden service address
`0x05`	`I2P`	32	I2P overlay network address
`0x06`	`CJDNS`	16	Cjdns overlay network address
`0x07`	`YGGDRASIL`	16	Yggdrasil overlay network address

Clients are RECOMMENDED to gossip addresses from all known networks even if they are currently not connected to some of them. That could help multi-homed nodes and make it more difficult for an observer to tell which networks a node is connected to.

Clients SHOULD NOT gossip addresses from unknown networks because they have no means to validate those addresses and so can be tricked to gossip invalid addresses.

Further network ID numbers MUST be reserved in a new BIP document.

Clients SHOULD reject messages that contain addresses that have a different length than specified in this table for a specific network ID, as these are meaningless.

Some ranges of the `IPV6` address space are reserved for embedding addresses of other networks (for example the IPv4-in-IPv6 range `::ffff:0:0/96`). Such addresses already have a canonical encoding under their own network ID and MUST NOT be sent with the `IPV6` network ID. Clients SHOULD ignore `IPV6` addresses that fall into these reserved ranges on receive, since the same address received under two network IDs would otherwise be treated as two distinct peers.

See the appendices for the address encodings to be used for the various networks.

Signaling support and compatibility

Introduce a new message type `sendaddrv2`. Sending such a message indicates that a node can understand and prefers to receive `addrv2` messages instead of `addr` messages. I.e. “Send me addrv2”. Sending or not sending this message does not imply any preference with respect to receiving unrequested address messages.

The `sendaddrv2` message MUST only be sent in response to the `version` message from a peer and prior to sending the `verack` message.

For older peers, that did not emit `sendaddrv2`, keep sending the legacy `addr` message, ignoring addresses with the newly introduced address types.

Reference implementation

Original reference implementation in Bitcoin Core:

- https://github.com/bitcoin/bitcoin/issues/18884
- https://github.com/bitcoin/bitcoin/pull/19954
- https://github.com/bitcoin/bitcoin/pull/20119

Acknowledgements

- Jonas Schnelli: change `services` field to CompactSize, to make the message more compact in the likely case instead of always using 8 bytes.

- Gregory Maxwell: various suggestions regarding extensibility

Appendix A: Tor v2 address encoding (no longer used)

The new message introduces a separate network ID for `TORV2`.

Clients MUST send Tor hidden service addresses with this network ID, with the 80-bit hidden service ID in the address field. This is the same as the representation in the legacy `addr` message, minus the 6 byte prefix of the OnionCat wrapping.

Clients SHOULD ignore OnionCat (`fd87:d87e:eb43::/48`) addresses on receive if they come with the `IPV6` network ID.

Appendix B: Tor v3 address encoding

According to the spec Tor Rendezvous Specification - Version 3: Encoding onion addresses, next-gen `.onion` addresses are encoded as follows:

```
onion_address = base32(PUBKEY | CHECKSUM | VERSION) + ".onion"
 CHECKSUM = H(".onion checksum" | PUBKEY | VERSION)[:2]

 where:
   - PUBKEY is the 32 bytes ed25519 master pubkey of the hidden service
   - VERSION is a one byte version field (default value '\x03')
   - ".onion checksum" is a constant string
```

```
- CHECKSUM is truncated to two bytes before inserting it in onion_address
- H() is the SHA3-256 cryptographic hash function
```

Tor v3 addresses MUST be sent with the `TORV3` network ID, with the 32-byte PUBKEY part in the address field. As VERSION will always be '' in the case of v3 addresses, this is enough to reconstruct the onion address.

Appendix C: I2P address encoding

Like Tor, I2P naming uses a base32-encoded address formatI2P: Naming and address book.

I2P uses 52 characters (256 bits) to represent the full SHA-256 hash, followed by `.b32.i2p`.

I2P addresses MUST be sent with the `I2P` network ID, with the decoded SHA-256 hash as address field.

Appendix D: Cjdns address encoding

Cjdns addresses are simply IPv6 addresses in the `fc00::/8` rangeCjdns whitepaper: Pulling It All Together. They MUST be sent with the `CJDNS` network ID.

Appendix E: Yggdrasil address encoding

Yggdrasil addresses are simply IPv6 addresses in the `0200::/7` rangeYggdrasil FAQ. They MUST be sent with the `YGGDRASIL` network ID.

Changelog

- 2.1.1 (2026-08-08):
  - Add links to reference implementation.
- 2.1.0 (2026-08-03):
  - Document that IPv4-in-IPv6 range MUST NOT be sent with the IPV6 network ID and SHOULD be ignored on receive.
- 2.0.0 (2025-10-01):
  - Add note that Tor v2 is no longer operational.
- 1.0.0 (2019-02-27):
  - Initial version

References

```
BIP: 157
Layer: Peer Services
Title: Client Side Block Filtering
Authors: Olaoluwa Osuntokun <laolu32@gmail.com>
         Alex Akselrod <alex@akselrod.org>
         Jim Posen <jimpo@coinbase.com>
Status: Deployed
Type: Specification
Assigned: 2017-05-24
License: CC0-1.0
Requires: 158
```

BIP 157

Abstract

This BIP describes a new light client protocol in Bitcoin that improves upon currently available options. The standard light client protocol in use today, defined in BIP 37https://github.com/bitcoin/bips/blob/master/bip-0037.mediawiki, has known flaws that weaken the security and privacy of clients and allow denial-of-service attack vectors on full nodeshttps://lists.linuxfoundation.org/pipermail/bitcoin-dev/2016-May/012636.html. The new protocol overcomes these issues by allowing light clients to obtain compact probabilistic filters of block content from full nodes and download full blocks if the filter matches relevant data.

New P2P messages empower light clients to securely sync the blockchain without relying on a trusted source. This BIP also defines a filter header, which serves as a commitment to all filters for previous blocks and provides the ability to efficiently detect malicious or faulty peers serving invalid filters. The resulting protocol guarantees that light clients with at least one honest peer are able to identify the correct block filters.

Motivation

Bitcoin light clients allow applications to read relevant transactions from the blockchain without incurring the full cost of downloading and validating all data. Such applications seek to simultaneously minimize the trust in peers and the amount of bandwidth, storage space, and computation required. They achieve this by downloading all block headers, verifying the proofs of work, and following the longest proof-of-work chain. Since block headers are a fixed 80-bytes and are generated every 10 minutes on average, the bandwidth required to sync the block headers is minimal. Light clients then download only the blockchain data relevant to them directly from peers and validate inclusion in the header chain. Though clients do not check the validity of all blocks in the longest proof-of-work chain, they rely on miner incentives for security.

BIP 37 is currently the most widely used light client execution mode for Bitcoin. With BIP 37, a client sends a Bloom filter it wants to watch to a full node peer, then receives notifications for each new transaction or block that matches the filter. The client then requests relevant transactions from the peer along with Merkle proofs of inclusion in the blocks containing them, which are verified against the block headers. The Bloom filters match data such as client addresses and unspent outputs, and the filter size must be carefully tuned to balance the false positive rate with the amount of information leaked to peer. It has been shown, however, that most implementations available offer virtually *zero privacy* to wallets and other applicationshttps://eprint.iacr.org/2014/763.pdfhttps://jonasnick.github.io/blog/2015/02/12/privacy-in-bitcoinj/. Additionally, malicious full nodes serving light clients can omit critical data with little risk of detection, which is unacceptable for some applications (such as Lightning Network clients) that must respond to certain on-chain events. Finally, honest nodes servicing BIP 37 light clients may incur significant I/O and CPU resource usage due to maliciously crafted Bloom filters, creating a denial-of-service (DoS) vector and disincentizing node operators from supporting the protocolhttps://github.com/bitcoin/bips/blob/master/bip-0111.mediawiki.

The alternative detailed in this document can be seen as the opposite of BIP 37: instead of the client sending a filter to a full node peer, full nodes generate deterministic filters on block data that are served to the client. A light client can then download an entire block if the filter matches the data it is watching for. Since filters are deterministic, they only need to be constructed once and stored on disk, whenever a new block is connected to the chain. This keeps the computation required to serve filters minimal, and eliminates the I/O asymmetry that makes BIP 37 enabled nodes vulnerable. Clients also get better assurance of seeing all relevant transactions

because they can check the validity of filters received from peers more easily than they can check completeness of filtered blocks. Finally, client privacy is improved because blocks can be downloaded from *any source*, so that no one peer gets complete information on the data required by a client. Extremely privacy conscious light clients may opt to anonymously fetch blocks using advanced techniques such a Private Information Retrievalhttps://en.wikipedia.org/wiki/Private_information_retrieval.

Definitions

`[]byte` represents a vector of bytes.

`[N]byte` represents a fixed-size byte array with length N.

CompactSize is a compact encoding of unsigned integers used in the Bitcoin P2P protocol.

double-SHA256 is a hash algorithm defined by two invocations of SHA-256: `double-SHA256(x) = SHA256(SHA256(x))`.

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

Specification

Filter Types

For the sake of future extensibility and reducing filter sizes, there are multiple *filter types* that determine which data is included in a block filter as well as the method of filter construction/querying. In this model, full nodes generate one filter per block per filter type supported.

Each type is identified by a one byte code, and specifies the contents and serialization format of the filter. A full node MAY signal support for particular filter types using service bits. The initial filter types are defined separately in BIP 158, and one service bit is allocated to signal support for them.

Filter Headers

This proposal draws inspiration from the headers-first mechanism that Bitcoin nodes use to sync the block chainhttps://bitcoin.org/en/developer-guide#headers-first. Similar to how block headers have a Merkle commitment to all transaction data in the block, we define filter headers that have commitments to the block filters. Also like block headers, filter headers each have a commitment to the preceding one. Before downloading the block filters themselves, a light client can download all filter headers for the current block chain and use them to verify the authenticity of the filters. If the filter header chains differ between multiple peers, the client can identify the point where they diverge, then download the full block and compute the correct filter, thus identifying which peer is faulty.

The canonical hash of a block filter is the double-SHA256 of the serialized filter. Filter headers are 32-byte hashes derived for each block filter. They are computed as the double-SHA256 of the concatenation of the filter hash with the previous filter header. The previous filter header used to calculate that of the genesis block is defined to be the 32-byte array of 0’s.

New Messages

getcfilters

`getcfilters` is used to request the compact filters of a particular type for a particular range of blocks. The message contains the following fields:

Field Name	Data Type	Byte Size	Description
FilterType	byte	1	Filter type for which headers are requested
StartHeight	uint32	4	The height of the first block in the requested range
StopHash	[32]byte	32	The hash of the last block in the requested range

1. Nodes SHOULD NOT send `getcfilters` unless the peer has signaled support for this filter type. Nodes receiving `getcfilters` with an unsupported filter type SHOULD NOT respond.
2. StopHash MUST be known to belong to a block accepted by the receiving peer. This is the case if the peer had previously sent a `headers` or `inv` message with that block or any descendents. A node that receives `getcfilters` with an unknown StopHash SHOULD NOT respond.
3. The height of the block with hash StopHash MUST be greater than or equal to StartHeight, and the difference MUST be strictly less than 1000.
4. The receiving node MUST respond to valid requests by sending one `cfilter` message for each block in the requested range, sequentially in order by block height.

cfilter

`cfilter` is sent in response to `getcfilters`, one for each block in the requested range. The message contains the following fields:

Field Name	Data Type	Byte Size	Description
FilterType	byte	1	Byte identifying the type of filter being returned
BlockHash	[32]byte	32	Block hash of the Bitcoin block for which the filter is being returned
NumFilterBytes	CompactSize	1-5	A variable length integer representing the size of the filter in the following field

Field Name	Data Type	Byte Size	Description
FilterBytes	[]byte	NumFilterBytes	The serialized compact filter for this block

1. The FilterType SHOULD match the field in the `getcfilters` request, and BlockHash must correspond to a block that is an ancestor of StopHash with height greater than or equal to StartHeight.

getcfheaders

`getcfheaders` is used to request verifiable filter headers for a range of blocks. The message contains the following fields:

Field Name	Data Type	Byte Size	Description
FilterType	byte	1	Filter type for which headers are requested
StartHeight	uint32	4	The height of the first block in the requested range
StopHash	[32]byte	32	The hash of the last block in the requested range

1. Nodes SHOULD NOT send `getcfheaders` unless the peer has signaled support for this filter type. Nodes receiving `getcfheaders` with an unsupported filter type SHOULD NOT respond.
2. StopHash MUST be known to belong to a block accepted by the receiving peer. This is the case if the peer had previously sent a `headers` or `inv` message with that block or any descendents. A node that receives `getcfheaders` with an unknown StopHash SHOULD NOT respond.
3. The height of the block with hash StopHash MUST be greater than or equal to StartHeight, and the difference MUST be strictly less than 2,000.

cfheaders

`cfheaders` is sent in response to `getcfheaders`. Instead of including the filter headers themselves, the response includes one filter header and a sequence of filter hashes, from which the headers can be derived. This has the benefit that the client can verify the binding links between the headers. The message contains the following fields:

Field Name	Data Type	Byte Size	Description
FilterType	byte	1	Filter type for which hashes are requested
StopHash	[32]byte	32	The hash of the last block in the requested range
PreviousFilterHeader	[32]byte	32	The filter header preceding the first block in the requested range
FilterHashesLength	CompactSize	1-3	

Field Name	Data Type	Byte Size	Description
			The length of the following vector of filter hashes
FilterHashes	[][32]byte	FilterHashesLength * 32	The filter hashes for each block in the requested range

1. The FilterType and StopHash SHOULD match the fields in the `getcfheaders` request.
2. FilterHashesLength MUST NOT be greater than 2,000.
3. FilterHashes MUST have one entry for each block on the chain terminating with tip StopHash, starting with the block at height StartHeight. The entries MUST be the filter hashes of the given type for each block in that range, in ascending order by height.
4. PreviousFilterHeader MUST be set to the previous filter header of first block in the requested range.

getcfcheckpt

`getcfcheckpt` is used to request filter headers at evenly spaced intervals over a range of blocks. Clients may use filter hashes from `getcfheaders` to connect these checkpoints, as is described in the Client Operation section below. The `getcfcheckpt` message contains the following fields:

Field Name	Data Type	Byte Size	Description
FilterType	byte	1	Filter type for which headers are requested
StopHash	[32]byte	32	The hash of the last block in the chain that headers are requested for

1. Nodes SHOULD NOT send `getcfcheckpt` unless the peer has signaled support for this filter type. Nodes receiving `getcfcheckpt` with an unsupported filter type SHOULD NOT respond.
2. StopHash MUST be known to belong to a block accepted by the receiving peer. This is the case if the peer had previously sent a `headers` or `inv` message with any descendent blocks. A node that receives `getcfcheckpt` with an unknown StopHash SHOULD NOT respond.

cfcheckpt

`cfcheckpt` is sent in response to `getcfcheckpt`. The filter headers included are the set of all filter headers on the requested chain where the height is a positive multiple of 1,000. The message contains the following fields:

Field Name	Data Type	Byte Size	Description
FilterType	byte	1	Filter type for which headers are requested
StopHash	[32]byte	32	The hash of the last block in the chain that headers are requested for

Field Name	Data Type	Byte Size	Description
FilterHeadersLength	CompactSize	1-3	The length of the following vector of filter headers
FilterHeaders	[][32]byte	FilterHeadersLength * 32	The filter headers at intervals of 1,000

1. The FilterType and StopHash SHOULD match the fields in the `getcfcheckpt` request.
2. FilterHeaders MUST have exactly one entry for each block on the chain terminating in StopHash, where the block height is a multiple of 1,000 greater than 0. The entries MUST be the filter headers of the given type for each such block, in ascending order by height.

Node Operation

Full nodes MAY opt to support this BIP and generate filters for any of the specified filter types. Such nodes SHOULD treat the filters as an additional index of the blockchain. For each new block that is connected to the main chain, nodes SHOULD generate filters for all supported types and persist them. Nodes that are missing filters and are already synced with the blockchain SHOULD reindex the chain upon start-up, constructing filters for each block from genesis to the current tip. They also SHOULD keep every checkpoint header in memory, so that `getcfcheckpt` requests do not result in many random-access disk reads.

Nodes SHOULD NOT generate filters dynamically on request, as malicious peers may be able to perform DoS attacks by requesting small filters derived from large blocks. This would require an asymmetrical amount of I/O on the node to compute and serve, similar to attacks against BIP 37 enabled nodes noted in BIP 111.

Nodes MAY prune block data after generating and storing all filters for a block.

Client Operation

This section provides recommendations for light clients to download filters with maximal security.

Clients SHOULD first sync the entire block header chain from peers using the standard headers-first syncing mechanism before downloading any block filters or filter headers. Clients configured with trusted checkpoints MAY only sync headers started from the last checkpoint. Clients SHOULD disconnect any outbound peers whose best chain has significantly less work than the known longest proof-of-work chain.

Once a client's block headers are in sync, it SHOULD download and verify filter headers for all blocks and filter types that it might later download. The client SHOULD send `getcfheaders` messages to peers and derive and store the filter headers for each block. The client MAY first fetch headers at evenly spaced intervals of 1,000 by sending `getcfcheckpt`. The header checkpoints allow the client to download filter headers for different intervals from multiple peers in parallel, verifying each range of 1,000 headers against the checkpoints.

Unless securely connected to a trusted peer that is serving filter headers, the client SHOULD connect to multiple outbound peers that support each filter type to mitigate the risk of downloading incorrect headers. If the client receives conflicting filter headers from different peers for any block and filter type, it SHOULD interrogate them to determine which is faulty. The client SHOULD use `getcfheaders` and/or `getcfcheckpt` to first identify the first filter headers that the peers disagree on. The client then SHOULD download the full block

from any peer and derive the correct filter and filter header. The client SHOULD ban any peers that sent a filter header that does not match the computed one.

Once the client has downloaded and verified all filter headers needed, *and* no outbound peers have sent conflicting headers, the client can download the actual block filters it needs. The client MAY backfill filter headers before the first verified one at this point if it only downloaded them starting at a later point. Clients SHOULD persist the verified filter headers for the last 100 blocks in the chain (or whatever finality depth is desired), to compare against headers received from new peers after restart. They MAY store more filter headers to avoid redownloading them if a rescan is later necessary.

Starting from the first block in the desired range, the client now MAY download the filters. The client SHOULD test that each filter links to its corresponding filter header and ban peers that send incorrect filters. The client MAY download multiple filters at once to increase throughput, though it SHOULD test the filters sequentially. The client MAY check if a filter is empty before requesting it by checking if the filter header commits to the hash of the empty filter, saving a round trip if that is the case.

Each time a new valid block header is received, the client SHOULD request the corresponding filter headers from all eligible peers. If two peers send conflicting filter headers, the client should interrogate them as described above and ban any peers that send an invalid header.

If a client is fetching full blocks from the P2P network, they SHOULD be downloaded from outbound peers at random to mitigate privacy loss due to transaction intersection analysis. Note that blocks may be downloaded from peers that do not support this BIP.

Rationale

The filter headers and checkpoints messages are defined to help clients identify the correct filter for a block when connected to peers sending conflicting information. An alternative solution is to require Bitcoin blocks to include commitments to derived block filters, so light clients can verify authenticity given block headers and some additional witness data. This would require a network-wide change to the Bitcoin consensus rules, however, whereas this document proposes a solution purely at the P2P layer.

The constant interval of 1,000 blocks between checkpoints was chosen so that, given the current chain height and rate of growth, the size of a `cfcheckpt` message is not drastically different from a `cfheaders` message between two checkpoints. Also, 1,000 is a nice round number, at least to those of us who think in decimal.

Compatibility

This light client mode is not compatible with current node deployments and requires support for the new P2P messages. The node implementation of this proposal is not incompatible with the current P2P network rules (ie. doesn't affect network topology of full nodes). Light clients may adopt protocols based on this as an alternative to the existing BIP 37. Adoption of this BIP may result in reduced network support for BIP 37.

Acknowledgments

We would like to thank bfd (from the bitcoin-dev mailing list) for bringing the basis of this BIP to our attention, Joseph Poon for suggesting the filter header chain scheme, and Pedro Martelletto for writing the initial indexing code for `btcd`.

We would also like to thank Dave Collins, JJ Jeffrey, Eric Lombrozo, and Matt Corallo for useful discussions.

Reference Implementation

Light client: 1

Full-node indexing: https://github.com/Roasbeef/btcd/tree/segwit-cbf

Golomb-Rice Coded sets: https://github.com/Roasbeef/btcutil/tree/gcs/gcs

References

Copyright

This document is licensed under the Creative Commons CC0 1.0 Universal license.

```
BIP: 158
Layer: Peer Services
Title: Compact Block Filters for Light Clients
Authors: Olaoluwa Osuntokun <laolu32@gmail.com>
         Alex Akselrod <alex@akselrod.org>
Status: Deployed
Type: Specification
Assigned: 2017-05-24
License: CC0-1.0
Requires: 157
```

BIP 158

Abstract

This BIP describes a structure for compact filters on block data, for use in the BIP 157 light client protocolbip-0157.mediawiki. The filter construction proposed is an alternative to Bloom filters, as used in BIP 37, that minimizes filter size by using Golomb-Rice coding for compression. This document specifies one initial filter type based on this construction that enables basic wallets and applications with more advanced smart contracts.

Motivation

BIP 157 defines a light client protocol based on deterministic filters of block content. The filters are designed to minimize the expected bandwidth consumed by light clients, downloading filters and full blocks. This document defines the initial filter type *basic* that is designed to reduce the filter size for regular wallets.

Definitions

`[]byte` represents a vector of bytes.

`[N]byte` represents a fixed-size byte array with length N.

CompactSize is a compact encoding of unsigned integers used in the Bitcoin P2P protocol.

Bit streams are readable and writable streams of individual bits. The following functions are used in the pseudocode in this document:

- `new_bit_stream` instantiates a new writable bit stream
- `new_bit_stream(vector)` instantiates a new bit stream reading data from `vector`
- `write_bit(stream, b)` appends the bit `b` to the end of the stream
- `read_bit(stream)` reads the next available bit from the stream
- `write_bits_big_endian(stream, n, k)` appends the `k` least significant bits of integer `n` to the end of the stream in big-endian bit order
- `read_bits_big_endian(stream, k)` reads the next available `k` bits from the stream and interprets them as the least significant bits of a big-endian integer

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

Specification

Golomb-Coded Sets

For each block, compact filters are derived containing sets of items associated with the block (eg. addresses sent to, outpoints spent, etc.). A set of such data objects is compressed into a probabilistic structure called a *Golomb-coded set* (GCS), which matches all items in the set with probability 1, and matches other items with probability `1/M` for some integer parameter `M`. The encoding is also parameterized by `P`, the bit length of the remainder code. Each filter defined specifies values for `P` and `M`.

At a high level, a GCS is constructed from a set of `N` items by:

1. hashing all items to 64-bit integers in the range `[0, N * M)`
2. sorting the hashed values in ascending order
3. computing the differences between each value and the previous one
4. writing the differences sequentially, compressed with Golomb-Rice coding

The following sections describe each step in greater detail.

Hashing Data Objects

The first step in the filter construction is hashing the variable-sized raw items in the set to the range `[0, F)`, where `F = N * M`. Customarily, `M` is set to `2^P`. However, if one is able to select both Parameters independently, then more optimal values can be selectedhttps://gist.github.com/sipa/576d5f09c3b86c3b1b75598d799fc845. Set membership queries against the hash outputs will have a false positive rate of `1 / M`. To avoid integer overflow, the number of items `N` MUST be <2^32 and `M` MUST be <2^32.

The items are first passed through the pseudorandom function *SipHash*, which takes a 128-bit key `k` and a variable-sized byte vector and produces a uniformly random 64-bit output. Implementations of this BIP MUST use the SipHash parameters `c = 2` and `d = 4`.

The 64-bit SipHash outputs are then mapped uniformly over the desired range by multiplying with F and taking the top 64 bits of the 128-bit result. This algorithm is a faster alternative to modulo reduction, as it

avoids the expensive division operationhttps://lemire.me/blog/2016/06/27/a-fast-alternative-to-the-modulo-reduction/. Note that care must be taken when implementing this reduction to ensure the upper 64 bits of the integer multiplication are not truncated; certain architectures and high level languages may require code that decomposes the 64-bit multiplication into four 32-bit multiplications and recombines into the result.

```
hash_to_range(item: []byte, F: uint64, k: [16]byte) -> uint64:
    return (siphash(k, item) * F) >> 64

hashed_set_construct(raw_items: [][]byte, k: [16]byte, M: uint) -> []uint64:
    let N = len(raw_items)
    let F = N * M

    let set_items = []

    for item in raw_items:
        let set_value = hash_to_range(item, F, k)
        set_items.append(set_value)

    return set_items
```

Golomb-Rice Coding

Instead of writing the items in the hashed set directly to the filter, greater compression is achieved by only writing the differences between successive items in sorted order. Since the items are distributed uniformly, it can be shown that the differences resemble a geometric distributionhttps://en.wikipedia.org/wiki/Geometric_distribution. *Golomb-Rice coding*https://en.wikipedia.org/wiki/Golomb_coding#Rice_coding is a technique that optimally compresses geometrically distributed values.

With Golomb-Rice, a value is split into a quotient and remainder modulo `2^P`, which are encoded separately. The quotient `q` is encoded as *unary*, with a string of `q` 1's followed by one 0. The remainder `r` is represented in big-endian by P bits. For example, this is a table of Golomb-Rice coded values using `P=2`:

n	**(q, r)**	**c**
0	(0, 0)	`0 00`
1	(0, 1)	`0 01`
2	(0, 2)	`0 10`
3	(0, 3)	`0 11`
4	(1, 0)	`10 00`
5	(1, 1)	`10 01`
6	(1, 2)	`10 10`
7	(1, 3)	`10 11`
8	(2, 0)	`110 00`
9	(2, 1)	`110 01`

```
golomb_encode(stream, x: uint64, P: uint):
    let q = x >> P
```

```
    while q > 0:
        write_bit(stream, 1)
        q--
    write_bit(stream, 0)

    write_bits_big_endian(stream, x, P)

golomb_decode(stream, P: uint) -> uint64:
    let q = 0
    while read_bit(stream) == 1:
        q++

    let r = read_bits_big_endian(stream, P)

    let x = (q << P) + r
    return x
```

Set Construction

A GCS is constructed from four parameters:

- `L`, a vector of `N` raw items
- `P`, the bit parameter of the Golomb-Rice coding
- `M`, the inverse of the target false positive rate
- `k`, the 128-bit key used to randomize the SipHash outputs

The result is a byte vector with a minimum size of `N * (P + 1)` bits.

The raw items in `L` are first hashed to 64-bit unsigned integers as specified above and sorted. The differences between consecutive values, hereafter referred to as *deltas*, are encoded sequentially to a bit stream with Golomb-Rice coding. Finally, the bit stream is padded with 0's to the nearest byte boundary and serialized to the output byte vector.

```
construct_gcs(L: [][]byte, P: uint, k: [16]byte, M: uint) -> []byte:
    let set_items = hashed_set_construct(L, k, M)

    set_items.sort()

    let output_stream = new_bit_stream()

    let last_value = 0
    for item in set_items:
        let delta = item - last_value
        golomb_encode(output_stream, delta, P)
        last_value = item

    return output_stream.bytes()
```

Set Querying/Decompression

To check membership of an item in a compressed GCS, one must reconstruct the hashed set members from the encoded deltas. The procedure to do so is the reverse of the compression: deltas are decoded one by one and added to a cumulative sum. Each intermediate sum represents a hashed value in the original set. The queried

item is hashed in the same way as the set members and compared against the reconstructed values. Note that querying does not require the entire decompressed set be held in memory at once.

```
gcs_match(key: [16]byte, compressed_set: []byte, target: []byte, P: uint, N: uint, M: uint) ->
bool:
    let F = N * M
    let target_hash = hash_to_range(target, F, k)

    stream = new_bit_stream(compressed_set)

    let last_value = 0

    loop N times:
        let delta = golomb_decode(stream, P)
        let set_item = last_value + delta

        if set_item == target_hash:
            return true

        // Since the values in the set are sorted, terminate the search once
        // the decoded value exceeds the target.
        if set_item > target_hash:
            break

        last_value = set_item

    return false
```

Some applications may need to check for set intersection instead of membership of a single item. This can be performed far more efficiently than checking each item individually by leveraging the sorted structure of the compressed GCS. First the query elements are all hashed and sorted, then compared in order against the decompressed GCS contents. See Appendix B for pseudocode.

Block Filters

This BIP defines one initial filter type:

- Basic (`0x00`)
    - `M = 784931`
    - `P = 19`

Contents

The basic filter is designed to contain everything that a light client needs to sync a regular Bitcoin wallet. A basic filter MUST contain exactly the following items for each transaction in a block:

- The previous output script (the script being spent) for each input, except for the coinbase transaction.
- The scriptPubKey of each output, aside from all `OP_RETURN` output scripts.

Any "nil" items MUST NOT be included into the final set of filter elements.

We exclude all outputs that start with `OP_RETURN` in order to allow filters to easily be committed to in the future via a soft-fork. A likely area for future commitments is an additional `OP_RETURN` output in the coinbase

transaction similar to the current witness commitment https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki. By excluding all `OP_RETURN` outputs we avoid a circular dependency between the commitment, and the item being committed to.

Construction

The basic type is constructed as Golomb-coded sets with the following parameters.

The parameter `P` MUST be set to `19`, and the parameter `M` MUST be set to `784931`. Analysis has shown that if one is able to select `P` and `M` independently, then setting `M=1.497137 * 2^P` is close to optimal https://gist.github.com/sipa/576d5f09c3b86c3b1b75598d799fc845.

Empirical analysis also shows that these parameters minimize the bandwidth utilized, considering both the expected number of blocks downloaded due to false positives and the size of the filters themselves.

The parameter `k` MUST be set to the first 16 bytes of the hash (in standard little-endian representation) of the block for which the filter is constructed. This ensures the key is deterministic while still varying from block to block.

Since the value `N` is required to decode a GCS, a serialized GCS includes it as a prefix, written as a `CompactSize`. Thus, the complete serialization of a filter is:

- `N`, encoded as a `CompactSize`
- The bytes of the compressed filter itself

A zero element filter MUST be written as one byte containing zeroes.

Signaling

This BIP allocates a new service bit:

NODE_COMPACT_FILTERS	`1 << 6`	If enabled, the node MUST respond to all BIP 157 messages for filter type `0x00`

Compatibility

This block filter construction is not incompatible with existing software, though it requires implementation of the new filters.

Acknowledgments

We would like to thank bfd (from the bitcoin-dev mailing list) for bringing the basis of this BIP to our attention, Greg Maxwell for pointing us in the direction of Golomb-Rice coding and fast range optimization, Pieter Wullie for his analysis of optimal GCS parameters, and Pedro Martelletto for writing the initial indexing code for `btcd`.

We would also like to thank Dave Collins, JJ Jeffrey, and Eric Lombrozo for useful discussions.

Reference Implementation

Light client: 1

Full-node indexing: https://github.com/Roasbeef/btcd/tree/segwit-cbf

Golomb-Rice Coded sets: https://github.com/btcsuite/btcd/tree/master/btcutil/gcs

Appendix A: Alternatives

A number of alternative set encodings were considered before Golomb-coded sets were settled upon. In this appendix section, we'll list a few of the alternatives along with our rationale for not pursuing them.

Bloom Filters

Bloom Filters are perhaps the best known probabilistic data structure for testing set membership, and were introduced into the Bitcoin protocol with BIP 37. The size of a Bloom filter is larger than the expected size of a GCS with the same false positive rate, which is the main reason the option was rejected.

Cryptographic Accumulators

Cryptographic accumulatorshttps://en.wikipedia.org/wiki/Accumulator_(cryptography) are a cryptographic data structures that enable (amongst other operations) a one way membership test. One advantage of accumulators are that they are constant size, independent of the number of elements inserted into the accumulator. However, current constructions of cryptographic accumulators require an initial trusted set up. Additionally, accumulators based on the Strong-RSA Assumption require mapping set items to prime representatives in the associated group which can be preemptively expensive.

Matrix Based Probabilistic Set Data Structures

There exist data structures based on matrix solving which are even more space efficient compared to Bloom filtershttps://arxiv.org/pdf/0804.1845.pdf. We instead opted for our GCS-based filters as they have a much lower implementation complexity and are easier to understand.

Appendix B: Pseudocode

Golomb-Coded Set Multi-Match

```
gcs_match_any(key: [16]byte, compressed_set: []byte, targets: [][]byte, P: uint, N: uint, M:
uint) -> bool:
    let F = N * M

    // Map targets to the same range as the set hashes.
    let target_hashes = []
    for target in targets:
        let target_hash = hash_to_range(target, F, k)
        target_hashes.append(target_hash)

    // Sort targets so matching can be checked in linear time.
    target_hashes.sort()
```

```
stream = new_bit_stream(compressed_set)

let value = 0
let target_idx = 0
let target_val = target_hashes[target_idx]

loop N times:
    let delta = golomb_decode(stream, P)
    value += delta

    inner loop:
        if target_val == value:
            return true

        // Move on to the next set value.
        else if target_val > value:
            break inner loop

        // Move on to the next target value.
        else if target_val < value:
            target_idx++

            // If there are no targets left, then there are no matches.
            if target_idx == len(targets):
                break outer loop

            target_val = target_hashes[target_idx]

return false
```

Appendix C: Test Vectors

Test vectors for basic block filters on five testnet blocks, including the filters and filter headers, can be found here. The code to generate them can be found here.

References

Copyright

This document is licensed under the Creative Commons CC0 1.0 Universal license.

```
BIP: 159
Layer: Peer Services
Title: NODE_NETWORK_LIMITED service bit
Authors: Jonas Schnelli <dev@jonasschnelli.ch>
Status: Deployed
Type: Specification
Assigned: 2017-05-11
License: BSD-2-Clause
```

BIP 159

Abstract

Define a service bit that allows pruned peers to signal their limited services.

Motivation

Pruned peers can offer the same services as traditional peers, except that of serving all historical blocks. Bitcoin right now only offers the `NODE_NETWORK` service bit to indicate that a peer can serve all historical blocks.

1. Pruned peers can relay blocks, headers, transactions, and addresses, but they only guarantee serving a minimum number of historical blocks; thus, they should have a way to announce their service(s)
2. Peers no longer in initial block download should consider connecting some of their outbound connections to pruned peers, to allow other peers to bootstrap from non-pruned peers

Specification

New service bit

This BIP proposes a new service bit

NODE_NETWORK_LIMITED	bit 10 (0x400)	If signaled, the peer *MUST* be capable of serving at least the last 288 blocks (~2 days).

Pruned/limited peers *MUST NOT* set a service bit that signals serving the complete block chain (e.g., `NODE_NETWORK`). Rationale: nodes that signal serving the complete block chain may also signal `NODE_NETWORK_LIMITED`.

A safety buffer of 144 blocks to handle chain reorganizations *SHOULD* be taken into account when connecting to a peer signaling the `NODE_NETWORK_LIMITED` service bit.

Address relay

Full nodes following this BIP *SHOULD* relay address/services (`addr` message) from peers they would connect to (including peers signaling `NODE_NETWORK_LIMITED`).

Counter-measures for peer fingerprinting

Peers may have different prune depths (depending on their configuration, disk space, etc.), which can result in a fingerprinting weakness (finding the prune depth through getdata requests).

Pruned nodes should therefore avoid leaking the prune depth and *SHOULD NOT* serve blocks deeper than the signaled `NODE_NETWORK_LIMITED` threshold of 288 blocks.

Risks

Pruned peers following this BIP may consume more outbound bandwidth.

Light clients (and such) who are not checking the `nServiceFlags` (service bits) from a relayed `addr`-message may unwittingly connect to a pruned peer and ask for (filtered) blocks at a depth below the peer's pruned depth. Light clients should therefore check the service bits and either (1) connect to peers signaling `NODE_NETWORK_LIMITED` that preferably do not also signal serving the full block chain, if they only require (filtered) blocks around the tip, or (2) connect to peers signaling serving the full block chain if they need data older than the latest 288 blocks. Light clients obtaining peer IPs through DNS seeds should use the DNS filtering option.

Compatibility

This proposal is backward compatible.

Reference implementation

- https://github.com/bitcoin/bitcoin/pull/11740 (signaling)
- https://github.com/bitcoin/bitcoin/pull/10387 (connection and relay)

Copyright

This BIP is licensed under the 2-clause BSD license.

```
BIP: 324
Layer: Peer Services
Title: Version 2 P2P Encrypted Transport Protocol
Authors: Dhruv Mehta <dhruv@bip324.com>
         Tim Ruffing <crypto@timruffing.de>
         Jonas Schnelli <dev@jonasschnelli.ch>
         Pieter Wuille <bitcoin-dev@wuille.net>
Status: Deployed
Type: Specification
Assigned: 2019-03-08
License: BSD-3-Clause
Version: 1.0.2
Replaces: 151
```

BIP 324

Introduction

Abstract

This document proposes a new Bitcoin P2P transport protocol, which features opportunistic encryption, a mild bandwidth reduction, and the ability to negotiate upgrades before exchanging application messages.

Copyright

This document is licensed under the 3-clause BSD license.

Motivation

Bitcoin is a permissionless network whose purpose is to reach consensus over public data. Since all data relayed in the Bitcoin P2P network is inherently public, and the protocol lacks a notion of cryptographic identities, peers talk to each other over unencrypted and unauthenticated connections. Nevertheless, this plaintext nature of the current P2P protocol (referred to as v1 in this document) has severe drawbacks in the presence of attackers:

- While the relayed data itself is public in nature, the associated metadata may reveal private information and hamper privacy of users. For example, a global passive attacker eavesdropping on all Bitcoin P2P connections can trivially identify the source and timing of a transaction.
- Since connections are unauthenticated, they can be tampered with at a low cost and often even with a low risk of detection. For example, an attacker can alter specific bytes of a connection (such as node flags) on-the-fly without the need to keep any state.
- The protocol is self-revealing. For example, deep packet inspection can identify a P2P connection trivially because connections start with a fixed sequence of magic bytes. The ability to detect connections enables censorship and facilitates the aforementioned attacks as well as other attacks which require the attacker to control the connections of victims, e.g., eclipse attacks targeted at miners.

This proposal for a new P2P protocol version (v2) aims to improve upon this by raising the costs for performing these attacks substantially, primarily through the use of unauthenticated, opportunistic transport encryption. In addition, the bytestream on the wire is made pseudorandom (i.e., indistinguishable from uniformly random bytes) to a passive eavesdropper.

- Encryption, even when it is unauthenticated and only used when both endpoints support v2, impedes eavesdropping by forcing the attacker to become active: either by performing a persistent man-in-the-middle (MitM) attack, by downgrading connections to v1, or by spinning up their own nodes and getting honest nodes to make connections to them. Active attacks at scale are more resource intensive in general, but in the case of manual, deliberate connections (as opposed to automatic, random ones), they are also in principle detectable: even very basic checks, e.g., operators manually comparing protocol versions and session IDs (as supported by the proposed protocol), will expose the attacker.
- Tampering, while already an inherently active attack, is costlier if the attacker is forced to maintain the state necessary for a full MitM interception.
- A pseudorandom bytestream excludes identification techniques based on pattern matching, and makes it easier to shape the bytestream in order to mimic other protocols used on the Internet. This raises the cost of a connection censoring firewall, forcing them to either resort to a full MitM attack, or operate on a more obvious allowlist basis, rather than a blocklist basis.

Why encrypt without authentication?

As we have argued above, unauthenticated encryption**What does *authentication* mean in this context?** Unfortunately, the term authentication in the context of secure channel protocols is ambiguous. It can refer to:

- The encryption scheme guaranteeing that a message obtained via successful decryption was encrypted by someone having access to the (symmetric) encryption key, and not modified after encryption by a third party. The proposal in this document achieves that property through the use of an AEAD.
- The communication protocol establishing that the communication partner's identity matches who we expect them to be, through some public key mechanism. The proposal in this document does **not** include such a mechanism. provides strictly better security than no encryption. Thus, all connections should use encryption, even if they are unauthenticated.

When it comes to authentication, the situation is not as clear as for encryption. Due to Bitcoin's permissionless nature, authentication will always be restricted to specific scenarios (e.g., connections between peers belonging to the same operator), and whether some form of (possibly partially anonymous) authentication is desired depends on the specific requirements of the involved peers. As a consequence, we believe that authentication should be addressed separately (if desired), and this proposal aims to provide a solid technical basis for future protocol upgrades, including the addition of optional authentication (see Private authentication protocols).

Why have a pseudorandom bytestream when traffic analysis is still possible?

Traffic analysis, e.g., observing packet lengths and timing, as well as active attacks can still reveal that the Bitcoin v2 P2P protocol is in use. Nevertheless, a pseudorandom bytestream raises the cost of fingerprinting the protocol substantially, and may force some intermediaries to attack any protocol they cannot identify, causing collateral cost.

A pseudorandom bytestream is not self-identifying. Moreover, it is unopinionated and thus a canonical choice for similar protocols. As a result, Bitcoin P2P traffic will be indistinguishable from traffic of other protocols which make the same choice (e.g., obfs4 and a recently proposed cTLS extension). Moreover, traffic shapers and protocol wrappers (for example, making the traffic look like HTTPS or SSH) can further mitigate traffic analysis and active attacks but are out of scope for this proposal.

Why not use a secure tunnel protocol?

Our goal includes making opportunistic encryption ubiquitously available, as that provides the best defense against large-scale attacks. That implies protecting both the manual, deliberate connections node operators instruct their software to make, and the automatic connections Bitcoin nodes make with each other based on IP addresses obtained via gossip. While encryption per se is already possible with proxy networks or VPN protocols, these are not desirable or applicable for automatic connections at scale:

- Proxy networks like Tor or I2P introduce a separate address space, independent of network topology, with a very low cost per address making eclipse attacks cheaper. In comparison, clearnet IPv4 and IPv6 networks make obtaining multiple network identities in distinct, well-known network partitions carry a non-trivial cost. Thus, it is not desirable to have a substantial portion of nodes be exclusively connected this way, as this would significantly reduce Eclipse attack costs.**Why is it a bad idea to have nodes exclusively connected over Tor?** See the Bitcoin over Tor isn't a Good Idea paper Additionally, Tor connections come with significant bandwidth and latency costs that may not be desirable for all network users.
- VPN protocols like WireGuard or OpenVPN inherently define a private network, which requires manual configuration and therefore is not a realistic avenue for automatic connections.

Thus, to achieve our goal, we need a solution that has minimal costs, works without configuration, and is always enabled – on top of any network layer rather than be part of the network layer.

Why not use a general-purpose transport encryption protocol?

While it would be possible to rely on an off-the-shelf transport encryption protocol such as TLS or Noise, the specific requirements of the Bitcoin P2P network laid out above make these protocols an unsuitable choice.

The primary requirement which existing protocols fail to meet is a sufficiently modular treatment of encryption and authentication. As we argue above, whether and which form of authentication is desired in the Bitcoin P2P network will depend on the specific requirements of the involved peers (resulting in a mix of authenticated and unauthenticated connections), and thus the question of authentication should be decoupled from encryption. However, native support for a handful of standard authentication scenarios (e.g., using digital signatures and certificates) is at the core of the design of existing general-purpose transport encryption protocols. This focus on authentication would not provide clear benefits for the Bitcoin P2P network but would come with a large amount of additional complexity.

In contrast, our proposal instead aims for a simple modular design that makes it possible to address authentication separately. Our proposal provides a foundation for authentication by exporting a *session ID* that uniquely identifies the encrypted channel. After an encrypted channel has been established, the two endpoints are able to use any authentication protocol to confirm that they have the same session ID. (This is sometimes called *channel binding* because the session ID binds the encrypted channel to the authentication protocol.) Since in our proposal, any authentication needs to run after an encrypted connection has been established, the price we pay for this modularity is a possibly higher number of roundtrips as opposed to other protocols that perform authentication alongside the Diffie-Hellman key exchange.**Do other protocols not support exporting a session ID?** While Noise and TLS (as a draft) offer similar protocol extensions for exporting session IDs, using channel binding for authentication is not at the focus of their design and would not avoid the bulk of additional complexity due to the native support of authentication methods. However, the resulting increase in connection establishment latency is a not a concern for Bitcoin's long-lived connections, which typically live for hours or even weeks.

Besides this fundamentally different treatment of authentication, further technical issues arise when applying TLS or Noise to our desired use case:

- Neither offers a pseudorandom bytestream.
- Neither offers native support for elliptic curve cryptography on the curve secp256k1 as otherwise used in Bitcoin. While using secp256k1 is not strictly necessary, it is the obvious choice is for any new asymmetric cryptography in Bitcoin because it minimizes the cryptographic hardness assumptions as well as the dependencies that Bitcoin software will need.
- Neither offers shapability of the bytestream.
- Both provide a stream-based interface to the application layer, whereas Bitcoin requires a packet-based interface, resulting in the need for an additional thin layer to perform packet serialization and deserialization.

While existing protocols could be amended to address all of the aforementioned issues, this would negate the benefits of using them as off-the-shelf solution, e.g., the possibility to re-use existing implementations and security analyses.

Goals

This proposal aims to achieve the following properties:

- Confidentiality against passive attacks: A passive attacker having recorded a v2 P2P bytestream (without timing and fragmentation information) must not be able to determine the plaintext being exchanged by the nodes.
- Observability of active attacks: A session ID identifying the encrypted channel uniquely is derived deterministically from a Diffie-Hellman negotiation. An active man-in-the-middle attacker is forced to incur a risk of being detected as peer operators can compare session IDs manually, or using optional authentication methods possibly introduced in future protocol versions.
- Pseudorandom bytestream: A passive attacker having recorded a v2 P2P bytestream (without timing information and fragmentation information) must not be able to distinguish it from a uniformly random bytestream.
- Shapable bytestream: It should be possible to shape the bytestream to increase resistance to traffic analysis (for example, to conceal block propagation), or censorship avoidance.**How can shapability help circumvent fragmentation-pattern based censoring?** See this Tor issue as an example.
- Forward secrecy: An eavesdropping attacker who compromises a peer's sessions secrets should not be able to decrypt past session traffic, except for the latest few packets.
- Upgradability: The proposal provides an upgrade path using transport versioning which can be used to add features like authentication, PQC handshake upgrade, etc. in the future.
- Compatibility: v2 clients will allow inbound v1 connections to minimize risk of network partitions.
- Low overhead: the introduction of a new P2P transport protocol should not substantially increase computational cost or bandwidth for nodes that implement it, compared to the current protocol.

Specification

The specification consists of three parts:

- The **Transport layer** concerns how to set up an encrypted connection between two nodes, capable of transporting application-level messages between them.
- The **Application layer** concerns how to encode Bitcoin P2P messages and commands for transport by the Transport Layer.
- The **Signaling** concerns how v2 nodes advertise their support for the v2 protocol to potential peers.

Transport layer specification

In this section, we define the encryption protocol for messages between peers.

Overview and design

We first give an informal overview of the entire protocol flow and packet encryption.

Protocol flow overview

Given a newly established connection (typically TCP/IP) between two v2 P2P nodes, there are 3 phases the connection goes through. The first starts immediately, i.e. there are no v1 messages or any other bytes

exchanged on the link beforehand. The two parties are called the **initiator** (who established the connection) and the **responder** (who accepted the connection).

1. The **Key exchange phase**, where nodes exchange data to establish shared secrets.
    - The initiator:
        - Generates a random ephemeral secp256k1 private key and sends a corresponding 64-byte ElligatorSwift**What is ElligatorSwift and why use it?** The SwiftEC paper describes a method called ElligatorSwift which allows encoding elliptic curve points in a way that is indistinguishable from a uniformly distributed bitstream. While a random 256-bit string has about 50% chance of being a valid X coordinate on the secp256k1 curve, every 512-bit string is a valid ElligatorSwift encoding of a curve point, making the encoded point indistinguishable from random when using an encoder that can sample uniformly.**How fast is ElligatorSwift?** Our benchmarks show that ElligatorSwift encoded ECDH is about 50% more expensive than unencoded ECDH. Given the fast performance of ECDH and the low frequency of new connections, we found the performance trade-off acceptable for the pseudorandom bytestream and future censorship resistance it can enable.-encoded public key to the responder.
        - May send up to 4095**How was the limit of 4095 bytes garbage chosen?** It is a balance between having sufficient freedom to hide information, and allowing it to be large enough so that the necessary 64 bytes of public key is small compared to it on the one hand, and bandwidth waste on the other hand. bytes of arbitrary data after their public key, called **garbage**, providing a form of shapability and avoiding a recognizable pattern of exactly 64 bytes.**Why does the affordance for garbage exist in the protocol?** The garbage strings after the public keys are needed for shapability of the handshake. Neither peer can send decoy packets before having received at least the other peer's public key, i.e., neither peer can send more than 64 bytes before having received 64 bytes.
    - The responder:
        - Waits until one byte is received which does not match the 16 bytes consisting of the network magic followed by "version". If the first 16 bytes do match, the connection is treated as using the v1 protocol instead.**What if a v2 initiator's public key starts accidentally with these 16 bytes?** This is so unlikely (probability of 2^{-128}) to happen randomly in the v2 protocol that the initiator does not need to specifically avoid it. The optional detection of wrong-network v1 peers has a probability of 2^{-96}, which is still negligible compared to random network failures.Bitcoin Core versions <=0.4.0 and >=22.0 ignore valid P2P messages that are received prior to a VERSION message. Bitcoin Core versions between 0.4.0 and 22.0 assign a misbehavior score to the peer upon receiving such messages. v2 clients implementing this proposal will interpret any message other than VERSION received as the first message to be the initiation of a v2 connection, and will result in disconnection for v1 initiators that send any message type other than VERSION as the first message. We are not aware of any implementations where this could pose a problem.
        - If the first 4 received bytes do not match the network magic, but the 12 bytes after that do match the version message encoding above, implementations may interpret this as a v1 peer of a different network, and disconnect them.
        - Similarly generates a random ephemeral private key and sends a corresponding 64-byte ElligatorSwift-encoded public key to the initiator.
        - Similarly may send up to 4095 bytes of garbage data after their public key.

- Both parties:
  - Receive (the remainder of) the full 64-byte public key from the other side.
  - Use X-only**Why use X-only ECDH?** Using only the X coordinate provides the same security as using a full encoding of the secret curve point but allows for more efficient implementation by avoiding the need for square roots to compute Y coordinates. ECDH to compute a shared secret from their private key and the exchanged public keys**Why is the shared secret computation a function of the exact 64-byte public encodings sent?** This makes sure that an attacker cannot modify the public key encoding used without modifying the rest of the stream. If a third party wants the ability to modify stream bytes, they need to perform a full MitM attack on the connection., and deterministically derive from the secret 4 **encryption keys** (two in each direction: one for packet lengths, one for content encryption), a **session id**, and two 16-byte **garbage terminatorsWhat length is sufficient for garbage terminators?** The length of the garbage terminators determines the probability of accidental termination of a legitimate v2 connection due to garbage bytes (sent prior to ECDH) inadvertently including the terminator. 16 byte terminators with 4095 bytes of garbage yield a negligible probability of such collision which is likely orders of magnitude lower than random connection failure on the Internet.**What does a garbage terminator in the wild look like?**



A garbage terminator model TX-v2 in the wild… sent by the responder

(one in each direction) using HKDF-SHA256.

#** Send their 16-byte garbage terminator.**Why does the protocol need a garbage terminator?** While it is in principle possible to use the first packet after the garbage directly as a terminator (scan until a valid packet follows), this would be significantly slower than just scanning for a fixed byte sequence, as it would require recomputing a Poly1305 tag after every received byte.

#** Receive up to 4111 bytes, stopping when encountering the garbage terminator.

#* At this point, both parties have the same keys, and all further communication proceeds in the form of **encrypted packets**.

#** Encrypted packets have an **ignore bit**, which makes them **decoy packets** if set. Decoy packets are to be ignored by the receiver apart from verifying they decrypt correctly. Either peer may send such decoy packets at any point from here on. These form the primary shapability mechanism in the protocol. How and when to use them is out of scope for this document.

#** For each of the two directions, the first encrypted packet that will be sent in that direction (regardless of it being a decoy packet or not) will make use of the associated authenticated data (AAD) feature of the AEAD to authenticate the garbage that has been sent in that direction.**Why does the protocol authenticate the garbage?** Without garbage authentication, the garbage would be modifiable by a third party without consequences. We want to force any active attacker to have to maintain a full protocol state. In addition, such malleability without the consequence of connection termination could enable protocol fingerprinting.

1. The **Version negotiation phase**, where parties negotiate what transport version they will use, as well as data defined by that version.**What features could be added in future protocol versions?** Examples of features that could be added in future versions include post-quantum cryptography upgrades to the handshake, and optional authentication.
    - The responder:
        - Sends a **version packet** with empty content, to indicate support for the v2 P2P protocol proposed by this document. Any other value for content is reserved for future versions.
    - The initiator:
        - Receives a packet, ignores its contents. The idea is that features added by future versions get negotiated based on what is supported by both parties. Since there is just one version so far, the contents here can simply be ignored. But in the future, receiving a non-empty contents here may trigger other behavior; we defer specifying the encoding for such version content until there is a need for it.**How will future versions encode version numbers in the version packet?** Future versions could, for example, specify that the contents of the version packet is to be interpreted as an integer version number (with empty representing 0), and if the minimum of both numbers is N, that being interpreted as choosing a "v2.N" protocol version. Alternatively, certain bytes of the version packet contents could be interpreted as a bitvector of optional features.
        - Sends a **version packet** with empty content as well, to indicate support for the v2 P2P protocol.
    - The responder:
        - Receives a packet, ignores its contents.
2. The **Application phase**, where the packets exchanged have contents to be interpreted as application data.
    - Whenever either peer has a message to send, it sends a packet with that application message as **contents**.

To avoid the recognizable pattern of first messages being at least 64 bytes, a future backwards-compatible upgrade to this protocol may allow both peers to send their public key + garbage + garbage terminator in multiple rounds, slicing those bytes up into messages arbitrarily, as long as progress is guaranteed.**How can progress be guaranteed in a backwards-compatible way?** In order to guarantee progress, it must be ensured that no deadlock occurs, i.e., no state is reached in which each party waits for the other party indefinitely. For example, any upgrade that adheres to the following conditions will guarantee progress:

- The initiator must start by sending at least as many bytes as necessary to mismatch the magic/version 16 bytes prefix.
- The responder must start sending after having received at least one byte that mismatches that 16-byte prefix.
- As soon as either party has received the other peer's garbage terminator, or has received 4095 bytes of garbage, they must send their own garbage terminator. (When either of these conditions is met, the other party has nothing to respond with anymore that would be needed to guarantee progress otherwise.)
- Whenever either party receives any nonzero number of bytes, while not having sent their garbage terminator completely yet, they must send at least one byte in response without waiting for more bytes.
- After either party has sent their garbage terminator, they must transition to the version negotiation phase without waiting for more bytes.

Since the protocol as specified here adheres to these conditions, any upgrade which also adheres to these conditions will be backwards-compatible.

Note that the version negotiation phase does not need to wait for the key exchange phase to complete; version packets can be sent immediately after sending the garbage terminator. So the first two phases together, jointly called **the handshake**, comprise just 1.5 roundtrips:

- the initiator sends public key + garbage
- the responder sends public key + garbage + garbage terminator + decoy packets (optional) + version packet
- the initiator sends garbage terminator + decoy packets (optional) + version packet

Packet encryption overview

All data on the wire after the garbage terminators takes the form of encrypted packets. Every packet encodes an encrypted variable-length byte array, called the **contents**, as well as an **ignore bit** as mentioned before. The total size of a packet is 20 bytes plus the length of its contents.

Each packet consists of:

- A 3-byte encrypted **length** field, encoding the length of the **contents** (between *0* and 2^{24}*-1***Is 2^{24}*-1* bytes sufficient as maximum content size?** The current Bitcoin P2P protocol has no messages which support more than 4000000 bytes of application payload. By supporting up to 2^{24}*-1* we can accommodate future evolutions needing more than 4 times that value. Hypothetical protocol changes that have even more data to exchange than that should probably use multiple separate messages anyway, because of the per-peer receive buffer sizes involved, and the inability to start processing a message before it is fully received. Of course, future versions of the transport protocol could change the size of the length field, if this were really needed., inclusive).

- An authenticated encryption of the **plaintext**, which consists of:
    - A 1-byte **header** which consists of transport layer protocol flags. Currently, only the highest bit is defined as the **ignore bit**. The other bits are ignored, but this may change in future versions**Why is the header a part of the plaintext and not included alongside the length field?** The packet length field is the minimum information that must be available before we can leverage the standard RFC8439 AEAD. Any other data, including metadata like the header being in the content encryption makes it easier to reason about the protocol security w.r.t. data being used before it is authenticated. If the ignore bit was not part of the content, another mechanism would be needed to authenticate it; for example, it could be fed as AAD to the AEAD cipher. We feel the complexity of such an approach outweighs the benefit of saving one byte per message..
    - The variable-length **contents**.

The encryption of the plaintext uses **ChaCha20Poly1305****Why is ChaCha20Poly1305 chosen as the basis for packet encryption?** It is a very widely used authenticated encryption cipher (used among others in SSH, TLS 1.2, TLS 1.3, QUIC, Noise, and WireGuard; in the latter it is currently even the only supported cipher), with very good performance in general purpose software implementations. While AES-based ciphers (including the winners in the CAESAR competition in non-lightweight categories) perform significantly better on systems with AES hardware acceleration, they are also significantly slower in pure software implementations. We choose to optimize for the weakest hardware., an authenticated encryption with associated data (AEAD) cipher specified in RFC 8439. Every packet's plaintext is treated as a separate AEAD message, with a different nonce for each.

The length must be dealt with specially, as it is needed to determine packet boundaries before the whole packet is received and authenticated. As we want a stream that is pseudorandom to a passive attacker, it still needs encryption. We use unauthenticated**Why is the length encryption not separately authenticated?** Informally, the relevant security goal we aim for is to hide the number of packets and their lengths (i.e., the packet boundaries) against a passive attacker that receives the bytestream without timing or fragmentation information. (A formal definition can be found for example in Hansen 2016 (Definition 22) under the name "boundary hiding against chosen-plaintext attacks (BH-CPA)".) However, we do not aim to hide packet boundaries against active attackers because active attackers can always exploit the fact that the Bitcoin P2P protocol is largely query-response based: they can trickle the bytes on the stream one-by-one unmodified and observe when a response comes (see Hansen 2016 (Section 3.9) for a in-depth discussion). With that in mind, we accept that an active (non-MitM) attacker is able to figure out some information about packet boundaries by flipping certain bits in the unauthenticated length field, and observing the other side disconnecting immediately or later. Thus, we choose to use unauthenticated encryption for the length data, which is sufficient to achieve boundary hiding against passive attackers, and saves 16 bytes of bandwidth per packet. **ChaCha20** encryption for this, with an independent key. Note that the plaintext length is still implicitly authenticated by the encryption of the plaintext, but this can only be verified after receiving the whole packet. This design is inspired by that of the ChaCha20Poly1305 cipher suite in OpenSSH.**How does packet encryption differ from the OpenSSH design?** The differences are:

- The length field is only 3 bytes instead of 4, as that is sufficient for our purposes.
- Length encryption keeps drawing pseudorandom bytes from the same ChaCha20 cipher for multiple packets, rather than incrementing the nonce for every packet.
- The Poly1305 authentication tag only covers the encrypted plaintext, and not the encrypted length field. This means that plaintext encryption uses the standard ChaCha20Poly1305 construction without any modifications, maximizing applicability of analysis and review of that cipher. The length encryption can be seen as a separate layer, using a separate key, and thus cannot affect any of the confidentiality or

integrity guarantees of the plaintext encryption. On the other hand, this change w.r.t. OpenSSH also does not worsen any properties, as incorrect lengths will still trigger authentication failure for the overall packet (the plaintext length is implicitly authenticated by ChaCha20Poly1305).

- A hash step is performed every 224**How was the rekeying interval 224 chosen?** Assuming a node sends only ping messages every 20 minutes (the timeout interval for post-BIP31 connections) on a connection, the node will transmit 224 packets in about 3.11 days. This means *soft rekeying* after a fixed number of packets automatically translates to an upper-bound of time interval for rekeying, while being much simpler to coordinate than an actual time-based rekeying regime. At the same time, doing it once every 224 messages is sufficiently infrequent that it has only negligible impact on performance. Furthermore, 224 times 3 bytes (the number of bytes consumed by each length encryption) is 672, which is a multiple of 64 minus 32. This means that at the end of 224 length encryptions, exactly 32 bytes of keystream data remain that can be used as next key. messages to rekey the encryption ciphers, in order to provide forward security.

```
Because only fixed-length chunks (3-
byte length fields) are encrypted, we do not need to treat all length chunks as separate messages. Instead,
it acceptable to use a less standard construction for length
encryption? The fact that multiple (non-
overlapping) bytes generated by a single ChaCha20 cipher are used for the encryption of multiple consecutive
```

In order to provide forward security**What value does forward security provide?** Re-keying ensures forward secrecy within a session, i.e., an attacker compromising the current session secrets cannot derive past encryption keys in the same session.**Why have a cipher with forward secrecy but no periodical refresh of the ECDH key exchange?** Our cipher ratchets encryption keys forward in order to protect messages encrypted under *past* encryption keys. In contrast, re-performing ECDH key exchange would protect messages encrypted under *future* encryption keys, i.e., it would re-establish security after the attacker had compromised one of the peers *temporarily* (e.g., the attacker obtains a memory dump). We do not believe protecting against that is a priority: an attacker that, for whatever reason, is capable of an attack that reveals encryption keys (or other session secrets) of a peer once is likely capable of performing the same attack again after peers have re-performed the ECDH key exchange. Thus, we do not believe the benefits of re-performing key exchange outweigh the additional complexity that comes with the necessary coordination between the peers. We note that the initiator could choose to close and re-open the entire connection to force a refresh of the ECDH key exchange, but that introduces other issues: a connection slot needs to be kept open at the responder side, it is not cryptographically guaranteed that really the same initiator will use it, and the observable TCP reset and handshake may create a detectable pattern., the encryption keys for both plaintext and length encryption are cycled every 224 messages, by switching to a new key that is generated by the key stream using the old key.

Handshake: key exchange and version negotiation

Next we specify the handshake of a connection in detail.

As explained before, these messages are sent to set up the connection:

```
------------------------------------------------------------------------------------------------
----
 | Initiator
Responder                                                    |
```


x, ellswift_X = ellswift_create()
ellswift_X + initiator_garbage (initiator_garbage_len bytes; max 4095)
y, ellswift_Y = ellswift_create()
ecdh_secret = v2_ecdh(y, ellswift_X, ellswift_Y, initiating=False)
initialize_v2_transport(initiator, ecdh_secret, initiating=False)
ellswift_Y + responder_garbage (responder_garbage_len bytes; max 4095)
responder_garbage_terminator (16 bytes)
v2_enc_packet(initiator, RESPONDER_TRANSPORT_VERSION, aad=responder_garbage)
ecdh_secret = v2_ecdh(x, ellswift_Y, ellswift_X, initiating=True)
initialize_v2_transport(responder, ecdh_secret, initiating=True)
initiator_garbage_terminator (16 bytes)
v2_enc_packet(responder, INITIATOR_TRANSPORT_VERSION, aad=initiator_garbage)

Shared secret computation

The peers derive their shared secret through X-only ECDH, hashed together with the exactly 64-byte public keys' encodings sent over the wire.

```
def v2_ecdh(priv, ellswift_theirs, ellswift_ours, initiating):
    ecdh_point_x32 = ellswift_ecdh_xonly(ellswift_theirs, priv)
    if initiating:
        # Initiating, place our public key encoding first.
        return sha256_tagged("bip324_ellswift_xonly_ecdh", ellswift_ours + ellswift_theirs +
ecdh_point_x32)
    else:
        # Responding, place their public key encoding first.
        return sha256_tagged("bip324_ellswift_xonly_ecdh", ellswift_theirs + ellswift_ours +
ecdh_point_x32)
```

Here, `sha256_tagged(tag, x)` returns a tagged hash value `SHA256(SHA256(tag) || SHA256(tag) || x)` as in BIP340.

ElligatorSwift encoding of curve X coordinates

The functions `ellswift_create` and `ellswift_ecdh_xonly` encapsulate the construction of ElligatorSwift-encoded public keys, and the computation of X-only ECDH with ElligatorSwift-encoded public keys.

First we define a constant:

- Let c = *0xa2d2ba93507f1df233770c2a797962cc61f6d15da14ecd47d8d27ae1cd5f852*.**What is the *c* constant used in *XSwiftEC*?** The algorithm requires a constant $\sqrt{-3}$ *(mod p)*; in other words, a number c such that $-c^2 \bmod p = 3$. There are two solutions to this equation, one which is itself a square modulo p, and its negation. We choose the square one.

To define the needed functions, we first introduce a helper function, matching the `XSwiftEC` function from the SwiftEC paper, instantiated for the secp256k1 curve, with minor modifications. It maps pairs of integers *(u, t)* (both in range *0..p-1*) to valid X coordinates on the curve. Note that the specification here does not attempt to be constant time, as it does not operate on secret data. In what follows, we use the notation from BIP340.

- *XSwiftEC(u, t)*:
  - Alter the inputs to guarantee an X coordinate on the curve:**Why do the inputs to the XSwiftEC algorithm need to be altered?** This step deviates from the paper, which maps a negligibly small subset of inputs (around $3/2^{256}$) to the point at infinity. To avoid the need to deal with the case where a peer could craft encodings that intentionally trigger this edge case, we remap them to inputs that yield a valid X coordinate.
    - If $u \bmod p = 0$, let $u = 1$ instead.
    - If $t \bmod p = 0$, let $t = 1$ instead.
    - If $(u^3 + t^2 + 7) \bmod p = 0$, let $t = 2t \pmod p$ instead.
  - Let $X = (u^3 + 7 - t^2)/(2t) \pmod p$.**What does the division (/) sign in modular arithmetic refer to?** Note that the division in these expressions corresponds to multiplication with the modular inverse modulo p, i.e. $a / b \pmod p$ with nonzero b is the unique solution x for which $bx = a \pmod p$. It can be computed as $ab^{p-2} \pmod p$, but more efficient algorithms exist.
  - Let $Y = (X + t)/(cu) \pmod p$.

    - For every x in $\{u + 4Y^2, (-X/Y - u)/2, (X/Y - u)/2\}$ (all *mod p*; the order matters):
        - If *lift_x(x)* succeeds, return x. There is at least one such x.

To find encodings of a given X coordinate x, we first need the inverse of *XSwiftEC*. The function *XSwiftECInv(x, u, case)* either returns t such that *XSwiftEC(u, t) = x*, or *None*. The *case* variable is an integer in range *0..7*, which selects which of the up to 8 valid such t values to return:

- *XSwiftECInv(x, u, case)*:
    - If *case & 2 = 0*:
        - If *lift_x(-x - u)* succeeds, return *None*.
        - Let $v = x$.
        - Let $s = -(u^3 + 7)/(u^2 + uv + v^2)$ *(mod p)*.
    - Else (*case & 2 = 2*):
        - Let $s = x - u$ *(mod p)*.
        - If $s = 0$, return *None*.
        - Let r be the square root of $-s(4(u^3 + 7) + 3u^2s)$ *(mod p)*.**How to compute a square root mod *p*?** Due to the structure of p, a candidate for the square root of a mod p can be computed as $x = a^{(p+1)/4}$ *mod p*. If a is not a square mod p, this formula returns the square root of $-a$ *mod p* instead, so it is necessary to verify that x^2 *mod p* $= a$. If that is the case $-x$ *mod p* is a solution too, but we define "the" square root to be equal to that expression (the square root will therefore always be a square itself, as *(p+1)/4* is even). This algorithm is a specialization of the Tonelli-Shanks algorithm. Return *None* if it does not exist.
        - If *case & 1 = 1* and $r = 0$, return *None*.
        - Let $v = (r/s - u)/2$.
    - Let w be the square root of s *(mod p)*. Return *None* if it does not exist.
    - If *case & 5 = 0*, return $-w(u(1 - c)/2 + v)$.
    - If *case & 5 = 1*, return $w(u(1 + c)/2 + v)$.
    - If *case & 5 = 4*, return $w(u(1 - c)/2 + v)$.
    - If *case & 5 = 5*, return $-w(u(1 + c)/2 + v)$.

The overall *XElligatorSwift* algorithm, matching the name used in the paper, then uses this inverse to randomly***Can the ElligatorSwift encoding be used to construct public key encodings that satisfy a certain structure (and not pseudorandom)?*** *The algorithm chooses the first 32 bytes (i.e., the value*u*) and then computes a corresponding*t*such that the mapping to the curve point holds. In general, picking*u*from a uniformly random distribution provides pseudorandomness. But we can also fix any of the 32 bytes in*u*, and the algorithm will still find a corresponding*t*. The fact that it is possible to fix the first 32 bytes, combined with the garbage bytes in the handshake, provides a limited but very simple method of parroting other protocols such as TLS 1.3, which can be deployed by one of the peers without explicit support from the other peer. More general methods of parroting, e.g., introduced by defining new protocol or a protocol upgrade, are not precluded. sample encodings of*x*":

- *XElligatorSwift(x)*:
    - Loop:
        - Let u be a random non-zero integer in range *1..p-1* inclusive.
        - Let *case* be a random integer in range *0..7* inclusive.
        - Compute t = *XSwiftECInv(x, u, case)*.
        - If t is not *None*, return *(u, t)*. Otherwise, restart loop.

This is used to define the `ellswift_create` algorithm used in the previous section; it generates a random private key, along with a uniformly sampled 64-byte ElligatorSwift-encoded public key corresponding to it:

- *ellswift_create()*:
  - Generate a random private key *priv* in range *1..p-1*.
  - Let *P = priv·G*, the corresponding public key point to *priv*.
  - Let *(u, t) = XElligatorSwift(x(P))*, an encoding of *x(P)*.
  - *ellswift_pub = bytes(u) || bytes(t)*, its encoding as 64 bytes.
  - Return *(priv, ellswift_pub)*.

Finally the `ellswift_ecdh_xonly` algorithm is:

- *ellswift_ecdh_xonly(ellswift_theirs, priv)*:
  - Let *u = int(ellswift_theirs[:32]) mod p*.
  - Let *t = int(ellswift_theirs[32:]) mod p*.
  - Return *bytes(x(priv·lift_x(XSwiftEC(u, t))))*.**Does it matter which point *lift_x* maps to?** Either point is valid, as they are negations of each other, and negations do not affect the output X coordinate.

Keys and session ID derivation

The authenticated encryption construction proposed here requires two 32-byte keys per communication direction. These (in addition to a session ID) are computed using HKDF**Why use HKDF for deriving key material?** The shared secret already involves a hash function to make sure the public key encodings contribute to it, which negates some of the need for HKDF already. We still use it as it is the standard mechanism for deriving many keys from a single secret, and its computational cost is low enough to be negligible compared to the rest of a connection setup. as specified in RFC 5869 with SHA256 as the hash function:

```
def initialize_v2_transport(peer, ecdh_secret, initiating):
    # Include NETWORK_MAGIC to ensure a connection between nodes on different networks will
immediately fail
    prk = HKDF_Extract(Hash=sha256, salt=b'bitcoin_v2_shared_secret' + NETWORK_MAGIC,
ikm=ecdh_secret)

    peer.session_id = HKDF_Expand(Hash=sha256, PRK=prk, info=b'session_id', L=32)

    # Initialize the packet encryption ciphers.
    initiator_L = HKDF_Expand(Hash=sha256, PRK=prk, info=b'initiator_L', L=32)
    initiator_P = HKDF_Expand(Hash=sha256, PRK=prk, info=b'initiator_P', L=32)
    responder_L = HKDF_Expand(Hash=sha256, PRK=prk, info=b'responder_L', L=32)
    responder_P = HKDF_Expand(Hash=sha256, PRK=prk, info=b'responder_P', L=32)
    garbage_terminators = HKDF_Expand(Hash=sha256, PRK=prk, info=b'garbage_terminators', L=32)
    initiator_garbage_terminator = garbage_terminators[:16]
    responder_garbage_terminator = garbage_terminators[16:]

    if initiating:
        peer.send_L = FSChaCha20(initiator_L)
        peer.send_P = FSChaCha20Poly1305(initiator_P)
        peer.send_garbage_terminator = initiator_garbage_terminator
        peer.recv_L = FSChaCha20(responder_L)
        peer.recv_P = FSChaCha20Poly1305(responder_P)
        peer.recv_garbage_terminator = responder_garbage_terminator
    else:
```

```
        peer.send_L = FSChaCha20(responder_L)
        peer.send_P = FSChaCha20Poly1305(responder_P)
        peer.send_garbage_terminator = responder_garbage_terminator
        peer.recv_L = FSChaCha20(initiator_L)
        peer.recv_P = FSChaCha20Poly1305(initiator_P)
        peer.recv_garbage_terminator = initiator_garbage_terminator

    # To achieve forward secrecy we must wipe the key material used to initialize the ciphers:
    memory_cleanse(ecdh_secret, prk, initiator_L, initiator_P, responder_L, responder_P)
```

The session ID uniquely identifies the encrypted channel. v2 clients supporting this proposal may present the entire session ID (encoded as a hex string) to the node operator to allow for manual, out of band comparison with the peer node operator. Future transport versions may introduce optional authentication methods that compare the session ID as seen by the two endpoints in order to bind the encrypted channel to the authentication.

Overall handshake pseudocode

To establish a v2 encrypted connection, the initiator generates an ephemeral secp256k1 keypair and sends an unencrypted ElligatorSwift encoding of the public key to the responding peer followed by unencrypted pseudorandom bytes `initiator_garbage` of length `garbage_len < 4096`.

```
def initiate_v2_handshake(peer, garbage_len):
    peer.privkey_ours, peer.ellswift_ours = ellswift_create()
    peer.sent_garbage = rand_bytes(garbage_len)
    send(peer, peer.ellswift_ours + peer.sent_garbage)
```

The responder generates an ephemeral keypair for itself and derives the shared ECDH secret (using the first 64 received bytes) which enables it to instantiate the encrypted transport. It then sends 64 bytes of the unencrypted ElligatorSwift encoding of its own public key and its own `responder_garbage` also of length `garbage_len < 4096`. If the first 16 bytes received match the v1 prefix, the v1 protocol is used instead.

```
TRANSPORT_VERSION = b''
NETWORK_MAGIC = b'\xf9\xbe\xb4\xd9' # Mainnet network magic; differs on other networks.
V1_PREFIX = NETWORK_MAGIC + b'version\x00\x00\x00\x00\x00'

def respond_v2_handshake(peer, garbage_len):
    peer.received_prefix = b""
    while len(peer.received_prefix) < len(V1_PREFIX):
        peer.received_prefix += receive(peer, 1)
        if peer.received_prefix[-1] != V1_PREFIX[len(peer.received_prefix) - 1]:
            peer.privkey_ours, peer.ellswift_ours = ellswift_create()
            peer.sent_garbage = rand_bytes(garbage_len)
            send(peer, peer.ellswift_ours + peer.sent_garbage)
            return
    use_v1_protocol()
```

Upon receiving the encoded responder public key, the initiator derives the shared ECDH secret and instantiates the encrypted transport. It then sends the derived 16-byte `initiator_garbage_terminator`, optionally followed by an arbitrary number of decoy packets. Afterwards, it receives the responder's garbage (delimited by the garbage terminator). The responder performs very similar steps but includes the earlier received prefix bytes in the public key. Both the initiator and the responder set the AAD of the first encrypted

packet they send after the garbage terminator (i.e., either an optional decoy packet or the version packet) to the garbage they have just sent, not including the garbage terminator.

```
def complete_handshake(peer, initiating, decoy_content_lengths=[]):
    received_prefix = b'' if initiating else peer.received_prefix
    ellswift_theirs = received_prefix + receive(peer, 64 - len(received_prefix))
    if not initiating and ellswift_theirs[4:16] == V1_PREFIX[4:16]:
        # Looks like a v1 peer from the wrong network.
        disconnect(peer)
    ecdh_secret = v2_ecdh(peer.privkey_ours, ellswift_theirs, peer.ellswift_ours,
                          initiating=initiating)
    initialize_v2_transport(peer, ecdh_secret, initiating=initiating)
    # Send garbage terminator
    send(peer, peer.send_garbage_terminator)
    # Optionally send decoy packets after garbage terminator.
    aad = peer.sent_garbage
    for decoy_content_len in decoy_content_lengths:
        send(v2_enc_packet(peer, decoy_content_len * b'\x00', aad=aad))
        aad = b''
    # Send version packet.
    send(v2_enc_packet(peer, TRANSPORT_VERSION, aad=aad))
    # Skip garbage, until encountering garbage terminator.
    received_garbage = recv(peer, 16)
    for i in range(4096):
        if received_garbage[-16:] == peer.recv_garbage_terminator:
            # Receive, decode, and ignore version packet.
            # This includes skipping decoys and authenticating the received garbage.
            v2_receive_packet(peer, aad=received_garbage[:-16])
            return
        else:
            received_garbage += recv(peer, 1)
    # Garbage terminator was not seen after 4 KiB of garbage.
    disconnect(peer)
```

Packet encryption

Lastly, we specify the packet encryption cipher in detail.

Existing cryptographic primitives

Packet encryption is built on two existing primitives:

- **ChaCha20Poly1305** is specified as `AEAD_CHACHA20_POLY1305` in RFC 8439 section 2.8. It is an authenticated encryption protocol with associated data (AEAD), taking a 256-bit key, 96-bit nonce, and an arbitrary-length byte array of associated authenticated data (AAD). Due to the built-in authentication tag, ciphertexts are 16 bytes longer than the corresponding plaintext. In what follows:
  - `aead_chacha20_poly1305_encrypt(key, nonce, aad, plaintext)` refers to a function that takes as input a 32-byte array *key*, a 12-byte array *nonce*, an arbitrary-length byte array *aad*, and an arbitrary-length byte array *plaintext*, and returns a byte array *ciphertext*, 16 bytes longer than the plaintext.
  - `aead_chacha20_poly1305_decrypt(key, nonce, aad, ciphertext)` refers to a function that takes as input a 32-byte array *key*, a 12-byte array *nonce*, an arbitrary-length byte array *aad*, and an

arbitrary-length byte array *ciphertext*, and returns either a byte array *plaintext* (16 bytes shorter than the ciphertext), or *None* in case the ciphertext was not a valid ChaCha20Poly1305 encryption of any plaintext with the specified *key*, *nonce*, and *aad*.

- The **ChaCha20 Block Function** is specified in RFC 8439 section 2.3. It is a pseudorandom function (PRF) taking a 256-bit key, 96-bit nonce, and 32-bit counter, and outputs 64 pseudorandom bytes. It is the underlying building block on which ChaCha20 (and ultimately, ChaCha20Poly1305) is built. In what follows:
  - `chacha20_block(key, nonce, count)` refers to a function that takes as input a 32-byte array *key*, a 12-byte array *nonce*, and an integer *count* in range $0..2^{32}-1$, and returns a byte array of length 64.

These will be used for plaintext encryption and length encryption, respectively.

Rekeying wrappers: FSChaCha20Poly1305 and FSChaCha20

To provide re-keying every 224 packets, we specify two wrappers.

The first is **FSChaCha20Poly1305**, which represents a ChaCha20Poly1305 AEAD, which automatically changes the nonce after every message, and rekeys every 224 messages by encrypting 32 zero bytes**Why is rekeying implemented in terms of an invocation of the AEAD?** This means the FSChaCha20Poly1305 wrapper can be thought of as a pure layer around the ChaCha20Poly1305 AEAD. Actual implementations can take advantage of the fact that this formulation is equivalent to using byte 64 through 95 of the keystream output of the underlying ChaCha20 cipher as new key, avoiding the need for Poly1305 in the process., and using the first 32 bytes of the result. Each message will be used for one packet. Note that in our protocol, any FSChaCha20Poly1305 instance is always either exclusively encryption or exclusively decryption, as separate instances are used for each direction of the protocol. The nonce used for a message is composed of the 32-bit little-endian encoding of the number of messages with the current key, followed by the 64-bit little-endian encoding of the number of rekeyings performed. For rekeying, the first 32-bit integer is set to *0xffffffff*.

```
REKEY_INTERVAL = 224

class FSChaCha20Poly1305:
    """Rekeying wrapper AEAD around ChaCha20Poly1305."""

    def __init__(self, initial_key):
        self.key = initial_key
        self.packet_counter = 0

    def crypt(self, aad, text, is_decrypt):
        nonce = ((self.packet_counter % REKEY_INTERVAL).to_bytes(4, 'little') +
                 (self.packet_counter // REKEY_INTERVAL).to_bytes(8, 'little'))
        if is_decrypt:
            ret = aead_chacha20_poly1305_decrypt(self.key, nonce, aad, text)
        else:
            ret = aead_chacha20_poly1305_encrypt(self.key, nonce, aad, text)
        if (self.packet_counter + 1) % REKEY_INTERVAL == 0:
            rekey_nonce = b"\xFF\xFF\xFF\xFF" + nonce[4:]
            self.key = aead_chacha20_poly1305_encrypt(self.key, rekey_nonce, b"", b"\x00" * 32)
[:32]
        self.packet_counter += 1
        return ret
```

```
    def decrypt(self, aad, ciphertext):
        return self.crypt(aad, ciphertext, True)

    def encrypt(self, aad, plaintext):
        return self.crypt(aad, plaintext, False)
```

The second is **FSChaCha20**, a (single) stream cipher which is used for the lengths of all packets. Encryption and decryption are identical here, so a single function `crypt` is exposed. It XORs the input with bytes generated using the ChaCha20 block function, rekeying every 224 chunks using the next 32 bytes of the block function output as new key. A *chunk* refers here to a single invocation of `crypt`. As explained before, the same cipher is used for 224 consecutive chunks, to avoid wasting cipher output. The nonce used for these batches of 224 chunks is composed of 4 zero bytes followed by the 64-bit little-endian encoding of the number of rekeyings performed. The block counter is reset to 0 after every rekeying.

```
class FSChaCha20:
    """Rekeying wrapper stream cipher around ChaCha20."""

    def __init__(self, initial_key):
        self.key = initial_key
        self.block_counter = 0
        self.chunk_counter = 0
        self.keystream = b''

    def get_keystream_bytes(self, nbytes):
        while len(self.keystream) < nbytes:
            nonce = ((0).to_bytes(4, 'little') +
                     (self.chunk_counter // REKEY_INTERVAL).to_bytes(8, 'little'))
            self.keystream += chacha20_block(self.key, nonce, self.block_counter)
            self.block_counter += 1
        ret = self.keystream[:nbytes]
        self.keystream = self.keystream[nbytes:]
        return ret

    def crypt(self, chunk):
        ks = self.get_keystream_bytes(len(chunk))
        ret = bytes([ks[i] ^ chunk[i] for i in range(len(chunk))])
        if ((self.chunk_counter + 1) % REKEY_INTERVAL) == 0:
            self.key = self.get_keystream_bytes(32)
            self.block_counter = 0
        self.chunk_counter += 1
        return ret
```

Overall packet encryption and decryption pseudocode

Encryption and decryption of packets then follow by composing the ciphers from the previous section as building blocks.

```
LENGTH_FIELD_LEN = 3
HEADER_LEN = 1
IGNORE_BIT_POS = 7

def v2_enc_packet(peer, contents, aad=b'', ignore=False):
    assert len(contents) <= 2**24 - 1
    header = (ignore << IGNORE_BIT_POS).to_bytes(HEADER_LEN, 'little')
```

```
    plaintext = header + contents
    aead_ciphertext = peer.send_P.encrypt(aad, plaintext)
    enc_contents_len = peer.send_L.encrypt(len(contents).to_bytes(LENGTH_FIELD_LEN, 'little'))
    return enc_contents_len + aead_ciphertext

CHACHA20POLY1305_EXPANSION = 16

def v2_receive_packet(peer, aad=b''):
    while True:
        enc_contents_len = receive(peer, LENGTH_FIELD_LEN)
        contents_len = int.from_bytes(peer.recv_L.crypt(enc_contents_len), 'little')
        aead_ciphertext = receive(peer, HEADER_LEN + contents_len + CHACHA20POLY1305_EXPANSION)
        plaintext = peer.recv_P.decrypt(aad, aead_ciphertext)
        if plaintext is None:
            disconnect(peer)
            break
        # Only the first packet is expected to have non-empty AAD.
        aad = b''
        header = plaintext[:HEADER_LEN]
        if not (header[0] & (1 << IGNORE_BIT_POS)):
            return plaintext[HEADER_LEN:]
```

Performance

Each v1 P2P message uses a double-SHA256 checksum truncated to 4 bytes. Roughly the same amount of computation power is required for encrypting and authenticating a v2 P2P message as proposed.

Application layer specification

v2 Bitcoin P2P message structure

v2 Bitcoin P2P transport layer packets use the encrypted message structure shown above. An unencrypted application layer **contents** is composed of:

Field	Size in bytes	Comments
`message_type`	1 or 13	either a one byte ID in range *1..255* or `b'\x00'` followed by a 12-byte ASCII message type (as in the v1 P2P protocol)
`message_payload`	`message_length`	message payload

If the first byte of `message_type` is `b'\x00'`, the following 12 bytes are interpreted as an ASCII message type (as in the v1 P2P protocol), trailing padded with `b'\x00'` as necessary. If the first byte of `message_type` is in the range *1..255*, it is interpreted as a message type ID. This structure results in smaller messages than the v1 protocol, as most messages sent/received will have a message type ID. We recommend reserving 1-byte type IDs for message types that are sent more than once per direction per connection.**How do the lengths between v1 and v2 compare?** For messages that use the 1-byte short message type ID, v2 packets use 3 bytes less per message than v1.**Why not allow variable length long message type IDs?** Allowing for variable length long IDs reduces the available 1-byte ID space by 12 (to encode the length itself) and incentivizes less descriptive message types. In addition, limiting message types to fixed lengths of 1 or 13 hampers traffic analysis.

The value of `message_length` is **length** minus the size of the `message_type`.

The following table lists currently defined message type IDs and the 12-byte ASCII message type (trimmed of trailing padding) that they are treated as equivalent to:

	0	**1**	**2**	**3**
+0	(12 bytes follow)	`addr`	`block`	`blocktxn`
+4	`cmpctblock`	`feefilter`	`filteradd`	`filterclear`
+8	`filterload`	`getblocks`	`getblocktxn`	`getdata`
+12	`getheaders`	`headers`	`inv`	`mempool`
+16	`merkleblock`	`notfound`	`ping`	`pong`
+20	`sendcmpct`	`tx`	`getcfilters`	`cfilter`
+24	`getcfheaders`	`cfheaders`	`getcfcheckpt`	`cfcheckpt`
+28	`addrv2`			
≥29	| colspan="4" | (undefined)			

When a message type has both a 1-byte encoding and a 13-byte encoding defined, peers that support receiving that message type should accept messages using either encoding (e.g., if the "getblocktxn" message type is supported, then both the 1-byte `b'\x0a'` encoding and the 13-byte `b'\x00getblocktxn\x00'` should be supported, and behavior should not depend on which of the two encodings is received).

Additional message type IDs may be defined by other BIPs. They should be added to the message type IDs table to ease coordination.

Signaling specification

Signaling v2 support

Peers supporting the v2 transport protocol signal support by advertising the `NODE_P2P_V2 = (1 << 11)` service flag in addr relay. If met with immediate disconnection when establishing a v2 connection, clients implementing this proposal are encouraged to retry connecting using the v1 protocol.**Why are v2 clients met with immediate disconnection encouraged to retry with a v1 connection?** Service flags propagated through untrusted intermediaries using ADDR and ADDRV2 P2P messages and are OR'ed when received from multiple sources. An untrusted intermediary could falsely advertise a potential peer as supportive of v2 connections. Connection downgrades to v1 mitigate the risk of a network participant being blackholed via false advertising.

Test Vectors

For development and testing purposes, we provide a collection of test vectors in CSV format, and a naive, highly inefficient, reference implementation of the relevant algorithms. This code is for demonstration purposes only:

- XElligatorSwift decoding vectors provide examples of ElligatorSwift-encoded public keys, and the X coordinate they map to.

- XSwiftECInv vectors provide examples of *(u, x)* pairs, and the various *t* values that *xswiftec_inv* maps them to.
- Packet encoding vectors illustrate the lifecycle of the authenticated encryption scheme proposed in this document.

Changelog

```
* 1.0.2 (2026-01-30)
  * Add message type ID table in auxiliary file
* 1.0.1 (2026-01-16)
  * Specify equivalence of 1-byte and 13-byte `message_type`
* 1.0.0 (2024-07-10)
  * Marked as Final
```

Rationale and References

Acknowledgements

Thanks to everyone (last name order) that helped invent and develop the ideas in this proposal:

- Matt Corallo
- Lloyd Fournier
- Gregory Maxwell
- Anthony Towns

```
BIP: 339
Layer: Peer Services
Title: WTXID-based transaction relay
Authors: Suhas Daftuar <sdaftuar@chaincode.com>
Status: Deployed
Type: Specification
Assigned: 2020-02-03
License: BSD-2-Clause
```

BIP 339

Abstract

This BIP describes two changes to the p2p protocol to support transaction relay based on the BIP 141 wtxid of a transaction, rather than its txid.

Motivation

Historically, the inv messages sent on the Bitcoin peer-to-peer network to announce transactions refer to transactions by their txid, which is a hash of the transaction that does not include the witness (see BIP 141). This has been the case even since Segregated Witness (BIP 141/143/144) has been adopted by the network.

Not committing to the witness in transaction announcements creates inefficiencies: because a transaction's witness can be malleated without altering the txid, a node in receipt of a witness transaction that the node does

not accept will generally still download that same transaction when announced by other peers. This is because the alternative – of not downloading a given txid after rejecting a transaction with that txid – would allow a third party to interfere with transaction relay by malleating a transaction's witness and announcing the resulting invalid transaction to nodes, preventing relay of the valid version of the transaction as well.

We can eliminate this concern by using the wtxid in place of the txid when announcing and fetching transactions.

Specification

1. A new wtxidrelay message is added, which is defined as an empty message where pchCommand == "wtxidrelay".
2. The protocol version of nodes implementing this BIP must be set to 70016 or higher.
3. The wtxidrelay message MUST be sent in response to a version message from a peer whose protocol version is >= 70016 and prior to sending a verack. A wtxidrelay message received after a verack message MUST be ignored or treated as invalid.
4. A new inv type MSG_WTX (0x00000005) is added, for use in both inv messages and getdata requests, indicating that the hash being referenced is a transaction's wtxid. In the case of getdata requests, MSG_WTX implies that the transaction being requested should be serialized with witness as well, as described in BIP 144.
5. After a node has received a wtxidrelay message from a peer, the node MUST use the MSG_WTX inv type when announcing transactions to that peer.
6. After a node has received a wtxidrelay message from a peer, the node SHOULD use a MSG_WTX getdata message to request any announced transactions. A node MAY still request transactions from that peer using MSG_TX getdata messages, such as for transactions not recently announced by that peer (like the parents of recently announced transactions).

Backward compatibility

As wtxid-based transaction relay is only enabled between peers that both support it, older clients remain fully compatible and interoperable after this change.

Implementation

https://github.com/bitcoin/bitcoin/pull/18044

Copyright

This BIP is licensed under the 2-clause BSD license.

```
BIP: 434
Layer: Peer Services
Title: Peer Feature Negotiation
Authors: Anthony Towns <aj@erisian.com.au>
Status: Complete
Type: Specification
Assigned: 2026-01-14
License: BSD-2-Clause
Discussion: 2025-12-19: https://gnusha.org/pi/bitcoindev/aUUXLgEUCgGb122o@erisian.com.au/T/#u
            2020-08-21: https://gnusha.org/pi/bitcoindev/
```

```
20200821023647.7eat4goqqrtaqnna@erisian.com.au/
  Version: 1.0.0
```

BIP 434

Abstract

This BIP defines a peer-to-peer (P2P) message that can be used for announcements and negotiation related to support of new peer-to-peer features.

Motivation

Historically, new peer-to-peer protocol changes have been tied to bumping the protocol version, so that nodes know to only attempt feature negotiation with peers that support the feature. Coordinating the protocol version across implementations, when different clients may have different priorities for features to implement, is an unnecessary burden in the upgrade process for P2P features that do not require universal support. And at a more philosophical level, having the P2P protocol be permissionlessly extensible, with no coordination required between implementations or developers, seems ideal for a decentralized system.

Many earlier P2P protocol upgrades were implemented as new messages sent after a peer connection is set up (ie, after receipt of a `verack` message by both sides). See BIP 130 (sendheaders), BIP 133 (feefilter), and BIP 152 (compact blocks) for some examples. However, for some P2P upgrades, it is helpful to perform feature negotiation prior to a connection being fully established (ie, prior to the `verack` being received by both sides). BIP 155 (addrv2) and BIP 339 (wtxid-relay) are examples of this approach, which involves sending and receiving a single new message (`sendaddrv2` and `wtxidrelay` respectively), in between `version` and `verack` to indicate support of the new feature.

In all these cases, sending new messages on the network raises the question of what non-implementing software will do with such messages. The common behavior observed on the network was for software to ignore unknown messages received from a peer, so these proposals posed minimal risk of potential network partitioning. In fact, supporting protocol extensibility in this manner was given as an explicit reason to ignore unknown messages in Bitcoin's first release.

However, if nodes respond to unknown messages by disconnecting, then the network might partition in the future as incompatible software is deployed. And in fact, some clients on the network have historically discouraged or disallowed unknown messages, both between `version` and `verack` (eg, Bitcoin Core discouraged such messages between PR#9720 and PR#19723, and btcd disallowed such messages until PR#1812, but see also discussion in #1661), as well as after `verack`.

To maximise compatibility with such clients, most of these BIPs require that peers bump the protocol version:

- BIP 130 requires version 70012 or higher,
- BIP 133 requires version 70013 or higher,
- BIP 152 recommends version 70014/70015 or higher, and
- BIP 339 requires version 70016 or higher.

And while BIP 155 does not specify a minimum protocol version, implementations have added a de facto requirement of version 70016 or higher.

In this BIP, we propose codifying and generalising the mechanism used by BIP 339 for future P2P upgrades, by adding a single new feature negotiation message that can be reused for advertising arbitrary new features, and requiring that implementing software ignore unknown features that might be advertised. This allows future upgrades to negotiate new features by exchanging messages prior to exchanging `verack` messages, without concerns of being unnecessarily disconnected by a peer which doesn't understand the messages, and without needing to coordinate updating the protocol version.

Specification

The key words "MUST", "MUST NOT", "REQUIRED", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119.

For the purposes of this section, `CompactSize` refers to the variable-length integer encoding used across the existing P2P protocol to encode array lengths, among other things, in 1, 3, 5 or 9 bytes. Only `CompactSize` encodings which are minimally-encoded (ie the shortest length possible) are used by this specification.

Nodes implementing this BIP:

- MUST advertise a protocol version number `>= 70017`,
- MUST NOT send `feature` messages to peers that advertise a protocol version number `< 70017`,
- MUST accept `feature` messages received after the `version` message and before the `verack` message, and
- MUST NOT send `feature` messages after sending the `verack` message.

In addition, nodes implementing this BIP:

- SHOULD ignore unknown messages received after the `version` message and before the `verack` message,
- MAY ignore `feature` messages sent after `verack`, and
- MAY disconnect peers who send `feature` messages after `verack`.

Feature specifications based on this BIP:

- MUST forbid sending messages it introduces after `verack` to a peer that has not indicated support for the feature via a `feature` message.

`feature` message

The payload of the `feature` message contains exactly the following data:

Type	Name	Description
string	`featureid`	Unique identifier for the feature
byte-vector	`featuredata`	Feature-specific configuration data

The `featureid` is encoded in the usual way, that is, as a `CompactSize` specifying the string length, followed by that many bytes. The string length MUST be between 4[25] and 80[26], inclusive. The string SHOULD include only printable ASCII characters (ie, each byte should have a value between 32 and 126, inclusive).

Likewise, `featuredata` is encoded as a `CompactSize` specifying the byte-vector size, followed by that many bytes. How these bytes are interpreted is part of the feature's specification. The byte-vector size MUST NOT be

more than 512 bytes[27]. Note that the `featuredata` field is not optional, so if no data is required, an empty vector should be provided, ie serialized as `CompactSize` of 0.

The `featureid` MUST be a globally unique identifier for the feature.

For features published as a BIP, the `featureid` SHOULD be the assigned BIP number, eg "BIP434", or be based on the BIP number (eg, "BIP434v2" where the "v2" suffix covers versioning, or "BIP434.3" where the ".3" suffix covers part 3 of the BIP). For experimental features that do not (yet) have a BIP number assigned, some other unique identifier MUST be chosen, such as a URL to the repository where development is taking place, or the sha256 digest of some longer reference.

Nodes implementing this BIP MUST ignore `feature` messages specifying a `featureid` they do not support, so long as the payload conforms to the requirements above.

Nodes implementing this BIP MAY disconnect peers that send `feature` messages where the `feature` message's payload cannot be correctly parsed (including having missing or additional data), even if they do not recognise the `featureid`.

`feature` message 1-byte identifier

Nodes implementing both this BIP and BIP 324 (v2 P2P encrypted transport) MUST treat a message with a 1-byte `message_type` equal to `37` that is received prior to `verack` as the `feature` message.

Feature negotiation

It is RECOMMENDED that feature negotiation be designed and implemented as follows:

- all `feature` messages and the `verack` message should be sent immediately on receipt of the peer's `version` message
- any negotiation calculations should be performed immediately on receipt of the peer's `verack` message

This structure is fairly easy to implement, and avoids introducing any significant latency that might result from more interactive negotiation methods.

Feature specifications defining a `featureid` MAY make use of the following approaches:

Feature advertisement:

1. Send a `feature` message advertising the `featureid` unconditionally
2. Accept messages related to the feature unconditionally
3. Only send messages defined by the feature if the peer sent a valid `feature` message for the `featureid`.

This approach is appropriate for many simple features that define new messages, particularly where an implementation might only implement sending or receiving a message, but not both, eg BIP 35 (mempool).

Feature coordination:

1. Send a `feature` message advertising the `featureid` unconditionally
2. Check if the peer sends the same `feature` message (or a compatible one), and enable the feature for this peer if so.

3. Only send/accept messages or encode data items according to the feature's specification if the feature is enabled for this peer.

This approach is appropriate for upgrades to data encoding in P2P messages, eg BIP 339 (wtxidrelay) or BIP 155 (addrv2).

Feature versioning:

1. Send `feature` messages for multiple incompatible features, eg `BIP434v3`, `BIP434v2`, `BIP434v1`, ordered from most preferred to least.
2. Track the corresponding `feature` messages from your peer.
3. If you were the listening peer, enable your highest preference feature that your peer also supports.
4. If you were the initiating peer, enable the first feature that your peer announced, that you also support.
5. For example if the listening peer sends `BIP434v3`, `BIP434v2`, `BIP434v1`, and the initiating peer sends `BIP434v1`, `BIP434v2`, then the listening peer should select `BIP434v2` when `verack` is received, and the initiating peer should select `BIP434v2` as soon as `feature BIP434v2` is received.
6. Conversely, if the initiating peer sends `BIP434v3`, `BIP434v2`, `BIP434v1`, and the listening peer sends `BIP434v1`, `BIP434v2`, then the listening peer should select `BIP434v1` when `verack` is received, and the initiating peer should select `BIP434v1` as soon as `feature BIP434v1` is received.
7. In most cases, implementations should simply advertise incompatible features in order from most recent to oldest, on the basis that the only reason to make incompatible updates is because there are significant improvements. Exceptions to that may occur when two incompatible features are both receiving active development, or when an implementation has only partially implemented the latest spec, and the older spec is better supported (and thus should be listed first, as the preferred protocol to adopt).

This approach may be appropriate when making substantial changes to a deployed protocol and backwards compatibility is desirable on a short-term basis, or when there is disagreement amongst implementations or users as to which approach is most desirable.

Considerations

The advantage this approach has over bumping the protocol version number when introducing new P2P messages or data structures, is that no coordination is required (that is, there is no longer a question whether version "n+1" belongs to Alice's new feature, or Bob's new feature), and there is no implication that supporting each new feature means all prior features are also supported.

The advantage this approach has over defining new messages for each feature is that the `featureid` can be much longer (at up to 80 bytes) than a message type id (which are limited to 12 bytes). With a BIP 324 one-byte `message_type`, the overhead compared to that approach is also kept small.

This approach is largely equivalent to adding a payload to the `verack` message (eg, a vector of `featureid`, `featuredata` pairs). It was chosen because:

- it retains compatibility with any implementations that expect `verack` to have no payload;
- it allows peers to process each feature request individually, rather than having to first load the configuration information for all features into memory at once (in order to validate the message's checksum), and then deal with each feature's configuration;
- limiting the maximum message payload size you accept (eg to 4MB) does not limit the number of features you can accept; and

- we have experience with negotiating features with individual messages, but no experience with doing so via `verack` payload.

A mild disadvantage compared to using a `verack` payload is that this approach allows the possibility of interactive feature negotiation prior to `verack`, which would make negotiation more complex and increase latency. However interactive feature negotiation is always possible simply by having the initiating peer disconnect and reconnect after discovering the listening peer's supported features.

This specification attempts to maximise compatibility with implementations that prefer to fully validate each message received:

- `feature` messages, even for unknown features, must always be fully parseable into a `featureid` and `featuredata`
- Ignoring unknown messages prior to `verack` is only a recommendation, not a requirement, so compliant implementations may disconnect on an unknown message that cannot be validated.
- Sending unknown messages after `verack` is explicitly forbidden, in so far as that is possible.

Reference implementation

This feature was implemented in Bitcoin Core PR#35221, which may be used as a reference implementation.

Backward compatibility

Clients specifying a version number prior to `70017` remain fully compatible with this change.

Clients specifying a version number of `70017` or higher that do not implement this BIP remain fully compatible provided they do not disconnect peers upon receiving unexpected messages received between `version` and `verack`.

Acknowledgements

Much of the logic here, and much of the text in the motivation section, is based on Suhas Daftuar's 2020 post on Generalizing feature negotiation.

Copyright

This BIP is licensed under the 2-clause BSD license.

Mining and Block Templates

Miners do not have to reimplement the consensus engine to build a block. They ask a full node for a template: transactions, the previous block hash, the version bits, and the other fields that have to be correct for the block to be valid.

BIP 22 is that interface. BIP 23 extends it for pooled mining. BIP 145 updates the template for SegWit, because a witness-aware block is not just a bag of legacy transactions. If you have ever pointed mining hardware at Bitcoin Core, you have spoken this protocol.

These three documents are the whole chapter on purpose. Stratum and other pool wires exist, but they are not BIPs in this set. The getblocktemplate family is the specification nodes actually ship.

In this chapter:

- BIP 22 — getblocktemplate - Fundamentals
- BIP 23 — getblocktemplate - Pooled Mining
- BIP 145 — getblocktemplate Updates for Segregated Witness

```
BIP: 22
Layer: API/RPC
Title: getblocktemplate - Fundamentals
Authors: Luke Dashjr <luke+bip22@dashjr.org>
Status: Deployed
Type: Specification
Assigned: 2012-02-28
License: BSD-2-Clause
```

BIP 22

Abstract

This BIP describes a new JSON-RPC method for "smart" Bitcoin miners and proxies. Instead of sending a simple block header for hashing, the entire block structure is sent, and left to the miner to (optionally) customize and assemble.

Copyright

This BIP is licensed under the BSD 2-clause license.

Specification

Block Template Request

A JSON-RPC method is defined, called "getblocktemplate". It accepts exactly one argument, which MUST be an Object of request parameters. If the request parameters include a "mode" key, that is used to explicitly select between the default "template" request or a "proposal".

Block template creation can be influenced by various parameters:

template request			
Key	Required	Type	Description
capabilities	No	Array of Strings	SHOULD contain a list of the following, to indicate client-side support: "longpoll", "coinbasetxn", "coinbasevalue", "proposal", "serverlist", "workid", and any of the mutations
mode	No	String	MUST be "template" or omitted

getblocktemplate MUST return a JSON Object containing the following keys:

template			
Key	Required	Type	Description
bits	Yes	String	the compressed difficulty in hexadecimal
curtime	Yes	Number	the current time as seen by the server (recommended for block time) - note this is not necessarily the system clock, and must fall within the mintime/maxtime rules
height	Yes	Number	the height of the block we are looking for
previousblockhash	Yes	String	the hash of the previous block, in big-endian hexadecimal
sigoplimit	No	Number	number of sigops allowed in blocks
sizelimit	No	Number	number of bytes allowed in blocks
transactions	Should	Array of Objects	Objects containing information for Bitcoin transactions (excluding coinbase)
version	Yes	Number	always 1 or 2 (at least for bitcoin) - clients MUST understand the implications of the version they use (eg, comply with BIP 0034 for version 2)
coinbaseaux	No	Object	data that SHOULD be included in the coinbase's scriptSig content. Only the values (hexadecimal byte-for-byte) in this Object should be included, not the keys. This does not include the block height, which is required to be included in the scriptSig by BIP 0034. It is advisable to encode values inside "PUSH" opcodes, so as to not inadvertently expend SIGOPs (which are counted toward limits, despite not being executed).
coinbasetxn	this or ↓	Object	information for coinbase transaction
coinbasevalue	this or ↑	Number	total funds available for the coinbase (in Satoshis)
workid	No	String	if provided, this value must be returned with results (see Block Submission)

Transactions Object Format

The Objects listed in the response's "transactions" key contains these keys:

template "transactions" element		
Key	Type	Description
data	String	transaction data encoded in hexadecimal (byte-for-byte)
depends	Array of Numbers	other transactions before this one (by 1-based index in "transactions" list) that must be present in the final block if this one is; if key is not present, dependencies are unknown and clients MUST NOT assume there aren't any
fee	Number	difference in value between transaction inputs and outputs (in Satoshis); for coinbase transactions, this is a negative Number of the total collected block fees (ie, not including the block subsidy); if key is not present, fee is unknown and clients MUST NOT assume there isn't one

template "transactions" element		
hash	String	hash/id encoded in little-endian hexadecimal
required	Boolean	if provided and true, this transaction must be in the final block
sigops	Number	total number of SigOps, as counted for purposes of block limits; if key is not present, sigop count is unknown and clients MUST NOT assume there aren't any

Only the "data" key is required, but servers should provide the others if they are known.

Block Submission

A JSON-RPC method is defined, called "submitblock", to submit potential blocks (or shares). It accepts two arguments: the first is always a String of the hex-encoded block data to submit; the second is an Object of parameters, and is optional if parameters are not needed.

submitblock parameters (2nd argument)		
Key	Type	Description
workid	String	if the server provided a workid, it MUST be included with submissions

This method MUST return either null (when a share is accepted), a String describing briefly the reason the share was rejected, or an Object of these with a key for each merged-mining chain the share was submitted to.

Optional: Long Polling

template request		
Key	Type	Description
capabilities	Array of Strings	miners which support long polling SHOULD provide a list including the String "longpoll"
longpollid	String	"longpollid" of job to monitor for expiration; required and valid only for long poll requests

template		
Key	Type	Description
longpollid	String	identifier for long poll request; MUST be omitted if the server does not support long polling
longpolluri	String	if provided, an alternate URI to use for long poll requests
submitold	Boolean	only relevant for long poll responses: indicates if work received prior to this response remains potentially valid (default) and should have its shares submitted; if false, the miner may wish to discard its share queue

If the server supports long polling, it MUST include a "longpollid" key in block templates, and it MUST be unique for each event: any given "longpollid" should check for only one condition and not be reused. For example, a server which has a long poll wakeup only for new blocks might use the previous block hash. However, clients should not assume the "longpollid" has any specific meaning. It MAY supply the "longpolluri" key with a relative or absolute URI, which MAY specify a completely different resource than the

original connection, including port number. If “longpolluri” is provided by the server, clients MUST only attempt to use that URI for longpoll requests.

Clients MAY start a longpoll request with a standard JSON-RPC request (in the case of HTTP transport, POST with data) and same authorization, setting the “longpollid” parameter in the request to the value provided by the server.

This request SHOULD NOT be processed nor answered by the server until it wishes to replace the current block data as identified by the “longpollid”. Clients SHOULD make this request with a very long request timeout and MUST accept servers sending a partial response in advance (such as HTTP headers with “chunked” Transfer-Encoding), and only delaying the completion of the final JSON response until processing.

Upon receiving a completed response:

- Only if “submitold” is provided and false, the client MAY discard the results of past operations and MUST begin working on the new work immediately.
- The client SHOULD begin working on the new work received as soon as possible, if not immediately.
- The client SHOULD make a new request to the same long polling URI.

If a client receives an incomplete or invalid response, it SHOULD retry the request with an exponential backoff. Clients MAY implement this backoff with limitations (such as maximum backoff time) or any algorithm as deemed suitable. It is, however, forbidden to simply retry immediately with no delay after more than one failure. In the case of a “Forbidden” response (for example, HTTP 403), a client SHOULD NOT attempt to retry without user intervention.

Optional: Template Tweaking

template request		
Key	Type	Description
sigoplimit	Number or Boolean	maximum number of sigops to include in template
sizelimit	Number or Boolean	maximum number of bytes to use for the entire block
maxversion	Number	highest block version number supported

For “sigoplimit” and “sizelimit”, negative values and zero are offset from the server-determined block maximum. If a Boolean is provided and true, the default limit is used; if false, the server is instructed not to use any limits on returned template. Servers SHOULD respect these desired maximums, but are NOT required to: clients SHOULD check that the returned template satisfies their requirements appropriately.

Appendix: Example Rejection Reasons

Possible reasons a share may be rejected include, but are not limited to:

share rejection reasons	
Reason	Description
bad-cb-flag	the server detected a feature-signifying flag that it does not allow
bad-cb-length	the coinbase was too long (bitcoin limit is 100 bytes)

share rejection reasons	
bad-cb-prefix	the server only allows appending to the coinbase, but it was modified beyond that
bad-diffbits	“bits” were changed
bad-prevblk	the previous-block is not the one the server intends to build on
bad-txnmrklroot	the block header’s merkle root did not match the transaction merkle tree
bad-txns	the server didn’t like something about the transactions in the block
bad-version	the version was wrong
duplicate	the server already processed this block data
high-hash	the block header did not hash to a value lower than the specified target
rejected	a generic rejection without details
stale-prevblk	the previous-block is no longer the one the server intends to build on
stale-work	the work this block was based on is no longer accepted
time-invalid	the time was not acceptable
time-too-new	the time was too far in the future
time-too-old	the time was too far in the past
unknown-user	the user submitting the block was not recognized
unknown-work	the template or workid could not be identified

Motivation

bitcoind’s JSON-RPC server can no longer support the load of generating the work required to productively mine Bitcoin, and external software specializing in work generation has become necessary. At the same time, new independent node implementations are maturing to the point where they will also be able to support miners.

A common standard for communicating block construction details is necessary to ensure compatibility between the full nodes and work generation software.

Rationale

Why not just deal with transactions as hashes (txids)?

- Servers might not have access to the transaction database, or miners may wish to include transactions not broadcast to the network as a whole.
- Miners may opt not to do full transaction verification, and may not have access to the transaction database on their end.

What is the purpose of “workid”?

- If servers allow all mutations, it may be hard to identify which job it is based on. While it may be possible to verify the submission by its content, it is much easier to compare it to the job issued. It is very easy for the miner to keep track of this. Therefore, using a “workid” is a very cheap solution to enable more mutations.

Why should “sigops” be provided for transactions?

- Due to the BIP 0016 changes regarding rules on block sigops, it is impossible to count sigops from the transactions themselves (the sigops in the scriptCheck must also be included in the count).

Reference Implementation

- Eloipool (server)
- libblkmaker (client)
- bitcoind (minimal server)

See Also

- BIP 23: getblocktemplate - Pooled Mining

```
BIP: 23
Layer: API/RPC
Title: getblocktemplate – Pooled Mining
Authors: Luke Dashjr <luke+bip22@dashjr.org>
Status: Deployed
Type: Specification
Assigned: 2012-02-28
License: BSD-2-Clause
```

BIP 23

Abstract

This BIP describes extensions to the getblocktemplate JSON-RPC call to enhance pooled mining.

Copyright

This BIP is licensed under the BSD 2-clause license.

Specification

Note that all sections of this specification are optional extensions on top of BIP 22.

Summary Support Levels

Something can be said to have BIP 23 Level 1 support if it implements at least:

- RFC 1945
- JSON-RPC 1.0
- BIP 22 (non-optional sections)
- BIP 22 Long Polling
- BIP 23 Basic Pool Extensions
- BIP 23 Mutation “coinbase/append”
- BIP 23 Submission Abbreviation “submit/coinbase”
- BIP 23 Mutation “time/increment” (only required for servers)

It can be said to have BIP 23 Level 2 support if it also implements:

- BIP 23 Mutation "transactions/add"
- BIP 23 Block Proposals

Basic Pool Extensions

template request		
Key	Type	Description
target	String	desired target for block template (may be ignored)

template		
Key	Type	Description
expires	Number	how many seconds (beginning from when the server sent the response) this work is valid for, at most
target	String	the number which valid results must be less than, in big-endian hexadecimal

Block Proposal

Servers may indicate support for proposing blocks by including a capability string in their original template:

template		
Key	Type	Description
capabilities	Array of Strings	MAY contain "proposal" to indicate support for block proposal
reject-reason	String	Reason the proposal was invalid as-is (only applicable in response to proposals)

If supported, a miner MAY propose a block to the server for general validation at any point before the job expires. This is accomplished by calling getblocktemplate with two keys:

getblocktemplate parameters		
Key	Type	Description
data	String	MUST be hex-encoded block data
mode	String	MUST be "proposal"
workid	String	if the server provided a workid, it MUST be included with proposals

The block data MUST be validated and checked against the server's usual acceptance rules (excluding the check for a valid proof-of-work). If it is found to be in violation of any of these rules, the server MUST return one of the following:

- Null if it is acceptable as-is, with the same workid (if any) as provided. Note that this SHOULD NOT invalidate the old template's claim to the same workid.
- A String giving the reason for the rejection (see example rejection reasons).

- A “delta” block template (with changes needed); in this case, any missing keys are assumed to default to those in the proposed block or, if not applicable, the original block template it was based on. This template MAY also include a “reject-reason” key with a String of the reason for rejection.

It is RECOMMENDED that servers which merely need to track the proposed block for later share/* submissions, return a simple Object of the form:

```
{"workid":"new workid"}
```

Clients SHOULD assume their proposed block will remain valid if the only changes they make are to the portion of the coinbase scriptSig they themselves provided (if any) and the time header. Servers SHOULD NOT break this assumption without good cause.

Mutations

template request		
Key	Type	Description
nonces	Number	size of nonce range the miner needs; if not provided, the server SHOULD assume the client requires 2^{32}

template		
Key	Type	Description
maxtime	Number	the maximum time allowed
maxtimeoff	Number	the maximum time allowed (as a moving offset from “curtime” - every second, the actual maxtime is incremented by 1; for example, “maxtimeoff”:0 means “time” may be incremented by 1 every second)
mintime	Number	the minimum time allowed
mintimeoff	Number	the minimum time allowed (as a moving offset from “curtime”)
mutable	Array of Strings	different manipulations that the server explicitly allows to be made
noncerange	String	two 32-bit integers, concatenated in big-endian hexadecimal, which represent the valid ranges of nonces the miner may scan

If the block template contains a “mutable” key, it is a list of these to signify modifications the miner is allowed to make:

mutations	
Value	Significance
coinbase/append	append the provided coinbase scriptSig
coinbase	provide their own coinbase; if one is provided, it may be replaced or modified (implied if “coinbasetxn” omitted)
generation	add or remove outputs from the coinbase/generation transaction (implied if “coinbasetxn” omitted)
time/increment	

mutations	
	change the time header to a value after “time” (implied if “maxtime” or “maxtimeoff” are provided)
time/decrement	change the time header to a value before “time” (implied if “mintime” is provided)
time	modify the time header of the block
transactions/add (or “transactions”)	add other valid transactions to the block (implied if “transactions” omitted from result)
prevblock	use the work with other previous-blocks; this implicitly allows removing transactions that are no longer valid (but clients SHOULD attempt to propose removal of any required transactions); it also implies adjusting “height” as necessary
version/force	encode the provide block version, even if the miner doesn’t understand it
version/reduce	use an older block version than the one provided (for example, if the client does not support the version provided)

Submission Abbreviation

template		
Key	Type	Description
fulltarget	String	the number which full results should be less than, in big-endian hexadecimal (see “share/*” mutations)
mutable	Array of Strings	different manipulations that the server explicitly allows to be made, including abbreviations

If the block template contains a “mutable” key, it is a list of these to signify modifications the miner is allowed to make:

abbreviation mutations	
Value	Significance
submit/hash	each transaction being sent in a request, that the client is certain the server knows about, may be replaced by its hash in little-endian hexadecimal, prepended by a “:” character
submit/coinbase	if the “transactions” provided by the server are used as-is with no changes, submissions may omit transactions after the coinbase (transaction count varint remains included with the full number of transactions)
submit/truncate	if the “coinbasetxn” and “transactions” provided by the server are used as-is with no changes, submissions may contain only the block header; if only the scriptSig of “coinbasetxn” is modified, the params Object MUST contain a “coinbasesig” key with the content, or a “coinbaseadd” with appended data (if only appending)
share/coinbase	same as “submit/coinbase”, but only if the block hash is greater than “fulltarget”
share/merkle	if the block hash is greater than “fulltarget”, the non-coinbase transactions may be replaced with a merkle chain connecting it to the root

abbreviation mutations	
share/truncate	same as “submit/truncate”, but only if the block hash is greater than “fulltarget”

Format of Data for Merkle-Only Shares

The format used for submitting shares with the “share/merkle” mutation shall be the 80-byte block header, the total number of transactions encoded in Bitcoin variable length number format, the coinbase transaction, and then finally the little-endian SHA256 hashes of each link in the merkle chain connecting it to the merkle root.

Logical Services

template request		
Key	Type	Description
capabilities	Array of Strings	miners which support this SHOULD provide a list including the String “serverlist”

template		
Key	Type	Description
serverlist	Array of Objects	list of servers in this single logical service

If the “serverlist” parameter is provided, clients MAY choose to intelligently treat the server as part of a larger single logical service.

Each host Object in the Array is comprised of the following fields:

serverlist element		
Key	Type	Description
uri	String	URI of the individual server; if authentication information is omitted, the same authentication used for this request MUST be assumed
avoid	Number	number of seconds to avoid using this server
priority	Number	an integer priority of this host (default: 0)
sticky	Number	number of seconds to stick to this server when used
update	Boolean	whether this server may update the serverlist (default: true)
weight	Number	a relative weight for hosts with the same priority (default: 1)

When choosing which actual server to get the next job from, URIs MUST be tried in order of their “priority” key, lowest Number first. Where the priority of URIs is the same, they should be chosen from in random order, weighed by their “weight” key. Work proposals and submissions MUST be made to the same server that issued the job. Clients MAY attempt to submit to other servers if, and only if, the original server cannot be reached. If cross-server share submissions are desired, services SHOULD instead use the equivalent domain name system (DNS) features (RFCs 1794 and 2782).

Updates to the Logical Service server list may only be made by the original server, or servers listed with the "update" key missing or true. Clients MAY choose to advertise serverlist capability to servers with a false "update" key, but if so, MUST treat the server list provided as a subset of the current one, only considered in the context of this server. At least one server with "update" privilege MUST be attempted at least once daily.

If the "sticky" key is provided, then when that server is used, it should be used consistently for at least that many seconds, if possible.

A permanent change in server URI MAY be indicated with a simple "serverlist" parameter:

```
"serverlist":[{"uri": "http://newserver"}]
```

A temporary delegation to another server for 5 minutes MAY be indicated likewise:

```
"serverlist":[{"uri": "", avoid: 300}, {"uri": "http://newserver", "update": false}]
```

Motivation

There is reasonable concerns about mining currently being too centralized on pools, and the amount of control these pools hold. By exposing the details of the block proposals to the miners, they are enabled to audit and possibly modify the block before hashing it.

To encourage widespread adoption, this BIP should be a complete superset of the existing centralized getwork protocol, so pools are not required to make substantial changes to adopt it.

Rationale

Why allow servers to restrict the complete coinbase and nonce range?

- This is necessary to provide a complete superset of JSON-RPC getwork functionality, so that pools may opt to enable auditing without significantly changing or increasing the complexity of their share validation or mining policies.
- Since noncerange is optional (both for getwork and this BIP), neither clients nor servers are required to support it.

Why specify "time/*" mutations at all?

- In most cases, these are implied by the mintime/mintimecur/maxtime/maxtimecur keys, but there may be cases that there are no applicable minimums/maximums.

What is the purpose of the "prevblock" mutation?

- There are often cases where a miner has processed a new block before the server. If the server allows "prevblock" mutation, the miner may begin mining on the new block immediately, without waiting for a new template.

Why must both "mintime"/"maxtime" and "mintimeoff"/"maxtimeoff" keys be defined?

- In some cases, the limits may be unrelated to the current time (such as the Bitcoin network itself; the minimum is always a fixed median time)

- In other cases, the limits may be bounded by other rules (many pools limit the time header to within 5 minutes of when the share is submitted to them).

Is “target” really needed?

- Some pools work with lower targets, and should not be expected to waste bandwidth ignoring shares that don’t meet it.
- Required to be a proper superset of getwork.
- As mining hashrates grow, some miners may need the ability to request a lower target from their pools to be able to manage their bandwidth use.

What is the purpose of the “hash” transaction list format?

- Non-mining tools may wish to simply get a list of memory pool transactions.
- Humans may wish to view their current memory pool.

Reference Implementation

- libblkmaker
- Eloipool
- bitcoind

See Also

- BIP 22: getblocktemplate - Fundamentals

```
BIP: 145
Layer: API/RPC
Title: getblocktemplate Updates for Segregated Witness
Authors: Luke Dashjr <luke+bip22@dashjr.org>
Status: Deployed
Type: Specification
Assigned: 2016-01-30
License: BSD-2-Clause OR OPUBL-1.0
```

BIP 145

Abstract

This BIP describes modifications to the getblocktemplate JSON-RPC call (BIP 22) to support segregated witness as defined by BIP 141.

Specification

Block Template

The template Object is revised to include a new key:

template			
Key	Required	Type	Description
weightlimit	No	Number	total weight allowed in blocks

The '!' rule prefix MUST be enabled on the "segwit" rule for templates including transactions with witness data. In particular, note that even if the client's "rules" list lacks "segwit", server MAY support old miners by producing a witness-free template and omitting the '!' rule prefix for "segwit" in the template's "rules" list. If the GBT server does not support producing witness-free templates after its activation, it must also use the '!' rule prefix in the "vbavailable" list prior to activation.

Transactions Object Format

The Objects listed in the response's "transactions" key is revised to include these keys:

template "transactions" element		
Key	Type	Description
txid	String	transaction id encoded in hexadecimal; required for transactions with witness data
weight	Number	numeric weight of the transaction, as counted for purposes of the block's weightlimit; if key is not present, weight is unknown and clients MUST NOT assume it is zero, although they MAY choose to calculate it themselves
hash	String	reversed hash of complete transaction (with witness data included) encoded in hexadecimal

Transactions with witness data may only be included if the template's "rules" list (see BIP 9) includes "segwit".

Sigops

For templates with "segwit" enabled as a rule, the "sigoplimit" and "sigops" keys must use the new values as calculated in BIP 141.

Block Assembly with Witness Transactions

When block assembly is done without witness transactions, no changes are made by this BIP, and it should be assembled as previously.

When witness transactions are included in the block, the primary merkle root MUST be calculated with those transactions' "txid" field instead of "hash". A secondary merkle root MUST be calculated as per BIP 141's commitment structure specification to be inserted into the generation (coinbase) transaction.

Servers MUST NOT include a commitment in the "coinbasetxn" key on the template. Clients MUST insert the commitment as an additional output at the end of the final generation (coinbase) transaction. Only if the template includes a "mutable" key (see BIP 23 Mutations) including "generation", the client MAY in that case place the commitment output in any position it chooses, provided that no later output matches the commitment pattern.

Motivation

Segregated witness substantially changes the structure of blocks, so the previous getblocktemplate specification is no longer sufficient. It additionally also adds a new way of counting resource limits, and so GBT must be extended to convey this information correctly as well.

Rationale

Why doesn't "weightlimit" simply redefine the existing "sizelimit"?

- "sizelimit" is already enforced by clients by counting the sum of bytes in transactions' "data" keys.
- Servers may wish to limit the overall size of a block, independently from the "weight" of the block.

Why is "sigoplimit" redefined instead of a new "sigopweightlimit" being added?

- The old limit was already arbitrarily defined, and could not be counted by clients on their own anyway. The concept of "sigop weight" is merely a change in the arbitrary formula used.

Why is "sigoplimit" divided by 4?

- To resemble the previous values. (FIXME: is this a good reason? maybe we shouldn't divide it?)

Why is the witness commitment required to be added to the end of the generation transaction rather than anywhere else?

- Servers which do not allow modification of the generation outputs ought to be checking this as part of the validity of submissions. By requiring a specific placement, they can simply strip the commitment and do a byte-for-byte comparison of the outputs. Placing it at the end avoids the possibility of a later output matching the pattern and overriding it.

Why shouldn't the server simply add the commitment upfront in the "coinbasetxn", and simply send the client stripped transaction data?

- It would become impossible for servers to specify only "coinbasevalue", since clients would no longer have the information required to construct the commitment.
- getblocktemplate is intended to be a *decentralised* mining protocol, and allowing clients to be blinded to the content of the block works contrary to that purpose.
- BIP 23's "transactions" mutations allow the client to modify the transaction-set on their own, which is impossible without the complete transaction data.

Reference Implementation

- libblkmaker
- Eloipool
- Bitcoin Core

See Also

- BIP 9: Version bits with timeout and delay
- BIP 22: getblocktemplate - Fundamentals

- BIP 23: getblocktemplate - Pooled Mining
- BIP 141: Segregated Witness (Consensus layer)

Copyright

This BIP is dual-licensed under the Open Publication License and BSD 2-clause license.

Paying People

An address is enough to send coins, but it is a blunt instrument. This chapter is the rest of the payment stack: how a merchant once asked for a signed payment request, how two wallets can cooperate on a payjoin, how a recipient can publish a silent-payment scan key instead of a fresh address, and how DNS can carry those instructions under a human-readable name.

BIP 70 through 75 are the old payment protocol — protobuf invoices, MIME types, URI extensions, and encrypted out-of-band exchange. They shipped, they are still in the archive, and most wallets have moved on. I kept them because they are complete specifications and because later payment designs are easier to understand as reactions to them.

The newer documents are the ones wallets are implementing now. BIP 78 is payjoin. BIP 47 is reusable payment codes: one published code, many on-chain addresses. BIP 352 is silent payments, a later answer to the same problem. BIP 353 puts payment instructions in DNS. BIP 327 is MuSig2, which is how several signers produce one Schnorr signature. BIP 137 and BIP 322 are how you prove you hold a key without moving coins — the first for the legacy message format, the second for a generic signed message that works with modern scripts.

In this chapter:

- BIP 70 — Payment Protocol
- BIP 71 — Payment Protocol MIME types
- BIP 72 — bitcoin: uri extensions for Payment Protocol
- BIP 73 — Accept header negotiation for Payment Request URLs
- BIP 75 — Out of Band Address Exchange
- BIP 78 — A Simple Payjoin Proposal
- BIP 47 — Reusable Payment Codes
- BIP 352 — Silent Payments
- BIP 353 — DNS Payment Instructions
- BIP 327 — MuSig2 for BIP340-compatible Multi-Signatures
- BIP 137 — Signatures of Messages using Private Keys
- BIP 322 — Generic Signed Message Format

```
BIP: 70
Layer: Applications
Title: Payment Protocol
Authors: Gavin Andresen <gavinandresen@gmail.com>
         Mike Hearn <mhearn@bitcoinfoundation.org>
Status: Deployed
Type: Specification
Assigned: 2013-07-29
```

BIP 70

Abstract

This BIP describes a protocol for communication between a merchant and their customer, enabling both a better customer experience and better security against man-in-the-middle attacks on the payment process.

Motivation

The current, minimal Bitcoin payment protocol operates as follows:

1. Customer adds items to an online shopping basket, and decides to pay using Bitcoin.
2. Merchant generates a unique payment address, associates it with the customer's order, and asks the customer to pay.
3. Customer copies the Bitcoin address from the merchant's web page and pastes it into whatever wallet they are using OR follows a bitcoin: link and their wallet is launched with the amount to be paid.
4. Customer authorizes payment to the merchant's address and broadcasts the transaction through the Bitcoin p2p network.
5. Merchant's server detects payment and after sufficient transaction confirmations considers the transaction final.

This BIP extends the above protocol to support several new features:

1. Human-readable, secure payment destinations– customers will be asked to authorize payment to "example.com" instead of an inscrutable, 34-character bitcoin address.
2. Secure proof of payment, which the customer can use in case of a dispute with the merchant.
3. Resistance from man-in-the-middle attacks that replace a merchant's bitcoin address with an attacker's address before a transaction is authorized with a hardware wallet.
4. Payment received messages, so the customer knows immediately that the merchant has received, and has processed (or is processing) their payment.
5. Refund addresses, automatically given to the merchant by the customer's wallet software, so merchants do not have to contact customers before refunding overpayments or orders that cannot be fulfilled for some reason.

Protocol

This BIP describes payment protocol messages encoded using Google's Protocol Buffers, authenticated using X.509 certificates, and communicated over http/https. Future BIPs might extend this payment protocol to other encodings, PKI systems, or transport protocols.

The payment protocol consists of three messages; PaymentRequest, Payment, and PaymentACK, and begins with the customer somehow indicating that they are ready to pay and the merchant's server responding with a PaymentRequest message:



Messages

The Protocol Buffers messages are defined in paymentrequest.proto.

Output

Outputs are used in PaymentRequest messages to specify where a payment (or part of a payment) should be sent. They are also used in Payment messages to specify where a refund should be sent.

```
message Output {
optional uint64 amount = 1 [default = 0];
    optional bytes script = 2;
}
```

amount	Number of satoshis (0.00000001 BTC) to be paid
script	a "TxOut" script where payment should be sent. This will normally be one of the standard Bitcoin transaction scripts (e.g. pubkey OP_CHECKSIG). This is optional to enable future extensions to this protocol that derive Outputs from a master public key and the PaymentRequest data itself.

PaymentDetails/PaymentRequest

Payment requests are split into two messages to support future extensibility. The bulk of the information is contained in the PaymentDetails message. It is wrapped inside a PaymentRequest message, which contains meta-information about the merchant and a digital signature.

```
message PaymentDetails {
    optional string network = 1 [default = "main"];
    repeated Output outputs = 2;
    required uint64 time = 3;
    optional uint64 expires = 4;
    optional string memo = 5;
    optional string payment_url = 6;
    optional bytes merchant_data = 7;
}
```

network	either "main" for payments on the production Bitcoin network, or "test" for payments on test network. If a client receives a PaymentRequest for a network it does not support it must reject the request.
outputs	one or more outputs where Bitcoins are to be sent. If the sum of outputs.amount is zero, the customer will be asked how much to pay, and the bitcoin client may choose any or all of the Outputs (if there are more than one) for payment. If the sum of outputs.amount is non-zero, then the customer will be asked to pay the sum, and the payment shall be split among the Outputs with non-zero amounts (if there are more than one; Outputs with zero amounts shall be ignored).
time	

	Unix timestamp (seconds since 1-Jan-1970 UTC) when the PaymentRequest was created.
expires	Unix timestamp (UTC) after which the PaymentRequest should be considered invalid.
memo	UTF-8 encoded, plain-text (no formatting) note that should be displayed to the customer, explaining what this PaymentRequest is for.
payment_url	Secure (usually https) location where a Payment message (see below) may be sent to obtain a PaymentACK.
merchant_data	Arbitrary data that may be used by the merchant to identify the PaymentRequest. May be omitted if the merchant does not need to associate Payments with PaymentRequest or if they associate each PaymentRequest with a separate payment address.

The payment_url specified in the PaymentDetails should remain valid at least until the PaymentDetails expires (or as long as possible if the PaymentDetails does not expire). Note that this is irrespective of any state change in the underlying payment request; for example cancellation of an order should not invalidate the payment_url, as it is important that the merchant's server can record mis-payments in order to refund the payment.

A PaymentRequest is PaymentDetails optionally tied to a merchant's identity:

```
message PaymentRequest {
    optional uint32 payment_details_version = 1 [default = 1];
    optional string pki_type = 2 [default = "none"];
    optional bytes pki_data = 3;
    required bytes serialized_payment_details = 4;
    optional bytes signature = 5;
}
```

payment_details_version	See below for a discussion of versioning/upgrading.
pki_type	public-key infrastructure (PKI) system being used to identify the merchant. All implementation should support "none", "x509+sha256" and "x509+sha1".
pki_data	PKI-system data that identifies the merchant and can be used to create a digital signature. In the case of X.509 certificates, pki_data contains one or more X.509 certificates (see Certificates section below).
serialized_payment_details	A protocol-buffer serialized PaymentDetails message.
signature	digital signature over a hash of the protocol buffer serialized variation of the PaymentRequest message, with all serialized fields serialized in numerical order (all current protocol buffer implementations serialize fields in numerical order) and signed using the

	private key that corresponds to the public key in pki_data. Optional fields that are not set are not serialized (however, setting a field to its default value will cause it to be serialized and will affect the signature). Before serialization, the signature field must be set to an empty value so that the field is included in the signed PaymentRequest hash but contains no data.

When a Bitcoin wallet application receives a PaymentRequest, it must authorize payment by doing the following:

1. Validate the merchant's identity and signature using the PKI system, if the pki_type is not "none".
2. Validate that customer's system unix time (UTC) is before PaymentDetails.expires. If it is not, then the payment request must be rejected.
3. Display the merchant's identity and ask the customer if they would like to submit payment (e.g. display the "Common Name" in the first X.509 certificate).

PaymentRequest messages larger than 50,000 bytes should be rejected by the wallet application, to mitigate denial-of-service attacks.

Payment

Payment messages are sent after the customer has authorized payment:

```
message Payment {
    optional bytes merchant_data = 1;
    repeated bytes transactions = 2;
    repeated Output refund_to = 3;
    optional string memo = 4;
}
```

merchant_data	copied from PaymentDetails.merchant_data. Merchants may use invoice numbers or any other data they require to match Payments to PaymentRequests. Note that malicious clients may modify the merchant_data, so should be authenticated in some way (for example, signed with a merchant-only key).
transactions	One or more valid, signed Bitcoin transactions that fully pay the PaymentRequest
refund_to	One or more outputs where the merchant may return funds, if necessary. The merchant may return funds using these outputs for up to 2 months after the time of the payment request. After that time has expired, parties must negotiate if returning of funds becomes necessary.
memo	

	UTF-8 encoded, plain-text note from the customer to the merchant.

If the customer authorizes payment, then the Bitcoin client:

1. Creates and signs one or more transactions that satisfy (pay in full) PaymentDetails.outputs
2. Validate that customer's system unix time (UTC) is still before PaymentDetails.expires. If it is not, the payment should be cancelled.
3. Broadcast the transactions on the Bitcoin p2p network.
4. If PaymentDetails.payment_url is specified, POST a Payment message to that URL. The Payment message is serialized and sent as the body of the POST request.

Errors communicating with the payment_url server should be communicated to the user. In the scenario where the merchant's server receives multiple identical Payment messages for an individual PaymentRequest, it must acknowledge each. The second and further PaymentACK messages sent from the merchant's server may vary by memo field to indicate current state of the Payment (for example number of confirmations seen on the network). This is required in order to ensure that in case of a transport level failure during transmission, recovery is possible by the Bitcoin client re-sending the Payment message.

PaymentDetails.payment_url should be secure against man-in-the-middle attacks that might alter Payment.refund_to (if using HTTP, it must be TLS-protected).

Wallet software sending Payment messages via HTTP must set appropriate Content-Type and Accept headers, as specified in BIP 71:

```
Content-Type: application/bitcoin-payment
Accept: application/bitcoin-paymentack
```

When the merchant's server receives the Payment message, it must determine whether or not the transactions satisfy conditions of payment. If and only if they do, it should broadcast the transaction(s) on the Bitcoin p2p network.

Payment messages larger than 50,000 bytes should be rejected by the merchant's server, to mitigate denial-of-service attacks.

PaymentACK

PaymentACK is the final message in the payment protocol; it is sent from the merchant's server to the bitcoin wallet in response to a Payment message:

```
message PaymentACK {
    required Payment payment = 1;
    optional string memo = 2;
}
```

payment	Copy of the Payment message that triggered this PaymentACK. Clients may ignore this if they implement another way of associating Payments with PaymentACKs.
memo	

	UTF-8 encoded note that should be displayed to the customer giving the status of the transaction (e.g. “Payment of 1 BTC for eleven tribbles accepted for processing.”)

PaymentACK messages larger than 60,000 bytes should be rejected by the wallet application, to mitigate denial-of-service attacks. This is larger than the limits on Payment and PaymentRequest messages as PaymentACK contains a full Payment message within it.

Localization

Merchants that support multiple languages should generate language-specific PaymentRequests, and either associate the language with the request or embed a language tag in the request’s merchant_data. They should also generate a language-specific PaymentACK based on the original request.

For example: A greek-speaking customer browsing the Greek version of a merchant’s website clicks on a “Αγορά τώρα” link, which generates a PaymentRequest with merchant_data set to “lang=el&basketId=11252”. The customer pays, their bitcoin client sends a Payment message, and the merchant’s website responds with PaymentACK.message “σας ευχαριστούμε”.

Certificates

The default PKI system is X.509 certificates (the same system used to authenticate web servers). The format of pki_data when pki_type is “x509+sha256” or “x509+sha1” is a protocol-buffer-encoded certificate chain:

```
message X509Certificates {
    repeated bytes certificate = 1;
}
```

If pki_type is “x509+sha256”, then the PaymentRequest message is hashed using the SHA256 algorithm to produce the message digest that is signed. If pki_type is “x509+sha1”, then the SHA1 algorithm is used.

Each certificate is a DER [ITU.X690.1994] PKIX certificate value. The certificate containing the public key of the entity that digitally signed the PaymentRequest must be the first certificate. This MUST be followed by additional certificates, with each subsequent certificate being the one used to certify the previous one, up to (but not including) a trusted root authority. The trusted root authority MAY be included. The recipient must verify the certificate chain according to [RFC5280] and reject the PaymentRequest if any validation failure occurs.

Trusted root certificates may be obtained from the operating system; if validation is done on a device without an operating system, the Mozilla root store is recommended.

Extensibility

The protocol buffers serialization format is designed to be extensible. In particular, new, optional fields can be added to a message and will be ignored (but saved/re-transmitted) by old implementations.

PaymentDetails messages may be extended with new optional fields and still be considered “version 1.” Old implementations will be able to validate signatures against PaymentRequests containing the new fields, but (obviously) will not be able to display whatever information is contained in the new, optional fields to the user.

If it becomes necessary at some point in the future for merchants to produce PaymentRequest messages that are accepted *only* by new implementations, they can do so by defining a new PaymentDetails message with version=2. Old implementations should let the user know that they need to upgrade their software when they get an up-version PaymentDetails message.

Implementations that need to extend messages in this specification shall use tags starting at 1000, and shall update the extensions page via pull-req to avoid conflicts with other extensions.

References

BIP 0071 : Payment Protocol mime types

BIP 0072 : Payment Protocol bitcoin: URI extensions

Public-Key Infrastructure (X.509) working group : http://datatracker.ietf.org/wg/pkix/charter/

Protocol Buffers : https://developers.google.com/protocol-buffers/

Reference implementation

Create Payment Request generator : https://developer.bitcoin.org/examples/payment_processing.html (source)

BitcoinJ : https://bitcoinj.github.io/payment-protocol#introduction

See Also

Javascript Object Signing and Encryption working group : http://datatracker.ietf.org/wg/jose/

Wikipedia's page on Invoices: http://en.wikipedia.org/wiki/Invoice especially the list of Electronic Invoice standards

sipa's payment protocol proposal: https://gist.github.com/sipa/1237788

ThomasV's "Signed Aliases" proposal : http://ecdsa.org/bitcoin_URIs.html

Homomorphic Payment Addresses and the Pay-to-Contract Protocol : http://arxiv.org/abs/1212.3257

```
BIP: 71
Layer: Applications
Title: Payment Protocol MIME types
Authors: Gavin Andresen <gavinandresen@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2013-07-29
```

BIP 71

Abstract

This BIP defines a MIME (RFC 2046) Media Type for Bitcoin payment request messages.

Motivation

Wallet or server software that sends payment protocol messages over email or http should follow Internet standards for properly encapsulating the messages.

Specification

The Media Type (Content-Type in HTML/email headers) for bitcoin protocol messages shall be:

Message	Type/Subtype
PaymentRequest	application/bitcoin-paymentrequest
Payment	application/bitcoin-payment
PaymentACK	application/bitcoin-paymentack

Payment protocol messages are encoded in binary.

Example

A web server generating a PaymentRequest message to initiate the payment protocol would precede the binary message data with the following headers:

```
Content-Type: application/bitcoin-paymentrequest
Content-Transfer-Encoding: binary
```

```
  BIP: 72
  Layer: Applications
  Title: bitcoin: uri extensions for Payment Protocol
  Authors: Gavin Andresen <gavinandresen@gmail.com>
  Status: Deployed
  Type: Specification
  Assigned: 2013-07-29
```

BIP 72

Abstract

This BIP describes an extension to the bitcoin: URI scheme (BIP 21) to support the payment protocol (BIP 70).

Motivation

Allow users to click on a link in a web page or email to initiate the payment protocol, while being backwards-compatible with existing bitcoin wallets.

Specification

The bitcoin: URI scheme is extended with an additional, optional “r” parameter, whose value is a URL from which a PaymentRequest message should be fetched (characters not allowed within the scope of a query parameter must be percent-encoded as described in RFC 3986 and bip-0021).

If the “r” parameter is provided and backwards compatibility is not required, then the bitcoin address portion of the URI may be omitted (the URI will be of the form: bitcoin:?r=…).

When Bitcoin wallet software that supports this BIP receives a bitcoin: URI with a request parameter, it should ignore the bitcoin address/amount/label/message in the URI and instead fetch a PaymentRequest message and then follow the payment protocol, as described in BIP 70.

Bitcoin wallets must support fetching PaymentRequests via http and https protocols; they may support other protocols. Wallets must include an “Accept” HTTP header in HTTP(s) requests (as defined in RFC 2616):

```
Accept: application/bitcoin-paymentrequest
```

If a PaymentRequest cannot be obtained (perhaps the server is unavailable), then the customer should be informed that the merchant’s payment processing system is unavailable. In the case of an HTTP request, status codes which are neither success nor error (such as redirect) should be handled as outlined in RFC 2616.

Compatibility

Wallet software that does not support this BIP will simply ignore the r parameter and will initiate a payment to bitcoin address.

Examples

A backwards-compatible request:

```
bitcoin:mq7se9wy2egettFxPbmn99cK8v5AFq55Lx?amount=0.11&r=https://merchant.com/pay.php?
h%3D2a8628fc2fbe
```

Non-backwards-compatible equivalent:

```
bitcoin:?r=https://merchant.com/pay.php?h%3D2a8628fc2fbe
```

References

RFC 2616 : Hypertext Transfer Protocol – HTTP/1.1

```
  BIP: 73
  Layer: Applications
  Title: Use "Accept" header for response type negotiation with Payment Request URLs
  Authors: Stephen Pair <stephen@bitpay.com>
  Status: Deployed
  Type: Specification
  Assigned: 2013-08-27
```

BIP 73

Abstract

This BIP describes an enhancement to the payment protocol (BIP 70) that addresses the need for short URLs when scanning from QR codes. It generalizes the specification for the behavior of a payment request URL in a way that allows the client and server to negotiate the content of the response using the HTTP Accept: header

field. Specifically, the client can indicate to the server whether it prefers to receive a bitcoin URI or a payment request.

Implementation of this BIP does not require full payment request (BIP 70) support.

Motivation

The payment protocol augments the bitcoin: uri scheme with an additional "payment" parameter that specifies a URL where a payment request can be downloaded. This creates long URIs that, when rendered as a QR code, have a high information density. Dense QR codes can be difficult to scan resulting in a more frustrating user experience. The goal is to create a standard that would allow QR scanning wallets to use less dense QR codes. It also makes general purpose QR code scanners more usable with bitcoin accepting websites.

Specification

QR scanning wallets will consider a non bitcoin URI scanned from a QR code to be an end point where either a bitcoin URI or a payment request can be obtained.

A wallet client uses the Accept: HTTP header to specify whether it can accept a payment request, a URI, or both. A media type of text/uri-list specifies that the client accepts a bitcoin URI. A media type of application/bitcoin-paymentrequest specifies that the client can process a payment request. In the absence of an Accept: header, the server is expected to respond with text/html suitable for rendering in a browser. An HTML response will ensure that QR codes scanned by non Bitcoin wallet QR scanners are useful (they could render an HTML page with a payment link that when clicked would open a wallet on the device).

It is not required that the client and server support the full semantics of an HTTP Accept header. If application/bitcoin-paymentrequest is specified in the header, the server should send a payment request regardless of anything else specified in the Accept header. If text/uri-list is specified (but not application/bitcoin-paymentrequest), a valid Bitcoin URI should be returned. If neither is specified, the server can return an HTML page. When a uri-list is returned only the first item in the list is used (and expected to be a bitcoin URI), any additional URIs should be ignored.

Compatibility

Only QR scanning wallets that implement this BIP will be able to process QR codes containing payment request URLs. There are two possible workarounds for QR scanning wallets that do not implement this BIP: 1) the server gives the user an option to change the QR code to a bitcoin: URI or 2) the user scans the code with a generic QR code scanner.

In the second scenario, if the server responds with a webpage containing a link to a bitcoin URI, the user can complete the payment by clicking that link provided the user has a wallet installed on their device and it supports bitcoin URIs. If the wallet/device does not have support for bitcoin URIs, the user can fall back on address copy/paste.

This BIP should be fully compatible with BIP 70 assuming it is required that wallets implementing BIP 70 make use of the Accept: HTTP header when retrieving a payment request.

Examples

The first image below is of a bitcoin URI with an amount and payment request specified (note, this is a fairly minimal URI as it does not contain a label and the request URL is of moderate size). The second image is a QR code with only the payment request url specified.

<img src=bip-0073/a.png></img><img src=bip-0073/b.png></img>

```
BIP: 75
Layer: Applications
Title: Out of Band Address Exchange using Payment Protocol Encryption
Authors: Justin Newton <justin@netki.com>
         Matt David <mgd@mgddev.com>
         Aaron Voisine <voisine@gmail.com>
         James MacWhyte <macwhyte@gmail.com>
Comments-Summary: Recommended for implementation (one person)
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0075
Status: Deployed
Type: Specification
Assigned: 2015-11-20
License: CC-BY-4.0
```

BIP 75

Abstract

This BIP is an extension to BIP 70 that provides two enhancements to the existing Payment Protocol.

1. It allows the requester (Sender) of a PaymentRequest to voluntarily sign the original request and provide a certificate to allow the payee to know the identity of who they are transacting with.

1. It encrypts the PaymentRequest that is returned, before handing it off to the SSL/TLS layer to prevent man in the middle viewing of the Payment Request details.

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

Copyright



https://licensebuttons.net/l/by/4.0/88x31.png

This work is licensed under a Creative Commons Attribution 4.0 International License.

Definitions

Sender	Entity wishing to transfer value that they control
Receiver	Entity receiving a value transfer

Motivation

The motivation for defining this extension to the BIP70 Payment Protocol is to allow two parties to exchange payment information in a permissioned and encrypted way, such that wallet address communication can become a more automated process. This extension also expands the types of PKI (public-key infrastructure) data that is supported, and allows it to be shared by both parties (with BIP70, only the receiver could provide PKI information). This allows for automated creation of off-blockchain transaction logs that are human readable, now including information about the sender and not just the recipient.

The motivation for this extension to BIP70 is threefold:

1. Ensure that the payment details can only be seen by the participants in the transaction, and not by any third party.

1. Enhance the Payment Protocol to allow for store and forward servers in order to allow, for example, mobile wallets to sign and serve Payment Requests.

1. Allow a sender of funds the option of sharing their identity with the receiver. This information could then be used to:

#* Make Bitcoin logs (wallet transaction history) more human readable

#* Give the user the ability to decide whether or not they share their Bitcoin address and other payment details when requested

#* Allow for an open standards based way for businesses to keep verifiable records of their financial transactions, to better meet the needs of accounting practices or other reporting and statutory requirements

#* Automate the active exchange of payment addresses, so static addresses and BIP32 X-Pubs can be avoided to maintain privacy and convenience

In short we wanted to make Bitcoin more human, while at the same time improving transaction privacy.

Example Use Cases

1. Address Book

A Bitcoin wallet developer would like to offer the ability to store an "address book" of payees, so users could send multiple payments to known entities without having to request an address every time. Static addresses compromise privacy, and address reuse is considered a security risk. BIP32 X-Pubs allow the generation of unique addresses, but watching an X-Pub chain for each person you wish to receive funds from is too resource-intensive for mobile applications, and there is always a risk of unknowingly sending funds to an X-Pub address after the owner has lost access to the corresponding private key.

With this BIP, Bitcoin wallets could maintain an "address book" that only needs to store each payee's public key. Adding an entry to one's address book could be done by using a Wallet Name, scanning a QR code, sending a URI through a text message or e-mail, or searching a public repository. When the user wishes to make a payment, their wallet would do all the work in the background to communicate with the payee's wallet to receive a unique payment address. If the payee's wallet has been lost, replaced, or destroyed, no communication will be possible, and the sending of funds to a "dead" address is prevented.

2. Individual Permissioned Address Release

A Bitcoin wallet developer would like to allow users to view a potential sending party's identifying information before deciding whether or not to share payment information with them. Currently, BIP70 shares the receiver's payment address and identity information with anyone who requests it.

With this BIP, Bitcoin wallets could use the sender's identifying information to make a determination of whether or not to share their own information. This gives the receiving party more control over who receives their payment and identity information. Additionally, this could be used to automatically provide new payment addresses to whitelisted senders, or to protect users' privacy from unsolicited payment requests.

3. Using Store & Forward Servers

A Bitcoin wallet developer would like to use a public Store & Forward service for an asynchronous address exchange. This is a common case for mobile and offline wallets.

With this BIP, returned payment information is encrypted with an ECDH-computed shared key before sending to a Store & Forward service. In this case, a successful attack against a Store & Forward service would not be able to read or modify wallet address or payment information, only delete encrypted messages.

Modifying BIP70 pki_type

This BIP adds additional possible values for the pki_type variable in the PaymentRequest message. The complete list is now as follows:

pki_type	**Description**
x509+sha256	A x.509 certificate, as described in BIP70
pgp+sha256	An OpenPGP certificate
ecdsa+sha256	A secp256k1 ECDSA public key

NOTE: Although SHA1 was supported in BIP70, it has been deprecated and BIP75 only supports SHA256. The hashing algorithm is still specified in the values listed above for forward and backwards compatibility.

New Messages

Updated [/bip-0075/paymentrequest.proto paymentrequest.proto] contains the existing PaymentRequest Protocol Buffer messages as well as the messages newly defined in this BIP.

NOTE: Public keys from both parties must be known to each other in order to facilitate encrypted communication. Although including both public keys in every message may get redundant, it provides the most flexibility as each message is completely self-contained.

InvoiceRequest

The **InvoiceRequest** message allows a Sender to send information to the Receiver such that the Receiver can create and return a PaymentRequest.

```
message InvoiceRequest {
        required bytes  sender_public_key = 1;
```

```
        optional uint64 amount = 2 [default = 0];
        optional string pki_type = 3 [default = "none"];
        optional bytes  pki_data = 4;
        optional string memo = 5;
        optional string notification_url = 6;
        optional bytes  signature = 7;
}
```

Field Name	Description
sender_public_key	Sender's SEC-encoded EC public key
amount	amount is integer-number-of-satoshis (default: 0)
pki_type	none / x509+sha256 / pgp+sha256 / ecdsa+sha256 (default: "none")
pki_data	Depends on pki_type
memo	Human-readable description of invoice request for the receiver
notification_url	Secure (usually TLS-protected HTTP) location where an EncryptedProtocolMessage SHOULD be sent when ready
signature	PKI-dependent signature

ProtocolMessageType Enum

This enum is used in the newly defined ProtocolMessage and EncryptedProtocolMessage messages to define the serialized message type. The **ProtocolMessageType** enum is defined in an extensible way to allow for new message type additions to the Payment Protocol.

```
enum ProtocolMessageType {
    UNKNOWN_MESSAGE_TYPE = 0;
    INVOICE_REQUEST = 1;
    PAYMENT_REQUEST = 2;
    PAYMENT = 3;
    PAYMENT_ACK = 4;
}
```

ProtocolMessage

The **ProtocolMessage** message is an encapsulating wrapper for any Payment Protocol message. It allows two-way, non-encrypted communication of Payment Protocol messages. The message also includes a status code and a status message that is used for error communication such that the protocol does not rely on transport-layer error handling.

```
message ProtocolMessage {
    required uint64 version = 1
    required uint64 status_code = 2;
    required ProtocolMessageType message_type = 3;
    required bytes serialized_message = 4;
    optional string status_message = 5;
```

```
    required bytes identifier = 6;
}
```

Field Name	Description
version	Protocol version number (Currently 1)
status_code	Payment Protocol Status Code
message_type	Message Type of serialized_message
serialized_message	Serialized Payment Protocol Message
status_message	Human-readable Payment Protocol status message
identifier	Unique key to identify this entire exchange on the server. Default value SHOULD be SHA256(Serialized Initial InvoiceRequest + Current Epoch Time in Seconds as a String)

Versioning

This BIP introduces version 1 of this protocol. All messages sent using these base requirements MUST use a value of 1 for the version number. Any future BIPs that modify this protocol (encryption schemes, etc) MUST each increment the version number by 1.

When initiating communication, the version field of the first message SHOULD be set to the highest version number the sender understands. All clients MUST be able to understand all version numbers less than the highest number they support. If a client receives a message with a version number higher than they understand, they MUST send the message back to the sender with a status code of 101 (“version too high”) and the version field set to the highest version number the recipient understands. The sender must then resend the original message using the same version number returned by the recipient or abort.

EncryptedProtocolMessage

The **EncryptedProtocolMessage** message is an encapsulating wrapper for any Payment Protocol message. It allows two-way, authenticated and encrypted communication of Payment Protocol messages in order to keep their contents secret. The message also includes a status code and status message that is used for error communication such that the protocol does not rely on transport-layer error handling.

```
message EncryptedProtocolMessage {
    required uint64 version = 1 [default = 1];
    required uint64 status_code = 2 [default = 1];
    required ProtocolMessageType message_type = 3;
    required bytes encrypted_message = 4;
    required bytes receiver_public_key = 5;
    required bytes sender_public_key = 6;
    required uint64 nonce = 7;
    required bytes identifier = 8;
    optional string status_message = 9;
    optional bytes signature = 10;
}
```

Field Name	Description
version	Protocol version number
status_code	Payment Protocol Status Code
message_type	Message Type of Decrypted encrypted_message
encrypted_message	AES-256-GCM Encrypted (as defined in BIP75) Payment Protocol Message
receiver_public_key	Receiver's SEC-encoded EC Public Key
sender_public_key	Sender's SEC-encoded EC Public Key
nonce	Microseconds since epoch
identifier	Unique key to identify this entire exchange on the server. Default value SHOULD be SHA256(Serialized Initial InvoiceRequest + Current Epoch Time in Seconds as a String)
status_message	Human-readable Payment Protocol status message
signature	DER-encoded Signature over the full EncryptedProtocolMessage with EC Key Belonging to Sender / Receiver, respectively

Payment Protocol Process with InvoiceRequests

The full process overview for using **InvoiceRequests** in the Payment Protocol is defined below.

All Payment Protocol messages MUST be encapsulated in either a ProtocolMessage or EncryptedProtocolMessage. Once the process begins using EncryptedProtocolMessage messages, all subsequent communications MUST use EncryptedProtocolMessages.

All Payment Protocol messages SHOULD be communicated using EncryptedProtocolMessage encapsulating messages with the exception that an InvoiceRequest MAY be communicated using the ProtocolMessage if the receiver's public key is unknown.

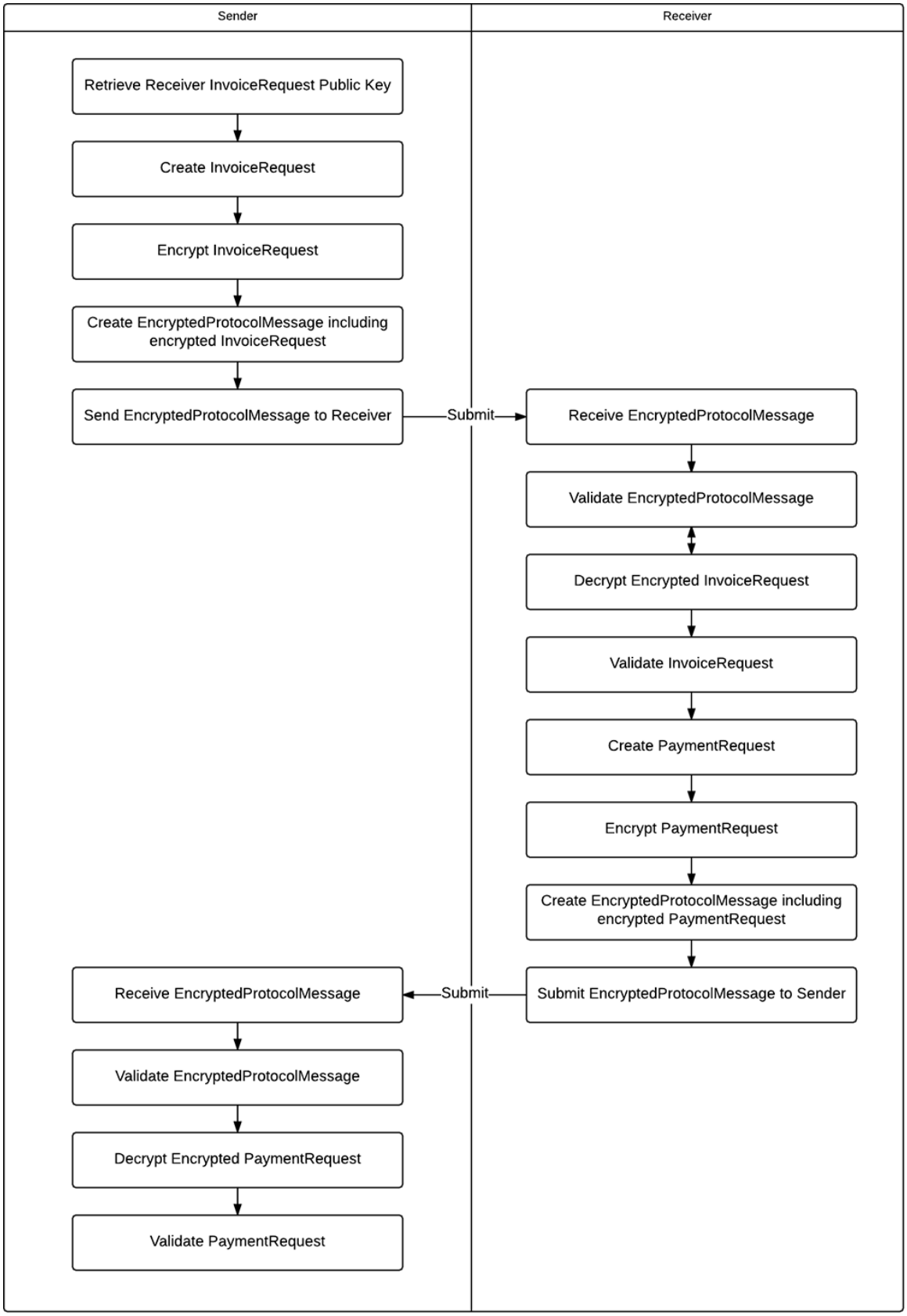
The process of creating encrypted Payment Protocol messages is enumerated in Sending Encrypted Payment Protocol Messages using EncryptedProtocolMessages, and the process of decrypting encrypted messages can be found under Validating and Decrypting Payment Protocol Messages using EncryptedProtocolMessages.

A standard exchange from start to finish would look like the following:

1. Sender creates InvoiceRequest
2. Sender encapsulates InvoiceRequest in (Encrypted)ProtocolMessage
3. Sender sends (Encrypted)ProtocolMessage to Receiver
4. Receiver retrieves InvoiceRequest in (Encrypted)ProtocolMessage from Sender
5. Receiver creates PaymentRequest
6. Receiver encapsulates PaymentRequest in EncryptedProtocolMessage
7. Receiver transmits EncryptedProtocolMessage to Sender
8. Sender validates PaymentRequest retrieved from the EncryptedProtocolMessage

9. The PaymentRequest is processed according to BIP70, including optional Payment and PaymentACK messages encapsulated in EncryptedProtocolMessage messages.

NOTE: See Initial Public Key Retrieval for InvoiceRequest Encryption for possible options to retrieve Receiver's public key.



bip-0075/encrypted-invoice-request-process.png

Message Interaction Details

HTTP Content Types for New Message Types

When communicated via **HTTP**, the listed messages MUST be transmitted via TLS-protected HTTP using the appropriate Content-Type header as defined here per message:
{| class="wikitable" ! Message Type !! Content Type |- | ProtocolMessage || application/bitcoin-paymentprotocol-message |- | EncryptedProtocolMessage || application/bitcoin-encrypted-paymentprotocol-message |}

Payment Protocol Status Communication

Every ProtocolMessage or EncryptedProtocolMessage MUST include a status code which conveys information about the last message received, if any (for the first message sent, use a status of 1 "OK" even though there was no previous message). In the case of an error that causes the Payment Protocol process to be stopped or requires that message be retried, a ProtocolMessage or EncryptedProtocolMessage SHOULD be returned by the party generating the error. The content of the message MUST contain the same **serialized_message** or **encrypted_message** and identifier (if present) and MUST have the status_code set appropriately.

The status_message value SHOULD be set with a human readable explanation of the status code.

Payment Protocol Status Codes

Status Code	Description
1	OK
2	Cancel
100	General / Unknown Error
101	Version Too High
102	Authentication Failed
103	Encrypted Message Required
200	Amount Too High
201	Amount Too Low
202	Amount Invalid
203	Payment Does Not Meet PaymentRequest Requirements
300	Certificate Required
301	Certificate Expired
302	Certificate Invalid for Transaction
303	Certificate Revoked
304	Certificate Not Well Rooted

+==Canceling A Message==+ If a participant to a transaction would like to inform the other party that a previous message should be canceled, they can send the same message with a status code of 2 ("Cancel") and,

where applicable, an updated nonce. How recipients make use of the "Cancel" message is up to developers. For example, wallet developers may want to offer users the ability to cancel payment requests they have sent to other users, and have that change reflected in the recipient's UI. Developers using the non-encrypted ProtocolMessage may want to ignore "Cancel" messages, as it may be difficult to authenticate that the message originated from the same user.

Transport Layer Communication Errors

Communication errors MUST be communicated to the party that initiated the communication via the communication layer's existing error messaging facilities. In the case of TLS-protected HTTP, this SHOULD be done through standard HTTP Status Code messaging (RFC 7231 Section 6).

Extended Payment Protocol Process Details

This BIP extends the Payment Protocol as defined in BIP70.

For the following we assume the Sender already knows the Receiver's public key, and the exchange is being facilitated by a Store & Forward server which requires valid signatures for authentication.

nonce MUST be set to a non-repeating number **and** MUST be chosen by the encryptor. The current epoch time in microseconds SHOULD be used, unless the creating device doesn't have access to a RTC (in the case of a smart card, for example). The service receiving the message containing the **nonce** MAY use whatever method to make sure that the **nonce** is never repeated.

InvoiceRequest Message Creation

- Create an InvoiceRequest message
- **sender_public_key** MUST be set to the public key of an EC keypair
- **amount** is optional. If the amount is not specified by the InvoiceRequest, the Receiver MAY specify the amount in the returned PaymentRequest. If an amount is specified by the InvoiceRequest and a PaymentRequest cannot be generated for that amount, the InvoiceRequest SHOULD return the same InvoiceRequest in a ProtocolMessage or EncryptedProtocolMessage with the status_code and status_message fields set appropriately.
- **memo** is optional. This MAY be set to a human readable description of the InvoiceRequest
- Set **notification_url** to URL that the Receiver will submit completed PaymentRequest (encapsulated in an EncryptedProtocolMessage) to
- If NOT including certificate, set **pki_type** to "none"
- If including certificate:
    - Set **pki_type** to "x509+sha256"
    - Set **pki_data** as it would be set in BIP-0070 (Certificates)
    - Sign InvoiceRequest with signature = "" using the X509 Certificate's private key
    - Set **signature** value to the computed signature

InvoiceRequest Validation

- Validate **sender_public_key** is a valid EC public key
- Validate **notification_url**, if set, contains characters deemed valid for a URL (avoiding XSS related characters, etc).

- If **pki_type** is None, InvoiceRequest is VALID
- If **pki_type** is x509+sha256 and **signature** is valid for the serialized InvoiceRequest where signature is set to "", InvoiceRequest is VALID

Sending Encrypted Payment Protocol Messages using EncryptedProtocolMessages

- Encrypt the serialized Payment Protocol message using AES-256-GCM setup as described in ECDH Point Generation and AES-256 (GCM Mode) Setup
- Create EncryptedProtocolMessage message
- Set **encrypted_message** to be the encrypted value of the Payment Protocol message
- **version** SHOULD be set to the highest version number the client understands (currently 1)
- **sender_public_key** MUST be set to the public key of the Sender's EC keypair
- **receiver_public_key** MUST be set to the public key of the Receiver's EC keypair
- **nonce** MUST be set to the nonce used in the AES-256-GCM encryption operation
- Set **identifier** to the identifier value received in the originating InvoiceRequest's ProtocolMessage or EncryptedProtocolMessage wrapper message
- Set **signature** to ""
- Sign the serialized EncryptedProtocolMessage message with the communicating party's EC public key
- Set **signature** to the result of the signature operation above

SIGNATURE NOTE: EncryptedProtocolMessage messages are signed with the public keys of the party transmitting the message. This allows a Store & Forward server or other transmission system to prevent spam or other abuses. For those who are privacy conscious and don't want the server to track the interactions between two public keys, the Sender can generate a new public key for each interaction to keep their identity anonymous.

Validating and Decrypting Payment Protocol Messages using EncryptedProtocolMessages

- The **nonce** MUST not be repeated. The service receiving the EncryptedProtocolMessage MAY use whatever method to make sure that the nonce is never repeated.
- Decrypt the serialized Payment Protocol message using AES-256-GCM setup as described in ECDH Point Generation and AES-256 (GCM Mode) Setup
- Deserialize the serialized Payment Protocol message

ECDH Point Generation and AES-256 (GCM Mode) Setup

NOTE: AES-256-GCM is used because it provides authenticated encryption facilities, thus negating the need for a separate message hash for authentication.

- Generate the **secret point** using ECDH using the local entity's private key and the remote entity's public key as inputs
- Initialize HMAC_DRBG
    - Use **SHA512(secret point's X value in Big-Endian bytes)** for Entropy
    - Use the given message's **nonce** field for Nonce, converted to byte string (Big Endian)

- Initialize AES-256 in GCM Mode
    - Initialize HMAC_DRBG with Security Strength of 256 bits
    - Use HMAC_DRBG.GENERATE(32) as the Encryption Key (256 bits)

- Use HMAC_DRBG.GENERATE(12) as the Initialization Vector (IV) (96 bits)

AES-256 GCM Authentication Tag Use

The 16 byte authentication tag resulting from the AES-GCM encrypt operation MUST be prefixed to the returned ciphertext. The decrypt operation will use the first 16 bytes of the ciphertext as the GCM authentication tag and the remainder of the ciphertext as the ciphertext in the decrypt operation.

AES-256 GCM Additional Authenticated Data

When either **status_code** OR **status_message** are present, the AES-256 GCM authenticated data used in both the encrypt and decrypt operations MUST be: STRING(status_code) || status_message. Otherwise, there is no additional authenticated data. This provides that, while not encrypted, the status_code and status_message are authenticated.

Initial Public Key Retrieval for InvoiceRequest Encryption

Initial public key retrieval for InvoiceRequest encryption via EncryptedProtocolMessage encapsulation can be done in a number of ways including, but not limited to, the following:

1. Wallet Name public key asset type resolution - DNSSEC-validated name resolution returns Base64 encoded DER-formatted EC public key via TXT Record RFC 5480
2. Key Server lookup - Key Server lookup (similar to PGP's pgp.mit.edu) based on key server identifier (i.e., e-mail address) returns Base64 encoded DER-formatted EC public key RFC 5480
3. QR Code - Use of QR-code to encode SEC-formatted EC public key RFC 5480
4. Address Service Public Key Exposure

Payment / PaymentACK Messages with a HTTP Store & Forward Server

If a Store & Forward server wishes to protect themselves from spam or abuse, they MAY enact whatever rules they deem fit, such as the following:

- Once an InvoiceRequest or PaymentRequest is received, all subsequent messages using the same identifier must use the same Sender and Receiver public keys.
- For each unique identifier, only one message each of type InvoiceRequest, PaymentRequest, and PaymentACK may be submitted. Payment messages may be submitted/overwritten multiple times. All messages submitted after a PaymentACK is received will be rejected.
- Specific messages are only saved until they have been verifiably received by the intended recipient or a certain amount of time has passed, whichever comes first.

Clients SHOULD keep in mind Receivers can broadcast a transaction without returning an ACK. If a Payment message needs to be updated, it SHOULD include at least one input referenced in the original transaction to prevent the Receiver from broadcasting both transactions and getting paid twice.

Public Key & Signature Encoding

- All x.509 certificates included in any message defined in this BIP MUST be DER [ITU.X690.1994] encoded.
- All EC public keys (**sender_public_key**, **receiver_public_key**) in any message defined in this BIP MUST be http://www.secg.org/sec2-v2.pdf ECDSA Public Key ECPoints encoded using http://www.secg.org/sec1-v2.pdf. Encoding MAY be compressed.
- All ECC signatures included in any message defined in this BIP MUST use the SHA-256 hashing algorithm and MUST be DER [ITU.X690.1994] encoded.
- All OpenPGP certificates must follow RFC4880, sections 5.5 and 12.1.

Implementation

A reference implementation for a Store & Forward server supporting this proposal can be found here:

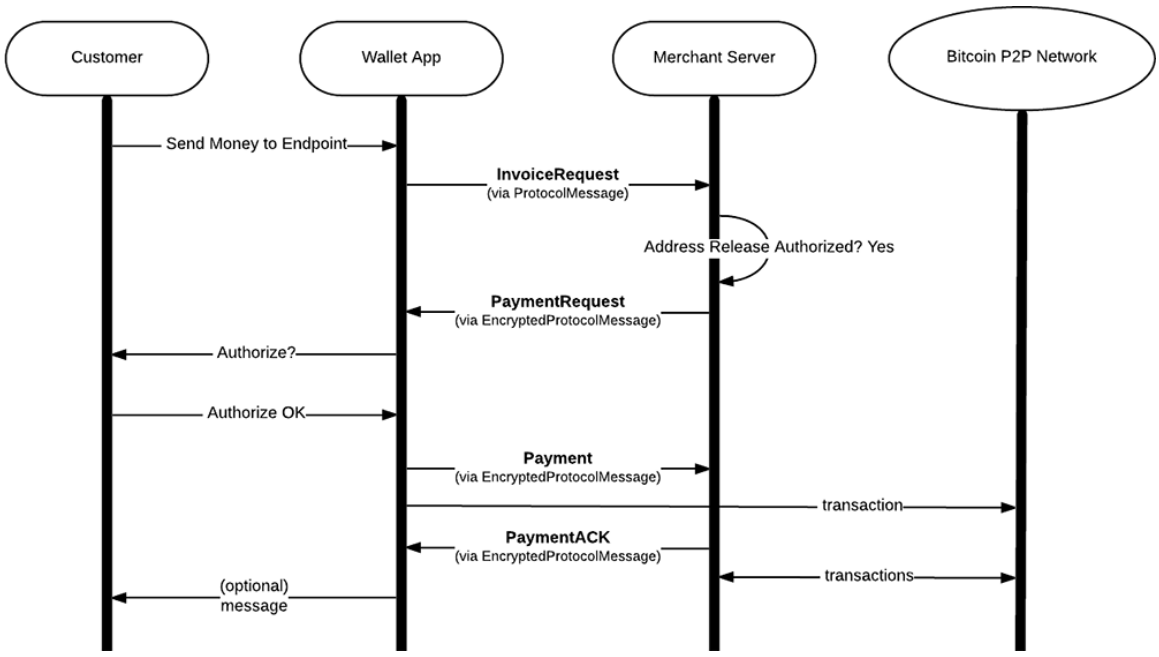
Addressimo

A reference client implementation can be found in the InvoiceRequest functional testing for Addressimo here:

BIP75 Client Reference Implementation

BIP70 Extension

The following flowchart is borrowed from BIP70 and expanded upon in order to visually describe how this BIP is an extension to BIP70.

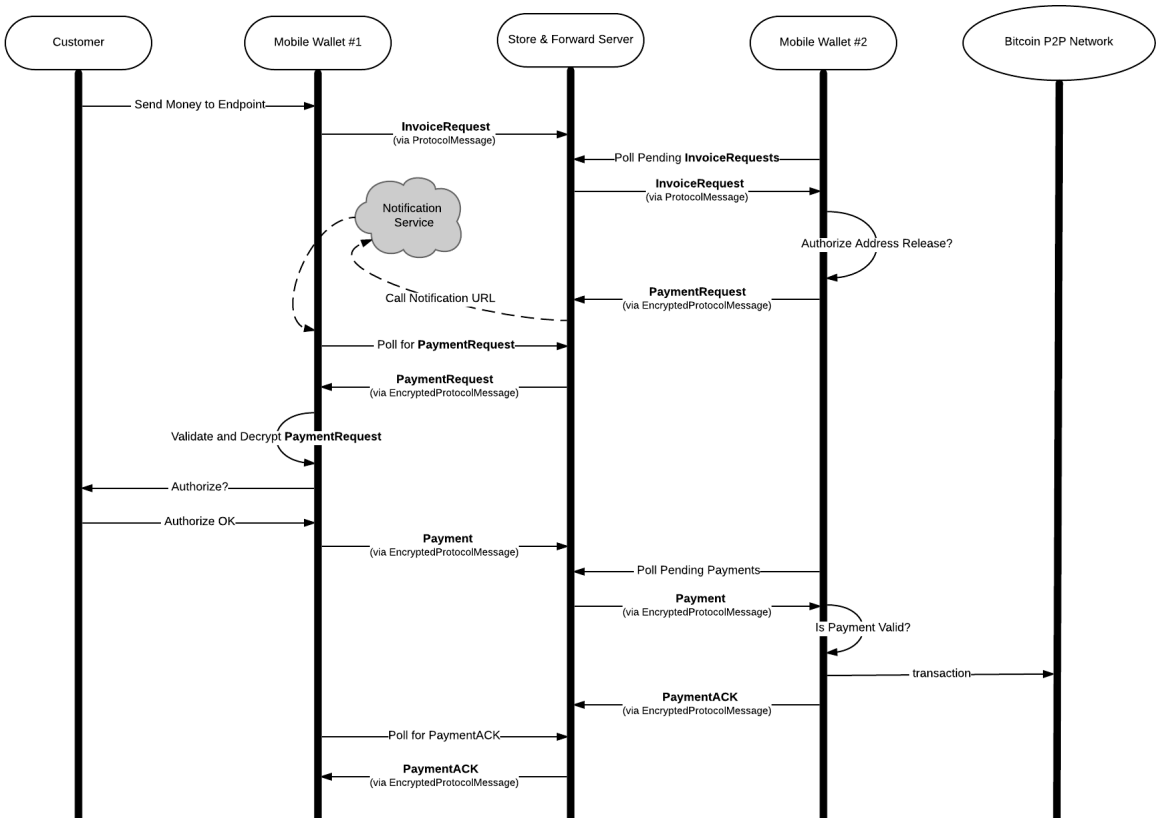


bip-0075/bip70-extension.png

Mobile to Mobile Examples

Full Payment Protocol

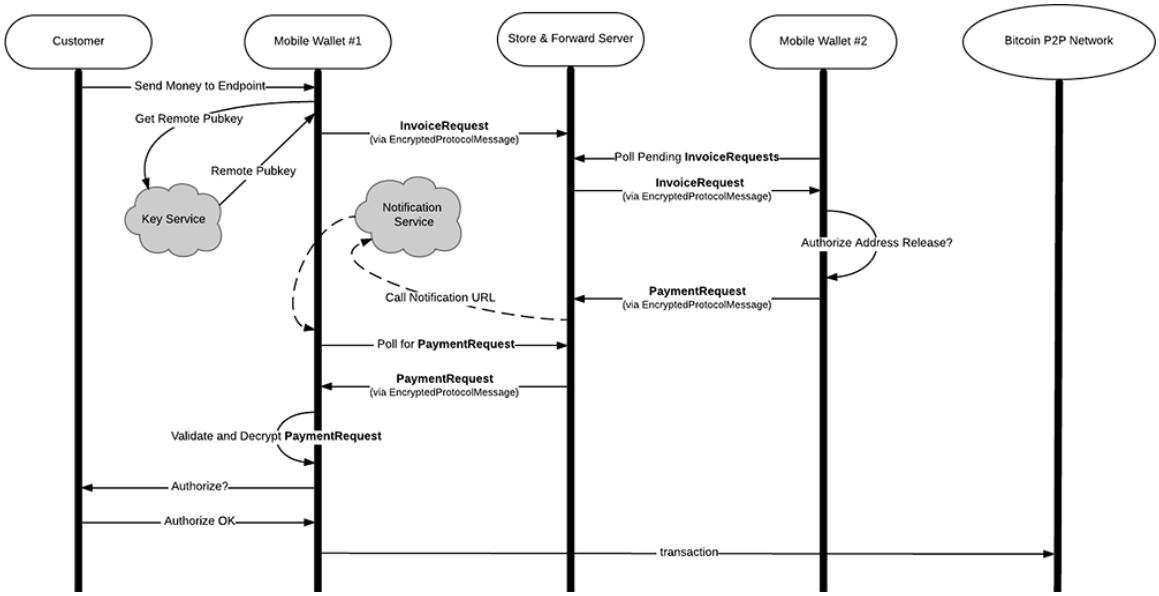
The following diagram shows a sample flow in which one mobile client is sending value to a second mobile client with the use of an InvoiceRequest, a Store & Forward server, PaymentRequest, Payment and PaymentACK. In this case, the PaymentRequest, Payment and PaymentACK messages are encrypted using EncryptedProtocolMessage **and** the Receiver submits the transaction to the Bitcoin network.



bip-0075/mobile-sf-ir-with-payment.png

Encrypting Initial InvoiceRequest via EncryptedProtocolMessage

The following diagram shows a sample flow in which one mobile client is sending value to a second mobile client using an EncryptedProtocolMessage to transmit the InvoiceRequest using encryption, Store & Forward server, and PaymentRequest. In this case, all Payment Protocol messages are encrypting using EncryptedProtocolMessage **and** the Sender submits the transaction to the Bitcoin network.



bip-0075/mobile-sf-encrypted-ir-without-payment.png

References

- BIP70 - Payment Protocol
- ECDH
- HMAC_DRBG
- NIST Special Publication 800-38D - Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC
- RFC6979
- Address Reuse
- FIPS 180-4 (Secure Hash Standard)

```
BIP: 78
Layer: Applications
Title: A Simple Payjoin Proposal
Authors: Nicolas Dorier <nicolas.dorier@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2019-05-01
License: BSD-2-Clause
Replaces: 79
```

BIP 78

Introduction

Abstract

This document proposes a protocol for two parties to negotiate a coinjoin transaction during a payment between them.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

When two parties (later referred to as sender and receiver) want to transact, most of the time, the sender creates a transaction spending their own Unspent Transaction Outputs (UTXOs), signs it and broadcasts it on the network.

This simple model gave birth to several heuristics impacting the privacy of the parties and of the network as a whole.

- Common input ownership heuristic: In most transactions, all the inputs belong to the same party.
- Change identification from scriptPubKey type: If all inputs are spending UTXOs of a certain scriptPubKey type, then the change output is likely to have the same scriptPubKey type, too.
- Change identification from round amount: If an output in the transaction has a round amount, it is likely an output belonging to the receiver.

We will designate these three heuristics as `common-input`, `change-scriptpubkey`, `change-round-amount`.

The problems we aim to solve are:

- For the receiver, there is a missed opportunity to consolidate their own UTXOs or making payment in the sender's transaction.
- For the sender, there are privacy leaks regarding their wallet that happen when someone applies the heuristics detailed above to their transaction.

Our proposal gives an opportunity for the receiver to consolidate their UTXOs while also batching their own payments, without creating a new transaction. (Saving fees in the process) For the sender, it allows them to invalidate the three heuristics above. With the receiver's involvement, the heuristics can even be poisoned. (ie, using the heuristics to intentionally mislead blockchain analysis)

Note that the existence of this proposal is also improving the privacy of parties who are not using it by making the three heuristics unreliable to the network as a whole.

Relation to BIP79 (Bustapay)

Another implementation proposal has been written: BIP79 Bustapay.

We decided to deviate from it for several reasons:

- It was not using PSBT, so if the receiver wanted to bump the fee, they would need the full UTXO set.
- Inability to change the payment output to match scriptPubKey type.
- Lack of basic versioning negotiation if the protocol evolves.
- No standardization of error condition for proper feedback to the sender.

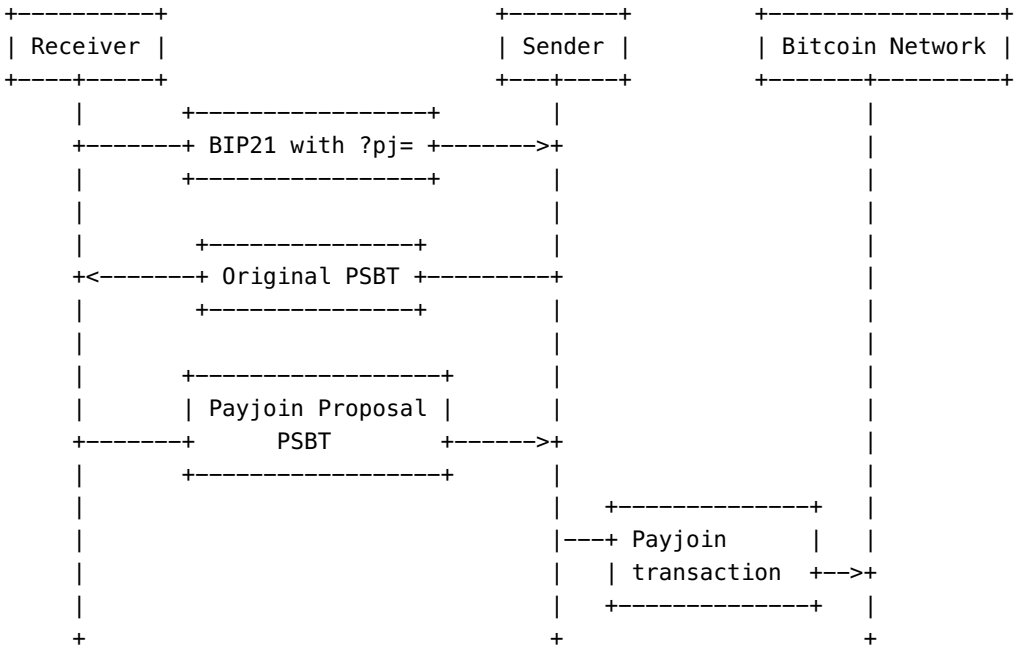
Other than that, our proposal is very similar.

Specification

Protocol

In a payjoin payment, the following steps happen:

- The receiver of the payment, presents a BIP 21 URI to the sender with a parameter `pj=` describing a payjoin endpoint.
- The sender creates a signed, finalized PSBT with witness UTXO or previous transactions of the inputs. We call this PSBT the `original`.
- The receiver replies back with a signed PSBT containing his own signed inputs/outputs and those of the sender. We call this PSBT `Payjoin proposal`.
- The sender verifies the proposal, re-signs his inputs and broadcasts the transaction to the Bitcoin network. We call this transaction `Payjoin transaction`.



The original PSBT is sent in the HTTP POST request body, base64 serialized, with `text/plain` in the `Content-Type` HTTP header and `Content-Length` set correctly. The payjoin proposal PSBT is sent in the HTTP response body, base64 serialized with HTTP code 200.

To ensure compatibility with web-wallets and browser-based-tools, all responses (including errors) must contain the HTTP header `Access-Control-Allow-Origin: *`.

The sender must ensure that the URL refers to a scheme or protocol using authenticated encryption, for example TLS with certificate validation, or a .onion link to a hidden service whose public key identifier has already been communicated via a TLS connection. Senders SHOULD NOT accept a URL representing an unencrypted or unauthenticated connection.

The original PSBT MUST:

- Have all the `witnessUTXO` or `nonWitnessUTXO` information filled in.
- Be finalized.

- Not include fields unneeded for the receiver such as global xpubs or keypath information.
- Be broadcastable.

The original PSBT MAY:

- Have outputs unrelated to the payment for batching purpose.

The original PSBT SHOULD NOT:

- Include mixed input types until September 2024. Mixed inputs were previously completely disallowed so this gives some grace period for receivers to update.

The payjoin proposal MUST:

- Use all the inputs from the original PSBT.
- Use all the outputs which do not belong to the receiver from the original PSBT.
- Only finalize the inputs added by the receiver. (Referred later as `additional inputs`)
- Only fill the `witnessUTXO` or `nonWitnessUTXO` for the additional inputs.

The payjoin proposal MAY:

- Add, or replace the outputs belonging to the receiver unless output substitution is disabled.

The payjoin proposal SHOULD NOT:

- Include mixed input types until September 2024. Mixed inputs were previously completely disallowed so this gives some grace period for senders to update.

The payjoin proposal MUST NOT:

- Shuffle the order of inputs or outputs, the additional outputs or additional inputs must be inserted at a random index.
- Decrease the absolute fee of the original transaction.

BIP21 payjoin parameters

This proposal is defining the following new BIP 21 URI parameters:

- `pj=`: Represents an http(s) endpoint which the sender can POST the original PSBT.
- `pjos=0`: Signal to the sender that they MUST disallow payment output substitution. (See Unsecured payjoin server)

Note: the `amount` parameter is *not* required.

Optional parameters

When the payjoin sender posts the original PSBT to the receiver, he can optionally specify the following HTTP query string parameters:

- `v=`, the version number of the payjoin protocol that the sender is using. The current version is `1`.

This can be used in the future so the receiver can reject a payjoin if the sender is using a version which is not supported via an error HTTP 400, `version-unsupported`. If not specified, the receiver will assume the sender is `v=1`.

If the receiver does not support the version of the sender, they should send an error with the list of supported versions:

```
{
    "errorCode": "version-unsupported",
    "supported" : [ 2, 3, 4 ],
    "message": "The version is not supported anymore"
}
```

- `additionalfeeoutputindex=`, if the sender is willing to pay for increased fee, this indicate output can have its value subtracted to pay for it.

If the `additionalfeeoutputindex` is out of bounds or pointing to the payment output meant for the receiver, the receiver should ignore the parameter. See fee output for more information.

- `maxadditionalfeecontribution=`, if the sender is willing to pay for increased fee, an integer defining the maximum amount in satoshis that the sender is willing to contribute towards fees for the additional inputs. `maxadditionalfeecontribution` must be ignored if set to less than zero. See fee output for more information.

Note that both `maxadditionalfeecontribution=` and `additionalfeeoutputindex=` must be specified and valid for the receiver to be allowed to decrease an output belonging to the sender. This fee contribution can't be used to pay for anything else than additional input's weight.

- `minfeerate=`, a decimal in satoshi per vbyte that the sender can use to constraint the receiver to not drop the minimum fee rate too much.
- `disableoutputsubstitution=`, a boolean indicating if the sender forbids the receiver to substitute the receiver's output, see payment output substitution. (default to `false`)

Receiver's well known errors

If for some reason the receiver is unable to create a payjoin proposal, it will reply with a HTTP code different than 200. The receiver is not constrained to specific set of errors, some are specified in this proposal.

The errors have the following format:

```
{
    "errorCode": "leaking-data",
    "message": "Key path information or GlobalXPubs should not be included in the original
PSBT."
}
```

The well-known error codes are:

Error code	**Meaning**
unavailable	The payjoin endpoint is not available for now.

Error code	Meaning
not-enough-money	The receiver added some inputs but could not bump the fee of the payjoin proposal.
version-unsupported	This version of payjoin is not supported.
original-psbt-rejected	The receiver rejected the original PSBT.

The receiver is allowed to return implementation-specific errors which may assist the sender to diagnose any issue.

However, it is important that error codes that are not well-known and that the message do not appear on the sender's software user interface. Such error codes or messages could be used maliciously to phish a non-technical user. Instead those errors or messages can only appear in debug logs.

It is advised to hard code the description of the well known error codes into the sender's software.

Fee output

In some situation, the sender might want to pay some additional fee in the payjoin proposal. If such is the case, the sender must use both optional parameters `additionalfeeoutputindex=` and `maxadditionalfeecontribution=` to indicate which output and how much the receiver can subtract fee.

There are several cases where a fee output is useful:

- The sender's original transaction's fee rate is at the minimum accepted by the network, aka `minimum relay transaction fee rate`, which is typically 1 satoshi per vbyte.

In such case, the receiver will need to increase the fee of the transaction after adding his own inputs to not drop below the minimum relay transaction fee rate.

- The sender's wallet software is using round fee rate.

If the sender's fee rate is always round, then a blockchain analyst can easily spot the transactions of the sender involving payjoin by checking if, when removing a single input to the suspected payjoin transaction, the resulting fee rate is round. To prevent this, the sender can agree to pay more fee so the receiver make sure that the payjoin transaction fee is also round.

- The sender's transaction is time sensitive.

When a sender picks a specific fee rate, the sender expects the transaction to be confirmed after a specific amount of time. But if the receiver adds an input without bumping the fee of the transaction, the payjoin transaction fee rate will be lower, and thus, longer to confirm.

Our recommendation for `maxadditionalfeecontribution=` is `originalPSBTFeeRate * 110`.

Receiver's original PSBT checklist

The receiver needs to do some check on the original PSBT before proceeding:

- Non-interactive receivers (like a payment processor) need to check that the original PSBT is broadcastable. ∗
- If the sender included inputs in the original PSBT owned by the receiver, the receiver must either return error `original-psbt-rejected` or make sure they do not sign those inputs in the payjoin proposal.
- Make sure that the inputs included in the original transaction have never been seen before.
    - This prevents probing attacks.
    - This prevents reentrant payjoin, where a sender attempts to use payjoin transaction as a new original transaction for a new payjoin.

∗: Interactive receivers are not required to validate the original PSBT because they are not exposed to probing attacks.

Sender's payjoin proposal checklist

The sender should check the payjoin proposal before signing it to prevent a malicious receiver from stealing money.

- Verify that the absolute fee of the payjoin proposal is equals or higher than the original PSBT.
- If the receiver's BIP21 signalled `pjos=0`, disable payment output substitution.
- Verify that the transaction version, and the nLockTime are unchanged.
- Check that the sender's inputs' sequence numbers are unchanged.
- For each input in the proposal:
    - Verify that no keypaths are in the PSBT input
    - Verify that no partial signature has been filled
    - If it is one of the sender's inputs:
        - Verify that input's sequence is unchanged.
        - Verify the PSBT input is not finalized
    - If it is one of the receiver's inputs:
        - Verify the PSBT input is finalized
        - Verify that `non_witness_utxo` or `witness_utxo` are filled in.
    - Verify that the payjoin proposal inputs all specify the same sequence value.
    - Verify that all of sender's inputs from the original PSBT are in the proposal.
- For each output in the proposal:
    - Verify that no keypaths are in the PSBT output
    - If the output is the fee output:
        - The amount that was subtracted from the output's value is less than or equal to `maxadditionalfeecontribution`. Let's call this amount `actual contribution`.
        - Make sure the actual contribution is only going towards fees: The `actual contribution` is less than or equals to the difference of absolute fee between the payjoin proposal and the original PSBT.
        - Make sure the actual contribution is only paying for fees incurred by additional inputs: `actual contribution` is less than or equal to `originalPSBTFeeRate * vsize(sender_input_type) * (count(payjoin_proposal_inputs) - count(original_psbt_inputs))`. (see Fee output section)

        - If the output is the payment output and payment output substitution is allowed,
            - Do not make any check
        - Else
            - Make sure the output's value did not decrease.
        - Verify that all sender's outputs (ie, all outputs except the output actually paid to the receiver) from the original PSBT are in the proposal.
- Once the proposal is signed, if `minfeerate` was specified, check that the fee rate of the payjoin transaction is not less than this value.

The sender must be careful to only sign the inputs that were present in the original PSBT and nothing else.

Note:

- The sender must allow the receiver to add/remove or modify the receiver's own outputs. (if payment output substitution is disabled, the receiver's outputs must not be removed or decreased in value)
- The sender should allow the receiver to not add any inputs. This is useful for the receiver to change the payment output scriptPubKey type.
- If the receiver added no inputs, the sender's wallet implementation should accept the payjoin proposal, but not mark the transaction as an actual payjoin in the user interface.

Our method of checking the fee allows the receiver and the sender to batch payments in the payjoin transaction. It also allows the receiver to pay the fee for batching adding his own outputs.

Rationale

There are several consequences of our proposal:

- The receiver can bump the fee of the original transaction.
- The receiver can modify the outputs of the original PSBT.
- The sender must provide the UTXO information (Witness or previous transaction) in the PSBT.

Respecting the minimum relay fee policy

To be properly relayed, a Bitcoin transaction needs to pay at least 1 satoshi per virtual byte. When blocks are not full, the original transaction might already at the minimum relay fee rate (currently 1 satoshi per virtual byte), so if the receiver adds their own input, they need to make sure the fee is increased such that the rate does not drop below the minimum relay fee rate. In such case, the sender must set both `maxadditionalfeecontribution=` and `additionalfeeoutputindex=`.

See the Fee output section for more information.

We also recommend the sender to set `minfeerate=`, as the sender's node policy might be different from the receiver's policy.

Defeating heuristics based on the fee calculation

Most wallets are creating a round fee rate (like 2 sat/b). If the payjoin transaction's fee was not increased by the added size, then those payjoin transactions could easily be identifiable on the blockchain.

Not only would those transactions stand out by not having a round fee (like 1.87 sat/b), but any suspicion of payjoin could be confirmed by checking if removing one input would create a round fee rate. In such case, the sender must set both `maxadditionalfeecontribution=` and `additionalfeeoutputindex=`.

The recommended value `maxadditionalfeecontribution=` is explained in the Fee output section. We also recommend the sender to set `minfeerate=`, as the sender's node policy might be different from the receiver's policy.

Receiver does not need to be a full node

Because the receiver needs to bump the fee to keep the same fee rate as the original PSBT, it needs the input's UTXO information to know what is the original fee rate. Without PSBT, light wallets like Wasabi Wallet would not be able to receive a payjoin transaction.

The validation (policy and consensus) of the original transaction is optional: a receiver without a full node can decide to create the payjoin transaction and automatically broadcast the original transaction after a timeout of 1 minute, and only verify that it has been propagated in the network.

However, non-interactive receivers (like a payment processor) need to verify the transaction to prevent UTXO probing attacks.

This is not a concern for interactive receivers like Wasabi Wallet, because those receivers can just limit the number of original PSBT proposals of a specific address to one. With such wallets, the attacker has no way to generate new deposit addresses to probe the UTXOs.

Spare change donation

Small change inside wallets are detrimental to privacy. Mixers like Wasabi wallet, because of its protocol, eventually generate such small change.

A common way to protect your privacy is to donate those spare changes, to deposit them in an exchange or on your favorite merchant's store account. Those kind of transactions can easily be spotted on the blockchain: There is only one output.

However, if you donate via payjoin, it will look like a normal transaction.

On top of this the receiver can poison analysis by randomly faking a round amount of satoshi for the additional output.

Payment output substitution

Unless disallowed by the sender explicitly via `disableoutputsubstitution=true` or by the BIP21 URL via the query parameter `pjos=0`, the receiver is free to decrease the amount or change the scriptPubKey output paying to himself. Note that if payment output substitution is disallowed, the receiver can still increase the amount of the output. (See the reference implementation)

For example, if the sender's scriptPubKey type is P2WPKH while the receiver's payment output in the original PSBT is P2SH, then the receiver can substitute the payment output to be P2WPKH to match the sender's scriptPubKey type.

Unsecured payjoin server

A receiver might run the payment server (generating the BIP21 invoice) on a different server than the payjoin server, which could be less trusted than the payment server.

In such case, the payment server can signal to the sender, via the BIP21 parameter `pjos=0`, that they MUST disallow payment output substitution. A compromised payjoin server could steal the hot wallet outputs of the receiver, but would not be able to re-route payment to himself.

Impacted heuristics

Our proposal of payjoin breaks the following blockchain heuristics:

- Common inputs heuristics.

Because payjoin is mixing the inputs of the sender and receiver, this heuristic becomes unreliable.

- Change identification from scriptPubKey type heuristics

When Alice pays Bob, if Alice is using P2SH but Bob's deposit address is P2WPKH, the heuristic would assume that the P2SH output is the change address of Alice. This is now however a broken assumption, as the payjoin receiver has the freedom to mislead analytics by purposefully changing the invoice's address in the payjoin transaction.

See payment output substitution.

- Change identification from round change amount

If Alice pays Bob, she might be tempted to pay him a round amount, like `1.23000000 BTC`. When this happens, blockchain analysis often identifies the output without the round amount as the change of the transaction.

For this reason, during a spare change case, the receiver may add an output with a rounded amount randomly.

Attack vectors

On the receiver side: UTXO probing attack

When the receiver creates a payjoin proposal, they expose one or more inputs belonging to them.

An attacker could create multiple original transactions in order to learn the UTXOs of the receiver, while not broadcasting the payjoin proposal.

While we cannot prevent this type of attack entirely, we implemented the following mitigations:

- When the receiver detects an original transaction being broadcast, or if the receiver detects that the original transaction has been double spent, then they will reuse the UTXO that was exposed for the next payjoin.
- While the exposed UTXO will be reused in priority to not leak other UTXOs, there is no strong guarantee about it. This prevents the attacker from detecting with certainty the next payjoin of the merchant to another peer.

Note that probing attacks are only a problem for automated payment systems such as BTCPay Server. End-user wallets with payjoin capabilities are not affected, as the attacker can't create multiple invoices to force the receiver to expose their UTXOs.

On the sender side: Double payment risk for hardware wallets

For a successful payjoin to happen, the sender needs to sign two transactions double spending each other: The original transaction and the payjoin proposal.

The sender's software wallet can verify that the payjoin proposal is legitimate by the sender's checklist.

However, a hardware wallet can't verify that this is indeed the case. This means that the security guarantee of the hardware wallet is decreased. If the sender's software is compromised, the hardware wallet would sign two valid transactions, thus sending two payments.

Without payjoin, the maximum amount of money that could be lost by a compromised software is equal to one payment (via payment output substitution). Note that the sender can disallow payment output substitution by using the optional parameter `disableoutputsubstitution=true`.

With payjoin, the maximum amount of money that can be lost is equal to two payments.

Reference sender's implementation

Here is pseudo code of a sender implementation. `RequestPayjoin` takes the BIP21 URI of the payment, the wallet and the `signedPSBT`.

The `signedPSBT` represents a PSBT which has been fully signed, but not yet finalized. We then prepare `originalPSBT` from the `signedPSBT` via the `CreateOriginalPSBT` function and get back the `proposal`.

While we verify the `proposal`, we also import into it information about our own inputs and outputs from the `signedPSBT`. At the end of this `RequestPayjoin`, the proposal is verified and ready to be signed.

We logged the different PSBT involved, and show the result in our test vectors.

```
public async Task<PSBT> RequestPayjoin(
    BIP21Uri bip21,
    Wallet wallet,
    PSBT signedPSBT,
    PayjoinClientParameters optionalParameters)
{
    Log("Unfinalized signed PSBT" + signedPSBT);
    // Extracting the pj link.
    var endpoint = bip21.ExtractPayjointEndpoint();
    if (signedPSBT.IsAllFinalized())
        throw new InvalidOperationException("The original PSBT should not be finalized.");
    PSBTOutput feePSBTOutput = null;

    bool allowOutputSubstitution = !optionalParameters.DisableOutputSubstitution;
    if (bip21.Parameters.Contains("pjos") && bip21.Parameters["pjos"] == "0")
        allowOutputSubstitution = false;

    if (optionalParameters.AdditionalFeeOutputIndex != null &&
```

```
optionalParameters.MaxAdditionalFeeContribution != null)
        feePSBTOutput = signedPSBT.Outputs[optionalParameters.AdditionalFeeOutputIndex];
    Script paymentScriptPubKey = bip21.Address == null ? null : bip21.Address.ScriptPubKey;
    decimal originalFee = signedPSBT.GetFee();
    PSBT originalPSBT = CreateOriginalPSBT(signedPSBT);
    Transaction originalGlobalTx = signedPSBT.GetGlobalTransaction();
    TxOut feeOutput = feePSBTOutput == null ? null :
originalGlobalTx.Outputs[feePSBTOutput.Index];
    var originalInputs = new Queue<(TxIn OriginalTxIn, PSBTInput SignedPSBTInput)>();
    for (int i = 0; i < originalGlobalTx.Inputs.Count; i++)
    {
        originalInputs.Enqueue((originalGlobalTx.Inputs[i], signedPSBT.Inputs[i]));
    }
    var originalOutputs = new Queue<(TxOut OriginalTxOut, PSBTOutput SignedPSBTOutput)>();
    for (int i = 0; i < originalGlobalTx.Outputs.Count; i++)
    {
      originalOutputs.Enqueue((originalGlobalTx.Outputs[i], signedPSBT.Outputs[i]));
    }
    // Add the client side query string parameters
    endpoint = ApplyOptionalParameters(endpoint, optionalParameters);
    Log("original PSBT" + originalPSBT);
    PSBT proposal = await SendOriginalTransaction(endpoint, originalPSBT, cancellationToken);
    Log("payjoin proposal" + proposal);
    // Checking that the PSBT of the receiver is clean
    if (proposal.GlobalXPubs.Any())
    {
        throw new PayjoinSenderException("GlobalXPubs should not be included in the receiver's
PSBT");
    }
    ////////////

    if (proposal.CheckSanity() is List<PSBTError> errors && errors.Count > 0)
        throw new PayjoinSenderException($"The proposal PSBT is not sane ({errors[0]})");

    var proposalGlobalTx = proposal.GetGlobalTransaction();
    // Verify that the transaction version, and nLockTime are unchanged.
    if (proposalGlobalTx.Version != originalGlobalTx.Version)
        throw new PayjoinSenderException($"The proposal PSBT changed the transaction version");
    if (proposalGlobalTx.LockTime != originalGlobalTx.LockTime)
        throw new PayjoinSenderException($"The proposal PSBT changed the nLocktime");

    HashSet<Sequence> sequences = new HashSet<Sequence>();
    // For each inputs in the proposal:
    foreach (PSBTInput proposedPSBTInput in proposal.Inputs)
    {
        if (proposedPSBTInput.HDKeyPaths.Count != 0)
            throw new PayjoinSenderException("The receiver added keypaths to an input");
        if (proposedPSBTInput.PartialSigs.Count != 0)
            throw new PayjoinSenderException("The receiver added partial signatures to an
input");
        PSBTInput proposedTxIn =
proposalGlobalTx.Inputs.FindIndexedInput(proposedPSBTInput.PrevOut).TxIn;
        bool isOurInput = originalInputs.Count > 0 && originalInputs.Peek().OriginalTxIn.PrevOut
== proposedPSBTInput.PrevOut;
        // If it is one of our input
```

```
        if (isOurInput)
        {
            OutPoint inputPrevout = ourPrevouts.Dequeue();
            TxIn originalTxin = originalGlobalTx.Inputs.FromOutpoint(inputPrevout);
            PSBTInput originalPSBTInput = originalPSBT.Inputs.FromOutpoint(inputPrevout);
            // Verify that sequence is unchanged.
            if (input.OriginalTxIn.Sequence != proposedTxIn.Sequence)
                throw new PayjoinSenderException("The proposedTxIn modified the sequence of one
of our inputs")
            // Verify the PSBT input is not finalized
            if (proposedPSBTInput.IsFinalized())
                throw new PayjoinSenderException("The receiver finalized one of our inputs");
            sequences.Add(proposedTxIn.Sequence);

            // Fill up the info from the original PSBT input so we can sign and get fees.
            proposedPSBTInput.NonWitnessUtxo = input.SignedPSBTInput.NonWitnessUtxo;
            proposedPSBTInput.WitnessUtxo = input.SignedPSBTInput.WitnessUtxo;
            // We fill up information we had on the signed PSBT, so we can sign it.
            foreach (var hdKey in input.SignedPSBTInput.HDKeyPaths)
            proposedPSBTInput.HDKeyPaths.Add(hdKey.Key, hdKey.Value);
            proposedPSBTInput.RedeemScript = signedPSBTInput.RedeemScript;
            proposedPSBTInput.RedeemScript = input.SignedPSBTInput.RedeemScript;
        }
        else
        {
            // Verify the PSBT input is finalized
            if (!proposedPSBTInput.IsFinalized())
                throw new PayjoinSenderException("The receiver did not finalized one of their
input");
            // Verify that non_witness_utxo or witness_utxo are filled in.
            if (proposedPSBTInput.NonWitnessUtxo == null && proposedPSBTInput.WitnessUtxo ==
null)
                throw new PayjoinSenderException("The receiver did not specify non_witness_utxo
or witness_utxo for one of their inputs");
            sequences.Add(proposedTxIn.Sequence);
        }
    }

    // Verify that all of sender's inputs from the original PSBT are in the proposal.
    if (originalInputs.Count != 0)
        throw new PayjoinSenderException("Some of our inputs are not included in the proposal");

    // Verify that the payjoin proposal did not introduced mixed inputs' sequence.
    if (sequences.Count != 1)
        throw new PayjoinSenderException("Mixed sequence detected in the proposal");

    decimal newFee = proposal.GetFee();
    decimal additionalFee = newFee - originalFee;
    if (additionalFee < 0)
      throw new PayjoinSenderException("The receiver decreased absolute fee");
    // For each outputs in the proposal:
    foreach (PSBTOutput proposedPSBTOutput in proposal.Outputs)
    {
        // Verify that no keypaths is in the PSBT output
        if (proposedPSBTOutput.HDKeyPaths.Count != 0)
```

```
            throw new PayjoinSenderException("The receiver added keypaths to an output");
        if (originalOutputs.Count == 0)
                continue;
        var originalOutput = originalOutputs.Peek();
        bool isOriginalOutput = originalOutput.OriginalTxOut.ScriptPubKey ==
proposedPSBTOutput.ScriptPubKey;
        bool substitutedOutput = !isOriginalOutput &&
                                 allowOutputSubstitution &&
                                 originalOutput.OriginalTxOut.ScriptPubKey ==
paymentScriptPubKey;
        if (isOriginalOutput || substitutedOutput)
        {
            originalOutputs.Dequeue();
            if (originalOutput.OriginalTxOut == feeOutput)
            {
                var actualContribution = feeOutput.Value - proposedPSBTOutput.Value;
                // The amount that was subtracted from the output's value is less than or equal
to maxadditionalfeecontribution
                if (actualContribution > optionalParameters.MaxAdditionalFeeContribution)
                    throw new PayjoinSenderException("The actual contribution is more than
maxadditionalfeecontribution");
                // Make sure the actual contribution is only paying fee
                if (actualContribution > additionalFee)
                    throw new PayjoinSenderException("The actual contribution is not only paying
fee");
                // Make sure the actual contribution is only paying for fee incurred by
additional inputs
                // This assumes an additional input can be up to 110 bytes.
                int additionalInputsCount = proposalGlobalTx.Inputs.Count -
originalGlobalTx.Inputs.Count;
                if (actualContribution > originalFeeRate * 110 * additionalInputsCount)
                    throw new PayjoinSenderException("The actual contribution is not only paying
for additional inputs");
            }
            else if (allowOutputSubstitution && output.OriginalTxOut.ScriptPubKey ==
paymentScriptPubKey)
            {
                // That's the payment output, the receiver may have changed it.
            }
            else
            {
                if (originalOutput.OriginalTxOut.Value > proposedPSBTOutput.Value)
                    throw new PayjoinSenderException("The receiver decreased the value of one of
the outputs");
            }
            // We fill up information we had on the signed PSBT, so we can sign it.
            foreach (var hdKey in output.SignedPSBTOutput.HDKeyPaths)
                proposedPSBTOutput.HDKeyPaths.Add(hdKey.Key, hdKey.Value);
            proposedPSBTOutput.RedeemScript = output.SignedPSBTOutput.RedeemScript;
        }
    }
    // Verify that all of sender's outputs from the original PSBT are in the proposal.
    if (originalOutputs.Count != 0)
    {
        // The payment output may have been substituted
```

```
        if (!allowOutputSubstitution ||
            originalOutputs.Count != 1 ||
            originalOutputs.Dequeue().OriginalTxOut.ScriptPubKey != paymentScriptPubKey)
            {
                throw new PayjoinSenderException("Some of our outputs are not included in the
proposal");
            }
    }

    // After signing this proposal, we should check if minfeerate is respected.
    Log("payjoin proposal filled with sender's information" + proposal);
    return proposal;
}

// Finalize the signedPSBT and remove confidential information
PSBT CreateOriginalPSBT(PSBT signedPSBT)
{
    var original = signedPSBT.Clone();
    original = original.Finalize();
    foreach (var input in original.Inputs)
    {
        input.HDKeyPaths.Clear();
        input.PartialSigs.Clear();
        input.Unknown.Clear();
    }
    foreach (var output in original.Outputs)
    {
        output.Unknown.Clear();
        output.HDKeyPaths.Clear();
    }
    original.GlobalXPubs.Clear();
    return original;
}
```

Test vectors

A successful exchange with:

InputScriptType	Original PSBT Fee rate	maxadditionalfeecontribution	additionalfeeoutputindex
P2SH-P2WPKH	2 sat/vbyte	0.00000182	0

```
Unfinalized signed PSBT

cHNidP8BAHMCAAAAAY8nutGgJdyYGXWiBEb45Hoe9lWGbkxh/6bNiOJdCDuDAAAAAAD+////
AtyVuAUAAAAAF6kUHehJ8GnSdBUOOv6ujXLrWmsJRDCHgIQeAAAAAAAXqRR3QJbbz0hnQ8IvQ0fptGn+votneofTAAAAAAEB
IKgb1wUAAAAAF6kU3k4ekGHKWRNbA1rV5tR5kEVDVNCHAQQWABTHikVyU1WCjVZYB03VJg1fy2mFMCICAxWawBqg1YdUxLTY
t9NJ7R7fzws2K09rVRBnI6KFj4UWRzBEAiB8Q+A6dep+Rz92vhy26lT0AjZn4PRLi8Bf9qoB/CMk0wIgP/
Rj2PWZ3gEjUkTlhDRNAQ0gXwTO7t9n+V14pZ6oljUBIgYDFZrAGqDVh1TEtNi300ntHt/
PCzYrT2tVEGcjooWPhRYYSFzWUDEAAIABAACAAAAAgAEAAAAAAAAAAAAEAFgAURvYaK7pzgo7lhbSl/
DeUan2MxRQiAgLKC8FYHmmul/HrXLUcMDCjfuRg/dhEkG8CO26cEC6vfBhIXNZQMQAAgAEAAIAAAACAAQAAAAEAAAAAAA==

Original PSBT
```

```
cHNidP8BAHMCAAAAAY8nutGgJdyYGXWiBEb45Hoe9lWGbkxh/6bNiOJdCDuDAAAAAAD+////
AtyVuAUAAAAAF6kUHehJ8GnSdBUOOv6ujXLrWmsJRDCHgIQeAAAAAAAXqRR3QJbbz0hnQ8IvQ0fptGn+votneofTAAAAAAEB
IKgb1wUAAAAAF6kU3k4ekGHKWRNbA1rV5tR5kEVDVNCHAQcXFgAUx4pFclNVgo1WWAdN1SYNX8tphTABCGsCRzBEAiB8Q+A6
dep+Rz92vhy26lT0AjZn4PRLi8Bf9qoB/CMk0wIgP/
Rj2PWZ3gEjUkTlhDRNAQ0gXwTO7t9n+V14pZ6oljUBIQMVmsAaoNWHVMS02LfTSe0e388LNitPa1UQZyOihY+FFgABABYAFE
b2Giu6c4KO5YW0pfw3lGp9jMUUAAA=

payjoin proposal

cHNidP8BAJwCAAAAAo8nutGgJdyYGXWiBEb45Hoe9lWGbkxh/6bNiOJdCDuDAAAAAAD+////
jye60aAl3JgZdaIERvjkeh72VYZuTGH/ps2I4l0IO4MBAAAAAP7///8CJpW4BQAAAAAXqRQd6EnwadJ0FQ46/
q6NcutaawlEMIcACT0AAAAABepFHdAltvPSGdDwi9DR+m0af6+i2d6h9MAAAAAAQEgqBvXBQAAAAAXqRTeTh6QYcpZE1sDW
tXm1HmQRUNU0IcAAQEggIQeAAAAAAAXqRTI8sv5ymFHLIjkZNRrNXSEXZHY1YcBBxcWABRfgGZV5ZJMkgTC1RvlOU9L+e2iE
AEIawJHMEQCIGe7e0DfJaVPRYEKWxddL2Pr0G37BoKz0lyNa02O2/
tWAiB7ZVgBoF4s8MHocYWWmo4Q1cyV2wl7MX0azlqa8NBENAEhAmXWPPW0G3yE3HajBOb7gO7iKzHSmZ0o0w0iONowcV+tAA
AA

payjoin proposal filled with sender's information

cHNidP8BAJwCAAAAAo8nutGgJdyYGXWiBEb45Hoe9lWGbkxh/6bNiOJdCDuDAAAAAAD+////
jye60aAl3JgZdaIERvjkeh72VYZuTGH/ps2I4l0IO4MBAAAAAP7///8CJpW4BQAAAAAXqRQd6EnwadJ0FQ46/
q6NcutaawlEMIcACT0AAAAABepFHdAltvPSGdDwi9DR+m0af6+i2d6h9MAAAAAAQEgqBvXBQAAAAAXqRTeTh6QYcpZE1sDW
tXm1HmQRUNU0IcBBBYAFMeKRXJTVYKNVlgHTdUmDV/LaYUwIgYDFZrAGqDVh1TEtNi300ntHt/
PCzYrT2tVEGcjooWPhRYYSFzWUDEAAIABAACAAAAAgAEAAAAAAAAAAAEBIICEHgAAAAAF6kUyPLL+cphRyyI5GTUazV0hF2
R2NWHAQcXFgAUX4BmVeWSTJIEwtUb5TlPS/
ntohABCGsCRzBEAiBnu3tA3yWlT0WBClsXXS9j69Bt+waCs9JcjWtNjtv7VgIge2VYAaBeLPDB6HGFlpqOENXMldsJezF9Gs
5amvDQRDQBIQJl1jz1tBt8hNx2owTm+4Du4isx0pmdKNMNIjjaMHFfrQABABYAFEb2Giu6c4KO5YW0pfw3lGp9jMUUIgICyg
vBWB5prpfx61y1HDAwo37kYP3YRJBvAjtunBAur3wYSFzWUDEAAIABAACAAAAAgAEAAAABAAAAAAA=
```

Implementations

- BlueWallet is in the process of implementing the protocol.
- BTCPay Server has implemented sender and receiver side of this protocol.
- Wasabi Wallet has merged sender's support.
- Join Market has implemented sender and receiver side of this protocol.
- JavaScript sender implementation.

Backward compatibility

The receivers advertise payjoin capabilities through BIP21's URI Scheme.

Senders not supporting payjoin will just ignore the `pj` variable and thus, will proceed to normal payment.

Special thanks

Special thanks to Kukks for developing the initial support to BTCPay Server, to junderw, AdamISZ, lukechilds, ncoelho, nopara73, lontivero, yahiheb, SomberNight, andrewkozlik, instagibbs, RHavar for all the feedback we received since our first implementation. Thanks again to RHavar who wrote the BIP79 Bustapay proposal, this gave a good starting point for our proposal.

```
BIP: 47
Layer: Applications
Title: Reusable Payment Codes for Hierarchical Deterministic Wallets
```

```
Authors: Justus Ranvier <justus@openbitcoinprivacyproject.org>
Status: Deployed
Type: Specification
Assigned: 2015-04-24
```

BIP 47

Status

This BIP can be considered final in terms of enabling compatibility with wallets that implement version 1 and version 2 reusable payment codes, however future developments of the reusable payment codes specification will not be distributed via the BIP process.

The Open Bitcoin Privacy Project RFC repo should be consulted for specifications related to version 3 or higher payment codes: https://github.com/OpenBitcoinPrivacyProject/rfc

Abstract

This BIP defines a technique for creating a payment code which can be publicly advertised and associated with a real-life identity without creating the loss of security or privacy inherent to P2PKH address reuse.

This BIP is a particular application of BIP43 and is intended to supplement HD wallets which implement BIP44.

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

Motivation

Payment codes add identity information to transactions which is useful in a merchant-customer interaction while protecting the privacy of users. Payment codes provide the privacy benefits of Darkwallet-style Stealth Addresses to SPV clients without requiring the assistance of a trusted full node and while greatly reducing reliance on blockchain storage.

Path levels

We define the following 3 levels in BIP32 path:

```
m / purpose' / coin_type' / identity'
```

The child keys derived from an identity are used in different ways:

```
m / purpose' / coin_type' / identity' / 0
```

The 0th (non-hardened) child is the notification key.

```
m / purpose' / coin_type' / identity' / 0 through 2147483647
```

These (non-hardened) keypairs are used for ECDH to generate deposit addresses.

```
m / purpose' / coin_type' / identity' / 0' through 2147483647'
```

These (hardened) keypairs are ephemeral payment codes.

Apostrophe in the path indicates that BIP32 hardened derivation is used.

Each level has a special meaning, described in the chapters below.

Purpose

Purpose is a constant set to 47′ (or 0x8000002F) following the BIP43 recommendation. It indicates that the subtree of this node is used according to this specification.

Coin type

The coin type field is identical to the same field in BIP44

Hardened derivation is used at this level.

Identity

The identity derivation level produces an extended public key and its associated extended private key.

When the extended public key at this level is combined with the metadata specified in the Representation section below, the resulting entity is called a "payment code."

This derivation level is equivalent to the Account level in BIP-44. Wallets SHOULD treat payment codes as intrinsically part of the BIP-44 account at the same index and create payment codes and accounts as pairs.

For example, the payment code created represented by (m / 47′ / 0′ / 0′) is part of the account represented by (m / 44′ / 0′ / 0′).

The second account in a wallet consists of the new account/payment code pair created by using an index of 1 in as the account/identity level of both paths.

Incoming payments received via this specification are equivalent to payments received to BIP-44 addresses, and unspent outputs from both types of addresses can be used as inputs in the same outgoing transaction.

Hardened derivation is used at this level.

Except where noted, all keys derived from a payment code use the public derivation method.

Versions

Payment codes contain a version byte which identifies a specific set of behavior.

Unless otherwise specified, payment codes of different versions are interoperable. If Alice uses a version x payment code, and Bob uses a version y payment code, they can still send and receive transactions between each other.

Currently specified versions:

- Version 1
    - Address type: P2PKH
    - Notification type: address
- Version 2
    - Address type: P2PKH
    - Notification type: bloom-multisig

Recommended Versions

- Wallets which have bloom filtering capabilities SHOULD create version 2 payment codes instead of version 1 payment codes.
- Version 1 payment codes are only recommended for wallets which lack access to bloom filtering capability.

Version 1

Representation

Binary Serialization

A payment code contains the following elements:

- Byte 0: version. required value: 0x01
- Byte 1: features bit field. All bits must be zero except where specified elsewhere in this specification
    - Bit 0: Bitmessage notification
    - Bits 1-7: reserved
- Byte 2: sign. required value: 0x02 or 0x03
- Bytes 3 - 34: x value, must be a member of the secp256k1 group
- Bytes 35 - 66: chain code
- Bytes 67 - 79: reserved for future expansion, zero-filled unless otherwise noted

Base58 Serialization

When a payment code is presented to the user, it SHOULD be presented encoded in Base58Check form.

- The version byte is: 0x47 (produces a “P” as the first character of the serialized form)
- The payload is the binary serialization of the payment code

Protocol

In the following examples, Alice and Bob are identities with a corresponding payment codes. Alice initiates a Bitcoin transaction, and Bob is the recipient of the transaction.

It is assumed that Alice can easily obtain Bob’s payment code via a suitable method outside the scope of the payment code protocol.

Definitions

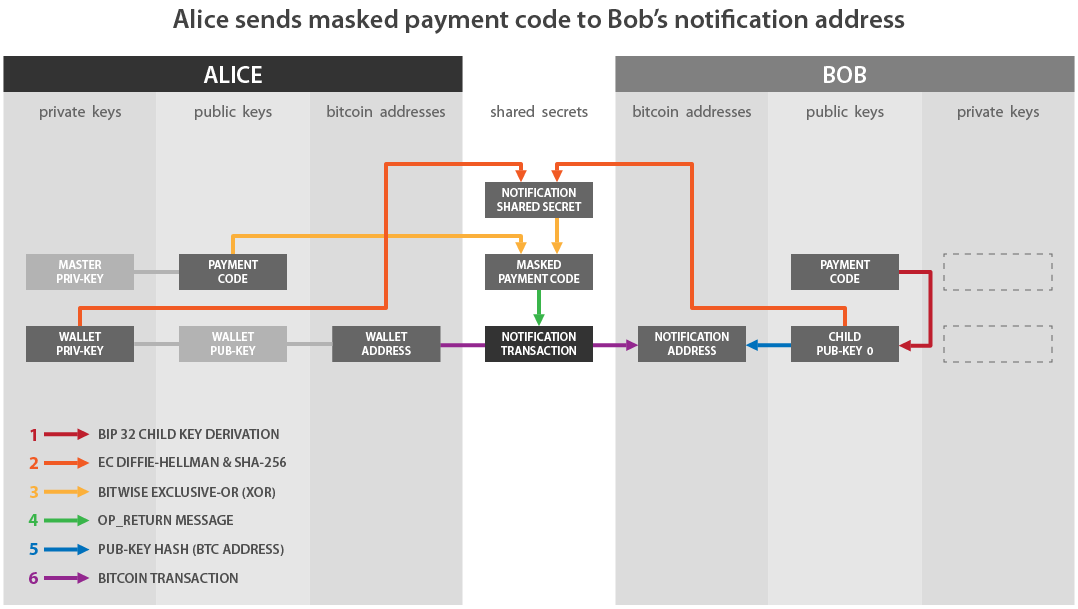
- Payment code: an extended public key and associated metadata which is associated with a particular identity/account
- Notification address: the P2PKH address associated with the 0th public key derived from a payment code
- Notification transaction: a transaction which sends an output to a notification address which includes an embedded payment code
- Designated input: the first input in the notification transaction which exposes an secp256k1 pubkey in either its signature script, or in the redeem script or pubkey script of the output being spent
- Designated pubkey: the first secp256k1 pubkey pushed to the stack during script execution for the designated input
- Outpoint: the specific output of a previous transaction which is being spent. See the Reference section for the binary serialization

Notification Transaction

Prior to the first time Alice initiates a transaction to Bob, Alice MUST inform Bob of her payment code via the following procedure:

Note: this procedure is used if Bob uses a version 1 payment code (regardless of the version of Alice's payment code). If Bob's payment code is not version 1, see the appropriate section in this specification.

1. Alice constructs a transaction which sends a small quantity of bitcoins to Bob's notification address (notification transaction)
    1. The inputs selected for this transaction MUST NOT be easily associated with Alice's notification address
2. Alice derives a unique shared secret using ECDH:
    1. Alice selects the private key corresponding to the designated pubkey: a
    2. Alice selects the public key associated with Bob's notification address: B, where B = bG
    3. Alice calculates a secret point: S = aB
    4. Alice calculates a 64 byte blinding factor: s = HMAC-SHA512(o, x)
        1. "x" is the x value of the secret point
        2. "o" is the outpoint being spent by the designated input
3. Alice serializes her payment code in binary form.
4. Alice renders her payment code (P) unreadable to anyone except Bob:
    1. Replace the x value with x': x' = x XOR (first 32 bytes of s)
    2. Replace the chain code with c': c' = c XOR (last 32 bytes of s)
5. Alice adds an OP_RETURN output to her transaction which consists of P.



bip-0047/reusable_payment_codes-01.png

1. Bob watches for any transactions which create an output at his notification address.
2. When a transaction is received, the client examines it to determine if it contains a standard OP_RETURN output with an 80 byte payload (notification transactions).
3. If the first byte of the payload in a notification transaction is 0x01:

    1. Bob selects the designated pubkey: A, where A = aG

    2. Bob selects the private key associated with his notification address: b

    3. Bob calculates a secret point: S = bA

    4. Bob calculates the blinding factor: s = HMAC-SHA512(x, o)

        1. "x" is the x value of the secret point
        2. "o" is the outpoint being spent by the designated input.

    5. Bob interprets the 80 byte payload as a payment code, except:

        1. Replace the x value with x': x' = x XOR (first 32 bytes of s)
        2. Replace the chain code with c': c' = c XOR (last 32 bytes of s)

    6. If the updated x value is a member of the secp256k1 group, the payment code is valid.

    7. If the updated x value is not a member of the secp256k1 group, the payment code is ignored.

Now that Bob's client has received Alice's payment code, it is possible for Alice to send payments (up to 2^{32} payments) to Bob.

Alice will never again need to send a notification transaction to Bob.

Bitcoins received via notification transactions require special handling in order to avoid privacy leaks:

1. The value of outputs received to notification addresses MUST NOT be displayed to the user as part of their spendable balance.
2. Outputs received to notification addresses MUST NOT be used as inputs for any transaction that involve ECDH calculations using any of the user's payment codes.
3. Outputs received to notification addresses MAY be passed through a mixing service before being added to the user's spendable balance.
4. Outputs received to notification addresses MAY be donated to miners using dust-b-gone or an equivalent procedure.

Standard Notification Transaction Scripts

Alice SHOULD use an input script in one of the following standard forms to expose a public key, and compliant applications SHOULD recognize all of these forms.

- P2PK (pay to pubkey)
- P2PKH (pay to pubkey hash)
- Multisig (bare multisig, without P2SH)
- a script which spends any of the above script forms via P2SH (pay to script hash)

Compatible wallets MAY provide a method for a user to manually specify the public key associated with a notification transaction in order to recover payment codes sent via non-standard notification transactions.

Post-Notification Privacy Considerations

Incautious handling of change outputs from notification transactions may cause unintended loss of privacy.

The recipient of a transaction which spends a change output from a prior notification transaction will learn about the potential connection between the sender and the recipient of the notification transaction.

The following actions are recommended to reduce this risk:

- Wallets which support mixing SHOULD mix change outputs from notification transactions prior to spending them
- Wallets which do not support mixing MAY simulate mixing by creating a transaction which spends the change output to the next external BIP44 address

Sending

1. Each time Alice wants to initiate a transaction to Bob, Alice derives a unique P2PKH address for the transaction using ECDH as follows:
    1. Alice selects the 0th private key derived from her payment code: a
    2. Alice selects the next unused public key derived from Bob's payment code, starting from zero: B, where B = bG
        1. The "next unused" public key is based on an index specific to the Alice-Bob context, not global to either Alice or Bob

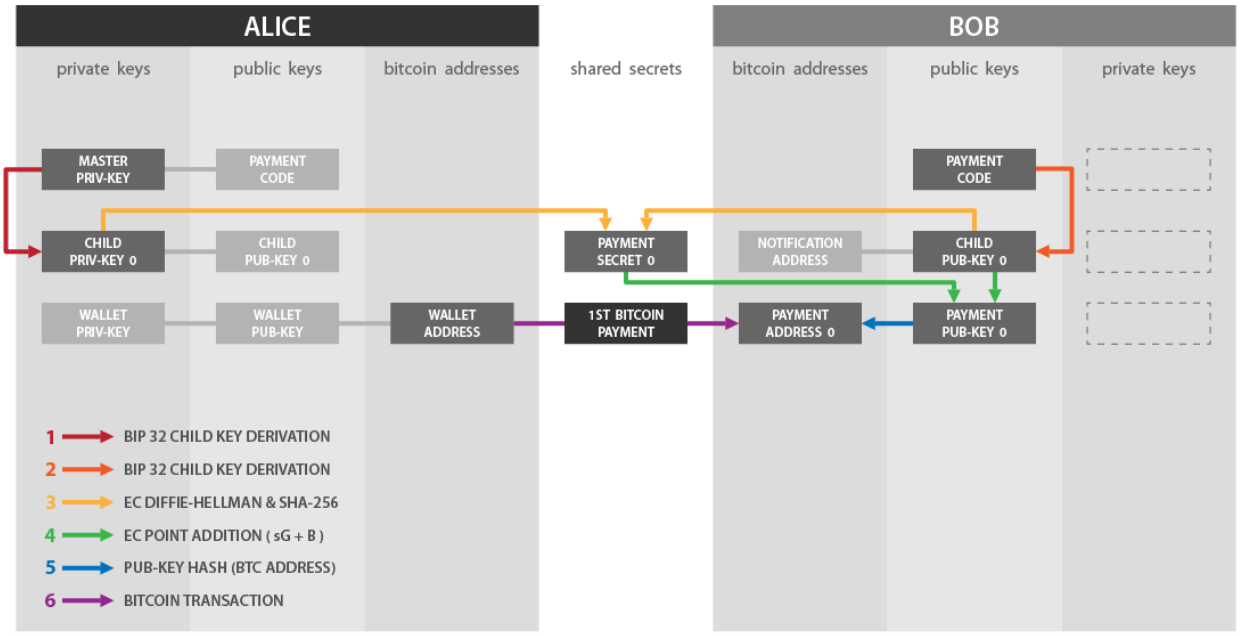
3. Alice calculates a secret point: S = aB

4. Alice calculates a scalar shared secret using the x value of S: s = SHA256(Sx)

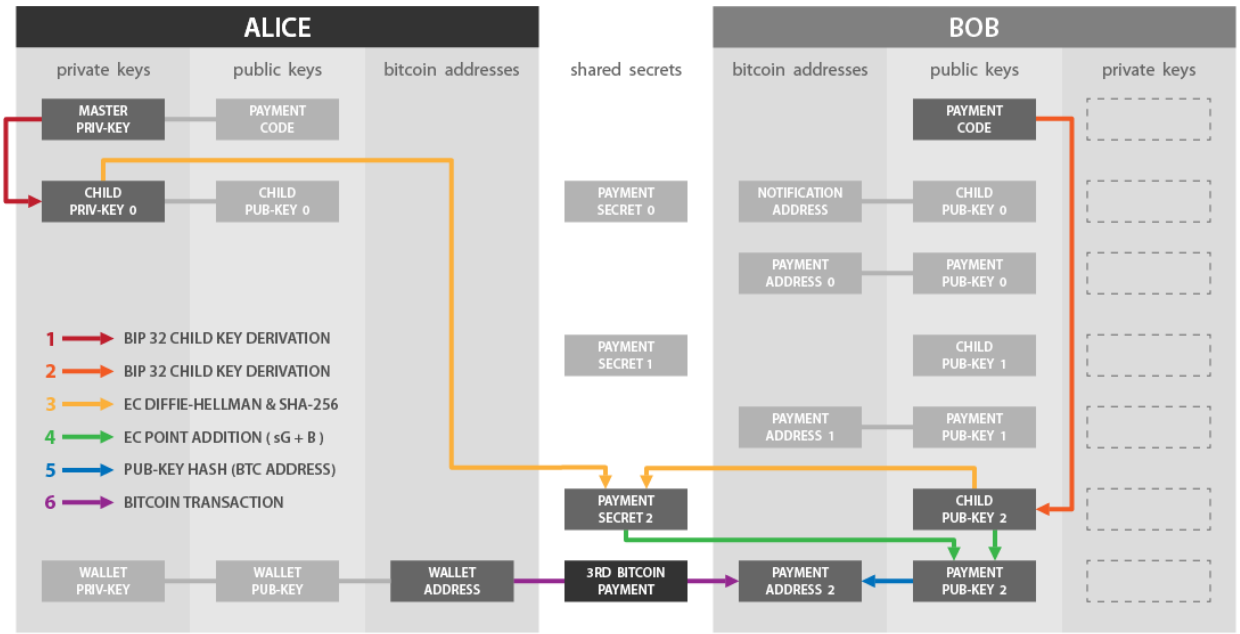
    1. If the value of s is not in the secp256k1 group, Alice MUST increment the index used to derive Bob's public key and try again.

5. Alice uses the scalar shared secret to calculate the ephemeral public key used to generate the P2PKH address for this transaction: B′ = B + sG

Alice sends bitcoins to Bob's first receiving address

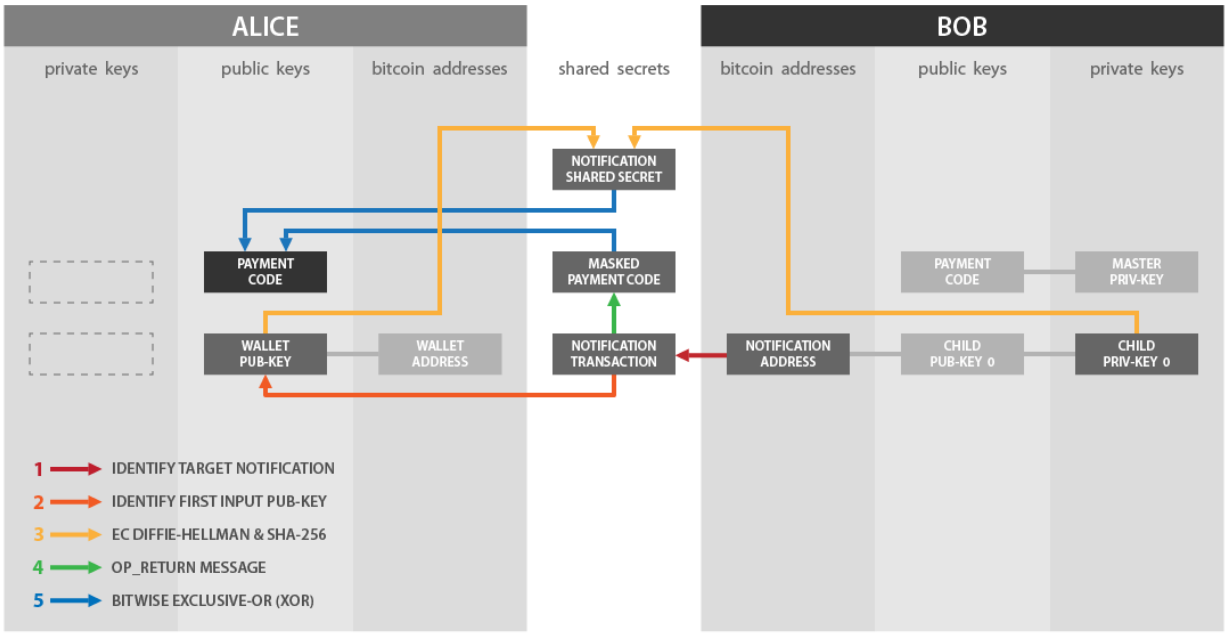


Alice sends bitcoins to Bob's third receiving address

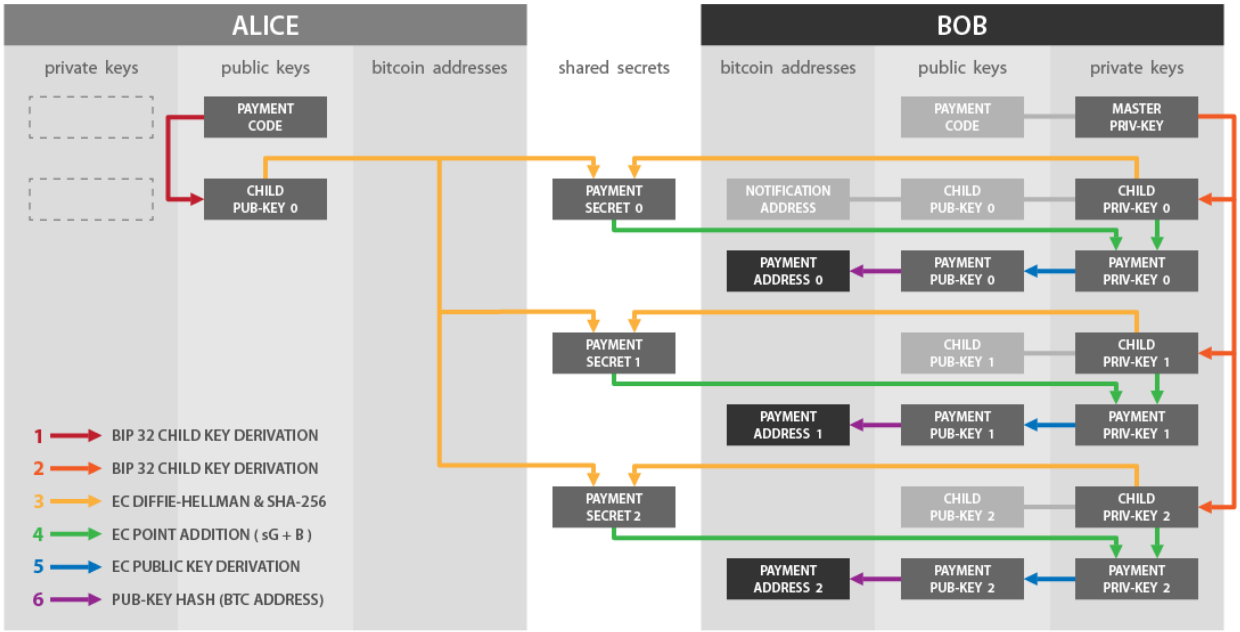


1. Bob is watching for incoming payments on B′ ever since he received the notification transaction from Alice.
    1. Bob calculates n shared secrets with Alice, using the 0th public key derived from Alice's payment code, and private keys 0 - n derived from Bob's payment code, where n is his desired lookahead window.
    2. Bob calculates the ephemeral deposit addresses using the same procedure as Alice: $B' = B + sG$
    3. Bob calculate the private key for each ephemeral address as: $b' = b + s$

Bob calculates the shared secret and unmasks Alice's payment code



Bob generates three deposit addresses to watch for payments from Alice

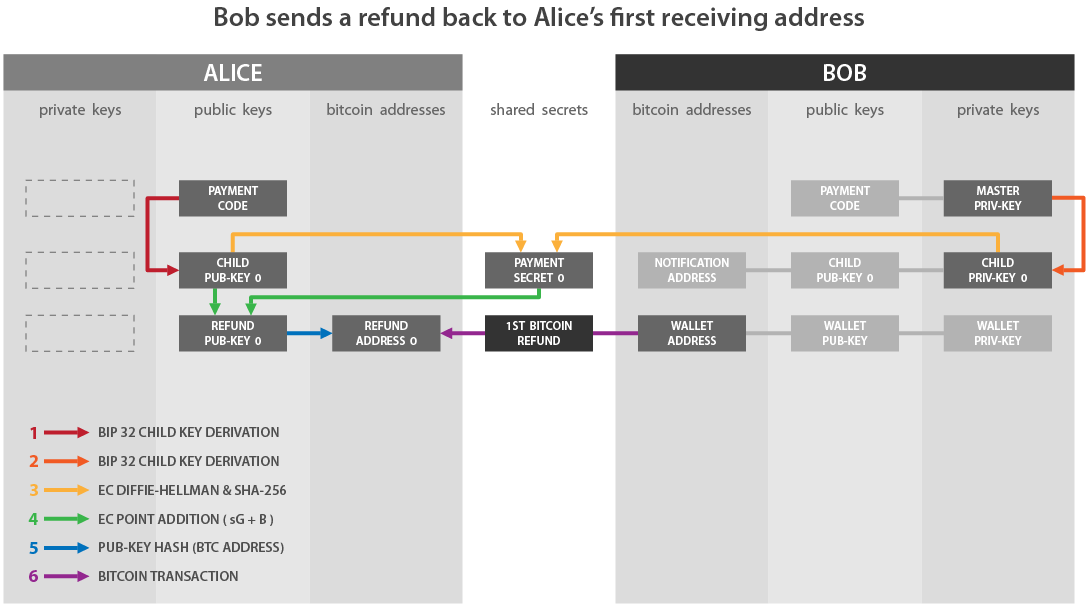


Refunds

Because Bob learns Alice's payment code as part of the process of receiving a payment, Bob has all the information he needs in order to send a refund to Alice.

A refund transaction is identical to a payment transactions, with only the roles of the participants switches.

Bob MUST send a notification transaction to Alice prior to the first time he sends funds to Alice, even if he has received transactions from her in the past.



bip-0047/reusable_payment_codes-06.png

Anonymous Payments

If Alice does not want her payment to Bob to be associated with her identity, she generates an ephemeral payment code to use for the transaction.

- Ephemeral payment codes are the hardened children of a payment code, starting from an index of zero.
- An ephemeral payment code SHOULD only be used for a single outgoing payment.
- The notification address of an ephemeral payment code MUST be monitored for notification transactions in order to detect incoming refund payments
- The correspondence between BIP44 accounts and ephemeral payment codes is 1:many

Cold Storage

- Unlike traditional watching-only wallets, those associated with payment codes held in cold storage can not detect incoming payments immediately.
- When the watching-only wallet detects an incoming notification transaction, it packages the transaction in an implementation-specific format suitable for transfer to the offline device.
- The offline device recovers the payment code, then pre-generates a large number of relevant keypairs (example: 10000) in order to minimize the need for air gap round trips.
- The offline device then packages the relevant public keys in an implementation-specific format suitable for transfer to the online device.
- The online device can then watch for incoming payments using a suitable lookahead window.
- If the lookahead window reaches the end of the pre-generated public keys, the user must generate more keys on the offline device and transfer them to the online device.

Wallet Recovery

Normal operation of a payment code-enabled wallet can be performed by an SPV client and does not require access to a complete copy of the blockchain.

Recovering a wallet from a seed, however, does require access to a fully-indexed blockchain.

The required data may be obtained from copy of the blockchain under the control of the user, or via a publicly-queryable blockchain explorer.

When querying a public blockchain explorer, wallets SHOULD connect to the explorer through Tor (or equivalent) and SHOULD avoid grouping queries in a manner that associates ephemeral addresses with each other.

Previously-spendable funds will generally not be lost or become inaccessible after a recovery from a seed, but all information regarding previous outgoing payments will be lost.

In order to recover received funds from a seed, the wallet must obtain every notification it has ever received to its notification address, including all spent transactions. It then re-establishes its lookahead window for each subchain by scanning every derived address sequentially until it locates a contiguous block of unused addresses of a user-specified length.

The metadata which a wallet must store to properly process outgoing transactions consists of:

1. A list of every payment code to which the identity has sent a notification transaction.
    1. This list is lost if a wallet must be recovered from a seed.
    2. The recovered wallet MUST send notification transactions as if it was a newly-created wallet
2. The index value corresponding to the next unused pubkey for each payment code on the previous list
    1. This value can be recovered by checking each ephemeral deposit address in sequence for transactions.
    2. Wallets MAY use a lookahead window capable of detecting gaps in the address sequence during this recovery operation.
3. The index value of the next unused ephemeral payment code.
    1. Recovering all incoming funds associated with ephemeral payment codes with 100% certainty requires exhausting the entire 2^{32} address space of potential ephemeral payment codes.
        1. In most cases, less than 100% certainty is acceptable as long as a fallback “deep scan” is available as an option to the user.
    2. The wallet checks the notification address for each ephemeral payment code for notification transactions in order to recover associated funds.
    3. Since most ephemeral payment codes will not receive a refund transaction wallets SHOULD use a large lookahead window for this recovery operation.
    4. The recovered value MUST be chosen as a number higher than any ephemeral payment code which has received a notification transaction.

Wallet Sharing

Wallets using payment codes generally should not be shared across multiple devices, given the need to synchronize metadata between each instance.

If wallets are shared between devices without a synchronization mechanism, undesirable address reuse can occur.

Wallets may perform an OPTIONAL check for existing transactions to an ephemeral deposit addresses prior to sending a transaction by checking a local copy of the blockchain or querying a public blockchain explorer via Tor or equivalent.

Alternate Notification Methods

In order to ensure that no funds will be lost in the event the recipient must recover their wallet from a seed, the sender MUST send a notification transaction the first time the sender interacts with a particular recipient.

A recipient MAY choose to designate alternate notification methods which the sender may use in addition to a notification transaction.

If the recipient specifies an alternate notification method, a compliant implementation MAY refrain from continually monitoring the notification address and SHOULD check the notification address periodically to detect payments sent by users who can not employ the alternate method.

A recipient specifies their preference for alternate notification by setting the appropriate bits in the feature byte of their payment code.

Bitmessage Notification

A recipient which prefers to receive notifications via Bitmessage indicates this preference by:

- Setting bit 0 of the features byte to 1
- Setting byte 67 of the serialized payment code to the desired Bitmessage address version
- Setting byte 68 of the serialized payment code to the desired Bitmessage stream number

The sender uses this information to construct a valid notification Bitmessage address:

1. Derive a Bitmessage signing key as: B = payment code / 0 / 0
2. Initialize a counter at 1: n
3. Derive a candidate encryption key as: B′ = payment code / 0 / n
4. If the combination of B and B` do not form a valid Bitmessage address, increment n by one and try again
5. Use the address version, signing key, encryption key, and stream number to construct a Bitmessage address per the Bitmessage protocol

The sender transmits their payment code in base58 form to the calculated Bitmessage address.

In order to use Bitmessage notification, the recipient must have a Bitmessage client which listens at the address which the senders will derive and is capable of relaying received payment codes to the Bitcoin wallet.

Version 2

Version 2 payment codes behave identifically to version 1 payment codes, except as modified below.

Representation

Binary Serialization

- Byte 0: version. required value: 0x02

Protocol

Definitions

- Notification change output: the change output from a notification transaction which resides in the sender's wallet, but can be automatically located by the intended recipient
- Payment code identifier: a 33 byte representation of a payment code constructed by prepending 0x02 to the SHA256 hash of the binary serialization of the payment code

Notification Transaction

Note: this procedure is used if Bob uses a version 2 payment code (regardless of the version of Alice's payment code). If Bob's payment code is not version 2, see the appropriate section in this specification.

1. Construct a notification transaction as per the version 1 instructions, except do not create the output to Bob's notification address
2. Create a notification change address as follows:
    1. Obtain the pubkey corresponding to the next change address
    2. Construct a multisig output in the form:

```
OP_1 <Bob's payment code identifier> <change address pubkey> OP_2 OP_CHECKMULTISIG
```

The relative ordering of the payment code identifier and change address pubkey in the above script MAY be randomized

Bob detects notification transactions by adding his payment code identifier to his bloom filter.

1. When the filter returns a notification transaction, the sender's payment code is unblinded using the same procedure as for version 1 notification transactions.

Alice's wallet should spend the notification change output at the next appropriate opportunity.

Test Vectors

- BIP47 Reusable Payment Codes Test Vectors

Reference

- BIP32 - Hierarchical Deterministic Wallets
- BIP43 - Purpose Field for Deterministic Wallets
- BIP44 - Multi-Account Hierarchy for Deterministic Wallets
- Outpoint
- dust-b-gone
- Base58Check encoding

- Bitmessage
- Mailing list discussion

Changelog

- 2021-02-15:
  - Finalize specification
- 2017-09-28:
  - Adjust text to match test vectors
- 2016-04-19:
  - Define version 2 payment codes

```
  BIP: 352
  Layer: Applications
  Title: Silent Payments
  Authors: josibake <josibake@protonmail.com>
           Ruben Somsen <rsomsen@gmail.com>
           Sebastian Falbesoner <sebastian.falbesoner@gmail.com>
  Status: Complete
  Type: Specification
  Assigned: 2023-03-09
  License: BSD-2-Clause
  Discussion: 2022-03-13: https://gist.github.com/RubenSomsen/c43b79517e7cb701ebf77eec6dbb46b8
[gist] Original proposal
              2022-03-28: https://gnusha.org/pi/bitcoindev/
CAPv7TjbXm953U2h+-12MfJ24YqOM5Kcq77_xFTjVK+R2nf-nYg@mail.gmail.com/ [bitcoin-dev] Silent
Payments - Non-interactive private payments with no on-chain overhead
              2022-10-11: https://gnusha.org/pi/bitcoindev/P_21MLHGJicZ-
hkbC4DGu86c5BtNKiH8spY4TOw5FJsfimdi_6VyHzU_y-s1mZsOcC2FA3EW_6w6W5qfV9dRK_7AvTAxDlwVfU-
yhWZPEuo=@protonmail.com/ [bitcoin-dev] Silent Payment v4 (coinjoin support added)
              2023-08-04: https://gnusha.org/pi/bitcoindev/ZM03twumu88V2NFH@petertodd.org/
[bitcoin-dev] BIP-352 Silent Payments addresses should have an expiration time
  Version: 1.1.2
  Requires: 340, 341, 350
```

BIP 352

Introduction

Abstract

This document specifies a protocol for static payment addresses in Bitcoin without on-chain linkability of payments or a need for on-chain notifications.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

Using a new address for each Bitcoin transaction is a crucial aspect of maintaining privacy. This often requires a secure interaction between sender and receiver, so that the receiver can hand out a fresh address, a batch of fresh addresses, or a method for the sender to generate addresses on-demand, such as an xpub.

However, interaction is often infeasible and in many cases undesirable. To solve for this, various protocols have been proposed which use a static payment address and notifications sent via the blockchain**Why not use out-of-band notifications** Out-of-band notifications (e.g. using something other than the Bitcoin blockchain) have been proposed as a way of addressing the privacy and cost concerns of using the Bitcoin blockchain as a messaging layer. This, however, simply moves the privacy and cost concerns somewhere else and increases the risk of losing money due to a notification not being reliably delivered, or even censored, and makes this notification data critical for backup to recover funds.. These protocols eliminate the need for interaction, but at the expense of increased costs for one-time payments and a noticeable footprint in the blockchain, potentially revealing metadata about the sender and receiver. Notification schemes also allow the receiver to link all payments from the same sender, compromising sender privacy.

This proposal aims to address the limitations of these current approaches by presenting a solution that eliminates the need for interaction, eliminates the need for notifications, and protects both sender and receiver privacy. These benefits come at the cost of requiring wallets to scan the blockchain in order to detect payments. This added requirement is generally feasible for full nodes but poses a challenge for light clients. While it is possible today to implement a privacy-preserving light client at the cost of increased bandwidth, light client support is considered an area of open research (see Appendix A: Light Client Support).

The design keeps collaborative transactions such as CoinJoins and inputs with MuSig and FROST keys in mind, but it is recommended that the keys of all inputs of a transaction belong to the same entity as there is no formal proof that the protocol is secure in a collaborative setting.

Goals

We aim to present a protocol which satisfies the following properties:

- No increase in the size or cost of transactions
- Resulting transactions blend in with other bitcoin transactions and can’t be distinguished
- Transactions can’t be linked to a silent payment address by an outside observer
- No sender-receiver interaction required
- No linking of multiple payments to the same sender
- Each silent payment goes to a unique address, avoiding accidental address reuse
- Supports payment labeling
- Uses existing seed phrase or descriptor methods for backup and recovery
- Separates scanning and spending responsibilities
- Compatible with other spending protocols, such as CoinJoin
- Light client/SPV wallet support
- Protocol is upgradeable

Overview

We first present an informal overview of the protocol. In what follows, uppercase letters represent public keys, lowercase letters represent private keys, || refers to byte concatenation, · refers to elliptic curve scalar multiplication, G represents the generator point for secp256k1, and n represents the curve order for secp256k1. Each section of the overview is incomplete on its own and is meant to build on the previous section in order to introduce and briefly explain each aspect of the protocol. For the full protocol specification, see Specification.

Simple case

Bob publishes a public key B as a silent payment address. Alice discovers Bob's silent payment address, selects a UTXO with private key a, public key A and creates a destination output P for Bob in the following manner:

- Let $P = B + hash(a \cdot B) \cdot G$
- Encode P as a BIP341 taproot output

Since $a \cdot B == b \cdot A$ (Elliptic-curve Diffie–Hellman), Bob scans with his private key b by collecting the input public keys for each transaction with at least one unspent taproot output and performing the ECDH calculation until P is found (i.e. calculating $P = B + hash(b \cdot A) \cdot G$ and seeing that P is present in the transaction outputs).

Creating more than one output

In order to allow Alice to create more than one output for Bob**Why allow for more than one output?** Allowing Alice to break her payment to Bob into multiple amounts opens up a number of privacy improving techniques for Alice, making the transaction look like a CoinJoin or better hiding the change amount by splitting both the payment and change outputs into multiple amounts. It also allows for Alice and Carol to both have their own unique output paying Bob in the event they are in a collaborative transaction and both paying Bob's silent payment address., we include an integer in the following manner:

- Let $k = 0$
- Let $P_0 = B + hash(a \cdot B \,||\, k) \cdot G$
- For additional outputs:
    - Increment k by one (k++)
    - Let $P_k = B + hash(a \cdot B \,||\, k) \cdot G$

Bob detects this output the same as before by searching for $P_0 = B + hash(b \cdot A \,||\, 0) \cdot G$. Once he detects the first output, he must:

- Check for $P_1 = B + hash(b \cdot A \,||\, 1) \cdot G$
- If P_1 is not found, stop
- If P_1 is found, continue to check for P_2 and so on until an additional output is not found

Since Bob will only perform these subsequent checks after a transaction with at least one output paying him is found, the increase to his overall scanning requirement is negligible. It should also be noted that the order in which these outputs appear in the transaction does not affect the outcome.

Preventing address reuse

If Alice were to use a different UTXO from the same public key A for a subsequent payment to Bob, she would end up deriving the same destinations P_k. To prevent this, Alice should include an input hash in the following manner:

- Let $input_hash = hash(outpoint \mid\mid A)$**Why include A in the input hash calculation?** By committing to A in input hash, this ensures that the sender cannot maliciously choose a private key a' in a subsequent transaction where $a' = input_hash \cdot a \,/\, input_hash'$, which would force address reuse in the protocol.
- Let $P_0 = B + hash(input_hash \cdot a \cdot B \mid\mid 0) \cdot G$

Bob must calculate the same *input_hash* when scanning.

Using all inputs

In our simplified example we have been referring to Alice's transactions as having only one input A, but in reality a Bitcoin transaction can have many inputs. Instead of requiring Alice to pick a particular input and requiring Bob to check each input separately, we can instead require Alice to perform the tweak with the sum of the input public keys**What about inputs without public keys?** Inputs without public keys can still be spent in the transaction but are simply ignored in the silent payments protocol.. This significantly reduces Bob's scanning requirement, makes light client support more feasible**How does using all inputs help light clients?** If Alice uses a random input for the tweak, Bob necessarily has to have access to and check all transaction inputs, which requires performing an ECC multiplication per input. If instead Alice performs the tweak with the sum of the input public keys, Bob only needs the summed 33 byte public key per transaction and only does one ECC multiplication per transaction. Bob can then use BIP158 block filters to determine if any of the outputs exist in a block and thus avoids downloading transactions which don't belong to him. It is still an open question as to how Bob can source the 33 bytes per transaction in a trustless manner, see Appendix A: Light Client Support for more details., and protects Alice's privacy in collaborative transaction protocols such as CoinJoin**Why does using all inputs matter for CoinJoin?** If Alice uses a random input to create the output for Bob, this necessarily reveals to Bob which input Alice has control of. If Alice is paying Bob as part of a CoinJoin, this would reveal which input belongs to her, degrading the anonymity set of the CoinJoin and giving Bob more information about Alice. If instead all inputs are used, Bob has no way of knowing which input(s) belong to Alice. This comes at the cost of increased complexity as the CoinJoin participants now need to coordinate to create the silent payment output and would need to use Blind Diffie–Hellman to prevent the other participants from learning who Alice is paying. Note it is currently not recommended to use this protocol for CoinJoins due to a lack of a formal security proof..

Alice performs the tweak with the sum of her input private keys in the following manner:

- Let $a = a_1 + a_2 + \ldots + a_n$
- Let $A = a \cdot G$
- Let $input_hash = hash(outpoint_L \mid\mid A)$, where $outpoint_L$ is the smallest outpoint lexicographically**Why use the lexicographically smallest outpoint for the hash?** Recall that the purpose of including the input hash is so that the sender and receiver can both come up with a deterministic nonce that ensures that a unique address is generated each time, even when reusing the same scriptPubKey as an input. Choosing the smallest outpoint lexicographically satisfies this requirement, while also ensuring that the generated output is not dependent on the final ordering of inputs in the transaction. Using a single outpoint also works well with memory constrained devices (such as hardware signing devices) as it does not require the device to have the entire transaction in memory in order to generate the silent payment output.
- Let $P_0 = B + hash(input_hash \cdot a \cdot B \mid\mid 0) \cdot G$

Spend and Scan Key

Since Bob needs his private key b to check for incoming payments, this requires b to be exposed to an online device. To minimize the risks involved, Bob can instead publish an address of the form (B_{scan}, B_{spend}). This allows Bob to keep b_{spend} in offline cold storage and perform the scanning with the public key B_{spend} and private key b_{scan}. Alice performs the tweak using both of Bob's public keys in the following manner:

- Let $P_0 = B_{spend} + hash(input_hash \cdot a \cdot B_{scan} \,||\, 0) \cdot G$

Bob detects this payment by calculating $P_0 = B_{spend} + hash(input_hash \cdot b_{scan} \cdot A \,||\, 0) \cdot G$ with his online device and can spend from his cold storage signing device using $(b_{spend} + hash(input_hash \cdot b_{scan} \cdot A \,||\, 0)) \bmod n$ as the private key.

Labels

For a single silent payment address of the form (B_{scan}, B_{spend}), Bob may wish to differentiate incoming payments. Naively, Bob could publish multiple silent payment addresses, but this would require him to scan for each one, which becomes prohibitively expensive. Instead, Bob can label his spend public key B_{spend} with an integer m in the following way:

- Let $B_m = B_{spend} + hash(b_{scan} \,||\, m) \cdot G$ where m is an incrementable integer starting from 1
- Publish (B_{scan}, B_1), (B_{scan}, B_2) etc.

Alice performs the tweak as before using one of the published (B_{scan}, B_m) pairs. Bob detects the labeled payment in the following manner:

- Let $P_0 = B_{spend} + hash(input_hash \cdot b_{scan} \cdot A \,||\, 0) \cdot G$
- Subtract P_0 from each of the transaction outputs and check if the remainder matches any of the labels ($hash(b_{scan} \,||\, 1) \cdot G$, $hash(b_{scan} \,||\, 2) \cdot G$ etc.) that the wallet has previously used

It is important to note that an outside observer can easily deduce that each published (B_{scan}, B_m) pair is owned by the same entity as each published address will have B_{scan} in common. As such, labels are not meant as a way for Bob to manage separate identities, but rather a way for Bob to determine the source of an incoming payment.

Labels for change

Bob can also use labels for managing his own change outputs. We reserve $m = 0$ for this use case. This gives Bob an alternative to using BIP32 for managing change, while still allowing him to know which of his unspent outputs were change when recovering his wallet from the master key. It is important that the wallet never hands out the label with $m = 0$ in order to ensure nobody else can create payments that are wrongly labeled as change.

While the use of labels is optional, every receiving silent payments wallet should at least scan for the change label when recovering from backup in order to ensure maximum cross-compatibility.

Specification

We use the following functions and conventions:

- *outpoint* (36 bytes): the `COutPoint` of an input (32-byte txid, least significant byte first || 4-byte vout, least significant byte first)**Why are outpoints little-endian?** Despite using big endian throughout the rest of the BIP, outpoints are sorted and hashed matching their transaction serialization, which is little-endian. This allows a wallet to parse a serialized transaction for use in silent payments without needing to re-order the bytes when computing the input hash. Note: despite outpoints being stored and serialized as little-endian, the transaction hash (txid) is always displayed as big-endian.
- $ser_{32}(i)$: serializes a 32-bit unsigned integer *i* as a 4-byte sequence, most significant byte first.
- $ser_{256}(p)$: serializes the integer p as a 32-byte sequence, most significant byte first.
- $ser_{P}(P)$: serializes the coordinate pair P = (x,y) as a byte sequence using SEC1's compressed form: (0x02 or 0x03) || $ser_{256}(x)$, where the header byte depends on the parity of the omitted Y coordinate.

For everything not defined above, we use the notation from BIP340. This includes the $hash_{tag}(x)$ notation to refer to *SHA256(SHA256(tag)* || *SHA256(tag)* || *x)*.

Versions

This document defines version 0 (*sp1q*). Version is communicated through the address in the same way as bech32 addresses (see BIP173. Future upgrades to silent payments will require a new version. As much as possible, future upgrades should support receiving from older wallets (e.g. a silent payments v0 wallet can send to both v0 and v1 addresses). Any changes that break compatibility with older silent payment versions should be a new BIP.

Future silent payments versions will use the following scheme:

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**Compatibility**
+0	q	p	z	r	y	9	x	8	backwards compatible
+8	g	f	2	t	v	d	w	0	
+16	s	3	j	n	5	4	k	h	
+24	c	e	6	m	u	a	7	-	

v31 (l) is reserved for a backwards incompatible change, if needed. For silent payments v0:

- If the receiver's silent payment address version is:
  - *v0*: check that the data part is exactly 66-bytes. Otherwise, fail
  - *v1* through *v30*: read the first 66-bytes of the data part and discard the remaining bytes
  - *v31*: fail
- Receiver addresses are always BIP341 taproot outputs**Why only taproot outputs?** Providing too much optionality for the protocol makes it difficult to implement and can be at odds with the goal of providing the best privacy. Limiting to taproot outputs helps simplify the implementation significantly while also putting users in the best eventual anonymity set.
- The sender should sign with one of the sighash flags *DEFAULT*, *ALL*, *SINGLE*, *NONE* (*ANYONECANPAY* is unsafe). It is strongly recommended implementations use *SIGHASH_ALL*

(*SIGHASH_DEFAULT* for taproot inputs) when possible**Why is it unsafe to use *SIGHASH_ANYONECANPAY*?** Since the output address for the receiver is derived from the sum of the Inputs For Shared Secret Derivation public keys, the inputs must not change once the sender has signed the transaction. If the inputs are allowed to change after the fact, the receiver will not be able to calculate the shared secret needed to find and spend the output. It is currently an open question on how a future version of silent payments could be made to work with new sighash flags such as *SIGHASH_GROUP* and *SIGHASH_ANYPREVOUT*.

- Inputs used to derive the shared secret are from the *Inputs For Shared Secret Derivation* list

Scanning silent payment eligible transactions

For silent payments v0 a transaction MUST be scanned if and only if all of the following are true:

- The transaction contains at least one BIP341 taproot output (note: spent transactions optionally can be skipped by only considering transactions with at least one unspent taproot output)
- The transaction has at least one input from the *Inputs For Shared Secret Derivation* list
- The transaction does not spend an output with SegWit version > 1**Why skip transactions that spend SegWit version > 1?** Skipping transactions that spend unknown output scripts allows us to have a clean upgrade path for silent payments by avoiding the need to scan the same transaction multiple times with different rule sets. If a new SegWit version is added in the future and silent payments v1 is released with support, we would want to avoid having to first scan the transaction with the silent payment v0 rules and then again with the silent payment v1 rules. Note: this restriction only applies to the inputs of a transaction.

Address encoding

A silent payment address is constructed in the following manner:

- Let B_{scan}, b_{scan} = *Receiver's scan public key and corresponding private key*
- Let B_{spend}, b_{spend} = *Receiver's spend public key and corresponding private key*
- Let $B_m = B_{spend} + hash_{BIP0352/Label}(ser_{256}(b_{scan}) \,||\, ser_{32}(m))\cdot G$, where $hash_{BIP0352/Label}(ser_{256}(b_{scan}) \,||\, ser_{32}(m))\cdot G$ is an optional integer tweak for labeling
    - If no label is applied then $B_m = B_{spend}$
- The final address is a Bech32m encoding of:
    - The human-readable part "sp" for mainnet, "tsp" for testnets (e.g. signet, testnet)
    - The data-part values:
        - The character "q", to represent a silent payment address of version 0
        - The 66-byte concatenation of the receiver's public keys, $ser_P(B_{scan}) \,||\, ser_P(B_m)$

Note: BIP173 imposes a 90 character limit for Bech32 segwit addresses and limits versions to 0 through 16, whereas a silent payment address requires *at least* 116 characters **Why do silent payment addresses need at least 116 characters?** A silent payment address is a bech32m encoding comprised of the following parts:

- HRP [2-3 characters]
- separator [1 character]
- version [1-2 characters]
- payload, 66 bytes concatenated pubkeys [ceil(66*8/5) = 106 characters]

- checksum [6 characters]

For a silent payments v0 address, this results in a 116-character address when using the 2-character mainnet HRP ("sp"), or a 117-character address when using the 3-character testnet HRP ("tsp"). Future versions of silent payment addresses may add to the payload, which is why a 1023-character limit is suggested. and allows versions up to 31. Additionally, since higher versions may add to the data field, it is recommended implementations use a limit of 1023 characters (see BIP173: Checksum design for more details).

Inputs For Shared Secret Derivation

While any UTXO with known output scripts can be used to fund the transaction, the sender and receiver MUST use inputs from the following list when deriving the shared secret:

- *P2TR*
- *P2WPKH*
- *P2SH-P2WPKH*
- *P2PKH*

Inputs with conditional branches or multiple public keys (e.g. *CHECKMULTISIG*) are excluded from shared secret derivation as this introduces malleability and would allow a sender to re-sign with a different set of public keys after the silent payment output has been derived. This is not a concern when the sender controls all of the inputs, but is an issue for CoinJoins and other collaborative protocols, where a malicious participant can participate in deriving the silent payment address with one set of keys and then re-broadcast the transaction with signatures for a different set of public keys. P2TR can have hidden conditional branches (script path), but we work around this by using only the output public key.

For all of the output types listed, only X-only and compressed public keys are permitted**Why only compressed public keys** Uncompressed and hybrid public keys are less common than compressed keys and generally considered to be a bad idea due to their blockspace inefficiency. Additionally, BIP143 recommends restricting P2WPKH inputs to compressed keys as a default policy..

P2TR

Keypath spend

```
witness:      <signature>
scriptSig:    (empty)
scriptPubKey: 1 <32-byte-x-only-key>
              (0x5120{32-byte-x-only-key})
```

The sender uses the private key corresponding to the taproot output key (i.e. the tweaked private key). This can be a single private key or an aggregate key (e.g. taproot outputs using MuSig or FROST)**Are key aggregation techniques like FROST and MuSig supported?** While we do not recommend it due to lack of a security proof (except if all participants are trusted or are the same entity), any taproot output able to do a key path theoretically is supported. Any offline key aggregation technique can be used, such as FROST or MuSig. This would require participants to perform the ECDH step collaboratively e.g. $ECDH = a_1 \cdot B_{scan} + a_2 \cdot B_{scan} + \ldots + a_t \cdot B_{scan}$ and $P = B_{spend} + hash(input_hash \cdot ECDH \mid\mid 0) \cdot G$. Additionally, it may be necessary for the participants to provide a DLEQ proof to ensure they are not acting maliciously.. The receiver obtains the public key from the *scriptPubKey* (i.e. the taproot output key).

Script path spend

```
witness:      <optional witness items> <leaf script> <control block>
scriptSig:    (empty)
scriptPubKey: 1 <32-byte-x-only-key>
              (0x5120{32-byte-x-only-key})
```

Same as a keypath spend, the sender MUST use the private key corresponding to the taproot output key. If this key is not available, the output cannot be included as an input to the transaction. Same as a keypath spend, the receiver obtains the public key from the *scriptPubKey* (i.e. the taproot output key)**Why not skip all taproot script path spends?** This causes malleability issues for CoinJoins. If the silent payments protocol skipped taproot script path spends, this would allow an attacker to join a CoinJoin round, participate in deriving the silent payment address using the tweaked private key for a key path spend, and then broadcast their own version of the transaction using the script path spend. If the receiver were to only consider key path spends, they would skip the attacker's script path spend input when deriving the shared secret and not be able to find the funds. Additionally, there may be scenarios where the sender can perform ECDH with the key path private key but spends the output using the script path..

The one exception is script path spends that use NUMS point *H* as their internal key (where *H* is constructed by taking the hash of the standard uncompressed encoding of the secp256k1 base point *G* as X coordinate, see BIP341: Constructing and spending Taproot outputs for more details), in which case the input will be skipped for the purposes of shared secret derivation**Why skip outputs with H as the internal taproot key?** If use cases get popularized where the taproot key path cannot be used, these outputs can still be included without getting in the way of making a silent payment, provided they specifically use H as their internal taproot key.. The receiver determines whether or not to skip the input by checking in the control block if the taproot internal key is equal to *H*.

P2WPKH

```
witness:      <signature> <33-byte-compressed-key>
scriptSig:    (empty)
scriptPubKey: 0 <20-byte-key-hash>
              (0x0014{20-byte-key-hash})
```

The sender performs the tweak using the private key for the output and the receiver obtains the public key as the last witness item.

P2SH-P2WPKH

```
witness:      <signature> <33-byte-compressed-key>
scriptSig:    <0 <20-byte-key-hash>>
              (0x160014{20-byte-key-hash})
scriptPubKey: HASH160 <20-byte-script-hash> EQUAL
              (0xA914{20-byte-script-hash}87)
```

The sender performs the tweak using the private key for the nested *P2WPKH* output and the receiver obtains the public key as the last witness item.

P2PKH

```
scriptSig:    <signature> <33-byte-compressed-key>
scriptPubKey: OP_DUP HASH160 <20-byte-key-hash> OP_EQUALVERIFY OP_CHECKSIG
              (0x76A914{20-byte-key-hash}88AC)
```

The receiver obtains the public key from the *scriptSig*. The receiver MUST parse the *scriptSig* for the public key, even if the *scriptSig* does not match the template specified (e.g. `<dummy> OP_DROP <Signature> <Public Key>`). This is to address the third-party malleability of *P2PKH scriptSigs*.

Sender

Selecting inputs

The sending wallet performs coin selection as usual with the following restrictions:

- At least one input MUST be from the *Inputs For Shared Secret Derivation* list
- Exclude inputs with SegWit version > 1 (see *Scanning silent payment eligible transactions*)
- For each taproot output spent the sending wallet MUST have access to the private key corresponding to the taproot output key, unless H is used as the internal public key

Creating outputs

After the inputs have been selected, the sender can create one or more outputs for one or more silent payment addresses in the following manner:

- Collect the private keys for each input from the *Inputs For Shared Secret Derivation* list
- For each private key a_i corresponding to a BIP341 taproot output, check that the private key produces a point with an even Y coordinate and negate the private key if not**Why do taproot private keys need to be checked?** Recall from BIP340 that each X-only public key has two corresponding private keys, d and $n - d$. To maintain parity between sender and receiver, it is necessary to use the private key corresponding to the even Y coordinate when performing the ECDH step since the receiver will assume the even Y coordinate when summing the taproot X-only public keys.
- Let $a = a_1 + a_2 + \ldots + a_n$, where each a_i has been negated if necessary
    - If $a = 0$, fail
- Let $input_hash = hash_{BIP0352/Inputs}(outpoint_L \mid\mid A)$, where $outpoint_L$ is the smallest *outpoint* lexicographically used in the transaction and $A = a \cdot G$
    - If *input_hash* is not a valid scalar, i.e., if $input_hash = 0$ or *input_hash* is larger or equal to the secp256k1 group order, fail
- Group receiver silent payment addresses by B_{scan} (e.g. each group consists of one B_{scan} and one or more B_m)
- If any of the groups exceed the limit of K_{max} (=2323) silent payment addresses, fail.**Why is the size of groups (i.e. silent payment addresses sharing the same scan public key) limited by K_{max}?** An adversary could construct a block filled with a single transaction consisting of N=23255 outputs (that's the theoretical maximum under current consensus rules, w.r.t. the block weight limit) that all target the same entity, consisting of one large group. Without a limit on the group size, scanning such a block with the algorithm described in this document would have a complexity of $O(N^2)$ for that entity, taking several minutes on modern systems. By capping the group size at K_{max}, we reduce the inner loop iterations to K_{max}, thereby decreasing the worst-case block scanning complexity to $O(N \cdot K_{max})$. This cuts

down the scanning cost to the order of tens of seconds. The chosen value of $K_{max} = 2323$ represents the maximum number of P2TR outputs that can fit into a 100kvB transaction, meaning a transaction that adheres to the current standardness rules is guaranteed to be within the limit. This ensures flexibility and also mitigates potential fingerprinting issues.

- For each group:
  - Let $ecdh_shared_secret = input_hash \cdot a \cdot B_{scan}$
  - Let $k = 0$
  - For each B_m in the group:
    - Let $t_k = hash_{BIP0352/SharedSecret}(ser_P(ecdh_shared_secret) \,||\, ser_{32}(k))$
      - If t_k is not a valid scalar, i.e., if $t_k = 0$ or t_k is larger or equal to the secp256k1 group order, fail
    - Let $P_{mn} = B_m + t_k \cdot G$
    - Encode P_{mn} as a BIP341 taproot output
    - Optionally, repeat with k++ to create additional outputs for the current B_m
    - If no additional outputs are required, continue to the next B_m with k++**Why not re-use t_k when paying different labels to the same receiver?** If paying the same entity but to two separate labeled addresses in the same transaction without incrementing k, an outside observer could subtract the two output values and observe that this value is the same as the difference between two published silent payment addresses and learn who the recipient is.
  - Optionally, if the sending wallet implements receiving silent payments, it can create change outputs by sending to its own silent payment address using label $m = 0$, following the steps above

All generated outputs MUST be present in the final transaction. If an output P_i with $i < k$ is omitted, the receiver will not be able to find outputs P_j where $i < j <= k$.

Receiver

Key Derivation

Two keys are needed to create a silent payments address: the spend key and the scan key. To ensure compatibility, wallets MAY use BIP32 derivation with the following derivation paths for the spend and scan key. When using BIP32 derivation, wallet software MUST use hardened derivation**Why use BIP32 hardened derivation?** Using BIP32 derivation allows users to add silent payments to an existing master seed. It also ensures that a user's silent payment funds are recoverable in any BIP32/BIP43 compatible wallet. Using hardened derivation ensures that it is safe to export the scan private key without exposing the master key or spend private key. for both the spend and scan key.

A scan and spend key pair using BIP32 derivation are defined (taking inspiration from BIP44) in the following manner:

```
 scan_private_key: m / purpose' / coin_type' / account' / 1' / 0
spend_private_key: m / purpose' / coin_type' / account' / 0' / 0
```

`purpose` is a constant set to *352* following the BIP43 recommendation. Refer to BIP43 and BIP44 for more details.

Scanning

If each of the checks in *Scanning silent payment eligible transactions* passes, the receiving wallet must:

- Let $A = A_1 + A_2 + \ldots + A_n$, where each A_i is the public key of an input from the *Inputs For Shared Secret Derivation* list
    - If A is the point at infinity, skip the transaction
- Let $input_hash = hash_{BIP0352/Inputs}(outpoint_L \mid\mid A)$, where $outpoint_L$ is the smallest *outpoint* lexicographically used in the transaction
    - If *input_hash* is not a valid scalar, i.e., if $input_hash = 0$ or *input_hash* is larger or equal to the secp256k1 group order, fail
- Let $ecdh_shared_secret = input_hash \cdot b_{scan} \cdot A$
- Check for outputs:
    - Let *outputs_to_check* be the taproot output keys from all taproot outputs in the transaction (spent and unspent).
    - Starting with $k = 0$:
        - If $k == K_{max}$ (=2323), stop scanning.
        - Let $t_k = hash_{BIP0352/SharedSecret}(ser_P(ecdh_shared_secret) \mid\mid ser_{32}(k))$
            - If t_k is not a valid scalar, i.e., if $t_k = 0$ or t_k is larger or equal to the secp256k1 group order, fail
        - Compute $P_k = B_{spend} + t_k \cdot G$
        - For each *output* in *outputs_to_check*:
            - If P_k equals *output*:
                - Add P_k to the wallet
                - Remove *output* from *outputs_to_check* and rescan *outputs_to_check* with *k++*
            - Else, check for labels (always check for the change label, i.e. $hash_{BIP0352/Label}(ser_{256}(b_{scan}) \mid\mid ser_{32}(m))$ where $m = 0$)**Why precompute labels?** Precomputing the labels is not strictly necessary: a wallet could track the max number of labels it has used (call it M) and scan for labels by adding $hash(b_{scan} \mid\mid m) \cdot G$ to P_0 for each label m up to M and comparing to the transaction outputs. This is more performant than precomputing the labels and checking via subtraction in cases where the number of eligible outputs exceeds the number of labels in use. In practice this will mainly apply to users that choose never to use labels, or users that use a single label for generating silent payment change outputs. If using a large number of labels, the wallet would need to add all possible labels to each output. This ends up being $n \cdot M$ additions, where n is the number of outputs in the transaction and M is the number of labels in the wallet. By precomputing the labels, the wallet only needs to compute $hash(b_{scan} \mid\mid m) \cdot G$ once when creating the labeled address and can determine if a label was used via a lookup, rather than adding each label to each output.:
                - Compute $label = output - P_k$
                - Check if *label* exists in the list of labels used by the wallet
                - If a match is found:
                    - Add $P_k + label$ to the wallet
                    - Remove *output* from *outputs_to_check* and rescan *outputs_to_check* with *k++*

        - If a label is not found, negate *output* and check a second time**Why negate the output?** Unfortunately taproot outputs are X-only, meaning we don't know what the correct Y coordinate is. This causes this specific calculation to fail 50% of the time, so we need to repeat it with the other Y coordinate by negating the output.
    - If no matches are found, stop**WARNING:** The decision to continue scanning (increment k) must be based on whether a cryptographic match was found, not on whether the wallet considers the output spendable or relevant. A match that is later filtered by wallet policy (e.g. dust) must still trigger a rescan with k++; stopping early may cause subsequent outputs for the same sender to be missed.

Spending

Recall that a silent payment output is of the form $B_{spend} + t_k \cdot G + hash_{BIP0352/Label}(ser_{256}(b_{scan}) \mid\mid ser_{32}(m)) \cdot G$, where $hash_{BIP0352/Label}(ser_{256}(b_{scan}) \mid\mid ser_{32}(m)) \cdot G$ is an optional label. To spend a silent payment output:

- Let $d = (b_{spend} + t_k + hash_{BIP0352/Label}(ser_{256}(b_{scan}) \mid\mid ser_{32}(m))) \bmod n$, where $hash_{BIP0352/Label}(ser_{256}(b_{scan}) \mid\mid ser_{32}(m))$ is the optional label
- Spend the BIP341 output with the private key d

Backup and Recovery

Since each silent payment output address is derived independently, regular backups are recommended. When recovering from a backup, the wallet will need to scan since the last backup to detect new payments.

If using a seed/seed phrase only style backup, the user can recover the wallet's unspent outputs from the UTXO set (i.e. only scanning transactions with at least one unspent taproot output) and can recover the full wallet history by scanning the blockchain starting from the wallet birthday. If a wallet uses labels, this information SHOULD be included in the backup. If the user does not know whether labels were used, it is strongly recommended they always precompute and check a large number of labels (e.g. 100k labels) to use when re-scanning. This ensures that the wallet can recover all funds from only a seed/seed phrase backup. The change label should simply always be scanned for, even when no other labels were used. This ensures the use of a change label is not critical for backups and maximizes cross-compatibility.

Backward Compatibility

Silent payments introduces a new address format and protocol for sending and as such is not compatible with older wallet software or wallets which have not implemented the silent payments protocol.

Test Vectors

A collection of test vectors in JSON format is provided, along with a python reference implementation. It uses a vendored copy of the secp256k1lab library at version 1.0.0 (commit 44dc4bd893b8f03e621585e3bf255253e0e0fbfb).

Each test vector consists of a sending test case and corresponding receiving test case. This is to allow sending and receiving to be implemented separately. To ensure determinism while testing, sort the array of B_m by amount (see the reference implementation). Test cases use the following schema:

test_case

```
{
    "comment": "Comment describing the behavior being tested",
    "sending": [<array of sender test objects>],
    "receiving": [<array of recipient test objects>],
}
```

sender

```
{
    "given": {
        "vin": [<array of vin objects with an added field for the private key. These objects are structur
        "recipients": [<array of recipient objects, consisting of the address and its contained scan/
spend public keys each>
            {
                "address": <bech32m encoding representing a silent payment address>,
                "scan_pub_key": <hex encoded scan public key>,
                "spend_pub_key": <hex encoded spend public key>,
                "count": <optional integer for specifying the same recipient repeatedly (1 by default)>
            },
            ...
        ]
    },
    "expected": {
        "outputs": [<array of strings, where each string is a hex encoding of 32-byte X-
only public key; contains all possible output sets, test must match a subset of size `n_outputs`>],
        "n_outputs": <integer for the exact number of expected outputs>,
    },
}
```

recipient

```
{
    "given": {
        "vin": [<array of vin objects. These objects are structured to match the `vin` output field from
        "key_material": {
            "scan_priv_key": <hex encoded scan private key>,
            "spend_priv_key": <hex encoded spend private key>,
        }
        "labels": [<array of ints, representing labels the receiver has used>],
     },
     "expected": {
         "addresses": [<array of bech32m strings, one for the silent payment address and each labeled add
         "outputs": [<optional array of outputs with tweak and signature, tester must match this set (alt
             {
                 "priv_key_tweak": <hex encoded private key tweak data>,
                 "pub_key": <hex encoded X-only public key>,
```

```
                "signature": <hex encoded signature for the output (produced with spend_priv_key + priv_
            },
            ...
        ],
        "n_outputs": <optional integer for the number of expected found outputs (alternative to "outputs
    }
}
```

Wallets should include inputs not in the *Inputs For Shared Secret Derivation* list when testing to ensure that only inputs from the list are being used for shared secret derivation. Additionally, receiving wallets should include non-silent payment outputs for themselves in testing to ensure silent payments scanning does not interfere with regular outputs detection.

Functional tests

Below is a list of functional tests which should be included in sending and receiving implementations.

Sending

- Ensure taproot outputs are excluded during coin selection if the sender does not have access to the key path private key (unless using *H* as the taproot internal key)
- Ensure the silent payment address is re-derived if inputs are added or removed during RBF

Receiving

- Ensure the public key can be extracted from non-standard *P2PKH* scriptSigs
- Ensure taproot script path spends are included, using the taproot output key (unless *H* is used as the taproot internal key)
- Ensure the scanner can extract the public key from each of the input types supported (e.g. *P2WPKH*, *P2SH-P2WPKH*, etc.)

Appendix A: Light Client Support

This section proposes a few ideas for how light clients could support scanning for incoming silent payments (sending is fairly straightforward) in ways that preserve bandwidth and privacy. While this is out of scope for the current BIP, it is included to motivate further research into this topic. In this context, a light client refers to any bitcoin wallet client which does not process blocks and does not have a direct connection to a node which does process blocks (e.g. a full node). Based on this definition, clients that directly connect to a personal electrum server or a bitcoin node are not light clients.

This distinction makes the problem for light clients more clear: light clients need a way to source the necessary data for performing the tweaks and a way of determining if any of the generated outputs exist in a block.

Tweak Data

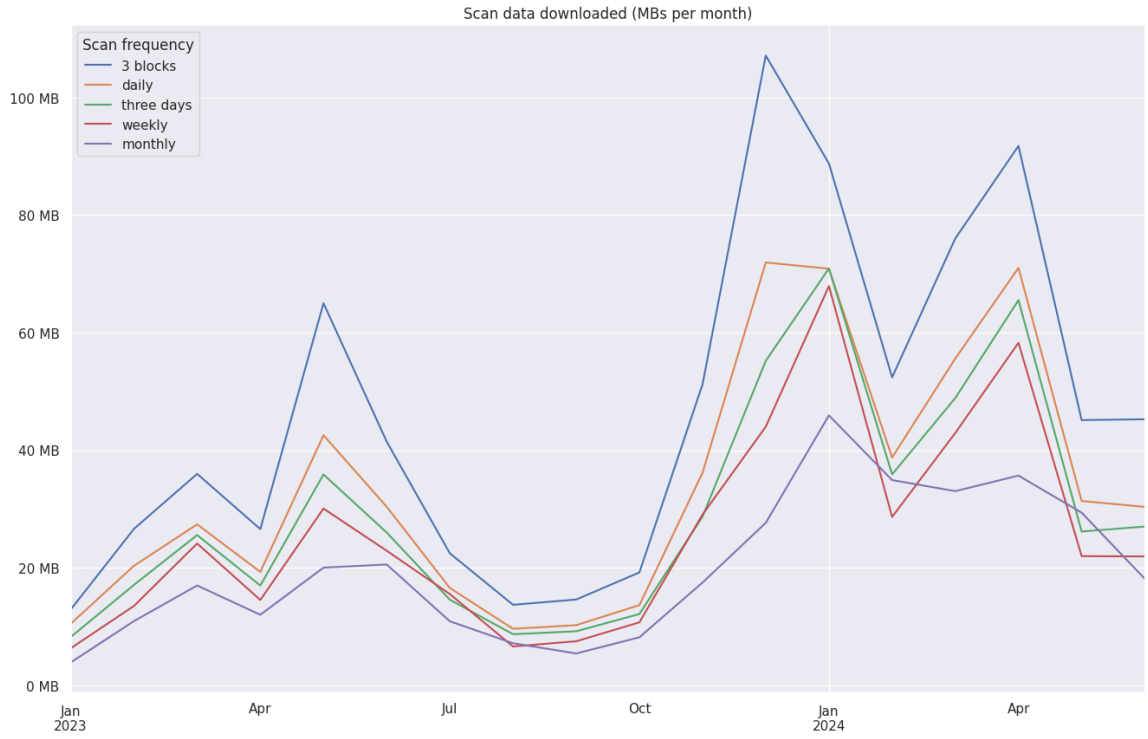
Recall that a silent payment eligible transaction follows certain conditions and should have at least one unspent taproot output. Full nodes (or any index server backed by a full node, such as electrum server) can build an index which collects all of the eligible public keys for a silent payments eligible transaction, sums

them up, multiplies the sum by the *input_hash*, and serves them to clients. This would be 33 bytes per silent payment eligible transaction.

For a typical bitcoin block of ~3500 txs, lets assume every transaction is a silent payments eligible transaction. This means a client would need to request *33 bytes * 3500* of data per block (roughly 100 kB per block). If a client were to request data for every block, this would amount to ~450 MB per month, assuming 100% taproot usage and all non-dust outputs remain unspent for > 1 month. As of today, these numbers are closer to 7–12 kB per block (30–50 MB per month)**Data for Appendix A** These numbers are based on data from January 2023 until July 2024. See Silent payments light client data for the full analysis..

Transaction cut-through

It is unlikely a light client would need to scan every block and as such can take advantage of transaction cut-through, depending on how often they choose to scan for new blocks. Empirically, ~75% of transactions with at least one non-dust unspent taproot output will have spent all non-dust taproot UTXOs in 150 blocks or less. This means a client that only scans once per day could *significantly* cut down on the number of blocks and the number of transactions per block that they need to request by only asking for data on transactions that were created since their last scan and that still have at least one non-dust unspent taproot output as of the current block height. Based on taproot adoption as of July 2024, a light client scanning once every 3 days would use roughly 30 MB per month.



bip-0352/scan_data_downloader_per_month.png

BIP158

Once a light client has the tweak data for a block, they can determine whether or not an output to them exists in the block using BIP158 block filters. Per BIP158, they would then request the entire block and add the transaction to their wallet, though it maybe be possible to only request the prevout txids and vouts for all

transactions with at least one taproot output, along with the scriptPubKeys and amounts. This would allow the client to download the necessary data for constructing a spending transaction, without downloading the entire block. How this affects the security assumptions of BIP158 is an open question.

Out-of-band notifications

Assuming a secure messaging protocol exists, the sender can send an encrypted (using the scan public key of the silent payment address) notification to the receiver with the following information:

- The spend public key (communicates the label)
- The shared secret portion of the private key (i.e *hash(ecdh_shared_secret || k)*)
- The outpoint and amount (so it's immediately spendable)

It is important to note that these notifications are not required. At any point, the receiver can fall back to scanning for silent payment transactions if they don't trust the notifications they are receiving, are being spammed with fake notifications, or if they are concerned that they are not receiving notifications.

A malicious notification could potentially cause the following issues:

- You did not actually receive money to the stated key
    - This can be probabilistically resolved by matching the key against the BIP158 block filters and assuming it's not a false positive, or fully resolved by downloading the block
- You received money but the outpoint or amount is incorrect, so attempts to spend it will fail or cause you to overpay fees
    - There doesn't seem to be much motivation for malicious senders to ever do this, but light clients need to take into account that this can occur and should ideally check for it by downloading the block
- The private key is correct but it wasn't actually derived using the silent payment protocol, causing recovery from back-up to fail (unsafe - no implementation should ever allow this)
    - This can be detected by downloading the tweak data of the corresponding block and should be resolved by immediately spending the output

Wallet designers can choose which tradeoffs they find appropriate. For example, a wallet could check the block filter to at least probabilistically confirm the likely existence of the UTXO, thus efficiently cutting down on spam. The payment could then be marked as unconfirmed until a scan is performed and the existence of the UTXO in accordance to the silent payment specification is verified.

Changelog

To help implementers understand updates to this document, we attach a version number that resembles *semantic versioning* (`MAJOR.MINOR.PATCH`). The `MAJOR` version is incremented if changes to the BIP are introduced that are incompatible with prior versions. The `MINOR` version is incremented whenever the inputs or the output of an algorithm changes in a backward-compatible way or new backward-compatible functionality is added. The `PATCH` version is incremented for other changes that are noteworthy (bug fixes, test vectors, important clarifications, etc.).

- **1.1.2** (2026-10-25):
    - Add test vector "Single recipient: taproot key-path spend with annex"

- **1.1.1** (2026-04-16):
  - Add test vector "Input keys intermediate sum is zero but final sum is non-zero"
- **1.1.0** (2026-03-02):
  - Introduce per-group recipient limit K_{max} to mitigate quadratic scanning behavior for adversarial transactions.
- **1.0.2** (2025-07-25):
  - Clarify how to handle the improbable corner case where the output of SHA256 is equal to 0 or greater than or equal to the secp256k1 curve order.
- **1.0.1** (2024-06-22):
  - Add steps to fail if private key sum is zero (for sender) or public key sum is point at infinity (for receiver), add corresponding test vectors.
- **1.0.0** (2024-05-08):
  - Initial version, merged as BIP-352.

Acknowledgements

This document is the result of many discussions and contains contributions by a number of people. The authors wish to thank all those who provided valuable feedback and reviews, including the participants of the BIP47 Prague discussion, the Advancing Bitcoin silent payments Workshop, and coredev. The authors would like to also thank w0xlt for writing the initial implementation of silent payments.

Rationale and References

```
  BIP: 353
  Layer: Applications
  Title: DNS Payment Instructions
  Authors: Matt Corallo <bip353@bluematt.me>
           Bastien Teinturier <bastien@acinq.fr>
  Status: Complete
  Type: Specification
  Assigned: 2024-02-10
  License: CC0-1.0
  Discussion: 2024-02-13: https://groups.google.com/g/bitcoindev/c/uATaflkYglQ [bitcoin-dev]
Mapping Human-Readable Names to Payment Instructions
```

BIP 353

Copyright

This BIP is licensed under the CC0-1.0 license.

Abstract

This BIP proposes a standard format for encoding BIP 21 URI schemes in DNS TXT records.

Motivation

Various Bitcoin and other cryptocurrency applications have developed human-readable names for payment instructions over time, with marketplace adoption signaling strong demand for it from users.

The DNS provides a standard, global, hierarchical namespace mapping human-readable labels to records of various forms. Using DNSSEC, the DNS provides cryptographic guarantees using a straightforward PKI which follows the hierarchical nature of the DNS, allowing for stateless and even offline validation of DNS records from a single trusted root.

Further, because DNS queries are generally proxied through ISP-provided or other resolvers, DNS queries usually do not directly expose the queryer's IP address. Further, because of the prevalence of open resolvers, the simplicity of the protocol, and broad availability of DNS recursive resolver implementations, finding a proxy for DNS records is trivial.

Thus, using TXT records to store Bitcoin payment instructions allows for human-readable Bitcoin payment destinations which can be trivially verified on hardware wallets and which can be resolved relatively privately.

Specification

General rules for handling

Bitcoin wallets MUST NOT prefer to use DNS-based resolving when methods with explicit public keys or addresses are available. In other words, if a standard Bitcoin address or direct BIP 21 URI is available or would suffice, Bitcoin wallets MUST prefer to use that instead.

Records

Payment instructions are indexed by both a user and a domain. Instructions for a given `user` and `domain` are stored at `user`.user._bitcoin-payment.`domain` in a single TXT RR.

The TXT RR's RDATA **MUST** consist of one or more DNS `<character-string>`s (see RFC 1035 §3.3.14), each ≤255 bytes.

All payment instructions MUST be DNSSEC-signed.

Payment instructions MAY resolve through CNAME or DNAME records as long as all such records and the ultimate records pointed to by them are DNSSEC signed.

User and domain names which are not expressible using standard printable ASCII MUST be encoded using the punycode IDN encoding defined in RFC 3492 and RFC 5891.

Note that because resolvers are not required to support resolving non-ASCII identifiers, wallets SHOULD avoid using non-ASCII identifiers.

For payment instructions that have a built-in expiry time (e.g. Lightning BOLT 12 offers), care must be taken to ensure that the DNS records expire prior to the expiry of the payment instructions. Otherwise, senders may have payment instructions cached locally which have expired, preventing payment.

Resolution

Clients resolving Bitcoin payment instructions MUST ignore any TXT records at the same label which do not begin with (ignoring case) "bitcoin:". Resolvers encountering multiple "bitcoin:%22-matching TXT records at the same label MUST treat the records as invalid and refuse to use any payment instructions therein.

Clients resolving Bitcoin payment instructions **MUST** reconstruct the `bitcoin:` URI by concatenating the TXT RR's `<character-string>` fields in **RDATA order**, without inserting separators, before parsing.DNS TXT RDATA consists of one or more length-prefixed strings with a maximum of 255 bytes of content; see RFC 1035 §3.3.14.

Clients **MUST NOT** concatenate across multiple TXT RRs at the same owner name.

Clients resolving Bitcoin payment instructions MUST fully validate DNSSEC signatures leading to the DNS root (including any relevant CNAME or DNAME records) and MUST NOT accept DNSSEC signatures which use SHA-1 or RSA with keys shorter than 1024 bits. Resolvers MAY accept SHA-1 DS records.

Clients resolving Bitcoin payment instructions MUST NOT trust a remote resolver to validate DNSSEC records on their behalf.

Clients resolving Bitcoin payment instructions MUST support resolving through CNAME or DNAME records.

Resolvers MAY support resolving non-ASCII user and domain identifiers. Resolvers which do support non-ASCII user and domain identifiers MUST take precautions to prevent homograph attacks and SHOULD consider denying paste functionality when entering non-ASCII identifiers. Wallets which do not take any such precautions MUST instead display non-ASCII user and domain identifiers using their raw punycode. As such, wallets SHOULD NOT create identifiers which are not entirely printable ASCII.

While clients MAY cache the payment instructions they receive from the DNS, clients MUST NOT cache the payment instructions received from the DNS for longer than the TTL provided by their DNS resolver, and further MUST NOT cache the payment instructions for longer than the lowest initial TTL (which is signed as a part of DNSSEC signatures) received in the full DNSSEC chain leading from the DNS root to the resolved TXT record.

Address Reuse

Payment instructions with on-chain addresses which will be re-used SHOULD be rotated as regularly as possible to reduce address reuse. Such payment instructions SHOULD also use a relatively short DNS TTL to ensure regular rotation takes effect quickly. In cases where this is not practical, payment instructions SHOULD NOT contain on-chain addresses (i.e. the URI path SHOULD be empty).

Payment instructions which do contain on-chain addresses which will be re-used SHOULD be rotated after any transaction to such an address is confirmed on-chain.

Display

When displaying a verified human-readable name, wallets SHOULD prefix it with ₿, i.e. ₿`user@domain`. They SHOULD parse recipient information in both `user@domain` and ₿`user@domain` forms and resolve such an entry into recipient information using the above record. For the avoidance of doubt, the ₿ is *not* included in the DNS label which is resolved.

Wallets providing the ability for users to "copy" their address information SHOULD copy the underlying URI directly, rather than the human-readable name. This avoids an additional DNS lookup by the application in which it is pasted. Wallets that nevertheless provide users the ability to copy their human-readable name, MUST include the ₿ prefix (i.e. copy it in the form ₿`user@domain`).

Wallets accepting payment information from external devices (e.g. hardware wallets) SHOULD accept RFC 9102-formatted proofs (as a series of unsorted `AuthenticationChain` records) and, if verification succeeds, SHOULD display the recipient in the form ₿`user@domain`.

PSBT types

Wallets accepting payment information from external devices (e.g. hardware wallets) MAY examine the following per-output PSBT fields to fetch RFC 9102-formatted proofs. Wallets creating PSBTs with recipient information derived from human-readable names SHOULD include the following fields.

When validating the contained proof, clients MUST enforce the inception on all contained RRSigs is no later than the current time and that the expiry of all RRSigs is no earlier than an hour in the past. Clients MAY allow for an expiry up to an hour in the past to allow for delays between PSBT construction and signing only if such a delay is likely to occur in their intended usecase.

Name	`<keytype>`	`<keydata>`	`<valuedata>`	`<valuedata>` Description	Versions Requiring Inclusion	Versions Requiring Exclusion	Versions Allowing Inclusion
BIP 353 DNSSEC proof	`PSBT_OUT_DNSSEC_PROOF = 0x35`	None	`<1-byte-length-prefixed BIP 353 human-readable name without the ₿ prefix><RFC 9102-formatted DNSSEC Proof>`	A BIP 353 human-readable name (without the ₿ prefix), prefixed by a 1-byte length. Followed by an RFC 9102 DNSSEC `AuthenticationChain` (i.e. a series of DNS Resource Records in no particular order) providing a DNSSEC proof to a BIP 353 DNS TXT record.			0, 2

Rationale

Display

There are several ways in which human-readable payment instructions could be displayed in wallets. In order to ensure compatibility with existing human-readable names schemes, @ is used as the separator between the `user` and `domain` parts. However, simply using the @ separator can lead to confusion between email addresses on a given domain and payment instructions on a domain. In order to somewhat reduce the incidence of such confusion, a ₿ prefix is used.

Rotation

On-chain addresses which are re-used (i.e. not including schemes like Silent Payments) need to be rotated to avoid contributing substantially to address reuse. However, rotating them on a timer or any time a transaction

enters the mempool could lead to substantial overhead from excess address generation. Instead, rotating addresses any time a transaction is confirmed on-chain ensures address rotation happens often while bounding the maximum number of addresses needed to one per block, which grows very slowly and will not generate an address set too large to handle while scanning the chain going forward.

Alternatives

There are many existing schemes to resolve human-readable names to cryptocurrency payment instructions. Sadly, these current schemes suffer from a myriad of drawbacks, including (a) lacking succinct proofs of namespace to public key mappings, (b) revealing sender IP addresses to recipients or other intermediaries as a side-effect of payment, (c) relying on the bloated TLS Certificate Authority infrastructure, or (d) lacking open access, not allowing anyone to create a namespace mapping.

DNS Rather than blockchain-based solutions

There are many blockchain-based alternatives to the DNS which feature better censorship-resistance and, in many cases, security. However, here we chose to use the standard ICANN-managed DNS namespace as many blockchain-based schemes suffer from (a), above (though in some cases this could be addressed with cryptographic SNARK schemes). Further, because they do not have simple client-side querying ability, many of these schemes use trusted intermediaries which resolve names on behalf of clients. This reintroduces drawbacks (b) and often (c) as well.

Finally, it is worth noting that none of the blockchain-based alternatives to the DNS have had material adoption outside of their specific silos, and committing Bitcoin wallets to rely on a separate system which doesn't see broad adoption may not be sustainable.

DNS Rather than HTTP-based solutions

HTTP(s)-based payment instruction resolution protocols suffer from drawbacks (a), (b), and (c), above, and generally shouldn't be considered a serious alternative for payment instruction resolution.

Private DNS Querying

While public recursive DNS resolvers are very common (e.g. 1.1.1.1, 8.8.8.8, and 9.9.9.9), using such resolvers directly (even after validating DNSSEC signatures) introduces drawback (b), at least in regard to a centralized intermediary. Resolving payment instructions recursively locally using a resolver on the same local network or in the paying application would instead introduce drawback (b) directly to the recipient, which may well be worse.

For payers not using VPN or other proxying technologies, they generally trust their ISP to not snoop on their DNS requests anyway, so using ISP-provided recursive DNS resolvers is likely the best option.

However, for the best privacy, payers are encouraged to perform DNS resolution over Tor or another VPN technology.

Lightning payers should consider utilizing DNS resolution over native onion messages, using the protocol described in BLIP 32

DNS Enumeration

In most cases where payments are accepted from any third-party, user enumeration is practical by simply attempting to send small value payments to a list of possible user names. However, storing all valid users in the DNS directly may make such enumeration marginally more practical. Thus, those wishing to avoid such enumeration should carefully ensure all DNS names return valid payment instructions. Note when doing so that wildcard records are identified as such by the DNSSEC RRSIG labels counter and are differentiable from non-wildcard records.

Backwards Compatibility

This work is intended to extend and subsume the existing "Lightning Address" scheme, which maps similar names (without the ₿ prefix) using HTTPS servers to Lightning BOLT 11 payment instructions. Wallets implementing this scheme MAY fall back to existing "Lightning Address" logic if DNS resolution fails but SHOULD NOT do so after this scheme is sufficiently broadly deployed to avoid leaking sender IP address information.

Examples

`matt@mattcorallo.com` resolves to

```
matt.user._bitcoin-payment.mattcorallo.com. 1800 IN TXT  "bitcoin:?
lno=lno1qsgr30k45jhvkfqxjqheaetac
u4guyxvqttftvqu0f5sneckep3lkwdut7mmhhpcyjmlmnjn4hze8ed7pq88xqkxt2dcw5mlxhz644fms82f7k4ymfxs2ehhp
jtxw
xly0w5k8xdtlvpqyd8xzdq4tq8lgupnueshgydr330lc3j5kdcqh54gt7jdg9n68j4eqqeu7ts8uh0qxamee6ndj37tc6mzg
ejth
vvjqj96p8dz2h"
"rsh5z5n27qfk6svjz5pmkh0smq26k0j2j4q36xgq3r5qzet9kuhq4lydpen5mskxgjdvs5faqgv8pmj7cfd7
ny84djksqpqk9ky6juc7fpezecxvg7sjx05dckyypnv9tmvfp6tkpehmtaqmvuupetxuzqf4t0azddjdcpasgw6hxuz9g"
```

- Note that `lno` indicates a value containing a lightning BOLT12 offer.
- Note that the complete URI is broken into two strings with maximum 255 characters each

Example proofs

The following are valid DNSSEC proofs to be included in the `PSBT_OUT_DNSSEC_PROOF` record in a PSBT.

This encodes address `bc1qztwy6xen3zdtt7z0vrgapmjtfz8acjkfp5fp7l` to `simple@dnssec_proof_tests.bitcoin.ninja` valid on August 7, 2025. Note that the second TXT record (which does not start with, case insensitively, "bitcoin:") is ignored.

```
2773696d706c6540646e737365635f70726f6f665f74657374732e626974636f696e2e6e696e6a610673696d706c650475736572105f
```

This encodes address `BC1QYYQ734DCNHHCWYW8YJXSWXG4P7K9NXU0QT84QZ` to `override.x_domain_cname_wild@dnssec_proof_tests.bitcoin.ninja` valid on August 6, 2025.

```
3d6f766572726964652e785f646f6d61696e5f636e616d655f77696c6440646e737365635f70726f6f665f74657374732e626974636f
```

The following encodes both the on-chain address `bc1qztwy6xen3zdtt7z0vrgapmjtfz8acjkfp5fp7l` and the BOLT 12 offer `lno1zr5qyugqgskrk70kqmuq7v3dnr2fnmhukps9n8hut48vkqpqnskt2svsqwjakp7k6pyhtkuxw7y2kqmsxlwruhzqv0zsnhh9q3t9xhx3`

to `a.x_domain_cname_wild@dnssec_proof_tests.bitcoin.ninja` valid on August 6, 2025. Note that it requires following a CNAME to a different domain.
`36612e785f646f6d61696e5f636e616d655f77696c6440646e737365635f70726f6f665f74657374732e626974636f696e2e6e696e6a`

The following is an *invalid* DNSSEC proof when included in the `PSBT_OUT_DNSSEC_PROOF` record of a PSBT.

The following encodes the on-chain address `bc1qztwy6xen3zdtt7z0vrgapmjtfz8acjkfp5fp7l` to `invalid@dnssec_proof_tests.bitcoin.ninja` but is **invalid**. This proof contains two TXT records which start with, case insensitively, "bitcoin:". It is a valid DNSSEC proof but should fail BIP 353-specific validation.
`28696e76616c696440646e737365635f70726f6f665f74657374732e626974636f696e2e6e696e6a6107696e76616c69640475736572`

The following encodes both the on-chain address `bc1qztwy6xen3zdtt7z0vrgapmjtfz8acjkfp5fp7l` and the BOLT 12 offer
`lno1zr5qyugqgskrk70kqmuq7v3dnr2fnmhukps9n8hut48vkqpqnskt2svsqwjakp7k6pyhtkuxw7y2kqmsxlwruhzqv0zsnhh9q3t9xhx3`
to `a.x_domain_cname_wild@dnssec_proof_tests.bitcoin.ninja` but is **invalid**. This proof is missing the required NSEC3 record to prove the lack of an override to the wildcard CNAME entry.
`36612e785f646f6d61696e5f636e616d655f77696c6440646e737365635f70726f6f665f74657374732e626974636f696e2e6e696e6a`

Further valid test cases can be generated by using https://satsto.me and selecting the "Look up using satsto.me's native proof server" option. This will cause lookups to be a single request to a server that returns a full DNSSEC proof (without the HRN prefix), which can be extracted using your browser's console.

Generated proofs can be tested against the below `dnssec-prover` implementation at https://satsto.me/prooftest.html

Reference Implementations

- A DNSSEC proof generation and validation implementation can be found at https://git.bitcoin.ninja/index.cgi?p=dnssec-prover;a=summary
- A lightning-specific name to payment instruction resolver can be found at https://git.bitcoin.ninja/index.cgi?p=lightning-resolver;a=summary
- Reference implementations for parsing the URI contents can be found in BIP 321.

Footnotes

Acknowledgements

Thanks to Rusty Russell for the concrete address rotation suggestion.

Thanks to the Bitcoin Design Community, and especially Christoph Ono for lots of discussion, analysis, and UX mockups in how human-readable payment instructions should be displayed.

Thanks to Andrew Kaizer for the suggestion to explicitly restrict cache lifetime to the relevant DNS TTLs.

```
BIP: 327
Title: MuSig2 for BIP340-compatible Multi-Signatures
Authors: Jonas Nick <jonasd.nick@gmail.com>
         Tim Ruffing <crypto@timruffing.de>
         Elliott Jin <elliott.jin@gmail.com>
Status: Deployed
Type: Informational
```

```
  Assigned: 2022-03-22
  License: BSD-3-Clause
  Discussion: 2022-04-05: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2022-April/
020198.html [bitcoin-dev] MuSig2 BIP
              2022-10-11: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2022-October/
021000.html [bitcoin-dev] MuSig2 BIP
  Version: 1.0.4
```

BIP 327

Introduction

Abstract

This document proposes a standard for the MuSig2 multi-signature scheme. The standard is compatible with BIP340 public keys and signatures. It supports *tweaking*, which allows deriving BIP32 child keys from aggregate public keys and creating BIP341 Taproot outputs with key and script paths.

Copyright

This document is licensed under the 3-clause BSD license.

Motivation

MuSig2 is a multi-signature scheme that allows multiple signers to create a single aggregate public key and cooperatively create ordinary Schnorr signatures valid under the aggregate public key. Signing requires interaction between *all* signers involved in key aggregation. (MuSig2 is a *n-of-n* multi-signature scheme and not a *t-of-n* threshold-signature scheme.)

The primary motivation is to create a standard that allows users of different software projects to jointly control Taproot outputs (BIP341). Such an output contains a public key which, in this case, would be the aggregate of all users' individual public keys. It can be spent using MuSig2 to produce a signature for the key-based spending path.

The on-chain footprint of a MuSig2 Taproot output is essentially a single BIP340 public key, and a transaction spending the output only requires a single signature cooperatively produced by all signers. This is **more compact** and has **lower verification cost** than each signer providing an individual public key and signature, as would be required by an *n-of-n* policy implemented using `OP_CHECKSIGADD` as introduced in (BIP342). As a side effect, the number *n* of signers is not limited by any consensus rules when using MuSig2.

Moreover, MuSig2 offers a **higher level of privacy** than `OP_CHECKSIGADD`: MuSig2 Taproot outputs are indistinguishable for a blockchain observer from regular, single-signer Taproot outputs even though they are actually controlled by multiple signers. By tweaking an aggregate public key, the shared Taproot output can have script spending paths that are hidden unless used.

There are multi-signature schemes other than MuSig2 that are fully compatible with Schnorr signatures. The MuSig2 variant proposed below stands out by combining all the following features:

- **Simple Key Setup**: Key aggregation is non-interactive and fully compatible with BIP340 public keys.

- **Two Communication Rounds**: MuSig2 is faster in practice than previous three-round multi-signature schemes such as MuSig1, particularly when signers are connected through high-latency anonymous links. Moreover, the need for fewer communication rounds simplifies the algorithms and reduces the probability that implementations and users make security-relevant mistakes.
- **Provable security**: MuSig2 has been proven existentially unforgeable under the algebraic one-more discrete logarithm (AOMDL) assumption (instead of the discrete logarithm assumption required for single-signer Schnorr signatures). AOMDL is a falsifiable and weaker variant of the well-studied OMDL problem.
- **Low complexity**: MuSig2 has a substantially lower computational and implementation complexity than alternative schemes like MuSig-DN. However, this comes at the cost of having no ability to generate nonces deterministically and the requirement to securely handle signing state.

Design

- **Compatibility with BIP340**: In this proposal, the aggregate public key is a BIP340 X-only public key, and the signature output at the end of the signing protocol is a BIP340 signature that passes BIP340 verification for the aggregate public key and a message. The individual public keys that are input to the key aggregation algorithm are *plain* public keys in compressed format.
- **Tweaking for BIP32 derivations and Taproot**: This proposal supports tweaking aggregate public keys and signing for tweaked aggregate public keys. We distinguish two modes of tweaking: *Plain* tweaking can be used to derive child aggregate public keys per BIP32. *X-only* tweaking, on the other hand, allows creating a BIP341 tweak to add script paths to a Taproot output. See below for details.
- **Non-interactive signing with preprocessing**: The first communication round, exchanging the nonces, can happen before the message or the exact set of signers is determined. Once the parameters of the signing session are finalized, the signers can send partial signatures without additional interaction.
- **Key aggregation optionally independent of order**: The output of the key aggregation algorithm depends on the order in which the individual public keys are provided as input. Key aggregation does not sort the individual public keys by default because applications often already have a canonical order of signers. Nonetheless, applications can mandate sorting before aggregation,Applications that sort individual public keys before aggregation should ensure that the implementation of sorting is reasonably efficient, and in particular does not degenerate to quadratic runtime on pathological inputs. and this proposal specifies a canonical order to sort the individual public keys before key aggregation. Sorting will ensure the same output, independent of the initial order.
- **Third-party nonce and partial signature aggregation**: Instead of every signer sending their nonce and partial signature to every other signer, it is possible to use an untrusted third-party *aggregator* in order to reduce the communication complexity from quadratic to linear in the number of signers. In each of the two rounds, the aggregator collects all signers' contributions (nonces or partial signatures), aggregates them, and broadcasts the aggregate back to the signers. A malicious aggregator can force the signing session to fail to produce a valid Schnorr signature but cannot negatively affect the unforgeability of the scheme.
- **Partial signature verification**: If any signer sends a partial signature contribution that was not created by honestly following the signing protocol, the signing session will fail to produce a valid Schnorr signature. This proposal specifies a partial signature verification algorithm to identify disruptive signers. It is incompatible with third-party nonce aggregation because the individual nonce is required for partial verification.
- **MuSig2* optimization**: This proposal uses an optimized scheme MuSig2*, which allows saving a point multiplication in key aggregation as compared to MuSig2. MuSig2* is proven secure in the appendix of

the MuSig2 paper. The optimization consists of assigning the constant key aggregation coefficient *1* to the second distinct key in the list of individual public keys to be aggregated (as well as to any key identical to this key).
- **Size of the nonce and security**: In this proposal, each signer's nonce consists of two elliptic curve points. The MuSig2 paper gives distinct security proofs depending on the number of points that constitute a nonce. See section Choosing the Size of the Nonce for a discussion.

Overview

Implementers must make sure to understand this section thoroughly to avoid subtle mistakes that may lead to catastrophic failure.

Optionality of Features

The goal of this proposal is to support a wide range of possible application scenarios. Given a specific application scenario, some features may be unnecessary or not desirable, and implementers can choose not to support them. Such optional features include:

- Applying plain tweaks after x-only tweaks.
- Applying tweaks at all.
- Dealing with messages that are not exactly 32 bytes.
- Identifying a disruptive signer after aborting (aborting itself remains mandatory).
- Dealing with duplicate individual public keys in key aggregation.

If applicable, the corresponding algorithms should simply fail when encountering inputs unsupported by a particular implementation. (For example, the signing algorithm may fail when given a message which is not 32 bytes.) Similarly, the test vectors that exercise the unimplemented features should be re-interpreted to expect an error, or be skipped if appropriate.

General Signing Flow

The signers start by exchanging their individual public keys and computing an aggregate public key using the *KeyAgg* algorithm. Whenever they want to sign a message, the basic order of operations to create a multi-signature is as follows:

First broadcast round: The signers start the signing session by running *NonceGen* to compute *secnonce* and *pubnonce*.We treat the *secnonce* and *pubnonce* as grammatically singular even though they include serializations of two scalars and two elliptic curve points, respectively. This treatment may be confusing for readers familiar with the MuSig2 paper. However, serialization is a technical detail that is irrelevant for users of MuSig2 interfaces. Then, the signers broadcast their *pubnonce* to each other and run *NonceAgg* to compute an aggregate nonce.

Second broadcast round: At this point, every signer has the required data to sign, which, in the algorithms specified below, is stored in a data structure called Session Context. Every signer computes a partial signature by running *Sign* with the secret signing key, the *secnonce* and the session context. Then, the signers broadcast their partial signatures to each other and run *PartialSigAgg* to obtain the final signature. If all signers behaved honestly, the result passes BIP340 verification.

Both broadcast rounds can be optimized by using an aggregator who collects all signers' nonces or partial signatures, aggregates them using *NonceAgg* or *PartialSigAgg*, respectively, and broadcasts the aggregate result back to the signers. A malicious aggregator can force the signing session to fail to produce a valid Schnorr signature but cannot negatively affect the unforgeability of the scheme, i.e., even a malicious aggregator colluding with all but one signer cannot forge a signature.

IMPORTANT: The *Sign* algorithm must **not** be executed twice with the same *secnonce*. Otherwise, it is possible to extract the secret signing key from the two partial signatures output by the two executions of *Sign*. To avoid accidental reuse of *secnonce*, an implementation may securely erase the *secnonce* argument by overwriting it with 64 zero bytes after it has been read by *Sign*. A *secnonce* consisting of only zero bytes is invalid for *Sign* and will cause it to fail.

To simplify the specification of the algorithms, some intermediary values are unnecessarily recomputed from scratch, e.g., when executing *GetSessionValues* multiple times. Actual implementations can cache these values. As a result, the Session Context may look very different in implementations or may not exist at all. However, computation of *GetSessionValues* and storage of the result must be protected against modification from an untrusted third party. This party would have complete control over the aggregate public key and message to be signed.

Public Key Aggregation

We distinguish between two public key types, namely *plain public keys*, the key type traditionally used in Bitcoin, and *X-only public keys*. Plain public keys are byte strings of length 33 (often called *compressed* format). In contrast, X-only public keys are 32-byte strings defined in BIP340.

The individual public keys of signers as input to the key aggregation algorithm *KeyAgg* (and to *GetSessionValues* and *PartialSigVerify*) are plain public keys. The output of *KeyAgg* is a KeyAgg Context which stores information required for tweaking the aggregate public key (see below), and it can be used to produce an X-only aggregate public key, or a plain aggregate public key. In order to obtain an X-only public key compatible with BIP340 verification, implementations call the *GetXonlyPubkey* function with the KeyAgg Context. To get the plain aggregate public key, which is required for some applications of tweaking, implementations call *GetPlainPubkey* instead.

The aggregate public key produced by *KeyAgg* (regardless of the type) depends on the order of the individual public keys. If the application does not have a canonical order of the signers, the individual public keys can be sorted with the *KeySort* algorithm to ensure that the aggregate public key is independent of the order of signers.

The same individual public key is allowed to occur more than once in the input of *KeyAgg* and *KeySort*. This is by design: All algorithms in this proposal handle multiple signers who (claim to) have identical individual public keys properly, and applications are not required to check for duplicate individual public keys. In fact, applications are recommended to omit checks for duplicate individual public keys in order to simplify error handling. Moreover, it is often impossible to tell at key aggregation which signer is to blame for the duplicate, i.e., which signer came up with an individual public key honestly and which disruptive signer copied it. In contrast, MuSig2 is designed to identify disruptive signers at signing time (see Identifying Disruptive Signers).

While the algorithms in this proposal are able to handle duplicate individual public keys, there are scenarios where applications may choose to abort when encountering duplicates. For example, we can imagine a scenario where a single entity creates a MuSig2 setup with multiple signing devices. In that case, duplicates

may not result from a malicious signing device copying an individual public key of another signing device but from accidental initialization of two devices with the same seed. Since MuSig2 key aggregation would accept the duplicate keys and not error out, which would in turn reduce the security compared to the intended key setup, applications may reject duplicate individual public keys before passing them to MuSig2 key aggregation and ask the user to investigate.

Nonce Generation

IMPORTANT: *NonceGen* must have access to a high-quality random generator to draw an unbiased, uniformly random value *rand'*. In contrast to BIP340 signing, the values k_1 and k_2 **must not be derived deterministically** from the session parameters because otherwise active adversaries can trick the victim into reusing a nonce.

The optional arguments to *NonceGen* enable a defense-in-depth mechanism that may prevent secret key exposure if *rand'* is accidentally not drawn uniformly at random. If the value *rand'* was identical in two *NonceGen* invocations, but any other argument was different, the *secnonce* would still be guaranteed to be different as well (with overwhelming probability), and thus accidentally using the same *secnonce* for *Sign* in both sessions would be avoided. Therefore, it is recommended to provide the optional arguments *sk*, *aggpk*, and *m* if these session parameters are already determined during nonce generation. The auxiliary input *extra_in* can contain additional contextual data that has a chance of changing between *NonceGen* runs, e.g., a supposedly unique session id (taken from the application), a session counter wide enough not to repeat in practice, any nonces by other signers (if already known), or the serialization of a data structure containing multiple of the above. However, the protection provided by the optional arguments should only be viewed as a last resort. In most conceivable scenarios, the assumption that the arguments are different between two executions of *NonceGen* is relatively strong, particularly when facing an active adversary.

In some applications, it is beneficial to generate and send a *pubnonce* before the other signers, their individual public keys, or the message to sign is known. In this case, only the available arguments are provided to the *NonceGen* algorithm. After this preprocessing phase, the *Sign* algorithm can be run immediately when the message and set of signers is determined. This way, the final signature is created quicker and with fewer round trips. However, applications that use this method presumably store the nonces for a longer time and must therefore be even more careful not to reuse them. Moreover, this method is not compatible with the defense-in-depth mechanism described in the previous paragraph.

Instead of every signer broadcasting their *pubnonce* to every other signer, the signers can send their *pubnonce* to a single aggregator node that runs *NonceAgg* and sends the *aggnonce* back to the signers. This technique reduces the overall communication. A malicious aggregator can force the signing session to fail to produce a valid Schnorr signature but cannot negatively affect the unforgeability of the scheme.

In general, MuSig2 signers are stateful in the sense that they first generate *secnonce* and then need to store it until they receive the other signers' *pubnonces* or the *aggnonce*. However, it is possible for one of the signers to be stateless. This signer waits until it receives the *pubnonce* of all the other signers and until session parameters such as a message to sign, individual public keys, and tweaks are determined. Then, the signer can run *NonceGen*, *NonceAgg* and *Sign* in sequence and send out its *pubnonce* along with its partial signature. Stateless signers may want to consider signing deterministically (see Modifications to Nonce Generation) to remove the reliance on the random number generator in the *NonceGen* algorithm.

Identifying Disruptive Signers

The signing protocol makes it possible to identify malicious signers who send invalid contributions to a signing session in order to make the signing session abort and prevent the honest signers from obtaining a valid signature. This property is called "identifiable aborts" and ensures that honest parties can assign blame to malicious signers who cause an abort in the signing protocol.

Aborts are identifiable for an honest party if the following conditions hold in a signing session:

- The contributions received from all signers have not been tampered with (e.g., because they were sent over authenticated connections).
- Nonce aggregation is performed honestly (e.g., because the honest signer performs nonce aggregation on its own or because the aggregator is trusted).
- The partial signatures received from all signers are verified using the algorithm *PartialSigVerify*.

If these conditions hold and an honest party (signer or aggregator) runs an algorithm that fails due to invalid protocol contributions from malicious signers, then the algorithm run by the honest party will output the index of exactly one malicious signer. Additionally, if the honest parties agree on the contributions sent by all signers in the signing session, all the honest parties who run the aborting algorithm will identify the same malicious signer.

Further Remarks

Some of the algorithms specified below may also assign blame to a malicious aggregator. While this is possible for some particular misbehavior of the aggregator, it is not guaranteed that a malicious aggregator can be identified. More specifically, a malicious aggregator (whose existence violates the second condition above) can always make signing abort and wrongly hold honest signers accountable for the abort (e.g., by claiming to have received an invalid contribution from a particular honest signer).

The only purpose of the algorithm *PartialSigVerify* is to ensure identifiable aborts, and it is not necessary to use it when identifiable aborts are not desired. In particular, partial signatures are *not* signatures. An adversary can forge a partial signature, i.e., create a partial signature without knowing the secret key for the claimed individual public key.Assume an adversary wants to forge a partial signature for individual public key P. It joins the signing session pretending to be two different signers, one with individual public key P and one with another individual public key. The adversary can then set the second signer's nonce such that it will be able to produce a partial signature for P but not for the other claimed signer. An explanation of the individual steps required to create a partial signature forgery can be found in a write up by Adam Gibson. However, if *PartialSigVerify* succeeds for all partial signatures then *PartialSigAgg* will return a valid Schnorr signature.Given a list of individual public keys, it is an open question whether a BIP-340 signature valid under the corresponding aggregate public key is a proof of knowledge of all secret keys of the individual public keys.

Tweaking the Aggregate Public Key

The aggregate public key can be *tweaked*, which modifies the key as defined in the Tweaking Definition subsection. In order to apply a tweak, the KeyAgg Context output by *KeyAgg* is provided to the *ApplyTweak* algorithm with the *is_xonly_t* argument set to false for plain tweaking and true for X-only tweaking. The resulting KeyAgg Context can be used to apply another tweak with *ApplyTweak* or obtain the aggregate public key with *GetXonlyPubkey* or *GetPlainPubkey*.

The purpose of supporting tweaking is to ensure compatibility with existing uses of tweaking, i.e., that the result of signing is a valid signature for the tweaked public key. The MuSig2 algorithms take arbitrary tweaks as input but accepting arbitrary tweaks may negatively affect the security of the scheme.It is an open question whether allowing arbitrary tweaks from an adversary affects the unforgeability of MuSig2. Instead, signers should obtain the tweaks according to other specifications. This typically involves deriving the tweaks from a hash of the aggregate public key and some other information. Depending on the specific scheme that is used for tweaking, either the plain or the X-only aggregate public key is required. For example, to do BIP32 derivation, you call *GetPlainPubkey* to be able to compute the tweak, whereas BIP341 TapTweaks require X-only public keys that are obtained with *GetXonlyPubkey*.

The tweak mode provided to *ApplyTweak* depends on the application: Plain tweaking can be used to derive child public keys from an aggregate public key using BIP32. On the other hand, X-only tweaking is required for Taproot tweaking per BIP341. A Taproot-tweaked public key commits to a *script path*, allowing users to create transaction outputs that are spendable either with a MuSig2 multi-signature or by providing inputs that satisfy the script path. Script path spends require a control block that contains a parity bit for the tweaked X-only public key. The bit can be obtained with *GetPlainPubkey(keyagg_ctx)[0] & 1*.

Algorithms

The following specification of the algorithms has been written with a focus on clarity. As a result, the specified algorithms are not always optimal in terms of computation and space. In particular, some values are recomputed but can be cached in actual implementations (see General Signing Flow).

Notation

The following conventions are used, with constants as defined for secp256k1. We note that adapting this proposal to other elliptic curves is not straightforward and can result in an insecure scheme.

- Lowercase variables represent integers or byte arrays.
  - The constant p refers to the field size, *0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F*.
  - The constant n refers to the curve order, *0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141*.
- Uppercase variables refer to points on the curve with equation $y^2 = x^3 + 7$ over the integers modulo p.
  - *is_infinite(P)* returns whether P is the point at infinity.
  - *x(P)* and *y(P)* are integers in the range *0..p-1* and refer to the X and Y coordinates of a point P (assuming it is not infinity).
  - The constant G refers to the base point, for which *x(G) = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798* and *y(G) = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8*.
  - Addition of points refers to the usual elliptic curve group operation.
  - Multiplication ($\cdot$) of an integer and a point refers to the repeated application of the group operation.
- Functions and operations:
  - $||$ refers to byte array concatenation.
  - The function *x[i:j]*, where x is a byte array and $i, j \geq 0$, returns a *(j - i)*-byte array with a copy of the i-th byte (inclusive) to the j-th byte (exclusive) of x.

    - The function *bytes(n, x)*, where *x* is an integer, returns the n-byte encoding of *x*, most significant byte first.
    - The constant *empty_bytestring* refers to the empty byte array. It holds that *len(empty_bytestring) = 0*.
    - The function *xbytes(P)*, where *P* is a point for which *not is_infinite(P)*, returns *bytes(32, x(P))*.
    - The function *len(x)* where *x* is a byte array returns the length of the array.
    - The function *has_even_y(P)*, where *P* is a point for which *not is_infinite(P)*, returns *y(P) mod 2 == 0*.
    - The function *with_even_y(P)*, where *P* is a point, returns *P* if *is_infinite(P)* or *has_even_y(P)*. Otherwise, *with_even_y(P)* returns *-P*.
    - The function *cbytes(P)*, where *P* is a point for which *not is_infinite(P)*, returns *a || xbytes(P)* where *a* is a byte that is *2* if *has_even_y(P)* and *3* otherwise.
    - The function *cbytes_ext(P)*, where *P* is a point, returns *bytes(33, 0)* if *is_infinite(P)*. Otherwise, it returns *cbytes(P)*.
    - The function *int(x)*, where *x* is a 32-byte array, returns the 256-bit unsigned integer whose most significant byte first encoding is *x*.
    - The function *lift_x(x)*, where *x* is an integer in range *0..2^256-1*, returns the point *P* for which *x(P) = x*

Given a candidate X coordinate *x* in the range *0..p-1*, there exist either exactly two or exactly zero vali
= *x*. The valid Y coordinates for a given candidate *x* are the square roots of *c = x^3+ 7 mod p* and they can be computed as *y = ±c^(p+1)/4 mod p* (see Quadratic residue) if they exist, which can be checked by squaring and comparing with *c*. and *has_even_y(P)*, or fails i

-
    - Fail if *x > p-1*.
    - Let *c = x^3 + 7 mod p*.
    - Let *y' = c^(p+1)/4 mod p*.
    - Fail if *c ≠ y'^2 mod p*.
    - Let *y = y'* if *y' mod 2 = 0*, otherwise let *y = p - y'*.
    - Return the unique point *P* such that *x(P) = x* and *y(P) = y*.
-
    - The function *cpoint(x)*, where *x* is a 33-byte array (compressed serialization), sets *P = lift_x(int(x[1:33]))* and fails if that fails. If *x[0] = 2* it returns *P* and if *x[0] = 3* it returns *-P*. Otherwise, it fails.
    - The function *cpoint_ext(x)*, where *x* is a 33-byte array (compressed serialization), returns the point at infinity if *x = bytes(33, 0)*. Otherwise, it returns *cpoint(x)* and fails if that fails.
    - The function *hash_tag(x)* where *tag* is a UTF-8 encoded tag name and *x* is a byte array returns the 32-byte hash *SHA256(SHA256(tag) || SHA256(tag) || x)*.
- Other:
    - Tuples are written by listing the elements within parentheses and separated by commas. For example, *(2, 3, 1)* is a tuple.

Key Generation and Aggregation

Key Generation of an Individual Signer

Algorithm *IndividualPubkey(sk)*:The *IndividualPubkey* algorithm matches the key generation procedure traditionally used for ECDSA in Bitcoin

- Inputs:
    - The secret key *sk*: a 32-byte array, freshly generated uniformly at random
- Let $d' = int(sk)$.
- Fail if $d' = 0$ or $d' \geq n$.
- Return $cbytes(d' \cdot G)$.

KeyAgg Context

The KeyAgg Context is a data structure consisting of the following elements:

- The point Q representing the potentially tweaked aggregate public key: an elliptic curve point
- The accumulated tweak *tacc*: an integer with $0 \leq tacc < n$
- The value *gacc* : 1 or -1 mod n

We write "Let *(Q, gacc, tacc) = keyagg_ctx*" to assign names to the elements of a KeyAgg Context.

Algorithm *GetXonlyPubkey(keyagg_ctx)*:

- Let *(Q, _, _) = keyagg_ctx*
- Return *xbytes(Q)*

Algorithm *GetPlainPubkey(keyagg_ctx)*:

- Let *(Q, _, _) = keyagg_ctx*
- Return *cbytes(Q)*

Key Sorting

Algorithm *KeySort*($pk_{1..u}$):

- Inputs:
    - The number u of individual public keys with $0 < u < 2^{32}$
    - The individual public keys $pk_{1..u}$: u 33-byte arrays
- Return $pk_{1..u}$ sorted in lexicographical order.

Key Aggregation

Algorithm *KeyAgg*($pk_{1..u}$):

- Inputs:
    - The number u of individual public keys with $0 < u < 2^{32}$
    - The individual public keys $pk_{1..u}$: u 33-byte arrays

- Let *pk2* = *GetSecondKey*($pk_{1..u}$)
- For $i = 1 .. u$:
    - Let P_i = *cpoint*(pk_i); fail if that fails and blame signer i for invalid individual public key.
    - Let a_i = *KeyAggCoeffInternal*($pk_{1..u}$, pk_i, *pk2*).
- Let $Q = a_1 \cdot P_1 + a_2 \cdot P_2 + \ldots + a_u \cdot P_u$
- Fail if *is_infinite*(*Q*).
- Let *gacc* = *1*
- Let *tacc* = *0*
- Return *keyagg_ctx* = *(Q, gacc, tacc)*.

Internal Algorithm *HashKeys*($pk_{1..u}$):

- Return $hash_{KeyAgg\ list}(pk_1 \,||\, pk_2 \,||\, \ldots \,||\, pk_u)$

Internal Algorithm *GetSecondKey*($pk_{1..u}$):

- For $j = 1 .. u$:
    - If $pk_j \neq pk_1$:
        - Return pk_j
- Return *bytes(33, 0)*

Internal Algorithm *KeyAggCoeff*($pk_{1..u}$, *pk'*):

- Let *pk2* = *GetSecondKey*($pk_{1..u}$):
- Return *KeyAggCoeffInternal*($pk_{1..u}$, *pk'*, *pk2*)

Internal Algorithm *KeyAggCoeffInternal*($pk_{1..u}$, *pk'*, *pk2*):

- Let *L* = *HashKeys*($pk_{1..u}$)
- If *pk'* = *pk2*:
    - Return 1
- Return $int(hash_{KeyAgg\ coefficient}(L \,||\, pk'))\ mod\ n$The key aggregation coefficient is computed by hashing the individual public key instead of its index, which requires one more invocation of the SHA-256 compression function. However, it results in significantly simpler implementations because signers do not need to translate between public key indices before and after sorting.

Applying Tweaks

Algorithm *ApplyTweak(keyagg_ctx, tweak, is_xonly_t)*:

- Inputs:
    - The *keyagg_ctx*: a KeyAgg Context data structure
    - The *tweak*: a 32-byte array
    - The tweak mode *is_xonly_t*: a boolean
- Let *(Q, gacc, tacc)* = *keyagg_ctx*
- If *is_xonly_t* and *not has_even_y(Q)*:
    - Let *g* = *-1 mod n*

- Else:
    - Let $g = 1$
- Let $t = int(tweak)$; fail if $t \geq n$
- Let $Q' = g \cdot Q + t \cdot G$
    - Fail if $is_infinite(Q')$
- Let $gacc' = g \cdot gacc \bmod n$
- Let $tacc' = t + g \cdot tacc \bmod n$
- Return $keyagg_ctx' = (Q', gacc', tacc')$

Nonce Generation

Algorithm $NonceGen(sk, pk, aggpk, m, extra_in)$:

- Inputs:
    - The secret signing key sk: a 32-byte array (optional argument)
    - The individual public key pk: a 33-byte array (see Signing with Tweaked Individual Keys for the reason that this argument is mandatory)
    - The x-only aggregate public key $aggpk$: a 32-byte array (optional argument)
    - The message m: a byte array (optional argument)In theory, the allowed message size is restricted because SHA256 accepts byte strings only up to size of 2^61-1 bytes (and because of the 8-byte length encoding).
    - The auxiliary input $extra_in$: a byte array with $0 \leq len(extra_in) \leq 2^{32}-1$ (optional argument)
- Let $rand'$ be a 32-byte array freshly drawn uniformly at random
- If the optional argument sk is present:
    - Let $rand$ be the byte-wise xor of sk and $hash_{MuSig/aux}(rand')$The random data is hashed (with a unique tag) as a precaution against situations where the randomness may be correlated with the secret signing key itself. It is xored with the secret key (rather than combined with it in a hash) to reduce the number of operations exposed to the actual secret key.
- Else:
    - Let $rand = rand'$
- If the optional argument $aggpk$ is not present:
    - Let $aggpk = empty_bytestring$
- If the optional argument m is not present:
    - Let $m_prefixed = bytes(1, 0)$
- Else:
    - Let $m_prefixed = bytes(1, 1) \,||\, bytes(8, len(m)) \,||\, m$
- If the optional argument $extra_in$ is not present:
    - Let $extra_in = empty_bytestring$
- Let $k_i = int(hash_{MuSig/nonce}(rand \,||\, bytes(1, len(pk)) \,||\, pk \,||\, bytes(1, len(aggpk)) \,||\, aggpk \,||\, m_prefixed \,||\, bytes(4, len(extra_in)) \,||\, extra_in \,||\, bytes(1, i - 1))) \bmod n$ for $i = 1,2$
- Fail if $k_1 = 0$ or $k_2 = 0$
- Let $R_{*,1} = k_1 \cdot G$, $R_{*,2} = k_2 \cdot G$
- Let $pubnonce = cbytes(R_{*,1}) \,||\, cbytes(R_{*,2})$
- Let $secnonce = bytes(32, k_1) \,||\, bytes(32, k_2) \,||\, pk$The algorithms as specified here assume that the $secnonce$ is stored as a 97-byte array using the serialization $secnonce = bytes(32, k_1) \,||\, bytes(32, k_2) \,||\, pk$. The same format is used in the reference implementation and in the test vectors. However, since the $secnonce$ is

(obviously) not meant to be sent over the wire, compatibility between implementations is not a concern, and this method of storing the *secnonce* is merely a suggestion.

The *secnonce* is effectively a local data structure of the signer which comprises the value triple (k_1, k_2, pk), and implementations may choose any suitable method to carry it from *NonceGen* (first communication round) to *Sign* (second communication round). In particular, implementations may choose to hide the *secnonce* in internal state without exposing it in an API explicitly, e.g., in an effort to prevent callers from reusing a *secnonce* accidentally.

- Return *(secnonce, pubnonce)*

Nonce Aggregation

Algorithm $NonceAgg(pubnonce_{1..u})$:

- Inputs:
  - The number u of *pubnonces* with $0 < u < 2^{32}$
  - The public nonces $pubnonce_{1..u}$: u 66-byte arrays
- For $j = 1 .. 2$:
  - For $i = 1 .. u$:
    - Let $R_{i,j} = cpoint(pubnonce_i[(j-1)*33:j*33])$; fail if that fails and blame signer i for invalid *pubnonce*.
  - Let $R_j = R_{1,j} + R_{2,j} + \ldots + R_{u,j}$
- Return $aggnonce = cbytes_ext(R_1) \,||\, cbytes_ext(R_2)$

Session Context

The Session Context is a data structure consisting of the following elements:

- The aggregate public nonce *aggnonce*: a 66-byte array
- The number u of individual public keys with $0 < u < 2^{32}$
- The individual public keys $pk_{1..u}$: u 33-byte arrays
- The number v of tweaks with $0 \leq v < 2^{32}$
- The tweaks $tweak_{1..v}$: v 32-byte arrays
- The tweak modes $is_xonly_t_{1..v}$: v booleans
- The message m: a byte array

We write "Let $(aggnonce, u, pk_{1..u}, v, tweak_{1..v}, is_xonly_t_{1..v}, m) = session_ctx$" to assign names to the elements of a Session Context.

Algorithm *GetSessionValues(session_ctx)*:

- Let $(aggnonce, u, pk_{1..u}, v, tweak_{1..v}, is_xonly_t_{1..v}, m) = session_ctx$
- Let $keyagg_ctx_0 = KeyAgg(pk_{1..u})$; fail if that fails
- For $i = 1 .. v$:
  - Let $keyagg_ctx_i = ApplyTweak(keyagg_ctx_{i-1}, tweak_i, is_xonly_t_i)$; fail if that fails
- Let $(Q, gacc, tacc) = keyagg_ctx_v$

- Let $b = int(hash_{MuSig/noncecoef}(aggnonce \mid\mid xbytes(Q) \mid\mid m)) \bmod n$
- Let $R_1 = cpoint_ext(aggnonce[0:33])$, $R_2 = cpoint_ext(aggnonce[33:66])$; fail if that fails and blame nonce aggregator for invalid *aggnonce*.
- Let $R' = R_1 + b \cdot R_2$
- If ''is_infinite(R'):
    - Let final nonce $R = G$ (see Dealing with Infinity in Nonce Aggregation)
- Else:
    - Let final nonce $R = R'$
- Let $e = int(hash_{BIP0340/challenge}(xbytes(R) \mid\mid xbytes(Q) \mid\mid m)) \bmod n$
- Return *(Q, gacc, tacc, b, R, e)*

Algorithm *GetSessionKeyAggCoeff(session_ctx, P)*:

- Let $(_, u, pk_{1..u}, _, _, _, _) = session_ctx$
- Let $pk = cbytes(P)$
- Fail if pk not in $pk_{1..u}$
- Return $KeyAggCoeff(pk_{1..u}, pk)$

Signing

Algorithm *Sign(secnonce, sk, session_ctx)*:

- Inputs:
    - The secret nonce *secnonce* that has never been used as input to *Sign* before: a 97-byte array
    - The secret key *sk*: a 32-byte array
    - The *session_ctx*: a Session Context data structure
- Let $(Q, gacc, _, b, R, e) = GetSessionValues(session_ctx)$; fail if that fails
- Let $k_1' = int(secnonce[0:32])$, $k_2' = int(secnonce[32:64])$
- Fail if $k_i' = 0$ or $k_i' \geq n$ for $i = 1..2$
- Let $k_1 = k_1'$, $k_2 = k_2'$ if $has_even_y(R)$, otherwise let $k_1 = n - k_1'$, $k_2 = n - k_2'$
- Let $d' = int(sk)$
- Fail if $d' = 0$ or $d' \geq n$
- Let $P = d' \cdot G$
- Let $pk = cbytes(P)$
- Fail if $pk \neq secnonce[64:97]$
- Let $a = GetSessionKeyAggCoeff(session_ctx, P)$; fail if that failsFailing *Sign* when *GetSessionKeyAggCoeff(session_ctx, P)* fails is not necessary for unforgeability. It merely indicates to the caller that the scheme is not being used correctly.

- Let $g = 1$ if $has_even_y(Q)$, otherwise let $g = -1 \bmod n$
- Let $d = g \cdot gacc \cdot d' \bmod n$ (See Negation Of The Secret Key When Signing)
- Let $s = (k_1 + b \cdot k_2 + e \cdot a \cdot d) \bmod n$
- Let $psig = bytes(32, s)$
- Let $pubnonce = cbytes(k_1' \cdot G) \,||\, cbytes(k_2' \cdot G)$
- If $PartialSigVerifyInternal(psig, pubnonce, pk, session_ctx)$ (see below) returns failure, failVerifying the signature before leaving the signer prevents random or adversarially provoked computation errors. This prevents publishing invalid signatures which may leak information about the secret key. It is recommended but can be omitted if the computation cost is prohibitive.
- Return partial signature $psig$

Partial Signature Verification

Algorithm $PartialSigVerify(psig, pubnonce_{1..u}, pk_{1..u}, tweak_{1..v}, is_xonly_t_{1..v}, m, i)$:

- Inputs:
  - The partial signature $psig$: a 32-byte array
  - The number u of public nonces and individual public keys with $0 < u < 2^{32}$
  - The public nonces $pubnonce_{1..u}$: u 66-byte arrays
  - The individual public keys $pk_{1..u}$: u 33-byte arrays
  - The number v of tweaks with $0 \leq v < 2^{32}$
  - The tweaks $tweak_{1..v}$: v 32-byte arrays
  - The tweak modes $is_xonly_t_{1..v}$: v booleans
  - The message m: a byte array
  - The index of the signer i in the public nonces and individual public keys with $0 < i \leq u$
- Let $aggnonce = NonceAgg(pubnonce_{1..u})$; fail if that fails
- Let $session_ctx = (aggnonce, u, pk_{1..u}, v, tweak_{1..v}, is_xonly_t_{1..v}, m)$
- Run $PartialSigVerifyInternal(psig, pubnonce_i, pk_i, session_ctx)$
- Return success iff no failure occurred before reaching this point.

Internal Algorithm $PartialSigVerifyInternal(psig, pubnonce, pk, session_ctx)$:

- Let $(Q, gacc, _, b, R, e) = GetSessionValues(session_ctx)$; fail if that fails
- Let $s = int(psig)$; fail if $s \geq n$
- Let $R_{*,1} = cpoint(pubnonce[0:33])$, $R_{*,2} = cpoint(pubnonce[33:66])$
- Let $Re_*' = R_{*,1} + b \cdot R_{*,2}$
- Let effective nonce $Re_* = Re_*'$ if $has_even_y(R)$, otherwise let $Re_* = -Re_*'$
- Let $P = cpoint(pk)$; fail if that fails

- Let a = *GetSessionKeyAggCoeff(session_ctx, P)GetSessionKeyAggCoeff(session_ctx, P)* cannot fail when called from *PartialSigVerifyInternal*.
- Let $g = 1$ if *has_even_y(Q)*, otherwise let g = *-1 mod n*
- Let $g' = g \cdot gacc \bmod n$ (See Negation Of The Individual Public Key When Partially Verifying)
- Fail if $s \cdot G \neq Re_{*} + e \cdot a \cdot g' \cdot P$
- Return success iff no failure occurred before reaching this point.

Partial Signature Aggregation

Algorithm *PartialSigAgg(*$psig_{1..u}$*, session_ctx)*:

- Inputs:
  - The number u of signatures with $0 < u < 2^{32}$
  - The partial signatures $psig_{1..u}$: u 32-byte arrays
  - The *session_ctx*: a Session Context data structure
- Let *(Q, _, tacc, _, R, e) = GetSessionValues(session_ctx)*; fail if that fails
- For $i = 1 .. u$:
  - Let $s_i = int(psig_i)$; fail if $s_i \geq n$ and blame signer i for invalid partial signature.
- Let $g = 1$ if *has_even_y(Q)*, otherwise let g = *-1 mod n*
- Let $s = s_1 + \ldots + s_u + e \cdot g \cdot tacc \bmod n$
- Return *sig* =xbytes(R) || bytes(32, s)''

Test Vectors and Reference Code

We provide a naive, highly inefficient, and non-constant time pure Python 3 reference implementation of the key aggregation, partial signing, and partial signature verification algorithms.

Standalone JSON test vectors are also available in the same directory, to facilitate porting the test vectors into other implementations.

The reference implementation is for demonstration purposes only and not to be used in production environments.

Remarks on Security and Correctness

Signing with Tweaked Individual Keys

The scheme in this proposal has been designed to be secure even if signers tweak their individual secret keys with tweaks known to the adversary (e.g., as in BIP32 unhardened derivation) before providing the corresponding individual public keys as input to key aggregation. In particular, the scheme as specified above requires each signer to provide a final individual public key *pk* already to *NonceGen*, which writes it into the *secnonce* array so that it can be checked against *IndividualPubkey(sk)* in the *Sign* algorithm. The purpose of this check in *Sign* is to ensure that *pk*, and thus the secret key *sk* that will be provided to *Sign*, is determined before the signer sends out the *pubnonce*.

If the check in *Sign* was omitted, and a signer supported signing with at least two different secret keys sk_1 and sk_2 which have been obtained via tweaking another secret key with tweaks known to the adversary, then the adversary could, after having seen the *pubnonce*, influence whether sk_1 or sk_2 is provided to *Sign*. This degree of freedom may allow the adversary to perform a generalized birthday attack and thereby forge a signature (see bitcoin-dev mailing list post and writeup for details).

Checking *pk* against *IndividualPubkey(sk)* is a simple way to ensure that the secret key provided to *Sign* is fully determined already when *NonceGen* is invoked. This removes the adversary's ability to influence the secret key after having seen the *pubnonce* and thus rules out the attack.Ensuring that the secret key provided to *Sign* is fully determined already when *NonceGen* is invoked is a simple policy to rule out the attack, but more flexible polices are conceivable. In fact, if the signer uses nothing but the message to be signed and the list of the individual public keys of all signers to decide which secret key to use, then it is not a problem that the adversary can influence this decision after having seen the *pubnonce*.
More formally, consider modified algorithms *NonceGen'* and *Sign'*, where *NonceGen'* does not take the individual public key of the signer as input and does not store it in pubnonce, and Sign' does not check read the individual public key from pubnonce and does not check it against the secret key taken as input. Then it suffices that for each invocation of *NonceGen'* with output *(secnonce, pubnonce)*, a function *fsk* is determined before sending out *pubnonce*, where *fsk* maps a pair consisting of a list of individual public keys and a message to a secret key, such that the secret key *sk* and the session context $session_ctx = (_, _, pk_{1..u}, _, _, _, m)$ provided to the corresponding invocation of *Sign'(secnonce, sk, session_ctx)*, adhere to the condition $fsk(pk_{1..u}, m) = sk$.
However, this requirement is complex and hard to enforce in implementations. The algorithms *NonceGen* and *Sign* specified in this BIP are effectively restricted to constant functions $fsk(_, _) = sk$. In other words, their usage ensure that the secret key *sk* of the signers is determined entirely when invoking *NonceGen*, which is enforced easily by letting *NonceGen* take the corresponding individual public key *pk* as input and checking *pk* against *IndividualPubKey(sk)* in *Sign*. Note that the scheme as given in the MuSig2 paper does not perform the check in *Sign*. However, the security model in the paper does not cover tweaking at all and assumes a single fixed secret key.

Modifications to Nonce Generation

Implementers must avoid modifying the *NonceGen* algorithm without being fully aware of the implications. We provide two modifications to *NonceGen* that are secure when applied correctly and may be useful in special circumstances, summarized in the following table.

	needs secure randomness	**needs secure counter**	**needs to keep state securely**	**needs aggregate nonce of all other signers (only possible for one signer)**
NonceGen	✓		✓	
CounterNonceGen		✓	✓	
DeterministicSign				✓

First, on systems where obtaining uniformly random values is much harder than maintaining a global atomic counter, it can be beneficial to modify *NonceGen*. The resulting algorithm *CounterNonceGen* does not draw *rand'*

uniformly at random but instead sets *rand'* to the value of an atomic counter that is incremented whenever it is read. With this modification, the secret signing key *sk* of the signer generating the nonce is **not** an optional argument and must be provided to *NonceGen*. The security of the resulting scheme then depends on the requirement that reading the counter must never yield the same counter value in two *NonceGen* invocations with the same *sk*.

Second, if there is a unique signer who is supposed to send the *pubnonce* last, it is possible to modify nonce generation for this single signer to not require high-quality randomness. Such a nonce generation algorithm *DeterministicSign* is specified below. Note that the only optional argument is *rand*, which can be omitted if randomness is entirely unavailable. *DeterministicSign* requires the argument *aggothernonce* which should be set to the output of *NonceAgg* run on the *pubnonce* value of **all** other signers (but can be provided by an untrusted party). Hence, using *DeterministicSign* is only possible for the last signer to generate a nonce and makes the signer stateless, similar to the stateless signer described in the Nonce Generation section.

Deterministic and Stateless Signing for a Single Signer

Algorithm *DeterministicSign(sk, aggothernonce,* $pk_{1..u}$*,* $tweak_{1..v}$*,* $is_xonly_t_{1..v}$*, m, rand)*:

- Inputs:
  - The secret signing key *sk*: a 32-byte array
  - The aggregate public nonce *aggothernonce* (see above): a 66-byte array
  - The number u of individual public keys with $0 < u < 2^{32}$
  - The individual public keys $pk_{1..u}$: u 33-byte arrays
  - The number v of tweaks with $0 \leq v < 2^{32}$
  - The tweaks $tweak_{1..v}$: v 32-byte arrays
  - The tweak methods $is_xonly_t_{1..v}$: v booleans
  - The message *m*: a byte array
  - The auxiliary randomness *rand*: a 32-byte array (optional argument)
- If the optional argument *rand* is present:
  - Let *sk'* be the byte-wise xor of *sk* and $hash_{MuSig/aux}(rand)$
- Else:
  - Let $sk' = sk$
- Let $keyagg_ctx_0 = KeyAgg(pk_{1..u})$; fail if that fails
- For $i = 1 .. v$:
  - Let $keyagg_ctx_i = ApplyTweak(keyagg_ctx_{i-1}, tweak_i, is_xonly_t_i)$; fail if that fails
- Let $aggpk = GetXonlyPubkey(keyagg_ctx_v)$
- Let $k_i = int(hash_{MuSig/deterministic/nonce}(sk' \,||\, aggothernonce \,||\, aggpk \,||\, bytes(8, len(m)) \,||\, m \,||\, bytes(1, i - 1))) \bmod n$ for $i = 1,2$
- Fail if $k_1 = 0$ or $k_2 = 0$
- Let $R_{*,1} = k_1 \cdot G$, $R_{*,2} = k_2 \cdot G$
- Let $pubnonce = cbytes(R_{*,1}) \,||\, cbytes(R_{*,2})$
- Let $d = int(sk)$
- Fail if $d = 0$ or $d \geq n$
- Let $pk = cbytes(d \cdot G)$
- Let $secnonce = bytes(32, k_1) \,||\, bytes(32, k_2) \,||\, pk$

- Let *aggnonce = NonceAgg((pubnonce, aggothernonce))*; fail if that fails and blame nonce aggregator for invalid *aggothernonce*.
- Let *session_ctx = (aggnonce, u, $pk_{1..u}$, v, $tweak_{1..v}$, $is_xonly_t_{1..v}$, m)*
- Return *(pubnonce, Sign(secnonce, sk, session_ctx))*

Tweaking Definition

Two modes of tweaking the aggregate public key are supported. They correspond to the following algorithms:

Algorithm *ApplyPlainTweak(P, t)*:

- Inputs:
  - *P*: a point
  - The tweak *t*: an integer with $0 \leq t < n$
- Return $P + t \cdot G$

Algorithm *ApplyXonlyTweak(P, t)*:

- Return *with_even_y(P) + t·G*

Negation Of The Secret Key When Signing

In order to produce a partial signature for an X-only aggregate public key that is an aggregate of *u* individual public keys and tweaked *v* times (X-only or plain), the *Sign* algorithm may need to negate the secret key during the signing process.

<poem> The following elliptic curve points arise as intermediate steps when creating a signature: • P_i as computed in *KeyAgg* is the point corresponding to the *i*-th signer's individual public key. Defining d_i' to be the *i*-th signer's secret key as an integer, i.e., the *d'* value as computed in the *Sign* algorithm of the *i*-th signer, we have

$$P_i = d_i' \cdot G.$$

• Q_0 is the aggregate of the individual public keys. It is identical to value *Q* computed in *KeyAgg* and therefore defined as

$$Q_0 = a_1 \cdot P_1 + a_2 \cdot P_2 + \ldots + a_u \cdot P_u.$$

• Q_i is the tweaked aggregate public key after the *i*-th execution of *ApplyTweak* for $1 \leq i \leq v$. It holds that

$$\begin{aligned} Q_i &= f(i-1) + t_i \cdot G \text{ for } i = 1, \ldots, v \text{ where} \\ &\quad f(i-1) := with_even_y(Q_{i-1}) \text{ if } is_xonly_t_i \text{ and} \\ &\quad f(i-1) := Q_{i-1} \text{ otherwise.} \end{aligned}$$

• *with_even_y*(Q_v) is the final result of the key aggregation and tweaking operations. It corresponds to the output of *GetXonlyPubkey* applied on the final KeyAgg Context. </poem>

The signer's goal is to produce a partial signature corresponding to the final result of key aggregation and tweaking, i.e., the X-only public key *with_even_y*(Q_v).

<poem> For $1 \le i \le v$, we denote the value g computed in the i-th execution of *ApplyTweak* by g_{i-1}. Therefore, g_{i-1} is *-1 mod n* if and only if *is_xonly_t*$_i$ is true and Q_{i-1} has an odd Y coordinate. In other words, g_{i-1} indicates whether Q_{i-1} needed to be negated to apply an X-only tweak:

$f(i-1) = g_{i-1} \cdot Q_{i-1}$ for $1 \le i \le v$.

Furthermore, the *Sign* and *PartialSigVerify* algorithms set value g depending on whether Q_v needed to be negated to produce the (X-only) final output. For consistency, this value g is referred to as g_v in this section.

$with_even_y(Q_v) = g_v \cdot Q_v$.

</poem>

<poem> So, the (X-only) final public key is

$$
\begin{aligned}
&''with_even_y(Q_v) \\
&\quad = g_v \cdot Q_v \\
&\quad = g_v \cdot (f(v-1) + t_v \cdot G) \\
&\quad = g_v \cdot (g_{v-1} \cdot (f(v-2) + t_{v-1} \cdot G) + t_v \cdot G) \\
&\quad = g_v \cdot g_{v-1} \cdot f(v-2) + g_v \cdot (t_v + g_{v-1} \cdot t_{v-1}) \cdot G \\
&\quad = g_v \cdot g_{v-1} \cdot f(v-2) + (sum_{i=v-1..v}\ t_i \cdot prod_{j=i..v}\ g_j) \cdot G \\
&\quad = g_v \cdot g_{v-1} \cdot \ldots \cdot g_1 \cdot f(0) + (sum_{i=1..v}\ t_i \cdot prod_{j=i..v}\ g_j) \cdot G \\
&\quad = g_v \cdot \ldots \cdot g_0 \cdot Q_0 + g_v \cdot tacc_v \cdot G''
\end{aligned}
$$

where $tacc_i$ is computed by *KeyAgg* and *ApplyTweak* as follows:

$$
\begin{aligned}
&''tacc_0 = 0 \\
&tacc_i = t_i + g_{i-1} \cdot tacc_{i-1} \text{ for } i=1..v \bmod n''
\end{aligned}
$$

for which it holds that $g_v \cdot tacc_v = sum_{i=1..v} t_i \cdot prod_{j=i..v} g_j$.

</poem>

<poem> *KeyAgg* and *ApplyTweak* compute

$$
\begin{aligned}
&''gacc_0 = 1 \\
&gacc_i = g_{i-1} \cdot gacc_{i-1} \text{ for } i=1..v \bmod n''
\end{aligned}
$$

So we can rewrite above equation for the final public key as

$with_even_y(Q_v) = g_v \cdot gacc_v \cdot Q_0 + g_v \cdot tacc_v \cdot G$.

</poem>

<poem> Then we have

$$
\begin{aligned}
&''with_even_y(Q_v) - g_v \cdot tacc_v \cdot G \\
&\quad = g_v \cdot gacc_v \cdot Q_0 \\
&\quad = g_v \cdot gacc_v \cdot (a_1 \cdot P_1 + \ldots + a_u \cdot P_u)
\end{aligned}
$$

$= g_v \cdot gacc_v \cdot (a_1 \cdot d_1' \cdot G + ... + a_u \cdot d_u' \cdot G)$
$= sum_{i=1..u}(g_v \cdot gacc_v \cdot a_i \cdot d_i') * G''.$

</poem>

Intuitively, $gacc_i$ tracks accumulated sign flipping and $tacc_i$ tracks the accumulated tweak value after applying the first i individual tweaks. Additionally, g_v indicates whether Q_v needed to be negated to produce the final X-only result. Thus, signer i multiplies its secret key d_i' with $g_v \cdot gacc_v$ in the *Sign* algorithm.

Negation Of The Individual Public Key When Partially Verifying

<poem> As explained in Negation Of The Secret Key When Signing the signer uses a possibly negated secret key

$$d = g_v \cdot gacc_v \cdot d' \mod n$$

when producing a partial signature to ensure that the aggregate signature will correspond to an aggregate public key with even Y coordinate. </poem>

<poem> The *PartialSigVerifyInternal* algorithm is supposed to check

$$s \cdot G = Re_* + e \cdot a \cdot d \cdot G.$$

</poem>

<poem> The verifier doesn't have access to $d \cdot G$ but can construct it using the individual public key pk as follows: ''d·G

$= g_v \cdot gacc_v \cdot d' \cdot G$
$= g_v \cdot gacc_v \cdot cpoint(pk)$*Note that the aggregate public key and list of tweaks are inputs to partial signature verification, so the verifier can also construct* g_v *and* $gacc_v$ *''.*

</poem>

Dealing with Infinity in Nonce Aggregation

If the nonce aggregator provides *aggnonce = bytes(33,0) || bytes(33,0)*, either the nonce aggregator is dishonest or there is at least one dishonest signer (except with negligible probability). If signing aborted in this case, it would be impossible to determine who is dishonest. Therefore, signing continues so that the culprit is revealed when collecting and verifying partial signatures.

However, the final nonce R of a BIP340 Schnorr signature cannot be the point at infinity. If we would nonetheless allow the final nonce to be the point at infinity, then the scheme would lose the following property: if *PartialSigVerify* succeeds for all partial signatures, then *PartialSigAgg* will return a valid Schnorr signature. Since this is a valuable feature, we modify MuSig2* (which is defined in the appendix of the MuSig2 paper) to avoid producing an invalid Schnorr signature while still allowing detection of the dishonest signer: In *GetSessionValues*, if the final nonce R would be the point at infinity, set it to the generator instead (an arbitrary choice).

This modification to *GetSessionValues* does not affect the unforgeability of the scheme. Given a successful adversary against the unforgeability game (EUF-CMA) for the modified scheme, a reduction can win the unforgeability game for the original scheme by simulating the modification towards the adversary: When the adversary provides $aggnonce' = bytes(33, 0) \,||\, bytes(33, 0)$, the reduction sets $aggnonce = cbytes_ext(G) \,||\, bytes(33, 0)$. For any other $aggnonce'$, the reduction sets $aggnonce = aggnonce'$. (The case that the adversary provides an $aggnonce' \neq bytes(33, 0) \,||\, bytes(33, 0)$ but nevertheless R' in *GetSessionValues* is the point at infinity happens only with negligible probability.)

Choosing the Size of the Nonce

The MuSig2 paper contains two security proofs that apply to different variants of the scheme. The first proof relies on the random oracle model (ROM) and applies to a scheme variant where each signer's nonce consists of four elliptic curve points. The second proof requires a stronger model, namely the combination of the ROM and the algebraic group model (AGM), and applies to an optimized scheme variant where the signers' nonces consist of only two points. This proposal uses the latter, optimized scheme variant. Relying on the stronger model is a legitimate choice for the following reasons:

First, an approach widely taken is interpreting a Forking Lemma proof in the ROM merely as design justification and ignoring the loss of security due to the Forking Lemma. If one believes in this approach, then the ROM may not be the optimal model in the first place because some parts of the concrete security bound are arbitrarily ignored. One may just as well move to the ROM+AGM model, which produces bounds close to the best-known attacks, e.g., for Schnorr signatures.

Second, as of this writing, there is no instance of a serious cryptographic scheme with a security proof in the AGM that is not secure in practice. There are, however, insecure toy schemes with AGM security proofs, but those explicitly violate the requirements of the AGM. Broken AGM proofs of toy schemes provide group elements to the adversary without declaring them as group element inputs. In contrast, in MuSig2, all group elements that arise in the scheme are known to the adversary and declared as group element inputs. A scheme very similar to MuSig2 and with two-point nonces was independently proven secure in the ROM and AGM by Alper and Burdges.

Backwards Compatibility

This document proposes a standard for the MuSig2 multi-signature scheme that is compatible with BIP340. MuSig2 is *not* compatible with ECDSA signatures traditionally used in Bitcoin.

Changelog

To help implementers understand updates to this document, we attach a version number that resembles *semantic versioning* (`MAJOR.MINOR.PATCH`). The `MAJOR` version is incremented if changes to the BIP are introduced that are incompatible with prior versions. An exception to this rule is `MAJOR` version zero (0.y.z) which is for development and does not need to be incremented if backwards incompatible changes are introduced. The `MINOR` version is incremented whenever the inputs or the output of an algorithm changes in a backward-compatible way or new backward-compatible functionality is added. The `PATCH` version is incremented for other changes that are noteworthy (bug fixes, test vectors, important clarifications, etc.).

- **1.0.4** (2026-08-19):
  - Fix two minor bugs in the specification of *PartialSigAgg*.

- **1.0.3** (2026-01-05):
  - Fix minor bugs in the specification of *DeterministicSign*.
- **1.0.2** (2024-07-22):
  - Fix minor bug in the specification of *DeterministicSign* and add small improvement to a *PartialSigAgg* test vector.
- **1.0.1** (2024-05-14):
  - Fix minor issue in *PartialSigVerify* vectors.
- **1.0.0** (2023-03-26):
  - Number 327 was assigned to this BIP.
- **1.0.0-rc.4** (2023-03-02):
  - Add expected value of *pubnonce* to *NonceGen* test vectors.
- **1.0.0-rc.3** (2023-02-28):
  - Improve *NonceGen* test vectors by not using an all-zero hex string as *rand_* values. This change addresses potential issues in some implementations that interpret this as a special value indicating uninitialized memory or a broken random number generator and therefore return an error.
  - Fix invalid length of a *pubnonce* in the *PartialSigVerify* test vectors.
  - Improve *KeySort* test vector.
  - Add explicit *IndividualPubkey* algorithm.
  - Rename KeyGen Context to KeyAgg Context.
- **1.0.0-rc.2** (2022-10-28):
  - Fix vulnerability that can occur in certain unusual scenarios (see bitcoin-dev mailing list: Add mandatory *pk* argument to *NonceGen*, append *pk* to *secnonce* and check in *Sign* that the *pk* in *secnonce* matches. Update test vectors.
  - Make sure that signer's key is in list of individual public keys by adding failure case to *GetSessionKeyAggCoeff* and add test vectors.
- **1.0.0-rc.1** (2022-10-03): Submit draft BIP to the BIPs repository
- **0.8.6** (2022-09-15): Clarify that implementations do not need to support every feature and add a test vector for signing with a tweaked key
- **0.8.5** (2022-09-05): Rename some functions to improve clarity.
- **0.8.4** (2022-09-02): Make naming of nonce variants *R* in specifications of the algorithms and reference code easier to read and more consistent.
- **0.8.3** (2022-09-01): Overwrite *secnonce* in *sign* reference implementation to help prevent accidental reuse and add test vector for invalid *secnonce*.
- **0.8.2** (2022-08-30): Fix *KeySort* input length and add test vectors
- **0.8.1** (2022-08-26): Add *DeterministicSign* algorithm
- **0.8.0** (2022-08-26): Switch from X-only to plain public key for individual public keys. This requires updating a large portion of the test vectors.
- **0.7.2** (2022-08-17): Add *NonceGen* and *Sign*/*PartialSigVerify* test vectors for messages longer than 32 bytes.
- **0.7.1** (2022-08-10): Extract test vectors into separate JSON file.
- **0.7.0** (2022-07-31): Change *NonceGen* such that output when message is not present is different from when message is present but has length 0.
- **0.6.0** (2022-07-31): Allow variable length messages, change serialization of the message in the *NonceGen* hash function, and add test vectors
- **0.5.2** (2022-06-26): Fix *aggpk* in *NonceGen* test vectors.
- **0.5.1** (2022-06-22): Rename "ordinary" tweaking to "plain" tweaking.
- **0.5.0** (2022-06-21): Separate ApplyTweak from KeyAgg and introduce KeyGen Context.

- **0.4.0** (2022-06-20): Allow the output of NonceAgg to be infinity and add test vectors
- **0.3.2** (2022-06-02): Add a lot of test vectors and improve handling of invalid contributions in reference code.
- **0.3.1** (2022-05-24): Add *NonceGen* test vectors
- **0.3.0** (2022-05-24): Hash *i - 1* instead of *i* in *NonceGen*
- **0.2.0** (2022-05-19): Change order of arguments in *NonceGen* hash function
- **0.1.0** (2022-05-19): Publication of draft BIP on the bitcoin-dev mailing list

Footnotes

Acknowledgements

We thank Brandon Black, Riccardo Casatta, Sivaram Dhakshinamoorthy, Lloyd Fournier, Russell O'Connor, and Pieter Wuille for their contributions to this document.

```
BIP: 137
Layer: Applications
Title: Signatures of Messages using Private Keys
Authors: Christopher Gilliard <christopher.gilliard@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2019-02-16
License: BSD-2-Clause
```

BIP 137

Abstract

This document describes a signature format for signing messages with Bitcoin private keys.

The specification is intended to describe the standard for signatures of messages that can be signed and verified between different clients that exist in the field today. Note: that a new signature format has been defined which has a number of advantages over this BIP, but to be backwards compatible with existing implementations this BIP will be useful. See BIP 322 [1] for full details on the new signature scheme.

One of the key problems in this area is that there are several different types of Bitcoin addresses and without introducing specific standards it is unclear which type of address format is being used. See [2]. This BIP will attempt to address these issues and define a clear and concise format for Bitcoin signatures.

Copyright

This BIP is licensed under the 2-clause BSD license.

Motivation

Since Bitcoin private keys can not only be used to sign Bitcoin transactions, but also any other message, it has become customary to use them to sign various messages for differing purposes. Some applications of signing messages with a Bitcoin private key are as follows: proof of funds for collateral, credit worthiness, entrance to events, airdrops, audits as well as other applications. While there was no BIP written for how to digitally sign

messages with Bitcoin private keys with P2PKH addresses it is a fairly well understood process, however with the introduction of Segwit (both in the form of P2SH and bech32) addresses, it is unclear how to distinguish a P2PKH, P2SH, or bech32 address from one another. This BIP proposes a standard signature format that will allow clients to distinguish between the different address formats.

Specification

Background on ECDSA Signatures

(For readers who already understand how ECDSA signatures work, you can skip this section as this is only intended as background information.) Elliptic Curve Digital Signature Algorithm or ECDSA is a cryptographic algorithm used by Bitcoin to ensure that funds can only be spent by their rightful owners.

A few concepts related to ECDSA:

private key: A secret number, known only to the person that generated it. A private key is essentially a randomly generated number. In Bitcoin, someone with the private key that corresponds to funds on the block chain can spend the funds. In Bitcoin, a private key is a single unsigned 256 bit integer (32 bytes).

public key: A number that corresponds to a private key, but does not need to be kept secret. A public key can be calculated from a private key, but not vice versa. A public key can be used to determine if a signature is genuine (in other words, produced with the proper key) without requiring the private key to be divulged. In Bitcoin, public keys are either compressed or uncompressed. Compressed public keys are 33 bytes, consisting of a prefix either 0x02 or 0x03, and a 256-bit integer called x. The older uncompressed keys are 65 bytes, consisting of constant prefix (0x04), followed by two 256-bit integers called x and y (2 * 32 bytes). The prefix of a compressed key allows for the y value to be derived from the x value.

signature: A number that proves that a signing operation took place. A signature is mathematically generated from a hash of something to be signed, plus a private key. The signature itself is two numbers known as r and s. With the public key, a mathematical algorithm can be used on the signature to determine that it was originally produced from the hash and the private key, without needing to know the private key. Signatures are either 73, 72, or 71 bytes long, with probabilities approximately 25%, 50% and 25% respectively, although sizes even smaller than that are possible with exponentially decreasing probability. Source [3].

Conventions with signatures used in Bitcoin

Bitcoin signatures have the r and s values mentioned above, and a header. The header is a single byte and the r and s are each 32 bytes so a signature's size is 65 bytes. The header is used to specify information about the signature. It can be thought of as a bitmask with each bit in this byte having a meaning. The serialization format of a Bitcoin signature is as follows:

[1 byte of header data][32 bytes for r value][32 bytes for s value]

The header byte has a few components to it. First, it stores something known as the recId. This value is stored in the least significant 2 bits of the header. If the header is between a value of 31 and 34, this indicates that it is a compressed address. If the header value is between 35 and 38 inclusive, it is a p2sh segwit address. If the header value is between 39 and 42, it is a bech32 address.

Procedure for signing/verifying a signature

As noted above the signature is composed of three components, the header, r and s values. r/s can be computed with standard ECDSA library functions. Part of the header includes something called a recId. This is part of every ECDSA signature and should be generated by the ECDSA library. The recId is a number between 0 and 3 inclusive. The header is the recId plus a constant which indicates what type of Bitcoin address this is. For P2PKH address using an uncompressed public key this value is 27. For P2PKH address using compressed public key this value is 31. For P2SH-P2WPKH this value is 35 and for P2WPKH (version 0 witness) address this value is 39. So, you have the following ranges:

- 27-30: P2PKH uncompressed
- 31-34: P2PKH compressed
- 35-38: Segwit P2SH
- 39-42: Segwit Bech32

To verify a signature, the recId is obtained by subtracting this constant from the header value.

Sample Code for processing a signature

Note: this code is a modification of the BitcoinJ code which is written in java.

```
   public static ECKey signedMessageToKey(String message, String signatureBase64) throws SignatureException
       byte[] signatureEncoded;
       try {
           signatureEncoded = Base64.decode(signatureBase64);
       } catch (RuntimeException e) {
           // This is what you get back from Bouncy Castle if base64 doesn't decode :(
           throw new SignatureException("Could not decode base64", e);
       }
       // Parse the signature bytes into r/s and the selector value.
       if (signatureEncoded.length < 65)
           throw new SignatureException("Signature truncated, expected 65 bytes and got " + signatureEncoded
       int header = signatureEncoded[0] & 0xFF;
       // The header byte: 0x1B = first key with even y, 0x1C = first key with odd y,
       //                  0x1D = second key with even y, 0x1E = second key with odd y
       if (header < 27 || header > 42)
           throw new SignatureException("Header byte out of range: " + header);
       BigInteger r = new BigInteger(1, Arrays.copyOfRange(signatureEncoded, 1, 33));
       BigInteger s = new BigInteger(1, Arrays.copyOfRange(signatureEncoded, 33, 65));
       ECDSASignature sig = new ECDSASignature(r, s);
       byte[] messageBytes = formatMessageForSigning(message);
       // Note that the C+
+ code doesn't actually seem to specify any character encoding. Presumably it's whatever
       // JSON-SPIRIT hands back. Assume UTF-8 for now.
       Sha256Hash messageHash = Sha256Hash.twiceOf(messageBytes);
       boolean compressed = false;
       // this section is added to support new signature types
       if(header>= 39) // this is a bech32 signature
```

```
        {
            header -= 12;
            compressed = true;
        } // this is a segwit p2sh signature
        else if(header >= 35)
        {
            header -= 8;
            compressed = true;
        } // this is a compressed key signature
        else if (header >= 31) {
            compressed = true;
            header -= 4;
        }
        int recId = header - 27;
        ECKey key = ECKey.recoverFromSignature(recId, sig, messageHash, compressed);
        if (key == null)
            throw new SignatureException("Could not recover public key from signature");
        return key;
    }
```

Backwards Compatibility

Since this format includes P2PKH keys, it is backwards compatible, but keep in mind some software has checks for ranges of headers and will report the newer segwit header types as errors.

Implications

Message signing is an important use case and potentially underused due to the fact that, up until now, there has not been a formal specification for how wallets can sign messages using Bitcoin private keys. Bitcoin wallets should be interoperable and use the same conventions for determining a signature's validity. This BIP can also be updated as new signature formats emerge.

Acknowledgements

- Konstantin Bay - review
- Holly Casaletto - review
- James Bryrer - review

Note that the background on ECDSA signatures was taken from en.bitcoin.it and code sample modified from BitcoinJ.

References

[1] - https://github.com/bitcoin/bips/blob/master/bip-0322.mediawiki

[2] - https://github.com/bitcoin/bitcoin/issues/10542

[3] - https://en.bitcoin.it/wiki/Elliptic_Curve_Digital_Signature_Algorithm

```
  BIP: 322
  Layer: Applications
  Title: Generic Signed Message Format
  Authors: Karl-Johan Alm <karljohan-alm@garage.co.jp>
           Oliver Gugger <gugger@gmail.com>
  Status: Complete
  Type: Specification
  Assigned: 2018-09-10
  License: CC0-1.0
  Discussion: 2018-03-14: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2018-March/
015818.html
              2019-07-23: https://github.com/bitcoin/bitcoin/pull/16440
              2022-01-13: https://github.com/bitcoin/bitcoin/pull/24058
              2022-08-06: https://bitcointalk.org/index.php?topic=5408898.0
              2024-05-04: https://groups.google.com/g/bitcoindev/c/RCi1Exs0ZvQ/m/vp6Xo36aBwAJ
              2025-05-10: https://bitcoin.stackexchange.com/questions/126277/where-can-i-use-
bip322-to-sign-a-message-to-verify-a-multisig-address
              2026-04-20: https://groups.google.com/g/bitcoindev/c/qd6BNz9gxCk/m/k1fHq4RKAQAJ
  Version: 2.1.0
  Requires: 174, 340, 341, 342
```

BIP 322

Abstract

A standard for interoperable signed messages based on the Bitcoin Script format, either for proving availability of funds, or for committing to a message as the intended recipient of funds sent to the invoice address.

Motivation

The current message signing standard only works for P2PKH (1…) invoice addresses. We propose to extend and generalize the standard by using an approach based on Bitcoin Script. This ensures that any coins, no matter what script they are controlled by, can in principle be signed for. For easy interoperability with existing signing hardware, we also define a signature message format that resembles a Bitcoin transaction (except that it contains an invalid input, so it cannot be spent on any real network).

Additionally, the current message signature format uses ECDSA signatures that do not commit to the public key, meaning that they do not actually prove knowledge of any secret keys. (Indeed, valid signatures can be tweaked by third parties to become valid signatures on certain related keys.)

Ultimately, no message signing protocol can actually prove control of funds, both because a signature is obsolete as soon as it is created, and because the possessor of a secret key may be willing to sign messages on others' behalf even if it would not sign actual transactions. No message signing protocol can fix these limitations.

Finally, this BIP only addresses the use case where a signer shows they will be able to control funds sent to the invoice address. Proving that a signer sent a prior transaction is not possible using this BIP.

Terminology

In the context of this BIP, whenever the word “signature” or similar is used, it refers to the output of the signing process described below, and, depending on the script type of the `message_challenge`, is either a full transaction input witness stack, a full transaction, or a PSBT packet that can be validated against a Bitcoin Script Interpreter. Such a “signature” may or may not contain an actual cryptographic (ECDSA or Schnorr) signature, depending on what is required to satisfy the script corresponding to the `message_challenge`.

Types of Signatures

This BIP specifies three formats for signing messages: *legacy*, *simple* and *full*. Additionally, a variant of the *full* format can be used to demonstrate control over a set of UTXOs.

	Compatible script types	**Signature prefix**	**Signature format**
Legacy	`P2PKH`, `P2SH-P2WPKH`[1], `P2WPKH`[1]	n/a	compact, public key recoverable ECDSA signature, base64-encoded
Simple	`P2WPKH`, `P2WSH`[2], `P2TR`[2]	`smp`	witness stack, consensus-encoded and base64-encoded
Full	`all`	`ful`	full `to_sign` transaction, consensus-encoded and base64-encoded
Full (Proof of Funds)	`all`	`pof`	full finalized PSBT of the `to_sign` transaction, consensus-encoded and base64-encoded

[1]: Possible on a technical level but SHOULD NOT be used anymore in the context of this BIP, see section Legacy below.
[2]: Excluding time lock scripts.

Signers MUST prefix the signature with the variant that was used to create the signature. To support backward compatibility with implementations of this BIP before it was finalized, a verifier might assume the *simple* variant in the absence of a prefix.

Legacy

New proofs SHOULD use the new format for all invoice address formats, including P2PKH.

The legacy format MAY be used, but MUST be restricted to the legacy P2PKH invoice address format.

Simple

A *simple* signature consists of a witness stack, consensus encoded as a vector of vectors of bytes, and base64-encoded, prefixed by the variant (`smp`). Validators should construct `to_spend` and `to_sign` as defined below, with default values for all fields except that

- ``message_hash`` is a BIP340-tagged hash of the message, as specified below
- ``message_challenge`` in ``to_spend`` is set to the scriptPubKey being signed with

- ``message_signature`` in ``to_sign`` is set to the provided simple signature.

and then proceed as they would for a full signature.

Full

Full signatures follow an analogous specification to the BIP325 challenges and solutions used by Signet.

Let there be two virtual transactions `to_spend` and `to_sign`.

The `to_spend` transaction is:

```
   nVersion = 0
   nLockTime = 0
   vin[0].prevout.hash = 0000...000
   vin[0].prevout.n = 0xFFFFFFFF
   vin[0].nSequence = 0
   vin[0].scriptSig = OP_0 PUSH32[ message_hash ]
   vin[0].scriptWitness = []
   vout[0].nValue = 0
   vout[0].scriptPubKey = message_challenge
```

where `message_hash` is a BIP340-tagged hash of the message, i.e., sha256_tag(m), where tag = `BIP0322-signed-message` and `m` is the message as-is without length prefix or null terminator, and `message_challenge` is the (public) key script to be proven.

The `to_sign` transaction is:

```
   nVersion = 0 or (FULL format only) as appropriate (e.g., 2 for time locks)
   nLockTime = 0 or (FULL format only) as appropriate (for time locks)
   vin[0].prevout.hash = to_spend.txid
   vin[0].prevout.n = 0
   vin[0].nSequence = 0 or (FULL format only) as appropriate (for time locks)
   vin[0].scriptSig = [] or (FULL format only) as appropriate (for non segwit-
native transactions)
   vin[0].scriptWitness = message_signature
   vout[0].nValue = 0
   vout[0].scriptPubKey = OP_RETURN
```

A *full* signature consists of the variant-prefixed (`ful`) base64-encoding of the `to_sign` transaction in standard network serialisation once it has been signed.

Full (Proof of Funds)

The Proof of Funds variant extends the basic scheme: in addition to signing a message under a single address's key, the signer proves control over an arbitrary set of UTXOs. This UTXO set is chosen freely by the signer and MAY be associated with the signing address (the `message_challenge`). For example, it may consist of outputs paid to that address, but any UTXOs the signer wants to show control over are permitted. In any case,

however, the UTXO list does not aim to prove completeness (e.g., it does NOT mean: “these are all UTXOs that exist for an address”), nor that they are unspent (e.g., a validator must consult the blockchain to verify that).

A signer may construct a proof of funds, demonstrating control of a set of UTXOs, by constructing a full signature as above, with the following modifications.

- The ``to_spend`` transaction is represented as a finalized PSBT instead of a raw transaction (see BIP174 for details on the finalization process).
- All outputs that the signer wishes to demonstrate control of are included as additional inputs of ``to_sign``, and their witness and scriptSig data should be set as though these outputs were actually being spent.
- The Non-Witness or Witness UTXO fields (as appropriate for the type) of each additional input must be set to the corresponding UTXO.
- As an optimization for large sets of Non-Witness UTXOs that spend outputs from the same transaction, the Non-Witness UTXO field may be omitted for any input that spends an output from the same transaction as an input earlier in the list.

A *full Proof of Funds* signature consists of the variant-prefixed (`pof`) base64-encoding of the finalized PSBT once it has been signed.

Unlike an ordinary signature, validators of a proof of funds need access to the current UTXO set, to learn that the claimed inputs exist on the blockchain and remain unspent. An offline validator therefore can only attest to the cryptographic validity of the additional inputs' witness stack, but not its blockchain state. An attested list of UTXOs can also never prove that there do not exist more UTXOs for a certain address.

Detailed Specification

For all signature types, except legacy, the `to_spend` and `to_sign` transactions must be valid transactions which pass all consensus checks, except of course that the output with prevout `000...000:FFFFFFFF` does not exist.

Verification

A validator is given as input an address *A*, signature *s* and message *m*, and outputs one of three states:

- *`valid at time T and age S`* indicates that the signature has set timelocks but is otherwise valid
- *`inconclusive`* means the validator was unable to check the scripts
- *`invalid`* means that some check failed

Verification Process

Validation consists of the following steps:

1. Basic validation
    1. Compute the transaction `to_spend` from *m* and *A*
    2. Decode *s* as the transaction `to_sign`

3. If *s* was a full transaction or PSBT, confirm all fields are set as specified above; in particular that
    - `to_sign` has at least one input and its first input spends the output of `to_spend`
    - `to_sign` with more than one input has an appropriate Witness UTXO or Non-Witness UTXO for each input
        - If (based on the input type) a Non-Witness UTXO is required but not provided, check if the first input with the same transaction ID has a Non-Witness UTXO set and use that; fail validation if no such Non-Witness UTXO can be found
    - `to_sign` has exactly one output, as specified above
4. Confirm that the two transactions together satisfy all consensus rules, except for `to_spend`'s missing input, and except that *nSequence* of `to_sign`'s first input and *nLockTime* of `to_sign` are not checked.

2. (Optional) If the validator does not have a full script interpreter, it should check that it understands all scripts being satisfied. If not, it should stop here and output *inconclusive*.
3. Check the **required rules**:
    1. All signatures MUST use the `SIGHASH_ALL` flag, unless the output type supports `SIGHASH_DEFAULT`, which then MAY be used alternatively (e.g., BIP341 P2TR).
    2. The use of `CODESEPARATOR` or `FindAndDelete` is forbidden.
    3. `LOW_S`, `STRICTENC` and `NULLFAIL`: valid ECDSA signatures must be strictly DER-encoded and have a low-S value; invalid ECDSA signature must be the empty push
    4. `MINIMALDATA`: all pushes must be minimally encoded
    5. `CLEANSTACK`: require that only a single stack element remains after evaluation
    6. `MINIMALIF`: the argument of `IF/NOTIF` must be exactly 0x01 or empty push
    7. If any of the above steps failed, the validator should stop and output the *invalid* state.
4. Check the **upgradeable rules**
    1. The version of `to_sign` must be 0 or 2.
    2. The use of NOPs reserved for upgrades is forbidden.
    3. The use of Segwit versions greater than 1 is forbidden.
    4. The use of the taproot upgrade mechanisms is forbidden: an unknown leaf version and the annex (BIP341), `OP_SUCCESSx` and an unknown public key type (BIP342).
    5. If any of the above steps failed, the validator should stop and output the *inconclusive* state.
5. Let *T* be the nLockTime of `to_sign` and *S* be the nSequence of the first input of `to_sign`. Output the state *valid at time T and age S*.
    1. For *full Proof of Funds* signatures, a validator MAY additionally report any non-zero nSequence values on the Proof of Funds inputs (i.e., `vin[1..n]`), and MAY verify them against the current chain state when the UTXO set is available; such checks are advisory and do not affect the output state.

Signing

Signers who control an address *A* who wish to sign a message *m* act as follows:

1. `They construct ``to_spend`` and ``to_sign`` as specified above, using the scriptPubKey of A for ``message_challenge`` and tagged hash of m as ``message_hash``.`
2. `Optionally, they may set nVersion/nLockTime of ``to_sign`` or nSequence of its first input.`

3. Optionally, they may add any additional inputs to ``to_sign`` that they wish to prove control of.
4. They satisfy ``to_sign`` as they would any other transaction.

They then encode their signature, choosing either *simple*, *full* or *full-pof* as follows:

- If they added no inputs to ``to_sign``, left nVersion, nSequence and nLockTime at 0, and *A* is a "native" Segwit address (P2WPKH, P2WSH, P2TR), then they may base64-encode ``message_signature`` with ``smp`` as prefix.
- If they added no inputs to ``to_sign``, they may base64-encode ``to_sign`` with ``ful`` as prefix.
- Otherwise, they must base64-encode the finalized PSBT of ``to_sign`` with ``pof`` as prefix.

PSBT-based signing

A valid witness stack for a multisig address must be constructed by coordinating different signers to produce a partial signature each. The coordination procedure is not specified by this BIP, but due to the use of PSBTs it should closely resemble the coordination of signing a multisig transaction for publishing to the network.

The main difference is a new global PSBT field and the way a signer presents the transaction signing request to the user. The new global type is defined as follows:

Name	**<keytype>**	**<keydata>**	**<keydata> Description**	**<valuedata>**	**<valuedata> Description**	**Versions Requiring Inclusion**	**Ver Rec Exc**
Generic Signed Message	PSBT_GLOBAL_GENERIC_SIGNED_MESSAGE = 0x09	None	No key data	<bytes message>	The UTF-8 encoded message to be signed.		

PSBT creator

The **transaction creator** of a BIP322 PSBT must follow these steps:

1. They construct ``to_spend`` and ``to_sign`` as specified above, using the scriptPubKey of *A* for ``message_challenge`` and tagged hash of *m* as ``message_hash``.
2. Optionally, they may set nVersion/nLockTime of ``to_sign`` or nSequence of its first input.
3. Optionally, they may add any additional inputs to ``to_sign`` that they wish to prove control of.
4. They set the appropriate ``witness_utxo`` and ``non_witness_utxo`` fields of the first input, using the ``to_spend`` transaction as a ``non_witness_utxo`` or the first output of the ``to_spend`` transaction as ``witness_utxo``.
5. They set the appropriate ``witness_utxo`` and ``non_witness_utxo`` fields of each additional input.

6. They set the appropriate PSBT input and global fields as required by the signers(s) to produce a partial signature.
7. They set the ``PSBT_GLOBAL_GENERIC_SIGNED_MESSAGE`` field, using the full UTF-8-encoded message as the ``valuedata``.

    1. There is no specified maximum length of an input's ``valuedata`` or a PSBT as a whole in BIP174, but different signers might impose safety limits. It is recommended to use a maximum length of a few kilobytes to maximize compatibility. Very large messages should be committed to by hash instead.

PSBT signer

A **transaction signer** of a BIP322 PSBT must follow these steps:

1. They decode the base64-encoded PSBT as specified in BIP174.
2. If they detect the following properties (all must be true, otherwise this is NOT a BIP322 PSBT and they should treat it as an ordinary PSBT):

    1. The PSBT has the ``PSBT_GLOBAL_GENERIC_SIGNED_MESSAGE`` field set. Extract and use as ``message`` in the next steps.

    2. The first PSBT input has either a ``witness_utxo`` or a ``non_witness_utxo`` field set and the ``scriptPubKey`` can be extracted. Use that as ``message_challenge`` in the next steps.

    3. The first PSBT input has ``prevout.n = 0``.

    4. The first PSBT input has ``prevout.hash = to_spend.txid`` where ``to_spend.txid`` is constructed using the rules described above using the ``message`` and ``message_challenge`` from the previous steps.

    5. The PSBT's unsigned transaction has a single output with a value of ``0`` and the ``scriptPubKey`` set to ``OP_RETURN`` (``0x6a``).

3. If all of the above steps are true, the signer must inform the user about the message they are signing and the address they are signing for.

    1. Even though the message being signed is a transaction, the user interaction (e.g., the steps and messages shown on a hardware signing device's screen) should resemble the steps to sign a legacy message, not the steps for signing a transaction.

    2. Example: Instead of showing "spending 0 satoshi from address <challenge_address>" the device should show "signing message <message> for address <challenge_address>".

4. Upon user approval, the signer adds a partial signature for each input it is capable of signing.

PSBT finalizer

A **transaction finalizer** of a BIP322 PSBT must follow these steps:

1. They decode the base64-encoded PSBT as specified in BIP174.
2. They finalize the PSBT as specified in BIP174.
3. They then encode the signature following the same steps as described in Signing above.

Compatibility

This specification is backwards-compatible with the legacy signmessage/verifymessage specification through the special case as described above. To support backward compatibility with implementations of this BIP before it was finalized, a verifier might assume the *simple* variant in the absence of a prefix.

Reference implementation

- Bitcoin Core pull request (basic support) at: https://github.com/bitcoin/bitcoin/pull/24058
- btcd pull request (complete support, source of test vectors) at: https://github.com/btcsuite/btcd/pull/2521

Acknowledgements

Thanks to David Harding, Jim Posen, Kalle Rosenbaum, Pieter Wuille, Andrew Poelstra, Luke Dashjr, and many others for their feedback on the specification.

References

1. Original mailing list thread: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2018-March/015818.html

Changelog

- **2.1.0** (2026-10-23):
  - Add the taproot upgrade mechanisms (unknown leaf version, annex, OP_SUCCESSx, unknown public key type) to the upgradeable rules, so that they give the *inconclusive* state.
- **2.0.0** (2026-06-04):
  - Clarify that the message_challenge is *not* optional for "Proof of Funds".
- **1.0.0** (2026-04-15):
  - Mark Complete
  - Breaking change: Add human-readable prefix to encoded signature
  - Breaking change: Format of "Proof of Funds" signatures to be base64-encoded finalized PSBT
  - Add new PSBT global field for PSBT-based signing
- **0.0.1** (2018-09-10):
  - Proposed as draft

Copyright

This document is licensed under the Creative Commons CC0 1.0 Universal license.

Test vectors

Basic test vectors for message hashing, transaction hashes and “simple” variant test cases can be found in `basic-test-vectors.json`.

Generated test vectors for more “simple” and “full” variant test cases can be found in `generated-test-vectors.json`.

They were generated using this code.

Scripts, Miniscript, and Policy

The last chapter is the practical layer on top of script: how wallets build standard multisig, how they sort keys and inputs so two implementations produce the same transaction, how a sender asks for replace-by-fee, and how a descriptor wallet describes a whole policy instead of a single script.

BIP 11 is M-of-N standard transactions, the original multisig that P2SH later made usable. BIP 67 sorts the public keys so two wallets independently derive the same P2SH address. BIP 69 sorts inputs and outputs lexicographically so a transaction has a canonical order. BIP 125 is opt-in full replace-by-fee — the signal that a transaction may be replaced with a higher fee.

BIP 388 sits at the end. Wallet policies are how a descriptor wallet talks about accounts, change, and multisig templates as one object. Descriptors themselves live in the earlier chapter; I did not repeat BIP 380 here.

Miniscript has a BIP number, but it is still a draft, so it is not in this book. When you want the compiler that sits between a policy and a script, that is the document to read next, upstream.

In this chapter:

- BIP 11 — M-of-N Standard Transactions
- BIP 67 — Deterministic P2SH multisig through public key sorting
- BIP 69 — Lexicographical Indexing of Transaction Inputs and Outputs
- BIP 125 — Opt-in Full Replace-by-Fee Signaling
- BIP 388 — Wallet Policies for Descriptor Wallets

```
BIP: 11
Layer: Applications
Title: M-of-N Standard Transactions
Authors: Gavin Andresen <gavinandresen@gmail.com>
Status: Deployed
Type: Specification
Assigned: 2011-10-18
Discussion: 2011-10-02
```

BIP 11

Abstract

This BIP proposes M-of-N-signatures required transactions as a new 'standard' transaction type.

Motivation

Enable secured wallets, escrow transactions, and other use cases where redeeming funds requires more than a single signature.

A couple of motivating use cases:

- A wallet secured by a "wallet protection service" (WPS). 2-of-2 signatures required transactions will be used, with one signature coming from the (possibly compromised) computer with the wallet and the second signature coming from the WPS. When sending protected bitcoins, the user's bitcoin client will contact the WPS with the proposed transaction and it can then contact the user for confirmation that they initiated the transaction and that the transaction details are correct. Details for how clients and WPS's communicate are outside the scope of this BIP. Side note: customers should insist that their wallet protection service provide them with copies of the private key(s) used to secure their wallets that they can safely store off-line, so that their coins can be spent even if the WPS goes out of business.
- Three-party escrow (buyer, seller, and trusted dispute agent). 2-of-3 signatures required transactions will be used. The buyer and seller and agent will each provide a public key, and the buyer will then send coins into a 2-of-3 CHECKMULTISIG transaction and send the seller and the agent the transaction id. The seller will fulfill their obligation and then ask the buyer to co-sign a transaction (already signed by seller) that sends the tied-up coins to him (seller).
  If the buyer and seller cannot agree, then the agent can, with the cooperation of either buyer or seller, decide what happens to the tied-up coins. Details of how buyer, seller, and agent communicate to gather signatures or public keys are outside the scope of this BIP.

Specification

A new standard transaction type (scriptPubKey) that is relayed by clients and included in mined blocks:

```
m {pubkey}...{pubkey} n OP_CHECKMULTISIG
```

But only for n less than or equal to 3.

OP_CHECKMULTISIG transactions are redeemed using a standard scriptSig:

```
OP_0 ...signatures...
```

(OP_0 is required because of a bug in OP_CHECKMULTISIG; it pops one too many items off the execution stack, so a dummy value must be placed on the stack).

The current Satoshi bitcoin client does not relay or mine transactions with scriptSigs larger than 200 bytes; to accommodate 3-signature transactions, this will be increased to 500 bytes.

Rationale

OP_CHECKMULTISIG is already an enabled opcode, and is the most straightforward way to support several important use cases.

One argument against using OP_CHECKMULTISIG is that old clients and miners count it as "20 sigops" for purposes of computing how many signature operations are in a block, and there is a hard limit of 20,000 sigops per block– meaning a maximum of 1,000 multisig transactions per block. Creating multisig transactions using multiple OP_CHECKSIG operations allows more of them per block.

The counter-argument is that these new multi-signature transactions will be used in combination with OP_EVAL (see the OP_EVAL BIP), and **will** be counted accurately. And in any case, as transaction volume rises the hard-coded maximum block size will have to be addressed, and the rules for counting number-of-signature-operations-in-a-block can be addressed at that time.

A weaker argument is OP_CHECKMULTISIG should not be used because it pops one too many items off the stack during validation. Adding an extra OP_0 placeholder to the scriptSig adds only 1 byte to the transaction, and any alternative that avoids OP_CHECKMULTISIG adds at least several bytes of opcodes.

Implementation

OP_CHECKMULTISIG is already supported by old clients and miners as a non-standard transaction type.

https://github.com/gavinandresen/bitcoin-git/tree/77f21f1583deb89bf3fffe80fe9b181fedb1dd60

Post History

- OP_EVAL proposal

```
BIP: 67
Layer: Applications
Title: Deterministic Pay-to-script-hash multi-signature addresses through public key sorting
Authors: Thomas Kerin <me@thomaskerin.io>
         Jean-Pierre Rupp <root@haskoin.com>
         Ruben de Vries <ruben@rubensayshi.com>
Status: Complete
Type: Specification
Assigned: 2015-02-08
License: PD
```

BIP 67

Abstract

This BIP describes a method to deterministically generate multi-signature pay-to-script-hash transaction scripts. It focuses on defining how the public keys must be encoded and sorted so that the redeem script and corresponding P2SH address are always the same for a given set of keys and number of required signatures.

Motivation

Pay-to-script-hash (BIP-0011BIP-0011) is a transaction type that allows funding of arbitrary scripts, where the recipient carries the cost of fee's associated with using longer, more complex scripts.

Multi-signature pay-to-script-hash transactions are defined in BIP-0016BIP-0016. The redeem script does not require a particular ordering or encoding for public keys. This means that for a given set of keys and number of required signatures, there are as many as 2(n!) possible standard redeem scripts, each with its separate P2SH address. Adhering to an ordering and key encoding would ensure that a multi-signature "account" (set of public keys and required signature count) has a canonical P2SH address.

By adopting a sorting and encoding standard, compliant wallets will always produce the same P2SH address for the same given set of keys and required signature count, making it easier to recognize transactions involving that multi-signature account. This is particularly attractive for multisignature hierarchical-deterministic wallets, as less state is required to setup multi-signature accounts: only the number of required signatures and master public keys of participants need to be shared, and all wallets will generate the same addresses.

While most web wallets do not presently facilitate the setup of multisignature accounts with users of a different service, conventions which ensure cross-compatibility should make it easier to achieve this.

Many wallet as a service providers use a 2of3 multi-signature schema where the user stores 1 of the keys (offline) as backup while using the other key for daily use and letting the service cosign his transactions. This standard will help in enabling a party other than the service provider to recover the wallet without any help from the service provider.

Specification

For a set of public keys, ensure that they have been received in compressed form:

```
022df8750480ad5b26950b25c7ba79d3e37d75f640f8e5d9bcd5b150a0f85014da
03e3818b65bcc73a7d64064106a859cc1a5a728c4345ff0b641209fba0d90de6e9
021f2f6e1e50cb6a953935c3601284925decd3fd21bc445712576873fb8c6ebc18
```

Sort them lexicographically according to their binary representation:

```
021f2f6e1e50cb6a953935c3601284925decd3fd21bc445712576873fb8c6ebc18
022df8750480ad5b26950b25c7ba79d3e37d75f640f8e5d9bcd5b150a0f85014da
03e3818b65bcc73a7d64064106a859cc1a5a728c4345ff0b641209fba0d90de6e9
```

..before using the resulting list of keys in a standard multisig redeem script:

```
OP_2 021f2f6e1e50cb6a953935c3601284925decd3fd21bc445712576873fb8c6ebc18 022df8750480ad5b26950b25c7ba79d3e
```

Hash the redeem script according to BIP-0016 to get the P2SH address.

```
3Q4sF6tv9wsdqu2NtARzNCpQgwifm2rAba
```

Compatibility

- Uncompressed keys are incompatible with this specification. A compatible implementation should not automatically compress keys. Receiving an uncompressed key from a multisig participant should be interpreted as a sign that the user has an incompatible implementation.
- P2SH addresses do not reveal information about the script that is receiving the funds. For this reason it is not technically possible to enforce this BIP as a rule on the network. Also, it would cause a hard fork.
- Implementations that do not conform with this BIP will have compatibility issues with strictly-compliant wallets.
- Implementations which do adopt this standard will be cross-compatible when choosing multisig addresses.
- If a group of users were not entirely compliant, there is the possibility that a participant will derive an address that the others will not recognize as part of the common multisig account.

Test vectors

Two signatures are required in each of these test vectors.

Vector 1

- List
    - 02ff12471208c14bd580709cb2358d98975247d8765f92bc25eab3b2763ed605f8
    - 02fe6f0a5a297eb38c391581c4413e084773ea23954d93f7753db7dc0adc188b2f
- Sorted
    - 02fe6f0a5a297eb38c391581c4413e084773ea23954d93f7753db7dc0adc188b2f
    - 02ff12471208c14bd580709cb2358d98975247d8765f92bc25eab3b2763ed605f8
- Script
    - 522102fe6f0a5a297eb38c391581c4413e084773ea23954d93f7753db7dc0adc188b2f2102ff12471208c14bd580709cb235
- Address
    - 39bgKC7RFbpoCRbtD5KEdkYKtNyhpsNa3Z

Vector 2 (Already sorted, no action required)

- List:
    - 02632b12f4ac5b1d1b72b2a3b508c19172de44f6f46bcee50ba33f3f9291e47ed0
    - 027735a29bae7780a9755fae7a1c4374c656ac6a69ea9f3697fda61bb99a4f3e77
    - 02e2cc6bd5f45edd43bebe7cb9b675f0ce9ed3efe613b177588290ad188d11b404
- Sorted:
    - 02632b12f4ac5b1d1b72b2a3b508c19172de44f6f46bcee50ba33f3f9291e47ed0
    - 027735a29bae7780a9755fae7a1c4374c656ac6a69ea9f3697fda61bb99a4f3e77
    - 02e2cc6bd5f45edd43bebe7cb9b675f0ce9ed3efe613b177588290ad188d11b404
- Script
    - 522102632b12f4ac5b1d1b72b2a3b508c19172de44f6f46bcee50ba33f3f9291e47ed021027735a29bae7780a9755fae7a1c
- Address
    - 3CKHTjBKxCARLzwABMu9yD85kvtm7WnMfH

Vector 3:

- List:
  - 03000000000000000000000000000000000000041414141414141414141414141
  - 02000000000000000000000000000000000000041414141414141414141414141
  - 02000000000000000000000000000000000000041414141414141414141414140
  - 03000000000000000000000000000000000000041414141414141414141414140
- Sorted:
  - 02000000000000000000000000000000000000041414141414141414141414140
  - 02000000000000000000000000000000000000041414141414141414141414141
  - 03000000000000000000000000000000000000041414141414141414141414140
  - 03000000000000000000000000000000000000041414141414141414141414141
- Script
  - 52210200000000000000000000000000000000000041414141414141414141414140210200000000000000000000000000
- Address
  - 32V85igBri9zcfBRVupVvwK18NFtS37FuD

Vector 4: (from bitcore)

- List:
  - 022df8750480ad5b26950b25c7ba79d3e37d75f640f8e5d9bcd5b150a0f85014da
  - 03e3818b65bcc73a7d64064106a859cc1a5a728c4345ff0b641209fba0d90de6e9
  - 021f2f6e1e50cb6a953935c3601284925decd3fd21bc445712576873fb8c6ebc18
- Sorted:
  - 021f2f6e1e50cb6a953935c3601284925decd3fd21bc445712576873fb8c6ebc18
  - 022df8750480ad5b26950b25c7ba79d3e37d75f640f8e5d9bcd5b150a0f85014da
  - 03e3818b65bcc73a7d64064106a859cc1a5a728c4345ff0b641209fba0d90de6e9
- Script
  - 5221021f2f6e1e50cb6a953935c3601284925decd3fd21bc445712576873fb8c6ebc1821022df8750480ad5b26950b25c7b
- Address
  - 3Q4sF6tv9wsdqu2NtARzNCpQgwifm2rAba

Acknowledgements

The authors wish to thank BtcDrak and Luke-Jr for their involvement & contributions in the early discussions of this BIP.

Usage & Implementations

- BIP-0045 - Structure for Deterministic P2SH Multisignature Wallets
- Bitcore
- Haskoin - Bitcoin implementation in Haskell
- Armory
- BitcoinJ

References

Copyright

This document is placed in the public domain.

```
BIP: 69
Layer: Applications
Title: Lexicographical Indexing of Transaction Inputs and Outputs
Authors: Kristov Atlas <kristov@openbitcoinprivacyproject.org>
Editor: Daniel Cousens <bips@dcousens.com>
Status: Complete
Type: Informational
Assigned: 2015-06-12
License: PD
```

BIP 69

Abstract

Currently there is no standard for bitcoin wallet clients when ordering transaction inputs and outputs. As a result, wallet clients often have a discernible blockchain fingerprint, and can leak private information about their users. By contrast, a standard for non-deterministic sorting could be difficult to audit. This document proposes deterministic lexicographical sorting, using hashes of previous transactions and output indices to sort transaction inputs, as well as values and scriptPubKeys to sort transaction outputs.

Copyright

This BIP is in the public domain.

Motivation

Currently, there is no clear standard for how wallet clients ought to order transaction inputs and outputs. Since wallet clients are left to their own devices to determine this ordering, they often leak information about their users' finances. For example, a wallet client might naively order inputs based on when addresses were added to a wallet by the user through importing or random generation. Many wallets will place spending outputs first and change outputs second, leaking information about both the sender and receiver's finances to passive blockchain observers. Such information should remain private not only for the benefit of consumers, but in higher order financial systems must be kept secret to prevent fraud. A researcher recently demonstrated this principle when he detected that Bitstamp leaked information when creating exchange transactions, enabling potential espionage among traders. [1]

One way to address these privacy weaknesses is by randomizing the order of inputs and outputs. [2] After all, the order of inputs and outputs does not impact the function of the transaction they belong to, making random sorting viable. Unfortunately, it can be difficult to prove that this sorting process is genuinely randomly sorted based on code or run-time analysis, especially if the software is closed source. A malicious software developer can abuse the ordering of inputs and outputs as a side channel of leaking information. For example, if an attacker can patch a victim's HD wallet client to order inputs and outputs based on the bits of a master private

key, then the attacker can eventually steal all of the victim's funds by monitoring the blockchain. Non-deterministic methods of sorting are difficult to audit because they are not repeatable.

The lack of standardization between wallet clients when ordering inputs and outputs can yield predictable quirks that characterize particular wallet clients or services. Such quirks create unique fingerprints that a privacy attacker can employ through simple passive blockchain observation.

The solution is to create an algorithm for sorting transaction inputs and outputs that is deterministic. Since it is deterministic, it should also be unambiguous — that is, given a particular transaction, the proper order of inputs and outputs should be obvious. To make this standard as widely applicable as possible, it should rely on information that is downloaded by both full nodes (with or without typical efficiency techniques such as pruning) and SPV nodes. In order to ensure that it does not leak confidential data, it must rely on information that is publicly accessible through the blockchain. The use of public blockchain information also allows a transaction to be sorted even when it is a multi-party transaction, such as in the example of a CoinJoin.

Specification

Applicability

This BIP applies to any transaction for which the order of its inputs and outputs does not impact the transaction's function. Currently, this refers to any transaction that employs the SIGHASH_ALL signature hash type, in which signatures commit to the exact order of inputs and outputs. Transactions that use SIGHASH_ANYONECANPAY and/or SIGHASH_NONE may include inputs and/or outputs that are not signed; however, compliant software should still emit transactions with lexicographically sorted inputs and outputs, even though they may later be modified by others.

In the event that future protocol upgrades introduce new signature hash types, compliant software should apply the lexicographical ordering principle analogously.

While out of scope of this BIP, protocols that do require a specified order of inputs/outputs (e.g. due to use of SIGHASH_SINGLE) should consider the goals of this BIP and how best to adapt them to the specific needs of those protocols.

Lexicographical Ordering

Lexicographical ordering is an algorithm for comparison used to sort two sets based on their cartesian order within their common superset. Lexicographic order is also often known as alphabetical order, or dictionary order.

Common implementations include:

- `std::lexicographical_compare` in C++ [5]
- `cmp` in Python 2.7
- `memcmp` in C [6]
- `Buffer.compare` in Node.js [7]

For more information, see the wikipedia entry on Lexicographical order. [8]

N.B. All comparisons do not need to operate in constant time since they are not processing secret information.

Transaction Inputs

Transaction inputs are defined by the hash of a previous transaction, the output index of a UTXO from that previous transaction, the size of an unlocking script, the unlocking script, and a sequence number. [3] For sorting inputs, the hash of the previous transaction and the output index within that transaction are sufficient for sorting purposes; each transaction hash has an extremely high probability of being unique in the blockchain — this is enforced for coinbase transactions by BIP30 — and output indices within a transaction are unique. For the sake of efficiency, transaction hashes should be compared first before output indices, since output indices from different transactions are often equivalent, while all bytes of the transaction hash are effectively random variables.

Previous transaction hashes (in reversed byte-order) are to be sorted in ascending order, lexicographically. In the event of two matching transaction hashes, the respective previous output indices will be compared by their integer value, in ascending order. If the previous output indices match, the inputs are considered equal.

Transaction malleability will not negatively impact the correctness of this process. Even if a wallet client follows this process using unconfirmed UTXOs as inputs and an attacker modifies the blockchain's record of the hash of the previous transaction, the wallet client will include the invalidated previous transaction hash in its input data, and will still correctly sort with respect to that invalidated hash.

Transaction Outputs

A transaction output is defined by its scriptPubKey and amount. [3] For the sake of efficiency, amounts should be compared first for sorting, since they contain fewer bytes of information (8 bytes) compared to a standard P2PKH scriptPubKey (25 bytes). [4]

Transaction output amounts (as 64-bit unsigned integers) are to be sorted in ascending order. In the event of two matching output amounts, the respective output scriptPubKeys (as a byte-array) will be compared lexicographically, in ascending order. If the scriptPubKeys match, the outputs are considered equal.

Examples

Transaction 0a6a357e2f7796444e02638749d9611c008b253fb55f5dc88b739b230ed0c4c3:

Inputs:

```
0: 0e53ec5dfb2cb8a71fec32dc9a634a35b7e24799295ddd5278217822e0b31f57[0]
 1: 26aa6e6d8b9e49bb0630aac301db6757c02e3619feb4ee0eea81eb1672947024[1]
 2: 28e0fdd185542f2c6ea19030b0796051e7772b6026dd5ddccd7a2f93b73e6fc2[0]
 3: 381de9b9ae1a94d9c17f6a08ef9d341a5ce29e2e60c36a52d333ff6203e58d5d[1]
 4: 3b8b2f8efceb60ba78ca8bba206a137f14cb5ea4035e761ee204302d46b98de2[0]
 5: 402b2c02411720bf409eff60d05adad684f135838962823f3614cc657dd7bc0a[1]
 6: 54ffff182965ed0957dba1239c27164ace5a73c9b62a660c74b7b7f15ff61e7a[1]
 7: 643e5f4e66373a57251fb173151e838ccd27d279aca882997e005016bb53d5aa[0]
 8: 6c1d56f31b2de4bfc6aaea28396b333102b1f600da9c6d6149e96ca43f1102b1[1]
 9: 7a1de137cbafb5c70405455c49c5104ca3057a1f1243e6563bb9245c9c88c191[0]
 10: 7d037ceb2ee0dc03e82f17be7935d238b35d1deabf953a892a4507bfbeeb3ba4[1]
 11: a5e899dddb28776ea9ddac0a502316d53a4a3fca607c72f66c470e0412e34086[0]
 12: b4112b8f900a7ca0c8b0e7c4dfad35c6be5f6be46b3458974988e1cdb2fa61b8[0]
```

```
 13: bafd65e3c7f3f9fdfdc1ddb026131b278c3be1af90a4a6ffa78c4658f9ec0c85[0]
 14: de0411a1e97484a2804ff1dbde260ac19de841bebad1880c782941aca883b4e9[1]
 15: f0a130a84912d03c1d284974f563c5949ac13f8342b8112edff52971599e6a45[0]
 16: f320832a9d2e2452af63154bc687493484a0e7745ebd3aaf9ca19eb80834ad60[0]
```

Outputs:

```
0:    400057456    76a9144a5fba237213a062f6f57978f796390bdcf8d01588ac
 1:    40000000000    76a9145be32612930b8323add2212a4ec03c1562084f8488ac
```

Transaction 28204cad1d7fc1d199e8ef4fa22f182de6258a3eaafe1bbe56ebdcacd3069a5f

Inputs:

```
0: 35288d269cee1941eaebb2ea85e32b42cdb2b04284a56d8b14dcc3f5c65d6055[0]
 1: 35288d269cee1941eaebb2ea85e32b42cdb2b04284a56d8b14dcc3f5c65d6055[1]
```

Outputs:

```
0:    100000000    41046a0765b5865641ce08dd39690aade26dfbf5511430ca428a3089261361cef170e3929a68aee3d8d4848b0
 1:    2400000000    41044a656f065871a353f216ca26cef8dde2f03e8c16202d2e8ad769f02032cb86a5eb5e56842e92e19141d
```

Discussion

- [Bitcoin-development] Lexicographical Indexing of Transaction Inputs and Outputs
- [Bitcoin-development] [RFC] Canonical input and output ordering in transactions

References

- 1: Bitstamp Info Leak
- 2: OBPP Random Indexing as Countermeasure
- 3: Mastering Bitcoin
- 4: Bitcoin Wiki on Script
- 5: std::lexicographical_compare
- 6: memcmp
- 7: Buffer.compare
- 8: Lexicographical order

Implementations

- Electrum
- BitcoinJS
- BitcoinJS Test Fixtures
- NodeJS
- Blockchain.info public beta
- Btcsuite

Acknowledgements

Danno Ferrin <danno@numisight.com>, Sergio Demian Lerner <sergiolerner@certimix.com>, Justus Ranvier <justus@openbitcoinprivacyproject.org>, and Peter Todd <pete@petertodd.org> contributed to the design and motivations for this BIP. A similar proposal was submitted to the Bitcoin-dev mailing list independently by Rusty Russell <rusty@rustcorp.com.au>

```
BIP: 125
Layer: Applications
Title: Opt-in Full Replace-by-Fee Signaling
Authors: David A. Harding <dave@dtrt.org>
         Peter Todd <pete@petertodd.org>
Comments-Summary: No comments yet.
Comments-URI: https://github.com/bitcoin/bips/wiki/Comments:BIP-0125
Status: Deployed
Type: Specification
Assigned: 2015-12-04
License: PD
```

BIP 125

Abstract

Many nodes today will not replace any transaction in their mempool with another transaction that spends the same inputs, making it difficult for spenders to adjust their previously-sent transactions to deal with unexpected confirmation delays or to perform other useful replacements.

The opt-in full Replace-by-Fee (opt-in full-RBF) signaling policy described here allows spenders to add a signal to a transaction indicating that they want to be able to replace that transaction in the future. In response to this signal,

- Nodes may allow transactions containing this signal to be replaced in their mempools.
- The recipient or recipients of a transaction containing this signal may choose not to treat it as payment until it has been confirmed, eliminating the risk that the spender will use allowed replacements to defraud them.

Nodes and recipients may continue to treat transactions without the signal the same way they treated them before, preserving the existing status quo.

Summary

This policy specifies two ways a transaction can signal that it is replaceable.

- **Explicit signaling:** A transaction is considered to have opted in to allowing replacement of itself if any of its inputs have an nSequence number less than (0xffffffff - 1).
- **Inherited signaling:** Transactions that don't explicitly signal replaceability are replaceable under this policy for as long as any one of their ancestors signals replaceability and remains unconfirmed.

Implementation Details

The initial implementation expected in Bitcoin Core 0.12.0 uses the following rules:

One or more transactions currently in the mempool (original transactions) will be replaced by a new transaction (replacement transaction) that spends one or more of the same inputs if,

1. The original transactions signal replaceability explicitly or through inheritance as described in the above Summary section.

1. The replacement transaction may only include an unconfirmed input if that input was included in one of the original transactions. (An unconfirmed input spends an output from a currently-unconfirmed transaction.)

1. The replacement transaction pays an absolute fee of at least the sum paid by the original transactions.

1. The replacement transaction must also pay for its own bandwidth at or above the rate set by the node's minimum relay fee setting. For example, if the minimum relay fee is 1 satoshi/byte and the replacement transaction is 500 bytes total, then the replacement must pay a fee at least 500 satoshis higher than the sum of the originals.

1. The number of original transactions to be replaced and their descendant transactions which will be evicted from the mempool must not exceed a total of 100 transactions.

The initial implementation may be seen in Bitcoin Core PR#6871 and specifically the master branch commits from 5891f870d68d90408aa5ce5b597fb574f2d2cbca to 16a2f93629f75d182871f288f0396afe6cdc8504 (inclusive).

Receiving wallet policy

Wallets that display unconfirmed transactions to users or that provide data about unconfirmed transactions to automated systems should consider doing one of the following:

1. Conveying additional suspicion about opt-in full-RBF transactions to the user or data consumer.

1. Ignoring the opt-in transaction until it has been confirmed.

Because descendant transactions may also be replaceable under this policy through inherited signaling, any method used to process opt-in full-RBF transactions should be inherited by any descendant transactions for as long as any ancestor opt-in full-RBF transactions remain unconfirmed.

Spending wallet policy

Wallets that don't want to signal replaceability should use either a max sequence number (0xffffffff) or a sequence number of (0xffffffff-1) when they also want to use locktime; all known wallets currently do this. They should also take care not to spend any unconfirmed transaction that signals replaceability explicitly or through inherited signaling; most wallets also currently do this by not spending any unconfirmed transactions except for those they created themselves.

Wallets that do want to make replacements should use explicit signaling and meet the criteria described above in the Implementation Details section. A Bitcoin Wiki page has been created to help wallet authors track deployed mempool policies relating to transaction replacement.

The initial implementation makes use of P2P protocol reject messages for rejected replacements, allowing P2P clients to determine whether their replacements were initially accepted by their peers. Standard P2P lightweight client practice of sending to some peers while listening for relays from other peers should allow clients to determine whether the replacement has propagated.

Motivation

Satoshi Nakamoto's original Bitcoin implementation provided the nSequence number field in each input to allow replacement of transactions containing that input within the mempool. When receiving replacements, nodes were supposed to replace transactions whose inputs had lower sequence numbers with transactions that had higher sequence numbers.

In that implementation, replacement transactions did not have to pay additional fees, so there was no direct incentive for miners to include the replacement and no built-in rate limiting that prevented overuse of relay node bandwidth. Nakamoto removed replacement from Bitcoin version 0.3.12, leaving only the comment, "Disable replacement feature for now".

Replacing transactions with higher-fee transactions provided a way for spenders to align their desires with miners, but by the time a Replace-by-Fee (RBF) patch was available to re-enable replacement, some receivers had begun to expect that the first version of a transaction they saw was highly likely to be the version of the transaction to be confirmed, and so some users advocated that replacement should be disallowed.

To address those concerns, a variation on RBF was created that required that the replacement transaction pay all of the same outputs as the original transaction in equal or greater amount. This was called RBF First Seen Safe (RBF-FSS), and the original RBF became known as full-RBF. Although agreeable to recipients who relied on the first-seen version of a transaction, each use of RBF-FSS required adding an extra input to a transaction, resulting in wallets being unable to use it if they had no spare inputs, a loss of privacy when inputs from different origins get used in the same transaction, and a wasteful increase in transaction byte size.

Opt-in full-RBF uses Nakamoto's original semantics (with a slight tweak to allow locktime users to opt-out) to signal that replacement is possible, providing first-seen users with the ability to ignore those transactions while also allowing for the efficiency benefits of full-RBF.

There are no known problematic interactions between opt-in full-RBF and other uses of nSequence. Specifically, opt-in full-RBF is compatible with consensus-enforced locktime as provided in the Bitcoin 0.1 implementation, BIP68 (Relative lock-time using consensus-enforced sequence numbers), and BIP112 (CHECKSEQUENCEVERIFY).

Deployment

Now, and since Bitcoin's first release, 100% of the network hash rate mines transactions using opt-in full-RBF semantics (sequence less than (0xffffffff - 1)).

Opt-in full-RBF as a default mempool replacement policy among nodes and miners is expected to become widespread as they upgrade to Bitcoin Core 0.12.0 (release expected Jan/Feb 2016) and similar node software such as Bitcoin LJR.

Actual replacement may be unreliable until two conditions have been satisfied:

1. Enough nodes have upgraded to support it, providing a relay path for replacements to go from spending wallets to miners controlling significant amounts of hash rate.

1. Enough hash rate has upgraded to support replacement, allowing for reasonable probability that a replacement can be mined.

Backwards compatibility

At the time opt-in RBF support was added/proposed, no known wallet created transactions by default with nSequence set below (0xffffffff - 1), so no known wallet explicitly signaled replaceability by default. Also no known popular wallet spent other users' unconfirmed transactions by default, so no known wallets signaled inherited replaceability.

See also

1. Transaction Replaceability on Bitcoin Wiki targeted at helping wallet authors use RBF

1. Tools for creating opt-in full-RBF transactions: https://github.com/petertodd/replace-by-fee-tools#replace-by-fee-tools

1. Reddit: Questions about opt-in RBF targeted at helping community members understand opt-in full-RBF

Copyright

This document is placed in the public domain.

```
  BIP: 388
  Layer: Applications
  Title: Wallet Policies for Descriptor Wallets
  Authors: Salvatore Ingala <salvatoshi@protonmail.com>
  Status: Complete
  Type: Specification
  Assigned: 2023-12-26
  License: BSD-2-Clause
  Discussion: 2022-05-10: https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2022-May/
020423.html
  Version: 1.1.0
```

BIP 388

Abstract

Software wallets and hardware signing devices typically partition funds into separate "accounts". When signing or visualizing a transaction, aggregate flows of funds of all accounts affected by the transaction may (and should) be displayed to the user.

Wallet policies build on top of output script descriptors to represent such accounts in a compact, reviewable way. An account encompasses a logical group of receive and change addresses, and each wallet policy represents all descriptors necessary to describe an account in its entirety.

We simplify the language to suit devices with limited memory, where even keeping the entire descriptor in memory could be a major hurdle, by reducing the generality of descriptors to just the essential features and by separating the extended pubkeys and other key information from the descriptor.

This results in a more compact representation and simplifies the inspection of the policy by the user.

The compilation of wallet policies to the corresponding descriptor is trivial, and the reverse process is easy for supported descriptors, because the language is kept similar to that of output script descriptors.

Copyright

This BIP is licensed under the BSD 2-clause license.

Motivation

Output Script Descriptors were introduced in Bitcoin Core as a way to represent collections of output scripts. It is a general and flexible language, designed to catch all the possible use-cases of bitcoin wallets (that is, if you know the script and you have the necessary keys, it will be possible to sign transactions with any descriptor-based software wallet).

Unfortunately, descriptors are not a perfect match for the typical usage of hardware signing devices (often also called *hardware wallets*). Most of them have some of the following limitations when compared to a general-purpose machine running Bitcoin Core:

- they are embedded devices with limited RAM and computational power;
- they cannot import additional private keys (that is, they can only sign with keys derived from a single seed via BIP-32);
- they have limited storage, or they might not have persistent storage at all (*stateless design*).

Moreover, other limitations like the limited size of the screen might affect what design choices are available in practice. Therefore, minimizing the amount of information shown on-screen is important for a good user experience. The ability for the user to completely validate on-screen the kind of script used (and each of the involved keys) is crucial for secure usage, as the machine that is interacting with the hardware signer (and running the software wallet) is considered untrusted.

A more native, compact representation of the wallet receive and change addresses might also benefit the UX of software wallets when they use descriptors (possibly with miniscript) for representing complex locking conditions.

We remark that wallet policies are not related to the *policy* language, a higher level language that can be compiled to miniscript.

Security, privacy and UX concerns for hardware signing devices

The usage of complex scripts presents challenges in terms of both security and user experience for a hardware signing device.

Security issues

Hardware signing devices strive to guarantee that no action can be performed without the user's consent as long as the user correctly verifies the information that is shown on the device's screen before approving.

This must hold even in scenarios where the attacker has full control of the machine that is connected to the signing device, and can execute arbitrary requests, or tamper with the legitimate user's requests.

Therefore, it is not at all trivial to allow complex scripts, especially if they contain keys that belong to third parties. The hardware signing device must guarantee that the user knows precisely what "policy" is being used to spend the funds, and that any "unspent" funds (if any) that is sent to a change address will be protected by the same policy.

This makes it impossible for an attacker to surreptitiously modify the policy, therefore stealing or burning the user's funds.

Avoiding key reuse

Reusing public keys within a script is a source of malleability when using miniscript policies, which has potential security implications.

Reusing keys across different UTXOs harms user privacy by allowing external parties to link these UTXOs to the same entity once they are spent.

By constraining the derivation path patterns to have a uniform structure, wallet policies prevent key reuse among the same or different UTXOs of the same account.

It is strongly recommended to avoid key reuse across accounts. Distinct public keys per account can be guaranteed by using distinct hardened derivation paths. This specification does not mandate hardened derivation in order to maintain compatibility with existing deployments that do not adhere to this recommendation.

It is out of scope for this document to guarantee that users do not reuse extended public keys among different wallet accounts. This is still very important, but the responsibility is left to the users and their software wallet.

UX issues

Miniscript (and taproot trees) allow substantially more complex spending policies. It is a challenge to ensure that the user can practically verify such spending policies on the screen.

We set two fundamental design goals:

- Minimize the amount of information that is shown on screen - so that the user can actually validate it.
- Minimize the number of times the user has to validate such information.

Designing a secure protocol for the coordination of a descriptor wallet among distant parties is also a challenging problem that is out of the scope of this document. See BIP-129 (Bitcoin Secure Multisig Setup) for an approach designed for multisig wallets. Regardless of the approach, the ability for the user to carefully verify all the details of the spending policies using the hardware signer's screen is a prerequisite for security in adversarial environments.

Policy registration as a solution

A solution to address the security concerns, and part of the UX concerns, is to have a registration flow for the wallet policy in the hardware signing device. The *wallet policy* must contain enough information to generate all the relevant addresses/scripts, and for the hardware signing device to identify the keys that it controls and that are needed to spend the funds sent to those addresses.

Before a new policy is used for the first time, the user will register a wallet policy into the hardware device. While the details of the process are out of scope in this document, the flow should be something similar to the following:

1. The software wallet initiates a *wallet policy registration* on the hardware signing device; the information should include the wallet policy, but also a unique *name* that identifies the policy.
2. The device shows the wallet policy to the user using the secure screen.
3. After inspecting the policy and comparing it with a trusted source (for example a printed backup), the user approves the policy.
4. If stateful, the hardware signing device persists the policy in its permanent memory; if stateless, it returns a "proof of registration".

The **proof of registration** will allow the hardware signer to verify that a certain policy was indeed previously approved by the user, and is therefore safe to use without repeating the expensive user verification procedure. The details of how to create a proof of registration are out of scope for this document; using a Message Authentication Code on a hash committing to the wallet policy, its name and any additional metadata is an effective solution if correctly executed.

Once a policy is registered, the hardware signing device can perform the typical operations securely:

- generating receive and change addresses;
- showing addresses on the secure screen;
- sign transactions spending from a wallet, while correctly identifying change addresses and computing the transaction fees.

Before any of the actions mentioned above, the hardware signing device will retrieve the policy from its permanent storage if stateful; if stateless it will validate the proof of registration before using the wallet policy provided by the client.

Once the previously registered policy is correctly identified and approved by the user (for example by showing its name), and as long as the policy registration was executed securely, hardware signing devices can provide a user experience similar to the usual one for single-signature transactions.

Avoiding blowup in descriptor size

While reusing a pubkey in different branches of a miniscript is explicitly forbidden by miniscript (as it has certain negative security implications), it is still reasonable to reuse the same xpub in multiple places, albeit with different final steps of derivation (so that the actual pubkeys that are used in the script are indeed different).

In fact, there are many reasonable spending policies with a quadratic size in the number of participants. For example, using Taproot, a 3-of-5 threshold wallet could use:

- a key path with a 5-of-5 MuSig2 aggregated key
- a script tree with 11 leaves:
    - 10 different scripts using a 3-of-3 MuSig2 aggregated key, plus
    - a final leaf with a fallback 3-of-5 multisig using `multi_a` (in case interactive signing is not available).

With each xpub being 118 bytes long, the repetition of xpubs makes the descriptor become extremely large.

Replacing the common part of the key with a short key placeholder and organizing all the key expressions in a separate list helps to keep the size of the wallet policy small, which is crucial to allow human inspection during the registration flow.

Specification

This section formally defines wallet policies, and how they relate to output script descriptors.

Formal definition

A *wallet policy* is composed of a *wallet descriptor template*, together with a vector of *key information items*.

Wallet descriptor template

A *wallet descriptor template* is a `SCRIPT` expression.

`SCRIPT` expressions:

- `sh(SCRIPT)` (top level only): P2SH embed the argument.
- `wsh(SCRIPT)` (top level or inside `sh` only): P2WSH embed the argument.
- `pkh(KEY)` (not inside `tr`): P2PKH output for the given public key.
- `wpkh(KEY)` (top level or inside `sh` only): P2WPKH output for the given compressed pubkey.
- `multi(k,KEY_1,KEY_2,...,KEY_n)` (inside `sh` or `wsh` only): *k*-of-*n* multisig script.
- `sortedmulti(k,KEY_1,KEY_2,...,KEY_n)` (inside `sh` or `wsh` only): *k*-of-*n* multisig script with keys sorted lexicographically in the resulting script.
- `tr(KEY)` or `tr(KEY,TREE)` (top level only): P2TR output with the specified key as internal key, and optionally a tree of script paths.
- any valid miniscript template (inside `wsh` or `tr` only).

See BIP-379 for a precise specification of all the valid miniscript `SCRIPT` expressions.

`TREE` expressions:

- any `SCRIPT` expression
- An open brace {, a `TREE` expression, a comma `,`, a `TREE` expression, and a closing brace }

`KEY` expressions consist of

- a `KP` expression

- *always* followed by either:
    - the string `/**`, or
    - a string of the form `/<NUM;NUM>/*`, for two distinct decimal numbers `NUM` representing unhardened derivations, or
    - any of the additional, implementation-specific valid derivation path patterns (see Optional derivation paths below).

`KP` expressions (key placeholders) consist of either:

- a `KI` (key index) expression, or
- (only inside `tr`): `musig(KI_1,KI_2,...,KI_n)`

A `KI` (key index) expression consists of:

- a single character `@`
- followed by a non-negative decimal number, with no leading zeros (except for `@0`)

The `/**` in the placeholder template represents commonly used paths for receive/change addresses, and is equivalent to `/<0;1>/*`.

Note that while BIP-389 allows multipath `/<NUM;NUM;...;NUM>` expressions with an arbitrary number of options, this specification restricts it to exactly 2 choices (with the typical meaning of receive/change addresses).

`SCRIPT`, `TREE` and `KEY` expressions map directly to the corresponding concepts defined in BIP-380 for output script descriptors.

Each `KEY` expression always corresponds to a precise public key in the final bitcoin Script. Therefore, all the derivation steps in the BIP-32 hierarchy are included in a `KEY` expression.

Each `KP` (key placeholder) expression, on the other hand, maps to the root of all the corresponding public keys for all the possible UTXOs that belong to the account represented in the wallet policy. Therefore, no derivation steps are allowed in a `KP` expression.

A `KI` (key index) `@i` for some number *i* represents the *i*-th key in the vector of key information items (which must be of size at least *i + 1*, or the wallet policy is invalid).

Note: while descriptor templates for miniscript are not formally defined in this version of the document (pending standardization), it is straightforward to adapt this approach by adding additional `SCRIPT` expressions.

Key information vector

Each element of the key origin information vector is a `KEY_INFO` expression, containing an extended public key, and (optionally) its key origin.

- Optionally, key origin information, consisting of:
    - An open bracket `[`
    - Exactly 8 hex characters for the fingerprint of the master key from which this key is derived from (see BIP-32 for details)

    - Followed by zero or more `/NUM'` or `/NUM` path elements to indicate hardened or unhardened derivation steps between the fingerprint and the xpub that follows
    - A closing bracket `]`
- Followed by the actual key, which is a serialized extended public key (as defined in BIP-32).

Additional rules

A wallet policy must have at least one key placeholder and the corresponding key.

The public keys obtained by deserializing elements of the key information vector must be pairwise distinct**Why must public keys be distinct?** Reusing pubkeys could be insecure in the context of wallet policies containing miniscript. Avoiding repeated public keys altogether avoids the problem at the source..

If two `KEY` are `KP/<M;N>/*` and `KP/<P;Q>/*` for the same key placeholder `KP`, then the sets `{M, N}` and `{P, Q}` must be disjoint. Two `musig` key placeholders are the same if they have exactly the same set of key indexes (regardless of the order).

Repeated `KI` expressions are not allowed inside a `musig` placeholder.

The key information vector should be ordered so that placeholder `@i` never appears for the first time before an occurrence of `@j` for some `j < i`; for example, the first placeholder is always `@0`, the next one is `@1`, etc.

Descriptor derivation

From a wallet descriptor template (and the associated vector of key information items), one can therefore obtain the corresponding multipath descriptor by:

- replacing each key placeholder with the corresponding key origin information;
- replacing every `/**` with `/<0;1>/*`.

For example, the wallet descriptor `pkh(@0/**)` with key information `["[d34db33f/44'/0'/0']xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcgJvLJuZ`
produces the following multipath descriptor:

`pkh([d34db33f/44'/0'/0']xpub6ERApfZwUNrhLCkDtcHTcxd75RbzS1ed54G1LkBUHQVHQKqhMkhgbmJbZRkrgZw4koxb5JaHWkY4ALHY2grBGRjaDMzQLcgJvLJuZ<0;1>/*)`

Implementation guidelines

It is acceptable to implement only a subset of the possible wallet policies defined by this standard. It is recommended that any limitations are clearly documented.

Implementations can add additional metadata that is stored together with the wallet policy for the purpose of wallet policy registration and later usage. Metadata can be vendor-specific and is out of the scope of this document.

Any implementation in a software wallet that allows wallet policies not matching any of the specifications in BIP-44, BIP-49, BIP-84, BIP-86 (especially if involving external cosigners) should put great care into a process for backing up the wallet policy that represents the account. In fact, unlike standard single-signature scenarios,

the seed alone is no longer enough to discover wallet policies with existing funds, and the loss of the backup is likely to lead to permanent loss of funds. Unlike the seed, leaking such backups only affects the privacy of the user, but it does not allow the attacker to steal funds.

Optional derivation paths

In order to allow supporting legacy derivation schemes (for example, using simply `/*` instead of the more common `/<M;N>/*` scheme most software wallets use today), or other schemes that are not covered in this document, implementations might choose to permit additional derivation patterns for the key placeholder (`KP`) expressions.

However, care needs to be taken in view of the following considerations:

- Allowing derivation schemes with a different length or cardinality in the same wallet policy would make it difficult to guarantee that there are no repeated pubkeys for every possible address generated by the policy. For example, `@0/<0;1>/*` and `@1/*` would generate the same pubkeys if the second public key in the key information vector is one of the first two unhardened children of the first public key. This could cause malleability with potential security implications (for example, in policies containing miniscript).
- Allowing naked pubkeys with no `/*` suffix (for example a descriptor template like `wsh(multi(2,@0,@1/<0;1>/*))`) would cause a pubkey to be repeated in every output generated from the policy, which would result in a total loss of privacy.

Examples

In the examples in this section, the vector of key information items is omitted. See the test vectors below for complete examples.

Common single-signature account patterns:

- `pkh(@0/**)` (legacy).
- `wpkh(@0/**)` (native segwit).
- `sh(wpkh(@0/**))` (nested segwit).
- `tr(@0/**)` (taproot single-signature account).

Common multisig schemes:

- `wsh(multi(2,@0/**,@1/**))` - SegWit 2-of-2 multisig, keys in order.
- `sh(sortedmulti(2,@0/**,@1/**,@2/**))` - Legacy 2-of-3 multisig, sorted keys.
- `tr(musig(@0,@1)/**)` - MuSig2 2-of-2 in the taproot keypath

Some miniscript policies in `wsh`:

- `wsh(and_v(v:pk(@0/**),or_d(pk(@1/**),older(12960))))` - Trust-minimized second factor, degrading to a single signature after about 90 days.
- `wsh(thresh(3,pk(@0/**),s:pk(@1/**),s:pk(@2/**),sln:older(12960)))` - A 3-of-3 wallet that becomes a 2-of-3 if coins are not spent for about 90 days.
- `wsh(or_d(pk(@0/**),and_v(v:multi(2,@1/**,@2/**,@3/**),older(65535))))` - A singlesig wallet with automatic inheritance to a timelocked 2-of-3 multisig of family members.

Test Vectors

Valid policies

BIP-44, first account

```
Descriptor template: pkh(@0/**)
Keys info: ["[6738736c/44'/0'/
0']xpub6Br37sWxruYfT8ASpCjVHKGwgdnYFEn98DwiN76i2oyY6fgH1LAPmmDcF46xjxJr22gw4jmVjTE2E3URMnRPEPYyo1zoPSUba563E
Descriptor: pkh([6738736c/44'/0'/
0']xpub6Br37sWxruYfT8ASpCjVHKGwgdnYFEn98DwiN76i2oyY6fgH1LAPmmDcF46xjxJr22gw4jmVjTE2E3URMnRPEPYyo1zoPSUba563E
<0;1>/*)
```

BIP-49, second account

```
Descriptor template: sh(wpkh(@0/**))
Keys info: ["[6738736c/49'/0'/
1']xpub6Bex1CHWGXNNwGVKHLqNC7kcV348FxkCxpZXyCWp1k27kin8sRPayjZUKDjyQeZzGUdyeAj2emoW5zStFFUAHRgd5w8iVVbLgZ7Pm
Descriptor: sh(wpkh([6738736c/49'/0'/
1']xpub6Bex1CHWGXNNwGVKHLqNC7kcV348FxkCxpZXyCWp1k27kin8sRPayjZUKDjyQeZzGUdyeAj2emoW5zStFFUAHRgd5w8iVVbLgZ7Pm
<0;1>/*))
```

BIP-84, third account

```
Descriptor template: wpkh(@0/**)
Keys info: ["[6738736c/84'/0'/
2']xpub6CRQzb8u9dmMcq5XAwwRn9gcoYCjndJkhKgD11WKzbVGd932UmrExWFxCAvRnDN3ez6ZujLmMvmLBaSWdfWVn75L83Qxu1qSX4fJN
Descriptor: wpkh([6738736c/84'/0'/
2']xpub6CRQzb8u9dmMcq5XAwwRn9gcoYCjndJkhKgD11WKzbVGd932UmrExWFxCAvRnDN3ez6ZujLmMvmLBaSWdfWVn75L83Qxu1qSX4fJN
<0;1>/*)
```

BIP-86, first account

```
Descriptor template: tr(@0/**)
Keys info: ["[6738736c/86'/0'/
0']xpub6CryUDWPS28eR2cDyojB8G354izmx294BdjeSvH469Ty3o2E6Tq5VjBJCn8rWBgesvTJnyXNAJ3QpLFGuNwqFXNt3gn612raffLWf
Descriptor: tr([6738736c/86'/0'/
0']xpub6CryUDWPS28eR2cDyojB8G354izmx294BdjeSvH469Ty3o2E6Tq5VjBJCn8rWBgesvTJnyXNAJ3QpLFGuNwqFXNt3gn612raffLWf
<0;1>/*)
```

BIP-48 P2WSH multisig

```
Descriptor template: wsh(sortedmulti(2,@0/**,@1/**))
Keys info: ["[6738736c/48'/0'/0'/
2']xpub6FC1fXFP1GXLX5TKtcjHGT4q89SDRehkQLtbKJ2PzWcvbBHtyDsJPLtpLtkGqYNYZdVVAjRQ5kug9CsapegmmeRutpP7PW4u4wVF9
```

```
48'/0'/0'/
2']xpub6EWhjpPa6FqrcaPBuGBZRJVjzGJ1ZsMygRF26RwN932Vfkn1gyCiTbECVitBjRCkexEvetLdiqzTcYimmzYxyR1BZ79KNevgt61PD
Descriptor: wsh(sortedmulti(2,[6738736c/48'/0'/0'/
2']xpub6FC1fXFP1GXLX5TKtcjHGT4q89SDRehkQLtbKJ2PzWcvbBHtyDsJPLtpLtkGqYNYZdVVAjRQ5kug9CsapegmmeRutpP7PW4u4wVF9
<0;1>/*,[b2b1f0cf/48'/0'/0'/
2']xpub6EWhjpPa6FqrcaPBuGBZRJVjzGJ1ZsMygRF26RwN932Vfkn1gyCiTbECVitBjRCkexEvetLdiqzTcYimmzYxyR1BZ79KNevgt61PD
<0;1>/*))
```

Miniscript: A 3-of-3 that becomes a 2-of-3 after 90 days

```
Descriptor template: wsh(thresh(3,pk(@0/**),s:pk(@1/**),s:pk(@2/**),sln:older(12960)))
Keys info: ["[6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbcoK42KRZbxw
0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
Descriptor: wsh(thresh(3,pk([6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
<0;1>/*),s:pk([b2b1f0cf/44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbcoK42KRZbxw
<0;1>/*),s:pk([a666a867/44'/0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
<0;1>/*),sln:older(12960)))
```

Miniscript: A singlesig wallet with automatic inheritance to a timelocked 2-of-3 multisig

```
Descriptor template: wsh(or_d(pk(@0/**),and_v(v:multi(2,@1/**,@2/**,@3/**),older(65535))))
Keys info: ["[6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbcoK42KRZbxw
0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
0'/0'/
100']xpub6Dz8PHFmXkYkykQ83ySkruky567XtJb9N69uXScJZqweYiQn6FyieajdiyjCvWzRZ2GoLHMRE1cwDfuJZ6461YvNRGVBJNnLA35
Descriptor: wsh(or_d(pk([6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
<0;1>/*),and_v(v:multi(2,[b2b1f0cf/44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbcoK42KRZbxw
<0;1>/*,[a666a867/44'/0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
<0;1>/*,[bb641298/44'/0'/0'/
100']xpub6Dz8PHFmXkYkykQ83ySkruky567XtJb9N69uXScJZqweYiQn6FyieajdiyjCvWzRZ2GoLHMRE1cwDfuJZ6461YvNRGVBJNnLA35
<0;1>/*),older(65535))))
```

Taproot wallet policy with sortedmulti_a and a miniscript leaf

```
Descriptor template: tr(@0/**,{sortedmulti_a(1,@0/<2;3>/*,@1/**),or_b(pk(@2/**),s:pk(@3/**))})
Keys info: ["[6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
Descriptor: tr([6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
<0;1>/*,{sortedmulti_a(1,[6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
<2;3>/
*,xpub6Fc2TRaCWNgfT49nRGG2G78d1dPnjhW66gEXi7oYZML7qEFN8e21b2DLDipTZZnfV6V7ivrMkvh4VbnHY2ChHTS9qM3XVLJiAgcfag
<0;1>/
*),or_b(pk(xpub6GxHB9kRdFfTqYka8tgtX9Gh3Td3A9XS8uakUGVcJ9NGZ1uLrGZrRVr67DjpMNCHprZmVmceFTY4X4wWfksy8nVwPiNvz
<0;1>/
*),s:pk(xpub6GjFUVVYewLj5no5uoNKCWuyWhQ1rKGvV8DgXBG9Uc6DvAKxt2dhrj1EZFrTNB5qxAoBkVW3wF8uCS3q1ri9fueAa6y7heFT
<0;1>/*))})
```

Taproot MuSig2 3-of-3 in the key path, with three 2-of-2 MuSig2 recovery paths after 90 days in the script paths

```
Descriptor template: tr(musig(@0,@1,@2)/**,{and_v(v:pk(musig(@0,@1)/**),older(12960)),
{and_v(v:pk(musig(@0,@2)/**),older(12960)),and_v(v:pk(musig(@1,@2)/**),older(12960))}})
Keys info: ["[6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbBcoK42KRZbxw
0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
Descriptor: tr(musig([6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
[b2b1f0cf/44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbBcoK42KRZbxw
[a666a867/44'/0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
<0;1>/*,{and_v(v:pk(musig([6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
[b2b1f0cf/44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbBcoK42KRZbxw
<0;1>/*),older(12960)),{and_v(v:pk(musig([6738736c/48'/0'/0'/
100']xpub6FC1fXFP1GXQpyRFfSE1vzzySqs3Vg63bzimYLeqtNUYbzA87kMNTcuy9ubr7MmavGRjW2FRYHP4WGKjwutbf1ghgkUW9H7e3ce
[a666a867/44'/0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
<0;1>/*),older(12960)),and_v(v:pk(musig([b2b1f0cf/44'/0'/0'/
100']xpub6EYajCJHe2CK53RLVXrN14uWoEttZgrRSaRztujsXg7yRhGtHmLBt9ot9Pd5ugfwWEu6eWyJYKSshyvZFKDXiNbBcoK42KRZbxw
[a666a867/44'/0'/0'/
100']xpub6Dgsze3ujLi1EiHoCtHFMS9VLS1UheVqxrHGfP7sBJ2DBfChEUHV4MDwmxAXR2ayeytpwm3zJEU3H3pjCR6q6U5sP2p2qzAD71x
<0;1>/*),older(12960))}})
```

Invalid policies

The following descriptor templates are invalid:

- `pkh(@0)`: Key placeholder with no path following it
- `pkh(@0/0/**)`: Key placeholder with an explicit path present
- `sh(multi(1,@1/**,@0/**))`: Key placeholders out of order
- `sh(multi(1,@0/**,@2/**))`: Skipped key placeholder `@1`
- `sh(multi(1,@0/**,@0/**))`: Repeated keys with the same path expression
- `sh(multi(1,@0/<0;1>/*,@0/<1;2>/*))`: Non-disjoint multipath expressions (`@0/1/*` appears twice)
- `sh(multi(1,@0/**,xpub6AHA9hZDN11k2ijHMeS5QqHx2KP9aMBRhTDqANMnwVtdyw2TDYRmF8PjpvwUFcL1Et8Hj59S3gTSMcUQ5gAqTz3Wd8EsMTm<0;1>/*))`: Expression with a non-KP key present
- `pkh(@0/<0;1;2>/*)`: Allowed cardinality > 2
- `tr(musig(@0/**,@1/**))`: Derivation before aggregation is not allowed in wallet policies (despite being allowed in BIP-390)

Remark: some of the examples of invalid descriptor templates may be valid via optional extensions.

Backwards Compatibility

The `@` character used for key placeholders is not part of the syntax of output script descriptors, therefore any valid descriptor with at least one `KEY` expression is not a valid descriptor template. Vice versa, any descriptor template with at least one key placeholder is not a valid output script descriptor.

Adoption of wallet policies in software and hardware wallets is opt-in. Conversion from wallet policies to the corresponding descriptors is programmatically extremely easy, and conversion from descriptors to wallet policies (when respecting the required patterns) can be automated. See the reference implementation below for some examples of conversion.

Software wallets are recommended to allow exporting plain descriptors for the purposes of interoperability with software not using wallet policies.

Reference Implementation

Wallet policies are implemented in

- the Ledger bitcoin application since version 2.1.0;
- the BitBox02 firmware since version v9.15.0;
- Blockstream Jade since version v1.0.24, via libwally-core v1.0.0.
- BTCPay Server from version v2.1.1. (Currently only a limited subset of singlesig and multisig policies)
- NBitcoin through `Miniscript.Parse`.

For development and testing purposes, we provide a Python 3.7 reference implementation of simple classes to handle wallet policies, and the conversion to/from output script descriptors. The reference implementation is for demonstration purposes only and not to be used in production environments.

Changelog

- **1.1.0** (2024-11):
  - Added support for `musig` key placeholders in descriptor templates.
- **1.0.0** (2024-05-12):
  - Initial version, advance to Proposed (later updated to Complete per BIP3 adoption).
- **0.0.1** (2022-11-16):
  - First draft.

Footnotes

Acknowledgments

The authors would like to thank the people who provided feedback in the bitcoin-dev list, and in person.

Conclusion

That is the set I wanted to put in one place: the deployed and complete BIPs, grouped so you can read related work together instead of hopping around by number.

I did not write these documents. The BIP authors did, and the bitcoin/bips editors keep the archive. If a specification is wrong, out of date, or missing a successor, the fix belongs upstream. This book is a reading copy, not a fork of the process.

Bitcoin keeps moving. New drafts will become complete, some complete documents will be buried under later rules, and wallets will keep inventing encodings on top of the same consensus. When that happens I will regroup the collection. Until then, I hope this order made the originals easier to sit with.

Thanks for reading. If this compilation helped you implement something, or just helped you see how the pieces fit, that is the whole point.

---

1. **When is a BIP "accepted"?**
   Many standards processes such as the PEPs or the IETF have a mechanism for a proposal to be formally accepted by some council or committee. The BIP Process does not have such a decision body. Bitcoin development and "acceptance" of BIPs emerges from the participation of stakeholders across the ecosystem, and refers to some vague notion of community interest and support for a proposal.
   BIP2 had made an attempt to gather community feedback into comment summaries in BIPs directly. Given the low participation and corresponding low information quality of the summaries that resulted from that feature, this BIP instead intends to leave the evaluation of BIPs to the reviewers and users. ↩

2. **Why does the BIPs repository no longer track adoption?**
   In the past, some BIPs maintained lists of projects that had implemented the BIP. These lists generated noise for subscribers of the repository, often listed implementations of questionable quality, and quickly grew outdated, therefore providing little value. The repository no longer tracks which projects have implemented BIPs. Instead, it is recommended that projects publish a list of the BIPs they implement. ↩

3. **Which flavor of Markdown is allowed?**
   The author of this proposal has no opinion on Markdown flavors, but recommends that proposals stick to the basic Markdown syntax features commonly shared across Markdown dialects.[?]

4. **Why was "Replaces" retained instead of changing it to "Proposes-to-Replace"?**
   When one BIP proposes to supersede another, it is on the original BIP where things get complicated. The BIP is an author document, but depending on its progress through the workflow, it may meanwhile be co-owned by the community. Who may decide whether the original document should endorse a potential replacement BIP? Is it the original authors, the authors of the new proposal, the BIP Editors, some sort of community process, or a mix of all of the above?
   On the new BIP these problems don't exist in the same manner. As it is freshly written, it is wholly owned by its authors. The community is not yet invested and the original BIP's authors do not have a privileged role in determining the content of the new BIP. The authors of the new BIP can unilaterally recommend that it be considered a replacement for a prior BIP. From there, the community can evaluate the proposal and adopt or reject it, thus establishing whether it is successful in superseding the original or not.[?]

5. **Why is Superseded-By replaced with Proposed-Replacement?**
   Reviewers asked who should get to decide whether a BIP is superseded in case of a disagreement among the authors of the original BIP, the authors of the new BIP, the editors, or the community? This is addressed by making the "Replaces" header part of the recommendation of the authors of the new document, and replacing the "Superseded-By" header with the "Proposed-Replacement" header that lists any proposals that recommend replacing the original document.[?]

6. **Why was the Proposed status renamed to Complete?**
   Some reviewers of this BIP raised that in a process which outlines the workflow of Bitcoin Improvement *Proposals* using "Proposed" as a status field value was overloading the term: clearly *proposals* are proposed at all stages of the process. "Complete" expresses that the authors have concluded planned work on all parts of the proposal and are ready to recommend their BIP for adoption. The term "ready" was also considered, but considered too subjective.[?]

7. **Why should the specification of an implemented BIP no longer be changed?**
   After a Complete or Deployed BIP has been deployed by one or more implementations, breaking changes to the specification could lead to a situation where multiple "compliant" implementations fail at being interoperable, because they implemented different versions of the same BIP. Therefore, even changes to the specification of Complete BIPs should be avoided, but Deployed BIPs should never be subject to breaking changes to their specification.[?]

8. **How is evidence for advancing to Deployed evaluated?**
   Whether evidence is deemed convincing to move a BIP to Deployed is up to the BIP Editors and Bitcoin community. Running a single instance of a personal fork of a software project might be rejected, while a small software project with dozens of users may be sufficient. The main point of the Deployed status is to indicate that changes to the BIP could negatively impact users of projects that have already implemented support.[?]

9. **Why should the specification of an implemented BIP no longer be changed?**
   After a Complete or Deployed BIP has been deployed by one or more implementations, breaking changes to the specification could lead to a situation where multiple "compliant" implementations fail at

being interoperable, because they implemented different versions of the same BIP. Therefore, even changes to the specification of Complete BIPs should be avoided, but Deployed BIPs should never be subject to breaking changes to their specification.

10. **Why are Process BIPs living documents?**
    In the past years, the existing BIPs process has not always provided a clear approach to all situations. For example, the content of BIP 2 appears to have been penned especially with fork proposals in mind. It seems clear that Bitcoin development will evolve in many surprising ways in the future. Instead of mandating the effort of writing a new process document every time new situations arise, it seems preferable to allow the process to adapt to the concerns of the future in specific aspects. Therefore, Process BIPs are defined as living documents that remain open to amendment. If a Process BIP requires large modifications or even a complete overhaul, a new BIP should be preferred.

11. **Why was the Closed Status introduced?**
    The Closed Status provides value to the audience by indicating which documents are only of historical significance. Previously, the process had Deferred, Obsolete, Rejected, Replaced, and Withdrawn, which all meant some flavor of "work has stopped on this." The many statuses complicated the process, may have contributed to process fatigue, and may have resulted in BIPs' statuses not being maintained well. The author of this BIP feels that all of the aforementioned can be represented by *Closed* without significantly impacting the information quality of the overview table. Where the many Status variants provided minuscule additional information, the simplification is more valuable and the Changelog section now collects specific details.

12. **Why are some BIP status changes announced to the mailing list?**
    The BIPs repository does not and cannot track who might be interested in or has deployed a BIP. While concerns were raised that making announcements to the Bitcoin Developer Mailing List would introduce unnecessary noise, our rationale is that 1) moving Complete and Deployed BIPs to the Closed status will be a rare occurrence, 2) status updates will usually not generate a lot of discussion, 3) while the mailing list should preferably only used for getting review for new BIPs, it is the only channel available to us that can be considered a public announcement to the audience of the BIPs repository: even if the authors, implementers, or other parties interested in a BIP do not see the announcement themselves, they may be made aware by someone that does see it.

13. **Why are some BIP status changes announced to the mailing list?**
    The BIPs repository does not and cannot track who might be interested in or has deployed a BIP. While concerns were raised that making announcements to the Bitcoin Developer Mailing List would introduce unnecessary noise, our rationale is that 1) moving Complete and Deployed BIPs to the Closed status will be a rare occurrence, 2) status updates will usually not generate a lot of discussion, 3) while the mailing list should preferably only used for getting review for new BIPs, it is the only channel available to us that can be considered a public announcement to the audience of the BIPs repository: even if the authors, implementers, or other parties interested in a BIP do not see the announcement themselves, they may be made aware by someone that does see it.

14. **What is the issue with "Other Implementations" sections in BIPs?**
    In the past, some BIPs had "Other Implementations" sections that caused frequent change requests to existing BIPs. This put a burden on the BIP authors and BIP Editors, and frequently led to lingering pull requests due to the corresponding BIPs' authors no longer participating in the process. Many of these

alternative implementations eventually became unmaintained or were low-quality to begin with. Therefore, "Other Implementations" sections are heavily discouraged.

15. **Why are some BIP status changes announced to the mailing list?**
    The BIPs repository does not and cannot track who might be interested in or has deployed a BIP. While concerns were raised that making announcements to the Bitcoin Developer Mailing List would introduce unnecessary noise, our rationale is that 1) moving Complete and Deployed BIPs to the Closed status will be a rare occurrence, 2) status updates will usually not generate a lot of discussion, 3) while the mailing list should preferably only used for getting review for new BIPs, it is the only channel available to us that can be considered a public announcement to the audience of the BIPs repository: even if the authors, implementers, or other parties interested in a BIP do not see the announcement themselves, they may be made aware by someone that does see it.

16. **Why were some licenses dropped?**
    Among the 141 BIPs with licenses in the repository, only nine licenses have ever been used to license BIPs (although, some BIPs were made available under more than one license) and only one license has been used to license code:

    Licenses used:

    - BSD-2-Clause: 55
    - PD: 42
    - CC0-1.0: 23
    - BSD-3-Clause: 19
    - OPUBL-1.0: 5
    - CC-BY-SA-4.0: 4
    - FSFAP: 3
    - MIT: 2
    - CC-BY-4.0: 1

    License-Code used (previous BIP2 format):

    - BSD-2-Clause: 1
    - CC0-1.0: 1
    - MIT: 5

    The following previously acceptable licenses were retained per request of reviewers, even though they have so far never been used in the BIPs process:

    - Apache-2.0: Apache License 2.0
    - BSL-1.0: Boost Software License 1.0

    The following previously acceptable licenses have never been used in the BIPs Process and have been dropped:

    - AGPL-3.0+: GNU Affero General Public License (AGPL) 3
    - FDL-1.3: GNU Free Documentation License 1.3
    - GPL-2.0+: GNU General Public License (GPL) 2 or newer
    - LGPL-2.1+: GNU Lesser General Public License (LGPL) 2.1 or newer

Why are software licenses included?

- Some BIPs, in particular those concerning the consensus layer, may include literal code in the BIP itself which may not be available under the license terms the authors wish to use for the BIP.
- The author of this BIP has been provided with a learned opinion indicating that software licenses are perfectly acceptable for licensing "human code" i.e., text as well as Markdown or MediaWiki code.

Why is CC-BY-SA-4.0 no longer acceptable for new BIPs?

- Specification BIPs are required to have a Reference Implementation and Test Vectors to be advanced to Complete. As the BIPs repository is aiming to make proposals easily adoptable, the intention is for the reference implementation and test vectors to be as accessible as possible. Copyleft licenses may introduce friction here, and therefore CC-BY-SA-4.0 (and the GPL-flavors) is no longer considered acceptable for new BIPs. As mentioned above, existing BIPs will retain their original licensing.

Why are Open Publication License and Public Domain no longer acceptable for new BIPs?

- Public domain is not universally recognised as a legitimate action, thus it is inadvisable.
- The Open Publication License is generally regarded as obsolete, and not a license suitable for new publications.

[?]

17. **Why was the Closed Status introduced?**
    The Closed Status provides value to the audience by indicating which documents are only of historical significance. Previously, the process had Deferred, Obsolete, Rejected, Replaced, and Withdrawn, which all meant some flavor of "work has stopped on this." The many statuses complicated the process, may have contributed to process fatigue, and may have resulted in BIPs' statuses not being maintained well. The author of this BIP feels that all of the aforementioned can be represented by *Closed* without significantly impacting the information quality of the overview table. Where the many Status variants provided minuscule additional information, the simplification is more valuable and the Changelog section now collects specific details.[?]

18. **Why were comments, Comments-URI, and Comments-Summary removed from the process?**
    The comments feature saw insignificant adoption. Few BIPs received any comments and barely any more than two with only a handful of contributors commenting at all. This led to many situations in which one or two comments ended up dominating the comment summary. While some of those comments may have been representative of broadly held opinions, it also overstated the importance of individual comments directly in the Preamble of BIPs. As collecting feedback in this accessible fashion failed, the new process puts the burden back on the audience to make their own evaluation.[?]

19. **Why can proposals remain in Draft or Complete indefinitely?**
    The automatic 3-year timeout of BIPs has led to some disagreement in the past and seems unnecessary in cases where the authors remain active in the community and still consider their idea worth pursuing. On the other hand, Draft proposals that appear stale may be closed after only one year, which should achieve the main goal of the original rule by limiting the effort and attention spent on proposals that never reach Complete.[?]

20. **Why was the Specification type introduced?**
    The definitions of Informational and Standards Track BIPs caused some confusion in the past. Due to Informational BIPs being described as optional, Standards Track BIPs were sometimes misunderstood to be generally recommended. This has led to a number of BIPs that propose new features affecting interoperability of implementations being assigned the Informational type. The situation is remedied by introducing a new *Specification BIP* type that is inclusive of any BIPs that can be implemented and affect interoperability of Bitcoin applications. Since all BIPs are individual recommendations by the authors (even if some may eventually achieve endorsement by the majority of the community), the prior reminder that Informational BIPs are optional is dropped.[?]

21. **What is the issue with "Other Implementations" sections in BIPs?**
    In the past, some BIPs had "Other Implementations" sections that caused frequent change requests to existing BIPs. This put a burden on the BIP authors and BIP Editors, and frequently led to lingering pull requests due to the corresponding BIPs' authors no longer participating in the process. Many of these alternative implementations eventually became unmaintained or were low-quality to begin with. Therefore, "Other Implementations" sections are heavily discouraged.[?]

22. **Why were comments, Comments-URI, and Comments-Summary removed from the process?**
    The comments feature saw insignificant adoption. Few BIPs received any comments and barely any more than two with only a handful of contributors commenting at all. This led to many situations in which one or two comments ended up dominating the comment summary. While some of those comments may have been representative of broadly held opinions, it also overstated the importance of individual comments directly in the Preamble of BIPs. As collecting feedback in this accessible fashion failed, the new process puts the burden back on the audience to make their own evaluation.[?]

23. **Why was the Created header renamed to Assigned?**
    Both BIP 1 and BIP 2 described the Created header as "date created on" in the preamble template, but followed that up with "Created header records the date that the BIP was assigned a number" as the description of the field. This has frequently led to confusion, with authors using the date of opening the pull request, the date they started writing their proposal, the date of number assignment (as prescribed), or various other dates. Aligning the name of the header and the text in the preamble template with the descriptions will reduce the confusion.[?]

24. **Why is the layer header now permitted for other BIP types?**
    The layer header had already been used by many Informational BIPs, so the rule that it is only available to Standards Track BIPs is dropped.[?]

25. The minimum `featureid` length of 4 characters corresponds with a "BIPx" string for BIPs in the range 1-9. Longer identifiers than this will almost always be needed in order to meet the "globally unique" requirement.[?]

26. The maximum values for the `featureid` and `featuredata` lengths serve only to bound the volume of the feature message payload, particularly as this data will be automatically advertised even to peers that do not support the features in question.[?]

27. The maximum values for the `featureid` and `featuredata` lengths serve only to bound the volume of the feature message payload, particularly as this data will be automatically advertised even to peers that do not support the features in question.[?]